MINI PROTHEUS - Codigo fonte do projeto

Documento gerado automaticamente com os arquivos relevantes do repositorio.
Exclusoes aplicadas: node_modules, dist, __pycache__, caches e binarios.

==========
ARQUIVO: .env.example
==========

```
0001: APP_TITLE=Mini Protheus - ERP (SQL Server + JWT)
0002: JWT_SECRET_KEY=change-me-in-production
0003: JWT_ALGORITHM=HS256
0004: JWT_EXPIRE_MINUTES=240
0005: CORS_ALLOW_ORIGINS=http://localhost:5173
0006: DB_CONN_STR=DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;DATABASE=ERP_PROTHEUS;Trus
ted_Connection=yes;Encrypt=yes;TrustServerCertificate=yes;
```

==========
ARQUIVO: .github/workflows/tests.yml
==========

```
0001: name: Tests
0002:
0003: on:
0004:   push:
0005:   pull_request:
0006:
0007: jobs:
0008:   backend-tests:
0009:     runs-on: ubuntu-latest
0010:     steps:
0011:       - uses: actions/checkout@v4
0012:
0013:       - uses: actions/setup-python@v5
0014:         with:
0015:           python-version: "3.12"
0016:
0017:       - name: Install Python dependencies
0018:         run: |
0019:           python -m pip install --upgrade pip
0020:           python -m pip install -r requirements-dev.txt
0021:
0022:       - name: Run default test suite
0023:         run: pytest -q
0024:
0025:   integration-db-tests:
0026:     runs-on: ubuntu-latest
0027:     services:
0028:       sqlserver:
0029:         image: mcr.microsoft.com/mssql/server:2022-latest
0030:         env:
0031:           ACCEPT_EULA: Y
0032:           MSSQL_SA_PASSWORD: ${{ secrets.CI_SQLSERVER_SA_PASSWORD != '' && secrets.CI_SQLSERVE
R_SA_PASSWORD || 'Strong_Passw0rd!123' }}
0033:         ports:
0034:           - 1433:1433
```

```
0035:     env:
0036:       CI_SQLSERVER_SA_PASSWORD: ${{ secrets.CI_SQLSERVER_SA_PASSWORD != '' && secrets.CI_SQLSE
RVER_SA_PASSWORD || 'Strong_Passw0rd!123' }}
0037:       TEST_DB_ADMIN_CONN_STR: DRIVER={ODBC Driver 18 for SQL Server};SERVER=127.0.0.1,1433;DAT
ABASE=master;UID=sa;PWD=${{ secrets.CI_SQLSERVER_SA_PASSWORD != '' && secrets.CI_SQLSERVER_SA_PASSWO
RD || 'Strong_Passw0rd!123' }};Encrypt=no;TrustServerCertificate=yes;
0038:       TEST_DB_CONN_STR: DRIVER={ODBC Driver 18 for SQL Server};SERVER=127.0.0.1,1433;DATABASE=
mini_protheus_ci;UID=sa;PWD=${{ secrets.CI_SQLSERVER_SA_PASSWORD != '' && secrets.CI_SQLSERVER_SA_PA
SSWORD || 'Strong_Passw0rd!123' }};Encrypt=no;TrustServerCertificate=yes;
0039:     steps:
0040:       - uses: actions/checkout@v4
0041:
0042:       - uses: actions/setup-python@v5
0043:         with:
0044:           python-version: "3.12"
0045:
0046:       - name: Install ODBC Driver 18
0047:         run: |
0048:           sudo apt-get update
0049:           sudo apt-get install -y curl gnupg apt-transport-https unixodbc-dev
0050:           curl https://packages.microsoft.com/keys/microsoft.asc | sudo gpg --dearmor -o /usr/
share/keyrings/microsoft-prod.gpg
0051:           curl https://packages.microsoft.com/config/ubuntu/24.04/prod.list | sudo tee /etc/ap
t/sources.list.d/microsoft-prod.list
0052:           sudo apt-get update
0053:           sudo ACCEPT_EULA=Y apt-get install -y msodbcsql18
0054:
0055:       - name: Install Python dependencies
0056:         run: |
0057:           python -m pip install --upgrade pip
0058:           python -m pip install -r requirements-dev.txt
0059:
0060:       - name: Initialize integration database
0061:         run: python tests/init_integration_db.py
0062:
0063:       - name: Run DB-backed business tests
0064:         run: pytest -q tests/test_business_flows_db.py

==========================================================================================
ARQUIVO: .gitignore
==========================================================================================
0001: __pycache__/
0002: *.pyc
0003: .pytest_cache/
0004: .env
0005: frontend/node_modules/
0006: frontend/dist/

==========================================================================================
ARQUIVO: app/__init__.py
==========================================================================================
0001:

==========================================================================================
```

ARQUIVO: app/api/__init__.py
==========
0001:

==========
ARQUIVO: app/api/deps.py
==========
```
0001: from datetime import datetime, timedelta, timezone
0002: from fastapi import Depends, HTTPException
0003: from fastapi.security import OAuth2PasswordBearer
0004: from jose import jwt, JWTError
0005:
0006: from app.core import settings
0007: from app.infra.db import set_current_user
0008:
0009: oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")
0010:
0011:
0012: def create_access_token(data: dict, expires_minutes: int = settings.JWT_EXPIRE_MINUTES) -> str
:
0013:     to_encode = data.copy()
0014:     expire = datetime.now(timezone.utc) + timedelta(minutes=expires_minutes)
0015:     to_encode.update({"exp": expire})
0016:     return jwt.encode(to_encode, settings.JWT_SECRET_KEY, algorithm=settings.JWT_ALGORITHM)
0017:
0018:
0019: def get_current_user(token: str = Depends(oauth2_scheme)) -> dict:
0020:     try:
0021:         payload = jwt.decode(
0022:             token,
0023:             settings.JWT_SECRET_KEY,
0024:             algorithms=[settings.JWT_ALGORITHM],
0025:         )
0026:         login: str = payload.get("sub")
0027:         perfil: str = payload.get("perfil")
0028:         if not login or not perfil:
0029:             raise HTTPException(status_code=401, detail="Token inválido")
0030:         set_current_user(login)
0031:         return {"login": login, "perfil": perfil}
0032:     except JWTError:
0033:         raise HTTPException(status_code=401, detail="Token inválido ou expirado")
```

==========
ARQUIVO: app/api/routers/__init__.py
==========
0001:

==========
ARQUIVO: app/api/routers/alertas.py
==========
```
0001: from fastapi import APIRouter, Depends
0002: from pydantic import BaseModel, ConfigDict
0003:
0004: from app.api.deps import get_current_user
```

```
0005: from app.api.schemas.errors import common_error_responses
0006: from app.infra.db import db_transaction
0007: from app.use_cases.alertas_uc import listar_alertas_uc_tx
0008:
0009: router = APIRouter(tags=["Alertas"])
0010:
0011:
0012: class AlertItem(BaseModel):
0013:     code: str
0014:     severity: str
0015:     title: str
0016:     text: str
0017:     route: str | None = None
0018:     count: int | None = None
0019:     action_label: str | None = None
0020:     business_context: str | None = None
0021:
0022:
0023: class AlertSummary(BaseModel):
0024:     critical: int
0025:     warning: int
0026:     info: int
0027:     total: int
0028:
0029:
0030: class AlertResponse(BaseModel):
0031:     model_config = ConfigDict(
0032:         json_schema_extra={
0033:             "example": {
0034:                 "filial": "01",
0035:                 "usuario": "admin",
0036:                 "generated_at": "2026-03-23T17:10:00",
0037:                 "summary": {"critical": 1, "warning": 2, "info": 0, "total": 3},
0038:                 "items": [
0039:                     {
0040:                         "code": "AR_OVERDUE",
0041:                         "severity": "critical",
0042:                         "title": "Recebiveis vencidos",
0043:                         "text": "3 titulo(s) de contas a receber estao vencidos e seguem em ab
erto na filial 01.",
0044:                         "route": "/financeiro/ar",
0045:                         "count": 3,
0046:                         "action_label": "Abrir recebiveis",
0047:                         "business_context": "financeiro.ar.overdue",
0048:                     }
0049:                 ],
0050:             }
0051:         }
0052:     )
0053:
0054:     filial: str
0055:     usuario: str
0056:     generated_at: str
0057:     summary: AlertSummary
```

```
0058:     items: list[AlertItem]
0059:
0060:
0061: @router.get(
0062:     "/alertas",
0063:     response_model=AlertResponse,
0064:     responses=common_error_responses(401, 403, 500),
0065: )
0066: def listar_alertas(filial: str = "01", me: dict = Depends(get_current_user)):
0067:     with db_transaction() as (_, cur):
0068:         return listar_alertas_uc_tx(cur, login=me["login"], perfil=me["perfil"], filial=filial
)
```

==========
ARQUIVO: app/api/routers/ap_lancamento.py
==========

```
0001: from __future__ import annotations
0002:
0003: from fastapi import APIRouter, Depends
0004: from pydantic import BaseModel, ConfigDict
0005:
0006: from app.api.deps import get_current_user
0007: from app.api.schemas.errors import common_error_responses
0008: from app.infra.db import db_transaction
0009: from app.use_cases.ap_lancamento_uc import lancar_ap_uc_tx
0010:
0011: router = APIRouter(prefix="/ap", tags=["AP (Lançamento)"])
0012:
0013:
0014: class APLancarInput(BaseModel):
0015:     model_config = ConfigDict(
0016:         json_schema_extra={"example": {"forn_cod": "F0001", "valor": 80.0, "ref": "MANUAL-1",
"venc_dias": 15, "filial": "01"}}
0017:     )
0018:
0019:     forn_cod: str
0020:     valor: float
0021:     ref: str = "MANUAL"
0022:     venc_dias: int = 30
0023:     filial: str = "01"
0024:
0025:
0026: @router.post("/lancar", responses=common_error_responses(400, 401, 403, 422, 500))
0027: def lancar(dados: APLancarInput, me: dict = Depends(get_current_user)):
0028:     with db_transaction() as (_, cur):
0029:         return lancar_ap_uc_tx(
0030:             cur,
0031:             usuario=me["login"],
0032:             filial=dados.filial,
0033:             forn_cod=dados.forn_cod,
0034:             valor=dados.valor,
0035:             ref=dados.ref,
0036:             venc_dias=dados.venc_dias,
0037:         )
```

==========
ARQUIVO: app/api/routers/auditoria.py
==========

```
0001: from fastapi import APIRouter, Depends
0002: from app.api.deps import get_current_user
0003: from app.infra.db import db_transaction
0004: from app.use_cases.auditoria_uc import listar_logs_uc_tx
0005:
0006: router = APIRouter(tags=["Auditoria"])
0007:
0008: @router.get("/auditoria")
0009: def auditoria(me: dict = Depends(get_current_user)):
0010:     with db_transaction() as (_, cur):
0011:         return listar_logs_uc_tx(cur, me["login"])
```

==========
ARQUIVO: app/api/routers/auth.py
==========

```
0001: from fastapi import APIRouter, Depends
0002: from fastapi.security import OAuth2PasswordRequestForm
0003:
0004: from app.api.deps import create_access_token
0005: from app.api.schemas.errors import common_error_responses
0006: from app.infra.db import db_transaction
0007: from app.use_cases.auth_uc import autenticar_usuario_tx
0008:
0009:
0010: router = APIRouter(tags=["Auth"])
0011:
0012: @router.post("/login", responses=common_error_responses(401, 422, 500))
0013: def login(form_data: OAuth2PasswordRequestForm = Depends()):
0014:     with db_transaction() as (_, cur):
0015:         user = autenticar_usuario_tx(cur, form_data.username, form_data.password)
0016:         token = create_access_token({"sub": user["U_LOGIN"], "perfil": user["U_PERFIL"]})
0017:         return {"access_token": token, "token_type": "bearer"}
0018:
```

==========
ARQUIVO: app/api/routers/clientes.py
==========

```
0001: from __future__ import annotations
0002:
0003: from fastapi import APIRouter, Depends
0004: from pydantic import BaseModel
0005: from typing import Optional
0006:
0007: from app.api.deps import get_current_user
0008: from app.infra.db import db_transaction
0009: from app.use_cases.clientes_uc import (
0010:     cliente_get_uc_tx,
0011:     cliente_listar_uc_tx,
0012:     cliente_buscar_uc_tx,
0013:     cliente_criar_uc_tx,
```

```
0014:     cliente_atualizar_uc_tx,
0015:     cliente_set_ativo_uc_tx,
0016: )
0017:
0018: router = APIRouter(prefix="/clientes", tags=["Clientes"])
0019:
0020: class ClienteCreateInput(BaseModel):
0021:     cod: str
0022:     nome: str
0023:     doc: Optional[str] = None
0024:     email: Optional[str] = None
0025:     fone: Optional[str] = None
0026:     filial: str = "01"
0027:
0028: class ClienteUpdateInput(BaseModel):
0029:     nome: Optional[str] = None
0030:     doc: Optional[str] = None
0031:     email: Optional[str] = None
0032:     fone: Optional[str] = None
0033:     filial: str = "01"
0034:
0035: class ClienteAtivoInput(BaseModel):
0036:     ativo: bool
0037:     filial: str = "01"
0038:
0039: @router.get("")
0040: def listar(ativos: bool = True, filial: str = "01", me: dict = Depends(get_current_user)):
0041:     with db_transaction() as (_, cur):
0042:         return cliente_listar_uc_tx(cur, usuario=me["login"], filial=filial, ativos=ativos)
0043:
0044: @router.get("/buscar")
0045: def buscar(q: str, filial: str = "01", me: dict = Depends(get_current_user)):
0046:     with db_transaction() as (_, cur):
0047:         return cliente_buscar_uc_tx(cur, usuario=me["login"], filial=filial, termo=q)
0048:
0049: @router.get("/{cod}")
0050: def get(cod: str, filial: str = "01", me: dict = Depends(get_current_user)):
0051:     with db_transaction() as (_, cur):
0052:         return cliente_get_uc_tx(cur, usuario=me["login"], filial=filial, cod=cod)
0053:
0054: @router.post("")
0055: def criar(dados: ClienteCreateInput, me: dict = Depends(get_current_user)):
0056:     with db_transaction() as (_, cur):
0057:         return cliente_criar_uc_tx(
0058:             cur,
0059:             usuario=me["login"],
0060:             filial=dados.filial,
0061:             cod=dados.cod,
0062:             nome=dados.nome,
0063:             doc=dados.doc,
0064:             email=dados.email,
0065:             fone=dados.fone,
0066:         )
0067:
```

```
0068: @router.put("/{cod}")
0069: def atualizar(cod: str, dados: ClienteUpdateInput, me: dict = Depends(get_current_user)):
0070:     with db_transaction() as (_, cur):
0071:         return cliente_atualizar_uc_tx(
0072:             cur,
0073:             usuario=me["login"],
0074:             filial=dados.filial,
0075:             cod=cod,
0076:             nome=dados.nome,
0077:             doc=dados.doc,
0078:             email=dados.email,
0079:             fone=dados.fone,
0080:         )
0081:
0082: @router.post("/{cod}/ativo")
0083: def set_ativo(cod: str, dados: ClienteAtivoInput, me: dict = Depends(get_current_user)):
0084:     with db_transaction() as (_, cur):
0085:         return cliente_set_ativo_uc_tx(cur, usuario=me["login"], filial=dados.filial, cod=cod,
 ativo=dados.ativo)
```

==========
ARQUIVO: app/api/routers/estoque.py
==========

```
0001: from fastapi import APIRouter, Depends
0002: from pydantic import BaseModel
0003: from typing import Optional
0004:
0005: from app.api.deps import get_current_user
0006: from app.infra.db import db_transaction
0007: from app.use_cases.estoque_uc import entrada_estoque_tx, consultar_estoque_tx, extrato_sd3_uc_
tx
0008: from app.use_cases.conciliacao_uc import recalcular_sb1_por_sd3_tx
0009:
0010: router = APIRouter(tags=["Estoque"])
0011:
0012: class EntradaInput(BaseModel):
0013:     cod: str
0014:     qtd: int
0015:     filial: str = "01"
0016:     custo_unit: float = 0.0
0017:     forn: Optional[str] = None
0018:     venc_dias: int = 30
0019:
0020: class RecalcularInput(BaseModel):
0021:     filial: str = "01"
0022:
0023: @router.post("/estoque/entrada")
0024: def entrada(dados: EntradaInput, me: dict = Depends(get_current_user)):
0025:     with db_transaction() as (_, cur):
0026:         return entrada_estoque_tx(cur, me["login"], dados.cod, dados.qtd, dados.filial, dados.
custo_unit,
0027: dados.forn, dados.venc_dias)
0028:
0029: @router.get("/estoque/{cod}")
```

```
0030: def consultar(cod: str, filial: str = "01", me: dict = Depends(get_current_user)):
0031:     with db_transaction() as (_, cur):
0032:         return consultar_estoque_tx(cur, cod, filial)
0033:
0034: @router.get("/estoque/extrato/{cod}")
0035: def extrato(cod: str, filial: str = "01", limite: int = 100, me: dict = Depends(get_current_us
er)):
0036:     with db_transaction() as (_, cur):
0037:         return extrato_sd3_uc_tx(cur, cod, filial, limite)
0038:
0039: @router.post("/estoque/recalcular")
0040: def recalcular(dados: RecalcularInput, me: dict = Depends(get_current_user)):
0041:     with db_transaction() as (_, cur):
0042:         return recalcular_sb1_por_sd3_tx(cur, me["login"], dados.filial)
```

==========
ARQUIVO: app/api/routers/financeiro.py
==========

```
0001: from datetime import date
0002:
0003: from fastapi import APIRouter, Depends
0004: from pydantic import BaseModel, ConfigDict, Field
0005:
0006: from app.api.deps import get_current_user
0007: from app.api.schemas.errors import common_error_responses
0008: from app.infra.db import db_transaction
0009: from app.use_cases.financeiro_uc import (
0010:     criar_categoria_uc_tx,
0011:     definir_categoria_mov_uc_tx,
0012:     dre_simples_uc_tx,
0013:     extrato_caixa_uc_tx,
0014:     listar_ap_uc_tx,
0015:     listar_ar_uc_tx,
0016:     listar_categorias_uc_tx,
0017:     listar_contas_caixa_uc_tx,
0018:     obter_movimento_uc_tx,
0019:     pagar_ap_uc_tx,
0020:     receber_ar_uc_tx,
0021:     saldo_caixa_uc_tx,
0022: )
0023:
0024: router = APIRouter(tags=["Financeiro"])
0025:
0026:
0027: class LiquidarInput(BaseModel):
0028:     model_config = ConfigDict(json_schema_extra={"example": {"titulo_id": 123, "conta_id": 1}}
)
0029:
0030:     titulo_id: int = Field(..., ge=1)
0031:     conta_id: int = Field(..., ge=1)
0032:
0033:
0034: class CategoriaCreateInput(BaseModel):
0035:     nome: str
```

```
0036:     tipo: str
0037:
0038:
0039: class MovSetCategoriaInput(BaseModel):
0040:     mov_id: int = Field(..., ge=1)
0041:     categ_id: int | None = Field(default=None, ge=1)
0042:
0043:
0044: @router.get("/financeiro/ar")
0045: def listar_ar(filial: str = "01", status: str = "ABERTO", me=Depends(get_current_user)):
0046:     with db_transaction() as (_, cur):
0047:         return listar_ar_uc_tx(cur, filial=filial, status=status)
0048:
0049:
0050: @router.get("/financeiro/ap")
0051: def listar_ap(filial: str = "01", status: str = "ABERTO", me: dict = Depends(get_current_user)
):
0052:     with db_transaction() as (_, cur):
0053:         return listar_ap_uc_tx(cur, usuario=me["login"], filial=filial, status=status)
0054:
0055:
0056: @router.post("/financeiro/ar/baixar")
0057: def baixar_ar(dados: LiquidarInput, me: dict = Depends(get_current_user)):
0058:     with db_transaction() as (_, cur):
0059:         return receber_ar_uc_tx(cur, me["login"], dados.titulo_id, dados.conta_id)
0060:
0061:
0062: @router.post("/financeiro/ap/baixar")
0063: def baixar_ap(dados: LiquidarInput, me: dict = Depends(get_current_user)):
0064:     with db_transaction() as (_, cur):
0065:         return pagar_ap_uc_tx(cur, me["login"], dados.titulo_id, dados.conta_id)
0066:
0067:
0068: @router.post("/financeiro/ar/receber", responses=common_error_responses(400, 401, 403, 422, 50
0))
0069: def receber_ar(dados: LiquidarInput, me: dict = Depends(get_current_user)):
0070:     with db_transaction() as (_, cur):
0071:         return receber_ar_uc_tx(cur, me["login"], dados.titulo_id, dados.conta_id)
0072:
0073:
0074: @router.post("/financeiro/ap/pagar", responses=common_error_responses(400, 401, 403, 422, 500)
)
0075: def pagar_ap(dados: LiquidarInput, me: dict = Depends(get_current_user)):
0076:     with db_transaction() as (_, cur):
0077:         return pagar_ap_uc_tx(cur, me["login"], dados.titulo_id, dados.conta_id)
0078:
0079:
0080: @router.get("/financeiro/caixa/contas")
0081: def listar_contas_caixa(filial: str = "01", me: dict = Depends(get_current_user)):
0082:     with db_transaction() as (_, cur):
0083:         return listar_contas_caixa_uc_tx(cur, filial=filial)
0084:
0085:
0086: @router.get("/financeiro/caixa/extrato")
```

```
0087: def extrato_caixa(
0088:     filial: str = "01",
0089:     conta_id: int = 1,
0090:     de: date | None = None,
0091:     ate: date | None = None,
0092:     me: dict = Depends(get_current_user),
0093: ):
0094:     with db_transaction() as (_, cur):
0095:         return extrato_caixa_uc_tx(cur, filial=filial, conta_id=conta_id, de=de, ate=ate)
0096:
0097:
0098: @router.get("/financeiro/caixa/saldo")
0099: def saldo_caixa(
0100:     filial: str = "01",
0101:     conta_id: int = 1,
0102:     ate: date | None = None,
0103:     me: dict = Depends(get_current_user),
0104: ):
0105:     with db_transaction() as (_, cur):
0106:         return saldo_caixa_uc_tx(cur, filial=filial, conta_id=conta_id, ate=ate)
0107:
0108:
0109: @router.get("/financeiro/categorias")
0110: def listar_categorias(filial: str = "01", ativas: bool = True, me: dict = Depends(get_current_
user)):
0111:     with db_transaction() as (_, cur):
0112:         return listar_categorias_uc_tx(cur, filial=filial, ativas=ativas)
0113:
0114:
0115: @router.post("/financeiro/categorias")
0116: def criar_categoria(dados: CategoriaCreateInput, filial: str = "01", me: dict = Depends(get_cu
rrent_user)):
0117:     with db_transaction() as (_, cur):
0118:         return criar_categoria_uc_tx(cur, usuario=me["login"], filial=filial, nome=dados.nome,
 tipo=dados.tipo)
0119:
0120:
0121: @router.post("/financeiro/movimentos/categoria")
0122: def set_categoria_mov(dados: MovSetCategoriaInput, me: dict = Depends(get_current_user)):
0123:     with db_transaction() as (_, cur):
0124:         return definir_categoria_mov_uc_tx(cur, usuario=me["login"], mov_id=dados.mov_id, cate
g_id=dados.categ_id)
0125:
0126:
0127: @router.get("/financeiro/movimentos/{mov_id}")
0128: def obter_movimento(mov_id: int, me: dict = Depends(get_current_user)):
0129:     with db_transaction() as (_, cur):
0130:         return obter_movimento_uc_tx(cur, mov_id=mov_id)
0131:
0132:
0133: @router.get("/financeiro/relatorios/dre")
0134: def dre_simples(
0135:     filial: str = "01",
0136:     de: date | None = None,
```

```
0137:     ate: date | None = None,
0138:     me: dict = Depends(get_current_user),
0139: ):
0140:     with db_transaction() as (_, cur):
0141:         return dre_simples_uc_tx(cur, filial=filial, de=de, ate=ate)
```

==========
ARQUIVO: app/api/routers/fornecedores.py
==========

```
0001: from __future__ import annotations
0002:
0003: from fastapi import APIRouter, Depends
0004: from pydantic import BaseModel
0005: from typing import Optional
0006:
0007: from app.api.deps import get_current_user
0008: from app.infra.db import db_transaction
0009: from app.use_cases.fornecedores_uc import (
0010:     forn_get_uc_tx,
0011:     forn_listar_uc_tx,
0012:     forn_buscar_uc_tx,
0013:     forn_criar_uc_tx,
0014:     forn_atualizar_uc_tx,
0015:     forn_set_ativo_uc_tx,
0016: )
0017:
0018: router = APIRouter(prefix="/fornecedores", tags=["Fornecedores"])
0019:
0020: class FornCreateInput(BaseModel):
0021:     cod: str
0022:     nome: str
0023:     doc: Optional[str] = None
0024:     email: Optional[str] = None
0025:     fone: Optional[str] = None
0026:     filial: str = "01"
0027:
0028: class FornUpdateInput(BaseModel):
0029:     nome: Optional[str] = None
0030:     doc: Optional[str] = None
0031:     email: Optional[str] = None
0032:     fone: Optional[str] = None
0033:     filial: str = "01"
0034:
0035: class FornAtivoInput(BaseModel):
0036:     ativo: bool
0037:     filial: str = "01"
0038:
0039: @router.get("")
0040: def listar(ativos: bool = True, filial: str = "01", me: dict = Depends(get_current_user)):
0041:     with db_transaction() as (_, cur):
0042:         return forn_listar_uc_tx(cur, usuario=me["login"], filial=filial, ativos=ativos)
0043:
0044: @router.get("/buscar")
0045: def buscar(q: str, filial: str = "01", me: dict = Depends(get_current_user)):
```

```
0046:     with db_transaction() as (_, cur):
0047:         return forn_buscar_uc_tx(cur, usuario=me["login"], filial=filial, termo=q)
0048:
0049: @router.get("/{cod}")
0050: def get(cod: str, filial: str = "01", me: dict = Depends(get_current_user)):
0051:     with db_transaction() as (_, cur):
0052:         return forn_get_uc_tx(cur, usuario=me["login"], filial=filial, cod=cod)
0053:
0054: @router.post("")
0055: def criar(dados: FornCreateInput, me: dict = Depends(get_current_user)):
0056:     with db_transaction() as (_, cur):
0057:         return forn_criar_uc_tx(
0058:             cur,
0059:             usuario=me["login"],
0060:             filial=dados.filial,
0061:             cod=dados.cod,
0062:             nome=dados.nome,
0063:             doc=dados.doc,
0064:             email=dados.email,
0065:             fone=dados.fone,
0066:         )
0067:
0068: @router.put("/{cod}")
0069: def atualizar(cod: str, dados: FornUpdateInput, me: dict = Depends(get_current_user)):
0070:     with db_transaction() as (_, cur):
0071:         return forn_atualizar_uc_tx(
0072:             cur,
0073:             usuario=me["login"],
0074:             filial=dados.filial,
0075:             cod=cod,
0076:             nome=dados.nome,
0077:             doc=dados.doc,
0078:             email=dados.email,
0079:             fone=dados.fone,
0080:         )
0081:
0082:
0083: @router.post("/{cod}/ativo")
0084: def set_ativo(cod: str, dados: FornAtivoInput, me: dict = Depends(get_current_user)):
0085:     with db_transaction() as (_, cur):
0086:         return forn_set_ativo_uc_tx(cur, usuario=me["login"], filial=dados.filial, cod=cod, at
ivo=dados.ativo)
```

==========
ARQUIVO: app/api/routers/parametros.py
==========

```
0001: from fastapi import APIRouter, Depends
0002: from pydantic import BaseModel
0003: from typing import Optional
0004:
0005: from app.api.deps import get_current_user
0006: from app.infra.db import db_transaction
0007: from app.use_cases.sx5_uc import (
0008:     sx5_listar_uc_tx,
```

```
0009:     sx5_criar_uc_tx,
0010:     sx5_atualizar_uc_tx,
0011:     sx5_inativar_uc_tx,
0012: )
0013: from app.use_cases.fiscal_uc import sf4_salvar_uc_tx, sf4_listar_uc_tx, sf4_inativar_uc_tx
0014: from app.use_cases.fiscal_regras_uc import (
0015:     fiscal_regra_listar_uc_tx,
0016:     fiscal_regra_salvar_uc_tx,
0017:     fiscal_regra_inativar_uc_tx,
0018: )
0019: from app.use_cases.fiscal_ncm_uc import ncm_listar_uc_tx, ncm_salvar_uc_tx
0020: from app.use_cases.series_uc import series_obter_uc_tx, series_definir_uc_tx, series_listar_sx
5_uc_tx, series_reset_uc_tx
0021: from app.use_cases.fechamento_uc import fechar_periodo_uc_tx, abrir_periodo_uc_tx
0022: from app.use_cases.filiais_uc import filiais_listar_uc_tx, filiais_criar_uc_tx, filiais_inativ
ar_uc_tx
0023:
0024: router = APIRouter(prefix="/parametros", tags=["Parâmetros"])
0025:
0026:
0027: class SX5CriarInput(BaseModel):
0028:     filial: str = "01"
0029:     tabela: str
0030:     chave: str
0031:     descr: str
0032:     ativo: bool = True
0033:
0034:
0035: class SX5AtualizarInput(BaseModel):
0036:     filial: str = "01"
0037:     tabela: str
0038:     chave: str
0039:     descr: Optional[str] = None
0040:     ativo: Optional[bool] = None
0041:
0042:
0043: class SF4SalvarInput(BaseModel):
0044:     filial: str = "01"
0045:     tes_cod: str
0046:     cfop: str
0047:     icms: float = 0.0
0048:     ipi: float = 0.0
0049:     pis: float = 0.0
0050:     cofins: float = 0.0
0051:     icms_st: float = 0.0
0052:     difal: float = 0.0
0053:     cst_icms: Optional[str] = None
0054:     csosn: Optional[str] = None
0055:     cst_pis: Optional[str] = None
0056:     cst_cofins: Optional[str] = None
0057:     tipo: str = "S"
0058:     gera_tit: bool = True
0059:     gera_est: bool = True
0060:     descr: Optional[str] = None
```

```
0061:
0062:
0063: class SeriesSalvarInput(BaseModel):
0064:     filial: str = "01"
0065:     serie_nf: Optional[str] = None
0066:     serie_ar: Optional[str] = None
0067:     serie_ap: Optional[str] = None
0068:
0069:
0070: class FechamentoInput(BaseModel):
0071:     filial: str = "01"
0072:     ano: int
0073:     mes: int
0074:
0075:
0076: class NCMInput(BaseModel):
0077:     filial: str = "01"
0078:     ncm: str
0079:     cfop: str
0080:     icms: Optional[float] = None
0081:     ipi: Optional[float] = None
0082:     ativo: bool = True
0083:
0084:
0085: class FilialInput(BaseModel):
0086:     filial: str
0087:     serie_nf: Optional[str] = None
0088:     serie_ar: Optional[str] = None
0089:     serie_ap: Optional[str] = None
0090:
0091:
0092: class SF5RegraInput(BaseModel):
0093:     filial: str = "01"
0094:     tes_cod: str
0095:     cliente_cod: Optional[str] = None
0096:     produto: Optional[str] = None
0097:     cfop: Optional[str] = None
0098:     icms: Optional[float] = None
0099:     ipi: Optional[float] = None
0100:     pis: Optional[float] = None
0101:     cofins: Optional[float] = None
0102:     icms_st: Optional[float] = None
0103:     difal: Optional[float] = None
0104:     cst_icms: Optional[str] = None
0105:     csosn: Optional[str] = None
0106:     cst_pis: Optional[str] = None
0107:     cst_cofins: Optional[str] = None
0108:     gera_tit: Optional[bool] = None
0109:     gera_est: Optional[bool] = None
0110:     prioridade: int = 0
0111:     ativo: bool = True
0112:     regra_id: Optional[int] = None
0113:
0114:
```

```
0115: @router.get("/sx5")
0116: def listar_sx5(
0117:     filial: str,
0118:     tabela: str,
0119:     ativos: bool = True,
0120:     me: dict = Depends(get_current_user),
0121: ):
0122:     with db_transaction() as (_, cur):
0123:         return sx5_listar_uc_tx(cur, usuario=me["login"], filial=filial, tabela=tabela, ativos
=ativos)
0124:
0125:
0126: @router.post("/sx5")
0127: def criar_sx5(dados: SX5CriarInput, me: dict = Depends(get_current_user)):
0128:     with db_transaction() as (_, cur):
0129:         return sx5_criar_uc_tx(
0130:             cur,
0131:             usuario=me["login"],
0132:             filial=dados.filial,
0133:             tabela=dados.tabela,
0134:             chave=dados.chave,
0135:             descr=dados.descr,
0136:             ativo=bool(dados.ativo),
0137:         )
0138:
0139:
0140: @router.put("/sx5")
0141: def atualizar_sx5(dados: SX5AtualizarInput, me: dict = Depends(get_current_user)):
0142:     with db_transaction() as (_, cur):
0143:         return sx5_atualizar_uc_tx(
0144:             cur,
0145:             usuario=me["login"],
0146:             filial=dados.filial,
0147:             tabela=dados.tabela,
0148:             chave=dados.chave,
0149:             descr=dados.descr,
0150:             ativo=dados.ativo,
0151:         )
0152:
0153:
0154: @router.delete("/sx5")
0155: def inativar_sx5(filial: str, tabela: str, chave: str, me: dict = Depends(get_current_user)):
0156:     with db_transaction() as (_, cur):
0157:         return sx5_inativar_uc_tx(cur, usuario=me["login"], filial=filial, tabela=tabela, chav
e=chave)
0158:
0159:
0160: @router.post("/sf4")
0161: def salvar_sf4(dados: SF4SalvarInput, me: dict = Depends(get_current_user)):
0162:     with db_transaction() as (_, cur):
0163:         return sf4_salvar_uc_tx(
0164:             cur,
0165:             usuario=me["login"],
0166:             filial=dados.filial,
```

```
0167:             tes_cod=dados.tes_cod,
0168:             cfop=dados.cfop,
0169:             icms=float(dados.icms),
0170:             ipi=float(dados.ipi),
0171:             pis=float(dados.pis),
0172:             cofins=float(dados.cofins),
0173:             icms_st=float(dados.icms_st),
0174:             difal=float(dados.difal),
0175:             cst_icms=dados.cst_icms,
0176:             csosn=dados.csosn,
0177:             cst_pis=dados.cst_pis,
0178:             cst_cofins=dados.cst_cofins,
0179:             tipo=dados.tipo,
0180:             gera_tit=bool(dados.gera_tit),
0181:             gera_est=bool(dados.gera_est),
0182:             descr=dados.descr,
0183:         )
0184:
0185:
0186: @router.get("/sf4")
0187: def listar_sf4(
0188:     filial: str,
0189:     ativos: bool = True,
0190:     me: dict = Depends(get_current_user),
0191: ):
0192:     with db_transaction() as (_, cur):
0193:         return sf4_listar_uc_tx(cur, usuario=me["login"], filial=filial, ativos=ativos)
0194:
0195:
0196: @router.delete("/sf4")
0197: def inativar_sf4(
0198:     filial: str,
0199:     tes_cod: str,
0200:     me: dict = Depends(get_current_user),
0201: ):
0202:     with db_transaction() as (_, cur):
0203:         return sf4_inativar_uc_tx(cur, usuario=me["login"], filial=filial, tes_cod=tes_cod)
0204:
0205:
0206: @router.get("/series")
0207: def obter_series(filial: str, me: dict = Depends(get_current_user)):
0208:     with db_transaction() as (_, cur):
0209:         return series_obter_uc_tx(cur, usuario=me["login"], filial=filial)
0210:
0211:
0212: @router.put("/series")
0213: def definir_series(dados: SeriesSalvarInput, me: dict = Depends(get_current_user)):
0214:     with db_transaction() as (_, cur):
0215:         return series_definir_uc_tx(
0216:             cur,
0217:             usuario=me["login"],
0218:             filial=dados.filial,
0219:             serie_nf=dados.serie_nf,
0220:             serie_ar=dados.serie_ar,
```

```
0221:             serie_ap=dados.serie_ap,
0222:         )
0223:
0224:
0225: @router.get("/series/sx5")
0226: def listar_series_sx5(filial: str, me: dict = Depends(get_current_user)):
0227:     with db_transaction() as (_, cur):
0228:         return series_listar_sx5_uc_tx(cur, usuario=me["login"], filial=filial)
0229:
0230:
0231: @router.post("/series/reset")
0232: def resetar_series(filial: str, me: dict = Depends(get_current_user)):
0233:     with db_transaction() as (_, cur):
0234:         return series_reset_uc_tx(cur, usuario=me["login"], filial=filial)
0235:
0236:
0237: @router.post("/fechamento/fechar")
0238: def fechar_periodo(dados: FechamentoInput, me: dict = Depends(get_current_user)):
0239:     with db_transaction() as (_, cur):
0240:         return fechar_periodo_uc_tx(
0241:             cur,
0242:             usuario=me["login"],
0243:             filial=dados.filial,
0244:             ano=int(dados.ano),
0245:             mes=int(dados.mes),
0246:         )
0247:
0248:
0249: @router.post("/fechamento/abrir")
0250: def abrir_periodo(dados: FechamentoInput, me: dict = Depends(get_current_user)):
0251:     with db_transaction() as (_, cur):
0252:         return abrir_periodo_uc_tx(
0253:             cur,
0254:             usuario=me["login"],
0255:             filial=dados.filial,
0256:             ano=int(dados.ano),
0257:             mes=int(dados.mes),
0258:         )
0259:
0260:
0261: @router.get("/sf5")
0262: def listar_sf5(
0263:     filial: str,
0264:     tes_cod: Optional[str] = None,
0265:     cliente_cod: Optional[str] = None,
0266:     produto: Optional[str] = None,
0267:     ativos: bool = True,
0268:     me: dict = Depends(get_current_user),
0269: ):
0270:     with db_transaction() as (_, cur):
0271:         return fiscal_regra_listar_uc_tx(
0272:             cur,
0273:             usuario=me["login"],
0274:             filial=filial,
```

```
0275:             tes_cod=tes_cod,
0276:             cliente_cod=cliente_cod,
0277:             produto=produto,
0278:             ativos=ativos,
0279:         )
0280:
0281:
0282: @router.post("/sf5")
0283: def salvar_sf5(dados: SF5RegraInput, me: dict = Depends(get_current_user)):
0284:     with db_transaction() as (_, cur):
0285:         return fiscal_regra_salvar_uc_tx(
0286:             cur,
0287:             usuario=me["login"],
0288:             filial=dados.filial,
0289:             tes_cod=dados.tes_cod,
0290:             cliente_cod=dados.cliente_cod,
0291:             produto=dados.produto,
0292:             cfop=dados.cfop,
0293:             icms=(float(dados.icms) if dados.icms is not None else None),
0294:             ipi=(float(dados.ipi) if dados.ipi is not None else None),
0295:             pis=(float(dados.pis) if dados.pis is not None else None),
0296:             cofins=(float(dados.cofins) if dados.cofins is not None else None),
0297:             icms_st=(float(dados.icms_st) if dados.icms_st is not None else None),
0298:             difal=(float(dados.difal) if dados.difal is not None else None),
0299:             cst_icms=dados.cst_icms,
0300:             csosn=dados.csosn,
0301:             cst_pis=dados.cst_pis,
0302:             cst_cofins=dados.cst_cofins,
0303:             gera_tit=dados.gera_tit,
0304:             gera_est=dados.gera_est,
0305:             prioridade=int(dados.prioridade),
0306:             ativo=bool(dados.ativo),
0307:             regra_id=(int(dados.regra_id) if dados.regra_id is not None else None),
0308:         )
0309:
0310:
0311: @router.delete("/sf5")
0312: def inativar_sf5(regra_id: int, me: dict = Depends(get_current_user)):
0313:     with db_transaction() as (_, cur):
0314:         return fiscal_regra_inativar_uc_tx(cur, usuario=me["login"], regra_id=int(regra_id))
0315:
0316:
0317: @router.get("/ncm")
0318: def listar_ncm(
0319:     filial: str,
0320:     ativos: bool = True,
0321:     me: dict = Depends(get_current_user),
0322: ):
0323:     with db_transaction() as (_, cur):
0324:         return ncm_listar_uc_tx(cur, usuario=me["login"], filial=filial, ativos=ativos)
0325:
0326:
0327: @router.post("/ncm")
0328: def salvar_ncm(dados: NCMInput, me: dict = Depends(get_current_user)):
```

```
0329:     with db_transaction() as (_, cur):
0330:         return ncm_salvar_uc_tx(
0331:             cur,
0332:             usuario=me["login"],
0333:             filial=dados.filial,
0334:             ncm=dados.ncm,
0335:             cfop=dados.cfop,
0336:             icms=(float(dados.icms) if dados.icms is not None else None),
0337:             ipi=(float(dados.ipi) if dados.ipi is not None else None),
0338:             ativo=bool(dados.ativo),
0339:         )
0340:
0341:
0342: @router.get("/filiais")
0343: def listar_filiais(me: dict = Depends(get_current_user)):
0344:     with db_transaction() as (_, cur):
0345:         return filiais_listar_uc_tx(cur, usuario=me["login"])
0346:
0347:
0348: @router.post("/filiais")
0349: def criar_filial(dados: FilialInput, me: dict = Depends(get_current_user)):
0350:     with db_transaction() as (_, cur):
0351:         return filiais_criar_uc_tx(
0352:             cur,
0353:             usuario=me["login"],
0354:             filial=dados.filial,
0355:             serie_nf=dados.serie_nf,
0356:             serie_ar=dados.serie_ar,
0357:             serie_ap=dados.serie_ap,
0358:         )
0359:
0360:
0361: @router.delete("/filiais")
0362: def inativar_filial(filial: str, me: dict = Depends(get_current_user)):
0363:     with db_transaction() as (_, cur):
0364:         return filiais_inativar_uc_tx(cur, usuario=me["login"], filial=filial)
```

==========
ARQUIVO: app/api/routers/parceiros.py
==========

```
0001: from fastapi import APIRouter, Depends
0002: from pydantic import BaseModel, Field
0003:
0004: from app.api.deps import get_current_user
0005: from app.infra.db import db_transaction
0006: from app.use_cases.parceiros_uc import (
0007:     criar_cliente_uc_tx,
0008:     criar_fornecedor_uc_tx,
0009:     inativar_cliente_uc_tx,
0010:     inativar_fornecedor_uc_tx,
0011:     listar_clientes_uc_tx,
0012:     listar_fornecedores_uc_tx,
0013:     obter_cliente_uc_tx,
0014:     obter_fornecedor_uc_tx,
```

```
0015:     set_tabela_cliente_uc_tx,
0016: )
0017:
0018: router = APIRouter(tags=["Parceiros"])
0019:
0020:
0021: class ClienteInput(BaseModel):
0022:     cod: str
0023:     nome: str
0024:     doc: str | None = None
0025:     email: str | None = None
0026:     tel: str | None = None
0027:     end: str | None = None
0028:     mun: str | None = None
0029:     uf: str | None = None
0030:     cep: str | None = None
0031:     tabela_id: int | None = Field(default=None, ge=1)
0032:
0033:
0034: class FornecedorInput(BaseModel):
0035:     cod: str
0036:     nome: str
0037:     doc: str | None = None
0038:     email: str | None = None
0039:     tel: str | None = None
0040:     end: str | None = None
0041:     mun: str | None = None
0042:     uf: str | None = None
0043:     cep: str | None = None
0044:
0045:
0046: class SetTabelaInput(BaseModel):
0047:     cod: str
0048:     tabela_id: int | None = Field(default=None, ge=1)
0049:
0050:
0051: @router.post("/parceiros/sa1/clientes")
0052: def criar_cliente(dados: ClienteInput, filial: str = "01", me: dict = Depends(get_current_user
)):
0053:     with db_transaction() as (_, cur):
0054:         return criar_cliente_uc_tx(cur, me["login"], filial, **dados.model_dump())
0055:
0056:
0057: @router.get("/parceiros/sa1/clientes")
0058: def listar_clientes(
0059:     filial: str = "01",
0060:     ativo: bool = True,
0061:     q: str | None = None,
0062:     me: dict = Depends(get_current_user),
0063: ):
0064:     with db_transaction() as (_, cur):
0065:         return listar_clientes_uc_tx(cur, filial=filial, ativo=ativo, q=q)
0066:
0067:
```

```
0068: @router.get("/parceiros/sa1/clientes/{cod}")
0069: def obter_cliente(cod: str, filial: str = "01", me: dict = Depends(get_current_user)):
0070:     with db_transaction() as (_, cur):
0071:         return obter_cliente_uc_tx(cur, filial=filial, cod=cod)
0072:
0073:
0074: @router.post("/parceiros/sa1/clientes/set-tabela")
0075: def set_tabela_cliente(dados: SetTabelaInput, filial: str = "01", me: dict = Depends(get_curre
nt_user)):
0076:     with db_transaction() as (_, cur):
0077:         return set_tabela_cliente_uc_tx(cur, me["login"], filial, dados.cod, dados.tabela_id)
0078:
0079:
0080: @router.delete("/parceiros/sa1/clientes/{cod}")
0081: def inativar_cliente(cod: str, filial: str = "01", me: dict = Depends(get_current_user)):
0082:     with db_transaction() as (_, cur):
0083:         return inativar_cliente_uc_tx(cur, me["login"], filial, cod)
0084:
0085:
0086: @router.post("/parceiros/sa2/fornecedores")
0087: def criar_fornecedor(dados: FornecedorInput, filial: str = "01", me: dict = Depends(get_curren
t_user)):
0088:     with db_transaction() as (_, cur):
0089:         return criar_fornecedor_uc_tx(cur, me["login"], filial, **dados.model_dump())
0090:
0091:
0092: @router.get("/parceiros/sa2/fornecedores")
0093: def listar_fornecedores(
0094:     filial: str = "01",
0095:     ativo: bool = True,
0096:     q: str | None = None,
0097:     me: dict = Depends(get_current_user),
0098: ):
0099:     with db_transaction() as (_, cur):
0100:         return listar_fornecedores_uc_tx(cur, filial=filial, ativo=ativo, q=q)
0101:
0102:
0103: @router.get("/parceiros/sa2/fornecedores/{cod}")
0104: def obter_fornecedor(cod: str, filial: str = "01", me: dict = Depends(get_current_user)):
0105:     with db_transaction() as (_, cur):
0106:         return obter_fornecedor_uc_tx(cur, filial=filial, cod=cod)
0107:
0108:
0109: @router.delete("/parceiros/sa2/fornecedores/{cod}")
0110: def inativar_fornecedor(cod: str, filial: str = "01", me: dict = Depends(get_current_user)):
0111:     with db_transaction() as (_, cur):
0112:         return inativar_fornecedor_uc_tx(cur, me["login"], filial, cod)
```

==========
ARQUIVO: app/api/routers/precos.py
==========

```
0001: from datetime import date
0002:
0003: from fastapi import APIRouter, Depends
```

```
0004: from pydantic import BaseModel
0005:
0006: from app.api.deps import get_current_user
0007: from app.infra.db import db_transaction
0008: from app.use_cases.preco_uc import (
0009:     buscar_preco_uc_tx,
0010:     criar_tabela_uc_tx,
0011:     definir_preco_uc_tx,
0012:     listar_precos_tabela_uc_tx,
0013:     listar_tabelas_uc_tx,
0014: )
0015:
0016: router = APIRouter(tags=["Preços"])
0017:
0018:
0019: class TabelaCreate(BaseModel):
0020:     codigo: str
0021:     descricao: str
0022:
0023:
0024: class PrecoInput(BaseModel):
0025:     tabela_id: int
0026:     produto: str
0027:     preco: float
0028:     dt_ini: date
0029:
0030:
0031: @router.get("/precos/tabelas")
0032: def listar_tabelas(filial: str = "01", me=Depends(get_current_user)):
0033:     with db_transaction() as (_, cur):
0034:         return listar_tabelas_uc_tx(cur, filial)
0035:
0036:
0037: @router.get("/precos/tabelas/{tabela_id}/itens")
0038: def listar_precos_tabela(tabela_id: int, filial: str = "01", me=Depends(get_current_user)):
0039:     with db_transaction() as (_, cur):
0040:         return listar_precos_tabela_uc_tx(cur, filial, tabela_id)
0041:
0042:
0043: @router.post("/precos/tabelas")
0044: def criar_tabela(dados: TabelaCreate, filial: str = "01", me=Depends(get_current_user)):
0045:     with db_transaction() as (_, cur):
0046:         return criar_tabela_uc_tx(cur, me["login"], filial, dados.codigo, dados.descricao)
0047:
0048:
0049: @router.post("/precos/definir")
0050: def definir_preco(dados: PrecoInput, filial: str = "01", me=Depends(get_current_user)):
0051:     with db_transaction() as (_, cur):
0052:         return definir_preco_uc_tx(
0053:             cur,
0054:             me["login"],
0055:             filial,
0056:             dados.tabela_id,
0057:             dados.produto,
```

```
0058:             dados.preco,
0059:             dados.dt_ini,
0060:         )
0061:
0062:
0063: @router.get("/precos/buscar")
0064: def buscar_preco(produto: str, tabela_id: int, data: date, filial: str = "01", me=Depends(get_
current_user)):
0065:     with db_transaction() as (_, cur):
0066:         return buscar_preco_uc_tx(cur, filial, tabela_id, produto, data)
```

==========
ARQUIVO: app/api/routers/produtos.py
==========

```
0001: from fastapi import APIRouter, Depends
0002: from pydantic import BaseModel
0003:
0004: from app.api.deps import get_current_user
0005: from app.infra.db import db_transaction
0006: from app.use_cases.produtos_uc import upsert_produto_tx, buscar_produtos_tx
0007:
0008: router = APIRouter(tags=["Produtos"])
0009:
0010: class ProdutoInput(BaseModel):
0011:     cod: str
0012:     desc: str
0013:     preco: float
0014:     filial: str = "01"
0015:
0016: @router.post("/produto")
0017: def upsert_produto(dados: ProdutoInput, me: dict = Depends(get_current_user)):
0018:     with db_transaction() as (_, cur):
0019:         return upsert_produto_tx(cur, me["login"], dados.cod, dados.desc, dados.preco, dados.f
ilial)
0020:
0021:
0022: @router.get("/produtos/buscar")
0023: def buscar_produtos(q: str, filial: str = "01", limite: int = 20, me: dict = Depends(get_curre
nt_user)):
0024:     with db_transaction() as (_, cur):
0025:         return buscar_produtos_tx(cur, filial=filial, q=q, limite=limite)
```

==========
ARQUIVO: app/api/routers/relatorios.py
==========

```
0001: from fastapi import APIRouter, Depends
0002: from datetime import date
0003:
0004: from app.api.deps import get_current_user
0005: from app.infra.db import db_transaction
0006: from app.infra.repositories.relatorios_repo import dre_simples_tx, margem_por_produto_tx
0007: from app.use_cases.conciliacao_nf_uc import conciliacao_nf_financeiro_uc_tx
0008: from app.use_cases.consistencia_uc import consistencia_fiscal_uc_tx
0009:
```

```
0010: router = APIRouter(tags=["Relatórios"])
0011:
0012: @router.get("/relatorios/dre")
0013: def dre(filial: str="01", de: date | None=None, ate: date | None=None, me=Depends(get_current_
user)):
0014:     with db_transaction() as (_, cur):
0015:         return dre_simples_tx(cur, filial, de, ate)
0016:
0017: @router.get("/relatorios/margem-produto")
0018: def margem_produto(filial: str="01", de: date | None=None, ate: date | None=None,
0019: me=Depends(get_current_user)):
0020:     with db_transaction() as (_, cur):
0021:         return margem_por_produto_tx(cur, filial, de, ate)
0022:
0023:
0024: @router.get("/relatorios/conciliacao-nf")
0025: def conciliacao_nf(
0026:     filial: str = "01",
0027:     de: date | None = None,
0028:     ate: date | None = None,
0029:     tolerancia: float = 0.01,
0030:     limit: int = 200,
0031:     me=Depends(get_current_user),
0032: ):
0033:     with db_transaction() as (_, cur):
0034:         return conciliacao_nf_financeiro_uc_tx(
0035:             cur,
0036:             usuario=me["login"],
0037:             filial=filial,
0038:             de=de,
0039:             ate=ate,
0040:             tolerancia=float(tolerancia),
0041:             limit=int(limit),
0042:         )
0043:
0044:
0045: @router.get("/relatorios/consistencia-fiscal")
0046: def consistencia_fiscal(
0047:     filial: str = "01",
0048:     de: date | None = None,
0049:     ate: date | None = None,
0050:     limit: int = 200,
0051:     me=Depends(get_current_user),
0052: ):
0053:     with db_transaction() as (_, cur):
0054:         return consistencia_fiscal_uc_tx(
0055:             cur,
0056:             usuario=me["login"],
0057:             filial=filial,
0058:             de=de,
0059:             ate=ate,
0060:             limit=int(limit),
0061:         )
```

```
0001: from typing import Optional
0002:
0003: from fastapi import APIRouter, Depends
0004: from pydantic import BaseModel, ConfigDict
0005:
0006: from app.api.deps import get_current_user
0007: from app.api.schemas.errors import common_error_responses
0008: from app.infra.db import db_transaction
0009: from app.use_cases.reservas_uc import cancelar_reserva_tx, confirmar_reserva_tx, criar_reserva
_tx
0010:
0011: router = APIRouter(prefix="/reserva", tags=["Reserva"])
0012:
0013:
0014: class ReservaInput(BaseModel):
0015:     model_config = ConfigDict(
0016:         json_schema_extra={"example": {"cod": "PROD002", "qtd": 2, "filial": "01", "cliente_co
d": "C0002", "tabela_cod": "0001"}}
0017:     )
0018:
0019:     cod: str
0020:     qtd: int
0021:     filial: str = "01"
0022:     cliente_cod: str | None = None
0023:     tabela_cod: str = "0001"
0024:
0025:
0026: class ConfirmarInput(BaseModel):
0027:     model_config = ConfigDict(
0028:         json_schema_extra={"example": {"reserva_id": 301, "cliente_cod": "C0002", "venc_dias":
 20, "tes_cod": "001"}}
0029:     )
0030:
0031:     reserva_id: int
0032:     cliente_cod: Optional[str] = None
0033:     venc_dias: int = 30
0034:     tes_cod: Optional[str] = "001"
0035:
0036:
0037: class CancelarInput(BaseModel):
0038:     model_config = ConfigDict(json_schema_extra={"example": {"reserva_id": 301}})
0039:
0040:     reserva_id: int
0041:
0042:
0043: @router.post("", responses=common_error_responses(400, 401, 403, 422, 500))
0044: def criar(dados: ReservaInput, me: dict = Depends(get_current_user)):
0045:     with db_transaction() as (_, cur):
0046:         cod = (dados.cod or "").strip()
0047:         filial = (dados.filial or "").strip()
0048:         tabela_cod = (getattr(dados, "tabela_cod", None) or "0001").strip()
```

```
0049:         cliente_cod = (getattr(dados, "cliente_cod", None) or None)
0050:         if isinstance(cliente_cod, str):
0051:             cliente_cod = cliente_cod.strip() or None
0052:
0053:         return criar_reserva_tx(
0054:             cur,
0055:             usuario=me["login"],
0056:             produto=cod,
0057:             qtd=dados.qtd,
0058:             filial=filial,
0059:             cliente_cod=cliente_cod,
0060:             tabela_cod=tabela_cod,
0061:         )
0062:
0063:
0064: @router.post("/cancelar", responses=common_error_responses(400, 401, 403, 422, 500))
0065: def cancelar(dados: CancelarInput, me: dict = Depends(get_current_user)):
0066:     with db_transaction() as (_, cur):
0067:         return cancelar_reserva_tx(cur, me["login"], dados.reserva_id)
0068:
0069:
0070: @router.post("/confirmar", responses=common_error_responses(400, 401, 403, 422, 500))
0071: def confirmar(dados: ConfirmarInput, me: dict = Depends(get_current_user)):
0072:     with db_transaction() as (_, cur):
0073:         return confirmar_reserva_tx(cur, me["login"], dados.reserva_id, dados.cliente_cod, dad
os.venc_dias, dados.tes_cod)

==========================================================================================
ARQUIVO: app/api/routers/usuarios.py
==========================================================================================
0001: from fastapi import APIRouter, Depends, HTTPException
0002: from pydantic import BaseModel, ConfigDict
0003:
0004: from app.api.deps import get_current_user
0005: from app.api.schemas.errors import common_error_responses
0006: from app.core import settings
0007: from app.infra.db import db_transaction
0008: from app.use_cases.usuarios_uc import criar_usuario_bootstrap_tx, criar_usuario_tx, obter_setu
p_status_tx
0009:
0010: router = APIRouter(tags=["Usuários"])
0011:
0012:
0013: class BootstrapInput(BaseModel):
0014:     model_config = ConfigDict(json_schema_extra={"example": {"login": "admin", "senha": "Troqu
e123!"}})
0015:
0016:     login: str
0017:     senha: str
0018:
0019:
0020: class UsuarioCreateInput(BaseModel):
0021:     model_config = ConfigDict(
```

```
0022:         json_schema_extra={"example": {"login": "operador", "senha": "SenhaForte123!", "perfil
": "user"}}
0023:     )
0024:
0025:     login: str
0026:     senha: str
0027:     perfil: str
0028:
0029:
0030: def bootstrap_is_available() -> bool:
0031:     return settings.BOOTSTRAP_ENABLED and settings.APP_ENV in settings.BOOTSTRAP_ALLOWED_ENVS
0032:
0033:
0034: @router.post(
0035:     "/bootstrap",
0036:     responses=common_error_responses(400, 404, 422, 500),
0037: )
0038: def bootstrap(dados: BootstrapInput):
0039:     # One-time setup endpoint: only exposed during explicit setup/dev windows.
0040:     if not bootstrap_is_available():
0041:         raise HTTPException(status_code=404, detail="Not Found")
0042:     with db_transaction() as (_, cur):
0043:         criar_usuario_bootstrap_tx(cur, dados.login, dados.senha)
0044:         return {"status": "admin criado", "login": dados.login}
0045:
0046:
0047: @router.post(
0048:     "/usuarios",
0049:     responses=common_error_responses(400, 401, 403, 409, 422, 500),
0050: )
0051: def criar_usuario(dados: UsuarioCreateInput, me: dict = Depends(get_current_user)):
0052:     with db_transaction() as (_, cur):
0053:         criar_usuario_tx(cur, me["login"], dados.login, dados.senha, dados.perfil)
0054:         return {"status": "usuário criado", "login": dados.login}
0055:
0056:
0057: @router.get(
0058:     "/setup-status",
0059:     responses=common_error_responses(401, 403, 500),
0060: )
0061: def obter_setup_status(me: dict = Depends(get_current_user)):
0062:     with db_transaction() as (_, cur):
0063:         return obter_setup_status_tx(cur, me["login"])
```

==========
ARQUIVO: app/api/routers/vendas.py
==========

```
0001: from datetime import date
0002: from typing import Optional
0003:
0004: from fastapi import APIRouter, Depends, Query
0005: from pydantic import BaseModel, ConfigDict
0006:
0007: from app.api.deps import get_current_user
```

```
0008: from app.api.schemas.errors import common_error_responses
0009: from app.infra.db import db_transaction
0010: from app.use_cases.vendas.buscar_nf_uc import buscar_nf_por_doc_serie_tx
0011: from app.use_cases.vendas.cancelar_nf_uc import cancelar_nf_tx
0012: from app.use_cases.vendas.cancelar_pedido_uc import cancelar_pedido_tx
0013: from app.use_cases.vendas.consultar_pedido_uc import consultar_pedido_tx
0014: from app.use_cases.vendas.devolver_nf_uc import devolver_nf_tx
0015: from app.use_cases.vendas.estornar_nf_uc import estornar_nf_tx
0016: from app.use_cases.vendas.estornar_pedido_uc import estornar_pedido_faturado_venda_tx
0017: from app.use_cases.vendas.faturar_pedido_uc import faturar_pedido_venda_tx
0018: from app.use_cases.vendas.itens_uc import editar_item_tx, excluir_item_tx
0019: from app.use_cases.vendas.liberacoes_uc import resumo_liberacao_pedido_tx
0020: from app.use_cases.vendas.liberar_pedido_uc import liberar_item_pedido_tx, liberar_pedido_tota
l_tx
0021: from app.use_cases.vendas.listar_pedidos_enriq_uc import listar_pedidos_enriq_tx
0022: from app.use_cases.vendas.listar_pedidos_uc import listar_pedidos_tx
0023: from app.use_cases.vendas.pedidos_uc import adicionar_item_tx, criar_pedido_tx
0024: from app.use_cases.vendas.recalcular_pedido_uc import pedido_recalcular_totais_tx
0025: from app.use_cases.vendas.relatorios_nf_id_uc import relatorio_nf_rastreio_tx
0026: from app.use_cases.vendas.relatorios_nf_uc import relatorio_nf_financeiro_tx
0027:
0028: router = APIRouter(prefix="/vendas", tags=["Vendas"])
0029:
0030:
0031: class PedidoCriarInput(BaseModel):
0032:     model_config = ConfigDict(json_schema_extra={"example": {"filial": "01", "valor_total": 15
0.0, "status": "ABERTO"}})
0033:
0034:     filial: str = "01"
0035:     valor_total: float
0036:     status: str = "ABERTO"
0037:     icms: Optional[float] = None
0038:     ipi: Optional[float] = None
0039:     total_bruto: Optional[float] = None
0040:
0041:
0042: class ItemCriarInput(BaseModel):
0043:     model_config = ConfigDict(
0044:         json_schema_extra={
0045:             "example": {
0046:                 "filial": "01",
0047:                 "produto": "PROD001",
0048:                 "qtd": 3,
0049:                 "total": 150.0,
0050:                 "preco_unit": 50.0,
0051:                 "cmv_unit": 32.5,
0052:             }
0053:         }
0054:     )
0055:
0056:     filial: str = "01"
0057:     produto: str
0058:     qtd: int
0059:     total: float
```

```
0060:     preco_unit: Optional[float] = None
0061:     cmv_unit: Optional[float] = None
0062:
0063:
0064: class FaturarPedidoInput(BaseModel):
0065:     model_config = ConfigDict(
0066:         json_schema_extra={"example": {"filial": "01", "cliente_cod": "C0001", "venc_dias": 30
, "tes_cod": "001"}}
0067:     )
0068:
0069:     filial: str = "01"
0070:     cliente_cod: Optional[str] = None
0071:     venc_dias: int = 30
0072:     tes_cod: Optional[str] = "001"
0073:
0074:
0075: class CancelarPedidoInput(BaseModel):
0076:     filial: str = "01"
0077:     modo: str = "AUTO"
0078:     reativar_reserva: bool = True
0079:
0080:
0081: class RecalcularPedidoInput(BaseModel):
0082:     forcar: bool = False
0083:
0084:
0085: class EditarItemInput(BaseModel):
0086:     filial: Optional[str] = "01"
0087:     qtd: int
0088:     total: float
0089:     preco_unit: float | None = None
0090:     cmv_unit: float | None = None
0091:
0092:
0093: class ExcluirItemInput(BaseModel):
0094:     filial: Optional[str] = "01"
0095:
0096:
0097: class EstornarPedidoInput(BaseModel):
0098:     filial: str = "01"
0099:     motivo: Optional[str] = None
0100:
0101:
0102: class CancelarNFInput(BaseModel):
0103:     filial: str = "01"
0104:     motivo: Optional[str] = None
0105:
0106:
0107: class EstornarNFInput(BaseModel):
0108:     filial: str = "01"
0109:     motivo: Optional[str] = None
0110:
0111:
0112: class DevolverNFInput(BaseModel):
```

```
0113:     filial: str = "01"
0114:     tes_cod: str
0115:     venc_dias: int = 30
0116:
0117:
0118: class LiberarItemInput(BaseModel):
0119:     filial: str = "01"
0120:     produto: str
0121:     qtd: int
0122:
0123:
0124: class LiberarTudoInput(BaseModel):
0125:     filial: str = "01"
0126:     usar_estoque: bool = True
0127:
0128:
0129: @router.post("/pedidos", responses=common_error_responses(400, 401, 403, 422, 500))
0130: def criar_pedido(dados: PedidoCriarInput, me: dict = Depends(get_current_user)):
0131:     with db_transaction() as (_, cur):
0132:         filial = (dados.filial or "").strip() or "01"
0133:         return criar_pedido_tx(
0134:             cur,
0135:             usuario=me["login"],
0136:             filial=filial,
0137:             valor_total=float(dados.valor_total),
0138:             status=(dados.status or "ABERTO").strip(),
0139:             icms=dados.icms,
0140:             ipi=dados.ipi,
0141:             total_bruto=dados.total_bruto,
0142:         )
0143:
0144:
0145: @router.post("/pedidos/{pedido_id}/itens", responses=common_error_responses(400, 401, 403, 422
, 500))
0146: def adicionar_item(pedido_id: int, dados: ItemCriarInput, me: dict = Depends(get_current_user)
):
0147:     with db_transaction() as (_, cur):
0148:         filial = (dados.filial or "").strip() or "01"
0149:         produto = (dados.produto or "").strip()
0150:         preco_unit = (float(dados.preco_unit) if dados.preco_unit is not None else None)
0151:         if dados.cmv_unit is not None:
0152:             return adicionar_item_tx(
0153:                 cur,
0154:                 usuario=me["login"],
0155:                 pedido_id=int(pedido_id),
0156:                 filial=filial,
0157:                 produto=produto,
0158:                 qtd=int(dados.qtd),
0159:                 total=float(dados.total),
0160:                 preco_unit=preco_unit,
0161:                 cmv_unit=float(dados.cmv_unit),
0162:             )
0163:         return adicionar_item_tx(
0164:             cur,
```

```
0165:             usuario=me["login"],
0166:             pedido_id=int(pedido_id),
0167:             filial=filial,
0168:             produto=produto,
0169:             qtd=int(dados.qtd),
0170:             total=float(dados.total),
0171:             preco_unit=preco_unit,
0172:         )
0173:
0174:
0175: @router.post("/pedidos/{pedido_id}/faturar", responses=common_error_responses(400, 401, 403, 4
22, 500))
0176: def faturar_pedido(pedido_id: int, dados: FaturarPedidoInput, me: dict = Depends(get_current_u
ser)):
0177:     with db_transaction() as (_, cur):
0178:         return faturar_pedido_venda_tx(
0179:             cur,
0180:             usuario=me["login"],
0181:             pedido_id=int(pedido_id),
0182:             filial=dados.filial,
0183:             cliente_cod=dados.cliente_cod,
0184:             venc_dias=int(dados.venc_dias),
0185:             tes_cod=(dados.tes_cod or "001"),
0186:         )
0187:
0188:
0189: @router.post("/pedidos/{pedido_id}/cancelar")
0190: def cancelar_pedido(pedido_id: int, dados: CancelarPedidoInput, me: dict = Depends(get_current
_user)):
0191:     with db_transaction() as (_, cur):
0192:         filial = (dados.filial or "").strip() or "01"
0193:         return cancelar_pedido_tx(
0194:             cur,
0195:             usuario=me["login"],
0196:             pedido_id=int(pedido_id),
0197:             filial=filial,
0198:             modo=(dados.modo or "AUTO"),
0199:             reativar_reserva=bool(dados.reativar_reserva),
0200:         )
0201:
0202:
0203: @router.get("/pedidos/enriquecido")
0204: def listar_pedidos_enriquecido(
0205:     filial: Optional[str] = None,
0206:     status: Optional[str] = None,
0207:     origem: Optional[str] = None,
0208:     de: Optional[date] = None,
0209:     ate: Optional[date] = None,
0210:     limit: int = Query(50, ge=1, le=200),
0211:     offset: int = Query(0, ge=0),
0212:     me: dict = Depends(get_current_user),
0213: ):
0214:     with db_transaction() as (_, cur):
0215:         return listar_pedidos_enriq_tx(
```

```
0216:             cur,
0217:             usuario=me["login"],
0218:             filial=filial,
0219:             status=status,
0220:             origem=origem,
0221:             de=de,
0222:             ate=ate,
0223:             limit=limit,
0224:             offset=offset,
0225:         )
0226:
0227:
0228: @router.get("/pedidos/{pedido_id}")
0229: def obter_pedido(pedido_id: int, me: dict = Depends(get_current_user)):
0230:     with db_transaction() as (_, cur):
0231:         return consultar_pedido_tx(cur, usuario=me["login"], pedido_id=int(pedido_id))
0232:
0233:
0234: @router.get("/pedidos")
0235: def listar_pedidos(
0236:     filial: Optional[str] = None,
0237:     status: Optional[str] = None,
0238:     origem: Optional[str] = None,
0239:     de: Optional[date] = None,
0240:     ate: Optional[date] = None,
0241:     limit: int = Query(50, ge=1, le=200),
0242:     offset: int = Query(0, ge=0),
0243:     me: dict = Depends(get_current_user),
0244: ):
0245:     with db_transaction() as (_, cur):
0246:         return listar_pedidos_tx(
0247:             cur,
0248:             usuario=me["login"],
0249:             filial=filial,
0250:             status=status,
0251:             origem=origem,
0252:             de=de,
0253:             ate=ate,
0254:             limit=limit,
0255:             offset=offset,
0256:         )
0257:
0258:
0259: @router.post("/pedidos/{pedido_id}/recalcular")
0260: def recalcular_pedido(pedido_id: int, dados: RecalcularPedidoInput, me: dict = Depends(get_cur
rent_user)):
0261:     with db_transaction() as (_, cur):
0262:         return pedido_recalcular_totais_tx(
0263:             cur,
0264:             usuario=me["login"],
0265:             pedido_id=int(pedido_id),
0266:             forcar=bool(dados.forcar),
0267:         )
0268:
```

```
0269:
0270: @router.put("/itens/{item_id}")
0271: def editar_item(item_id: int, dados: EditarItemInput, me: dict = Depends(get_current_user)):
0272:     with db_transaction() as (_, cur):
0273:         if dados.cmv_unit is not None:
0274:             return editar_item_tx(
0275:                 cur,
0276:                 usuario=me["login"],
0277:                 item_id=int(item_id),
0278:                 filial=dados.filial,
0279:                 qtd=dados.qtd,
0280:                 total=dados.total,
0281:                 preco_unit=dados.preco_unit,
0282:                 cmv_unit=dados.cmv_unit,
0283:             )
0284:         return editar_item_tx(
0285:             cur,
0286:             usuario=me["login"],
0287:             item_id=int(item_id),
0288:             filial=dados.filial,
0289:             qtd=dados.qtd,
0290:             total=dados.total,
0291:             preco_unit=dados.preco_unit,
0292:         )
0293:
0294:
0295: @router.delete("/itens/{item_id}")
0296: def excluir_item(item_id: int, dados: ExcluirItemInput, me: dict = Depends(get_current_user)):
0297:     with db_transaction() as (_, cur):
0298:         return excluir_item_tx(
0299:             cur,
0300:             usuario=me["login"],
0301:             item_id=int(item_id),
0302:             filial=dados.filial,
0303:         )
0304:
0305:
0306: @router.post("/pedidos/{pedido_id}/estornar")
0307: def estornar_pedido(pedido_id: int, dados: EstornarPedidoInput, me: dict = Depends(get_current
_user)):
0308:     with db_transaction() as (_, cur):
0309:         return estornar_pedido_faturado_venda_tx(
0310:             cur,
0311:             usuario=me["login"],
0312:             pedido_id=int(pedido_id),
0313:             filial=dados.filial,
0314:             motivo=dados.motivo,
0315:         )
0316:
0317:
0318: @router.post("/nf/{nf_id}/cancelar")
0319: def cancelar_nf(nf_id: int, dados: CancelarNFInput, me: dict = Depends(get_current_user)):
0320:     with db_transaction() as (_, cur):
0321:         return cancelar_nf_tx(
```

```
0322:             cur,
0323:             usuario=me["login"],
0324:             nf_id=int(nf_id),
0325:             filial=dados.filial,
0326:             motivo=dados.motivo,
0327:         )
0328:
0329:
0330: @router.post("/nf/{nf_id}/estornar")
0331: def estornar_nf(nf_id: int, dados: EstornarNFInput, me: dict = Depends(get_current_user)):
0332:     with db_transaction() as (_, cur):
0333:         return estornar_nf_tx(
0334:             cur,
0335:             usuario=me["login"],
0336:             nf_id=int(nf_id),
0337:             filial=dados.filial,
0338:             motivo=dados.motivo,
0339:         )
0340:
0341:
0342: @router.post("/nf/{nf_id}/devolver")
0343: def devolver_nf(nf_id: int, dados: DevolverNFInput, me: dict = Depends(get_current_user)):
0344:     with db_transaction() as (_, cur):
0345:         return devolver_nf_tx(
0346:             cur,
0347:             usuario=me["login"],
0348:             nf_origem_id=int(nf_id),
0349:             filial=dados.filial,
0350:             tes_cod=dados.tes_cod,
0351:             venc_dias=int(dados.venc_dias),
0352:         )
0353:
0354:
0355: @router.post("/pedidos/{pedido_id}/liberar")
0356: def liberar_pedido_item(pedido_id: int, dados: LiberarItemInput, me: dict = Depends(get_curren
t_user)):
0357:     with db_transaction() as (_, cur):
0358:         return liberar_item_pedido_tx(
0359:             cur,
0360:             usuario=me["login"],
0361:             pedido_id=int(pedido_id),
0362:             filial=dados.filial,
0363:             produto=dados.produto,
0364:             qtd=int(dados.qtd),
0365:         )
0366:
0367:
0368: @router.post("/pedidos/{pedido_id}/liberar-tudo")
0369: def liberar_pedido_total(pedido_id: int, dados: LiberarTudoInput, me: dict = Depends(get_curre
nt_user)):
0370:     with db_transaction() as (_, cur):
0371:         return liberar_pedido_total_tx(
0372:             cur,
0373:             usuario=me["login"],
```

```
0374:             pedido_id=int(pedido_id),
0375:             filial=dados.filial,
0376:             usar_estoque=bool(dados.usar_estoque),
0377:         )
0378:
0379:
0380: @router.get("/pedidos/{pedido_id}/liberacoes")
0381: def resumo_liberacao_pedido(pedido_id: int, me: dict = Depends(get_current_user)):
0382:     with db_transaction() as (_, cur):
0383:         return resumo_liberacao_pedido_tx(cur, usuario=me["login"], pedido_id=int(pedido_id))
0384:
0385:
0386: @router.get("/pedidos/{pedido_id}/nf-financeiro")
0387: def relatorio_nf_financeiro(
0388:     pedido_id: int,
0389:     nf_status: Optional[str] = None,
0390:     ar_status: Optional[str] = None,
0391:     sc9_status: Optional[str] = None,
0392:     me: dict = Depends(get_current_user),
0393: ):
0394:     with db_transaction() as (_, cur):
0395:         return relatorio_nf_financeiro_tx(
0396:             cur,
0397:             usuario=me["login"],
0398:             pedido_id=int(pedido_id),
0399:             nf_status=nf_status,
0400:             ar_status=ar_status,
0401:             sc9_status=sc9_status,
0402:         )
0403:
0404:
0405: @router.get("/nf/{nf_id}/rastreio")
0406: def relatorio_nf_rastreio(nf_id: int, me: dict = Depends(get_current_user)):
0407:     with db_transaction() as (_, cur):
0408:         return relatorio_nf_rastreio_tx(cur, usuario=me["login"], nf_id=int(nf_id))
0409:
0410:
0411: @router.get("/nf/buscar")
0412: def buscar_nf(
0413:     filial: str,
0414:     doc: str,
0415:     serie: str,
0416:     me: dict = Depends(get_current_user),
0417: ):
0418:     with db_transaction() as (_, cur):
0419:         return buscar_nf_por_doc_serie_tx(
0420:             cur,
0421:             usuario=me["login"],
0422:             filial=filial,
0423:             doc=doc,
0424:             serie=serie,
0425:         )
```

ARQUIVO: app/api/schemas/__init__.py
==========
```
0001: """Schemas compartilhados da camada HTTP."""
```

==========
ARQUIVO: app/api/schemas/errors.py
==========
```
0001: from fastapi import status
0002: from pydantic import BaseModel
0003:
0004:
0005: class ErrorResponse(BaseModel):
0006:     detail: str
0007:     code: str
0008:     error_type: str
0009:     request_id: str | None = None
0010:
0011:
0012: def build_error_response(
0013:     *,
0014:     detail: str,
0015:     code: str,
0016:     error_type: str,
0017:     request_id: str | None,
0018: ) -> dict:
0019:     return {
0020:         "detail": detail,
0021:         "code": code,
0022:         "error_type": error_type,
0023:         "request_id": request_id,
0024:     }
0025:
0026:
0027: def common_error_responses(*status_codes: int) -> dict[int, dict]:
0028:     examples = {
0029:         status.HTTP_400_BAD_REQUEST: {
0030:             "description": "Falha de regra de negócio.",
0031:             "content": {
0032:                 "application/json": {
0033:                     "example": {
0034:                         "detail": "Pedido sem itens (SC6)",
0035:                         "code": "BUSINESS_ERROR",
0036:                         "error_type": "business",
0037:                         "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0",
0038:                     }
0039:                 }
0040:             },
0041:         },
0042:         status.HTTP_401_UNAUTHORIZED: {
0043:             "description": "Falha de autenticação.",
0044:             "content": {
0045:                 "application/json": {
0046:                     "example": {
0047:                         "detail": "Token inválido ou expirado",
```

0048:   "code": "AUTH_ERROR",
0049:   "error_type": "auth",
0050:   "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0",
0051:   }
0052:   }
0053:   },
0054:   },
0055:   status.HTTP_403_FORBIDDEN: {
0056:   "description": "Acesso negado.",
0057:   "content": {
0058:   "application/json": {
0059:   "example": {
0060:   "detail": "Sem permissão. Perfil atual: user",
0061:   "code": "AUTHZ_ERROR",
0062:   "error_type": "authorization",
0063:   "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0",
0064:   }
0065:   }
0066:   },
0067:   },
0068:   status.HTTP_404_NOT_FOUND: {
0069:   "description": "Recurso não encontrado ou endpoint indisponível nesse ambiente.",
0070:   "content": {
0071:   "application/json": {
0072:   "example": {
0073:   "detail": "Not Found",
0074:   "code": "HTTP_404",
0075:   "error_type": "http",
0076:   "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0",
0077:   }
0078:   }
0079:   },
0080:   },
0081:   status.HTTP_409_CONFLICT: {
0082:   "description": "Conflito de dados.",
0083:   "content": {
0084:   "application/json": {
0085:   "example": {
0086:   "detail": "Conflito de dados",
0087:   "code": "CONFLICT_ERROR",
0088:   "error_type": "conflict",
0089:   "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0",
0090:   }
0091:   }
0092:   },
0093:   },
0094:   status.HTTP_422_UNPROCESSABLE_CONTENT: {
0095:   "description": "Payload inválido.",
0096:   "content": {
0097:   "application/json": {
0098:   "example": {
0099:   "detail": "Payload inválido: body.qtd - Input should be greater than o
r equal to 1",
0100:   "code": "VALIDATION_ERROR",

```
0101:                           "error_type": "validation",
0102:                           "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0",
0103:                       }
0104:                   }
0105:               },
0106:           },
0107:           status.HTTP_500_INTERNAL_SERVER_ERROR: {
0108:               "description": "Erro interno.",
0109:               "content": {
0110:                   "application/json": {
0111:                       "example": {
0112:                           "detail": "Erro interno",
0113:                           "code": "INTERNAL_ERROR",
0114:                           "error_type": "internal",
0115:                           "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0",
0116:                       }
0117:                   }
0118:               },
0119:           },
0120:       }
0121:
0122:       response_map: dict[int, dict] = {}
0123:       for status_code in status_codes:
0124:           normalized_status = status.HTTP_422_UNPROCESSABLE_CONTENT if status_code == 422 else s
tatus_code
0125:           response_map[status_code] = {
0126:               "model": ErrorResponse,
0127:               **examples[normalized_status],
0128:           }
0129:       return response_map
```

==========
ARQUIVO: app/core/__init__.py
==========

```
0001:
```

==========
ARQUIVO: app/core/exceptions.py
==========

```
0001: class BusinessError(Exception):
0002:     """Regra de negocio (vira HTTP 400)."""
0003:
0004:     def __init__(self, message: str, code: str = "BUSINESS_ERROR"):
0005:         super().__init__(message)
0006:         self.code = code
0007:         self.message = message
0008:
0009:
0010: class AuthzError(Exception):
0011:     """Sem permissao (vira HTTP 403)."""
0012:
0013:     def __init__(self, message: str = "Sem permissao", code: str = "AUTHZ_ERROR"):
0014:         super().__init__(message)
0015:         self.code = code
```

```
0016:         self.message = message
0017:
0018:
0019: class ConflictError(Exception):
0020:     """Conflito (vira HTTP 409)."""
0021:
0022:     def __init__(self, message: str = "Conflito de dados", code: str = "CONFLICT_ERROR"):
0023:         super().__init__(message)
0024:         self.code = code
0025:         self.message = message
```

==========
ARQUIVO: app/core/security.py
==========

```
0001: from passlib.context import CryptContext
0002:
0003: pwd_ctx = CryptContext(schemes=["pbkdf2_sha256"], deprecated="auto")
0004:
0005: def hash_password(raw: str) -> str:
0006:     return pwd_ctx.hash(raw)
0007:
0008: def verify_password(raw: str, hashed: str) -> bool:
0009:     return pwd_ctx.verify(raw, hashed)
```

==========
ARQUIVO: app/core/settings.py
==========

```
0001: import os
0002:
0003:
0004: def _to_bool(value: str | None, default: bool = False) -> bool:
0005:     if value is None:
0006:         return default
0007:     return value.strip().lower() in {"1", "true", "yes", "on"}
0008:
0009:
0010: def _require_safe_jwt_secret(app_env: str, secret: str) -> str:
0011:     secret = (secret or "").strip()
0012:     if app_env == "production" and secret in {"", "dev-only-change-me", "change-me-in-producti
on"}:
0013:         raise RuntimeError(
0014:             "JWT_SECRET_KEY must be explicitly set to a non-default value when APP_ENV=product
ion."
0015:         )
0016:     return secret
0017:
0018:
0019: APP_TITLE = os.getenv("APP_TITLE", "Mini Protheus - ERP (SQL Server + JWT)")
0020: APP_ENV = os.getenv("APP_ENV", "production").strip().lower() or "production"
0021:
0022: JWT_SECRET_KEY = _require_safe_jwt_secret(APP_ENV, os.getenv("JWT_SECRET_KEY", "dev-only-chang
e-me"))
0023: JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")
0024: JWT_EXPIRE_MINUTES = int(os.getenv("JWT_EXPIRE_MINUTES", "240"))
```

```
0025: BOOTSTRAP_ENABLED = _to_bool(os.getenv("BOOTSTRAP_ENABLED"), default=False)
0026: BOOTSTRAP_ALLOWED_ENVS = {"development", "setup"}
0027: BOOTSTRAP_POLICY = (
0028:     "One-time setup: /bootstrap only responds in development/setup environments, "
0029:     "must be explicitly enabled, and is permanently blocked by business rule after "
0030:     "the first user exists."
0031: )
0032: CORS_ALLOW_ORIGINS = [
0033:     origin.strip()
0034:     for origin in os.getenv("CORS_ALLOW_ORIGINS", "http://localhost:5173").split(",")
0035:     if origin.strip()
0036: ]
0037:
0038: DB_CONN_STR = os.getenv(
0039:     "DB_CONN_STR",
0040:     (
0041:         "DRIVER={ODBC Driver 18 for SQL Server};"
0042:         "SERVER=localhost;"
0043:         "DATABASE=ERP_PROTHEUS;"
0044:         "Trusted_Connection=yes;"
0045:         "Encrypt=yes;"
0046:         "TrustServerCertificate=yes;"
0047:     ),
0048: )
```

==========
ARQUIVO: app/domain/__init__.py
==========

```
0001:
```

==========
ARQUIVO: app/domain/errors.py
==========

```
0001: class BusinessError(Exception):
0002:     """
0003:     Erro de regra de negócio (vira 400/422 na API).
0004:     """
0005:
0006:     def __init__(self, message: str, code: str = "BUSINESS_ERROR"):
0007:         super().__init__(message)
0008:         self.code = code
0009:         self.message = message
0010:
0011:
0012: class AuthError(Exception):
0013:     """
0014:     Erro de autenticação/autorização (vira 401/403).
0015:     """
0016:
0017:     def __init__(self, message: str = "Não autorizado", code: str = "AUTH_ERROR"):
0018:         super().__init__(message)
0019:         self.code = code
0020:         self.message = message
```

==========
ARQUIVO: app/domain/preco_models.py
==========

```
0001: from dataclasses import dataclass
0002: from datetime import date
0003: from typing import Optional
0004:
0005: @dataclass
0006: class TabelaPreco:
0007:     id: int
0008:     filial: str
0009:     codigo: str
0010:     descricao: str
0011:     ativa: bool
0012:
0013: @dataclass
0014: class PrecoProduto:
0015:     id: int
0016:     filial: str
0017:     tabela_id: int
0018:     produto: str
0019:     preco: float
0020:     dt_ini: date
0021:     dt_fim: Optional[date]
```

==========
ARQUIVO: app/infra/__init__.py
==========

```
0001:
```

==========
ARQUIVO: app/infra/db.py
==========

```
0001: import pyodbc
0002: from contextlib import contextmanager
0003: from contextvars import ContextVar
0004: from typing import Iterator, Tuple, Optional, List, Dict, Any
0005:
0006: CONN_STR = (
0007:     "DRIVER={ODBC Driver 18 for SQL Server};"
0008:     "SERVER=localhost;"
0009:     "DATABASE=ERP_PROTHEUS;"
0010:     "Trusted_Connection=yes;"
0011:     "Encrypt=yes;"
0012:     "TrustServerCertificate=yes;"
0013: )
0014:
0015: _current_user: ContextVar[str | None] = ContextVar("db_current_user", default=None)
0016: _current_request_id: ContextVar[str | None] = ContextVar("db_request_id", default=None)
0017: _current_request_path: ContextVar[str | None] = ContextVar("db_request_path", default=None)
0018:
0019:
0020: def set_current_user(login: str | None) -> None:
0021:     _current_user.set((login or "").strip() or None)
```

```
0022:
0023:
0024: def get_current_user_ctx() -> str | None:
0025:     return _current_user.get()
0026:
0027:
0028: def set_current_request(request_id: str | None, path: str | None) -> None:
0029:     _current_request_id.set((request_id or "").strip() or None)
0030:     _current_request_path.set((path or "").strip() or None)
0031:
0032:
0033: def get_current_request_id() -> str | None:
0034:     return _current_request_id.get()
0035:
0036:
0037: def get_current_request_path() -> str | None:
0038:     return _current_request_path.get()
0039:
0040: def get_connection() -> pyodbc.Connection:
0041:     return pyodbc.connect(CONN_STR, autocommit=False, timeout=30)
0042:
0043: @contextmanager
0044: def db_transaction() -> Iterator[Tuple[pyodbc.Connection, pyodbc.Cursor]]:
0045:     conn = get_connection()
0046:     cur = conn.cursor()
0047:     try:
0048:         cur.execute("SET NOCOUNT ON;")
0049:         user = get_current_user_ctx()
0050:         if user:
0051:             # registra usuário da sessão para auditoria via trigger
0052:             cur.execute("EXEC sp_set_session_context @key=N'app_user', @value=?", (user,))
0053:         req_id = get_current_request_id()
0054:         req_path = get_current_request_path()
0055:         if req_id:
0056:             cur.execute("EXEC sp_set_session_context @key=N'app_req_id', @value=?", (req_id,))
0057:         if req_path:
0058:             cur.execute("EXEC sp_set_session_context @key=N'app_path', @value=?", (req_path,))
0059:         yield conn, cur
0060:         conn.commit()
0061:     except Exception:
0062:         conn.rollback()
0063:         raise
0064:     finally:
0065:         try:
0066:             cur.close()
0067:         finally:
0068:             conn.close()
0069:
0070: def fetchone_dict(cur: pyodbc.Cursor) -> Optional[Dict[str, Any]]:
0071:     row = cur.fetchone()
0072:     if row is None:
0073:         return None
0074:     if cur.description is None:
0075:         raise RuntimeError("fetchone_dict chamado sem SELECT (cur.description=None)")
```

```
0076:     cols = [c[0] for c in cur.description]
0077:     return dict(zip(cols, row))
0078:
0079: def fetchall_dict(cur: pyodbc.Cursor) -> List[Dict[str, Any]]:
0080:     rows = cur.fetchall()
0081:     if cur.description is None:
0082:         raise RuntimeError("fetchall_dict chamado sem SELECT (cur.description=None)")
0083:     cols = [c[0] for c in cur.description]
0084:     return [dict(zip(cols, r)) for r in rows]
```

==========
ARQUIVO: app/infra/repositories/__init__.py
==========

```
0001:
```

==========
ARQUIVO: app/infra/repositories/alertas_repo.py
==========

```
0001: from datetime import date
0002:
0003: from app.infra.db import fetchone_dict
0004:
0005:
0006: def _count_query(cur, sql: str, params: tuple) -> int:
0007:     cur.execute(sql, params)
0008:     row = fetchone_dict(cur) or {}
0009:     return int(row.get("total") or 0)
0010:
0011:
0012: def contar_ar_aberto_tx(cur, filial: str) -> int:
0013:     return _count_query(
0014:         cur,
0015:         """
0016:         SELECT COUNT(1) AS total
0017:         FROM dbo.SE1_AR
0018:         WHERE E1_FILIAL = ? AND E1_STATUS = 'ABERTO'
0019:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0020:         """,
0021:         (filial,),
0022:     )
0023:
0024:
0025: def contar_ar_vencido_tx(cur, filial: str, hoje: date) -> int:
0026:     return _count_query(
0027:         cur,
0028:         """
0029:         SELECT COUNT(1) AS total
0030:         FROM dbo.SE1_AR
0031:         WHERE E1_FILIAL = ? AND E1_STATUS = 'ABERTO' AND E1_VENC < ?
0032:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0033:         """,
0034:         (filial, hoje),
0035:     )
0036:
```

```
0037:
0038: def contar_ar_vence_hoje_tx(cur, filial: str, hoje: date) -> int:
0039:     return _count_query(
0040:         cur,
0041:         """
0042:         SELECT COUNT(1) AS total
0043:         FROM dbo.SE1_AR
0044:         WHERE E1_FILIAL = ? AND E1_STATUS = 'ABERTO' AND CAST(E1_VENC AS DATE) = ?
0045:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0046:         """,
0047:         (filial, hoje),
0048:     )
0049:
0050:
0051: def contar_ar_vencendo_ate_tx(cur, filial: str, de: date, ate: date) -> int:
0052:     return _count_query(
0053:         cur,
0054:         """
0055:         SELECT COUNT(1) AS total
0056:         FROM dbo.SE1_AR
0057:         WHERE E1_FILIAL = ? AND E1_STATUS = 'ABERTO'
0058:           AND CAST(E1_VENC AS DATE) >= ? AND CAST(E1_VENC AS DATE) <= ?
0059:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0060:         """,
0061:         (filial, de, ate),
0062:     )
0063:
0064:
0065: def contar_ap_aberto_tx(cur, filial: str) -> int:
0066:     return _count_query(
0067:         cur,
0068:         """
0069:         SELECT COUNT(1) AS total
0070:         FROM dbo.SF1_AP
0071:         WHERE F1_FILIAL = ? AND F1_STATUS = 'ABERTO'
0072:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0073:         """,
0074:         (filial,),
0075:     )
0076:
0077:
0078: def contar_ap_vencido_tx(cur, filial: str, hoje: date) -> int:
0079:     return _count_query(
0080:         cur,
0081:         """
0082:         SELECT COUNT(1) AS total
0083:         FROM dbo.SF1_AP
0084:         WHERE F1_FILIAL = ? AND F1_STATUS = 'ABERTO' AND F1_VENC < ?
0085:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0086:         """,
0087:         (filial, hoje),
0088:     )
0089:
0090:
```

```
0091: def contar_ap_vence_hoje_tx(cur, filial: str, hoje: date) -> int:
0092:     return _count_query(
0093:         cur,
0094:         """
0095:         SELECT COUNT(1) AS total
0096:         FROM dbo.SF1_AP
0097:         WHERE F1_FILIAL = ? AND F1_STATUS = 'ABERTO' AND CAST(F1_VENC AS DATE) = ?
0098:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0099:         """,
0100:         (filial, hoje),
0101:     )
0102:
0103:
0104: def contar_ap_vencendo_ate_tx(cur, filial: str, de: date, ate: date) -> int:
0105:     return _count_query(
0106:         cur,
0107:         """
0108:         SELECT COUNT(1) AS total
0109:         FROM dbo.SF1_AP
0110:         WHERE F1_FILIAL = ? AND F1_STATUS = 'ABERTO'
0111:           AND CAST(F1_VENC AS DATE) >= ? AND CAST(F1_VENC AS DATE) <= ?
0112:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0113:         """,
0114:         (filial, de, ate),
0115:     )
0116:
0117:
0118: def contar_reservas_abertas_tx(cur, filial: str) -> int:
0119:     return _count_query(
0120:         cur,
0121:         """
0122:         SELECT COUNT(1) AS total
0123:         FROM dbo.SZ0_RESERVA
0124:         WHERE Z0_FILIAL = ? AND Z0_STATUS = 'ABERTA'
0125:         """,
0126:         (filial,),
0127:     )
0128:
0129:
0130: def contar_pedidos_abertos_sem_itens_tx(cur, filial: str) -> int:
0131:     return _count_query(
0132:         cur,
0133:         """
0134:         WITH AggSC6 AS (
0135:           SELECT C6_PEDIDO_ID, COUNT(1) AS qtd_itens
0136:           FROM dbo.SC6_ITENS
0137:           WHERE (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0138:           GROUP BY C6_PEDIDO_ID
0139:         )
0140:         SELECT COUNT(1) AS total
0141:         FROM dbo.SC5_PEDIDOS p
0142:         LEFT JOIN AggSC6 s ON s.C6_PEDIDO_ID = p.ID
0143:         WHERE p.C5_FILIAL = ? AND p.C5_STATUS = 'ABERTO'
0144:           AND COALESCE(s.qtd_itens, 0) = 0
```

```
0145:           AND (p.D_E_L_E_T_ IS NULL OR p.D_E_L_E_T_ <> '*')
0146:         """,
0147:         (filial,),
0148:     )
0149:
0150:
0151: def contar_pedidos_abertos_com_itens_tx(cur, filial: str) -> int:
0152:     return _count_query(
0153:         cur,
0154:         """
0155:         WITH AggSC6 AS (
0156:           SELECT C6_PEDIDO_ID, COUNT(1) AS qtd_itens
0157:           FROM dbo.SC6_ITENS
0158:           WHERE (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0159:           GROUP BY C6_PEDIDO_ID
0160:         )
0161:         SELECT COUNT(1) AS total
0162:         FROM dbo.SC5_PEDIDOS p
0163:         INNER JOIN AggSC6 s ON s.C6_PEDIDO_ID = p.ID
0164:         WHERE p.C5_FILIAL = ? AND p.C5_STATUS = 'ABERTO'
0165:           AND s.qtd_itens > 0
0166:           AND (p.D_E_L_E_T_ IS NULL OR p.D_E_L_E_T_ <> '*')
0167:         """,
0168:         (filial,),
0169:     )
0170:
0171:
0172: def contar_contas_caixa_ativas_tx(cur, filial: str) -> int:
0173:     return _count_query(
0174:         cur,
0175:         """
0176:         SELECT COUNT(1) AS total
0177:         FROM dbo.SE5_CONTAS
0178:         WHERE E5_FILIAL = ? AND E5_ATIVA = 1
0179:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0180:         """,
0181:         (filial,),
0182:     )
```

==========
ARQUIVO: app/infra/repositories/app_setup_repo.py
==========

```
0001: from app.infra.db import fetchone_dict
0002:
0003:
0004: def ensure_app_setup_state_table_tx(cur) -> None:
0005:     cur.execute(
0006:         """
0007:         IF OBJECT_ID('dbo.APP_SETUP_STATE', 'U') IS NULL
0008:         BEGIN
0009:             CREATE TABLE dbo.APP_SETUP_STATE (
0010:                 ID INT IDENTITY(1,1) PRIMARY KEY,
0011:                 SETUP_COMPLETED BIT NOT NULL DEFAULT 0,
0012:                 COMPLETED_AT DATETIME2 NULL,
```

```
0013:                 COMPLETED_BY NVARCHAR(100) NULL
0014:             );
0015:
0016:             INSERT INTO dbo.APP_SETUP_STATE (SETUP_COMPLETED, COMPLETED_AT, COMPLETED_BY)
0017:             VALUES (0, NULL, NULL);
0018:         END
0019:         """
0020:     )
0021:
0022:
0023: def app_setup_is_completed_tx(cur) -> bool:
0024:     ensure_app_setup_state_table_tx(cur)
0025:     cur.execute(
0026:         """
0027:         SELECT TOP 1 SETUP_COMPLETED
0028:         FROM dbo.APP_SETUP_STATE
0029:         ORDER BY ID
0030:         """
0031:     )
0032:     row = fetchone_dict(cur)
0033:     return bool(row and row.get("SETUP_COMPLETED"))
0034:
0035:
0036: def app_setup_status_tx(cur) -> dict:
0037:     ensure_app_setup_state_table_tx(cur)
0038:     cur.execute(
0039:         """
0040:         SELECT TOP 1 SETUP_COMPLETED, COMPLETED_AT, COMPLETED_BY
0041:         FROM dbo.APP_SETUP_STATE
0042:         ORDER BY ID
0043:         """
0044:     )
0045:     row = fetchone_dict(cur) or {}
0046:     return {
0047:         "setup_completed": bool(row.get("SETUP_COMPLETED")),
0048:         "completed_at": row.get("COMPLETED_AT"),
0049:         "completed_by": row.get("COMPLETED_BY"),
0050:     }
0051:
0052:
0053: def app_setup_mark_completed_tx(cur, login: str) -> None:
0054:     ensure_app_setup_state_table_tx(cur)
0055:     cur.execute(
0056:         """
0057:         UPDATE dbo.APP_SETUP_STATE
0058:         SET SETUP_COMPLETED = 1,
0059:             COMPLETED_AT = SYSDATETIME(),
0060:             COMPLETED_BY = ?
0061:         WHERE ID = (
0062:             SELECT TOP 1 ID
0063:             FROM dbo.APP_SETUP_STATE
0064:             ORDER BY ID
0065:         )
0066:         """,
```

```
0067:         (login,),
0068:     )
```

==========
ARQUIVO: app/infra/repositories/faturamento_repo.py
==========

```
0001: from __future__ import annotations
0002:
0003: from datetime import datetime
0004: from app.infra.db import fetchone_dict
0005: from app.infra.repositories.sx6_repo import sx6_next_tx
0006: from app.infra.repositories.serie_repo import resolve_serie_tx, validar_serie_tx
0007:
0008:
0009: def _nf_doc_from_id(nf_id: int) -> str:
0010:     # Protheus-like: número sequencial como string (zero-padded)
0011:     return f"{int(nf_id):09d}"
0012:
0013:
0014: def sf2_criar_tx(
0015:     cur,
0016:     filial: str,
0017:     pedido_id: int | None,
0018:     cliente: str | None,
0019:     cliente_cod: str | None,
0020:     valor_total: float,
0021:     icms: float,
0022:     ipi: float,
0023:     total_bruto: float,
0024:     tes_cod: str,
0025:     cfop: str | None = None,
0026:     doc: str | None = None,
0027:     serie: str | None = None,
0028:     status: str = "EMITIDA",
0029:     origem: str = "VENDA",
0030:     pis: float = 0.0,
0031:     cofins: float = 0.0,
0032:     icms_st: float = 0.0,
0033:     difal: float = 0.0,
0034: ) -> dict:
0035:     if not serie:
0036:         serie = resolve_serie_tx(cur, filial=filial, tabela="NF")
0037:     validar_serie_tx(cur, filial=filial, tabela="NF", serie=serie)
0038:     if not doc:
0039:         doc = str(sx6_next_tx(cur, filial=filial, serie=serie, tabela="SF2")).zfill(9)
0040:
0041:     cur.execute(
0042:         """
0043:         INSERT INTO dbo.SF2_NOTAS
0044:           (F2_DOC, F2_SERIE, F2_EMISSAO, F2_FILIAL, F2_CLIENTE, F2_CLIENTE_COD,
0045:            F2_VALOR, F2_ICMS, F2_IPI, F2_PIS, F2_COFINS, F2_ICMS_ST, F2_DIFAL,
0046:            F2_TOTAL_BRUTO, F2_STATUS, F2_PEDIDO_ID,
0047:            F2_TES, F2_CFOP, F2_ORIGEM)
0048:         VALUES (?, ?, SYSDATETIME(), ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
```

```
0049:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0050:         """,
0051:         (
0052:             doc,
0053:             serie,
0054:             filial,
0055:             cliente,
0056:             cliente_cod,
0057:             float(valor_total),
0058:             float(icms),
0059:             float(ipi),
0060:             float(pis),
0061:             float(cofins),
0062:             float(icms_st),
0063:             float(difal),
0064:             float(total_bruto),
0065:             status,
0066:             int(pedido_id) if pedido_id is not None else None,
0067:             tes_cod,
0068:             cfop,
0069:             origem,
0070:         ),
0071:     )
0072:     row = fetchone_dict(cur)
0073:     nf_id = int(row["id"])
0074:
0075:     return {"id": nf_id, "doc": doc, "serie": serie}
0076:
0077:
0078: def sd2_item_criar_tx(
0079:     cur,
0080:     nf_id: int,
0081:     doc: str,
0082:     serie: str,
0083:     item: int,
0084:     filial: str,
0085:     pedido_id: int | None,
0086:     produto: str,
0087:     qtd: int,
0088:     preco_unit: float | None,
0089:     total: float,
0090:     icms: float,
0091:     ipi: float,
0092:     tes_cod: str,
0093:     cfop: str | None = None,
0094:     pis: float = 0.0,
0095:     cofins: float = 0.0,
0096:     icms_st: float = 0.0,
0097:     difal: float = 0.0,
0098:     cst_icms: str | None = None,
0099:     csosn: str | None = None,
0100:     cst_pis: str | None = None,
0101:     cst_cofins: str | None = None,
0102: ) -> None:
```

```
0103:     d2_item = str(int(item)).zfill(4)
0104:     cur.execute(
0105:         """
0106:         INSERT INTO dbo.SD2_ITENS
0107:           (D2_NF_ID, D2_DOC, D2_SERIE, D2_ITEM, D2_FILIAL, D2_PEDIDO_ID,
0108:            D2_PRODUTO, D2_QTD, D2_PRECO_UNIT, D2_TOTAL, D2_ICMS, D2_IPI,
0109:            D2_PIS, D2_COFINS, D2_ICMS_ST, D2_DIFAL,
0110:            D2_CST_ICMS, D2_CSOSN, D2_CST_PIS, D2_CST_COFINS,
0111:            D2_TES, D2_CFOP, D2_ITEM_NUM)
0112:         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
0113:         """,
0114:         (
0115:             int(nf_id),
0116:             doc,
0117:             serie,
0118:             int(item),
0119:             filial,
0120:             int(pedido_id) if pedido_id is not None else None,
0121:             produto,
0122:             int(qtd),
0123:             float(preco_unit) if preco_unit is not None else None,
0124:             float(total),
0125:             float(icms),
0126:             float(ipi),
0127:             float(pis),
0128:             float(cofins),
0129:             float(icms_st),
0130:             float(difal),
0131:             cst_icms,
0132:             csosn,
0133:             cst_pis,
0134:             cst_cofins,
0135:             tes_cod,
0136:             cfop,
0137:             d2_item,
0138:         ),
0139:     )
0140:
0141:
0142: def sd2_itens_por_nf_tx(cur, nf_id: int, somente_ativos: bool = True) -> list[dict]:
0143:     sql = """
0144:         SELECT *
0145:         FROM dbo.SD2_ITENS
0146:         WHERE D2_NF_ID=?
0147:     """
0148:     params: list = [int(nf_id)]
0149:     if somente_ativos:
0150:         sql += " AND (D2_STATUS IS NULL OR D2_STATUS<>'CANCELADO')"
0151:     sql += " ORDER BY ID ASC"
0152:     cur.execute(sql, tuple(params))
0153:     rows = cur.fetchall()
0154:     if not rows:
0155:         return []
0156:     cols = [c[0] for c in cur.description]
```

```
0157:     return [dict(zip(cols, r)) for r in rows]
0158:
0159:
0160: def sd2_count_por_nf_tx(cur, nf_id: int) -> int:
0161:     cur.execute(
0162:         """
0163:         SELECT COUNT(*) AS qtd
0164:         FROM dbo.SD2_ITENS
0165:         WHERE D2_NF_ID=?
0166:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0167:         """,
0168:         (int(nf_id),),
0169:     )
0170:     row = fetchone_dict(cur)
0171:     return int(row["qtd"] or 0) if row else 0
0172:
0173:
0174: def sf2_buscar_por_pedido_tx(cur, filial: str, pedido_id: int) -> list[dict]:
0175:     cur.execute(
0176:         """
0177:         SELECT *
0178:         FROM dbo.SF2_NOTAS
0179:         WHERE F2_FILIAL=? AND F2_PEDIDO_ID=?
0180:         ORDER BY ID DESC
0181:         """,
0182:         (filial, int(pedido_id)),
0183:     )
0184:     rows = cur.fetchall()
0185:     if not rows:
0186:         return []
0187:     cols = [c[0] for c in cur.description]
0188:     return [dict(zip(cols, r)) for r in rows]
0189:
0190:
0191: def sf2_get_tx(cur, nf_id: int) -> dict | None:
0192:     cur.execute(
0193:         """
0194:         SELECT *
0195:         FROM dbo.SF2_NOTAS
0196:         WHERE ID=?
0197:         """,
0198:         (int(nf_id),),
0199:     )
0200:     row = fetchone_dict(cur)
0201:     return row
0202:
0203:
0204: def sf2_buscar_por_doc_serie_tx(cur, filial: str, doc: str, serie: str) -> dict | None:
0205:     cur.execute(
0206:         """
0207:         SELECT TOP 1 *
0208:         FROM dbo.SF2_NOTAS
0209:         WHERE F2_FILIAL=? AND F2_DOC=? AND F2_SERIE=?
0210:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
```

```
0211:         ORDER BY ID DESC
0212:         """,
0213:         (filial, str(doc).strip(), str(serie).strip()),
0214:     )
0215:     return fetchone_dict(cur)
0216:
0217:
0218: def sd2_itens_por_nf_id_tx(cur, nf_id: int) -> list[dict]:
0219:     cur.execute(
0220:         """
0221:         SELECT *
0222:         FROM dbo.SD2_ITENS
0223:         WHERE D2_NF_ID=?
0224:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0225:         ORDER BY ID ASC
0226:         """,
0227:         (int(nf_id),),
0228:     )
0229:     rows = cur.fetchall()
0230:     if not rows:
0231:         return []
0232:     cols = [c[0] for c in cur.description]
0233:     return [dict(zip(cols, r)) for r in rows]
0234:
0235:
0236: def sf2_cancelar_tx(cur, nf_id: int, usuario: str | None, motivo: str | None) -> None:
0237:     cur.execute(
0238:         """
0239:         UPDATE dbo.SF2_NOTAS
0240:         SET F2_STATUS='CANCELADA',
0241:             F2_DATA_CANCEL=SYSDATETIME(),
0242:             F2_USUARIO_CANCEL=?,
0243:             F2_MOTIVO_CANCEL=?,
0244:             D_E_L_E_T_='*',
0245:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0246:         WHERE ID=? AND F2_STATUS<>'CANCELADA'
0247:         """,
0248:         (usuario, motivo, int(nf_id)),
0249:     )
0250:
0251:
0252: def sd2_cancelar_por_nf_tx(cur, nf_id: int, usuario: str | None, motivo: str | None) -> int:
0253:     cur.execute(
0254:         """
0255:         UPDATE dbo.SD2_ITENS
0256:         SET D2_STATUS='CANCELADO',
0257:             D2_DATA_CANCEL=SYSDATETIME(),
0258:             D2_USUARIO_CANCEL=?,
0259:             D2_MOTIVO_CANCEL=?,
0260:             D_E_L_E_T_='*',
0261:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0262:         WHERE D2_NF_ID=? AND (D2_STATUS IS NULL OR D2_STATUS<>'CANCELADO')
0263:         """,
0264:         (usuario, motivo, int(nf_id)),
```

```
0265:     )
0266:     return int(cur.rowcount or 0)
```

==========
ARQUIVO: app/infra/repositories/fechamento_repo.py
==========

```
0001: from datetime import date
0002:
0003: from app.core.exceptions import BusinessError
0004: from app.infra.db import fetchone_dict
0005:
0006:
0007: def fechamento_periodo_aberto_tx(cur, filial: str, data_ref: date) -> bool:
0008:     cur.execute(
0009:         """
0010:         SELECT TOP 1 F7_FECHADO
0011:         FROM dbo.SF7_FECHAMENTO
0012:         WHERE F7_FILIAL=? AND F7_ANO=? AND F7_MES=?
0013:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0014:         """,
0015:         (filial, int(data_ref.year), int(data_ref.month)),
0016:     )
0017:     row = fetchone_dict(cur)
0018:     if not row:
0019:         return True
0020:     return not bool(row["F7_FECHADO"])
0021:
0022:
0023: def fechamento_validar_periodo_tx(cur, filial: str, data_ref: date) -> None:
0024:     if not fechamento_periodo_aberto_tx(cur, filial, data_ref):
0025:         raise BusinessError("Período fechado para esta filial")
0026:
0027:
0028: def fechamento_fechar_tx(cur, filial: str, ano: int, mes: int, usuario: str | None):
0029:     cur.execute(
0030:         """
0031:         SELECT ID FROM dbo.SF7_FECHAMENTO
0032:         WHERE F7_FILIAL=? AND F7_ANO=? AND F7_MES=?
0033:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0034:         """,
0035:         (filial, int(ano), int(mes)),
0036:     )
0037:     row = cur.fetchone()
0038:     if row:
0039:         cur.execute(
0040:             """
0041:             UPDATE dbo.SF7_FECHAMENTO
0042:             SET F7_FECHADO=1, F7_USUARIO=?, F7_DATA=SYSDATETIME()
0043:             WHERE F7_FILIAL=? AND F7_ANO=? AND F7_MES=?
0044:             """,
0045:             (usuario, filial, int(ano), int(mes)),
0046:         )
0047:         return
0048:     cur.execute(
```

```
0049:         """
0050:         INSERT INTO dbo.SF7_FECHAMENTO (F7_FILIAL, F7_ANO, F7_MES, F7_FECHADO, F7_USUARIO)
0051:         VALUES (?, ?, ?, 1, ?)
0052:         """,
0053:         (filial, int(ano), int(mes), usuario),
0054:     )
0055:
0056:
0057: def fechamento_abrir_tx(cur, filial: str, ano: int, mes: int, usuario: str | None):
0058:     cur.execute(
0059:         """
0060:         UPDATE dbo.SF7_FECHAMENTO
0061:         SET F7_FECHADO=0, F7_USUARIO=?, F7_DATA=SYSDATETIME()
0062:         WHERE F7_FILIAL=? AND F7_ANO=? AND F7_MES=?
0063:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0064:         """,
0065:         (usuario, filial, int(ano), int(mes)),
0066:     )
```

==========
ARQUIVO: app/infra/repositories/financeiro_repo.py
==========

```
0001: from datetime import date, datetime, timedelta
0002:
0003: import pyodbc
0004:
0005: from app.core.exceptions import BusinessError
0006: from app.infra.db import fetchall_dict, fetchone_dict
0007: from app.infra.repositories.serie_repo import resolve_serie_tx, validar_serie_tx
0008: from app.infra.repositories.sx6_repo import sx6_next_tx
0009:
0010:
0011: def _obter_maior_numero_serie_tx(cur, tabela_fisica: str, campo_num: str, filial: str, serie:
str) -> int:
0012:     cur.execute(
0013:         f"""
0014:         SELECT MAX(TRY_CONVERT(INT, {campo_num})) AS maior_num
0015:         FROM dbo.{tabela_fisica}
0016:         WHERE {"E1_FILIAL" if tabela_fisica == "SE1_AR" else "F1_FILIAL"} = ?
0017:           AND {"E1_SERIE" if tabela_fisica == "SE1_AR" else "F1_SERIE"} = ?
0018:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0019:         """,
0020:         (filial, serie),
0021:     )
0022:     row = fetchone_dict(cur) or {}
0023:     valor = row.get("maior_num")
0024:     return int(valor) if valor is not None else 0
0025:
0026:
0027: def _alinhar_sx6_com_serie_tx(cur, filial: str, serie: str, tabela_seq: str, tabela_fisica: st
r, campo_num: str) -> None:
0028:     maior_existente = _obter_maior_numero_serie_tx(cur, tabela_fisica=tabela_fisica, campo_num
=campo_num, filial=filial, serie=serie)
0029:     cur.execute(
```

```
0030:         """
0031:         SELECT TOP 1 ID, X6_SEQ
0032:         FROM dbo.SX6_SEQ WITH (UPDLOCK, ROWLOCK)
0033:         WHERE X6_FILIAL=? AND X6_SERIE=? AND X6_TABELA=?
0034:         """,
0035:         (filial, serie, tabela_seq),
0036:     )
0037:     row = fetchone_dict(cur)
0038:     if row:
0039:         seq_atual = int(row["X6_SEQ"])
0040:         if seq_atual < maior_existente:
0041:             cur.execute("UPDATE dbo.SX6_SEQ SET X6_SEQ=? WHERE ID=?", (maior_existente, int(ro
w["ID"])))
0042:         return
0043:
0044:     cur.execute(
0045:         """
0046:         INSERT INTO dbo.SX6_SEQ (X6_FILIAL, X6_SERIE, X6_TABELA, X6_SEQ)
0047:         VALUES (?, ?, ?, ?)
0048:         """,
0049:         (filial, serie, tabela_seq, maior_existente),
0050:     )
0051:
0052:
0053: def ar_criar_tx(
0054:     cur,
0055:     filial: str,
0056:     cliente: str | None,
0057:     valor: float,
0058:     ref_id: int,
0059:     venc_dias: int,
0060:     cliente_cod: str | None = None,
0061:     serie: str | None = None,
0062: ) -> None:
0063:     """Cria título em SE1_AR (Contas a Receber)."""
0064:
0065:     data = datetime.now()
0066:     venc: date = (data + timedelta(days=int(venc_dias))).date()
0067:
0068:     serie = (serie or resolve_serie_tx(cur, filial=filial, tabela="AR")).strip()
0069:     validar_serie_tx(cur, filial=filial, tabela="AR", serie=serie)
0070:     _alinhar_sx6_com_serie_tx(
0071:         cur,
0072:         filial=filial,
0073:         serie=serie,
0074:         tabela_seq="SE1",
0075:         tabela_fisica="SE1_AR",
0076:         campo_num="E1_NUM",
0077:     )
0078:
0079:     for _ in range(5):
0080:         e1_num = str(sx6_next_tx(cur, filial=filial, serie=serie, tabela="SE1")).zfill(9)
0081:         try:
0082:             cur.execute(
```

```
0083:                 """
0084:                 INSERT INTO dbo.SE1_AR
0085:                     (E1_DATA,
0086:                      E1_VENC,
0087:                      E1_CLIENTE,
0088:                      E1_CLIENTE_COD,
0089:                      E1_VALOR,
0090:                      E1_STATUS,
0091:                      E1_REF,
0092:                      E1_NUM,
0093:                      E1_SERIE,
0094:                      E1_FILIAL)
0095:                 VALUES (?, ?, ?, ?, ?, 'ABERTO', ?, ?, ?, ?)
0096:                 """,
0097:                 (
0098:                     data,
0099:                     venc,
0100:                     cliente,
0101:                     cliente_cod,
0102:                     float(valor),
0103:                     str(ref_id),
0104:                     e1_num,
0105:                     serie,
0106:                     filial,
0107:                 ),
0108:             )
0109:             break
0110:         except pyodbc.IntegrityError as exc:
0111:             msg = str(exc)
0112:             if "UX_SE1_FILIAL_NUM_SERIE" in msg or "2601" in msg or "2627" in msg:
0113:                 continue
0114:             raise
0115:     else:
0116:         raise BusinessError("Falha ao gerar E1_NUM único para SE1_AR")
0117:
0118:
0119: def ap_criar_tx(
0120:     cur,
0121:     filial: str,
0122:     forn: str | None,
0123:     valor: float,
0124:     ref: str,
0125:     venc_dias: int,
0126:     forn_cod: str | None = None,
0127:     serie: str | None = None,
0128: ) -> None:
0129:     """Cria título em SF1_AP (Contas a Pagar)."""
0130:
0131:     data = datetime.now()
0132:     venc: date = (data + timedelta(days=int(venc_dias))).date()
0133:
0134:     serie = (serie or resolve_serie_tx(cur, filial=filial, tabela="AP")).strip()
0135:     validar_serie_tx(cur, filial=filial, tabela="AP", serie=serie)
0136:     _alinhar_sx6_com_serie_tx(
```

```
0137:         cur,
0138:         filial=filial,
0139:         serie=serie,
0140:         tabela_seq="SF1",
0141:         tabela_fisica="SF1_AP",
0142:         campo_num="F1_NUM",
0143:     )
0144:
0145:     for _ in range(5):
0146:         f1_num = str(sx6_next_tx(cur, filial=filial, serie=serie, tabela="SF1")).zfill(9)
0147:         try:
0148:             cur.execute(
0149:                 """
0150:                 INSERT INTO dbo.SF1_AP
0151:                     (F1_DATA,
0152:                      F1_VENC,
0153:                      F1_FORN,
0154:                      F1_FORN_COD,
0155:                      F1_VALOR,
0156:                      F1_STATUS,
0157:                      F1_REF,
0158:                      F1_NUM,
0159:                      F1_SERIE,
0160:                      F1_FILIAL)
0161:                 VALUES (?, ?, ?, ?, ?, 'ABERTO', ?, ?, ?, ?)
0162:                 """,
0163:                 (
0164:                     data,
0165:                     venc,
0166:                     forn,
0167:                     forn_cod,
0168:                     float(valor),
0169:                     ref,
0170:                     f1_num,
0171:                     serie,
0172:                     filial,
0173:                 ),
0174:             )
0175:             break
0176:         except pyodbc.IntegrityError as exc:
0177:             msg = str(exc)
0178:             if "UX_SF1_FILIAL_NUM_SERIE" in msg or "2601" in msg or "2627" in msg:
0179:                 continue
0180:             raise
0181:     else:
0182:         raise BusinessError("Falha ao gerar F1_NUM único para SF1_AP")
0183:
0184:
0185: def listar_ar_tx(cur, filial: str, status: str):
0186:     cur.execute(
0187:         """
0188:         SELECT *
0189:         FROM dbo.SE1_AR
0190:         WHERE E1_FILIAL = ? AND E1_STATUS = ?
```

```
0191:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0192:         ORDER BY ID DESC
0193:         """,
0194:         (filial, status),
0195:     )
0196:     return fetchall_dict(cur)
0197:
0198:
0199: def listar_ap_tx(cur, filial: str, status: str):
0200:     cur.execute(
0201:         """
0202:         SELECT *
0203:         FROM dbo.SF1_AP
0204:         WHERE F1_FILIAL = ? AND F1_STATUS = ?
0205:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0206:         ORDER BY ID DESC
0207:         """,
0208:         (filial, status),
0209:     )
0210:     return fetchall_dict(cur)
0211:
0212:
0213: def _get_ar_for_update(cur, titulo_id: int):
0214:     cur.execute(
0215:         """
0216:         SELECT *
0217:         FROM dbo.SE1_AR WITH (UPDLOCK, ROWLOCK)
0218:         WHERE ID = ?
0219:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0220:         """,
0221:         (int(titulo_id),),
0222:     )
0223:     return fetchone_dict(cur)
0224:
0225:
0226: def _get_ap_for_update(cur, titulo_id: int):
0227:     cur.execute(
0228:         """
0229:         SELECT *
0230:         FROM dbo.SF1_AP WITH (UPDLOCK, ROWLOCK)
0231:         WHERE ID = ?
0232:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0233:         """,
0234:         (int(titulo_id),),
0235:     )
0236:     return fetchone_dict(cur)
0237:
0238:
0239: def criar_mov_caixa_tx(
0240:     cur,
0241:     filial: str,
0242:     conta_id: int,
0243:     tipo: str,
0244:     valor: float,
```

```
0245:     origem_tipo: str,
0246:     origem_id: int | None,
0247:     usuario: str | None,
0248:     hist: str | None = None,
0249: ) -> None:
0250:     cur.execute(
0251:         """
0252:         INSERT INTO dbo.SE5_MOV
0253:           (E5_FILIAL, E5_CONTA_ID, E5_TIPO, E5_VALOR, E5_HIST, E5_ORIGEM_TIPO, E5_ORIGEM_ID, E
5_USUARIO)
0254:         VALUES (?, ?, ?, ?, ?, ?, ?, ?)
0255:         """,
0256:         (filial, int(conta_id), tipo, float(valor), hist, origem_tipo, origem_id, usuario),
0257:     )
0258:
0259:
0260: def baixar_ar_atomico_tx(cur, titulo_id: int) -> None:
0261:     cur.execute(
0262:         """
0263:         UPDATE dbo.SE1_AR
0264:         SET E1_STATUS='BAIXADO'
0265:         WHERE ID=? AND E1_STATUS='ABERTO'
0266:         """,
0267:         (int(titulo_id),),
0268:     )
0269:     if cur.rowcount == 0:
0270:         cur.execute(
0271:             "SELECT E1_STATUS FROM dbo.SE1_AR WHERE ID=? AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_
 <> '*')",
0272:             (int(titulo_id),),
0273:         )
0274:         titulo = fetchone_dict(cur)
0275:         if not titulo:
0276:             raise BusinessError("Título não encontrado")
0277:         raise BusinessError("Título não está em aberto")
0278:
0279:
0280: def baixar_ap_atomico_tx(cur, titulo_id: int) -> None:
0281:     cur.execute(
0282:         """
0283:         UPDATE dbo.SF1_AP
0284:         SET F1_STATUS='BAIXADO'
0285:         WHERE ID=? AND F1_STATUS='ABERTO'
0286:         """,
0287:         (int(titulo_id),),
0288:     )
0289:     if cur.rowcount == 0:
0290:         cur.execute(
0291:             "SELECT F1_STATUS FROM dbo.SF1_AP WHERE ID=? AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_
 <> '*')",
0292:             (int(titulo_id),),
0293:         )
0294:         titulo = fetchone_dict(cur)
0295:         if not titulo:
```

```
0296:             raise BusinessError("Título não encontrado")
0297:         raise BusinessError("Título não está em aberto")
0298:
0299:
0300: def se5_mov_tx(
0301:     cur,
0302:     filial: str,
0303:     conta_id: int,
0304:     tipo: str,
0305:     valor: float,
0306:     hist: str,
0307:     origem_tipo: str,
0308:     origem_id: int | None,
0309:     usuario: str,
0310:     categ_id: int | None = None,
0311: ) -> int:
0312:     cur.execute(
0313:         """
0314:         INSERT INTO dbo.SE5_MOV
0315:           (E5_DATA, E5_FILIAL, E5_CONTA_ID, E5_TIPO, E5_VALOR,
0316:            E5_HIST, E5_ORIGEM_TIPO, E5_ORIGEM_ID, E5_USUARIO, E5_CATEG_ID)
0317:         VALUES (SYSDATETIME(), ?, ?, ?, ?, ?, ?, ?, ?, ?);
0318:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0319:         """,
0320:         (
0321:             filial,
0322:             int(conta_id),
0323:             tipo,
0324:             float(valor),
0325:             hist,
0326:             origem_tipo,
0327:             int(origem_id) if origem_id is not None else None,
0328:             usuario,
0329:             categ_id,
0330:         ),
0331:     )
0332:
0333:     row = fetchone_dict(cur)
0334:     return int(row["id"])
0335:
0336:
0337: def receber_ar_e_gerar_caixa_tx(
0338:     cur,
0339:     titulo_id: int,
0340:     conta_id: int,
0341:     usuario: str,
0342:     categ_id: int | None = None,
0343: ) -> int:
0344:     cur.execute(
0345:         """
0346:         SELECT ID, E1_VALOR, E1_STATUS, E1_FILIAL
0347:         FROM dbo.SE1_AR
0348:         WHERE ID = ?
0349:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
```

```
0350:         """,
0351:         (int(titulo_id),),
0352:     )
0353:     titulo = fetchone_dict(cur)
0354:     if not titulo:
0355:         raise BusinessError("Título AR não encontrado")
0356:     if titulo["E1_STATUS"] != "ABERTO":
0357:         raise BusinessError("Título AR não está em aberto")
0358:
0359:     mov_id = se5_mov_tx(
0360:         cur,
0361:         filial=titulo["E1_FILIAL"],
0362:         conta_id=int(conta_id),
0363:         tipo="E",
0364:         valor=float(titulo["E1_VALOR"]),
0365:         hist=f"Recebimento AR {titulo_id}",
0366:         origem_tipo="SE1",
0367:         origem_id=titulo_id,
0368:         usuario=usuario,
0369:         categ_id=categ_id,
0370:     )
0371:     cur.execute(
0372:         """
0373:         SELECT 1
0374:         FROM dbo.SE5_CONTAS
0375:         WHERE ID=? AND E5_FILIAL=? AND E5_ATIVA=1
0376:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0377:         """,
0378:         (int(conta_id), titulo["E1_FILIAL"]),
0379:     )
0380:     if cur.fetchone() is None:
0381:         raise BusinessError("Conta de caixa inválida/inativa para esta filial")
0382:
0383:     cur.execute(
0384:         """
0385:         UPDATE dbo.SE1_AR
0386:         SET E1_STATUS='BAIXADO',
0387:             E1_SE5_ID=?
0388:         WHERE ID=? AND E1_STATUS='ABERTO'
0389:         """,
0390:         (mov_id, int(titulo_id)),
0391:     )
0392:     if cur.rowcount == 0:
0393:         raise BusinessError("Falha ao baixar AR")
0394:
0395:     return mov_id
0396:
0397:
0398: def pagar_ap_e_gerar_caixa_tx(
0399:     cur,
0400:     titulo_id: int,
0401:     conta_id: int,
0402:     usuario: str,
0403:     categ_id: int | None = None,
```

```
0404: ) -> int:
0405:     cur.execute(
0406:         """
0407:         SELECT ID, F1_VALOR, F1_STATUS, F1_FILIAL
0408:         FROM dbo.SF1_AP
0409:         WHERE ID = ?
0410:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0411:         """,
0412:         (int(titulo_id),),
0413:     )
0414:     titulo = fetchone_dict(cur)
0415:     if not titulo:
0416:         raise BusinessError("Título AP não encontrado")
0417:     if titulo["F1_STATUS"] != "ABERTO":
0418:         raise BusinessError("Título AP não está em aberto")
0419:
0420:     mov_id = se5_mov_tx(
0421:         cur,
0422:         filial=titulo["F1_FILIAL"],
0423:         conta_id=int(conta_id),
0424:         tipo="S",
0425:         valor=float(titulo["F1_VALOR"]),
0426:         hist=f"Pagamento AP {titulo_id}",
0427:         origem_tipo="SF1",
0428:         origem_id=titulo_id,
0429:         usuario=usuario,
0430:         categ_id=categ_id,
0431:     )
0432:     cur.execute(
0433:         """
0434:         SELECT 1
0435:         FROM dbo.SE5_CONTAS
0436:         WHERE ID=? AND E5_FILIAL=? AND E5_ATIVA=1
0437:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0438:         """,
0439:         (int(conta_id), titulo["F1_FILIAL"]),
0440:     )
0441:     if cur.fetchone() is None:
0442:         raise BusinessError("Conta de caixa inválida/inativa para esta filial")
0443:
0444:     cur.execute(
0445:         """
0446:         UPDATE dbo.SF1_AP
0447:         SET F1_STATUS='BAIXADO',
0448:             F1_SE5_ID=?
0449:         WHERE ID=? AND F1_STATUS='ABERTO'
0450:         """,
0451:         (mov_id, int(titulo_id)),
0452:     )
0453:     if cur.rowcount == 0:
0454:         raise BusinessError("Falha ao baixar AP")
0455:
0456:     return mov_id
0457:
```

```
0458:
0459: def listar_contas_caixa_tx(cur, filial: str):
0460:     cur.execute(
0461:         """
0462:         SELECT ID, E5_FILIAL, E5_NOME, E5_TIPO, E5_ATIVA
0463:         FROM dbo.SE5_CONTAS
0464:         WHERE E5_FILIAL = ?
0465:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0466:         ORDER BY E5_ATIVA DESC, E5_NOME ASC
0467:         """,
0468:         (filial,),
0469:     )
0470:     return fetchall_dict(cur)
0471:
0472:
0473: def extrato_caixa_tx(cur, filial: str, conta_id: int, de: date | None, ate: date | None):
0474:     filtros = ["m.E5_FILIAL = ?", "m.E5_CONTA_ID = ?"]
0475:     params = [filial, int(conta_id)]
0476:
0477:     if de is not None:
0478:         filtros.append("m.E5_DATA >= ?")
0479:         params.append(de)
0480:
0481:     if ate is not None:
0482:         filtros.append("m.E5_DATA < DATEADD(DAY, 1, ?)")
0483:         params.append(ate)
0484:
0485:     where_sql = " AND ".join(filtros)
0486:
0487:     cur.execute(
0488:         f"""
0489:         WITH Mov AS (
0490:           SELECT
0491:             m.ID,
0492:             m.E5_DATA,
0493:             m.E5_FILIAL,
0494:             m.E5_CONTA_ID,
0495:             m.E5_TIPO,
0496:             m.E5_VALOR,
0497:             CASE WHEN m.E5_TIPO='E' THEN m.E5_VALOR ELSE -m.E5_VALOR END AS E5_VALOR_ASSINADO,
0498:             m.E5_HIST,
0499:             m.E5_ORIGEM_TIPO,
0500:             m.E5_ORIGEM_ID,
0501:             m.E5_USUARIO,
0502:             m.E5_CATEG_ID,
0503:             SUM(CASE WHEN m.E5_TIPO='E' THEN m.E5_VALOR ELSE -m.E5_VALOR END)
0504:               OVER (ORDER BY m.E5_DATA ASC, m.ID ASC ROWS UNBOUNDED PRECEDING) AS E5_SALDO_ACU
MULADO
0505:           FROM dbo.SE5_MOV m
0506:           WHERE {where_sql}
0507:             AND (m.D_E_L_E_T_ IS NULL OR m.D_E_L_E_T_ <> '*')
0508:         )
0509:         SELECT
0510:           mv.*,
```

```
0511:           c.C5_NOME AS CATEG_NOME,
0512:           c.C5_TIPO AS CATEG_TIPO
0513:         FROM Mov mv
0514:         LEFT JOIN dbo.SE5_CATEG c ON c.ID = mv.E5_CATEG_ID
0515:          AND (c.D_E_L_E_T_ IS NULL OR c.D_E_L_E_T_ <> '*')
0516:         ORDER BY mv.E5_DATA DESC, mv.ID DESC
0517:         """,
0518:         tuple(params),
0519:     )
0520:     return fetchall_dict(cur)
0521:
0522:
0523: def saldo_caixa_simples_tx(cur, filial: str, conta_id: int, ate: date | None):
0524:     filtros = ["E5_FILIAL = ?", "E5_CONTA_ID = ?"]
0525:     params = [filial, int(conta_id)]
0526:
0527:     if ate is not None:
0528:         filtros.append("E5_DATA < DATEADD(DAY, 1, ?)")
0529:         params.append(ate)
0530:
0531:     where_sql = " AND ".join(filtros)
0532:
0533:     cur.execute(
0534:         f"""
0535:         SELECT
0536:           COALESCE(SUM(CASE WHEN E5_TIPO='E' THEN E5_VALOR ELSE 0 END), 0) AS entradas,
0537:           COALESCE(SUM(CASE WHEN E5_TIPO='S' THEN E5_VALOR ELSE 0 END), 0) AS saidas,
0538:           COALESCE(SUM(CASE WHEN E5_TIPO='E' THEN E5_VALOR ELSE -E5_VALOR END), 0) AS saldo
0539:         FROM dbo.SE5_MOV
0540:         WHERE {where_sql}
0541:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0542:         """,
0543:         tuple(params),
0544:     )
0545:     return fetchone_dict(cur) or {"entradas": 0, "saidas": 0, "saldo": 0}
0546:
0547:
0548: def listar_categorias_tx(cur, filial: str, ativas: bool = True):
0549:     if ativas:
0550:         cur.execute(
0551:             """
0552:             SELECT ID, C5_FILIAL, C5_NOME, C5_TIPO, C5_ATIVA
0553:             FROM dbo.SE5_CATEG
0554:             WHERE C5_FILIAL=? AND C5_ATIVA=1
0555:               AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0556:             ORDER BY C5_TIPO, C5_NOME
0557:             """,
0558:             (filial,),
0559:         )
0560:     else:
0561:         cur.execute(
0562:             """
0563:             SELECT ID, C5_FILIAL, C5_NOME, C5_TIPO, C5_ATIVA
0564:             FROM dbo.SE5_CATEG
```

```
0565:             WHERE C5_FILIAL=?
0566:               AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0567:             ORDER BY C5_ATIVA DESC, C5_TIPO, C5_NOME
0568:             """,
0569:             (filial,),
0570:         )
0571:     return fetchall_dict(cur)
0572:
0573:
0574: def criar_categoria_tx(cur, filial: str, nome: str, tipo: str) -> int:
0575:     cur.execute(
0576:         """
0577:         INSERT INTO dbo.SE5_CATEG (C5_FILIAL, C5_NOME, C5_TIPO, C5_ATIVA)
0578:         VALUES (?, ?, ?, 1);
0579:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0580:         """,
0581:         (filial, nome, tipo),
0582:     )
0583:     row = fetchone_dict(cur)
0584:     return int(row["id"])
0585:
0586:
0587: def definir_categoria_mov_tx(cur, mov_id: int, categ_id: int | None):
0588:     cur.execute(
0589:         """
0590:         UPDATE dbo.SE5_MOV
0591:         SET E5_CATEG_ID = ?
0592:         WHERE ID = ?
0593:         """,
0594:         (categ_id, int(mov_id)),
0595:     )
0596:     if cur.rowcount == 0:
0597:         raise BusinessError("Movimento não encontrado")
0598:
0599:
0600: def obter_movimento_tx(cur, mov_id: int):
0601:     cur.execute(
0602:         """
0603:         SELECT
0604:           m.ID,
0605:           m.E5_FILIAL,
0606:           m.E5_DATA,
0607:           m.E5_TIPO,
0608:           m.E5_VALOR,
0609:           m.E5_HIST,
0610:           m.E5_ORIGEM_TIPO,
0611:           m.E5_ORIGEM_ID,
0612:           m.E5_USUARIO,
0613:           m.E5_CATEG_ID,
0614:           c.C5_NOME AS CATEG_NOME,
0615:           c.C5_TIPO AS CATEG_TIPO
0616:         FROM dbo.SE5_MOV m
0617:         LEFT JOIN dbo.SE5_CATEG c
0618:           ON c.ID = m.E5_CATEG_ID
```

```
0619:          AND (c.D_E_L_E_T_ IS NULL OR c.D_E_L_E_T_ <> '*')
0620:         WHERE m.ID = ?
0621:           AND (m.D_E_L_E_T_ IS NULL OR m.D_E_L_E_T_ <> '*')
0622:         """,
0623:         (int(mov_id),),
0624:     )
0625:     return fetchone_dict(cur)
0626:
0627:
0628: def dre_simples_tx(cur, filial: str, de: date | None, ate: date | None):
0629:     filtros = ["m.E5_FILIAL = ?"]
0630:     params = [filial]
0631:
0632:     if de is not None:
0633:         filtros.append("m.E5_DATA >= ?")
0634:         params.append(de)
0635:     if ate is not None:
0636:         filtros.append("m.E5_DATA < DATEADD(DAY, 1, ?)")
0637:         params.append(ate)
0638:
0639:     where_sql = " AND ".join(filtros)
0640:
0641:     cur.execute(
0642:         f"""
0643:         SELECT
0644:           c.C5_TIPO AS grupo,
0645:           COALESCE(SUM(CASE WHEN m.E5_TIPO='E' THEN m.E5_VALOR ELSE -m.E5_VALOR END), 0) AS va
lor
0646:         FROM dbo.SE5_MOV m
0647:         LEFT JOIN dbo.SE5_CATEG c ON c.ID = m.E5_CATEG_ID
0648:          AND (c.D_E_L_E_T_ IS NULL OR c.D_E_L_E_T_ <> '*')
0649:         WHERE {where_sql}
0650:           AND (m.D_E_L_E_T_ IS NULL OR m.D_E_L_E_T_ <> '*')
0651:         GROUP BY c.C5_TIPO
0652:         ORDER BY c.C5_TIPO
0653:         """,
0654:         tuple(params),
0655:     )
0656:     rows = fetchall_dict(cur)
0657:
0658:     cur.execute(
0659:         f"""
0660:         SELECT
0661:           COALESCE(SUM(CASE WHEN m.E5_TIPO='E' THEN m.E5_VALOR ELSE -m.E5_VALOR END), 0) AS to
tal
0662:         FROM dbo.SE5_MOV m
0663:         WHERE {where_sql}
0664:           AND (m.D_E_L_E_T_ IS NULL OR m.D_E_L_E_T_ <> '*')
0665:         """,
0666:         tuple(params),
0667:     )
0668:     total = fetchone_dict(cur) or {"total": 0}
0669:
0670:     return {"filial": filial, "de": de, "ate": ate, "linhas": rows, "total": total["total"]}
```

```
0671:
0672:
0673: def cancelar_ar_por_ref_tx(cur, filial: str, pedido_id: int) -> int:
0674:     """Cancela títulos AR abertos vinculados às NFs do pedido."""
0675:
0676:     cur.execute(
0677:         """
0678:         SELECT ID
0679:         FROM dbo.SF2_NOTAS
0680:         WHERE F2_FILIAL=? AND F2_PEDIDO_ID=?
0681:         """,
0682:         (filial, int(pedido_id)),
0683:     )
0684:     nf_ids = [str(row[0]) for row in cur.fetchall()]
0685:     if not nf_ids:
0686:         return 0
0687:
0688:     cur.execute(
0689:         f"""
0690:         SELECT TOP 1 ID
0691:         FROM dbo.SE1_AR
0692:         WHERE E1_FILIAL=? AND E1_REF IN ({",".join(["?"] * len(nf_ids))})
0693:           AND E1_STATUS='BAIXADO'
0694:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0695:         """,
0696:         tuple([filial] + nf_ids),
0697:     )
0698:     if cur.fetchone() is not None:
0699:         raise BusinessError("Não é possível cancelar: AR já está baixado (recebido)")
0700:
0701:     cur.execute(
0702:         f"""
0703:         UPDATE dbo.SE1_AR
0704:         SET E1_STATUS='CANCELADO',
0705:             D_E_L_E_T_='*',
0706:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0707:         WHERE E1_FILIAL=? AND E1_REF IN ({",".join(["?"] * len(nf_ids))})
0708:           AND E1_STATUS='ABERTO'
0709:         """,
0710:         tuple([filial] + nf_ids),
0711:     )
0712:     return int(cur.rowcount or 0)
0713:
0714:
0715: def cancelar_ar_por_nf_tx(cur, filial: str, nf_id: int) -> int:
0716:     """Cancela títulos AR abertos vinculados a uma NF específica."""
0717:
0718:     cur.execute(
0719:         """
0720:         SELECT TOP 1 ID
0721:         FROM dbo.SE1_AR
0722:         WHERE E1_FILIAL=? AND E1_REF=? AND E1_STATUS='BAIXADO'
0723:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0724:         """,
```

```
0725:         (filial, str(nf_id)),
0726:     )
0727:     if cur.fetchone() is not None:
0728:         raise BusinessError("Não é possível cancelar: AR já está baixado (recebido)")
0729:
0730:     cur.execute(
0731:         """
0732:         UPDATE dbo.SE1_AR
0733:         SET E1_STATUS='CANCELADO',
0734:             D_E_L_E_T_='*',
0735:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0736:         WHERE E1_FILIAL=? AND E1_REF=? AND E1_STATUS='ABERTO'
0737:         """,
0738:         (filial, str(nf_id)),
0739:     )
0740:     return int(cur.rowcount or 0)
0741:
0742:
0743: def ar_listar_por_pedido_tx(cur, pedido_id: int) -> list[dict]:
0744:     cur.execute(
0745:         """
0746:         SELECT e1.*
0747:         FROM dbo.SE1_AR e1
0748:         INNER JOIN dbo.SF2_NOTAS f2 ON f2.ID = TRY_CAST(e1.E1_REF AS INT)
0749:         WHERE f2.F2_PEDIDO_ID = ?
0750:           AND (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0751:         ORDER BY e1.ID DESC
0752:         """,
0753:         (int(pedido_id),),
0754:     )
0755:     return fetchall_dict(cur)
0756:
0757:
0758: def ar_ultimo_por_pedido_tx(cur, pedido_id: int) -> dict | None:
0759:     cur.execute(
0760:         """
0761:         SELECT TOP 1 e1.*
0762:         FROM dbo.SE1_AR e1
0763:         INNER JOIN dbo.SF2_NOTAS f2 ON f2.ID = TRY_CAST(e1.E1_REF AS INT)
0764:         WHERE f2.F2_PEDIDO_ID = ?
0765:           AND (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0766:         ORDER BY e1.ID DESC
0767:         """,
0768:         (int(pedido_id),),
0769:     )
0770:     return fetchone_dict(cur)
0771:
0772:
0773: def ar_cancelar_atomico_tx(cur, titulo_id: int) -> None:
0774:     cur.execute(
0775:         """
0776:         UPDATE dbo.SE1_AR
0777:         SET E1_STATUS='CANCELADO',
0778:             D_E_L_E_T_='*',
```

```
0779:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0780:         WHERE ID=? AND E1_STATUS='ABERTO'
0781:         """,
0782:         (int(titulo_id),),
0783:     )
0784:     if cur.rowcount == 0:
0785:         cur.execute(
0786:             "SELECT E1_STATUS FROM dbo.SE1_AR WHERE ID=? AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_
 <> '*')",
0787:             (int(titulo_id),),
0788:         )
0789:         titulo = fetchone_dict(cur)
0790:         if not titulo:
0791:             raise BusinessError("Título AR não encontrado")
0792:         raise BusinessError("Título AR não está em aberto")
```

==========
ARQUIVO: app/infra/repositories/fiscal_ncm_repo.py
==========

```
0001: from app.infra.db import fetchone_dict, fetchall_dict
0002: from app.core.exceptions import BusinessError
0003: from app.infra.repositories.sx5_repo import sx5_obter_tx
0004:
0005:
0006: def ncm_obter_tx(cur, filial: str, ncm: str) -> dict | None:
0007:     cur.execute(
0008:         """
0009:         SELECT TOP 1 *
0010:         FROM dbo.SF8_FISCAL_NCM
0011:         WHERE F8_FILIAL=? AND F8_NCM=?
0012:           AND F8_ATIVO=1
0013:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0014:         """,
0015:         (filial, str(ncm).strip()),
0016:     )
0017:     return fetchone_dict(cur)
0018:
0019:
0020: def ncm_listar_tx(cur, filial: str, ativos: bool = True):
0021:     sql = """
0022:         SELECT *
0023:         FROM dbo.SF8_FISCAL_NCM
0024:         WHERE F8_FILIAL=?
0025:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0026:     """
0027:     params: list = [filial]
0028:     if ativos:
0029:         sql += " AND F8_ATIVO=1"
0030:     sql += " ORDER BY F8_NCM"
0031:     cur.execute(sql, tuple(params))
0032:     return fetchall_dict(cur)
0033:
0034:
0035: def ncm_upsert_tx(
```

```
0036:     cur,
0037:     filial: str,
0038:     ncm: str,
0039:     cfop: str,
0040:     icms: float | None,
0041:     ipi: float | None,
0042:     ativo: bool = True,
0043: ):
0044:     ncm = (ncm or "").strip()
0045:     cfop = (cfop or "").strip()
0046:     if not ncm:
0047:         raise BusinessError("NCM é obrigatório")
0048:     if not cfop:
0049:         raise BusinessError("CFOP é obrigatório")
0050:     if sx5_obter_tx(cur, filial=filial, tabela="CFOP", chave=cfop) is None:
0051:         raise BusinessError("CFOP não cadastrado no SX5 (tabela CFOP)")
0052:
0053:     cur.execute(
0054:         """
0055:         SELECT ID FROM dbo.SF8_FISCAL_NCM
0056:         WHERE F8_FILIAL=? AND F8_NCM=?
0057:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0058:         """,
0059:         (filial, ncm),
0060:     )
0061:     row = cur.fetchone()
0062:     if row:
0063:         cur.execute(
0064:             """
0065:             UPDATE dbo.SF8_FISCAL_NCM
0066:             SET F8_CFOP=?, F8_ICMS=?, F8_IPI=?, F8_ATIVO=?
0067:             WHERE F8_FILIAL=? AND F8_NCM=?
0068:             """,
0069:             (cfop, icms, ipi, 1 if ativo else 0, filial, ncm),
0070:         )
0071:         return
0072:
0073:     cur.execute(
0074:         """
0075:         INSERT INTO dbo.SF8_FISCAL_NCM
0076:           (F8_FILIAL, F8_NCM, F8_CFOP, F8_ICMS, F8_IPI, F8_ATIVO)
0077:         VALUES (?, ?, ?, ?, ?, ?)
0078:         """,
0079:         (filial, ncm, cfop, icms, ipi, 1 if ativo else 0),
0080:     )
```

==========
ARQUIVO: app/infra/repositories/fiscal_repo.py
==========

```
0001: from app.infra.db import fetchone_dict, fetchall_dict
0002: from app.domain.errors import BusinessError
0003: from app.infra.repositories.sx5_repo import sx5_obter_tx, sx5_criar_tx
0004: from app.infra.repositories.fiscal_ncm_repo import ncm_obter_tx
0005:
```

```
0006:
0007: def fiscal_get_tes_tx(cur, filial: str, tes_cod: str) -> dict:
0008:     cur.execute(
0009:         """
0010:         SELECT TOP 1
0011:           F4_COD,
0012:           F4_TIPO,
0013:           F4_CFOP,
0014:           F4_GERA_TIT,
0015:           F4_GERA_EST,
0016:           F4_ICMS,
0017:           F4_IPI,
0018:           F4_PIS,
0019:           F4_COFINS,
0020:           F4_ICMS_ST,
0021:           F4_DIFAL,
0022:           F4_CST_ICMS,
0023:           F4_CSOSN,
0024:           F4_CST_PIS,
0025:           F4_CST_COFINS
0026:         FROM dbo.SF4_FISCAL
0027:         WHERE F4_FILIAL=? AND F4_COD=?
0028:         """,
0029:         (filial, str(tes_cod).strip()),
0030:     )
0031:     row = fetchone_dict(cur)
0032:     if not row:
0033:         raise BusinessError("TES não encontrado para a filial")
0034:
0035:     # valida TES no dicionário SX5 (tabela TES)
0036:     if sx5_obter_tx(cur, filial=filial, tabela="TES", chave=str(tes_cod).strip()) is None:
0037:         raise BusinessError("TES não cadastrado no SX5 (tabela TES)")
0038:
0039:     # valida CFOP no dicionário SX5 (tabela CFOP)
0040:     cfop = (row.get("F4_CFOP") or "").strip()
0041:     if cfop and sx5_obter_tx(cur, filial=filial, tabela="CFOP", chave=cfop) is None:
0042:         raise BusinessError("CFOP não cadastrado no SX5 (tabela CFOP)")
0043:     return row
0044:
0045:
0046: def fiscal_get_tx(cur, filial: str, tes_cod: str | None = None) -> dict:
0047:     if tes_cod:
0048:         return fiscal_get_tes_tx(cur, filial, tes_cod)
0049:
0050:     cur.execute(
0051:         """
0052:         SELECT TOP 1
0053:           F4_COD,
0054:           F4_TIPO,
0055:           F4_CFOP,
0056:           F4_GERA_TIT,
0057:           F4_GERA_EST,
0058:           F4_ICMS,
0059:           F4_IPI,
```

```
0060:           F4_PIS,
0061:           F4_COFINS,
0062:           F4_ICMS_ST,
0063:           F4_DIFAL,
0064:           F4_CST_ICMS,
0065:           F4_CSOSN,
0066:           F4_CST_PIS,
0067:           F4_CST_COFINS
0068:         FROM dbo.SF4_FISCAL
0069:         WHERE F4_FILIAL=?
0070:         ORDER BY F4_COD
0071:         """,
0072:         (filial,),
0073:     )
0074:     row = fetchone_dict(cur)
0075:     if not row:
0076:         raise BusinessError("Parâmetros fiscais não cadastrados para a filial")
0077:     return row
0078:
0079:
0080: def fiscal_upsert_tx(
0081:     cur,
0082:     filial: str,
0083:     tes_cod: str,
0084:     cfop: str,
0085:     icms: float,
0086:     ipi: float,
0087:     pis: float = 0.0,
0088:     cofins: float = 0.0,
0089:     icms_st: float = 0.0,
0090:     difal: float = 0.0,
0091:     cst_icms: str | None = None,
0092:     csosn: str | None = None,
0093:     cst_pis: str | None = None,
0094:     cst_cofins: str | None = None,
0095:     tipo: str = "S",
0096:     gera_tit: bool = True,
0097:     gera_est: bool = True,
0098:     descr: str | None = None,
0099: ) -> None:
0100:     tes_cod = (tes_cod or "").strip()
0101:     cfop = (cfop or "").strip()
0102:     tipo = (tipo or "S").strip().upper()
0103:
0104:     if not tes_cod:
0105:         raise BusinessError("TES é obrigatório")
0106:     if not cfop:
0107:         raise BusinessError("CFOP é obrigatório")
0108:     if tipo not in {"S", "E"}:
0109:         raise BusinessError("Tipo TES inválido (S/E)")
0110:
0111:     # garante SX5 (TES/CFOP)
0112:     if sx5_obter_tx(cur, filial=filial, tabela="TES", chave=tes_cod) is None:
```

```
0113:         sx5_criar_tx(cur, filial=filial, tabela="TES", chave=tes_cod, descr=(descr or f"TES {t
es_cod}"), ativo=True)
0114:     if sx5_obter_tx(cur, filial=filial, tabela="CFOP", chave=cfop) is None:
0115:         sx5_criar_tx(cur, filial=filial, tabela="CFOP", chave=cfop, descr=(f"CFOP {cfop}"), at
ivo=True)
0116:
0117:     # upsert SF4
0118:     cur.execute(
0119:         """
0120:         SELECT ID
0121:         FROM dbo.SF4_FISCAL
0122:         WHERE F4_FILIAL=? AND F4_COD=?
0123:         """,
0124:         (filial, tes_cod),
0125:     )
0126:     row = cur.fetchone()
0127:     if row:
0128:         cur.execute(
0129:             """
0130:             UPDATE dbo.SF4_FISCAL
0131:             SET F4_CFOP=?, F4_TIPO=?, F4_GERA_TIT=?, F4_GERA_EST=?, F4_ICMS=?, F4_IPI=?,
0132:                 F4_PIS=?, F4_COFINS=?, F4_ICMS_ST=?, F4_DIFAL=?,
0133:                 F4_CST_ICMS=?, F4_CSOSN=?, F4_CST_PIS=?, F4_CST_COFINS=?,
0134:                 F4_DESC=?
0135:             WHERE F4_FILIAL=? AND F4_COD=?
0136:             """,
0137:             (
0138:                 cfop,
0139:                 tipo,
0140:                 1 if gera_tit else 0,
0141:                 1 if gera_est else 0,
0142:                 float(icms),
0143:                 float(ipi),
0144:                 float(pis),
0145:                 float(cofins),
0146:                 float(icms_st),
0147:                 float(difal),
0148:                 cst_icms,
0149:                 csosn,
0150:                 cst_pis,
0151:                 cst_cofins,
0152:                 descr,
0153:                 filial,
0154:                 tes_cod,
0155:             ),
0156:         )
0157:         return
0158:
0159:     cur.execute(
0160:         """
0161:         INSERT INTO dbo.SF4_FISCAL
0162:           (F4_FILIAL, F4_COD, F4_TIPO, F4_CFOP, F4_GERA_TIT, F4_GERA_EST, F4_ICMS, F4_IPI,
0163:            F4_PIS, F4_COFINS, F4_ICMS_ST, F4_DIFAL, F4_CST_ICMS, F4_CSOSN, F4_CST_PIS, F4_CST_
COFINS, F4_DESC)
```

```
0164:         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
0165:         """,
0166:         (
0167:             filial,
0168:             tes_cod,
0169:             tipo,
0170:             cfop,
0171:             1 if gera_tit else 0,
0172:             1 if gera_est else 0,
0173:             float(icms),
0174:             float(ipi),
0175:             float(pis),
0176:             float(cofins),
0177:             float(icms_st),
0178:             float(difal),
0179:             cst_icms,
0180:             csosn,
0181:             cst_pis,
0182:             cst_cofins,
0183:             descr,
0184:         ),
0185:     )
0186:
0187:
0188: def fiscal_listar_tx(cur, filial: str, ativo: bool = True):
0189:     sql = """
0190:         SELECT *
0191:         FROM dbo.SF4_FISCAL
0192:         WHERE F4_FILIAL=?
0193:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0194:     """
0195:     params: list = [filial]
0196:     if ativo:
0197:         sql += " AND F4_ATIVO=1"
0198:     sql += " ORDER BY F4_COD"
0199:     cur.execute(sql, tuple(params))
0200:     return fetchall_dict(cur)
0201:
0202:
0203: def fiscal_inativar_tx(cur, filial: str, tes_cod: str, usuario: str | None = None):
0204:     cur.execute(
0205:         """
0206:         UPDATE dbo.SF4_FISCAL
0207:         SET F4_ATIVO=0,
0208:             D_E_L_E_T_='*',
0209:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0210:         WHERE F4_FILIAL=? AND F4_COD=?
0211:         """,
0212:         (filial, tes_cod),
0213:     )
0214:     if cur.rowcount == 0:
0215:         raise BusinessError("TES não encontrado para inativação")
0216:
0217:
```

```
0218: def fiscal_regra_match_tx(
0219:     cur,
0220:     filial: str,
0221:     tes_cod: str,
0222:     cliente_cod: str | None,
0223:     produto: str | None,
0224: ) -> dict | None:
0225:     cur.execute(
0226:         """
0227:         SELECT TOP 1 *
0228:         FROM dbo.SF5_FISCAL_REGRAS
0229:         WHERE F5_FILIAL=? AND F5_TES=? AND F5_ATIVO=1
0230:           AND (F5_CLIENTE_COD IS NULL OR F5_CLIENTE_COD=?)
0231:           AND (F5_PRODUTO IS NULL OR F5_PRODUTO=?)
0232:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0233:         ORDER BY
0234:           (CASE WHEN F5_CLIENTE_COD IS NULL THEN 0 ELSE 1 END +
0235:            CASE WHEN F5_PRODUTO IS NULL THEN 0 ELSE 1 END) DESC,
0236:           F5_PRIORIDADE DESC,
0237:           ID DESC
0238:         """,
0239:         (filial, str(tes_cod).strip(), cliente_cod, produto),
0240:     )
0241:     return fetchone_dict(cur)
0242:
0243:
0244: def fiscal_resolver_item_tx(
0245:     cur,
0246:     filial: str,
0247:     tes_cod: str,
0248:     cliente_cod: str | None,
0249:     produto: str | None,
0250: ) -> dict:
0251:     base = fiscal_get_tes_tx(cur, filial=filial, tes_cod=tes_cod)
0252:     ncm_row = None
0253:     if produto:
0254:         cur.execute(
0255:             """
0256:             SELECT B1_NCM
0257:             FROM dbo.SB1_PRODUTOS
0258:             WHERE B1_COD=? AND B1_FILIAL=?
0259:               AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0260:             """,
0261:             (produto, filial),
0262:         )
0263:         r = fetchone_dict(cur)
0264:         if r:
0265:             ncm = (r.get("B1_NCM") or "").strip()
0266:             if ncm:
0267:                 ncm_row = ncm_obter_tx(cur, filial=filial, ncm=ncm)
0268:     regra = fiscal_regra_match_tx(
0269:         cur,
0270:         filial=filial,
0271:         tes_cod=tes_cod,
```

```
0272:         cliente_cod=cliente_cod,
0273:         produto=produto,
0274:     )
0275:
0276:     cfop = (
0277:         (regra.get("F5_CFOP") if regra and regra.get("F5_CFOP") is not None else None)
0278:         or (ncm_row.get("F8_CFOP") if ncm_row and ncm_row.get("F8_CFOP") is not None else None
)
0279:         or base.get("F4_CFOP")
0280:         or ""
0281:     )
0282:     icms = float(regra["F5_ICMS"]) if regra and regra.get("F5_ICMS") is not None else (
0283:         float(ncm_row["F8_ICMS"]) if ncm_row and ncm_row.get("F8_ICMS") is not None else float
(base["F4_ICMS"])
0284:     )
0285:     ipi = float(regra["F5_IPI"]) if regra and regra.get("F5_IPI") is not None else (
0286:         float(ncm_row["F8_IPI"]) if ncm_row and ncm_row.get("F8_IPI") is not None else float(b
ase["F4_IPI"])
0287:     )
0288:     pis = float(regra["F5_PIS"]) if regra and regra.get("F5_PIS") is not None else float(base.
get("F4_PIS") or 0)
0289:     cofins = float(regra["F5_COFINS"]) if regra and regra.get("F5_COFINS") is not None else fl
oat(base.get("F4_COFINS") or 0)
0290:     icms_st = float(regra["F5_ICMS_ST"]) if regra and regra.get("F5_ICMS_ST") is not None else
 float(base.get("F4_ICMS_ST") or 0)
0291:     difal = float(regra["F5_DIFAL"]) if regra and regra.get("F5_DIFAL") is not None else float
(base.get("F4_DIFAL") or 0)
0292:     cst_icms = (regra.get("F5_CST_ICMS") if regra and regra.get("F5_CST_ICMS") is not None els
e base.get("F4_CST_ICMS"))
0293:     csosn = (regra.get("F5_CSOSN") if regra and regra.get("F5_CSOSN") is not None else base.ge
t("F4_CSOSN"))
0294:     cst_pis = (regra.get("F5_CST_PIS") if regra and regra.get("F5_CST_PIS") is not None else b
ase.get("F4_CST_PIS"))
0295:     cst_cofins = (regra.get("F5_CST_COFINS") if regra and regra.get("F5_CST_COFINS") is not No
ne else base.get("F4_CST_COFINS"))
0296:     gera_tit = bool(regra["F5_GERA_TIT"]) if regra and regra.get("F5_GERA_TIT") is not None el
se bool(base.get("F4_GERA_TIT"))
0297:     gera_est = bool(regra["F5_GERA_EST"]) if regra and regra.get("F5_GERA_EST") is not None el
se bool(base.get("F4_GERA_EST"))
0298:
0299:     if not str(cfop).strip():
0300:         raise BusinessError("CFOP n??o definido para o item (TES/regra)")
0301:     if sx5_obter_tx(cur, filial=filial, tabela="CFOP", chave=str(cfop).strip()) is None:
0302:         raise BusinessError("CFOP n??o cadastrado no SX5 (tabela CFOP)")
0303:     return {
0304:         "tes_cod": str(base.get("F4_COD") or tes_cod).strip(),
0305:         "tes_tipo": str(base.get("F4_TIPO") or "").strip().upper(),
0306:         "cfop": str(cfop).strip(),
0307:         "icms": float(icms),
0308:         "ipi": float(ipi),
0309:         "pis": float(pis),
0310:         "cofins": float(cofins),
0311:         "icms_st": float(icms_st),
0312:         "difal": float(difal),
```

```
0313:         "cst_icms": (str(cst_icms).strip() if cst_icms is not None else None),
0314:         "csosn": (str(csosn).strip() if csosn is not None else None),
0315:         "cst_pis": (str(cst_pis).strip() if cst_pis is not None else None),
0316:         "cst_cofins": (str(cst_cofins).strip() if cst_cofins is not None else None),
0317:         "gera_tit": bool(gera_tit),
0318:         "gera_est": bool(gera_est),
0319:         "regra_id": int(regra["ID"]) if regra else None,
0320:     }
0321:
0322:
0323: def fiscal_regra_listar_tx(
0324:     cur,
0325:     filial: str,
0326:     tes_cod: str | None = None,
0327:     cliente_cod: str | None = None,
0328:     produto: str | None = None,
0329:     ativos: bool = True,
0330: ):
0331:     sql = """
0332:         SELECT *
0333:         FROM dbo.SF5_FISCAL_REGRAS
0334:         WHERE F5_FILIAL=?
0335:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0336:     """
0337:     params: list = [filial]
0338:     if tes_cod:
0339:         sql += " AND F5_TES=?"
0340:         params.append(str(tes_cod).strip())
0341:     if cliente_cod:
0342:         sql += " AND F5_CLIENTE_COD=?"
0343:         params.append(str(cliente_cod).strip())
0344:     if produto:
0345:         sql += " AND F5_PRODUTO=?"
0346:         params.append(str(produto).strip())
0347:     if ativos:
0348:         sql += " AND F5_ATIVO=1"
0349:     sql += " ORDER BY F5_PRIORIDADE DESC, ID DESC"
0350:     cur.execute(sql, tuple(params))
0351:     return fetchall_dict(cur)
0352:
0353:
0354: def fiscal_regra_upsert_tx(
0355:     cur,
0356:     filial: str,
0357:     tes_cod: str,
0358:     cliente_cod: str | None,
0359:     produto: str | None,
0360:     cfop: str | None,
0361:     icms: float | None,
0362:     ipi: float | None,
0363:     pis: float | None = None,
0364:     cofins: float | None = None,
0365:     icms_st: float | None = None,
0366:     difal: float | None = None,
```

```
0367:     cst_icms: str | None = None,
0368:     csosn: str | None = None,
0369:     cst_pis: str | None = None,
0370:     cst_cofins: str | None = None,
0371:     gera_tit: bool | None = None,
0372:     gera_est: bool | None = None,
0373:     prioridade: int = 0,
0374:     ativo: bool = True,
0375:     regra_id: int | None = None,
0376: ) -> int:
0377:     tes_cod = (tes_cod or "").strip()
0378:     if not tes_cod:
0379:         raise BusinessError("TES é obrigatório")
0380:
0381:     if cfop:
0382:         if sx5_obter_tx(cur, filial=filial, tabela="CFOP", chave=str(cfop).strip()) is None:
0383:             raise BusinessError("CFOP não cadastrado no SX5 (tabela CFOP)")
0384:
0385:     if regra_id:
0386:         cur.execute(
0387:             """
0388:             UPDATE dbo.SF5_FISCAL_REGRAS
0389:             SET F5_TES=?,
0390:                 F5_CLIENTE_COD=?,
0391:                 F5_PRODUTO=?,
0392:                 F5_CFOP=?,
0393:                 F5_ICMS=?,
0394:                 F5_IPI=?,
0395:                 F5_PIS=?,
0396:                 F5_COFINS=?,
0397:                 F5_ICMS_ST=?,
0398:                 F5_DIFAL=?,
0399:                 F5_CST_ICMS=?,
0400:                 F5_CSOSN=?,
0401:                 F5_CST_PIS=?,
0402:                 F5_CST_COFINS=?,
0403:                 F5_GERA_TIT=?,
0404:                 F5_GERA_EST=?,
0405:                 F5_PRIORIDADE=?,
0406:                 F5_ATIVO=?
0407:             WHERE ID=? AND F5_FILIAL=?
0408:             """,
0409:             (
0410:                 tes_cod,
0411:                 cliente_cod,
0412:                 produto,
0413:                 cfop,
0414:                 icms,
0415:                 ipi,
0416:                 pis,
0417:                 cofins,
0418:                 icms_st,
0419:                 difal,
0420:                 cst_icms,
```

```
0421:                 csosn,
0422:                 cst_pis,
0423:                 cst_cofins,
0424:                 1 if gera_tit is True else (0 if gera_tit is False else None),
0425:                 1 if gera_est is True else (0 if gera_est is False else None),
0426:                 int(prioridade),
0427:                 1 if ativo else 0,
0428:                 int(regra_id),
0429:                 filial,
0430:             ),
0431:         )
0432:         if cur.rowcount == 0:
0433:             raise BusinessError("Regra fiscal não encontrada")
0434:         return int(regra_id)
0435:
0436:     cur.execute(
0437:         """
0438:         INSERT INTO dbo.SF5_FISCAL_REGRAS
0439:           (F5_FILIAL, F5_TES, F5_CLIENTE_COD, F5_PRODUTO, F5_CFOP, F5_ICMS, F5_IPI,
0440:            F5_PIS, F5_COFINS, F5_ICMS_ST, F5_DIFAL, F5_CST_ICMS, F5_CSOSN, F5_CST_PIS, F5_CST_
COFINS,
0441:            F5_GERA_TIT, F5_GERA_EST, F5_PRIORIDADE, F5_ATIVO)
0442:         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
0443:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0444:         """,
0445:         (
0446:             filial,
0447:             tes_cod,
0448:             cliente_cod,
0449:             produto,
0450:             cfop,
0451:             icms,
0452:             ipi,
0453:             pis,
0454:             cofins,
0455:             icms_st,
0456:             difal,
0457:             cst_icms,
0458:             csosn,
0459:             cst_pis,
0460:             cst_cofins,
0461:             1 if gera_tit is True else (0 if gera_tit is False else None),
0462:             1 if gera_est is True else (0 if gera_est is False else None),
0463:             int(prioridade),
0464:             1 if ativo else 0,
0465:         ),
0466:     )
0467:     row = fetchone_dict(cur)
0468:     return int(row["id"])
0469:
0470:
0471: def fiscal_regra_inativar_tx(cur, regra_id: int):
0472:     cur.execute(
0473:         """
```

```
0474:         UPDATE dbo.SF5_FISCAL_REGRAS
0475:         SET F5_ATIVO=0,
0476:             D_E_L_E_T_='*',
0477:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0478:         WHERE ID=?
0479:         """,
0480:         (int(regra_id),),
0481:     )
0482:     if cur.rowcount == 0:
0483:         raise BusinessError("Regra fiscal não encontrada")
```

==========
ARQUIVO: app/infra/repositories/logs_repo.py
==========

```
0001: from datetime import datetime
0002: from app.infra.db import fetchall_dict
0003:
0004: def log_tx(cur, usuario: str, acao: str) -> None:
0005:     cur.execute(
0006:         "INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA) VALUES (?, ?, ?)",
0007:         (usuario, acao, datetime.now()),
0008:     )
0009:
0010: def listar_logs_tx(cur, usuario: str, perfil: str):
0011:     if perfil in {"admin", "auditor"}:
0012:         cur.execute("""
0013:             SELECT *
0014:             FROM dbo.LOGS
0015:             ORDER BY ID DESC
0016:             OFFSET 0 ROWS FETCH NEXT 200 ROWS ONLY
0017:         """)
0018:     else:
0019:         cur.execute("""
0020:             SELECT *
0021:             FROM dbo.LOGS
0022:             WHERE L_USUARIO=?
0023:             ORDER BY ID DESC
0024:             OFFSET 0 ROWS FETCH NEXT 200 ROWS ONLY
0025:         """, (usuario,))
0026:     return fetchall_dict(cur)
```

==========
ARQUIVO: app/infra/repositories/parceiros_repo.py
==========

```
0001: from app.domain.errors import BusinessError
0002: from app.infra.db import fetchall_dict, fetchone_dict
0003:
0004:
0005: def _has_column(cur, table_name: str, column_name: str) -> bool:
0006:     cur.execute(f"SELECT COL_LENGTH('{table_name}', '{column_name}') AS col")
0007:     row = cur.fetchone()
0008:     return bool(row and row[0] is not None)
0009:
0010:
```

```
0011: def sa1_criar_tx(
0012:     cur,
0013:     filial: str,
0014:     cod: str,
0015:     nome: str,
0016:     doc: str | None,
0017:     email: str | None,
0018:     tel: str | None,
0019:     end: str | None,
0020:     mun: str | None,
0021:     uf: str | None,
0022:     cep: str | None,
0023:     tabela_id: int | None,
0024: ) -> int:
0025:     cur.execute(
0026:         """
0027:         INSERT INTO dbo.SA1_CLIENTES
0028:           (A1_FILIAL, A1_COD, A1_NOME, A1_DOC, A1_EMAIL, A1_TEL, A1_END, A1_MUN, A1_UF, A1_CEP
,
0029:            A1_TABELA_ID, A1_ATIVO)
0030:         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 1);
0031:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0032:     """,
0033:         (filial, cod, nome, doc, email, tel, end, mun, uf, cep, tabela_id),
0034:     )
0035:     return int(fetchone_dict(cur)["id"])
0036:
0037:
0038: def sa1_listar_tx(cur, filial: str, ativo: bool = True, q: str | None = None):
0039:     sql = """
0040:       SELECT
0041:         c.*,
0042:         t.P0_COD AS TABELA_COD,
0043:         t.P0_DESC AS TABELA_DESC
0044:       FROM dbo.SA1_CLIENTES c
0045:       LEFT JOIN dbo.PR0_TABELA t ON t.ID = c.A1_TABELA_ID
0046:       WHERE c.A1_FILIAL = ?
0047:         AND (c.D_E_L_E_T_ IS NULL OR c.D_E_L_E_T_ <> '*')
0048:     """
0049:     params = [filial]
0050:     if ativo:
0051:         sql += " AND c.A1_ATIVO = 1"
0052:     if q:
0053:         sql += " AND (c.A1_NOME LIKE ? OR c.A1_COD LIKE ? OR c.A1_DOC LIKE ?)"
0054:         like = f"%{q}%"
0055:         params.extend([like, like, like])
0056:     if _has_column(cur, "dbo.SA1_CLIENTES", "ID"):
0057:         sql += " ORDER BY c.ID DESC"
0058:     else:
0059:         sql += " ORDER BY c.A1_COD DESC"
0060:     cur.execute(sql, tuple(params))
0061:     return fetchall_dict(cur)
0062:
0063:
```

```
0064: def sa1_obter_tx(cur, filial: str, cod: str):
0065:     cur.execute(
0066:         """
0067:         SELECT
0068:           c.*,
0069:           t.P0_COD AS TABELA_COD,
0070:           t.P0_DESC AS TABELA_DESC
0071:         FROM dbo.SA1_CLIENTES c
0072:         LEFT JOIN dbo.PR0_TABELA t ON t.ID = c.A1_TABELA_ID
0073:         WHERE c.A1_FILIAL=? AND c.A1_COD=?
0074:           AND (c.D_E_L_E_T_ IS NULL OR c.D_E_L_E_T_ <> '*')
0075:     """,
0076:         (filial, cod),
0077:     )
0078:     return fetchone_dict(cur)
0079:
0080:
0081: def sa1_set_tabela_tx(cur, filial: str, cod: str, tabela_id: int | None):
0082:     cur.execute(
0083:         """
0084:         UPDATE dbo.SA1_CLIENTES
0085:         SET A1_TABELA_ID = ?
0086:         WHERE A1_FILIAL=? AND A1_COD=? AND A1_ATIVO=1
0087:     """,
0088:         (tabela_id, filial, cod),
0089:     )
0090:     if cur.rowcount == 0:
0091:         raise BusinessError("Cliente não encontrado/ativo")
0092:
0093:
0094: def sa1_inativar_tx(cur, filial: str, cod: str):
0095:     cur.execute(
0096:         """
0097:         UPDATE dbo.SA1_CLIENTES
0098:         SET A1_ATIVO=0
0099:         WHERE A1_FILIAL=? AND A1_COD=? AND A1_ATIVO=1
0100:     """,
0101:         (filial, cod),
0102:     )
0103:     if cur.rowcount == 0:
0104:         raise BusinessError("Cliente não encontrado/ativo")
0105:
0106:
0107: def sa2_criar_tx(
0108:     cur,
0109:     filial: str,
0110:     cod: str,
0111:     nome: str,
0112:     doc: str | None,
0113:     email: str | None,
0114:     tel: str | None,
0115:     end: str | None,
0116:     mun: str | None,
0117:     uf: str | None,
```

```
0118:     cep: str | None,
0119: ) -> int:
0120:     cur.execute(
0121:         """
0122:         INSERT INTO dbo.SA2_FORNECEDORES
0123:           (A2_FILIAL, A2_COD, A2_NOME, A2_DOC, A2_EMAIL, A2_TEL, A2_END, A2_MUN, A2_UF, A2_CEP
,
0124:            A2_ATIVO)
0125:         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 1);
0126:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0127:     """,
0128:         (filial, cod, nome, doc, email, tel, end, mun, uf, cep),
0129:     )
0130:     return int(fetchone_dict(cur)["id"])
0131:
0132:
0133: def sa2_listar_tx(cur, filial: str, ativo: bool = True, q: str | None = None):
0134:     sql = "SELECT * FROM dbo.SA2_FORNECEDORES WHERE A2_FILIAL=? AND (D_E_L_E_T_ IS NULL OR D_E
_L_E_T_ <> '*')"
0135:     params = [filial]
0136:     if ativo:
0137:         sql += " AND A2_ATIVO=1"
0138:     if q:
0139:         sql += " AND (A2_NOME LIKE ? OR A2_COD LIKE ? OR A2_DOC LIKE ?)"
0140:         like = f"%{q}%"
0141:         params.extend([like, like, like])
0142:     if _has_column(cur, "dbo.SA2_FORNECEDORES", "ID"):
0143:         sql += " ORDER BY ID DESC"
0144:     else:
0145:         sql += " ORDER BY A2_COD DESC"
0146:     cur.execute(sql, tuple(params))
0147:     return fetchall_dict(cur)
0148:
0149:
0150: def sa2_obter_tx(cur, filial: str, cod: str):
0151:     cur.execute(
0152:         """
0153:         SELECT * FROM dbo.SA2_FORNECEDORES
0154:         WHERE A2_FILIAL=? AND A2_COD=?
0155:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0156:     """,
0157:         (filial, cod),
0158:     )
0159:     return fetchone_dict(cur)
0160:
0161:
0162: def sa2_inativar_tx(cur, filial: str, cod: str):
0163:     cur.execute(
0164:         """
0165:         UPDATE dbo.SA2_FORNECEDORES
0166:         SET A2_ATIVO=0
0167:         WHERE A2_FILIAL=? AND A2_COD=? AND A2_ATIVO=1
0168:     """,
0169:         (filial, cod),
```

```
0170:     )
0171:     if cur.rowcount == 0:
0172:         raise BusinessError("Fornecedor não encontrado/ativo")
```

==========
ARQUIVO: app/infra/repositories/preco_repo.py
==========

```
0001: from datetime import date
0002:
0003: from app.domain.errors import BusinessError
0004: from app.infra.db import fetchall_dict, fetchone_dict
0005:
0006:
0007: def _has_deleted_column(cur, table_name: str) -> bool:
0008:     cur.execute(f"SELECT COL_LENGTH('{table_name}','D_E_L_E_T_') AS col")
0009:     row = cur.fetchone()
0010:     return bool(row and row[0] is not None)
0011:
0012:
0013: def listar_tabelas_tx(cur, filial: str):
0014:     sql = """
0015:         SELECT ID, P0_FILIAL, P0_COD, P0_DESC, P0_ATIVA
0016:         FROM dbo.PR0_TABELA
0017:         WHERE P0_FILIAL=? AND P0_ATIVA=1
0018:     """
0019:     if _has_deleted_column(cur, "dbo.PR0_TABELA"):
0020:         sql += " AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')"
0021:     sql += " ORDER BY P0_COD"
0022:
0023:     cur.execute(sql, (filial,))
0024:     return fetchall_dict(cur)
0025:
0026:
0027: def criar_tabela_tx(cur, filial: str, cod: str, desc: str) -> int:
0028:     cur.execute(
0029:         """
0030:         INSERT INTO dbo.PR0_TABELA (P0_FILIAL, P0_COD, P0_DESC)
0031:         VALUES (?, ?, ?);
0032:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0033:         """,
0034:         (filial, cod, desc),
0035:     )
0036:     return int(fetchone_dict(cur)["id"])
0037:
0038:
0039: def definir_preco_tx(cur, filial: str, tabela_id: int, produto: str, preco: float, dt_ini: dat
e):
0040:     sql = """
0041:         UPDATE dbo.PR1_PRECO
0042:         SET P1_DTFIM = DATEADD(DAY, -1, ?)
0043:         WHERE P1_FILIAL=? AND P1_TABELA_ID=? AND P1_PRODUTO=? AND P1_DTFIM IS NULL
0044:     """
0045:     if _has_deleted_column(cur, "dbo.PR1_PRECO"):
0046:         sql += " AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')"
```

```
0047:
0048:     cur.execute(sql, (dt_ini, filial, tabela_id, produto))
0049:
0050:     cur.execute(
0051:         """
0052:         INSERT INTO dbo.PR1_PRECO
0053:           (P1_FILIAL, P1_TABELA_ID, P1_PRODUTO, P1_PRECO, P1_DTINI)
0054:         VALUES (?, ?, ?, ?, ?)
0055:         """,
0056:         (filial, tabela_id, produto, float(preco), dt_ini),
0057:     )
0058:
0059:
0060: def listar_precos_tabela_tx(cur, filial: str, tabela_id: int):
0061:     sql = """
0062:         SELECT ID, P1_FILIAL, P1_TABELA_ID, P1_PRODUTO, P1_PRECO, P1_DTINI, P1_DTFIM
0063:         FROM dbo.PR1_PRECO
0064:         WHERE P1_FILIAL=? AND P1_TABELA_ID=?
0065:     """
0066:     if _has_deleted_column(cur, "dbo.PR1_PRECO"):
0067:         sql += " AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')"
0068:     sql += " ORDER BY P1_PRODUTO, P1_DTINI DESC"
0069:
0070:     cur.execute(sql, (filial, tabela_id))
0071:     return fetchall_dict(cur)
0072:
0073:
0074: def buscar_preco_vigente_tx(cur, filial: str, tabela_id: int, produto: str, data: date):
0075:     sql = """
0076:         SELECT TOP 1 P1_PRECO
0077:         FROM dbo.PR1_PRECO
0078:         WHERE P1_FILIAL=? AND P1_TABELA_ID=? AND P1_PRODUTO=?
0079:           AND P1_DTINI <= ?
0080:           AND (P1_DTFIM IS NULL OR P1_DTFIM >= ?)
0081:     """
0082:     if _has_deleted_column(cur, "dbo.PR1_PRECO"):
0083:         sql += " AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')"
0084:     sql += " ORDER BY P1_DTINI DESC"
0085:
0086:     cur.execute(sql, (filial, tabela_id, produto, data, data))
0087:     row = fetchone_dict(cur)
0088:     if not row:
0089:         raise BusinessError("Preco nao encontrado")
0090:     return float(row["P1_PRECO"])
```

==========
ARQUIVO: app/infra/repositories/relatorios_repo.py
==========

```
0001: from datetime import date
0002: from app.infra.db import fetchone_dict, fetchall_dict
0003:
0004: def dre_simples_tx(cur, filial: str, de: date | None, ate: date | None):
0005:     filtros = ["C5_FILIAL=?", "(D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')"]
0006:     params = [filial]
```

```
0007:     if de:
0008:         filtros.append("C5_DATA >= ?")
0009:         params.append(de)
0010:     if ate:
0011:         filtros.append("C5_DATA < DATEADD(DAY, 1, ?)")
0012:         params.append(ate)
0013:     where_sql = " AND ".join(filtros)
0014:
0015:     # Receita + impostos (SC5)
0016:     cur.execute(f"""
0017:         SELECT
0018:           COALESCE(SUM(C5_VALOR_TOTAL),0) AS receita,
0019:           COALESCE(SUM(C5_ICMS),0) AS icms,
0020:           COALESCE(SUM(C5_IPI),0)  AS ipi
0021:         FROM dbo.SC5_PEDIDOS
0022:         WHERE {where_sql} AND C5_STATUS='FATURADO'
0023:     """, tuple(params))
0024:     r = fetchone_dict(cur) or {"receita": 0, "icms": 0, "ipi": 0}
0025:
0026:     # Impostos adicionais (SF2)
0027:     cur.execute(
0028:         f"""
0029:         SELECT
0030:           COALESCE(SUM(f2.F2_PIS),0) AS pis,
0031:           COALESCE(SUM(f2.F2_COFINS),0) AS cofins,
0032:           COALESCE(SUM(f2.F2_ICMS_ST),0) AS icms_st,
0033:           COALESCE(SUM(f2.F2_DIFAL),0) AS difal
0034:         FROM dbo.SF2_NOTAS f2
0035:         WHERE f2.F2_FILIAL=?
0036:           AND (f2.D_E_L_E_T_ IS NULL OR f2.D_E_L_E_T_ <> '*')
0037:           AND f2.F2_STATUS='EMITIDA'
0038:           AND (f2.F2_EMISSAO >= COALESCE(?, '19000101'))
0039:           AND (f2.F2_EMISSAO <  DATEADD(DAY,1, COALESCE(?, '29991231')))
0040:         """,
0041:         (filial, de, ate),
0042:     )
0043:     imp = fetchone_dict(cur) or {"pis": 0, "cofins": 0, "icms_st": 0, "difal": 0}
0044:
0045:     # CMV (SD3 saídas vinculadas a pedido)
0046:     cur.execute(f"""
0047:         SELECT COALESCE(SUM(D3_VALOR),0) AS cmv
0048:         FROM dbo.SD3_MOV
0049:         WHERE D3_FILIAL=? AND D3_TIPO='S'
0050:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0051:           AND D3_ORIGEM='RESERVA_FATURADA'
0052:           AND (D3_DATA >= COALESCE(?, '19000101'))
0053:           AND (D3_DATA <  DATEADD(DAY,1, COALESCE(?, '29991231')))
0054:     """, (filial, de, ate))
0055:     c = fetchone_dict(cur) or {"cmv": 0}
0056:
0057:     receita = float(r["receita"])
0058:     icms = float(r["icms"])
0059:     ipi = float(r["ipi"])
0060:     cmv = float(c["cmv"])
```

```
0061:     pis = float(imp["pis"])
0062:     cofins = float(imp["cofins"])
0063:     icms_st = float(imp["icms_st"])
0064:     difal = float(imp["difal"])
0065:
0066:     margem_bruta = receita - cmv
0067:     resultado = receita - cmv - icms - ipi - pis - cofins - icms_st - difal
0068:
0069:     return {
0070:         "filial": filial,
0071:         "de": de.isoformat() if de else None,
0072:         "ate": ate.isoformat() if ate else None,
0073:         "receita": receita,
0074:         "cmv": cmv,
0075:         "margem_bruta": margem_bruta,
0076:         "icms": icms,
0077:         "ipi": ipi,
0078:         "pis": pis,
0079:         "cofins": cofins,
0080:         "icms_st": icms_st,
0081:         "difal": difal,
0082:         "resultado": resultado,
0083:     }
0084:
0085: def margem_por_produto_tx(cur, filial: str, de: date | None, ate: date | None):
0086:     filtros_sc5 = ["p.C5_FILIAL=?","p.C5_STATUS='FATURADO'", "(p.D_E_L_E_T_ IS NULL OR p.D_E_L
_E_T_ <> '*')"]
0087:     params = [filial]
0088:     if de:
0089:         filtros_sc5.append("p.C5_DATA >= ?"); params.append(de)
0090:     if ate:
0091:         filtros_sc5.append("p.C5_DATA < DATEADD(DAY,1,?)"); params.append(ate)
0092:     where_sc5 = " AND ".join(filtros_sc5)
0093:
0094:     # Receita por produto (SC6)
0095:     cur.execute(f"""
0096:       SELECT
0097:         i.C6_PRODUTO AS produto,
0098:         SUM(i.C6_TOTAL) AS receita
0099:       FROM dbo.SC6_ITENS i
0100:       JOIN dbo.SC5_PEDIDOS p ON p.ID = i.C6_PEDIDO_ID
0101:       WHERE (i.D_E_L_E_T_ IS NULL OR i.D_E_L_E_T_ <> '*')
0102:         AND {where_sc5}
0103:       GROUP BY i.C6_PRODUTO
0104:       ORDER BY receita DESC
0105:     """, tuple(params))
0106:     rec = fetchall_dict(cur)
0107:
0108:     # CMV por produto (SD3)
0109:     cur.execute("""
0110:       SELECT D3_PRODUTO AS produto, SUM(D3_VALOR) AS cmv
0111:       FROM dbo.SD3_MOV
0112:       WHERE D3_FILIAL=? AND D3_TIPO='S' AND D3_ORIGEM='RESERVA_FATURADA'
0113:         AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
```

```
0114:         AND (D3_DATA >= COALESCE(?, '19000101'))
0115:         AND (D3_DATA <  DATEADD(DAY,1, COALESCE(?, '29991231')))
0116:       GROUP BY D3_PRODUTO
0117:     """, (filial, de, ate))
0118:     cmv = {row["produto"]: float(row["cmv"]) for row in fetchall_dict(cur)}
0119:
0120:     out = []
0121:     for r in rec:
0122:         prod = r["produto"]
0123:         receita = float(r["receita"])
0124:         custo = float(cmv.get(prod, 0))
0125:         out.append({
0126:             "produto": prod,
0127:             "receita": receita,
0128:             "cmv": custo,
0129:             "margem": receita - custo
0130:         })
0131:     return out
0132:
0133:
0134: def conciliacao_nf_financeiro_tx(
0135:     cur,
0136:     filial: str,
0137:     de: date | None,
0138:     ate: date | None,
0139:     tolerancia: float = 0.01,
0140:     limit: int = 200,
0141: ):
0142:     filtros_nf = ["f2.F2_FILIAL=?", "(f2.D_E_L_E_T_ IS NULL OR f2.D_E_L_E_T_ <> '*')"]
0143:     params_nf: list = [filial]
0144:     if de:
0145:         filtros_nf.append("f2.F2_EMISSAO >= ?")
0146:         params_nf.append(de)
0147:     if ate:
0148:         filtros_nf.append("f2.F2_EMISSAO < DATEADD(DAY,1,?)")
0149:         params_nf.append(ate)
0150:     where_nf = " AND ".join(filtros_nf)
0151:
0152:     # NF sem AR
0153:     cur.execute(
0154:         f"""
0155:         SELECT TOP ({int(limit)})
0156:           'NF_SEM_AR' AS issue,
0157:           f2.ID AS nf_id, f2.F2_DOC AS nf_doc, f2.F2_SERIE AS nf_serie,
0158:           f2.F2_STATUS AS nf_status, f2.F2_VALOR AS nf_valor, f2.F2_EMISSAO AS nf_emissao,
0159:           NULL AS ar_id, NULL AS ar_num, NULL AS ar_serie, NULL AS ar_status, NULL AS ar_valor
, NULL AS ar_venc,
0160:           'NF ativa sem título AR' AS info
0161:         FROM dbo.SF2_NOTAS f2
0162:         LEFT JOIN dbo.SE1_AR e1
0163:           ON TRY_CAST(e1.E1_REF AS INT) = f2.ID
0164:          AND (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0165:         WHERE {where_nf}
0166:           AND f2.F2_STATUS <> 'CANCELADA'
```

```
0167:           AND e1.ID IS NULL
0168:         ORDER BY f2.ID DESC
0169:         """,
0170:         tuple(params_nf),
0171:     )
0172:     nf_sem_ar = fetchall_dict(cur)
0173:
0174:     # AR sem NF
0175:     filtros_ar = ["e1.E1_FILIAL=?", "(e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')"]
0176:     params_ar: list = [filial]
0177:     if de:
0178:         filtros_ar.append("e1.E1_DATA >= ?")
0179:         params_ar.append(de)
0180:     if ate:
0181:         filtros_ar.append("e1.E1_DATA < DATEADD(DAY,1,?)")
0182:         params_ar.append(ate)
0183:     where_ar = " AND ".join(filtros_ar)
0184:
0185:     cur.execute(
0186:         f"""
0187:         SELECT TOP ({int(limit)})
0188:           'AR_SEM_NF' AS issue,
0189:           NULL AS nf_id, NULL AS nf_doc, NULL AS nf_serie, NULL AS nf_status, NULL AS nf_valor
, NULL AS nf_emissao,
0190:           e1.ID AS ar_id, e1.E1_NUM AS ar_num, e1.E1_SERIE AS ar_serie,
0191:           e1.E1_STATUS AS ar_status, e1.E1_VALOR AS ar_valor, e1.E1_VENC AS ar_venc,
0192:           'Título AR sem NF vinculada' AS info
0193:         FROM dbo.SE1_AR e1
0194:         LEFT JOIN dbo.SF2_NOTAS f2 ON f2.ID = TRY_CAST(e1.E1_REF AS INT)
0195:          AND (f2.D_E_L_E_T_ IS NULL OR f2.D_E_L_E_T_ <> '*')
0196:         WHERE {where_ar}
0197:           AND f2.ID IS NULL
0198:         ORDER BY e1.ID DESC
0199:         """,
0200:         tuple(params_ar),
0201:     )
0202:     ar_sem_nf = fetchall_dict(cur)
0203:
0204:     # Divergência de valores
0205:     cur.execute(
0206:         f"""
0207:         SELECT TOP ({int(limit)})
0208:           'VALOR_DIVERGENTE' AS issue,
0209:           f2.ID AS nf_id, f2.F2_DOC AS nf_doc, f2.F2_SERIE AS nf_serie,
0210:           f2.F2_STATUS AS nf_status, f2.F2_VALOR AS nf_valor, f2.F2_EMISSAO AS nf_emissao,
0211:           e1.ID AS ar_id, e1.E1_NUM AS ar_num, e1.E1_SERIE AS ar_serie,
0212:           e1.E1_STATUS AS ar_status, e1.E1_VALOR AS ar_valor, e1.E1_VENC AS ar_venc,
0213:           'AR x NF com valor diferente' AS info
0214:         FROM dbo.SF2_NOTAS f2
0215:         JOIN dbo.SE1_AR e1
0216:           ON TRY_CAST(e1.E1_REF AS INT) = f2.ID
0217:          AND (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0218:         WHERE {where_nf}
0219:           AND ABS(COALESCE(e1.E1_VALOR,0) - COALESCE(f2.F2_VALOR,0)) > ?
```

```
0220:         ORDER BY f2.ID DESC
0221:         """,
0222:         tuple(params_nf + [float(tolerancia)]),
0223:     )
0224:     valor_div = fetchall_dict(cur)
0225:
0226:     # NF cancelada com AR aberto/baixado
0227:     cur.execute(
0228:         f"""
0229:         SELECT TOP ({int(limit)})
0230:           CASE WHEN e1.E1_STATUS='BAIXADO' THEN 'NF_CANCELADA_AR_BAIXADO' ELSE 'NF_CANCELADA_A
R_ABERTO' END AS issue,
0231:           f2.ID AS nf_id, f2.F2_DOC AS nf_doc, f2.F2_SERIE AS nf_serie,
0232:           f2.F2_STATUS AS nf_status, f2.F2_VALOR AS nf_valor, f2.F2_EMISSAO AS nf_emissao,
0233:           e1.ID AS ar_id, e1.E1_NUM AS ar_num, e1.E1_SERIE AS ar_serie,
0234:           e1.E1_STATUS AS ar_status, e1.E1_VALOR AS ar_valor, e1.E1_VENC AS ar_venc,
0235:           'NF cancelada com AR ativo' AS info
0236:         FROM dbo.SF2_NOTAS f2
0237:         JOIN dbo.SE1_AR e1
0238:           ON TRY_CAST(e1.E1_REF AS INT) = f2.ID
0239:          AND (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0240:         WHERE {where_nf}
0241:           AND f2.F2_STATUS = 'CANCELADA'
0242:           AND e1.E1_STATUS IN ('ABERTO','BAIXADO')
0243:         ORDER BY f2.ID DESC
0244:         """,
0245:         tuple(params_nf),
0246:     )
0247:     nf_cancel_ar = fetchall_dict(cur)
0248:
0249:     return {
0250:         "nf_sem_ar": nf_sem_ar,
0251:         "ar_sem_nf": ar_sem_nf,
0252:         "valor_divergente": valor_div,
0253:         "nf_cancelada_ar": nf_cancel_ar,
0254:     }
0255:
0256:
0257: def consistencia_fiscal_tx(
0258:     cur,
0259:     filial: str,
0260:     de: date | None,
0261:     ate: date | None,
0262:     limit: int = 200,
0263: ):
0264:     filtros_nf = ["f2.F2_FILIAL=?", "(f2.D_E_L_E_T_ IS NULL OR f2.D_E_L_E_T_ <> '*')"]
0265:     params_nf: list = [filial]
0266:     if de:
0267:         filtros_nf.append("f2.F2_EMISSAO >= ?")
0268:         params_nf.append(de)
0269:     if ate:
0270:         filtros_nf.append("f2.F2_EMISSAO < DATEADD(DAY,1,?)")
0271:         params_nf.append(ate)
0272:     where_nf = " AND ".join(filtros_nf)
```

```
0273:
0274:     # NF sem itens SD2
0275:     cur.execute(
0276:         f"""
0277:         SELECT TOP ({int(limit)})
0278:           'NF_SEM_SD2' AS issue,
0279:           f2.ID AS nf_id, f2.F2_DOC AS nf_doc, f2.F2_SERIE AS nf_serie,
0280:           f2.F2_STATUS AS nf_status, f2.F2_EMISSAO AS nf_emissao
0281:         FROM dbo.SF2_NOTAS f2
0282:         LEFT JOIN dbo.SD2_ITENS d2
0283:           ON d2.D2_NF_ID = f2.ID
0284:          AND (d2.D_E_L_E_T_ IS NULL OR d2.D_E_L_E_T_ <> '*')
0285:         WHERE {where_nf}
0286:           AND d2.ID IS NULL
0287:         ORDER BY f2.ID DESC
0288:         """,
0289:         tuple(params_nf),
0290:     )
0291:     nf_sem_sd2 = fetchall_dict(cur)
0292:
0293:     # SD2 sem NF
0294:     cur.execute(
0295:         f"""
0296:         SELECT TOP ({int(limit)})
0297:           'SD2_SEM_NF' AS issue,
0298:           d2.ID AS sd2_id, d2.D2_NF_ID AS nf_id, d2.D2_DOC AS nf_doc, d2.D2_SERIE AS nf_serie,
0299:           d2.D2_PRODUTO AS produto, d2.D2_QTD AS qtd
0300:         FROM dbo.SD2_ITENS d2
0301:         LEFT JOIN dbo.SF2_NOTAS f2 ON f2.ID = d2.D2_NF_ID
0302:          AND (f2.D_E_L_E_T_ IS NULL OR f2.D_E_L_E_T_ <> '*')
0303:         WHERE (d2.D_E_L_E_T_ IS NULL OR d2.D_E_L_E_T_ <> '*')
0304:           AND f2.ID IS NULL
0305:         ORDER BY d2.ID DESC
0306:         """,
0307:         tuple(),
0308:     )
0309:     sd2_sem_nf = fetchall_dict(cur)
0310:
0311:     # NF cancelada com SD2 ativo
0312:     cur.execute(
0313:         f"""
0314:         SELECT TOP ({int(limit)})
0315:           'NF_CANCELADA_SD2_ATIVO' AS issue,
0316:           f2.ID AS nf_id, f2.F2_DOC AS nf_doc, f2.F2_SERIE AS nf_serie,
0317:           f2.F2_STATUS AS nf_status, f2.F2_EMISSAO AS nf_emissao,
0318:           d2.ID AS sd2_id, d2.D2_PRODUTO AS produto
0319:         FROM dbo.SF2_NOTAS f2
0320:         JOIN dbo.SD2_ITENS d2 ON d2.D2_NF_ID = f2.ID
0321:          AND (d2.D_E_L_E_T_ IS NULL OR d2.D_E_L_E_T_ <> '*')
0322:         WHERE {where_nf}
0323:           AND f2.F2_STATUS = 'CANCELADA'
0324:           AND (d2.D2_STATUS IS NULL OR d2.D2_STATUS <> 'CANCELADO')
0325:         ORDER BY f2.ID DESC
0326:         """,
```

```
0327:         tuple(params_nf),
0328:     )
0329:     nf_cancel_sd2 = fetchall_dict(cur)
0330:
0331:     return {
0332:         "nf_sem_sd2": nf_sem_sd2,
0333:         "sd2_sem_nf": sd2_sem_nf,
0334:         "nf_cancelada_sd2": nf_cancel_sd2,
0335:     }
```

==========
ARQUIVO: app/infra/repositories/reservas_repo.py
==========

```
0001:
0002: from app.infra.db import fetchone_dict
0003: from app.domain.errors import BusinessError  # se você usa BusinessError
0004:
0005: def reserva_get_tx(cur, reserva_id: int) -> dict | None:
0006:     cur.execute("SELECT * FROM dbo.SZ0_RESERVA WHERE ID=?", (int(reserva_id),))
0007:     return fetchone_dict(cur)
0008:
0009: def reserva_criar_tx(
0010:     cur,
0011:     filial: str,
0012:     produto: str,
0013:     qtd: int,
0014:     usuario: str,
0015:     cliente_cod: str | None = None,
0016:     tabela_id: int | None = None,
0017:     preco_unit: float | None = None,
0018:     total: float | None = None,
0019: ) -> int:
0020:     cur.execute("""
0021:         INSERT INTO dbo.SZ0_RESERVA
0022:           (Z0_DATA, Z0_FILIAL, Z0_PRODUTO, Z0_QTD, Z0_USUARIO, Z0_STATUS,
0023:            Z0_CLIENTE_COD, Z0_TABELA_ID, Z0_PRECO_UNIT, Z0_TOTAL)
0024:         VALUES (SYSDATETIME(), ?, ?, ?, ?, 'ABERTA', ?, ?, ?, ?);
0025:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0026:     """, (filial, produto, int(qtd), usuario, cliente_cod, tabela_id, preco_unit, total))
0027:     row = fetchone_dict(cur)
0028:     return int(row["id"])
0029:
0030: def reserva_status_tx(cur, reserva_id: int, status: str) -> None:
0031:     cur.execute("UPDATE dbo.SZ0_RESERVA SET Z0_STATUS=? WHERE ID=?", (status, int(reserva_id))
)
0032:     if cur.rowcount == 0:
0033:         raise BusinessError("Reserva não encontrada para atualizar status")
0034:
0035: def reserva_set_cliente_preco_tx(
0036:     cur,
0037:     reserva_id: int,
0038:     cliente_cod: str | None,
0039:     tabela_id: int | None,
0040:     preco_unit: float | None,
```

```
0041:     total: float | None,
0042: ) -> None:
0043:     cur.execute("""
0044:         UPDATE dbo.SZ0_RESERVA
0045:         SET
0046:           Z0_CLIENTE_COD = COALESCE(?, Z0_CLIENTE_COD),
0047:           Z0_TABELA_ID   = COALESCE(?, Z0_TABELA_ID),
0048:           Z0_PRECO_UNIT  = COALESCE(?, Z0_PRECO_UNIT),
0049:           Z0_TOTAL       = COALESCE(?, Z0_TOTAL)
0050:         WHERE ID = ?
0051:     """, (cliente_cod, tabela_id, preco_unit, total, int(reserva_id)))
0052:     if cur.rowcount == 0:
0053:         raise BusinessError("Reserva não encontrada para atualizar snapshot")

==========================================================================================
ARQUIVO: app/infra/repositories/sa1_repo.py
==========================================================================================
0001: from __future__ import annotations
0002: from typing import Optional, List, Dict, Any
0003: from app.infra.db import fetchone_dict, fetchall_dict
0004: from app.core.exceptions import BusinessError
0005:
0006: def sa1_get_tx(cur, filial: str, cod: str) -> Optional[Dict[str, Any]]:
0007:     cur.execute("""
0008:         SELECT *
0009:         FROM dbo.SA1_CLIENTES
0010:         WHERE A1_FILIAL = ? AND A1_COD = ?
0011:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0012:     """, (filial, cod))
0013:     return fetchone_dict(cur)
0014:
0015: def sa1_listar_tx(cur, filial: str, ativos: bool = True, limit: int = 200) -> List[Dict[str, A
ny]]:
0016:     if ativos:
0017:         cur.execute("""
0018:             SELECT TOP (?) *
0019:             FROM dbo.SA1_CLIENTES
0020:             WHERE A1_FILIAL = ? AND A1_ATIVO = 1
0021:               AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0022:             ORDER BY A1_NOME
0023:         """, (int(limit), filial))
0024:     else:
0025:         cur.execute("""
0026:             SELECT TOP (?) *
0027:             FROM dbo.SA1_CLIENTES
0028:             WHERE A1_FILIAL = ?
0029:               AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0030:             ORDER BY A1_NOME
0031:         """, (int(limit), filial))
0032:     return fetchall_dict(cur)
0033:
0034: def sa1_buscar_tx(cur, filial: str, termo: str, limit: int = 50) -> List[Dict[str, Any]]:
0035:     termo = (termo or "").strip()
0036:     if not termo:
```

```
0037:         return []
0038:     like = f"%{termo}%"
0039:     cur.execute("""
0040:         SELECT TOP (?) *
0041:         FROM dbo.SA1_CLIENTES
0042:         WHERE A1_FILIAL = ?
0043:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0044:           AND (A1_COD LIKE ? OR A1_NOME LIKE ? OR A1_DOC LIKE ? OR A1_EMAIL LIKE ?)
0045:         ORDER BY A1_NOME
0046:     """, (int(limit), filial, like, like, like, like))
0047:     return fetchall_dict(cur)
0048:
0049: def sa1_criar_tx(
0050:     cur,
0051:     filial: str,
0052:     cod: str,
0053:     nome: str,
0054:     doc: Optional[str] = None,
0055:     email: Optional[str] = None,
0056:     fone: Optional[str] = None,
0057: ) -> Dict[str, Any]:
0058:     cod = (cod or "").strip()
0059:     nome = (nome or "").strip()
0060:     doc = (doc or None)
0061:     email = (email or None)
0062:     fone = (fone or None)
0063:
0064:     if not cod:
0065:         raise BusinessError("A1_COD é obrigatório")
0066:     if not nome:
0067:         raise BusinessError("A1_NOME é obrigatório")
0068:
0069:     # evita sobrescrever sem querer
0070:     existe = sa1_get_tx(cur, filial=filial, cod=cod)
0071:     if existe:
0072:         raise BusinessError("Cliente já existe nesta filial")
0073:
0074:     cur.execute("""
0075:         INSERT INTO dbo.SA1_CLIENTES (A1_COD, A1_FILIAL, A1_NOME, A1_DOC, A1_EMAIL, A1_FONE,
0076: A1_ATIVO)
0077:         VALUES (?, ?, ?, ?, ?, ?, 1)
0078:     """, (cod, filial, nome, doc, email, fone))
0079:
0080:     return sa1_get_tx(cur, filial=filial, cod=cod) or {"A1_COD": cod, "A1_FILIAL": filial}
0081:
0082: def sa1_atualizar_tx(
0083:     cur,
0084:     filial: str,
0085:     cod: str,
0086:     nome: Optional[str] = None,
0087:     doc: Optional[str] = None,
0088:     email: Optional[str] = None,
0089:     fone: Optional[str] = None,
0090: ) -> Dict[str, Any]:
```

```
0091:     atual = sa1_get_tx(cur, filial=filial, cod=cod)
0092:     if not atual:
0093:         raise BusinessError("Cliente não encontrado")
0094:
0095:     # mantém valores atuais quando None
0096:     nome2 = atual["A1_NOME"] if nome is None else (nome or "").strip()
0097:     doc2 = atual.get("A1_DOC") if doc is None else (doc or None)
0098:     email2 = atual.get("A1_EMAIL") if email is None else (email or None)
0099:     fone2 = atual.get("A1_FONE") if fone is None else (fone or None)
0100:
0101:     if not nome2:
0102:         raise BusinessError("A1_NOME não pode ficar vazio")
0103:
0104:     cur.execute("""
0105:         UPDATE dbo.SA1_CLIENTES
0106:         SET A1_NOME=?, A1_DOC=?, A1_EMAIL=?, A1_FONE=?
0107:         WHERE A1_FILIAL=? AND A1_COD=?
0108:     """, (nome2, doc2, email2, fone2, filial, cod))
0109:
0110:     return sa1_get_tx(cur, filial=filial, cod=cod) or atual
0111:
0112: def sa1_set_ativo_tx(cur, filial: str, cod: str, ativo: bool) -> None:
0113:     cur.execute("""
0114:         UPDATE dbo.SA1_CLIENTES
0115:         SET A1_ATIVO=?
0116:         WHERE A1_FILIAL=? AND A1_COD=?
0117:     """, (1 if ativo else 0, filial, cod))
0118:     if cur.rowcount == 0:
0119:         raise BusinessError("Cliente não encontrado")
```

==========
ARQUIVO: app/infra/repositories/sa2_repo.py
==========

```
0001: from __future__ import annotations
0002:
0003: from typing import Optional, List, Dict, Any
0004: from app.infra.db import fetchone_dict, fetchall_dict
0005: from app.core.exceptions import BusinessError
0006:
0007: def sa2_get_tx(cur, filial: str, cod: str) -> Optional[Dict[str, Any]]:
0008:     cur.execute("""
0009:         SELECT *
0010:         FROM dbo.SA2_FORNECEDORES
0011:         WHERE A2_FILIAL = ? AND A2_COD = ?
0012:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0013:     """, (filial, cod))
0014:     return fetchone_dict(cur)
0015:
0016: def sa2_listar_tx(cur, filial: str, ativos: bool = True, limit: int = 200) -> List[Dict[str, A
ny]]:
0017:     if ativos:
0018:         cur.execute("""
0019:             SELECT TOP (?) *
0020:             FROM dbo.SA2_FORNECEDORES
```

```
0021:             WHERE A2_FILIAL = ? AND A2_ATIVO = 1
0022:               AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0023:             ORDER BY A2_NOME
0024:         """, (int(limit), filial))
0025:     else:
0026:         cur.execute("""
0027:             SELECT TOP (?) *
0028:             FROM dbo.SA2_FORNECEDORES
0029:             WHERE A2_FILIAL = ?
0030:               AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0031:             ORDER BY A2_NOME
0032:         """, (int(limit), filial))
0033:     return fetchall_dict(cur)
0034:
0035: def sa2_buscar_tx(cur, filial: str, termo: str, limit: int = 50) -> List[Dict[str, Any]]:
0036:     termo = (termo or "").strip()
0037:     if not termo:
0038:         return []
0039:     like = f"%{termo}%"
0040:     cur.execute("""
0041:         SELECT TOP (?) *
0042:         FROM dbo.SA2_FORNECEDORES
0043:         WHERE A2_FILIAL = ?
0044:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0045:           AND (A2_COD LIKE ? OR A2_NOME LIKE ? OR A2_DOC LIKE ? OR A2_EMAIL LIKE ?)
0046:         ORDER BY A2_NOME
0047:     """, (int(limit), filial, like, like, like, like))
0048:     return fetchall_dict(cur)
0049:
0050: def sa2_criar_tx(
0051:     cur,
0052:     filial: str,
0053:     cod: str,
0054:     nome: str,
0055:     doc: Optional[str] = None,
0056:     email: Optional[str] = None,
0057:     fone: Optional[str] = None,
0058: ) -> Dict[str, Any]:
0059:     cod = (cod or "").strip()
0060:     nome = (nome or "").strip()
0061:     doc = (doc or None)
0062:     email = (email or None)
0063:     fone = (fone or None)
0064:
0065:     if not cod:
0066:         raise BusinessError("A2_COD é obrigatório")
0067:     if not nome:
0068:         raise BusinessError("A2_NOME é obrigatório")
0069:
0070:     existe = sa2_get_tx(cur, filial=filial, cod=cod)
0071:     if existe:
0072:         raise BusinessError("Fornecedor já existe nesta filial")
0073:
0074:     cur.execute("""
```

```
0075:         INSERT INTO dbo.SA2_FORNECEDORES (A2_COD, A2_FILIAL, A2_NOME, A2_DOC, A2_EMAIL,
0076: A2_FONE, A2_ATIVO)
0077:         VALUES (?, ?, ?, ?, ?, ?, 1)
0078:     """, (cod, filial, nome, doc, email, fone))
0079:
0080:     return sa2_get_tx(cur, filial=filial, cod=cod) or {"A2_COD": cod, "A2_FILIAL": filial}
0081:
0082: def sa2_atualizar_tx(
0083:     cur,
0084:     filial: str,
0085:     cod: str,
0086:     nome: Optional[str] = None,
0087:     doc: Optional[str] = None,
0088:     email: Optional[str] = None,
0089:     fone: Optional[str] = None,
0090: ) -> Dict[str, Any]:
0091:     atual = sa2_get_tx(cur, filial=filial, cod=cod)
0092:     if not atual:
0093:         raise BusinessError("Fornecedor não encontrado")
0094:
0095:     nome2 = atual["A2_NOME"] if nome is None else (nome or "").strip()
0096:     doc2 = atual.get("A2_DOC") if doc is None else (doc or None)
0097:     email2 = atual.get("A2_EMAIL") if email is None else (email or None)
0098:     fone2 = atual.get("A2_FONE") if fone is None else (fone or None)
0099:
0100:     if not nome2:
0101:         raise BusinessError("A2_NOME não pode ficar vazio")
0102:
0103:     cur.execute("""
0104:         UPDATE dbo.SA2_FORNECEDORES
0105:         SET A2_NOME=?, A2_DOC=?, A2_EMAIL=?, A2_FONE=?
0106:         WHERE A2_FILIAL=? AND A2_COD=?
0107:     """, (nome2, doc2, email2, fone2, filial, cod))
0108:
0109:     return sa2_get_tx(cur, filial=filial, cod=cod) or atual
0110:
0111: def sa2_set_ativo_tx(cur, filial: str, cod: str, ativo: bool) -> None:
0112:     cur.execute("""
0113:         UPDATE dbo.SA2_FORNECEDORES
0114:         SET A2_ATIVO=?
0115:         WHERE A2_FILIAL=? AND A2_COD=?
0116:     """, (1 if ativo else 0, filial, cod))
0117:     if cur.rowcount == 0:
0118:         raise BusinessError("Fornecedor não encontrado")
```

==========
ARQUIVO: app/infra/repositories/sb1_repo.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.infra.db import fetchone_dict, fetchall_dict
0003:
0004:
0005: def sb1_get_tx(cur, produto: str, filial: str) -> dict:
0006:     cur.execute(
```

```
0007:         """
0008:         SELECT B1_COD, B1_DESC, B1_PRECO, B1_ESTOQUE, B1_RESERVADO, B1_CM, B1_FILIAL, B1_NCM
0009:         FROM SB1_PRODUTOS
0010:         WHERE B1_COD=? AND B1_FILIAL=?
0011:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0012:         """,
0013:         (produto, filial),
0014:     )
0015:     row = fetchone_dict(cur)
0016:     if not row:
0017:         raise BusinessError("Produto não encontrado na filial")
0018:     return row
0019:
0020:
0021: def sb1_upsert_tx(cur, cod: str, desc: str, preco: float, filial: str) -> None:
0022:     cur.execute(
0023:         "SELECT 1 FROM SB1_PRODUTOS WHERE B1_COD=? AND B1_FILIAL=? AND (D_E_L_E_T_ IS NULL OR
D_E_L_E_T_ <> '*')",
0024:         (cod, filial),
0025:     )
0026:     existe = cur.fetchone() is not None
0027:     if existe:
0028:         cur.execute(
0029:             "UPDATE SB1_PRODUTOS SET B1_DESC=?, B1_PRECO=? WHERE B1_COD=? AND B1_FILIAL=?",
0030:             (desc, float(preco), cod, filial),
0031:         )
0032:     else:
0033:         cur.execute(
0034:             """
0035:             INSERT INTO SB1_PRODUTOS (B1_COD, B1_DESC, B1_PRECO, B1_ESTOQUE, B1_RESERVADO, B1_
CM, B1_FILIAL)
0036:             VALUES (?, ?, ?, 0, 0, 0, ?)
0037:             """,
0038:             (cod, desc, float(preco), filial),
0039:         )
0040:
0041:
0042: def sb1_atualizar_cm_tx(cur, produto: str, filial: str, cm_novo: float) -> None:
0043:     cur.execute(
0044:         "UPDATE SB1_PRODUTOS SET B1_CM=? WHERE B1_COD=? AND B1_FILIAL=?",
0045:         (float(cm_novo), produto, filial),
0046:     )
0047:
0048:
0049: def sb1_entrada_tx(cur, produto: str, filial: str, qtd: int) -> None:
0050:     cur.execute(
0051:         "UPDATE SB1_PRODUTOS SET B1_ESTOQUE = B1_ESTOQUE + ? WHERE B1_COD=? AND B1_FILIAL=?",
0052:         (int(qtd), produto, filial),
0053:     )
0054:
0055:
0056: def sb1_reservar_atomico_tx(cur, produto: str, filial: str, qtd: int) -> None:
0057:     cur.execute(
0058:         """
```

```
0059:         UPDATE SB1_PRODUTOS
0060:         SET B1_RESERVADO = B1_RESERVADO + ?
0061:         WHERE B1_COD=? AND B1_FILIAL=?
0062:           AND (B1_ESTOQUE - B1_RESERVADO) >= ?
0063:         """,
0064:         (int(qtd), produto, filial, int(qtd)),
0065:     )
0066:     if cur.rowcount == 0:
0067:         raise BusinessError("Saldo disponível insuficiente para reservar")
0068:
0069:
0070: def sb1_cancelar_reserva_atomico_tx(cur, produto: str, filial: str, qtd: int) -> None:
0071:     cur.execute(
0072:         """
0073:         UPDATE SB1_PRODUTOS
0074:         SET B1_RESERVADO = B1_RESERVADO - ?
0075:         WHERE B1_COD=? AND B1_FILIAL=?
0076:           AND B1_RESERVADO >= ?
0077:         """,
0078:         (int(qtd), produto, filial, int(qtd)),
0079:     )
0080:     if cur.rowcount == 0:
0081:         raise BusinessError("Não foi possível desreservar (reservado insuficiente)")
0082:
0083:
0084: def sb1_baixar_e_desreservar_atomico_tx(cur, produto: str, filial: str, qtd: int) -> None:
0085:     cur.execute(
0086:         """
0087:         UPDATE SB1_PRODUTOS
0088:         SET B1_ESTOQUE = B1_ESTOQUE - ?,
0089:             B1_RESERVADO = B1_RESERVADO - ?
0090:         WHERE B1_COD=? AND B1_FILIAL=?
0091:           AND B1_ESTOQUE >= ?
0092:           AND B1_RESERVADO >= ?
0093:         """,
0094:         (int(qtd), int(qtd), produto, filial, int(qtd), int(qtd)),
0095:     )
0096:     if cur.rowcount == 0:
0097:         raise BusinessError("Estoque/reservado insuficiente para confirmar")
0098:
0099:
0100: def sb1_baixar_atomico_tx(cur, produto: str, filial: str, qtd: int) -> None:
0101:     cur.execute(
0102:         """
0103:         UPDATE SB1_PRODUTOS
0104:         SET B1_ESTOQUE = B1_ESTOQUE - ?
0105:         WHERE B1_COD=? AND B1_FILIAL=?
0106:           AND B1_ESTOQUE - B1_RESERVADO >= ?
0107:         """,
0108:         (int(qtd), produto, filial, int(qtd)),
0109:     )
0110:     if cur.rowcount == 0:
0111:         raise BusinessError("Saldo disponível insuficiente")
0112:
```

```
0113:
0114: def sb1_buscar_tx(cur, filial: str, q: str, limite: int = 20) -> list[dict]:
0115:     termo = (q or "").strip()
0116:     if not termo:
0117:         return []
0118:
0119:     like = f"%{termo}%"
0120:     cur.execute(
0121:         """
0122:         SELECT TOP (?) B1_COD, B1_DESC, B1_PRECO, B1_ESTOQUE, B1_RESERVADO, B1_CM, B1_FILIAL,
B1_NCM
0123:         FROM SB1_PRODUTOS
0124:         WHERE B1_FILIAL=?
0125:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0126:           AND (
0127:             B1_COD LIKE ?
0128:             OR B1_DESC LIKE ?
0129:           )
0130:         ORDER BY B1_COD ASC
0131:         """,
0132:         (int(limite), filial, like, like),
0133:     )
0134:     return fetchall_dict(cur)
```

==========
ARQUIVO: app/infra/repositories/sd3_repo.py
==========

```
0001: from datetime import datetime
0002: from app.infra.db import fetchone_dict, fetchall_dict
0003: from app.core.exceptions import BusinessError
0004:
0005: def sd3_mov_tx(
0006:     cur,
0007:     filial: str,
0008:     produto: str,
0009:     tipo: str,     # 'E'/'S'
0010:     qtd: int,
0011:     custo_unit: float,
0012:     valor: float,
0013:     origem: str,
0014:     ref: str | None,
0015:     usuario: str,
0016: ) -> None:
0017:     if tipo not in {"E", "S"}:
0018:         raise BusinessError("Tipo inválido (E/S)")
0019:     if qtd <= 0:
0020:         raise BusinessError("Quantidade deve ser > 0")
0021:
0022:     cur.execute(
0023:         """
0024:         INSERT INTO SD3_MOV (
0025:             D3_DATA, D3_FILIAL, D3_PRODUTO, D3_TIPO, D3_QTD,
0026:             D3_CUSTO_UNIT, D3_VALOR, D3_ORIGEM, D3_REF, D3_USUARIO
0027:         )
```

```
0028:         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
0029:         """,
0030:         (datetime.now(), filial, produto, tipo, int(qtd), float(custo_unit), float(valor), ori
gem, ref, usuario),
0031:     )
0032:
0033: def sd3_saldo_tx(cur, produto: str, filial: str) -> int:
0034:     cur.execute(
0035:         """
0036:         SELECT COALESCE(SUM(CASE WHEN D3_TIPO='E' THEN D3_QTD ELSE -D3_QTD END), 0) AS SALDO
0037:         FROM SD3_MOV
0038:         WHERE D3_PRODUTO=? AND D3_FILIAL=?
0039:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0040:         """,
0041:         (produto, filial),
0042:     )
0043:     row = fetchone_dict(cur)
0044:     return int(row["SALDO"]) if row else 0
0045: def extrato_sd3_tx(cur, produto: str, filial: str, limite: int) -> list[dict]:
0046:     cur.execute("""
0047:         SELECT ID, D3_DATA, D3_TIPO, D3_QTD, D3_CUSTO_UNIT, D3_VALOR, D3_ORIGEM, D3_REF,
0048: D3_USUARIO
0049:         FROM SD3_MOV
0050:         WHERE D3_PRODUTO=? AND D3_FILIAL=?
0051:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0052:         ORDER BY ID DESC
0053:         OFFSET 0 ROWS FETCH NEXT ? ROWS ONLY
0054:     """, (produto, filial, int(limite)))
0055:     return fetchall_dict(cur)
0056:
0057: saldo_sd3_tx = sd3_saldo_tx
```

==========
ARQUIVO: app/infra/repositories/serie_repo.py
==========

```
0001: from __future__ import annotations
0002:
0003: from app.infra.repositories.sm0_repo import sm0_obter_series_tx
0004: from app.infra.repositories.sx5_repo import sx5_obter_tx, sx5_criar_tx
0005: from app.core.exceptions import BusinessError
0006:
0007:
0008: def resolve_serie_tx(cur, filial: str, tabela: str) -> str:
0009:     tabela = (tabela or "").strip().upper()
0010:     filial = (filial or "01").strip()
0011:
0012:     sm0 = sm0_obter_series_tx(cur, filial)
0013:     if sm0:
0014:         if tabela == "NF" and sm0.get("M0_SERIE_NF"):
0015:             return str(sm0["M0_SERIE_NF"]).strip()
0016:         if tabela == "AR" and sm0.get("M0_SERIE_AR"):
0017:             return str(sm0["M0_SERIE_AR"]).strip()
0018:         if tabela == "AP" and sm0.get("M0_SERIE_AP"):
0019:             return str(sm0["M0_SERIE_AP"]).strip()
```

```
0020:
0021:     sx5 = sx5_obter_tx(cur, filial=filial, tabela="SERIE", chave=tabela)
0022:     if sx5 and sx5.get("X5_DESCR"):
0023:         return str(sx5["X5_DESCR"]).strip()
0024:
0025:     return "1"
0026:
0027:
0028: def validar_serie_tx(cur, filial: str, tabela: str, serie: str) -> None:
0029:     tabela = (tabela or "").strip().upper()
0030:     serie = (serie or "").strip()
0031:     if not serie:
0032:         raise BusinessError("Série inválida")
0033:
0034:     sx5 = sx5_obter_tx(cur, filial=filial, tabela="SERIE", chave=tabela)
0035:     if sx5 is None:
0036:         raise BusinessError("Série não cadastrada no SX5 (tabela SERIE)")
0037:     if str(sx5.get("X5_DESCR") or "").strip() != serie:
0038:         raise BusinessError("Série informada não confere com SX5 (SERIE)")
0039:
0040:
0041: def seed_series_basicas_tx(cur, filial: str) -> None:
0042:     for chave in ("NF", "AR", "AP"):
0043:         if sx5_obter_tx(cur, filial=filial, tabela="SERIE", chave=chave) is None:
0044:             sx5_criar_tx(cur, filial=filial, tabela="SERIE", chave=chave, descr="1", ativo=Tru
e)
```

==========
ARQUIVO: app/infra/repositories/sm0_repo.py
==========

```
0001: from __future__ import annotations
0002:
0003: from app.infra.db import fetchone_dict, fetchall_dict
0004: from app.core.exceptions import BusinessError
0005:
0006:
0007: def sm0_obter_series_tx(cur, filial: str) -> dict:
0008:     cur.execute(
0009:         """
0010:         SELECT TOP 1 M0_FILIAL, M0_SERIE_NF, M0_SERIE_AR, M0_SERIE_AP
0011:         FROM dbo.SM0_EMPRESA
0012:         WHERE M0_FILIAL=?
0013:         """,
0014:         (filial,),
0015:     )
0016:     return fetchone_dict(cur) or {}
0017:
0018:
0019: def sm0_existe_tx(cur, filial: str) -> bool:
0020:     cur.execute(
0021:         """
0022:         SELECT 1
0023:         FROM dbo.SM0_EMPRESA
0024:         WHERE M0_FILIAL=?
```

```
0025:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0026:         """,
0027:         (filial,),
0028:     )
0029:     return cur.fetchone() is not None
0030:
0031:
0032: def sm0_listar_tx(cur):
0033:     cur.execute(
0034:         """
0035:         SELECT M0_FILIAL, M0_SERIE_NF, M0_SERIE_AR, M0_SERIE_AP
0036:         FROM dbo.SM0_EMPRESA
0037:         WHERE (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0038:         ORDER BY M0_FILIAL
0039:         """
0040:     )
0041:     return fetchall_dict(cur)
0042:
0043:
0044: def sm0_criar_tx(
0045:     cur,
0046:     filial: str,
0047:     serie_nf: str | None,
0048:     serie_ar: str | None,
0049:     serie_ap: str | None,
0050: ):
0051:     cur.execute(
0052:         "SELECT 1 FROM dbo.SM0_EMPRESA WHERE M0_FILIAL=?",
0053:         (filial,),
0054:     )
0055:     if cur.fetchone() is not None:
0056:         raise BusinessError("Filial já existe")
0057:     cur.execute(
0058:         """
0059:         INSERT INTO dbo.SM0_EMPRESA (M0_FILIAL, M0_SERIE_NF, M0_SERIE_AR, M0_SERIE_AP)
0060:         VALUES (?, ?, ?, ?)
0061:         """,
0062:         (filial, serie_nf, serie_ar, serie_ap),
0063:     )
0064:
0065:
0066: def sm0_inativar_tx(cur, filial: str):
0067:     cur.execute(
0068:         """
0069:         UPDATE dbo.SM0_EMPRESA
0070:         SET D_E_L_E_T_='*',
0071:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0072:         WHERE M0_FILIAL=?
0073:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0074:         """,
0075:         (filial,),
0076:     )
0077:     if cur.rowcount == 0:
0078:         raise BusinessError("Filial não encontrada")
```

```
0079:
0080:
0081: def sm0_definir_series_tx(cur, filial: str, serie_nf: str | None, serie_ar: str | None, serie_
ap: str | None):
0082:     atual = sm0_obter_series_tx(cur, filial)
0083:     if not atual:
0084:         raise BusinessError("Filial não encontrada em SM0_EMPRESA")
0085:
0086:     cur.execute(
0087:         """
0088:         UPDATE dbo.SM0_EMPRESA
0089:         SET M0_SERIE_NF = COALESCE(?, M0_SERIE_NF),
0090:             M0_SERIE_AR = COALESCE(?, M0_SERIE_AR),
0091:             M0_SERIE_AP = COALESCE(?, M0_SERIE_AP)
0092:         WHERE M0_FILIAL=?
0093:         """,
0094:         (serie_nf, serie_ar, serie_ap, filial),
0095:     )
```

==========
ARQUIVO: app/infra/repositories/sx5_repo.py
==========

```
0001: from __future__ import annotations
0002:
0003: from app.infra.db import fetchall_dict, fetchone_dict
0004: from app.core.exceptions import BusinessError
0005:
0006:
0007: def sx5_listar_tx(cur, filial: str, tabela: str, ativos: bool = True):
0008:     sql = """
0009:         SELECT *
0010:         FROM dbo.SX5_TABELAS
0011:         WHERE X5_FILIAL=? AND X5_TABELA=?
0012:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0013:     """
0014:     params: list = [filial, tabela]
0015:     if ativos:
0016:         sql += " AND X5_ATIVO=1"
0017:     sql += " ORDER BY X5_CHAVE"
0018:     cur.execute(sql, tuple(params))
0019:     return fetchall_dict(cur)
0020:
0021:
0022: def sx5_obter_tx(cur, filial: str, tabela: str, chave: str):
0023:     cur.execute(
0024:         """
0025:         SELECT *
0026:         FROM dbo.SX5_TABELAS
0027:         WHERE X5_FILIAL=? AND X5_TABELA=? AND X5_CHAVE=?
0028:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0029:         """,
0030:         (filial, tabela, chave),
0031:     )
0032:     return fetchone_dict(cur)
```

```
0033:
0034:
0035: def sx5_criar_tx(cur, filial: str, tabela: str, chave: str, descr: str, ativo: bool = True) ->
 None:
0036:     existe = sx5_obter_tx(cur, filial, tabela, chave)
0037:     if existe:
0038:         raise BusinessError("Registro SX5 já existe")
0039:     cur.execute(
0040:         """
0041:         INSERT INTO dbo.SX5_TABELAS
0042:           (X5_FILIAL, X5_TABELA, X5_CHAVE, X5_DESCR, X5_ATIVO)
0043:         VALUES (?, ?, ?, ?, ?)
0044:         """,
0045:         (filial, tabela, chave, descr, 1 if ativo else 0),
0046:     )
0047:
0048:
0049: def sx5_atualizar_tx(cur, filial: str, tabela: str, chave: str, descr: str | None, ativo: bool
 | None):
0050:     atual = sx5_obter_tx(cur, filial, tabela, chave)
0051:     if not atual:
0052:         raise BusinessError("Registro SX5 não encontrado")
0053:
0054:     descr2 = descr if descr is not None else atual.get("X5_DESCR")
0055:     ativo2 = atual.get("X5_ATIVO") if ativo is None else (1 if ativo else 0)
0056:
0057:     cur.execute(
0058:         """
0059:         UPDATE dbo.SX5_TABELAS
0060:         SET X5_DESCR=?, X5_ATIVO=?
0061:         WHERE X5_FILIAL=? AND X5_TABELA=? AND X5_CHAVE=?
0062:         """,
0063:         (descr2, ativo2, filial, tabela, chave),
0064:     )
0065:
0066:
0067: def sx5_inativar_tx(cur, filial: str, tabela: str, chave: str) -> None:
0068:     cur.execute(
0069:         """
0070:         UPDATE dbo.SX5_TABELAS
0071:         SET X5_ATIVO=0,
0072:             D_E_L_E_T_='*',
0073:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0074:         WHERE X5_FILIAL=? AND X5_TABELA=? AND X5_CHAVE=?
0075:         """,
0076:         (filial, tabela, chave),
0077:     )
0078:     if cur.rowcount == 0:
0079:         raise BusinessError("Registro SX5 não encontrado")

==========================================================================================
ARQUIVO: app/infra/repositories/sx6_repo.py
==========================================================================================
0001: from __future__ import annotations
```

```
0002:
0003: from app.infra.db import fetchone_dict
0004: from app.core.exceptions import BusinessError
0005:
0006:
0007: def sx6_next_tx(cur, filial: str, serie: str, tabela: str) -> int:
0008:     filial = (filial or "01").strip()
0009:     serie = (serie or "").strip()
0010:     tabela = (tabela or "").strip().upper()
0011:     if not tabela:
0012:         raise BusinessError("Tabela SX6 inválida")
0013:
0014:     # lock row for update
0015:     cur.execute(
0016:         """
0017:         SELECT TOP 1 ID, X6_SEQ
0018:         FROM dbo.SX6_SEQ WITH (UPDLOCK, ROWLOCK)
0019:         WHERE X6_FILIAL=? AND X6_SERIE=? AND X6_TABELA=?
0020:         """,
0021:         (filial, serie, tabela),
0022:     )
0023:     row = fetchone_dict(cur)
0024:     if row:
0025:         seq = int(row["X6_SEQ"]) + 1
0026:         cur.execute(
0027:             "UPDATE dbo.SX6_SEQ SET X6_SEQ=? WHERE ID=?",
0028:             (seq, int(row["ID"])),
0029:         )
0030:         return seq
0031:
0032:     # create first
0033:     cur.execute(
0034:         """
0035:         INSERT INTO dbo.SX6_SEQ (X6_FILIAL, X6_SERIE, X6_TABELA, X6_SEQ)
0036:         VALUES (?, ?, ?, 1);
0037:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0038:         """,
0039:         (filial, serie, tabela),
0040:     )
0041:     return 1
```

==========
ARQUIVO: app/infra/repositories/transfer_repo.py
==========

```
0001: from datetime import datetime
0002:
0003: def transfer_registrar_tx(cur, produto: str, qtd: int, origem: str, destino: str, usuario: str
, ref: str) -> None:
0004:     cur.execute("""
0005:         INSERT INTO ST0_TRANSFER (T0_DATA, T0_PRODUTO, T0_QTD, T0_ORIGEM, T0_DESTINO,
0006: T0_USUARIO, T0_REF)
0007:         VALUES (?, ?, ?, ?, ?, ?, ?)
0008:     """, (datetime.now(), produto, int(qtd), origem, destino, usuario, ref))
```

==========
ARQUIVO: app/infra/repositories/usuarios_repo.py
==========

```
0001: from app.infra.db import fetchone_dict
0002:
0003: def usuarios_existem_tx(cur) -> bool:
0004:     cur.execute("SELECT TOP 1 1 AS X FROM USUARIOS")
0005:     return cur.fetchone() is not None
0006:
0007: def inserir_usuario_tx(cur, login: str, senha_hash: str, perfil: str) -> None:
0008:     cur.execute(
0009:         "INSERT INTO USUARIOS (U_LOGIN, U_SENHA_HASH, U_PERFIL) VALUES (?, ?, ?)",
0010:         (login, senha_hash, perfil),
0011:     )
0012:
0013: def obter_usuario_por_login_tx(cur, login: str):
0014:     cur.execute("SELECT * FROM USUARIOS WHERE U_LOGIN=?", (login,))
0015:     return fetchone_dict(cur)
0016:
0017: def obter_perfil_tx(cur, login: str) -> str:
0018:     cur.execute("SELECT U_PERFIL FROM USUARIOS WHERE U_LOGIN=?", (login,))
0019:     row = fetchone_dict(cur)
0020:     return row["U_PERFIL"] if row else None
```

==========
ARQUIVO: app/infra/repositories/vendas/__init__.py
==========

```
0001:
```

==========
ARQUIVO: app/infra/repositories/vendas/origem_repo.py
==========

```
0001: from app.infra.db import fetchone_dict
0002:
0003: def venda_buscar_reserva_id_por_pedido_tx(cur, pedido_id: int) -> int | None:
0004:     """
0005:     Retorna a reserva (SZ0_RESERVA.ID) que gerou esse pedido, se existir.
0006:     """
0007:     cur.execute("""
0008:         SELECT TOP 1 ID
0009:         FROM dbo.SZ0_RESERVA
0010:         WHERE Z0_PEDIDO_ID=?
0011:         ORDER BY ID DESC
0012:     """, (int(pedido_id),))
0013:     row = fetchone_dict(cur)
0014:     return int(row["ID"]) if row else None
```

==========
ARQUIVO: app/infra/repositories/vendas/relatorios_nf_id_repo.py
==========

```
0001: from __future__ import annotations
0002:
0003: from app.infra.db import fetchall_dict
0004:
```

```
0005:
0006: def nf_rastreio_tx(cur, nf_id: int) -> list[dict]:
0007:     cur.execute(
0008:         """
0009:         SELECT
0010:           f2.ID           AS nf_id,
0011:           f2.F2_DOC       AS nf_doc,
0012:           f2.F2_SERIE     AS nf_serie,
0013:           f2.F2_EMISSAO   AS nf_emissao,
0014:           f2.F2_STATUS    AS nf_status,
0015:           f2.F2_TES       AS nf_tes,
0016:           f2.F2_CFOP      AS nf_cfop,
0017:           f2.F2_VALOR     AS nf_valor,
0018:           f2.F2_ICMS      AS nf_icms,
0019:           f2.F2_IPI       AS nf_ipi,
0020:           f2.F2_TOTAL_BRUTO AS nf_total_bruto,
0021:
0022:           p.ID            AS pedido_id,
0023:           p.C5_STATUS     AS pedido_status,
0024:           p.C5_FILIAL     AS pedido_filial,
0025:           p.C5_DATA       AS pedido_data,
0026:
0027:           i.ID            AS item_id,
0028:           i.C6_PRODUTO    AS item_produto,
0029:           i.C6_QTD        AS item_qtd,
0030:           i.C6_TOTAL      AS item_total,
0031:
0032:           s.C9_ITEM       AS sc9_item,
0033:           s.C9_QTD        AS sc9_qtd,
0034:           s.C9_STATUS     AS sc9_status,
0035:
0036:           e1.ID           AS ar_id,
0037:           e1.E1_NUM       AS ar_num,
0038:           e1.E1_SERIE     AS ar_serie,
0039:           e1.E1_VALOR     AS ar_valor,
0040:           e1.E1_STATUS    AS ar_status,
0041:           e1.E1_VENC      AS ar_venc
0042:         FROM dbo.SF2_NOTAS f2
0043:         LEFT JOIN dbo.SC5_PEDIDOS p ON p.ID = f2.F2_PEDIDO_ID
0044:          AND (p.D_E_L_E_T_ IS NULL OR p.D_E_L_E_T_ <> '*')
0045:         LEFT JOIN dbo.SC6_ITENS i ON i.C6_PEDIDO_ID = p.ID
0046:          AND (i.D_E_L_E_T_ IS NULL OR i.D_E_L_E_T_ <> '*')
0047:         LEFT JOIN dbo.SC9_LIBERACAO s ON s.C9_PEDIDO_ID = p.ID AND s.C9_ITEM = i.ID
0048:          AND (s.D_E_L_E_T_ IS NULL OR s.D_E_L_E_T_ <> '*')
0049:         LEFT JOIN dbo.SE1_AR e1 ON TRY_CAST(e1.E1_REF AS INT) = f2.ID
0050:          AND (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0051:         WHERE f2.ID = ?
0052:           AND (f2.D_E_L_E_T_ IS NULL OR f2.D_E_L_E_T_ <> '*')
0053:         ORDER BY i.ID ASC
0054:         """,
0055:         (int(nf_id),),
0056:     )
0057:     return fetchall_dict(cur)
```

```
0001: from __future__ import annotations
0002:
0003: from app.infra.db import fetchall_dict
0004:
0005:
0006: def nf_pedido_financeiro_resumo_tx(
0007:     cur,
0008:     pedido_id: int,
0009:     nf_status: str | None = None,
0010:     ar_status: str | None = None,
0011:     sc9_status: str | None = None,
0012: ) -> list[dict]:
0013:     filtros = ["f2.F2_PEDIDO_ID = ?"]
0014:     params: list = [int(pedido_id)]
0015:
0016:     if nf_status:
0017:         filtros.append("f2.F2_STATUS = ?")
0018:         params.append(str(nf_status).strip())
0019:     if ar_status:
0020:         filtros.append("e1.E1_STATUS = ?")
0021:         params.append(str(ar_status).strip())
0022:     if sc9_status:
0023:         filtros.append("s.C9_STATUS = ?")
0024:         params.append(str(sc9_status).strip())
0025:
0026:     where_sql = " AND ".join(filtros)
0027:     cur.execute(
0028:         f"""
0029:         SELECT
0030:           f2.ID           AS nf_id,
0031:           f2.F2_DOC       AS nf_doc,
0032:           f2.F2_SERIE     AS nf_serie,
0033:           f2.F2_EMISSAO   AS nf_emissao,
0034:           f2.F2_STATUS    AS nf_status,
0035:           f2.F2_TES       AS nf_tes,
0036:           f2.F2_CFOP      AS nf_cfop,
0037:           f2.F2_VALOR     AS nf_valor,
0038:           f2.F2_ICMS      AS nf_icms,
0039:           f2.F2_IPI       AS nf_ipi,
0040:           f2.F2_PIS       AS nf_pis,
0041:           f2.F2_COFINS    AS nf_cofins,
0042:           f2.F2_ICMS_ST   AS nf_icms_st,
0043:           f2.F2_DIFAL     AS nf_difal,
0044:           f2.F2_TOTAL_BRUTO AS nf_total_bruto,
0045:
0046:           s.C9_ITEM       AS sc9_item,
0047:           s.C9_PRODUTO    AS sc9_produto,
0048:           s.C9_QTD        AS sc9_qtd,
0049:           s.C9_STATUS     AS sc9_status,
0050:
0051:           e1.ID           AS ar_id,
```

```
0052:           e1.E1_NUM       AS ar_num,
0053:           e1.E1_SERIE     AS ar_serie,
0054:           e1.E1_VALOR     AS ar_valor,
0055:           e1.E1_STATUS    AS ar_status,
0056:           e1.E1_VENC      AS ar_venc
0057:         FROM dbo.SF2_NOTAS f2
0058:         LEFT JOIN dbo.SC9_LIBERACAO s
0059:           ON s.C9_PEDIDO_ID = f2.F2_PEDIDO_ID
0060:          AND (s.D_E_L_E_T_ IS NULL OR s.D_E_L_E_T_ <> '*')
0061:         LEFT JOIN dbo.SE1_AR e1
0062:           ON TRY_CAST(e1.E1_REF AS INT) = f2.ID
0063:          AND (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0064:         WHERE {where_sql}
0065:           AND (f2.D_E_L_E_T_ IS NULL OR f2.D_E_L_E_T_ <> '*')
0066:         ORDER BY f2.ID DESC, s.C9_ITEM ASC
0067:         """,
0068:         tuple(params),
0069:     )
0070:     return fetchall_dict(cur)
```

==========
ARQUIVO: app/infra/repositories/vendas/sc5_pedidos_enriq_repo.py
==========

```
0001: from datetime import date
0002: from app.infra.db import fetchall_dict, fetchone_dict
0003:
0004: def sc5_pedidos_enriq_listar_tx(
0005:     cur,
0006:     filial: str | None,
0007:     status: str | None,
0008:     origem: str | None,
0009:     de: date | None,
0010:     ate: date | None,
0011:     limit: int,
0012:     offset: int,
0013: ) -> list[dict]:
0014:     filtros = []
0015:     params: list = []
0016:
0017:     if filial:
0018:         filtros.append("p.C5_FILIAL = ?")
0019:         params.append(str(filial))
0020:
0021:     if status:
0022:         filtros.append("p.C5_STATUS = ?")
0023:         params.append(str(status))
0024:
0025:     if origem:
0026:         filtros.append("p.C5_ORIGEM = ?")
0027:         params.append(str(origem))
0028:
0029:     if de is not None:
0030:         filtros.append("CAST(p.C5_DATA AS DATE) >= ?")
0031:         params.append(de)
```

```
0032:
0033:     if ate is not None:
0034:         filtros.append("CAST(p.C5_DATA AS DATE) <= ?")
0035:         params.append(ate)
0036:
0037:     filtros.append("(p.D_E_L_E_T_ IS NULL OR p.D_E_L_E_T_ <> '*')")
0038:     where_sql = ("WHERE " + " AND ".join(filtros)) if filtros else ""
0039:
0040:     # Observações:
0041:     # - AggSC6 agrega itens por pedido
0042:     # - ARLast pega o último título por pedido (maior ID)
0043:     # - ARStat calcula status consolidado (BAIXADO > ABERTO > CANCELADO)
0044:     cur.execute(
0045:         f"""
0046:         WITH AggSC6 AS (
0047:           SELECT
0048:             C6_PEDIDO_ID,
0049:             SUM(CAST(C6_QTD AS INT)) AS qtd_itens,
0050:             SUM(CAST(COALESCE(C6_TOTAL, C6_VALOR, 0) AS FLOAT)) AS valor_itens
0051:           FROM dbo.SC6_ITENS
0052:           WHERE (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0053:           GROUP BY C6_PEDIDO_ID
0054:         ),
0055:         ARJoin AS (
0056:           SELECT
0057:             f2.F2_PEDIDO_ID AS pedido_id,
0058:             e1.*
0059:           FROM dbo.SE1_AR e1
0060:           INNER JOIN dbo.SF2_NOTAS f2 ON f2.ID = TRY_CAST(e1.E1_REF AS INT)
0061:           WHERE (e1.D_E_L_E_T_ IS NULL OR e1.D_E_L_E_T_ <> '*')
0062:         ),
0063:         ARLast AS (
0064:           SELECT a.*
0065:           FROM ARJoin a
0066:           JOIN (
0067:             SELECT pedido_id, MAX(ID) AS max_id
0068:             FROM ARJoin
0069:             GROUP BY pedido_id
0070:           ) x ON x.pedido_id = a.pedido_id AND x.max_id = a.ID
0071:         ),
0072:         ARStat AS (
0073:           SELECT
0074:             pedido_id,
0075:             CASE
0076:               WHEN SUM(CASE WHEN E1_STATUS='BAIXADO' THEN 1 ELSE 0 END) > 0 THEN 'BAIXADO'
0077:               WHEN SUM(CASE WHEN E1_STATUS='ABERTO'  THEN 1 ELSE 0 END) > 0 THEN 'ABERTO'
0078:               WHEN SUM(CASE WHEN E1_STATUS='CANCELADO' THEN 1 ELSE 0 END) > 0 THEN 'CANCELADO'

0079:               ELSE NULL
0080:             END AS ar_status
0081:           FROM ARJoin
0082:           GROUP BY pedido_id
0083:         )
0084:         SELECT
```

```
0085:           p.ID,
0086:           p.C5_DATA,
0087:           p.C5_FILIAL,
0088:           p.C5_STATUS,
0089:           p.C5_USUARIO,
0090:           p.C5_VALOR_TOTAL,
0091:           p.C5_ICMS,
0092:           p.C5_IPI,
0093:           p.C5_TOTAL_BRUTO,
0094:           p.C5_ORIGEM,
0095:           p.C5_DATA_CANCEL,
0096:           p.C5_USUARIO_CANCEL,
0097:
0098:           COALESCE(s.qtd_itens, 0) AS qtd_itens,
0099:           COALESCE(s.valor_itens, 0) AS valor_itens,
0100:
0101:           st.ar_status,
0102:           al.ID        AS ar_id,
0103:           al.E1_VALOR  AS ar_valor,
0104:           al.E1_VENC   AS ar_venc,
0105:           al.E1_SE5_ID AS ar_se5_id,
0106:           al.E1_STATUS AS ar_status_ultimo
0107:         FROM dbo.SC5_PEDIDOS p
0108:         LEFT JOIN AggSC6 s
0109:           ON s.C6_PEDIDO_ID = p.ID
0110:         LEFT JOIN ARStat st
0111:           ON st.pedido_id = p.ID
0112:         LEFT JOIN ARLast al
0113:           ON al.pedido_id = p.ID
0114:         {where_sql}
0115:         ORDER BY p.ID DESC
0116:         OFFSET ? ROWS FETCH NEXT ? ROWS ONLY
0117:         """,
0118:         tuple(params + [int(offset), int(limit)]),
0119:     )
0120:     return fetchall_dict(cur)
0121:
0122: def sc5_pedidos_enriq_contar_tx(
0123:     cur,
0124:     filial: str | None,
0125:     status: str | None,
0126:     origem: str | None,
0127:     de: date | None,
0128:     ate: date | None,
0129: ) -> int:
0130:     filtros = []
0131:     params: list = []
0132:
0133:     if filial:
0134:         filtros.append("C5_FILIAL = ?")
0135:         params.append(str(filial))
0136:
0137:     if status:
0138:         filtros.append("C5_STATUS = ?")
```

```
0139:         params.append(str(status))
0140:
0141:     if origem:
0142:         filtros.append("C5_ORIGEM = ?")
0143:         params.append(str(origem))
0144:
0145:     if de is not None:
0146:         filtros.append("CAST(C5_DATA AS DATE) >= ?")
0147:         params.append(de)
0148:
0149:     if ate is not None:
0150:         filtros.append("CAST(C5_DATA AS DATE) <= ?")
0151:         params.append(ate)
0152:
0153:     filtros.append("(D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')")
0154:
0155:     where_sql = ("WHERE " + " AND ".join(filtros)) if filtros else ""
0156:
0157:     cur.execute(
0158:         f"SELECT COUNT(1) AS total FROM dbo.SC5_PEDIDOS {where_sql}",
0159:         tuple(params),
0160:     )
0161:     row = fetchone_dict(cur) or {"total": 0}
0162:     return int(row["total"])
```

==========
ARQUIVO: app/infra/repositories/vendas/sc5_pedidos_query_repo.py
==========

```
0001: from datetime import date
0002: from app.infra.db import fetchall_dict, fetchone_dict
0003:
0004: def sc5_pedidos_listar_tx(
0005:     cur,
0006:     filial: str | None,
0007:     status: str | None,
0008:     origem: str | None,
0009:     de: date | None,
0010:     ate: date | None,
0011:     limit: int,
0012:     offset: int,
0013: ) -> list[dict]:
0014:     filtros = []
0015:     params: list = []
0016:
0017:     if filial:
0018:         filtros.append("C5_FILIAL = ?")
0019:         params.append(str(filial))
0020:
0021:     if status:
0022:         filtros.append("C5_STATUS = ?")
0023:         params.append(str(status))
0024:
0025:     if origem:
0026:         filtros.append("C5_ORIGEM = ?")
```

```
0027:         params.append(str(origem))
0028:
0029:     if de is not None:
0030:         filtros.append("CAST(C5_DATA AS DATE) >= ?")
0031:         params.append(de)
0032:
0033:     if ate is not None:
0034:         filtros.append("CAST(C5_DATA AS DATE) <= ?")
0035:         params.append(ate)
0036:
0037:     filtros.append("(D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')")
0038:     where_sql = ("WHERE " + " AND ".join(filtros)) if filtros else ""
0039:
0040:     cur.execute(
0041:         f"""
0042:         SELECT
0043:           ID,
0044:           C5_DATA,
0045:           C5_FILIAL,
0046:           C5_STATUS,
0047:           C5_USUARIO,
0048:           C5_VALOR_TOTAL,
0049:           C5_ICMS,
0050:           C5_IPI,
0051:           C5_TOTAL_BRUTO,
0052:           C5_ORIGEM,
0053:           C5_DATA_CANCEL,
0054:           C5_USUARIO_CANCEL
0055:         FROM dbo.SC5_PEDIDOS
0056:         {where_sql}
0057:         ORDER BY ID DESC
0058:         OFFSET ? ROWS FETCH NEXT ? ROWS ONLY
0059:         """,
0060:         tuple(params + [int(offset), int(limit)]),
0061:     )
0062:     return fetchall_dict(cur)
0063:
0064: def sc5_pedidos_contar_tx(
0065:     cur,
0066:     filial: str | None,
0067:     status: str | None,
0068:     origem: str | None,
0069:     de: date | None,
0070:     ate: date | None,
0071: ) -> int:
0072:     filtros = []
0073:     params: list = []
0074:
0075:     if filial:
0076:         filtros.append("C5_FILIAL = ?")
0077:         params.append(str(filial))
0078:
0079:     if status:
0080:         filtros.append("C5_STATUS = ?")
```

```
0081:         params.append(str(status))
0082:
0083:     if origem:
0084:         filtros.append("C5_ORIGEM = ?")
0085:         params.append(str(origem))
0086:
0087:     if de is not None:
0088:         filtros.append("CAST(C5_DATA AS DATE) >= ?")
0089:         params.append(de)
0090:
0091:     if ate is not None:
0092:         filtros.append("CAST(C5_DATA AS DATE) <= ?")
0093:         params.append(ate)
0094:
0095:     filtros.append("(D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')")
0096:     where_sql = ("WHERE " + " AND ".join(filtros)) if filtros else ""
0097:
0098:     cur.execute(
0099:         f"SELECT COUNT(1) AS total FROM dbo.SC5_PEDIDOS {where_sql}",
0100:         tuple(params),
0101:     )
0102:     row = fetchone_dict(cur) or {"total": 0}
0103:     return int(row["total"])
```

==========
ARQUIVO: app/infra/repositories/vendas/sc5_pedidos_repo.py
==========

```
0001: from app.infra.db import fetchone_dict
0002: from app.infra.repositories.sx6_repo import sx6_next_tx
0003: from app.core.exceptions import BusinessError
0004:
0005: def sc5_pedido_criar_simples_tx(
0006:     cur,
0007:     filial: str,
0008:     valor_total: float,
0009:     usuario: str,
0010:     status: str = "FATURADO",
0011: ) -> int:
0012:     """
0013:     Versão simples (legada): cria SC5 com campos básicos.
0014:     Útil se algum fluxo antigo não calcula impostos.
0015:     """
0016:     cur.execute(
0017:         """
0018:         INSERT INTO dbo.SC5_PEDIDOS (C5_DATA, C5_FILIAL, C5_VALOR_TOTAL, C5_USUARIO, C5_STATUS
)
0019:         VALUES (SYSDATETIME(), ?, ?, ?, ?);
0020:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0021:         """,
0022:         (filial, float(valor_total), usuario, status),
0023:     )
0024:     row = fetchone_dict(cur)
0025:     return int(row["id"])
0026:
```

```
0027: from app.infra.db import fetchone_dict
0028:
0029: def sc5_pedido_criar_tx(
0030:     cur,
0031:     filial: str,
0032:     valor_total: float,
0033:     usuario: str,
0034:     status: str = "ABERTO",
0035:     icms: float | None = None,
0036:     ipi: float | None = None,
0037:     total_bruto: float | None = None,
0038:     origem: str = "VENDA",   # <-- novo
0039: ) -> int:
0040:     origem = (origem or "VENDA").strip().upper()
0041:     if origem not in {"VENDA", "RESERVA"}:
0042:         origem = "VENDA"
0043:
0044:     c5_num = sx6_next_tx(cur, filial=filial, serie="1", tabela="SC5")
0045:     cur.execute(
0046:         """
0047:         INSERT INTO dbo.SC5_PEDIDOS
0048:           (C5_DATA, C5_FILIAL, C5_NUM, C5_VALOR_TOTAL, C5_USUARIO, C5_STATUS,
0049:            C5_ICMS, C5_IPI, C5_TOTAL_BRUTO, C5_ORIGEM)
0050:         VALUES (SYSDATETIME(), ?, ?, ?, ?, ?, ?, ?, ?, ?);
0051:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0052:         """,
0053:         (filial, str(c5_num).zfill(9), float(valor_total), usuario, status, icms, ipi, total_b
ruto, origem),
0054:     )
0055:     row = fetchone_dict(cur)
0056:     return int(row["id"])
0057:
0058: from app.infra.db import fetchone_dict
0059:
0060: def sc5_pedido_get_tx(cur, pedido_id: int) -> dict | None:
0061:     cur.execute(
0062:         """
0063:         SELECT TOP 1
0064:           ID,
0065:           C5_DATA,
0066:           C5_FILIAL,
0067:           C5_STATUS,
0068:           C5_USUARIO,
0069:           C5_VALOR_TOTAL,
0070:           C5_ICMS,
0071:           C5_IPI,
0072:           C5_TOTAL_BRUTO,
0073:           C5_ORIGEM,
0074:           C5_DATA_CANCEL,
0075:           C5_USUARIO_CANCEL
0076:         FROM dbo.SC5_PEDIDOS
0077:         WHERE ID=?
0078:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0079:         """,
```

```
0080:         (int(pedido_id),),
0081:     )
0082:     return fetchone_dict(cur)
0083:
0084: def sc5_pedido_atualizar_valor_total_tx(cur, pedido_id: int, valor_total: float) -> None:
0085:     cur.execute(
0086:         """
0087:         UPDATE dbo.SC5_PEDIDOS
0088:         SET C5_VALOR_TOTAL=?
0089:         WHERE ID=?
0090:         """,
0091:         (float(valor_total), int(pedido_id)),
0092:     )
0093:
0094: def sc5_pedido_status_tx(cur, pedido_id: int, status: str, usuario: str | None = None, motivo:
 str | None = None) -> None:
0095:     status_norm = str(status).strip().upper()
0096:     if status_norm == "CANCELADO":
0097:         cur.execute(
0098:             """
0099:             UPDATE dbo.SC5_PEDIDOS
0100:             SET C5_STATUS='CANCELADO',
0101:                 C5_MOTIVO_CANCEL=?,
0102:                 C5_DATA_CANCEL=SYSDATETIME(),
0103:                 C5_USUARIO_CANCEL=?,
0104:                 D_E_L_E_T_='*',
0105:                 R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0106:             WHERE ID=?
0107:             """,
0108:             (motivo, usuario, int(pedido_id)),
0109:         )
0110:         return
0111:
0112:     cur.execute(
0113:         "UPDATE dbo.SC5_PEDIDOS SET C5_STATUS=? WHERE ID=?",
0114:         (status_norm, int(pedido_id)),
0115:     )
0116:
0117: def sc5_pedido_atualizar_totais_tx(
0118:     cur,
0119:     pedido_id: int,
0120:     valor_total: float,
0121:     icms: float,
0122:     ipi: float,
0123:     total_bruto: float,
0124:     status: str = "FATURADO",
0125: ) -> None:
0126:     cur.execute(
0127:         """
0128:         UPDATE dbo.SC5_PEDIDOS
0129:         SET C5_VALOR_TOTAL=?
0130:         WHERE ID=?
0131:         """,
0132:         (float(valor_total), int(pedido_id)),
```

```
0133:     )
0134:
0135: def sc5_pedido_marcar_faturado_tx(cur, pedido_id: int, icms: float, ipi: float, total_bruto: f
loat) -> None:
0136:     cur.execute("""
0137:         UPDATE dbo.SC5_PEDIDOS
0138:         SET C5_STATUS='FATURADO',
0139:             C5_ICMS=?,
0140:             C5_IPI=?,
0141:             C5_TOTAL_BRUTO=?
0142:         WHERE ID=? AND C5_STATUS='ABERTO'
0143:     """, (float(icms), float(ipi), float(total_bruto), int(pedido_id)))
0144:     if cur.rowcount == 0:
0145:         # ou não existe, ou não está ABERTO
0146:         pass
0147:
0148: def sc5_pedido_cancelar_faturado_tx(cur, pedido_id: int, usuario: str, motivo: str | None) ->
None:
0149:     cur.execute("""
0150:         UPDATE dbo.SC5_PEDIDOS
0151:         SET C5_STATUS='CANCELADO',
0152:             C5_MOTIVO_CANCEL=?,
0153:             C5_DATA_CANCEL=SYSDATETIME(),
0154:             C5_USUARIO_CANCEL=?,
0155:             D_E_L_E_T_='*',
0156:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0157:         WHERE ID=? AND C5_STATUS='FATURADO'
0158:     """, (motivo, usuario, int(pedido_id)))
0159:     if cur.rowcount == 0:
0160:         raise BusinessError("Pedido não está FATURADO (não é possível estornar)")
```

==========
ARQUIVO: app/infra/repositories/vendas/sc6_itens_query_repo.py
==========

```
0001: from app.infra.db import fetchall_dict
0002:
0003: def sc6_itens_do_pedido_tx(cur, pedido_id: int) -> list[dict]:
0004:     cur.execute("SELECT CASE WHEN COL_LENGTH('dbo.SC6_ITENS', 'C6_CMV_UNIT') IS NULL THEN 0 EL
SE 1 END")
0005:     row = cur.fetchone()
0006:     cmv_select = "C6_CMV_UNIT" if row and int(row[0]) == 1 else "CAST(NULL AS DECIMAL(18,2)) A
S C6_CMV_UNIT"
0007:     cur.execute(
0008:         f"""
0009:         SELECT ID, C6_PEDIDO_ID, C6_ITEM, C6_PRODUTO, C6_QTD, C6_VALOR, C6_PRECO_UNIT, C6_TOTA
L, C6_ATIVO, {cmv_select}
0010:         FROM dbo.SC6_ITENS
0011:         WHERE C6_PEDIDO_ID=?
0012:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0013:         ORDER BY ID
0014:         """,
0015:         (int(pedido_id),),
0016:     )
0017:     return fetchall_dict(cur)
```

```
0001: from app.infra.db import fetchone_dict, fetchall_dict
0002:
0003:
0004: def _sc6_has_cmv_unit(cur) -> bool:
0005:     cur.execute("SELECT CASE WHEN COL_LENGTH('dbo.SC6_ITENS', 'C6_CMV_UNIT') IS NULL THEN 0 EL
SE 1 END")
0006:     row = cur.fetchone()
0007:     return bool(row and int(row[0]) == 1)
0008:
0009:
0010: def _sc6_ensure_cmv_unit_column(cur) -> None:
0011:     if _sc6_has_cmv_unit(cur):
0012:         return
0013:     cur.execute("ALTER TABLE dbo.SC6_ITENS ADD C6_CMV_UNIT DECIMAL(18,2) NULL")
0014:
0015: def sc6_item_get_tx(cur, item_id: int) -> dict | None:
0016:     cur.execute("""
0017:         SELECT *
0018:         FROM dbo.SC6_ITENS
0019:         WHERE ID=?
0020:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0021:     """, (int(item_id),))
0022:     return fetchone_dict(cur)
0023:
0024: def sc6_itens_do_pedido_tx(cur, pedido_id: int) -> list[dict]:
0025:     cmv_select = "C6_CMV_UNIT" if _sc6_has_cmv_unit(cur) else "CAST(NULL AS DECIMAL(18,2)) AS
C6_CMV_UNIT"
0026:     cur.execute(f"""
0027:         SELECT ID, C6_PEDIDO_ID, C6_ITEM, C6_PRODUTO, C6_QTD,
0028:                C6_PRECO_UNIT, C6_TOTAL, C6_VALOR, C6_ATIVO, {cmv_select}
0029:         FROM dbo.SC6_ITENS
0030:         WHERE C6_PEDIDO_ID=? AND C6_ATIVO=1
0031:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0032:         ORDER BY ID ASC
0033:     """, (int(pedido_id),))
0034:     return fetchall_dict(cur)
0035:
0036: def sc6_item_atualizar_tx(
0037:     cur,
0038:     item_id: int,
0039:     qtd: int,
0040:     preco_unit: float,
0041:     total: float,
0042:     usuario: str,
0043:     cmv_unit: float | None = None,
0044: ) -> None:
0045:     _sc6_ensure_cmv_unit_column(cur)
0046:     cur.execute("""
0047:         UPDATE dbo.SC6_ITENS
0048:         SET C6_QTD=?,
```

```
0049:             C6_PRECO_UNIT=?,
0050:             C6_TOTAL=?,
0051:             C6_VALOR=?,
0052:             C6_CMV_UNIT=?,
0053:             C6_DATA_ALT=SYSDATETIME(),
0054:             C6_USUARIO_ALT=?
0055:         WHERE ID=? AND C6_ATIVO=1
0056:     """, (int(qtd), float(preco_unit), float(total), float(total), (float(cmv_unit) if cmv_uni
t is not None else None), usuario, int(item_id)))
0057:
0058: def sc6_item_excluir_logico_tx(cur, item_id: int, usuario: str) -> None:
0059:     cur.execute("""
0060:         UPDATE dbo.SC6_ITENS
0061:         SET C6_ATIVO=0,
0062:             C6_DATA_DEL=SYSDATETIME(),
0063:             C6_USUARIO_DEL=?,
0064:             D_E_L_E_T_='*',
0065:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0066:         WHERE ID=? AND C6_ATIVO=1
0067:     """, (usuario, int(item_id)))
0068:
0069:
0070: def sc6_itens_cancelar_por_pedido_tx(cur, pedido_id: int, usuario: str) -> int:
0071:     cur.execute(
0072:         """
0073:         UPDATE dbo.SC6_ITENS
0074:         SET C6_ATIVO=0,
0075:             C6_DATA_DEL=SYSDATETIME(),
0076:             C6_USUARIO_DEL=?,
0077:             D_E_L_E_T_='*',
0078:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0079:         WHERE C6_PEDIDO_ID=? AND C6_ATIVO=1
0080:         """,
0081:         (usuario, int(pedido_id)),
0082:     )
0083:     return int(cur.rowcount or 0)
```

==========
ARQUIVO: app/infra/repositories/vendas/sc6_pedidos_repo.py
==========

```
0001: from app.infra.db import fetchone_dict
0002:
0003:
0004: def _sc6_has_cmv_unit(cur) -> bool:
0005:     cur.execute("SELECT CASE WHEN COL_LENGTH('dbo.SC6_ITENS', 'C6_CMV_UNIT') IS NULL THEN 0 EL
SE 1 END")
0006:     row = cur.fetchone()
0007:     return bool(row and int(row[0]) == 1)
0008:
0009:
0010: def _sc6_ensure_cmv_unit_column(cur) -> None:
0011:     if _sc6_has_cmv_unit(cur):
0012:         return
0013:     cur.execute("ALTER TABLE dbo.SC6_ITENS ADD C6_CMV_UNIT DECIMAL(18,2) NULL")
```

```
0014:
0015:
0016: def sc6_item_criar_tx(
0017:     cur,
0018:     pedido_id: int,
0019:     produto: str,
0020:     qtd: int,
0021:     total: float,
0022:     preco_unit: float | None = None,
0023:     cmv_unit: float | None = None,
0024: ) -> dict:
0025:     """
0026:     Insere item SC6. Mantém compatibilidade com C6_VALOR (total),
0027:     e também grava C6_PRECO_UNIT e C6_TOTAL (novo padrão).
0028:     """
0029:     if preco_unit is None:
0030:         preco_unit = (float(total) / int(qtd)) if int(qtd) > 0 else float(total)
0031:
0032:     cur.execute(
0033:         """
0034:         SELECT COALESCE(MAX(C6_ITEM), 0) + 1 AS prox
0035:         FROM dbo.SC6_ITENS
0036:         WHERE C6_PEDIDO_ID=?
0037:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0038:         """,
0039:         (int(pedido_id),),
0040:     )
0041:     prox = fetchone_dict(cur) or {"prox": 1}
0042:     c6_item = int(prox["prox"])
0043:
0044:     _sc6_ensure_cmv_unit_column(cur)
0045:
0046:     cur.execute(
0047:         """
0048:         INSERT INTO dbo.SC6_ITENS
0049:           (C6_PEDIDO_ID, C6_ITEM, C6_PRODUTO, C6_QTD, C6_VALOR, C6_PRECO_UNIT, C6_TOTAL, C6_CM
V_UNIT)
0050:         VALUES (?, ?, ?, ?, ?, ?, ?, ?);
0051:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0052:         """,
0053:         (
0054:             int(pedido_id),
0055:             c6_item,
0056:             str(produto),
0057:             int(qtd),
0058:             float(total),
0059:             float(preco_unit),
0060:             float(total),
0061:             (float(cmv_unit) if cmv_unit is not None else None),
0062:         ),
0063:     )
0064:     row = cur.fetchone()
0065:     item_id = int(row[0]) if row else 0
0066:     return {"id": item_id, "item": c6_item}
```

==========
ARQUIVO: app/infra/repositories/vendas/sc6_totais_repo.py
==========

```
0001: from app.infra.db import fetchone_dict
0002:
0003: def sc6_totais_do_pedido_tx(cur, pedido_id: int) -> dict:
0004:     """
0005:     Retorna totais calculados a partir dos itens SC6:
0006:       - qtd_itens: soma de C6_QTD
0007:       - valor_itens: soma de C6_TOTAL (ou C6_VALOR)
0008:     """
0009:     cur.execute(
0010:         """
0011:         SELECT
0012:           COALESCE(SUM(CAST(C6_QTD AS INT)), 0) AS qtd_itens,
0013:           COALESCE(SUM(CAST(COALESCE(C6_TOTAL, C6_VALOR, 0) AS FLOAT)), 0) AS valor_itens
0014:         FROM dbo.SC6_ITENS
0015:         WHERE C6_PEDIDO_ID=?
0016:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0017:         """,
0018:         (int(pedido_id),),
0019:     )
0020:     row = fetchone_dict(cur) or {"qtd_itens": 0, "valor_itens": 0}
0021:     return {"qtd_itens": int(row["qtd_itens"]), "valor_itens": float(row["valor_itens"])}
```

==========
ARQUIVO: app/infra/repositories/vendas/sc9_liberacao_query_repo.py
==========

```
0001: from __future__ import annotations
0002:
0003: from app.infra.db import fetchall_dict
0004:
0005:
0006: def sc9_resumo_por_pedido_tx(cur, pedido_id: int) -> list[dict]:
0007:     cur.execute(
0008:         """
0009:         SELECT
0010:           i.ID AS item_id,
0011:           i.C6_PRODUTO AS produto,
0012:           i.C6_QTD AS qtd_pedido,
0013:           COALESCE(l.qtd_liberada, 0) AS qtd_liberada,
0014:           (i.C6_QTD - COALESCE(l.qtd_liberada, 0)) AS qtd_pendente
0015:         FROM dbo.SC6_ITENS i
0016:         LEFT JOIN (
0017:           SELECT C9_PEDIDO_ID, C9_PRODUTO, C9_ITEM, SUM(C9_QTD) AS qtd_liberada
0018:           FROM dbo.SC9_LIBERACAO
0019:           WHERE C9_PEDIDO_ID = ?
0020:             AND C9_STATUS = 'LIBERADO'
0021:             AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0022:           GROUP BY C9_PEDIDO_ID, C9_PRODUTO, C9_ITEM
0023:         ) l ON l.C9_PEDIDO_ID = i.C6_PEDIDO_ID AND l.C9_PRODUTO = i.C6_PRODUTO AND l.C9_ITEM =
 i.ID
0024:         WHERE i.C6_PEDIDO_ID = ?
```

```
0025:           AND i.C6_ATIVO = 1
0026:           AND (i.D_E_L_E_T_ IS NULL OR i.D_E_L_E_T_ <> '*')
0027:         ORDER BY i.C6_PRODUTO
0028:         """,
0029:         (int(pedido_id), int(pedido_id)),
0030:     )
0031:     return fetchall_dict(cur)
```

==========

ARQUIVO: app/infra/repositories/vendas/sc9_liberacao_repo.py

==========

```
0001: from __future__ import annotations
0002:
0003: from app.infra.db import fetchall_dict, fetchone_dict
0004:
0005:
0006: def sc9_liberar_tx(
0007:     cur,
0008:     pedido_id: int,
0009:     filial: str,
0010:     produto: str,
0011:     qtd: int,
0012:     usuario: str,
0013:     item: int | None = None,
0014: ) -> int:
0015:     cur.execute(
0016:         """
0017:         INSERT INTO dbo.SC9_LIBERACAO
0018:           (C9_PEDIDO_ID, C9_FILIAL, C9_PRODUTO, C9_QTD, C9_STATUS, C9_USUARIO, C9_DATA, C9_ITE
M)
0019:         VALUES (?, ?, ?, ?, 'LIBERADO', ?, SYSDATETIME(), ?);
0020:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0021:         """,
0022:         (int(pedido_id), filial, produto, int(qtd), usuario, int(item) if item is not None els
e None),
0023:     )
0024:     row = fetchone_dict(cur)
0025:     return int(row["id"])
0026:
0027:
0028: def sc9_qtd_liberada_por_pedido_tx(cur, pedido_id: int) -> dict:
0029:     cur.execute(
0030:         """
0031:         SELECT
0032:           C9_ITEM,
0033:           COALESCE(SUM(C9_QTD), 0) AS qtd_liberada
0034:         FROM dbo.SC9_LIBERACAO
0035:         WHERE C9_PEDIDO_ID = ?
0036:           AND C9_STATUS = 'LIBERADO'
0037:           AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')
0038:         GROUP BY C9_ITEM
0039:         """,
0040:         (int(pedido_id),),
0041:     )
```

```
0042:     rows = fetchall_dict(cur)
0043:     return {int(r["C9_ITEM"]): int(r["qtd_liberada"]) for r in rows if r["C9_ITEM"] is not Non
e}
0044:
0045:
0046: def sc9_cancelar_por_pedido_tx(cur, pedido_id: int, usuario: str | None = None) -> int:
0047:     cur.execute(
0048:         """
0049:         UPDATE dbo.SC9_LIBERACAO
0050:         SET C9_STATUS='CANCELADO',
0051:             C9_USUARIO_CANCEL=?,
0052:             C9_DATA_CANCEL=SYSDATETIME(),
0053:             D_E_L_E_T_='*',
0054:             R_E_C_D_E_L_=COALESCE(R_E_C_D_E_L_, R_E_C_N_O_)
0055:         WHERE C9_PEDIDO_ID=? AND C9_STATUS='LIBERADO'
0056:         """,
0057:         (usuario, int(pedido_id)),
0058:     )
0059:     return int(cur.rowcount or 0)
```

==========
ARQUIVO: app/main.py
==========

```
0001: from uuid import uuid4
0002:
0003: from fastapi import FastAPI, HTTPException, Request
0004: from fastapi.exceptions import RequestValidationError
0005: from fastapi.middleware.cors import CORSMiddleware
0006: from fastapi.responses import JSONResponse
0007:
0008: from app.api.schemas.errors import build_error_response
0009: from app.api.routers import (
0010:     ap_lancamento,
0011:     alertas,
0012:     auditoria,
0013:     auth,
0014:     clientes,
0015:     estoque,
0016:     financeiro,
0017:     fornecedores,
0018:     parametros,
0019:     parceiros,
0020:     precos,
0021:     produtos,
0022:     relatorios,
0023:     reservas,
0024:     usuarios,
0025: )
0026: from app.api.routers.vendas import router as vendas_router
0027: from app.core import settings
0028: from app.infra.db import set_current_request
0029: from app.core.exceptions import (
0030:     AuthzError,
0031:     BusinessError as CoreBusinessError,
```

```
0032:     ConflictError,
0033: )
0034: from app.domain.errors import AuthError, BusinessError as DomainBusinessError
0035:
0036: app = FastAPI(title=settings.APP_TITLE)
0037:
0038: app.add_middleware(
0039:     CORSMiddleware,
0040:     allow_origins=settings.CORS_ALLOW_ORIGINS,
0041:     allow_credentials=True,
0042:     allow_methods=["*"],
0043:     allow_headers=["*"],
0044: )
0045:
0046:
0047: @app.middleware("http")
0048: async def request_context_middleware(request: Request, call_next):
0049:     req_id = request.headers.get("X-Request-Id") or str(uuid4())
0050:     request.state.request_id = req_id
0051:     set_current_request(req_id, request.url.path)
0052:     response = await call_next(request)
0053:     response.headers["X-Request-Id"] = req_id
0054:     return response
0055:
0056:
0057: def _request_id(request: Request) -> str | None:
0058:     return getattr(request.state, "request_id", None)
0059:
0060:
0061: @app.exception_handler(CoreBusinessError)
0062: @app.exception_handler(DomainBusinessError)
0063: def business_error_handler(request: Request, exc: Exception):
0064:     return JSONResponse(
0065:         status_code=400,
0066:         content=build_error_response(
0067:             detail=str(exc),
0068:             code=getattr(exc, "code", "BUSINESS_ERROR"),
0069:             error_type="business",
0070:             request_id=_request_id(request),
0071:         ),
0072:     )
0073:
0074:
0075: @app.exception_handler(AuthError)
0076: def auth_error_handler(request: Request, exc: AuthError):
0077:     return JSONResponse(
0078:         status_code=401,
0079:         content=build_error_response(
0080:             detail=exc.message,
0081:             code=exc.code,
0082:             error_type="auth",
0083:             request_id=_request_id(request),
0084:         ),
0085:     )
```

```
0086:
0087:
0088: @app.exception_handler(AuthzError)
0089: def authz_error_handler(request: Request, exc: AuthzError):
0090:     return JSONResponse(
0091:         status_code=403,
0092:         content=build_error_response(
0093:             detail=exc.message,
0094:             code=exc.code,
0095:             error_type="authorization",
0096:             request_id=_request_id(request),
0097:         ),
0098:     )
0099:
0100:
0101: @app.exception_handler(ConflictError)
0102: def conflict_handler(request: Request, exc: ConflictError):
0103:     return JSONResponse(
0104:         status_code=409,
0105:         content=build_error_response(
0106:             detail=exc.message,
0107:             code=exc.code,
0108:             error_type="conflict",
0109:             request_id=_request_id(request),
0110:         ),
0111:     )
0112:
0113:
0114: @app.exception_handler(HTTPException)
0115: def http_exception_handler(request: Request, exc: HTTPException):
0116:     detail = exc.detail if isinstance(exc.detail, str) else "Erro HTTP"
0117:     return JSONResponse(
0118:         status_code=exc.status_code,
0119:         content=build_error_response(
0120:             detail=detail,
0121:             code=f"HTTP_{exc.status_code}",
0122:             error_type="http",
0123:             request_id=_request_id(request),
0124:         ),
0125:     )
0126:
0127:
0128: @app.exception_handler(RequestValidationError)
0129: def validation_handler(request: Request, exc: RequestValidationError):
0130:     first_error = exc.errors()[0] if exc.errors() else {}
0131:     loc = ".".join(str(part) for part in first_error.get("loc", []))
0132:     msg = first_error.get("msg", "Payload inválido")
0133:     detail = f"Payload inválido: {loc} - {msg}" if loc else f"Payload inválido: {msg}"
0134:     return JSONResponse(
0135:         status_code=422,
0136:         content=build_error_response(
0137:             detail=detail,
0138:             code="VALIDATION_ERROR",
0139:             error_type="validation",
```

```
0140:             request_id=_request_id(request),
0141:         ),
0142:     )
0143:
0144:
0145: @app.exception_handler(Exception)
0146: def generic_handler(request: Request, __: Exception):
0147:     return JSONResponse(
0148:         status_code=500,
0149:         content=build_error_response(
0150:             detail="Erro interno",
0151:             code="INTERNAL_ERROR",
0152:             error_type="internal",
0153:             request_id=_request_id(request),
0154:         ),
0155:     )
0156:
0157:
0158: app.include_router(auth.router)
0159: app.include_router(usuarios.router)
0160: app.include_router(alertas.router)
0161: app.include_router(produtos.router)
0162: app.include_router(estoque.router)
0163: app.include_router(reservas.router)
0164: app.include_router(financeiro.router)
0165: app.include_router(auditoria.router)
0166: app.include_router(parceiros.router)
0167: app.include_router(precos.router)
0168: app.include_router(relatorios.router)
0169: app.include_router(clientes.router)
0170: app.include_router(fornecedores.router)
0171: app.include_router(ap_lancamento.router)
0172: app.include_router(parametros.router)
0173: app.include_router(vendas_router)

================================================================================================
ARQUIVO: app/security/__init__.py
================================================================================================
0001:

================================================================================================
ARQUIVO: app/security/passwords.py
================================================================================================
0001: from passlib.context import CryptContext
0002:
0003: _pwd_ctx = CryptContext(
0004:     schemes=["pbkdf2_sha256"],
0005:     deprecated="auto"
0006: )
0007:
0008: def hash_password(password: str) -> str:
0009:     return _pwd_ctx.hash(password)
0010:
0011: def verify_password(password: str, password_hash: str) -> bool:
```

```
0012:     return _pwd_ctx.verify(password, password_hash)
```

==========
ARQUIVO: app/spa/frontend/app.tsx
==========

```
0001:
```

==========
ARQUIVO: app/spa/frontend/main.tsx
==========

```
0001:
```

==========
ARQUIVO: app/use_cases/__init__.py
==========

```
0001:
```

==========
ARQUIVO: app/use_cases/alertas_uc.py
==========

```
0001: from __future__ import annotations
0002:
0003: from datetime import date, datetime
0004:
0005: from app.infra.repositories.alertas_repo import (
0006:     contar_ap_aberto_tx,
0007:     contar_ap_vence_hoje_tx,
0008:     contar_ap_vencendo_ate_tx,
0009:     contar_ap_vencido_tx,
0010:     contar_ar_aberto_tx,
0011:     contar_ar_vence_hoje_tx,
0012:     contar_ar_vencendo_ate_tx,
0013:     contar_ar_vencido_tx,
0014:     contar_contas_caixa_ativas_tx,
0015:     contar_pedidos_abertos_com_itens_tx,
0016:     contar_pedidos_abertos_sem_itens_tx,
0017:     contar_reservas_abertas_tx,
0018: )
0019: from app.infra.repositories.app_setup_repo import app_setup_status_tx
0020: from app.infra.repositories.fechamento_repo import fechamento_periodo_aberto_tx
0021:
0022:
0023: def _append_alert(
0024:     items: list[dict],
0025:     *,
0026:     code: str,
0027:     severity: str,
0028:     title: str,
0029:     text: str,
0030:     route: str | None = None,
0031:     count: int | None = None,
0032:     action_label: str | None = None,
0033:     business_context: str | None = None,
0034: ) -> None:
```

```
0035:     items.append(
0036:         {
0037:             "code": code,
0038:             "severity": severity,
0039:             "title": title,
0040:             "text": text,
0041:             "route": route,
0042:             "count": count,
0043:             "action_label": action_label,
0044:             "business_context": business_context,
0045:         }
0046:     )
0047:
0048:
0049: def listar_alertas_uc_tx(cur, login: str, perfil: str, filial: str) -> dict:
0050:     hoje = date.today()
0051:     horizonte = date.fromordinal(hoje.toordinal() + 3)
0052:     items: list[dict] = []
0053:
0054:     ar_aberto = contar_ar_aberto_tx(cur, filial)
0055:     ar_vencido = contar_ar_vencido_tx(cur, filial, hoje)
0056:     ar_vence_hoje = contar_ar_vence_hoje_tx(cur, filial, hoje)
0057:     ar_vencendo = contar_ar_vencendo_ate_tx(cur, filial, date.fromordinal(hoje.toordinal() + 1
), horizonte)
0058:     ap_aberto = contar_ap_aberto_tx(cur, filial)
0059:     ap_vencido = contar_ap_vencido_tx(cur, filial, hoje)
0060:     ap_vence_hoje = contar_ap_vence_hoje_tx(cur, filial, hoje)
0061:     ap_vencendo = contar_ap_vencendo_ate_tx(cur, filial, date.fromordinal(hoje.toordinal() + 1
), horizonte)
0062:     reservas_abertas = contar_reservas_abertas_tx(cur, filial)
0063:     pedidos_abertos_sem_itens = contar_pedidos_abertos_sem_itens_tx(cur, filial)
0064:     pedidos_abertos_com_itens = contar_pedidos_abertos_com_itens_tx(cur, filial)
0065:     contas_ativas = contar_contas_caixa_ativas_tx(cur, filial)
0066:     periodo_aberto = fechamento_periodo_aberto_tx(cur, filial, hoje)
0067:
0068:     if ar_vencido > 0:
0069:         _append_alert(
0070:             items,
0071:             code="AR_OVERDUE",
0072:             severity="critical",
0073:             title="Recebiveis vencidos",
0074:             text=f"{ar_vencido} titulo(s) de contas a receber estao vencidos e seguem em abert
o na filial {filial}.",
0075:             route="/financeiro/ar",
0076:             count=ar_vencido,
0077:             action_label="Abrir recebiveis",
0078:             business_context="financeiro.ar.overdue",
0079:         )
0080:
0081:     if ap_vencido > 0:
0082:         _append_alert(
0083:             items,
0084:             code="AP_OVERDUE",
0085:             severity="critical",
```

```
0086:             title="Pagamentos vencidos",
0087:             text=f"{ap_vencido} titulo(s) de contas a pagar estao vencidos e seguem em aberto
na filial {filial}.",
0088:             route="/financeiro/ap",
0089:             count=ap_vencido,
0090:             action_label="Abrir pagamentos",
0091:             business_context="financeiro.ap.overdue",
0092:         )
0093:
0094:     if perfil in {"admin", "operador"} and pedidos_abertos_sem_itens > 0:
0095:         _append_alert(
0096:             items,
0097:             code="PEDIDOS_NO_ITEMS",
0098:             severity="warning",
0099:             title="Pedidos abertos sem itens",
0100:             text=f"{pedidos_abertos_sem_itens} pedido(s) seguem abertos sem item lancado.",
0101:             route="/vendas/pedidos",
0102:             count=pedidos_abertos_sem_itens,
0103:             action_label="Revisar pedidos",
0104:             business_context="vendas.pedidos.sem_itens",
0105:         )
0106:
0107:     if perfil in {"admin", "operador"} and pedidos_abertos_com_itens > 0:
0108:         _append_alert(
0109:             items,
0110:             code="PEDIDOS_OPEN",
0111:             severity="info",
0112:             title="Pedidos aguardando faturamento",
0113:             text=f"{pedidos_abertos_com_itens} pedido(s) abertos possuem itens e ainda aguarda
m faturamento.",
0114:             route="/vendas/pedidos",
0115:             count=pedidos_abertos_com_itens,
0116:             action_label="Ir para faturamento",
0117:             business_context="vendas.pedidos.abertos",
0118:         )
0119:
0120:     if reservas_abertas > 0:
0121:         _append_alert(
0122:             items,
0123:             code="RESERVAS_OPEN",
0124:             severity="warning",
0125:             title="Reservas pendentes",
0126:             text=f"{reservas_abertas} reserva(s) ainda estao abertas aguardando confirmacao ou
 cancelamento.",
0127:             route="/reservas",
0128:             count=reservas_abertas,
0129:             action_label="Abrir reservas",
0130:             business_context="vendas.reservas.abertas",
0131:         )
0132:
0133:     if ar_vence_hoje > 0:
0134:         _append_alert(
0135:             items,
0136:             code="AR_DUE_TODAY",
```

```
0137:             severity="warning",
0138:             title="Recebiveis vencem hoje",
0139:             text=f"{ar_vence_hoje} titulo(s) de AR vencem hoje na filial {filial}.",
0140:             route="/financeiro/ar",
0141:             count=ar_vence_hoje,
0142:             action_label="Priorizar recebimento",
0143:             business_context="financeiro.ar.due_today",
0144:         )
0145:
0146:     if ap_vence_hoje > 0:
0147:         _append_alert(
0148:             items,
0149:             code="AP_DUE_TODAY",
0150:             severity="warning",
0151:             title="Pagamentos vencem hoje",
0152:             text=f"{ap_vence_hoje} titulo(s) de AP vencem hoje na filial {filial}.",
0153:             route="/financeiro/ap",
0154:             count=ap_vence_hoje,
0155:             action_label="Priorizar pagamento",
0156:             business_context="financeiro.ap.due_today",
0157:         )
0158:
0159:     if ar_vencendo > 0:
0160:         _append_alert(
0161:             items,
0162:             code="AR_DUE_SOON",
0163:             severity="info",
0164:             title="Recebiveis a vencer",
0165:             text=f"{ar_vencendo} titulo(s) de AR vencem nos proximos 3 dias.",
0166:             route="/financeiro/ar",
0167:             count=ar_vencendo,
0168:             action_label="Planejar cobranca",
0169:             business_context="financeiro.ar.due_soon",
0170:         )
0171:
0172:     if ap_vencendo > 0:
0173:         _append_alert(
0174:             items,
0175:             code="AP_DUE_SOON",
0176:             severity="info",
0177:             title="Pagamentos a vencer",
0178:             text=f"{ap_vencendo} titulo(s) de AP vencem nos proximos 3 dias.",
0179:             route="/financeiro/ap",
0180:             count=ap_vencendo,
0181:             action_label="Planejar pagamento",
0182:             business_context="financeiro.ap.due_soon",
0183:         )
0184:
0185:     if not periodo_aberto:
0186:         _append_alert(
0187:             items,
0188:             code="PERIODO_FECHADO",
0189:             severity="warning",
0190:             title="Periodo fechado",
```

```
0191:             text=f"O periodo corrente da filial {filial} esta fechado para lancamentos operaci
onais.",
0192:             route="/dashboard",
0193:             action_label="Revisar fechamento",
0194:             business_context="parametros.fechamento.closed",
0195:         )
0196:
0197:     if contas_ativas == 0:
0198:         _append_alert(
0199:             items,
0200:             code="CAIXA_SEM_CONTA",
0201:             severity="warning",
0202:             title="Caixa sem conta ativa",
0203:             text=f"Nao existe conta de caixa ativa cadastrada para a filial {filial}.",
0204:             route="/financeiro/caixa",
0205:             action_label="Configurar caixa",
0206:             business_context="financeiro.caixa.sem_conta",
0207:         )
0208:
0209:     if ar_aberto == 0 and ap_aberto == 0 and reservas_abertas == 0:
0210:         _append_alert(
0211:             items,
0212:             code="OPERACAO_ESTAVEL",
0213:             severity="info",
0214:             title="Operacao sem pendencias abertas",
0215:             text=f"Nao ha AR/AP em aberto nem reservas pendentes na filial {filial}.",
0216:             route="/dashboard",
0217:             action_label="Ver painel",
0218:             business_context="operacao.estavel",
0219:         )
0220:
0221:     if perfil == "admin":
0222:         setup = app_setup_status_tx(cur)
0223:         if not setup["setup_completed"]:
0224:             _append_alert(
0225:                 items,
0226:                 code="SETUP_PENDING",
0227:                 severity="warning",
0228:                 title="Setup inicial pendente",
0229:                 text="O estado persistido de setup ainda nao foi marcado como concluido.",
0230:                 route="/usuarios",
0231:                 action_label="Revisar setup",
0232:                 business_context="administracao.setup.pending",
0233:             )
0234:
0235:     severity_order = {"critical": 0, "warning": 1, "info": 2}
0236:     profile_order = {
0237:         "admin": {
0238:             "SETUP_PENDING": 0,
0239:             "PERIODO_FECHADO": 1,
0240:             "AR_OVERDUE": 2,
0241:             "AP_OVERDUE": 3,
0242:         },
0243:         "operador": {
```

```
0244:             "PEDIDOS_NO_ITEMS": 0,
0245:             "PEDIDOS_OPEN": 1,
0246:             "RESERVAS_OPEN": 2,
0247:             "AR_DUE_TODAY": 3,
0248:             "AP_DUE_TODAY": 4,
0249:         },
0250:         "user": {
0251:             "AR_OVERDUE": 0,
0252:             "AP_OVERDUE": 1,
0253:             "AR_DUE_TODAY": 2,
0254:             "AP_DUE_TODAY": 3,
0255:         },
0256:     }
0257:     code_priority = profile_order.get(perfil, {})
0258:     items.sort(
0259:         key=lambda item: (
0260:             severity_order.get(item["severity"], 9),
0261:             code_priority.get(item["code"], 99),
0262:             item["title"],
0263:         )
0264:     )
0265:
0266:     summary = {
0267:         "critical": sum(1 for item in items if item["severity"] == "critical"),
0268:         "warning": sum(1 for item in items if item["severity"] == "warning"),
0269:         "info": sum(1 for item in items if item["severity"] == "info"),
0270:         "total": len(items),
0271:     }
0272:
0273:     return {
0274:         "filial": filial,
0275:         "usuario": login,
0276:         "generated_at": datetime.now().isoformat(timespec="seconds"),
0277:         "summary": summary,
0278:         "items": items,
0279:     }
```

==========
ARQUIVO: app/use_cases/ap_lancamento_uc.py
==========

```
0001: from __future__ import annotations
0002:
0003: from typing import Optional, Dict, Any
0004: from app.core.exceptions import BusinessError
0005: from app.use_cases.usuarios_uc import exigir_perfil_tx
0006: from app.infra.repositories.sa2_repo import sa2_get_tx
0007: from app.infra.repositories.financeiro_repo import ap_criar_tx
0008: from app.infra.repositories.logs_repo import log_tx
0009:
0010: def lancar_ap_uc_tx(
0011:     cur,
0012:     usuario: str,
0013:     filial: str,
0014:     forn_cod: str,
```

```
0015:     valor: float,
0016:     ref: str,
0017:     venc_dias: int = 30,
0018: ) -> Dict[str, Any]:
0019:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0020:
0021:     forn_cod = (forn_cod or "").strip()
0022:     if not forn_cod:
0023:         raise BusinessError("forn_cod é obrigatório")
0024:     if float(valor) <= 0:
0025:         raise BusinessError("valor deve ser > 0")
0026:
0027:     f = sa2_get_tx(cur, filial=filial, cod=forn_cod)
0028:     if not f:
0029:         raise BusinessError("Fornecedor não encontrado (SA2)")
0030:
0031:     forn_nome = f["A2_NOME"]
0032:
0033:     ap_criar_tx(
0034:         cur,
0035:         filial=filial,
0036:         forn=forn_nome,          # snapshot
0037:         forn_cod=forn_cod,       # chave SA2
0038:         valor=float(valor),
0039:         ref=ref,
0040:         venc_dias=int(venc_dias),
0041:     )
0042:
0043:     log_tx(cur, usuario, f"SF1.LANCAR filial={filial} forn_cod={forn_cod} valor={valor} ref={r
ef}")
0044:     return {"ok": True, "filial": filial, "forn_cod": forn_cod, "forn_nome": forn_nome, "valor
": float(valor), "ref": ref}
```

==========
ARQUIVO: app/use_cases/auditoria_uc.py
==========

```
0001: from app.infra.repositories.logs_repo import listar_logs_tx
0002: from app.infra.repositories.usuarios_repo import obter_perfil_tx
0003:
0004:
0005: def listar_logs_uc_tx(cur, usuario: str) -> list[dict]:
0006:     perfil = obter_perfil_tx(cur, usuario) or ""
0007:     return listar_logs_tx(cur, usuario, perfil)
```

==========
ARQUIVO: app/use_cases/auth_uc.py
==========

```
0001: import pyodbc
0002:
0003: from app.infra.repositories.usuarios_repo import obter_usuario_por_login_tx
0004: from app.security.passwords import verify_password
0005:
0006: def autenticar_usuario_tx(cur: pyodbc.Cursor, username: str, password: str) -> dict:
0007:     user = obter_usuario_por_login_tx(cur, username)
```

```
0008:     if not user:
0009:         raise ValueError("Credenciais inválidas")
0010:
0011:     if not verify_password(password, user["U_SENHA_HASH"]):
0012:         raise ValueError("Credenciais inválidas")
0013:
0014:     return user
```

==========
ARQUIVO: app/use_cases/clientes_uc.py
==========

```
0001: from __future__ import annotations
0002:
0003: from typing import Optional, Dict, Any, List
0004: from app.core.exceptions import BusinessError
0005: from app.infra.repositories.logs_repo import log_tx
0006: from app.infra.repositories.sa1_repo import (
0007:     sa1_get_tx,
0008:     sa1_listar_tx,
0009:     sa1_buscar_tx,
0010:     sa1_criar_tx,
0011:     sa1_atualizar_tx,
0012:     sa1_set_ativo_tx,
0013: )
0014:
0015: # (opcional) se você já usa exigir_perfil_tx, mantenha
0016: from app.use_cases.usuarios_uc import exigir_perfil_tx
0017:
0018: def cliente_get_uc_tx(cur, usuario: str, filial: str, cod: str) -> Dict[str, Any]:
0019:     exigir_perfil_tx(cur, usuario, {"admin", "operador", "auditor"})
0020:     c = sa1_get_tx(cur, filial=filial, cod=cod)
0021:     if not c:
0022:         raise BusinessError("Cliente não encontrado")
0023:     return c
0024:
0025: def cliente_listar_uc_tx(cur, usuario: str, filial: str, ativos: bool = True) -> List[Dict[str
, Any]]:
0026:     exigir_perfil_tx(cur, usuario, {"admin", "operador", "auditor"})
0027:     return sa1_listar_tx(cur, filial=filial, ativos=ativos)
0028:
0029: def cliente_buscar_uc_tx(cur, usuario: str, filial: str, termo: str) -> List[Dict[str, Any]]:
0030:     exigir_perfil_tx(cur, usuario, {"admin", "operador", "auditor"})
0031:     return sa1_buscar_tx(cur, filial=filial, termo=termo)
0032:
0033: def cliente_criar_uc_tx(
0034:     cur,
0035:     usuario: str,
0036:     filial: str,
0037:     cod: str,
0038:     nome: str,
0039:     doc: Optional[str] = None,
0040:     email: Optional[str] = None,
0041:     fone: Optional[str] = None,
0042: ) -> Dict[str, Any]:
```

```
0043:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0044:     c = sa1_criar_tx(cur, filial=filial, cod=cod, nome=nome, doc=doc, email=email, fone=fone)
0045:     log_tx(cur, usuario, f"SA1.CRIAR filial={filial} cod={cod} nome={nome}")
0046:     return c
0047:
0048: def cliente_atualizar_uc_tx(
0049:     cur,
0050:     usuario: str,
0051:     filial: str,
0052:     cod: str,
0053:     nome: Optional[str] = None,
0054:     doc: Optional[str] = None,
0055:     email: Optional[str] = None,
0056:     fone: Optional[str] = None,
0057: ) -> Dict[str, Any]:
0058:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0059:     c = sa1_atualizar_tx(cur, filial=filial, cod=cod, nome=nome, doc=doc, email=email, fone=fo
ne)
0060:     log_tx(cur, usuario, f"SA1.ATUALIZAR filial={filial} cod={cod}")
0061:     return c
0062:
0063: def cliente_set_ativo_uc_tx(cur, usuario: str, filial: str, cod: str, ativo: bool) -> Dict[str
, Any]:
0064:     exigir_perfil_tx(cur, usuario, {"admin"})
0065:     sa1_set_ativo_tx(cur, filial=filial, cod=cod, ativo=ativo)
0066:     log_tx(cur, usuario, f"SA1.ATIVO filial={filial} cod={cod} ativo={ativo}")
0067:     return {"filial": filial, "cod": cod, "ativo": bool(ativo)}
```

==========
ARQUIVO: app/use_cases/conciliacao_nf_uc.py
==========

```
0001: from datetime import date
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.relatorios_repo import conciliacao_nf_financeiro_tx
0004:
0005:
0006: def conciliacao_nf_financeiro_uc_tx(
0007:     cur,
0008:     usuario: str,
0009:     filial: str,
0010:     de: date | None,
0011:     ate: date | None,
0012:     tolerancia: float = 0.01,
0013:     limit: int = 200,
0014: ) -> dict:
0015:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0016:     return conciliacao_nf_financeiro_tx(
0017:         cur,
0018:         filial=filial,
0019:         de=de,
0020:         ate=ate,
0021:         tolerancia=float(tolerancia),
0022:         limit=int(limit),
0023:     )
```

==========
ARQUIVO: app/use_cases/conciliacao_uc.py
==========

```
0001: from app.use_cases.usuarios_uc import exigir_perfil_tx
0002: from app.infra.repositories.sd3_repo import saldo_sd3_tx
0003: from app.infra.repositories.logs_repo import log_tx
0004: from app.infra.db import fetchall_dict
0005:
0006: def recalcular_sb1_por_sd3_tx(cur, usuario: str, filial: str) -> dict:
0007:     exigir_perfil_tx(cur, usuario, {"admin"})
0008:     cur.execute("SELECT B1_COD FROM SB1_PRODUTOS WHERE B1_FILIAL=?", (filial,))
0009:     cods = [r["B1_COD"] for r in fetchall_dict(cur)]
0010:
0011:     for cod in cods:
0012:         saldo = saldo_sd3_tx(cur, cod, filial)
0013:         cur.execute(
0014:             "UPDATE SB1_PRODUTOS SET B1_ESTOQUE=? WHERE B1_COD=? AND B1_FILIAL=?",
0015:             (int(saldo), cod, filial),
0016:         )
0017:
0018:     log_tx(cur, usuario, f"Recalcular SB1 por SD3 (filial={filial}) itens={len(cods)}")
0019:     return {"filial": filial, "itens": len(cods)}
```

==========
ARQUIVO: app/use_cases/consistencia_uc.py
==========

```
0001: from datetime import date
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.relatorios_repo import consistencia_fiscal_tx
0004:
0005:
0006: def consistencia_fiscal_uc_tx(
0007:     cur,
0008:     usuario: str,
0009:     filial: str,
0010:     de: date | None,
0011:     ate: date | None,
0012:     limit: int = 200,
0013: ) -> dict:
0014:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0015:     return consistencia_fiscal_tx(
0016:         cur,
0017:         filial=filial,
0018:         de=de,
0019:         ate=ate,
0020:         limit=int(limit),
0021:     )
```

==========
ARQUIVO: app/use_cases/estoque_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
```

```
0003: from app.infra.repositories.sb1_repo import (
0004:     sb1_get_tx, sb1_atualizar_cm_tx, sb1_entrada_tx, sb1_baixar_atomico_tx
0005: )
0006: from app.infra.repositories.sd3_repo import sd3_mov_tx, saldo_sd3_tx, extrato_sd3_tx
0007: from app.infra.repositories.financeiro_repo import ap_criar_tx
0008: from app.infra.repositories.logs_repo import log_tx
0009:
0010: def recalcular_cm(cm_atual: float, saldo: int, qtd_entrada: int, custo_unit: float) -> float:
0011:     novo = saldo + qtd_entrada
0012:     if novo <= 0:
0013:         return 0.0
0014:     return ((saldo * cm_atual) + (qtd_entrada * custo_unit)) / novo
0015:
0016: def entrada_estoque_tx(cur, usuario: str, produto: str, qtd: int, filial: str,
0017:                        custo_unit: float, forn: str | None, venc_dias: int) -> dict:
0018:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0019:     if qtd <= 0:
0020:         raise BusinessError("qtd deve ser > 0")
0021:     if custo_unit < 0:
0022:         raise BusinessError("custo_unit não pode ser negativo")
0023:
0024:     s = sb1_get_tx(cur, produto, filial)
0025:     cm_novo = recalcular_cm(float(s["B1_CM"]), int(s["B1_ESTOQUE"]), int(qtd), float(custo_uni
t))
0026:     sb1_atualizar_cm_tx(cur, produto, filial, cm_novo)
0027:
0028:     valor = float(qtd) * float(custo_unit)
0029:     sd3_mov_tx(cur, filial, produto, "E", qtd, float(custo_unit), valor, "ENTRADA_ESTOQUE", No
ne,
0030: usuario)
0031:     sb1_entrada_tx(cur, produto, filial, qtd)
0032:
0033:     if forn:
0034:         ap_criar_tx(cur, filial, forn, valor, ref=f"ENTRADA:{produto}", venc_dias=venc_dias)
0035:
0036:     log_tx(cur, usuario, f"Entrada {produto}/{filial} +{qtd} custo={custo_unit} cm={cm_novo}")

0037:
0038:     snap = sb1_get_tx(cur, produto, filial)
0039:     snap["saldo_sd3"] = saldo_sd3_tx(cur, produto, filial)
0040:     return snap
0041:
0042: def consultar_estoque_tx(cur, produto: str, filial: str) -> dict:
0043:     s = sb1_get_tx(cur, produto, filial)
0044:     s["DISPONIVEL"] = int(s["B1_ESTOQUE"]) - int(s["B1_RESERVADO"])
0045:     s["saldo_sd3"] = saldo_sd3_tx(cur, produto, filial)
0046:     return s
0047:
0048: def extrato_sd3_uc_tx(cur, produto: str, filial: str, limite: int) -> list[dict]:
0049:     return extrato_sd3_tx(cur, produto, filial, limite)
```

==========
ARQUIVO: app/use_cases/fechamento_uc.py
==========

```
0001: from app.use_cases.usuarios_uc import exigir_perfil_tx
0002: from app.infra.repositories.fechamento_repo import (
0003:     fechamento_fechar_tx,
0004:     fechamento_abrir_tx,
0005: )
0006:
0007:
0008: def fechar_periodo_uc_tx(cur, usuario: str, filial: str, ano: int, mes: int):
0009:     exigir_perfil_tx(cur, usuario, {"admin"})
0010:     fechamento_fechar_tx(cur, filial=filial, ano=int(ano), mes=int(mes), usuario=usuario)
0011:     return {"filial": filial, "ano": int(ano), "mes": int(mes), "fechado": True}
0012:
0013:
0014: def abrir_periodo_uc_tx(cur, usuario: str, filial: str, ano: int, mes: int):
0015:     exigir_perfil_tx(cur, usuario, {"admin"})
0016:     fechamento_abrir_tx(cur, filial=filial, ano=int(ano), mes=int(mes), usuario=usuario)
0017:     return {"filial": filial, "ano": int(ano), "mes": int(mes), "fechado": False}
```

==========
ARQUIVO: app/use_cases/filiais_uc.py
==========

```
0001: from app.use_cases.usuarios_uc import exigir_perfil_tx
0002: from app.infra.repositories.sm0_repo import (
0003:     sm0_listar_tx,
0004:     sm0_criar_tx,
0005:     sm0_inativar_tx,
0006: )
0007: from app.infra.repositories.serie_repo import seed_series_basicas_tx
0008:
0009:
0010: def filiais_listar_uc_tx(cur, usuario: str):
0011:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0012:     return sm0_listar_tx(cur)
0013:
0014:
0015: def filiais_criar_uc_tx(
0016:     cur,
0017:     usuario: str,
0018:     filial: str,
0019:     serie_nf: str | None,
0020:     serie_ar: str | None,
0021:     serie_ap: str | None,
0022: ):
0023:     exigir_perfil_tx(cur, usuario, {"admin"})
0024:     sm0_criar_tx(cur, filial=filial, serie_nf=serie_nf, serie_ar=serie_ar, serie_ap=serie_ap)
0025:     seed_series_basicas_tx(cur, filial=filial)
0026:     return {"filial": filial}
0027:
0028:
0029: def filiais_inativar_uc_tx(cur, usuario: str, filial: str):
0030:     exigir_perfil_tx(cur, usuario, {"admin"})
0031:     sm0_inativar_tx(cur, filial=filial)
0032:     return {"filial": filial, "ativo": False}
```

```
0001: from __future__ import annotations
0002:
0003: from datetime import date
0004: from typing import Any, Dict
0005:
0006: import pyodbc
0007:
0008: from app.domain.errors import BusinessError
0009: from app.infra.repositories.financeiro_repo import (
0010:     criar_categoria_tx,
0011:     definir_categoria_mov_tx,
0012:     dre_simples_tx,
0013:     extrato_caixa_tx,
0014:     listar_ap_tx,
0015:     listar_ar_tx,
0016:     listar_categorias_tx,
0017:     listar_contas_caixa_tx,
0018:     obter_movimento_tx,
0019:     pagar_ap_e_gerar_caixa_tx,
0020:     receber_ar_e_gerar_caixa_tx,
0021:     saldo_caixa_simples_tx,
0022: )
0023: from app.infra.repositories.logs_repo import log_tx
0024:
0025:
0026: def listar_ar_uc_tx(cur, filial: str, status: str = "ABERTO"):
0027:     return listar_ar_tx(cur, filial=filial, status=status)
0028:
0029:
0030: def listar_ap_uc_tx(cur, usuario: str, filial: str, status: str = "ABERTO"):
0031:     return listar_ap_tx(cur, filial=filial, status=status)
0032:
0033:
0034: def receber_ar_uc_tx(cur, usuario: str, titulo_id: int, conta_id: int) -> dict:
0035:     mov_id = receber_ar_e_gerar_caixa_tx(
0036:         cur,
0037:         titulo_id=int(titulo_id),
0038:         conta_id=int(conta_id),
0039:         usuario=usuario,
0040:     )
0041:     log_tx(cur, usuario, f"AR recebido id={titulo_id} se5={mov_id}")
0042:     return {
0043:         "titulo_id": int(titulo_id),
0044:         "status": "BAIXADO",
0045:         "se5_id": int(mov_id),
0046:     }
0047:
0048:
0049: def pagar_ap_uc_tx(cur: pyodbc.Cursor, usuario: str, titulo_id: int, conta_id: int) -> Dict[st
r, Any]:
0050:     """
```

```
0051:     Paga um título do AP (SF1) e gera movimento de caixa (SE5) na mesma transação.
0052:     """
0053:
0054:     mov_id = pagar_ap_e_gerar_caixa_tx(
0055:         cur,
0056:         titulo_id=int(titulo_id),
0057:         conta_id=int(conta_id),
0058:         usuario=usuario,
0059:     )
0060:     log_tx(cur, usuario, f"AP pago id={titulo_id} se5={mov_id}")
0061:     return {
0062:         "id": int(titulo_id),
0063:         "status": "BAIXADO",
0064:         "se5_id": int(mov_id),
0065:         "conta_id": int(conta_id),
0066:     }
0067:
0068:
0069: def listar_contas_caixa_uc_tx(cur, filial: str):
0070:     return listar_contas_caixa_tx(cur, filial=filial)
0071:
0072:
0073: def listar_categorias_uc_tx(cur, filial: str, ativas: bool = True):
0074:     return listar_categorias_tx(cur, filial=filial, ativas=ativas)
0075:
0076:
0077: def criar_categoria_uc_tx(cur, usuario: str, filial: str, nome: str, tipo: str):
0078:     tipo = (tipo or "").upper().strip()
0079:     if tipo not in {"RECEITA", "DESPESA", "CUSTO", "TRANSFERENCIA", "OUTROS"}:
0080:         raise BusinessError("Tipo de categoria inválido")
0081:     cat_id = criar_categoria_tx(cur, filial=filial, nome=nome.strip(), tipo=tipo)
0082:     log_tx(cur, usuario, f"FIN.SE5.CATEG.CRIAR id={cat_id} filial={filial} tipo={tipo} nome={n
ome}")
0083:     return {"id": cat_id, "filial": filial, "tipo": tipo, "nome": nome}
0084:
0085:
0086: def definir_categoria_mov_uc_tx(cur, usuario: str, mov_id: int, categ_id: int | None):
0087:     definir_categoria_mov_tx(
0088:         cur,
0089:         mov_id=int(mov_id),
0090:         categ_id=(int(categ_id) if categ_id is not None else None),
0091:     )
0092:     log_tx(cur, usuario, f"FIN.SE5.MOV.SETCATEG mov_id={mov_id} categ_id={categ_id}")
0093:     return {"mov_id": int(mov_id), "categ_id": (int(categ_id) if categ_id is not None else Non
e)}
0094:
0095:
0096: def obter_movimento_uc_tx(cur, mov_id: int):
0097:     mov = obter_movimento_tx(cur, mov_id=int(mov_id))
0098:     if not mov:
0099:         raise BusinessError("Movimento não encontrado")
0100:     if "E5_VALOR" in mov and mov["E5_VALOR"] is not None:
0101:         mov["E5_VALOR"] = float(mov["E5_VALOR"])
0102:     return mov
```

```
0103:
0104:
0105: def extrato_caixa_uc_tx(cur, filial: str, conta_id: int, de: date | None, ate: date | None):
0106:     rows = extrato_caixa_tx(cur, filial=filial, conta_id=int(conta_id), de=de, ate=ate)
0107:     for r in rows:
0108:         if "E5_VALOR" in r:
0109:             r["E5_VALOR"] = float(r["E5_VALOR"])
0110:         if "E5_VALOR_ASSINADO" in r:
0111:             r["E5_VALOR_ASSINADO"] = float(r["E5_VALOR_ASSINADO"])
0112:         if "E5_SALDO_ACUMULADO" in r:
0113:             r["E5_SALDO_ACUMULADO"] = float(r["E5_SALDO_ACUMULADO"])
0114:     return rows
0115:
0116:
0117: def saldo_caixa_uc_tx(cur, filial: str, conta_id: int, ate: date | None):
0118:     s = saldo_caixa_simples_tx(cur, filial=filial, conta_id=int(conta_id), ate=ate)
0119:     return {
0120:         "filial": filial,
0121:         "conta_id": int(conta_id),
0122:         "ate": ate.isoformat() if ate else None,
0123:         "entradas": float(s["entradas"]),
0124:         "saidas": float(s["saidas"]),
0125:         "saldo": float(s["saldo"]),
0126:     }
0127:
0128:
0129: def dre_simples_uc_tx(cur, filial: str, de: date | None, ate: date | None):
0130:     res = dre_simples_tx(cur, filial=filial, de=de, ate=ate)
0131:     for l in res["linhas"]:
0132:         l["valor"] = float(l["valor"])
0133:     res["total"] = float(res["total"])
0134:     res["de"] = de.isoformat() if de else None
0135:     res["ate"] = ate.isoformat() if ate else None
0136:     return res
```

==========
ARQUIVO: app/use_cases/fiscal_ncm_uc.py
==========

```
0001: from app.use_cases.usuarios_uc import exigir_perfil_tx
0002: from app.infra.repositories.fiscal_ncm_repo import ncm_listar_tx, ncm_upsert_tx
0003:
0004:
0005: def ncm_listar_uc_tx(cur, usuario: str, filial: str, ativos: bool = True):
0006:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0007:     return ncm_listar_tx(cur, filial=filial, ativos=ativos)
0008:
0009:
0010: def ncm_salvar_uc_tx(
0011:     cur,
0012:     usuario: str,
0013:     filial: str,
0014:     ncm: str,
0015:     cfop: str,
0016:     icms: float | None,
```

```
0017:     ipi: float | None,
0018:     ativo: bool = True,
0019: ):
0020:     exigir_perfil_tx(cur, usuario, {"admin"})
0021:     ncm_upsert_tx(
0022:         cur,
0023:         filial=filial,
0024:         ncm=ncm,
0025:         cfop=cfop,
0026:         icms=icms,
0027:         ipi=ipi,
0028:         ativo=ativo,
0029:     )
0030:     return {"filial": filial, "ncm": ncm}
```

==========
ARQUIVO: app/use_cases/fiscal_regras_uc.py
==========

```
0001: from app.use_cases.usuarios_uc import exigir_perfil_tx
0002: from app.infra.repositories.fiscal_repo import (
0003:     fiscal_regra_listar_tx,
0004:     fiscal_regra_upsert_tx,
0005:     fiscal_regra_inativar_tx,
0006: )
0007:
0008:
0009: def fiscal_regra_listar_uc_tx(
0010:     cur,
0011:     usuario: str,
0012:     filial: str,
0013:     tes_cod: str | None = None,
0014:     cliente_cod: str | None = None,
0015:     produto: str | None = None,
0016:     ativos: bool = True,
0017: ):
0018:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0019:     return fiscal_regra_listar_tx(
0020:         cur,
0021:         filial=filial,
0022:         tes_cod=tes_cod,
0023:         cliente_cod=cliente_cod,
0024:         produto=produto,
0025:         ativos=ativos,
0026:     )
0027:
0028:
0029: def fiscal_regra_salvar_uc_tx(
0030:     cur,
0031:     usuario: str,
0032:     filial: str,
0033:     tes_cod: str,
0034:     cliente_cod: str | None,
0035:     produto: str | None,
0036:     cfop: str | None,
```

```
0037:     icms: float | None,
0038:     ipi: float | None,
0039:     pis: float | None = None,
0040:     cofins: float | None = None,
0041:     icms_st: float | None = None,
0042:     difal: float | None = None,
0043:     cst_icms: str | None = None,
0044:     csosn: str | None = None,
0045:     cst_pis: str | None = None,
0046:     cst_cofins: str | None = None,
0047:     gera_tit: bool | None = None,
0048:     gera_est: bool | None = None,
0049:     prioridade: int = 0,
0050:     ativo: bool = True,
0051:     regra_id: int | None = None,
0052: ) -> dict:
0053:     exigir_perfil_tx(cur, usuario, {"admin"})
0054:     regra_id = fiscal_regra_upsert_tx(
0055:         cur,
0056:         filial=filial,
0057:         tes_cod=tes_cod,
0058:         cliente_cod=cliente_cod,
0059:         produto=produto,
0060:         cfop=cfop,
0061:         icms=icms,
0062:         ipi=ipi,
0063:         pis=pis,
0064:         cofins=cofins,
0065:         icms_st=icms_st,
0066:         difal=difal,
0067:         cst_icms=cst_icms,
0068:         csosn=csosn,
0069:         cst_pis=cst_pis,
0070:         cst_cofins=cst_cofins,
0071:         gera_tit=gera_tit,
0072:         gera_est=gera_est,
0073:         prioridade=prioridade,
0074:         ativo=ativo,
0075:         regra_id=regra_id,
0076:     )
0077:     return {"id": int(regra_id), "filial": filial, "tes_cod": tes_cod}
0078:
0079:
0080: def fiscal_regra_inativar_uc_tx(cur, usuario: str, regra_id: int):
0081:     exigir_perfil_tx(cur, usuario, {"admin"})
0082:     fiscal_regra_inativar_tx(cur, regra_id=int(regra_id))
0083:     return {"id": int(regra_id), "ativo": False}
```

==========
ARQUIVO: app/use_cases/fiscal_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
```

```
0004: from app.infra.repositories.fiscal_repo import (
0005:     fiscal_upsert_tx,
0006:     fiscal_get_tes_tx,
0007:     fiscal_listar_tx,
0008:     fiscal_inativar_tx,
0009: )
0010:
0011:
0012: def sf4_salvar_uc_tx(
0013:     cur,
0014:     usuario: str,
0015:     filial: str,
0016:     tes_cod: str,
0017:     cfop: str,
0018:     icms: float,
0019:     ipi: float,
0020:     tipo: str = "S",
0021:     pis: float = 0.0,
0022:     cofins: float = 0.0,
0023:     icms_st: float = 0.0,
0024:     difal: float = 0.0,
0025:     cst_icms: str | None = None,
0026:     csosn: str | None = None,
0027:     cst_pis: str | None = None,
0028:     cst_cofins: str | None = None,
0029:     gera_tit: bool = True,
0030:     gera_est: bool = True,
0031:     descr: str | None = None,
0032: ) -> dict:
0033:     exigir_perfil_tx(cur, usuario, {"admin"})
0034:     def _norm_code(v: str | None) -> str | None:
0035:         if v is None:
0036:             return None
0037:         s = str(v).strip()
0038:         if not s or s.lower() in {"null", "none"}:
0039:             return None
0040:         return s
0041:
0042:     cst_icms = _norm_code(cst_icms)
0043:     csosn = _norm_code(csosn)
0044:     cst_pis = _norm_code(cst_pis)
0045:     cst_cofins = _norm_code(cst_cofins)
0046:     fiscal_upsert_tx(
0047:         cur,
0048:         filial=filial,
0049:         tes_cod=tes_cod,
0050:         cfop=cfop,
0051:         icms=icms,
0052:         ipi=ipi,
0053:         pis=pis,
0054:         cofins=cofins,
0055:         icms_st=icms_st,
0056:         difal=difal,
0057:         cst_icms=cst_icms,
```

```
0058:         csosn=csosn,
0059:         cst_pis=cst_pis,
0060:         cst_cofins=cst_cofins,
0061:         tipo=tipo,
0062:         gera_tit=gera_tit,
0063:         gera_est=gera_est,
0064:         descr=descr,
0065:     )
0066:     return fiscal_get_tes_tx(cur, filial=filial, tes_cod=tes_cod)
0067:
0068:
0069: def sf4_listar_uc_tx(cur, usuario: str, filial: str, ativos: bool = True):
0070:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0071:     return fiscal_listar_tx(cur, filial=filial, ativo=ativos)
0072:
0073:
0074: def sf4_inativar_uc_tx(cur, usuario: str, filial: str, tes_cod: str):
0075:     exigir_perfil_tx(cur, usuario, {"admin"})
0076:     fiscal_inativar_tx(cur, filial=filial, tes_cod=tes_cod, usuario=usuario)
0077:     return {"filial": filial, "tes_cod": tes_cod, "ativo": False}
```

==========
ARQUIVO: app/use_cases/fornecedores_uc.py
==========

```
0001: from __future__ import annotations
0002:
0003: from typing import Optional, Dict, Any, List
0004: from app.core.exceptions import BusinessError
0005: from app.infra.repositories.logs_repo import log_tx
0006: from app.infra.repositories.sa2_repo import (
0007:     sa2_get_tx,
0008:     sa2_listar_tx,
0009:     sa2_buscar_tx,
0010:     sa2_criar_tx,
0011:     sa2_atualizar_tx,
0012:     sa2_set_ativo_tx,
0013: )
0014: from app.use_cases.usuarios_uc import exigir_perfil_tx
0015:
0016: def forn_get_uc_tx(cur, usuario: str, filial: str, cod: str) -> Dict[str, Any]:
0017:     exigir_perfil_tx(cur, usuario, {"admin", "operador", "auditor"})
0018:     f = sa2_get_tx(cur, filial=filial, cod=cod)
0019:     if not f:
0020:         raise BusinessError("Fornecedor não encontrado")
0021:     return f
0022:
0023: def forn_listar_uc_tx(cur, usuario: str, filial: str, ativos: bool = True) -> List[Dict[str, A
ny]]:
0024:     exigir_perfil_tx(cur, usuario, {"admin", "operador", "auditor"})
0025:     return sa2_listar_tx(cur, filial=filial, ativos=ativos)
0026:
0027: def forn_buscar_uc_tx(cur, usuario: str, filial: str, termo: str) -> List[Dict[str, Any]]:
0028:     exigir_perfil_tx(cur, usuario, {"admin", "operador", "auditor"})
0029:     return sa2_buscar_tx(cur, filial=filial, termo=termo)
```

```
0030:
0031: def forn_criar_uc_tx(
0032:     cur,
0033:     usuario: str,
0034:     filial: str,
0035:     cod: str,
0036:     nome: str,
0037:     doc: Optional[str] = None,
0038:     email: Optional[str] = None,
0039:     fone: Optional[str] = None,
0040: ) -> Dict[str, Any]:
0041:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0042:     f = sa2_criar_tx(cur, filial=filial, cod=cod, nome=nome, doc=doc, email=email, fone=fone)
0043:     log_tx(cur, usuario, f"SA2.CRIAR filial={filial} cod={cod} nome={nome}")
0044:     return f
0045:
0046: def forn_atualizar_uc_tx(
0047:     cur,
0048:     usuario: str,
0049:     filial: str,
0050:     cod: str,
0051:     nome: Optional[str] = None,
0052:     doc: Optional[str] = None,
0053:     email: Optional[str] = None,
0054:     fone: Optional[str] = None,
0055: ) -> Dict[str, Any]:
0056:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0057:     f = sa2_atualizar_tx(cur, filial=filial, cod=cod, nome=nome, doc=doc, email=email, fone=fo
ne)
0058:     log_tx(cur, usuario, f"SA2.ATUALIZAR filial={filial} cod={cod}")
0059:     return f
0060:
0061: def forn_set_ativo_uc_tx(cur, usuario: str, filial: str, cod: str, ativo: bool) -> Dict[str, A
ny]:
0062:     exigir_perfil_tx(cur, usuario, {"admin"})
0063:     sa2_set_ativo_tx(cur, filial=filial, cod=cod, ativo=ativo)
0064:     log_tx(cur, usuario, f"SA2.ATIVO filial={filial} cod={cod} ativo={ativo}")
0065:     return {"filial": filial, "cod": cod, "ativo": bool(ativo)}
```

==========
ARQUIVO: app/use_cases/parceiros_uc.py
==========

```
0001: from app.domain.errors import BusinessError
0002: from app.infra.repositories.logs_repo import log_tx
0003: from app.infra.repositories.parceiros_repo import (
0004:     sa1_criar_tx,
0005:     sa1_inativar_tx,
0006:     sa1_listar_tx,
0007:     sa1_obter_tx,
0008:     sa1_set_tabela_tx,
0009:     sa2_criar_tx,
0010:     sa2_inativar_tx,
0011:     sa2_listar_tx,
0012:     sa2_obter_tx,
```

```
0013: )
0014:
0015:
0016: def criar_cliente_uc_tx(
0017:     cur,
0018:     usuario: str,
0019:     filial: str,
0020:     cod: str,
0021:     nome: str,
0022:     doc: str | None,
0023:     email: str | None,
0024:     tel: str | None,
0025:     end: str | None,
0026:     mun: str | None,
0027:     uf: str | None,
0028:     cep: str | None,
0029:     tabela_id: int | None,
0030: ):
0031:     cod = cod.strip()
0032:     nome = nome.strip()
0033:     if not cod or not nome:
0034:         raise BusinessError("Código e nome são obrigatórios")
0035:
0036:     cliente_id = sa1_criar_tx(cur, filial, cod, nome, doc, email, tel, end, mun, uf, cep, tabe
la_id)
0037:     log_tx(cur, usuario, f"SA1.CRIAR id={cliente_id} cod={cod} filial={filial} tabela_id={tabe
la_id}")
0038:     return {"id": cliente_id, "cod": cod, "nome": nome, "filial": filial, "tabela_id": tabela_
id}
0039:
0040:
0041: def listar_clientes_uc_tx(cur, filial: str, ativo: bool = True, q: str | None = None):
0042:     return sa1_listar_tx(cur, filial=filial, ativo=ativo, q=q)
0043:
0044:
0045: def obter_cliente_uc_tx(cur, filial: str, cod: str):
0046:     cliente = sa1_obter_tx(cur, filial=filial, cod=cod)
0047:     if not cliente:
0048:         raise BusinessError("Cliente não encontrado")
0049:     return cliente
0050:
0051:
0052: def set_tabela_cliente_uc_tx(cur, usuario: str, filial: str, cod: str, tabela_id: int | None):
0053:     sa1_set_tabela_tx(cur, filial=filial, cod=cod, tabela_id=tabela_id)
0054:     log_tx(cur, usuario, f"SA1.SET_TABELA cod={cod} filial={filial} tabela_id={tabela_id}")
0055:     return {"cod": cod, "filial": filial, "tabela_id": tabela_id}
0056:
0057:
0058: def inativar_cliente_uc_tx(cur, usuario: str, filial: str, cod: str):
0059:     sa1_inativar_tx(cur, filial=filial, cod=cod)
0060:     log_tx(cur, usuario, f"SA1.INATIVAR cod={cod} filial={filial}")
0061:     return {"cod": cod, "filial": filial, "ativo": False}
0062:
0063:
```

```
0064: def criar_fornecedor_uc_tx(
0065:     cur,
0066:     usuario: str,
0067:     filial: str,
0068:     cod: str,
0069:     nome: str,
0070:     doc: str | None,
0071:     email: str | None,
0072:     tel: str | None,
0073:     end: str | None,
0074:     mun: str | None,
0075:     uf: str | None,
0076:     cep: str | None,
0077: ):
0078:     cod = cod.strip()
0079:     nome = nome.strip()
0080:     if not cod or not nome:
0081:         raise BusinessError("Código e nome são obrigatórios")
0082:
0083:     fornecedor_id = sa2_criar_tx(cur, filial, cod, nome, doc, email, tel, end, mun, uf, cep)
0084:     log_tx(cur, usuario, f"SA2.CRIAR id={fornecedor_id} cod={cod} filial={filial}")
0085:     return {"id": fornecedor_id, "cod": cod, "nome": nome, "filial": filial}
0086:
0087:
0088: def listar_fornecedores_uc_tx(cur, filial: str, ativo: bool = True, q: str | None = None):
0089:     return sa2_listar_tx(cur, filial=filial, ativo=ativo, q=q)
0090:
0091:
0092: def obter_fornecedor_uc_tx(cur, filial: str, cod: str):
0093:     fornecedor = sa2_obter_tx(cur, filial=filial, cod=cod)
0094:     if not fornecedor:
0095:         raise BusinessError("Fornecedor não encontrado")
0096:     return fornecedor
0097:
0098:
0099: def inativar_fornecedor_uc_tx(cur, usuario: str, filial: str, cod: str):
0100:     sa2_inativar_tx(cur, filial=filial, cod=cod)
0101:     log_tx(cur, usuario, f"SA2.INATIVAR cod={cod} filial={filial}")
0102:     return {"cod": cod, "filial": filial, "ativo": False}
```

==========
ARQUIVO: app/use_cases/preco_uc.py
==========

```
0001: from datetime import date
0002:
0003: from app.domain.errors import BusinessError
0004: from app.infra.repositories.logs_repo import log_tx
0005: from app.infra.repositories.preco_repo import (
0006:     buscar_preco_vigente_tx,
0007:     criar_tabela_tx,
0008:     definir_preco_tx,
0009:     listar_precos_tabela_tx,
0010:     listar_tabelas_tx,
0011: )
```

```
0012:
0013:
0014: def listar_tabelas_uc_tx(cur, filial: str):
0015:     rows = listar_tabelas_tx(cur, filial)
0016:     return [
0017:         {
0018:             "id": int(row["ID"]),
0019:             "filial": row["P0_FILIAL"],
0020:             "codigo": row["P0_COD"],
0021:             "descricao": row["P0_DESC"],
0022:             "ativa": bool(row["P0_ATIVA"]),
0023:         }
0024:         for row in rows
0025:     ]
0026:
0027:
0028: def criar_tabela_uc_tx(cur, usuario: str, filial: str, cod: str, desc: str):
0029:     tabela_id = criar_tabela_tx(cur, filial, cod, desc)
0030:     log_tx(cur, usuario, f"PRECO.TABELA.CRIAR id={tabela_id} filial={filial}")
0031:     return {"id": tabela_id, "codigo": cod, "descricao": desc}
0032:
0033:
0034: def listar_precos_tabela_uc_tx(cur, filial: str, tabela_id: int):
0035:     rows = listar_precos_tabela_tx(cur, filial, tabela_id)
0036:     return [
0037:         {
0038:             "id": int(row["ID"]),
0039:             "filial": row["P1_FILIAL"],
0040:             "tabela_id": int(row["P1_TABELA_ID"]),
0041:             "produto": row["P1_PRODUTO"],
0042:             "preco": float(row["P1_PRECO"]),
0043:             "dt_ini": row["P1_DTINI"].isoformat() if row["P1_DTINI"] else None,
0044:             "dt_fim": row["P1_DTFIM"].isoformat() if row["P1_DTFIM"] else None,
0045:             "vigente": row["P1_DTFIM"] is None,
0046:         }
0047:         for row in rows
0048:     ]
0049:
0050:
0051: def definir_preco_uc_tx(
0052:     cur,
0053:     usuario: str,
0054:     filial: str,
0055:     tabela_id: int,
0056:     produto: str,
0057:     preco: float,
0058:     dt_ini: date,
0059: ):
0060:     if preco <= 0:
0061:         raise BusinessError("Preço inválido")
0062:
0063:     definir_preco_tx(cur, filial, tabela_id, produto, preco, dt_ini)
0064:     log_tx(cur, usuario, f"PRECO.DEF produto={produto} tabela={tabela_id} preco={preco}")
0065:     return {"produto": produto, "preco": preco, "dt_ini": dt_ini.isoformat()}
```

```
0066:
0067:
0068: def buscar_preco_uc_tx(cur, filial: str, tabela_id: int, produto: str, data: date):
0069:     return {
0070:         "produto": produto,
0071:         "preco": buscar_preco_vigente_tx(cur, filial, tabela_id, produto, data),
0072:     }
```

==========
ARQUIVO: app/use_cases/produtos_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.sb1_repo import sb1_upsert_tx, sb1_get_tx, sb1_buscar_tx
0004: from app.infra.repositories.sd3_repo import saldo_sd3_tx
0005: from app.infra.repositories.logs_repo import log_tx
0006:
0007: def upsert_produto_tx(cur, usuario: str, cod: str, desc: str, preco: float, filial: str) -> di
ct:
0008:     exigir_perfil_tx(cur, usuario, {"admin"})
0009:     if preco < 0:
0010:         raise BusinessError("Preço não pode ser negativo")
0011:
0012:     sb1_upsert_tx(cur, cod, desc, preco, filial)
0013:     log_tx(cur, usuario, f"Produto upsert {cod}/{filial}")
0014:
0015:     snap = sb1_get_tx(cur, cod, filial)
0016:     snap["saldo_sd3"] = saldo_sd3_tx(cur, cod, filial)
0017:     return snap
0018:
0019:
0020: def buscar_produtos_tx(cur, filial: str, q: str, limite: int = 20) -> list[dict]:
0021:     return sb1_buscar_tx(cur, filial, q, limite)
```

==========
ARQUIVO: app/use_cases/reservas_uc.py
==========

```
0001: from datetime import date
0002:
0003: from app.core.exceptions import BusinessError
0004: from app.infra.db import fetchone_dict
0005: from app.infra.repositories.logs_repo import log_tx
0006: from app.infra.repositories.preco_repo import buscar_preco_vigente_tx
0007: from app.infra.repositories.reservas_repo import (
0008:     reserva_criar_tx,
0009:     reserva_get_tx,
0010:     reserva_set_cliente_preco_tx,
0011:     reserva_status_tx,
0012: )
0013: from app.infra.repositories.sa1_repo import sa1_get_tx
0014: from app.infra.repositories.sb1_repo import (
0015:     sb1_baixar_e_desreservar_atomico_tx,
0016:     sb1_cancelar_reserva_atomico_tx,
0017:     sb1_get_tx,
```

```
0018:     sb1_reservar_atomico_tx,
0019: )
0020: from app.infra.repositories.sd3_repo import saldo_sd3_tx, sd3_mov_tx
0021: from app.use_cases.usuarios_uc import exigir_perfil_tx
0022: from app.use_cases.vendas.faturamento_uc import faturar_reserva_tx
0023:
0024:
0025: def _tabela_padrao_id_tx(cur, filial: str, cod: str = "0001") -> int:
0026:     cur.execute("SELECT COL_LENGTH('dbo.PR0_TABELA','D_E_L_E_T_') AS col")
0027:     has_deleted_column = cur.fetchone()
0028:     has_deleted_column = bool(has_deleted_column and has_deleted_column[0] is not None)
0029:
0030:     sql = """
0031:         SELECT TOP 1 ID
0032:         FROM dbo.PR0_TABELA
0033:         WHERE P0_FILIAL=? AND P0_COD=? AND P0_ATIVA=1
0034:     """
0035:     if has_deleted_column:
0036:         sql += " AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*')"
0037:     sql += " ORDER BY ID"
0038:
0039:     cur.execute(sql, (filial, cod))
0040:     row = fetchone_dict(cur)
0041:     if not row:
0042:         raise BusinessError(f"Não existe tabela de preço ativa (PR0) filial={filial} cod={cod}
")
0043:     return int(row["ID"])
0044:
0045:
0046: def criar_reserva_tx(
0047:     cur,
0048:     usuario: str,
0049:     produto: str,
0050:     qtd: int,
0051:     filial: str,
0052:     cliente_cod: str | None = None,
0053:     tabela_cod: str = "0001",
0054: ) -> dict:
0055:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0056:     if int(qtd) <= 0:
0057:         raise BusinessError("qtd deve ser > 0")
0058:
0059:     produto_row = sb1_get_tx(cur, produto, filial)
0060:
0061:     tabela_id = _tabela_padrao_id_tx(cur, filial, cod=tabela_cod)
0062:     preco_source = "TABELA"
0063:     try:
0064:         preco_unit = buscar_preco_vigente_tx(
0065:             cur,
0066:             filial=filial,
0067:             tabela_id=tabela_id,
0068:             produto=produto,
0069:             data=date.today(),
0070:         )
```

```
0071:     except BusinessError as exc:
0072:         if str(exc) != "Preco nao encontrado":
0073:             raise
0074:         preco_unit = float(produto_row.get("B1_PRECO") or 0.0)
0075:         if preco_unit <= 0:
0076:             raise
0077:         preco_source = "PRODUTO"
0078:     total = float(preco_unit) * int(qtd)
0079:
0080:     sb1_reservar_atomico_tx(cur, produto, filial, int(qtd))
0081:
0082:     reserva_id = reserva_criar_tx(
0083:         cur,
0084:         filial=filial,
0085:         produto=produto,
0086:         qtd=int(qtd),
0087:         usuario=usuario,
0088:         cliente_cod=cliente_cod,
0089:         tabela_id=tabela_id,
0090:         preco_unit=float(preco_unit),
0091:         total=float(total),
0092:     )
0093:
0094:     log_tx(
0095:         cur,
0096:         usuario,
0097:         f"Reserva {reserva_id} criada {produto}/{filial} qtd={qtd} cliente_cod={cliente_cod} t
ab_id={tabela_id} preco_unit={preco_unit} fonte_preco={preco_source}",
0098:     )
0099:
0100:     saldo_snapshot = sb1_get_tx(cur, produto, filial)
0101:     saldo_snapshot["saldo_sd3"] = saldo_sd3_tx(cur, produto, filial)
0102:
0103:     return {
0104:         "reserva_id": int(reserva_id),
0105:         "status": "ABERTA",
0106:         "cliente_cod": cliente_cod,
0107:         "tabela_id": int(tabela_id),
0108:         "preco_unit": float(preco_unit),
0109:         "preco_source": preco_source,
0110:         "total": float(total),
0111:         "sb1": saldo_snapshot,
0112:     }
0113:
0114:
0115: def cancelar_reserva_tx(cur, usuario: str, reserva_id: int) -> dict:
0116:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0117:     reserva = reserva_get_tx(cur, int(reserva_id))
0118:     if not reserva:
0119:         raise BusinessError("Reserva não encontrada")
0120:     if reserva["Z0_STATUS"] != "ABERTA":
0121:         raise BusinessError("Apenas reserva ABERTA pode ser cancelada")
0122:
0123:     produto = reserva["Z0_PRODUTO"]
```

```
0124:     filial = reserva["Z0_FILIAL"]
0125:     qtd = int(reserva["Z0_QTD"])
0126:
0127:     sb1_cancelar_reserva_atomico_tx(cur, produto, filial, qtd)
0128:     reserva_status_tx(cur, int(reserva_id), "CANCELADA")
0129:     log_tx(cur, usuario, f"Reserva {reserva_id} cancelada")
0130:
0131:     saldo_snapshot = sb1_get_tx(cur, produto, filial)
0132:     saldo_snapshot["saldo_sd3"] = saldo_sd3_tx(cur, produto, filial)
0133:     return {"reserva_id": int(reserva_id), "status": "CANCELADA", "sb1": saldo_snapshot}
0134:
0135:
0136: def confirmar_reserva_tx(
0137:     cur,
0138:     usuario: str,
0139:     reserva_id: int,
0140:     cliente_cod: str | None,
0141:     venc_dias: int,
0142:     tes_cod: str | None = None,
0143: ) -> dict:
0144:     return faturar_reserva_tx(
0145:         cur=cur,
0146:         usuario=usuario,
0147:         reserva_id=int(reserva_id),
0148:         cliente_cod=cliente_cod,
0149:         venc_dias=int(venc_dias),
0150:         tes_cod=(tes_cod or "001"),
0151:     )
```

==========
ARQUIVO: app/use_cases/series_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.sm0_repo import sm0_obter_series_tx, sm0_definir_series_tx
0005: from app.infra.repositories.serie_repo import resolve_serie_tx, seed_series_basicas_tx, valida
r_serie_tx
0006: from app.infra.repositories.sx5_repo import sx5_listar_tx, sx5_obter_tx, sx5_criar_tx
0007:
0008:
0009: def series_obter_uc_tx(cur, usuario: str, filial: str) -> dict:
0010:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0011:     seed_series_basicas_tx(cur, filial)
0012:     sm0 = sm0_obter_series_tx(cur, filial)
0013:     return {
0014:         "filial": filial,
0015:         "sm0": sm0,
0016:         "serie_nf": resolve_serie_tx(cur, filial, "NF"),
0017:         "serie_ar": resolve_serie_tx(cur, filial, "AR"),
0018:         "serie_ap": resolve_serie_tx(cur, filial, "AP"),
0019:     }
0020:
0021:
```

```
0022: def series_definir_uc_tx(cur, usuario: str, filial: str, serie_nf: str | None, serie_ar: str |
 None, serie_ap: str | None):
0023:     exigir_perfil_tx(cur, usuario, {"admin"})
0024:     if serie_nf is not None and not str(serie_nf).strip():
0025:         raise BusinessError("Série NF inválida")
0026:     if serie_ar is not None and not str(serie_ar).strip():
0027:         raise BusinessError("Série AR inválida")
0028:     if serie_ap is not None and not str(serie_ap).strip():
0029:         raise BusinessError("Série AP inválida")
0030:
0031:     # garante SX5 para as séries informadas
0032:     if serie_nf is not None:
0033:         if sx5_obter_tx(cur, filial=filial, tabela="SERIE", chave="NF") is None:
0034:             sx5_criar_tx(cur, filial=filial, tabela="SERIE", chave="NF", descr=str(serie_nf).s
trip(), ativo=True)
0035:         validar_serie_tx(cur, filial=filial, tabela="NF", serie=str(serie_nf).strip())
0036:     if serie_ar is not None:
0037:         if sx5_obter_tx(cur, filial=filial, tabela="SERIE", chave="AR") is None:
0038:             sx5_criar_tx(cur, filial=filial, tabela="SERIE", chave="AR", descr=str(serie_ar).s
trip(), ativo=True)
0039:         validar_serie_tx(cur, filial=filial, tabela="AR", serie=str(serie_ar).strip())
0040:     if serie_ap is not None:
0041:         if sx5_obter_tx(cur, filial=filial, tabela="SERIE", chave="AP") is None:
0042:             sx5_criar_tx(cur, filial=filial, tabela="SERIE", chave="AP", descr=str(serie_ap).s
trip(), ativo=True)
0043:         validar_serie_tx(cur, filial=filial, tabela="AP", serie=str(serie_ap).strip())
0044:
0045:     sm0_definir_series_tx(cur, filial, serie_nf=serie_nf, serie_ar=serie_ar, serie_ap=serie_ap
)
0046:     return series_obter_uc_tx(cur, usuario, filial)
0047:
0048:
0049: def series_listar_sx5_uc_tx(cur, usuario: str, filial: str):
0050:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0051:     seed_series_basicas_tx(cur, filial)
0052:     return sx5_listar_tx(cur, filial=filial, tabela="SERIE", ativos=True)
0053:
0054:
0055: def series_reset_uc_tx(cur, usuario: str, filial: str):
0056:     exigir_perfil_tx(cur, usuario, {"admin"})
0057:     # garante SX5 básico e valida antes de resetar
0058:     seed_series_basicas_tx(cur, filial)
0059:     validar_serie_tx(cur, filial, "NF", "1")
0060:     validar_serie_tx(cur, filial, "AR", "1")
0061:     validar_serie_tx(cur, filial, "AP", "1")
0062:     sm0_definir_series_tx(cur, filial, serie_nf="1", serie_ar="1", serie_ap="1")
0063:     return series_obter_uc_tx(cur, usuario, filial)
```

==========
ARQUIVO: app/use_cases/sx5_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
```

```
0004: from app.infra.repositories.sx5_repo import (
0005:     sx5_listar_tx,
0006:     sx5_obter_tx,
0007:     sx5_criar_tx,
0008:     sx5_atualizar_tx,
0009:     sx5_inativar_tx,
0010: )
0011:
0012:
0013: def sx5_listar_uc_tx(cur, usuario: str, filial: str, tabela: str, ativos: bool = True):
0014:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0015:     return sx5_listar_tx(cur, filial=filial, tabela=tabela, ativos=ativos)
0016:
0017:
0018: def sx5_criar_uc_tx(cur, usuario: str, filial: str, tabela: str, chave: str, descr: str, ativo
: bool = True):
0019:     exigir_perfil_tx(cur, usuario, {"admin"})
0020:     sx5_criar_tx(cur, filial=filial, tabela=tabela, chave=chave, descr=descr, ativo=ativo)
0021:     return {"filial": filial, "tabela": tabela, "chave": chave, "descr": descr, "ativo": ativo
}
0022:
0023:
0024: def sx5_atualizar_uc_tx(cur, usuario: str, filial: str, tabela: str, chave: str, descr: str |
None, ativo: bool | None):
0025:     exigir_perfil_tx(cur, usuario, {"admin"})
0026:     sx5_atualizar_tx(cur, filial=filial, tabela=tabela, chave=chave, descr=descr, ativo=ativo)
0027:     return sx5_obter_tx(cur, filial=filial, tabela=tabela, chave=chave)
0028:
0029:
0030: def sx5_inativar_uc_tx(cur, usuario: str, filial: str, tabela: str, chave: str):
0031:     exigir_perfil_tx(cur, usuario, {"admin"})
0032:     sx5_inativar_tx(cur, filial=filial, tabela=tabela, chave=chave)
0033:     return {"filial": filial, "tabela": tabela, "chave": chave, "ativo": False}

==========================================================================================
ARQUIVO: app/use_cases/usuarios_uc.py
==========================================================================================
0001: from app.core.exceptions import AuthzError, BusinessError
0002: from app.core.security import hash_password
0003: from app.infra.repositories.app_setup_repo import (
0004:     app_setup_is_completed_tx,
0005:     app_setup_mark_completed_tx,
0006:     app_setup_status_tx,
0007: )
0008: from app.infra.repositories.logs_repo import log_tx
0009: from app.infra.repositories.usuarios_repo import inserir_usuario_tx, obter_perfil_tx, usuarios
_existem_tx
0010:
0011:
0012: def exigir_perfil_tx(cur, login: str, perfis: set[str]) -> str:
0013:     perfil = obter_perfil_tx(cur, login)
0014:     if not perfil:
0015:         raise BusinessError("Usuário não encontrado")
0016:     if perfil not in perfis:
```

```
0017:         raise AuthzError(f"Sem permissão. Perfil atual: {perfil}")
0018:     return perfil
0019:
0020:
0021: def criar_usuario_bootstrap_tx(cur, login: str, senha: str) -> None:
0022:     """
0023:     One-time setup policy:
0024:     - The route layer must only expose/bootstrap this flow in setup/development.
0025:     - Setup completion is persisted in the database and blocks future bootstrap attempts.
0026:     - Once any user exists, bootstrap is irreversibly blocked by business rule.
0027:     """
0028:     if app_setup_is_completed_tx(cur):
0029:         raise BusinessError("Bootstrap bloqueado: setup inicial já foi concluído.")
0030:     if usuarios_existem_tx(cur):
0031:         raise BusinessError("Bootstrap bloqueado: já existem usuários.")
0032:     inserir_usuario_tx(cur, login, hash_password(senha), "admin")
0033:     app_setup_mark_completed_tx(cur, login)
0034:     log_tx(cur, login, "Bootstrap admin criado")
0035:
0036:
0037: def criar_usuario_tx(cur, admin_login: str, login: str, senha: str, perfil: str) -> None:
0038:     exigir_perfil_tx(cur, admin_login, {"admin"})
0039:     inserir_usuario_tx(cur, login, hash_password(senha), perfil)
0040:     log_tx(cur, admin_login, f"Usuário criado: {login} (perfil={perfil})")
0041:
0042:
0043: def obter_setup_status_tx(cur, admin_login: str) -> dict:
0044:     exigir_perfil_tx(cur, admin_login, {"admin"})
0045:     return app_setup_status_tx(cur)
```

==========
ARQUIVO: app/use_cases/vendas/__init__.py
==========

```
0001:
```

==========
ARQUIVO: app/use_cases/vendas/buscar_nf_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.faturamento_repo import sf2_buscar_por_doc_serie_tx, sd2_itens_por
_nf_id_tx
0004:
0005:
0006: def buscar_nf_por_doc_serie_tx(
0007:     cur,
0008:     usuario: str,
0009:     filial: str,
0010:     doc: str,
0011:     serie: str,
0012: ) -> dict:
0013:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0014:
0015:     filial = (filial or "").strip() or "01"
```

```
0016:     doc = (doc or "").strip()
0017:     serie = (serie or "").strip()
0018:     if not doc or not serie:
0019:         raise BusinessError("doc e serie são obrigatórios")
0020:
0021:     nf = sf2_buscar_por_doc_serie_tx(cur, filial=filial, doc=doc, serie=serie)
0022:     if not nf:
0023:         raise BusinessError("NF não encontrada")
0024:     itens = sd2_itens_por_nf_id_tx(cur, int(nf["ID"]))
0025:     return {"nf": nf, "itens": itens}
```

==========
ARQUIVO: app/use_cases/vendas/cancelar_nf_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.faturamento_repo import sf2_get_tx, sf2_cancelar_tx, sd2_cancelar_
por_nf_tx, sd2_count_por_nf_tx
0004: from app.infra.repositories.financeiro_repo import cancelar_ar_por_nf_tx
0005: from app.infra.repositories.logs_repo import log_tx
0006: from app.infra.repositories.fechamento_repo import fechamento_validar_periodo_tx
0007:
0008:
0009: def cancelar_nf_tx(
0010:     cur,
0011:     usuario: str,
0012:     nf_id: int,
0013:     filial: str,
0014:     motivo: str | None = None,
0015: ) -> dict:
0016:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0017:
0018:     nf_id = int(nf_id)
0019:     filial = (filial or "").strip() or "01"
0020:
0021:     nf = sf2_get_tx(cur, nf_id)
0022:     if not nf:
0023:         raise BusinessError("NF não encontrada")
0024:
0025:     nf_filial = str(nf.get("F2_FILIAL") or "").strip()
0026:     if nf_filial and nf_filial != filial:
0027:         raise BusinessError("NF não pertence à filial informada")
0028:
0029:     status = str(nf.get("F2_STATUS") or "").strip().upper()
0030:     if status == "CANCELADA":
0031:         raise BusinessError("NF já está CANCELADA")
0032:
0033:     fechamento_validar_periodo_tx(cur, filial, nf["F2_EMISSAO"])
0034:
0035:     # 1) Cancela AR vinculado (se existir). Bloqueia se BAIXADO.
0036:     ar_cancelados = cancelar_ar_por_nf_tx(cur, filial=filial, nf_id=nf_id)
0037:
0038:     # 2) valida SD2
0039:     if sd2_count_por_nf_tx(cur, nf_id) == 0:
```

```
0040:         raise BusinessError("NF sem itens SD2 (consistência)")
0041:
0042:     # 3) Cancela SD2 da NF
0043:     sd2_canceladas = sd2_cancelar_por_nf_tx(cur, nf_id=nf_id, usuario=usuario, motivo=motivo)
0044:
0045:     # 4) Cancela SF2
0046:     sf2_cancelar_tx(cur, nf_id=nf_id, usuario=usuario, motivo=motivo)
0047:
0048:     log_tx(
0049:         cur,
0050:         usuario,
0051:         f"NF {nf_id} cancelada filial={filial} ar_cancelados={ar_cancelados} sd2_canceladas={s
d2_canceladas} motivo={motivo}",
0052:     )
0053:
0054:     return {
0055:         "nf_id": int(nf_id),
0056:         "filial": filial,
0057:         "status": "CANCELADA",
0058:         "ar_cancelados": int(ar_cancelados),
0059:         "sd2_canceladas": int(sd2_canceladas),
0060:     }
```

==========
ARQUIVO: app/use_cases/vendas/cancelar_pedido_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx, sc5_pedido_statu
s_tx
0005: from app.infra.repositories.vendas.sc6_itens_query_repo import sc6_itens_do_pedido_tx
0006: from app.infra.repositories.vendas.sc6_itens_repo import sc6_itens_cancelar_por_pedido_tx
0007: from app.infra.repositories.vendas.sc9_liberacao_repo import sc9_cancelar_por_pedido_tx
0008: from app.infra.repositories.vendas.origem_repo import venda_buscar_reserva_id_por_pedido_tx
0009:
0010: from app.infra.repositories.sd3_repo import sd3_mov_tx
0011: from app.infra.repositories.sb1_repo import sb1_get_tx, sb1_entrada_tx, sb1_reservar_atomico_t
x
0012: from app.infra.repositories.reservas_repo import reserva_status_tx  # se quiser reabrir reserv
a
0013: from app.infra.repositories.financeiro_repo import cancelar_ar_por_ref_tx
0014: from app.infra.repositories.faturamento_repo import (
0015:     sf2_buscar_por_pedido_tx,
0016:     sf2_cancelar_tx,
0017:     sd2_cancelar_por_nf_tx,
0018: )
0019: from app.infra.repositories.logs_repo import log_tx
0020:
0021: def cancelar_pedido_tx(
0022:     cur,
0023:     usuario: str,
0024:     pedido_id: int,
0025:     filial: str,
```

```
0026:     modo: str = "AUTO",   # AUTO | SIMPLES | ESTORNAR
0027:     reativar_reserva: bool = True,
0028: ) -> dict:
0029:     """
0030:     Cancela pedido SC5.
0031:
0032:     modo:
0033:       - SIMPLES: apenas muda status do SC5 -> CANCELADO (sem mexer em estoque/financeiro)
0034:       - ESTORNAR: faz estorno completo (SD3 entrada + devolve estoque + cancela AR)
0035:       - AUTO: se status FATURADO => ESTORNAR; senão => SIMPLES
0036:     """
0037:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0038:
0039:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0040:     if not p:
0041:         raise BusinessError("Pedido não encontrado")
0042:
0043:     if str(p.get("C5_FILIAL", "")).strip() != str(filial).strip():
0044:         raise BusinessError(f"Pedido {pedido_id} não pertence à filial {filial}")
0045:
0046:     status_atual = str(p.get("C5_STATUS", "")).strip().upper()
0047:     if status_atual == "CANCELADO":
0048:         raise BusinessError("Pedido já está CANCELADO")
0049:
0050:     modo2 = (modo or "AUTO").strip().upper()
0051:     if modo2 not in {"AUTO", "SIMPLES", "ESTORNAR"}:
0052:         raise BusinessError("modo inválido (AUTO/SIMPLES/ESTORNAR)")
0053:
0054:     if modo2 == "AUTO":
0055:         modo2 = "ESTORNAR" if status_atual == "FATURADO" else "SIMPLES"
0056:
0057:     # --- SIMPLES: só cancela SC5 ---
0058:     if modo2 == "SIMPLES":
0059:         sc5_pedido_status_tx(cur, int(pedido_id), "CANCELADO", usuario=usuario, motivo=None)
0060:         itens_cancelados = sc6_itens_cancelar_por_pedido_tx(cur, int(pedido_id), usuario=usuar
io)
0061:         sc9_canceladas = sc9_cancelar_por_pedido_tx(cur, int(pedido_id), usuario=usuario)
0062:         log_tx(cur, usuario, f"Pedido {pedido_id} cancelado (SIMPLES) filial={filial} status_a
nt={status_atual} sc6_cancelados={itens_cancelados}")
0063:         return {
0064:             "pedido_id": int(pedido_id),
0065:             "filial": filial,
0066:             "status": "CANCELADO",
0067:             "modo": "SIMPLES",
0068:             "sc6_cancelados": int(itens_cancelados),
0069:             "sc9_canceladas": int(sc9_canceladas),
0070:         }
0071:
0072:     # --- ESTORNAR: reverte estoque + SD3 + financeiro ---
0073:     itens = sc6_itens_do_pedido_tx(cur, int(pedido_id))
0074:     if not itens:
0075:         raise BusinessError("Pedido sem itens (SC6)")
0076:
0077:     # 1) cancelar AR (se existir). Se AR já baixado, bloqueia (ERP real).
```

```
0078:     ar_cancelados = cancelar_ar_por_ref_tx(cur, filial=filial, pedido_id=int(pedido_id))
0079:
0080:     # 1b) cancelar NF (SF2) associada ao pedido
0081:     nfs = sf2_buscar_por_pedido_tx(cur, filial=filial, pedido_id=int(pedido_id))
0082:     if not nfs:
0083:         raise BusinessError("Pedido FATURADO sem NF (consistência)")
0084:     sd2_canceladas = 0
0085:     for nf in nfs:
0086:         nf_id = int(nf["ID"])
0087:         sf2_cancelar_tx(cur, nf_id, usuario=usuario, motivo=f"Cancelamento pedido {pedido_id}"
)
0088:         sd2_canceladas += sd2_cancelar_por_nf_tx(
0089:             cur,
0090:             nf_id=nf_id,
0091:             usuario=usuario,
0092:             motivo=f"Cancelamento pedido {pedido_id}",
0093:         )
0094:
0095:     # 2) estornar SD3 e devolver estoque por item
0096:     valor_cmv_estornado = 0.0
0097:     qtd_total_itens = 0
0098:
0099:     for it in itens:
0100:         produto = str(it.get("C6_PRODUTO") or "").strip()
0101:         qtd = int(it.get("C6_QTD") or 0)
0102:         if not produto or qtd <= 0:
0103:             raise BusinessError("Item SC6 inválido para estorno")
0104:
0105:         sb1 = sb1_get_tx(cur, produto, filial)
0106:         cm = float(sb1["B1_CM"])
0107:         cmv_item = cm * qtd
0108:         valor_cmv_estornado += cmv_item
0109:         qtd_total_itens += qtd
0110:
0111:         # SD3 entrada (estorno do custo)
0112:         sd3_mov_tx(
0113:             cur,
0114:             filial=filial,
0115:             produto=produto,
0116:             tipo="E",
0117:             qtd=qtd,
0118:             custo_unit=cm,
0119:             valor=cmv_item,
0120:             origem="PEDIDO_CANCELADO",
0121:             ref=str(pedido_id),
0122:             usuario=usuario,
0123:         )
0124:
0125:         # devolve estoque físico
0126:         sb1_entrada_tx(cur, produto, filial, qtd)
0127:
0128:     # 3) Se o pedido veio de reserva, opcionalmente reabre e reserva de volta
0129:     reserva_id = venda_buscar_reserva_id_por_pedido_tx(cur, int(pedido_id))
0130:     if reserva_id is not None and reativar_reserva:
```

```
0131:         # reabrir reserva (regra: volta para ABERTA)
0132:         reserva_status_tx(cur, int(reserva_id), "ABERTA")
0133:
0134:         # re-reservar exatamente as quantidades do pedido
0135:         for it in itens:
0136:             produto = str(it.get("C6_PRODUTO") or "").strip()
0137:             qtd = int(it.get("C6_QTD") or 0)
0138:             if produto and qtd > 0:
0139:                 sb1_reservar_atomico_tx(cur, produto, filial, qtd)
0140:
0141:     # 4) cancelar SC5 + itens
0142:     sc5_pedido_status_tx(cur, int(pedido_id), "CANCELADO", usuario=usuario, motivo=None)
0143:     itens_cancelados = sc6_itens_cancelar_por_pedido_tx(cur, int(pedido_id), usuario=usuario)
0144:     sc9_canceladas = sc9_cancelar_por_pedido_tx(cur, int(pedido_id), usuario=usuario)
0145:
0146:     log_tx(
0147:         cur,
0148:         usuario,
0149:         f"Pedido {pedido_id} cancelado (ESTORNAR) filial={filial} ar_cancelados={ar_cancelados
} nf_canceladas={len(nfs)} sd2_canceladas={sd2_canceladas} sc6_cancelados={itens_cancelados} sc9_can
celadas={sc9_canceladas} cmv_estornado={valor_cmv_estornado} reserva={reserva_id}",
0150:     )
0151:
0152:     return {
0153:         "pedido_id": int(pedido_id),
0154:         "filial": filial,
0155:         "status": "CANCELADO",
0156:         "modo": "ESTORNAR",
0157:         "status_anterior": status_atual,
0158:         "ar_cancelados": int(ar_cancelados),
0159:         "nf_canceladas": int(len(nfs)),
0160:         "sd2_canceladas": int(sd2_canceladas),
0161:         "sc6_cancelados": int(itens_cancelados),
0162:         "sc9_canceladas": int(sc9_canceladas),
0163:         "cmv_estornado": float(valor_cmv_estornado),
0164:         "reserva_id": int(reserva_id) if reserva_id is not None else None,
0165:         "reserva_reativada": bool(reserva_id is not None and reativar_reserva),
0166:     }
```

==========
ARQUIVO: app/use_cases/vendas/consultar_pedido_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx
0005: from app.infra.repositories.vendas.sc6_itens_query_repo import sc6_itens_do_pedido_tx
0006: from app.infra.repositories.financeiro_repo import ar_listar_por_pedido_tx
0007:
0008: def consultar_pedido_tx(cur, usuario: str, pedido_id: int) -> dict:
0009:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0010:
0011:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0012:     if not p:
```

```
0013:         raise BusinessError("Pedido não encontrado")
0014:
0015:     itens = sc6_itens_do_pedido_tx(cur, int(pedido_id))
0016:     ar = ar_listar_por_pedido_tx(cur, int(pedido_id))
0017:
0018:     # resumo simples do AR (último título, se existir)
0019:     ar_resumo = None
0020:     if ar:
0021:         t = ar[0]
0022:         ar_resumo = {
0023:             "ar_id": int(t["ID"]),
0024:             "status": t.get("E1_STATUS"),
0025:             "valor": float(t.get("E1_VALOR") or 0),
0026:             "venc": t.get("E1_VENC"),
0027:             "se5_id": t.get("E1_SE5_ID"),
0028:         }
0029:
0030:     return {
0031:         "pedido": p,
0032:         "itens": itens,
0033:         "ar": ar,               # lista completa (auditoria)
0034:         "ar_resumo": ar_resumo, # fácil pro front
0035:     }
```

==========
ARQUIVO: app/use_cases/vendas/devolver_nf_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.faturamento_repo import (
0004:     sf2_get_tx,
0005:     sf2_criar_tx,
0006:     sd2_item_criar_tx,
0007:     sd2_itens_por_nf_tx,
0008: )
0009: from app.infra.repositories.fiscal_repo import fiscal_get_tes_tx
0010: from app.infra.repositories.sb1_repo import sb1_get_tx, sb1_entrada_tx
0011: from app.infra.repositories.sd3_repo import sd3_mov_tx
0012: from app.infra.repositories.logs_repo import log_tx
0013: from app.infra.repositories.fechamento_repo import fechamento_validar_periodo_tx
0014: from app.infra.repositories.financeiro_repo import ar_criar_tx, ap_criar_tx
0015:
0016:
0017: def devolver_nf_tx(
0018:     cur,
0019:     usuario: str,
0020:     nf_origem_id: int,
0021:     filial: str,
0022:     tes_cod: str,
0023:     venc_dias: int = 30,
0024:     titulo_tipo: str = "AP",  # compat: ignorado (Protheus-like)
0025: ) -> dict:
0026:     """
0027:     Gera NF de devolução (entrada) a partir de uma NF de venda.
```

```
0028:     - TES deve ser tipo 'E'.
0029:     - Gera SD2 e, se configurado, entrada de estoque + SD3.
0030:     """
0031:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0032:
0033:     filial = (filial or "").strip() or "01"
0034:     nf_origem_id = int(nf_origem_id)
0035:     tes_cod = (tes_cod or "").strip()
0036:     if not tes_cod:
0037:         raise BusinessError("TES é obrigatório para devolução")
0038:
0039:     nf_origem = sf2_get_tx(cur, nf_origem_id)
0040:     if not nf_origem:
0041:         raise BusinessError("NF origem não encontrada")
0042:
0043:     nf_filial = str(nf_origem.get("F2_FILIAL") or "").strip()
0044:     if nf_filial and nf_filial != filial:
0045:         raise BusinessError("NF origem não pertence à filial informada")
0046:
0047:     if str(nf_origem.get("F2_STATUS") or "").strip().upper() == "CANCELADA":
0048:         raise BusinessError("NF origem está CANCELADA")
0049:
0050:     fechamento_validar_periodo_tx(cur, filial, nf_origem["F2_EMISSAO"])
0051:
0052:     itens_origem = sd2_itens_por_nf_tx(cur, nf_origem_id, somente_ativos=True)
0053:     if not itens_origem:
0054:         raise BusinessError("NF origem sem itens ativos")
0055:
0056:     # TES para devolução (entrada)
0057:     tes = fiscal_get_tes_tx(cur, filial=filial, tes_cod=tes_cod)
0058:     tes_tipo = str(tes.get("F4_TIPO") or "").strip().upper()
0059:     if tes_tipo != "E":
0060:         raise BusinessError("TES incompatível para devolução (F4_TIPO deve ser 'E')")
0061:     cfop = (tes.get("F4_CFOP") or "").strip()
0062:     if not cfop:
0063:         raise BusinessError("TES sem CFOP configurado (F4_CFOP)")
0064:
0065:     gera_estoque = bool(tes.get("F4_GERA_EST"))
0066:     gera_titulo = bool(tes.get("F4_GERA_TIT"))
0067:
0068:     # Totais a partir da NF origem
0069:     valor_total = float(nf_origem.get("F2_VALOR") or 0)
0070:     valor_icms = float(nf_origem.get("F2_ICMS") or 0)
0071:     valor_ipi = float(nf_origem.get("F2_IPI") or 0)
0072:     total_bruto = float(nf_origem.get("F2_TOTAL_BRUTO") or 0)
0073:
0074:     nf_dev = sf2_criar_tx(
0075:         cur,
0076:         filial=filial,
0077:         pedido_id=nf_origem.get("F2_PEDIDO_ID"),
0078:         cliente=nf_origem.get("F2_CLIENTE"),
0079:         cliente_cod=nf_origem.get("F2_CLIENTE_COD"),
0080:         valor_total=valor_total,
0081:         icms=valor_icms,
```

```
0082:         ipi=valor_ipi,
0083:         total_bruto=total_bruto,
0084:         tes_cod=tes_cod,
0085:         cfop=cfop,
0086:         origem="DEVOLUCAO",
0087:     )
0088:
0089:     # Itens SD2 da devolução
0090:     for it in itens_origem:
0091:         item_no = int(it.get("D2_ITEM") or 1)
0092:         sd2_item_criar_tx(
0093:             cur,
0094:             nf_id=nf_dev["id"],
0095:             doc=nf_dev["doc"],
0096:             serie=nf_dev["serie"],
0097:             item=item_no,
0098:             filial=filial,
0099:             pedido_id=nf_origem.get("F2_PEDIDO_ID"),
0100:             produto=str(it["D2_PRODUTO"]).strip(),
0101:             qtd=int(it["D2_QTD"]),
0102:             preco_unit=(float(it["D2_PRECO_UNIT"]) if it.get("D2_PRECO_UNIT") is not None else
 None),
0103:             total=float(it["D2_TOTAL"]),
0104:             icms=float(it["D2_ICMS"]),
0105:             ipi=float(it["D2_IPI"]),
0106:             tes_cod=tes_cod,
0107:             cfop=cfop,
0108:         )
0109:
0110:     # Entrada de estoque + SD3 (se configurado)
0111:     valor_cmv_total = 0.0
0112:     if gera_estoque:
0113:         for it in itens_origem:
0114:             produto = str(it["D2_PRODUTO"]).strip()
0115:             qtd = int(it["D2_QTD"])
0116:             if qtd <= 0:
0117:                 continue
0118:             sb1 = sb1_get_tx(cur, produto, filial)
0119:             cm_unit = float(sb1["B1_CM"])
0120:             valor_cmv = cm_unit * qtd
0121:             valor_cmv_total += valor_cmv
0122:
0123:             sb1_entrada_tx(cur, produto, filial, qtd)
0124:             sd3_mov_tx(
0125:                 cur,
0126:                 filial=filial,
0127:                 produto=produto,
0128:                 tipo="E",
0129:                 qtd=qtd,
0130:                 custo_unit=cm_unit,
0131:                 valor=valor_cmv,
0132:                 origem="NF_DEVOLUCAO",
0133:                 ref=str(nf_dev["id"]),
0134:                 usuario=usuario,
```

```
0135:             )
0136:
0137:     # Geração financeira (devolução)
0138:     titulo_gerado = False
0139:     # Protheus-like: devolução (TES tipo E) gera AP quando configurado
0140:     if gera_titulo:
0141:         ap_criar_tx(
0142:             cur,
0143:             filial=filial,
0144:             forn=nf_origem.get("F2_CLIENTE"),
0145:             forn_cod=nf_origem.get("F2_CLIENTE_COD"),
0146:             valor=float(valor_total),
0147:             ref=str(nf_dev["id"]),
0148:             venc_dias=int(venc_dias),
0149:         )
0150:         titulo_gerado = True
0151:
0152:     log_tx(
0153:         cur,
0154:         usuario,
0155:         f"NF devolução gerada nf_origem={nf_origem_id} nf_dev={nf_dev['doc']}/{nf_dev['serie']
} tes={tes_cod} cfop={cfop} cmv={valor_cmv_total}",
0156:     )
0157:
0158:     return {
0159:         "nf_origem_id": int(nf_origem_id),
0160:         "nf_dev_id": int(nf_dev["id"]),
0161:         "nf_dev_doc": nf_dev["doc"],
0162:         "nf_dev_serie": nf_dev["serie"],
0163:         "tes": tes_cod,
0164:         "cfop": cfop,
0165:         "valor_total": float(valor_total),
0166:         "icms": float(valor_icms),
0167:         "ipi": float(valor_ipi),
0168:         "total_bruto": float(total_bruto),
0169:         "valor_cmv_entrada": float(round(valor_cmv_total, 2)),
0170:         "titulo_gerado": bool(titulo_gerado),
0171:         "titulo_tipo": "AP" if gera_titulo else None,
0172:     }
```

==========
ARQUIVO: app/use_cases/vendas/estornar_nf_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.faturamento_repo import (
0004:     sf2_get_tx,
0005:     sf2_cancelar_tx,
0006:     sd2_cancelar_por_nf_tx,
0007:     sd2_itens_por_nf_tx,
0008:     sd2_count_por_nf_tx,
0009: )
0010: from app.infra.repositories.financeiro_repo import cancelar_ar_por_nf_tx
0011: from app.infra.repositories.fiscal_repo import fiscal_get_tes_tx
```

```
0012: from app.infra.repositories.sb1_repo import sb1_get_tx, sb1_entrada_tx
0013: from app.infra.repositories.sd3_repo import sd3_mov_tx
0014: from app.infra.repositories.logs_repo import log_tx
0015: from app.infra.repositories.fechamento_repo import fechamento_validar_periodo_tx
0016:
0017:
0018: def estornar_nf_tx(
0019:     cur,
0020:     usuario: str,
0021:     nf_id: int,
0022:     filial: str,
0023:     motivo: str | None = None,
0024: ) -> dict:
0025:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0026:
0027:     nf_id = int(nf_id)
0028:     filial = (filial or "").strip() or "01"
0029:
0030:     nf = sf2_get_tx(cur, nf_id)
0031:     if not nf:
0032:         raise BusinessError("NF não encontrada")
0033:
0034:     nf_filial = str(nf.get("F2_FILIAL") or "").strip()
0035:     if nf_filial and nf_filial != filial:
0036:         raise BusinessError("NF não pertence à filial informada")
0037:
0038:     status = str(nf.get("F2_STATUS") or "").strip().upper()
0039:     if status == "CANCELADA":
0040:         raise BusinessError("NF já está CANCELADA")
0041:
0042:     tes_cod = str(nf.get("F2_TES") or "").strip()
0043:     gera_estoque = True
0044:     if tes_cod:
0045:         tes = fiscal_get_tes_tx(cur, filial=filial, tes_cod=tes_cod)
0046:         gera_estoque = bool(tes.get("F4_GERA_EST"))
0047:
0048:     fechamento_validar_periodo_tx(cur, filial, nf["F2_EMISSAO"])
0049:
0050:     # 1) Cancela AR vinculado (se existir). Bloqueia se BAIXADO.
0051:     ar_cancelados = cancelar_ar_por_nf_tx(cur, filial=filial, nf_id=nf_id)
0052:
0053:     # 2) valida SD2
0054:     if sd2_count_por_nf_tx(cur, nf_id) == 0:
0055:         raise BusinessError("NF sem itens SD2 (consistência)")
0056:
0057:     # 3) Estorno de estoque/SD3 (somente itens não cancelados)
0058:     sd2_itens = sd2_itens_por_nf_tx(cur, nf_id=nf_id, somente_ativos=True)
0059:     valor_cmv_total = 0.0
0060:     if gera_estoque and sd2_itens:
0061:         for it in sd2_itens:
0062:             produto = str(it["D2_PRODUTO"]).strip()
0063:             qtd = int(it["D2_QTD"])
0064:             if qtd <= 0:
0065:                 continue
```

```
0066:             sb1 = sb1_get_tx(cur, produto, filial)
0067:             cm_unit = float(sb1["B1_CM"])
0068:             valor_cmv = cm_unit * qtd
0069:             valor_cmv_total += valor_cmv
0070:
0071:             sb1_entrada_tx(cur, produto, filial, qtd)
0072:             sd3_mov_tx(
0073:                 cur,
0074:                 filial=filial,
0075:                 produto=produto,
0076:                 tipo="E",
0077:                 qtd=qtd,
0078:                 custo_unit=cm_unit,
0079:                 valor=valor_cmv,
0080:                 origem="NF_ESTORNO",
0081:                 ref=str(nf_id),
0082:                 usuario=usuario,
0083:             )
0084:
0085:     # 4) Cancela SD2 e SF2
0086:     sd2_canceladas = sd2_cancelar_por_nf_tx(cur, nf_id=nf_id, usuario=usuario, motivo=motivo)
0087:     sf2_cancelar_tx(cur, nf_id=nf_id, usuario=usuario, motivo=motivo)
0088:
0089:     log_tx(
0090:         cur,
0091:         usuario,
0092:         f"NF {nf_id} ESTORNADA filial={filial} ar_cancelados={ar_cancelados} sd2_canceladas={s
d2_canceladas} cmv={valor_cmv_total} motivo={motivo}",
0093:     )
0094:
0095:     return {
0096:         "nf_id": int(nf_id),
0097:         "filial": filial,
0098:         "status": "CANCELADA",
0099:         "ar_cancelados": int(ar_cancelados),
0100:         "sd2_canceladas": int(sd2_canceladas),
0101:         "valor_cmv_estornado": float(round(valor_cmv_total, 2)),
0102:     }
```

==========
ARQUIVO: app/use_cases/vendas/estornar_pedido_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx, sc5_pedido_cance
lar_faturado_tx
0005: from app.infra.repositories.vendas.sc6_itens_repo import sc6_itens_do_pedido_tx
0006:
0007: from app.infra.repositories.sb1_repo import sb1_get_tx, sb1_entrada_tx
0008: from app.infra.repositories.sd3_repo import sd3_mov_tx
0009:
0010: from app.infra.repositories.financeiro_repo import ar_ultimo_por_pedido_tx, ar_cancelar_atomic
o_tx
```

```
0011: from app.infra.repositories.faturamento_repo import (
0012:     sf2_buscar_por_pedido_tx,
0013:     sf2_cancelar_tx,
0014:     sd2_cancelar_por_nf_tx,
0015: )
0016: from app.infra.repositories.vendas.sc9_liberacao_repo import sc9_cancelar_por_pedido_tx
0017: from app.infra.repositories.logs_repo import log_tx
0018:
0019: def estornar_pedido_faturado_venda_tx(
0020:     cur,
0021:     usuario: str,
0022:     pedido_id: int,
0023:     filial: str,
0024:     motivo: str | None = None,
0025: ) -> dict:
0026:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0027:
0028:     filial = (filial or "").strip() or "01"
0029:     pedido_id = int(pedido_id)
0030:
0031:     p = sc5_pedido_get_tx(cur, pedido_id)
0032:     if not p:
0033:         raise BusinessError("Pedido não encontrado")
0034:
0035:     if str(p.get("C5_FILIAL") or "").strip() != filial:
0036:         raise BusinessError(f"Pedido {pedido_id} não pertence à filial {filial}")
0037:
0038:     status = str(p.get("C5_STATUS") or "").strip().upper()
0039:     if status != "FATURADO":
0040:         raise BusinessError("Apenas pedido FATURADO pode ser estornado")
0041:
0042:     origem = str(p.get("C5_ORIGEM") or "VENDA").strip().upper()
0043:     if origem != "VENDA":
0044:         raise BusinessError("Este estorno é apenas para pedidos de origem VENDA")
0045:
0046:     # 1) AR: precisa existir e estar ABERTO
0047:     ar = ar_ultimo_por_pedido_tx(cur, pedido_id)
0048:     if not ar:
0049:         raise BusinessError("Não existe título AR para este pedido (SE1)")
0050:     ar_status = str(ar.get("E1_STATUS") or "").strip().upper()
0051:
0052:     if ar_status == "BAIXADO":
0053:         raise BusinessError("Não é permitido estornar: AR já BAIXADO (exige contra-lançamento)
")
0054:     if ar_status != "ABERTO":
0055:         raise BusinessError(f"Não é permitido estornar: AR status={ar_status}")
0056:
0057:     # 2) Itens (ativos)
0058:     itens = sc6_itens_do_pedido_tx(cur, pedido_id)
0059:     if not itens:
0060:         raise BusinessError("Pedido sem itens ativos (SC6)")
0061:
0062:     # 3) Devolver estoque + SD3 entrada (inverso do faturamento)
0063:     valor_cmv_total = 0.0
```

```
0064:     for it in itens:
0065:         prod = str(it["C6_PRODUTO"]).strip()
0066:         qtd = int(it["C6_QTD"])
0067:
0068:         sb1 = sb1_get_tx(cur, prod, filial)
0069:         cm_unit = float(sb1["B1_CM"])
0070:         valor_cmv = cm_unit * qtd
0071:         valor_cmv_total += valor_cmv
0072:
0073:         # devolve estoque (entrada)
0074:         sb1_entrada_tx(cur, prod, filial, qtd)
0075:
0076:         # SD3 entrada
0077:         sd3_mov_tx(
0078:             cur,
0079:             filial=filial,
0080:             produto=prod,
0081:             tipo="E",
0082:             qtd=qtd,
0083:             custo_unit=cm_unit,
0084:             valor=valor_cmv,
0085:             origem="ESTORNO_PEDIDO",
0086:             ref=str(pedido_id),
0087:             usuario=usuario,
0088:         )
0089:
0090:     # 4) Cancela AR
0091:     ar_cancelar_atomico_tx(cur, int(ar["ID"]))
0092:
0093:     # 4b) Cancela NF(s)
0094:     nfs = sf2_buscar_por_pedido_tx(cur, filial=filial, pedido_id=int(pedido_id))
0095:     sd2_canceladas = 0
0096:     for nf in nfs:
0097:         nf_id = int(nf["ID"])
0098:         sf2_cancelar_tx(cur, nf_id, usuario=usuario, motivo=motivo or f"Estorno pedido {pedido
_id}")
0099:         sd2_canceladas += sd2_cancelar_por_nf_tx(
0100:             cur,
0101:             nf_id=nf_id,
0102:             usuario=usuario,
0103:             motivo=motivo or f"Estorno pedido {pedido_id}",
0104:         )
0105:
0106:     sc9_canceladas = sc9_cancelar_por_pedido_tx(cur, int(pedido_id), usuario=usuario)
0107:
0108:     # 5) Cancela pedido FATURADO (vira CANCELADO)
0109:     sc5_pedido_cancelar_faturado_tx(cur, pedido_id, usuario=usuario, motivo=motivo)
0110:
0111:     log_tx(cur, usuario, f"Pedido {pedido_id} ESTORNADO (VENDA) motivo={motivo} ar_id={ar['ID'
]} nf_canceladas={len(nfs)} sd2_canceladas={sd2_canceladas} sc9_canceladas={sc9_canceladas} cmv={val
or_cmv_total}")
0112:
0113:     return {
0114:         "pedido_id": pedido_id,
```

```
0115:         "filial": filial,
0116:         "status_novo": "CANCELADO",
0117:         "ar_id": int(ar["ID"]),
0118:         "ar_status_novo": "CANCELADO",
0119:         "nf_canceladas": int(len(nfs)),
0120:         "sd2_canceladas": int(sd2_canceladas),
0121:         "sc9_canceladas": int(sc9_canceladas),
0122:         "valor_cmv_estornado": float(round(valor_cmv_total, 2)),
0123:         "motivo": motivo,
0124:     }
```

==========
ARQUIVO: app/use_cases/vendas/faturamento_uc.py
==========

```
0001: from datetime import date
0002:
0003: from app.core.exceptions import BusinessError
0004: from app.infra.repositories.faturamento_repo import sd2_item_criar_tx, sf2_criar_tx
0005: from app.infra.repositories.fechamento_repo import fechamento_validar_periodo_tx
0006: from app.infra.repositories.financeiro_repo import ar_criar_tx
0007: from app.infra.repositories.fiscal_repo import fiscal_get_tes_tx, fiscal_resolver_item_tx
0008: from app.infra.repositories.logs_repo import log_tx
0009: from app.infra.repositories.preco_repo import buscar_preco_vigente_tx
0010: from app.infra.repositories.reservas_repo import (
0011:     reserva_get_tx,
0012:     reserva_set_cliente_preco_tx,
0013:     reserva_status_tx,
0014: )
0015: from app.infra.repositories.sa1_repo import sa1_get_tx
0016: from app.infra.repositories.sb1_repo import (
0017:     sb1_baixar_e_desreservar_atomico_tx,
0018:     sb1_get_tx,
0019: )
0020: from app.infra.repositories.sd3_repo import saldo_sd3_tx, sd3_mov_tx
0021: from app.infra.repositories.sm0_repo import sm0_existe_tx
0022: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_criar_tx
0023: from app.infra.repositories.vendas.sc6_pedidos_repo import sc6_item_criar_tx
0024: from app.infra.repositories.vendas.sc9_liberacao_repo import sc9_liberar_tx
0025: from app.use_cases.usuarios_uc import exigir_perfil_tx
0026:
0027:
0028: def faturar_reserva_tx(
0029:     cur,
0030:     usuario: str,
0031:     reserva_id: int,
0032:     cliente_cod: str | None,
0033:     venc_dias: int,
0034:     tes_cod: str = "001",
0035: ) -> dict:
0036:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0037:
0038:     reserva = reserva_get_tx(cur, int(reserva_id))
0039:     if not reserva:
0040:         raise BusinessError("Reserva não encontrada")
```

```
0041:     if reserva["Z0_STATUS"] != "ABERTA":
0042:         raise BusinessError("Apenas reserva ABERTA pode ser confirmada")
0043:
0044:     produto = reserva["Z0_PRODUTO"]
0045:     filial = reserva["Z0_FILIAL"]
0046:     qtd = int(reserva["Z0_QTD"])
0047:     if not sm0_existe_tx(cur, filial):
0048:         raise BusinessError("Filial não cadastrada (SM0_EMPRESA)")
0049:     fechamento_validar_periodo_tx(cur, filial, date.today())
0050:     produto_snapshot = sb1_get_tx(cur, produto, filial)
0051:
0052:     cliente_nome = None
0053:     if cliente_cod:
0054:         cliente = sa1_get_tx(cur, filial=filial, cod=cliente_cod)
0055:         if not cliente:
0056:             raise BusinessError("Cliente não encontrado (SA1)")
0057:         cliente_nome = cliente["A1_NOME"]
0058:
0059:     preco_unit = reserva.get("Z0_PRECO_UNIT")
0060:     total = reserva.get("Z0_TOTAL")
0061:     tabela_id = reserva.get("Z0_TABELA_ID")
0062:
0063:     if preco_unit is not None and total is not None:
0064:         preco = float(preco_unit)
0065:         valor_venda = float(total)
0066:     else:
0067:         preco = float(produto_snapshot["B1_PRECO"])
0068:         if tabela_id is not None:
0069:             try:
0070:                 preco = float(
0071:                     buscar_preco_vigente_tx(
0072:                         cur,
0073:                         filial=filial,
0074:                         tabela_id=int(tabela_id),
0075:                         produto=produto,
0076:                         data=date.today(),
0077:                     )
0078:                 )
0079:             except BusinessError:
0080:                 pass
0081:
0082:         valor_venda = preco * qtd
0083:
0084:         reserva_set_cliente_preco_tx(
0085:             cur,
0086:             reserva_id=int(reserva_id),
0087:             cliente_cod=cliente_cod,
0088:             tabela_id=(int(tabela_id) if tabela_id is not None else None),
0089:             preco_unit=float(preco),
0090:             total=float(valor_venda),
0091:         )
0092:
0093:     cm_unit = float(produto_snapshot["B1_CM"])
0094:     valor_cmv = cm_unit * qtd
```

```
0095:
0096:     tes = fiscal_get_tes_tx(cur, filial, tes_cod=tes_cod)
0097:     tes_cod = str(tes.get("F4_COD") or tes_cod).strip()
0098:     tes_tipo = str(tes.get("F4_TIPO") or "").strip().upper()
0099:     cfop_base = (tes.get("F4_CFOP") or "").strip()
0100:
0101:     if tes_tipo and tes_tipo != "S":
0102:         raise BusinessError("TES incompatível para venda (F4_TIPO deve ser 'S')")
0103:     if not cfop_base:
0104:         raise BusinessError("TES sem CFOP configurado (F4_CFOP)")
0105:
0106:     fiscal_item = fiscal_resolver_item_tx(
0107:         cur,
0108:         filial=filial,
0109:         tes_cod=tes_cod,
0110:         cliente_cod=cliente_cod,
0111:         produto=produto,
0112:     )
0113:     valor_icms = round(valor_venda * float(fiscal_item["icms"]), 2)
0114:     valor_ipi = round(valor_venda * float(fiscal_item["ipi"]), 2)
0115:     valor_pis = round(valor_venda * float(fiscal_item["pis"]), 2)
0116:     valor_cofins = round(valor_venda * float(fiscal_item["cofins"]), 2)
0117:     valor_icms_st = round(valor_venda * float(fiscal_item["icms_st"]), 2)
0118:     valor_difal = round(valor_venda * float(fiscal_item["difal"]), 2)
0119:     total_bruto = round(valor_venda + valor_ipi + valor_icms_st + valor_difal, 2)
0120:     gera_titulo = bool(fiscal_item["gera_tit"])
0121:     gera_estoque = bool(fiscal_item["gera_est"])
0122:     cfop_item = str(fiscal_item["cfop"] or cfop_base).strip()
0123:
0124:     pedido_id = sc5_pedido_criar_tx(
0125:         cur,
0126:         filial,
0127:         valor_venda,
0128:         usuario,
0129:         status="ABERTO",
0130:         icms=valor_icms,
0131:         ipi=valor_ipi,
0132:         total_bruto=total_bruto,
0133:     )
0134:
0135:     cur.execute(
0136:         "UPDATE dbo.SZ0_RESERVA SET Z0_PEDIDO_ID=? WHERE ID=?",
0137:         (int(pedido_id), int(reserva_id)),
0138:     )
0139:
0140:     item = sc6_item_criar_tx(cur, pedido_id, produto, qtd, total=valor_venda, preco_unit=preco
)
0141:
0142:     sc9_liberar_tx(
0143:         cur,
0144:         pedido_id=pedido_id,
0145:         filial=filial,
0146:         produto=produto,
0147:         qtd=qtd,
```

```
0148:         usuario=usuario,
0149:         item=int(item["item"]) if item else None,
0150:     )
0151:
0152:     nota = sf2_criar_tx(
0153:         cur,
0154:         filial=filial,
0155:         pedido_id=pedido_id,
0156:         cliente=cliente_nome,
0157:         cliente_cod=cliente_cod,
0158:         valor_total=valor_venda,
0159:         icms=valor_icms,
0160:         ipi=valor_ipi,
0161:         total_bruto=total_bruto,
0162:         tes_cod=tes_cod,
0163:         cfop=cfop_base,
0164:         origem="RESERVA",
0165:         pis=valor_pis,
0166:         cofins=valor_cofins,
0167:         icms_st=valor_icms_st,
0168:         difal=valor_difal,
0169:     )
0170:
0171:     sd2_item_criar_tx(
0172:         cur,
0173:         nf_id=nota["id"],
0174:         doc=nota["doc"],
0175:         serie=nota["serie"],
0176:         item=int(item["item"]) if item else 1,
0177:         filial=filial,
0178:         pedido_id=pedido_id,
0179:         produto=produto,
0180:         qtd=qtd,
0181:         preco_unit=preco,
0182:         total=valor_venda,
0183:         icms=valor_icms,
0184:         ipi=valor_ipi,
0185:         tes_cod=tes_cod,
0186:         cfop=cfop_item,
0187:         pis=valor_pis,
0188:         cofins=valor_cofins,
0189:         icms_st=valor_icms_st,
0190:         difal=valor_difal,
0191:         cst_icms=fiscal_item.get("cst_icms"),
0192:         csosn=fiscal_item.get("csosn"),
0193:         cst_pis=fiscal_item.get("cst_pis"),
0194:         cst_cofins=fiscal_item.get("cst_cofins"),
0195:     )
0196:
0197:     if gera_estoque:
0198:         sd3_mov_tx(
0199:             cur,
0200:             filial,
0201:             produto,
```

```
0202:             "S",
0203:             qtd,
0204:             cm_unit,
0205:             valor_cmv,
0206:             "NF_EMITIDA",
0207:             str(nota["id"]),
0208:             usuario,
0209:         )
0210:         sb1_baixar_e_desreservar_atomico_tx(cur, produto, filial, qtd)
0211:
0212:     reserva_status_tx(cur, int(reserva_id), "CONFIRMADA")
0213:
0214:     if gera_titulo:
0215:         ar_criar_tx(
0216:             cur,
0217:             filial=filial,
0218:             cliente=cliente_nome,
0219:             cliente_cod=cliente_cod,
0220:             valor=valor_venda,
0221:             ref_id=nota["id"],
0222:             venc_dias=int(venc_dias),
0223:             serie=nota["serie"],
0224:         )
0225:
0226:     log_tx(
0227:         cur,
0228:         usuario,
0229:         f"Reserva {reserva_id} confirmada -> pedido {pedido_id} nf={nota['doc']}/{nota['serie'
]} tes={tes_cod} cfop={cfop_item}",
0230:     )
0231:
0232:     saldo_snapshot = sb1_get_tx(cur, produto, filial)
0233:     saldo_snapshot["saldo_sd3"] = saldo_sd3_tx(cur, produto, filial)
0234:
0235:     return {
0236:         "reserva_id": int(reserva_id),
0237:         "pedido_id": int(pedido_id),
0238:         "nf_id": int(nota["id"]),
0239:         "nf_doc": nota["doc"],
0240:         "nf_serie": nota["serie"],
0241:         "tes": tes_cod,
0242:         "cfop": cfop_item,
0243:         "cliente_cod": cliente_cod,
0244:         "cliente_nome": cliente_nome,
0245:         "preco_unit": float(preco),
0246:         "valor_venda": float(valor_venda),
0247:         "valor_cmv": float(valor_cmv),
0248:         "icms": float(valor_icms),
0249:         "ipi": float(valor_ipi),
0250:         "pis": float(valor_pis),
0251:         "cofins": float(valor_cofins),
0252:         "icms_st": float(valor_icms_st),
0253:         "difal": float(valor_difal),
0254:         "total_bruto": float(total_bruto),
```

```
0255:         "sb1": saldo_snapshot,
0256:     }
```

==========
ARQUIVO: app/use_cases/vendas/faturar_pedido_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.faturamento_repo import sd2_item_criar_tx, sf2_criar_tx
0005: from app.infra.repositories.fechamento_repo import fechamento_validar_periodo_tx
0006: from app.infra.repositories.financeiro_repo import ar_criar_tx
0007: from app.infra.repositories.fiscal_repo import fiscal_get_tes_tx, fiscal_resolver_item_tx
0008: from app.infra.repositories.logs_repo import log_tx
0009: from app.infra.repositories.sa1_repo import sa1_get_tx
0010: from app.infra.repositories.sb1_repo import sb1_baixar_atomico_tx, sb1_get_tx
0011: from app.infra.repositories.sd3_repo import sd3_mov_tx
0012: from app.infra.repositories.sm0_repo import sm0_existe_tx
0013: from app.infra.repositories.vendas.sc5_pedidos_repo import (
0014:     sc5_pedido_atualizar_valor_total_tx,
0015:     sc5_pedido_get_tx,
0016:     sc5_pedido_marcar_faturado_tx,
0017: )
0018: from app.infra.repositories.vendas.sc6_itens_repo import sc6_itens_do_pedido_tx
0019: from app.infra.repositories.vendas.sc6_totais_repo import sc6_totais_do_pedido_tx
0020: from app.infra.repositories.vendas.sc9_liberacao_repo import sc9_qtd_liberada_por_pedido_tx
0021:
0022:
0023: def _item_total(it: dict) -> float:
0024:     total = it.get("C6_TOTAL")
0025:     if total is None:
0026:         total = it.get("C6_VALOR")
0027:     if total is None:
0028:         preco_unit = it.get("C6_PRECO_UNIT") or 0
0029:         total = float(preco_unit) * int(it.get("C6_QTD") or 0)
0030:     return float(total)
0031:
0032:
0033: def faturar_pedido_venda_tx(
0034:     cur,
0035:     usuario: str,
0036:     pedido_id: int,
0037:     filial: str,
0038:     cliente_cod: str | None,
0039:     venc_dias: int = 30,
0040:     tes_cod: str = "001",
0041: ) -> dict:
0042:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0043:
0044:     filial = (filial or "").strip() or "01"
0045:     if not sm0_existe_tx(cur, filial):
0046:         raise BusinessError("Filial não cadastrada (SM0_EMPRESA)")
0047:     pedido_id = int(pedido_id)
0048:
```

```
0049:     p = sc5_pedido_get_tx(cur, pedido_id)
0050:     if not p:
0051:         raise BusinessError("Pedido não encontrado")
0052:     fechamento_validar_periodo_tx(cur, filial, p["C5_DATA"])
0053:
0054:     if str(p.get("C5_FILIAL") or "").strip() != filial:
0055:         raise BusinessError(f"Pedido {pedido_id} não pertence à filial {filial}")
0056:
0057:     status = str(p.get("C5_STATUS") or "").strip().upper()
0058:     if status != "ABERTO":
0059:         raise BusinessError("Apenas pedido ABERTO pode ser faturado")
0060:
0061:     origem = str(p.get("C5_ORIGEM") or "VENDA").strip().upper()
0062:     if origem != "VENDA":
0063:         raise BusinessError("Este endpoint fatura apenas pedidos de origem VENDA")
0064:
0065:     itens = sc6_itens_do_pedido_tx(cur, pedido_id)
0066:     if not itens:
0067:         raise BusinessError("Pedido sem itens (SC6)")
0068:     for it in itens:
0069:         if not it.get("C6_ITEM"):
0070:             raise BusinessError("Item sem C6_ITEM (numeração) - recalcule ou gere C6_ITEM")
0071:
0072:     # 0) valida liberações (SC9)
0073:     liberados = sc9_qtd_liberada_por_pedido_tx(cur, pedido_id)
0074:     for it in itens:
0075:         prod = str(it["C6_PRODUTO"]).strip()
0076:         qtd = int(it["C6_QTD"])
0077:         item_no = int(it.get("C6_ITEM") or 0)
0078:         qtd_lib = int(liberados.get(item_no, 0))
0079:         if qtd_lib < qtd:
0080:             raise BusinessError(
0081:                 f"Item {prod} não liberado (SC9). Liberado={qtd_lib} Pedido={qtd} (liberação p
arcial)"
0082:             )
0083:
0084:     # 1) recalcula total (fonte de verdade = SC6) e atualiza SC5
0085:     tot = sc6_totais_do_pedido_tx(cur, pedido_id)
0086:     valor_venda = float(tot["valor_itens"])
0087:     if valor_venda <= 0:
0088:         raise BusinessError("Valor do pedido deve ser > 0")
0089:
0090:     sc5_pedido_atualizar_valor_total_tx(cur, pedido_id, valor_venda)
0091:
0092:     # 2) snapshot cliente (opcional)
0093:     cliente_nome = None
0094:     if cliente_cod:
0095:         c = sa1_get_tx(cur, filial=filial, cod=str(cliente_cod).strip())
0096:         if not c:
0097:             raise BusinessError("Cliente não encontrado (SA1)")
0098:         cliente_nome = c["A1_NOME"]
0099:
0100:     # 3) TES / fiscal
0101:     f = fiscal_get_tes_tx(cur, filial, tes_cod=tes_cod)
```

```
0102:     tes_cod = str(f.get("F4_COD") or tes_cod).strip()
0103:     tes_tipo = str(f.get("F4_TIPO") or "").strip().upper()
0104:     cfop_base = (f.get("F4_CFOP") or "").strip()
0105:
0106:     if tes_tipo and tes_tipo != "S":
0107:         raise BusinessError("TES incompatível para venda (F4_TIPO deve ser 'S')")
0108:     if not cfop_base:
0109:         raise BusinessError("TES sem CFOP configurado (F4_CFOP)")
0110:
0111:     # 4) calcula impostos por item
0112:     valor_icms = 0.0
0113:     valor_ipi = 0.0
0114:     valor_pis = 0.0
0115:     valor_cofins = 0.0
0116:     valor_icms_st = 0.0
0117:     valor_difal = 0.0
0118:     itens_calc = []
0119:     gera_titulo = False
0120:     for it in itens:
0121:         item_total = _item_total(it)
0122:         fiscal_item = fiscal_resolver_item_tx(
0123:             cur,
0124:             filial=filial,
0125:             tes_cod=tes_cod,
0126:             cliente_cod=cliente_cod,
0127:             produto=str(it["C6_PRODUTO"]).strip(),
0128:         )
0129:         item_icms = round(item_total * float(fiscal_item["icms"]), 2)
0130:         item_ipi = round(item_total * float(fiscal_item["ipi"]), 2)
0131:         item_pis = round(item_total * float(fiscal_item["pis"]), 2)
0132:         item_cofins = round(item_total * float(fiscal_item["cofins"]), 2)
0133:         item_icms_st = round(item_total * float(fiscal_item["icms_st"]), 2)
0134:         item_difal = round(item_total * float(fiscal_item["difal"]), 2)
0135:         itens_calc.append(
0136:             (
0137:                 it,
0138:                 item_total,
0139:                 item_icms,
0140:                 item_ipi,
0141:                 item_pis,
0142:                 item_cofins,
0143:                 item_icms_st,
0144:                 item_difal,
0145:                 fiscal_item["cfop"],
0146:                 bool(fiscal_item["gera_est"]),
0147:                 fiscal_item.get("cst_icms"),
0148:                 fiscal_item.get("csosn"),
0149:                 fiscal_item.get("cst_pis"),
0150:                 fiscal_item.get("cst_cofins"),
0151:             )
0152:         )
0153:         if bool(fiscal_item["gera_tit"]):
0154:             gera_titulo = True
0155:         valor_icms += item_icms
```

```
0156:         valor_ipi += item_ipi
0157:         valor_pis += item_pis
0158:         valor_cofins += item_cofins
0159:         valor_icms_st += item_icms_st
0160:         valor_difal += item_difal
0161:
0162:     total_bruto = round(valor_venda + valor_ipi + valor_icms_st + valor_difal, 2)
0163:
0164:     # 5) SF2 + SD2 (NF)
0165:     nf = sf2_criar_tx(
0166:         cur,
0167:         filial=filial,
0168:         pedido_id=pedido_id,
0169:         cliente=cliente_nome,
0170:         cliente_cod=cliente_cod,
0171:         valor_total=valor_venda,
0172:         icms=valor_icms,
0173:         ipi=valor_ipi,
0174:         total_bruto=total_bruto,
0175:         tes_cod=tes_cod,
0176:         cfop=cfop_base,
0177:         origem="VENDA",
0178:         pis=valor_pis,
0179:         cofins=valor_cofins,
0180:         icms_st=valor_icms_st,
0181:         difal=valor_difal,
0182:     )
0183:
0184:     for idx, (
0185:         it,
0186:         item_total,
0187:         item_icms,
0188:         item_ipi,
0189:         item_pis,
0190:         item_cofins,
0191:         item_icms_st,
0192:         item_difal,
0193:         cfop_item,
0194:         _,
0195:         cst_icms,
0196:         csosn,
0197:         cst_pis,
0198:         cst_cofins,
0199:     ) in enumerate(itens_calc, start=1):
0200:         item_no = int(it.get("C6_ITEM") or idx)
0201:         sd2_item_criar_tx(
0202:             cur,
0203:             nf_id=nf["id"],
0204:             doc=nf["doc"],
0205:             serie=nf["serie"],
0206:             item=item_no,
0207:             filial=filial,
0208:             pedido_id=pedido_id,
0209:             produto=str(it["C6_PRODUTO"]).strip(),
```

```
0210:             qtd=int(it["C6_QTD"]),
0211:             preco_unit=(float(it["C6_PRECO_UNIT"]) if it.get("C6_PRECO_UNIT") is not None else
 None),
0212:             total=float(item_total),
0213:             icms=float(item_icms),
0214:             ipi=float(item_ipi),
0215:             tes_cod=tes_cod,
0216:             cfop=cfop_item or cfop_base,
0217:             pis=float(item_pis),
0218:             cofins=float(item_cofins),
0219:             icms_st=float(item_icms_st),
0220:             difal=float(item_difal),
0221:             cst_icms=cst_icms,
0222:             csosn=csosn,
0223:             cst_pis=cst_pis,
0224:             cst_cofins=cst_cofins,
0225:         )
0226:
0227:     # 6) estoque + SD3 (saída), conforme TES
0228:     valor_cmv_total = 0.0
0229:     if itens_calc:
0230:         for it, _, _, _, _, _, _, _, _, gera_est, _, _, _, _ in itens_calc:
0231:             if not gera_est:
0232:                 continue
0233:             prod = str(it["C6_PRODUTO"]).strip()
0234:             qtd = int(it["C6_QTD"])
0235:
0236:             sb1 = sb1_get_tx(cur, prod, filial)
0237:             cmv_override = it.get("C6_CMV_UNIT")
0238:             cm_unit = float(cmv_override) if cmv_override is not None else float(sb1["B1_CM"])
0239:             valor_cmv = cm_unit * qtd
0240:             valor_cmv_total += valor_cmv
0241:
0242:             sb1_baixar_atomico_tx(cur, prod, filial, qtd)
0243:
0244:             sd3_mov_tx(
0245:                 cur,
0246:                 filial=filial,
0247:                 produto=prod,
0248:                 tipo="S",
0249:                 qtd=qtd,
0250:                 custo_unit=cm_unit,
0251:                 valor=valor_cmv,
0252:                 origem="NF_EMITIDA",
0253:                 ref=str(nf["id"]),
0254:                 usuario=usuario,
0255:             )
0256:
0257:     # 7) marca pedido como FATURADO (com impostos)
0258:     sc5_pedido_marcar_faturado_tx(cur, pedido_id, icms=valor_icms, ipi=valor_ipi, total_bruto=
total_bruto)
0259:
0260:     # 8) cria AR (SE1) vinculado à NF (SF2)
0261:     if gera_titulo:
```

```
0262:         ar_criar_tx(
0263:             cur,
0264:             filial=filial,
0265:             cliente=cliente_nome,
0266:             cliente_cod=cliente_cod,
0267:             valor=valor_venda,
0268:             ref_id=nf["id"],
0269:             venc_dias=int(venc_dias),
0270:             serie=nf["serie"],
0271:         )
0272:
0273:     log_tx(
0274:         cur,
0275:         usuario,
0276:         f"Pedido {pedido_id} FATURADO origem=VENDA nf={nf['doc']}/{nf['serie']} valor={valor_v
enda} icms={valor_icms} ipi={valor_ipi} pis={valor_pis} cofins={valor_cofins} icms_st={valor_icms_st
} difal={valor_difal} cmv={valor_cmv_total} cliente_cod={cliente_cod} tes={tes_cod} cfop={cfop_base}
",
0277:     )
0278:
0279:     return {
0280:         "pedido_id": pedido_id,
0281:         "filial": filial,
0282:         "status": "FATURADO",
0283:         "origem": "VENDA",
0284:         "nf_id": int(nf["id"]),
0285:         "nf_doc": nf["doc"],
0286:         "nf_serie": nf["serie"],
0287:         "tes": tes_cod,
0288:         "cfop": cfop_base,
0289:         "valor_venda": float(valor_venda),
0290:         "icms": float(valor_icms),
0291:         "ipi": float(valor_ipi),
0292:         "pis": float(valor_pis),
0293:         "cofins": float(valor_cofins),
0294:         "icms_st": float(valor_icms_st),
0295:         "difal": float(valor_difal),
0296:         "total_bruto": float(total_bruto),
0297:         "valor_cmv": float(round(valor_cmv_total, 2)),
0298:         "cliente_cod": cliente_cod,
0299:         "cliente_nome": cliente_nome,
0300:         "qtd_itens": int(tot["qtd_itens"]),
0301:     }
```

==========
ARQUIVO: app/use_cases/vendas/itens_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx, sc5_pedido_atual
izar_valor_total_tx
0004: from app.infra.repositories.vendas.sc6_totais_repo import sc6_totais_do_pedido_tx
0005: from app.infra.repositories.vendas.sc6_itens_repo import sc6_item_get_tx, sc6_item_atualizar_t
x, sc6_item_excluir_logico_tx
```

```
0006: from app.infra.repositories.logs_repo import log_tx
0007:
0008: def _exigir_pedido_aberto_tx(cur, pedido_id: int, filial: str | None):
0009:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0010:     if not p:
0011:         raise BusinessError("Pedido não encontrado")
0012:     if filial and str(p.get("C5_FILIAL") or "").strip() != str(filial).strip():
0013:         raise BusinessError("Pedido não pertence à filial informada")
0014:
0015:     st = str(p.get("C5_STATUS") or "").strip().upper()
0016:     if st != "ABERTO":
0017:         raise BusinessError("Operação permitida apenas para pedido ABERTO")
0018:     return p
0019:
0020: def editar_item_tx(
0021:     cur,
0022:     usuario: str,
0023:     item_id: int,
0024:     filial: str | None,
0025:     qtd: int,
0026:     total: float,
0027:     preco_unit: float | None = None,
0028:     cmv_unit: float | None = None,
0029: ) -> dict:
0030:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0031:
0032:     it = sc6_item_get_tx(cur, int(item_id))
0033:     if not it or int(it.get("C6_ATIVO") or 0) != 1:
0034:         raise BusinessError("Item não encontrado/INATIVO")
0035:
0036:     pedido_id = int(it["C6_PEDIDO_ID"])
0037:     _exigir_pedido_aberto_tx(cur, pedido_id, filial)
0038:
0039:     qtd = int(qtd)
0040:     if qtd <= 0:
0041:         raise BusinessError("qtd deve ser > 0")
0042:
0043:     total = float(total)
0044:     if total < 0:
0045:         raise BusinessError("total deve ser >= 0")
0046:
0047:     if preco_unit is None:
0048:         preco_unit = (total / qtd) if qtd > 0 else total
0049:
0050:     sc6_item_atualizar_tx(cur, int(item_id), qtd, float(preco_unit), float(total), usuario, cm
v_unit=cmv_unit)
0051:
0052:     tot = sc6_totais_do_pedido_tx(cur, pedido_id)
0053:     sc5_pedido_atualizar_valor_total_tx(cur, pedido_id, float(tot["valor_itens"]))
0054:
0055:     log_tx(cur, usuario, f"Item {item_id} editado pedido={pedido_id} qtd={qtd} total={total} n
ovo_total={tot['valor_itens']}")
0056:
0057:     return {
```

```
0058:         "pedido_id": pedido_id,
0059:         "item_id": int(item_id),
0060:         "cmv_unit": (float(cmv_unit) if cmv_unit is not None else None),
0061:         "novo_total": float(tot["valor_itens"]),
0062:         "qtd_itens": int(tot["qtd_itens"]),
0063:     }
0064:
0065: def excluir_item_tx(cur, usuario: str, item_id: int, filial: str | None) -> dict:
0066:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0067:
0068:     it = sc6_item_get_tx(cur, int(item_id))
0069:     ativo = it.get("C6_ATIVO") if it else None
0070:     if not it:
0071:         raise BusinessError("Item não encontrado")
0072:     if ativo in (0, False):  # já deletado lógico
0073:         raise BusinessError("Item INATIVO")
0074:         # se ativo vier None por migração antiga, trate como ativo e deixe o delete funciona
0075:
0076:     pedido_id = int(it["C6_PEDIDO_ID"])
0077:     _exigir_pedido_aberto_tx(cur, pedido_id, filial)
0078:
0079:     sc6_item_excluir_logico_tx(cur, int(item_id), usuario)
0080:
0081:     tot = sc6_totais_do_pedido_tx(cur, pedido_id)
0082:     sc5_pedido_atualizar_valor_total_tx(cur, pedido_id, float(tot["valor_itens"]))
0083:
0084:     log_tx(cur, usuario, f"Item {item_id} excluído pedido={pedido_id} novo_total={tot['valor_i
tens']}")
0085:
0086:     return {
0087:         "pedido_id": pedido_id,
0088:         "item_id": int(item_id),
0089:         "novo_total": float(tot["valor_itens"]),
0090:         "qtd_itens": int(tot["qtd_itens"]),
0091:     }
```

==========
ARQUIVO: app/use_cases/vendas/liberacoes_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx
0005: from app.infra.repositories.vendas.sc9_liberacao_query_repo import sc9_resumo_por_pedido_tx
0006:
0007:
0008: def resumo_liberacao_pedido_tx(cur, usuario: str, pedido_id: int) -> dict:
0009:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0010:
0011:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0012:     if not p:
0013:         raise BusinessError("Pedido não encontrado")
0014:
0015:     resumo = sc9_resumo_por_pedido_tx(cur, int(pedido_id))
```

```
0016:     return {"pedido_id": int(pedido_id), "itens": resumo}
```

==========
ARQUIVO: app/use_cases/vendas/liberar_pedido_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx
0005: from app.infra.repositories.vendas.sc6_itens_repo import sc6_itens_do_pedido_tx
0006: from app.infra.repositories.sb1_repo import sb1_get_tx
0007: from app.infra.repositories.vendas.sc9_liberacao_repo import (
0008:     sc9_liberar_tx,
0009:     sc9_qtd_liberada_por_pedido_tx,
0010: )
0011:
0012:
0013: def liberar_item_pedido_tx(
0014:     cur,
0015:     usuario: str,
0016:     pedido_id: int,
0017:     filial: str,
0018:     produto: str,
0019:     qtd: int,
0020: ) -> dict:
0021:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0022:
0023:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0024:     if not p:
0025:         raise BusinessError("Pedido não encontrado")
0026:
0027:     if str(p.get("C5_FILIAL") or "").strip() != str(filial).strip():
0028:         raise BusinessError(f"Pedido {pedido_id} não pertence à filial {filial}")
0029:
0030:     status = str(p.get("C5_STATUS") or "").strip().upper()
0031:     if status != "ABERTO":
0032:         raise BusinessError("Apenas pedido ABERTO pode ser liberado")
0033:
0034:     itens = sc6_itens_do_pedido_tx(cur, int(pedido_id))
0035:     if not itens:
0036:         raise BusinessError("Pedido sem itens")
0037:
0038:     produto = (produto or "").strip()
0039:     if not produto:
0040:         raise BusinessError("produto é obrigatório")
0041:
0042:     qtd = int(qtd)
0043:     if qtd <= 0:
0044:         raise BusinessError("qtd deve ser > 0")
0045:
0046:     item = next((it for it in itens if str(it["C6_PRODUTO"]).strip() == produto), None)
0047:     if not item:
0048:         raise BusinessError("Produto não encontrado no pedido")
0049:
```

```
0050:     qtd_pedido = int(item["C6_QTD"])
0051:     liberados = sc9_qtd_liberada_por_pedido_tx(cur, int(pedido_id))
0052:     item_no = int(item.get("C6_ITEM") or 0)
0053:     qtd_ja = int(liberados.get(item_no, 0))
0054:     if qtd > (qtd_pedido - qtd_ja):
0055:         raise BusinessError("qtd liberada maior que saldo do pedido")
0056:
0057:     lib_id = sc9_liberar_tx(
0058:         cur,
0059:         pedido_id=int(pedido_id),
0060:         filial=str(filial).strip(),
0061:         produto=produto,
0062:         qtd=qtd,
0063:         usuario=usuario,
0064:         item=int(item["C6_ITEM"]) if item.get("C6_ITEM") is not None else None,
0065:     )
0066:
0067:     return {
0068:         "pedido_id": int(pedido_id),
0069:         "filial": str(filial).strip(),
0070:         "produto": produto,
0071:         "qtd": int(qtd),
0072:         "liberacao_id": int(lib_id),
0073:     }
0074:
0075:
0076: def liberar_pedido_total_tx(
0077:     cur,
0078:     usuario: str,
0079:     pedido_id: int,
0080:     filial: str,
0081:     usar_estoque: bool = True,
0082: ) -> dict:
0083:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0084:
0085:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0086:     if not p:
0087:         raise BusinessError("Pedido não encontrado")
0088:
0089:     if str(p.get("C5_FILIAL") or "").strip() != str(filial).strip():
0090:         raise BusinessError(f"Pedido {pedido_id} não pertence à filial {filial}")
0091:
0092:     status = str(p.get("C5_STATUS") or "").strip().upper()
0093:     if status != "ABERTO":
0094:         raise BusinessError("Apenas pedido ABERTO pode ser liberado")
0095:
0096:     itens = sc6_itens_do_pedido_tx(cur, int(pedido_id))
0097:     if not itens:
0098:         raise BusinessError("Pedido sem itens")
0099:
0100:     liberados = sc9_qtd_liberada_por_pedido_tx(cur, int(pedido_id))
0101:     liberacoes = []
0102:
0103:     for it in itens:
```

```
0104:         produto = str(it["C6_PRODUTO"]).strip()
0105:         qtd_pedido = int(it["C6_QTD"])
0106:         item_no = int(it.get("C6_ITEM") or 0)
0107:         qtd_ja = int(liberados.get(item_no, 0))
0108:         saldo = qtd_pedido - qtd_ja
0109:         if saldo <= 0:
0110:             continue
0111:         if usar_estoque:
0112:             sb1 = sb1_get_tx(cur, produto, str(filial).strip())
0113:             disponivel = int(sb1["B1_ESTOQUE"]) - int(sb1["B1_RESERVADO"])
0114:             if disponivel <= 0:
0115:                 continue
0116:             saldo = min(saldo, disponivel)
0117:         lib_id = sc9_liberar_tx(
0118:             cur,
0119:             pedido_id=int(pedido_id),
0120:             filial=str(filial).strip(),
0121:             produto=produto,
0122:             qtd=saldo,
0123:             usuario=usuario,
0124:             item=int(it["C6_ITEM"]) if it.get("C6_ITEM") is not None else None,
0125:         )
0126:         liberacoes.append({"produto": produto, "qtd": saldo, "liberacao_id": int(lib_id)})
0127:
0128:     return {
0129:         "pedido_id": int(pedido_id),
0130:         "filial": str(filial).strip(),
0131:         "liberacoes": liberacoes,
0132:         "usar_estoque": bool(usar_estoque),
0133:     }
```

==========
ARQUIVO: app/use_cases/vendas/listar_pedidos_enriq_uc.py
==========

```
0001: from datetime import date
0002: from app.core.exceptions import BusinessError
0003: from app.use_cases.usuarios_uc import exigir_perfil_tx
0004:
0005: from app.infra.repositories.vendas.sc5_pedidos_enriq_repo import (
0006:     sc5_pedidos_enriq_listar_tx,
0007:     sc5_pedidos_enriq_contar_tx,
0008: )
0009:
0010: def listar_pedidos_enriq_tx(
0011:     cur,
0012:     usuario: str,
0013:     filial: str | None,
0014:     status: str | None,
0015:     origem: str | None,
0016:     de: date | None,
0017:     ate: date | None,
0018:     limit: int,
0019:     offset: int,
0020: ) -> dict:
```

```
0021:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0022:
0023:     filial = (filial or "").strip() or None
0024:     status = (status or "").strip().upper() or None
0025:     origem = (origem or "").strip().upper() or None
0026:
0027:     if status and status not in {"ABERTO", "FATURADO", "CANCELADO"}:
0028:         raise BusinessError("status inválido (ABERTO/FATURADO/CANCELADO)")
0029:
0030:     if origem and origem not in {"VENDA", "RESERVA"}:
0031:         raise BusinessError("origem inválida (VENDA/RESERVA)")
0032:
0033:     limit = int(limit)
0034:     offset = int(offset)
0035:     if limit <= 0 or limit > 200:
0036:         raise BusinessError("limit deve estar entre 1 e 200")
0037:     if offset < 0:
0038:         raise BusinessError("offset deve ser >= 0")
0039:
0040:     if de is not None and ate is not None and de > ate:
0041:         raise BusinessError("de não pode ser maior que ate")
0042:
0043:     total = sc5_pedidos_enriq_contar_tx(cur, filial, status, origem, de, ate)
0044:     items = sc5_pedidos_enriq_listar_tx(cur, filial, status, origem, de, ate, limit, offset)
0045:
0046:     return {"total": total, "limit": limit, "offset": offset, "items": items}
```

==========
ARQUIVO: app/use_cases/vendas/listar_pedidos_uc.py
==========

```
0001: from datetime import date
0002: from app.core.exceptions import BusinessError
0003: from app.use_cases.usuarios_uc import exigir_perfil_tx
0004:
0005: from app.infra.repositories.vendas.sc5_pedidos_query_repo import (
0006:     sc5_pedidos_listar_tx,
0007:     sc5_pedidos_contar_tx,
0008: )
0009:
0010: def listar_pedidos_tx(
0011:     cur,
0012:     usuario: str,
0013:     filial: str | None,
0014:     status: str | None,
0015:     origem: str | None,
0016:     de: date | None,
0017:     ate: date | None,
0018:     limit: int,
0019:     offset: int,
0020: ) -> dict:
0021:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0022:
0023:     # normalização
0024:     filial = (filial or "").strip() or None
```

```
0025:     status = (status or "").strip().upper() or None
0026:     origem = (origem or "").strip().upper() or None
0027:
0028:     if status and status not in {"ABERTO", "FATURADO", "CANCELADO"}:
0029:         raise BusinessError("status inválido (ABERTO/FATURADO/CANCELADO)")
0030:
0031:     if origem and origem not in {"VENDA", "RESERVA"}:
0032:         raise BusinessError("origem inválida (VENDA/RESERVA)")
0033:
0034:     limit = int(limit)
0035:     offset = int(offset)
0036:     if limit <= 0 or limit > 200:
0037:         raise BusinessError("limit deve estar entre 1 e 200")
0038:     if offset < 0:
0039:         raise BusinessError("offset deve ser >= 0")
0040:
0041:     if de is not None and ate is not None and de > ate:
0042:         raise BusinessError("de não pode ser maior que ate")
0043:
0044:     total = sc5_pedidos_contar_tx(cur, filial, status, origem, de, ate)
0045:     items = sc5_pedidos_listar_tx(cur, filial, status, origem, de, ate, limit, offset)
0046:
0047:     return {
0048:         "total": int(total),
0049:         "limit": int(limit),
0050:         "offset": int(offset),
0051:         "items": items,
0052:     }
```

==========
ARQUIVO: app/use_cases/vendas/pedidos_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import (
0005:     sc5_pedido_criar_tx,
0006:     sc5_pedido_get_tx,
0007:     sc5_pedido_atualizar_valor_total_tx,
0008: )
0009: from app.infra.repositories.vendas.sc6_pedidos_repo import sc6_item_criar_tx
0010: from app.infra.repositories.vendas.sc6_totais_repo import sc6_totais_do_pedido_tx
0011:
0012: from app.infra.repositories.sb1_repo import sb1_get_tx
0013: from app.infra.repositories.logs_repo import log_tx
0014: from app.infra.repositories.vendas.sc9_liberacao_repo import sc9_liberar_tx
0015: from app.infra.repositories.sm0_repo import sm0_existe_tx
0016:
0017: def criar_pedido_tx(
0018:     cur,
0019:     usuario: str,
0020:     filial: str,
0021:     valor_total: float,
0022:     status: str = "ABERTO",
```

```
0023:     icms: float | None = None,
0024:     ipi: float | None = None,
0025:     total_bruto: float | None = None,
0026: ) -> dict:
0027:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0028:
0029:     filial = (filial or "").strip() or "01"
0030:     status = (status or "ABERTO").strip().upper()
0031:
0032:     if not sm0_existe_tx(cur, filial):
0033:         raise BusinessError("Filial não cadastrada (SM0_EMPRESA)")
0034:
0035:     # rigor ERP: pedido nasce ABERTO
0036:     if status != "ABERTO":
0037:         raise BusinessError("Pedido só pode ser criado com status ABERTO")
0038:
0039:     # valor_total no create vira placeholder (fonte de verdade = SC6)
0040:     if float(valor_total) < 0:
0041:         raise BusinessError("valor_total deve ser >= 0")
0042:
0043:     pedido_id = sc5_pedido_criar_tx(
0044:         cur,
0045:         filial=filial,
0046:         valor_total=float(valor_total),
0047:         usuario=usuario,
0048:         status="ABERTO",
0049:         icms=icms,
0050:         ipi=ipi,
0051:         total_bruto=total_bruto,
0052:         origem="VENDA",  # Protheus-like
0053:     )
0054:
0055:     log_tx(cur, usuario, f"Pedido {pedido_id} criado filial={filial} origem=VENDA status=ABERT
O total={valor_total}")
0056:
0057:     return {
0058:         "pedido_id": int(pedido_id),
0059:         "filial": filial,
0060:         "status": "ABERTO",
0061:         "valor_total": float(valor_total),
0062:     }
0063:
0064: def adicionar_item_tx(
0065:     cur,
0066:     usuario: str,
0067:     pedido_id: int,
0068:     filial: str,
0069:     produto: str,
0070:     qtd: int,
0071:     total: float,
0072:     preco_unit: float | None = None,
0073:     cmv_unit: float | None = None,
0074: ) -> dict:
0075:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
```

```
0076:
0077:     filial = (filial or "").strip() or "01"
0078:     produto = (produto or "").strip()
0079:     if not produto:
0080:         raise BusinessError("produto é obrigatório")
0081:
0082:     pedido_id = int(pedido_id)
0083:
0084:     # pedido existe + filial
0085:     p = sc5_pedido_get_tx(cur, pedido_id)
0086:     if not p:
0087:         raise BusinessError("Pedido não encontrado")
0088:
0089:     if str(p.get("C5_FILIAL") or "").strip() != filial:
0090:         raise BusinessError(f"Pedido {pedido_id} não pertence à filial {filial}")
0091:
0092:     # status bloqueios
0093:     status = str(p.get("C5_STATUS") or "").strip().upper()
0094:     if status == "CANCELADO":
0095:         raise BusinessError("Não é permitido incluir item: pedido CANCELADO")
0096:     if status == "FATURADO":
0097:         raise BusinessError("Não é permitido incluir item: pedido FATURADO")
0098:     if status != "ABERTO":
0099:         raise BusinessError("Status inválido para inclusão de item")
0100:
0101:     # validações do item
0102:     qtd = int(qtd)
0103:     if qtd <= 0:
0104:         raise BusinessError("qtd deve ser > 0")
0105:
0106:     total = float(total)
0107:     if total < 0:
0108:         raise BusinessError("total deve ser >= 0")
0109:
0110:     # garante produto existe
0111:     sb1_get_tx(cur, produto, filial)
0112:
0113:     # cria item (SC6)
0114:     item = sc6_item_criar_tx(
0115:         cur,
0116:         pedido_id=pedido_id,
0117:         produto=produto,
0118:         qtd=qtd,
0119:         total=total,
0120:         preco_unit=preco_unit,
0121:         cmv_unit=cmv_unit,
0122:     )
0123:
0124:     # liberação automática do item (SC9)
0125:     sc9_liberar_tx(
0126:         cur,
0127:         pedido_id=pedido_id,
0128:         filial=filial,
0129:         produto=produto,
```

```
0130:         qtd=qtd,
0131:         usuario=usuario,
0132:         item=int(item["item"]) if item else None,
0133:     )
0134:
0135:     # recalcula SC5 a partir da soma do SC6 (fonte de verdade)
0136:     tot = sc6_totais_do_pedido_tx(cur, pedido_id)
0137:     sc5_pedido_atualizar_valor_total_tx(cur, pedido_id, float(tot["valor_itens"]))
0138:
0139:     log_tx(
0140:         cur,
0141:         usuario,
0142:         f"Item adicionado pedido={pedido_id} produto={produto} qtd={qtd} total={total} novo_to
tal={tot['valor_itens']} lib=SC9",
0143:     )
0144:
0145:     return {
0146:         "pedido_id": int(pedido_id),
0147:         "produto": produto,
0148:         "qtd": int(qtd),
0149:         "total": float(total),
0150:         "cmv_unit": (float(cmv_unit) if cmv_unit is not None else None),
0151:         "novo_total": float(tot["valor_itens"]),
0152:         "qtd_itens": int(tot["qtd_itens"]),
0153:     }
```

==========
ARQUIVO: app/use_cases/vendas/recalcular_pedido_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx, sc5_pedido_atual
izar_valor_total_tx
0005: from app.infra.repositories.vendas.sc6_totais_repo import sc6_totais_do_pedido_tx
0006: from app.infra.repositories.logs_repo import log_tx
0007:
0008: def pedido_recalcular_totais_tx(
0009:     cur,
0010:     usuario: str,
0011:     pedido_id: int,
0012:     forcar: bool = False,  # permite recalcular até FATURADO (admin), se você quiser
0013: ) -> dict:
0014:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0015:
0016:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0017:     if not p:
0018:         raise BusinessError("Pedido não encontrado")
0019:
0020:     status = str(p.get("C5_STATUS") or "").strip().upper()
0021:     if status == "CANCELADO":
0022:         raise BusinessError("Não é permitido recalcular: pedido CANCELADO")
0023:
0024:     if status == "FATURADO" and not forcar:
```

```
0025:         raise BusinessError("Não é permitido recalcular: pedido FATURADO (use forcar=true se n
ecessário)")
0026:
0027:     # totais calculados
0028:     tot = sc6_totais_do_pedido_tx(cur, int(pedido_id))
0029:     novo_total = float(tot["valor_itens"])
0030:
0031:     antigo_total = float(p.get("C5_VALOR_TOTAL") or 0.0)
0032:     if abs(antigo_total - novo_total) < 0.0001:
0033:         return {
0034:             "pedido_id": int(pedido_id),
0035:             "status": status,
0036:             "alterado": False,
0037:             "valor_total_antigo": antigo_total,
0038:             "valor_total_novo": novo_total,
0039:             "qtd_itens": int(tot["qtd_itens"]),
0040:             "valor_itens": float(tot["valor_itens"]),
0041:         }
0042:
0043:     sc5_pedido_atualizar_valor_total_tx(cur, int(pedido_id), novo_total)
0044:
0045:     log_tx(
0046:         cur,
0047:         usuario,
0048:         f"Pedido {pedido_id} recalculado status={status} total:{antigo_total}->{novo_total} qt
d_itens={tot['qtd_itens']}",
0049:     )
0050:
0051:     return {
0052:         "pedido_id": int(pedido_id),
0053:         "status": status,
0054:         "alterado": True,
0055:         "valor_total_antigo": antigo_total,
0056:         "valor_total_novo": novo_total,
0057:         "qtd_itens": int(tot["qtd_itens"]),
0058:         "valor_itens": float(tot["valor_itens"]),
0059:     }
```

==========
ARQUIVO: app/use_cases/vendas/relatorios_nf_id_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.relatorios_nf_id_repo import nf_rastreio_tx
0005:
0006:
0007: def relatorio_nf_rastreio_tx(cur, usuario: str, nf_id: int) -> dict:
0008:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0009:
0010:     rows = nf_rastreio_tx(cur, int(nf_id))
0011:     if not rows:
0012:         raise BusinessError("NF não encontrada")
0013:
```

```
0014:     return {"nf_id": int(nf_id), "rows": rows}
```

==========
ARQUIVO: app/use_cases/vendas/relatorios_nf_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases.usuarios_uc import exigir_perfil_tx
0003:
0004: from app.infra.repositories.vendas.sc5_pedidos_repo import sc5_pedido_get_tx
0005: from app.infra.repositories.vendas.relatorios_nf_repo import nf_pedido_financeiro_resumo_tx
0006:
0007:
0008: def relatorio_nf_financeiro_tx(
0009:     cur,
0010:     usuario: str,
0011:     pedido_id: int,
0012:     nf_status: str | None = None,
0013:     ar_status: str | None = None,
0014:     sc9_status: str | None = None,
0015: ) -> dict:
0016:     exigir_perfil_tx(cur, usuario, {"admin", "operador"})
0017:
0018:     p = sc5_pedido_get_tx(cur, int(pedido_id))
0019:     if not p:
0020:         raise BusinessError("Pedido não encontrado")
0021:
0022:     rows = nf_pedido_financeiro_resumo_tx(
0023:         cur,
0024:         int(pedido_id),
0025:         nf_status=nf_status,
0026:         ar_status=ar_status,
0027:         sc9_status=sc9_status,
0028:     )
0029:     return {"pedido_id": int(pedido_id), "rows": rows}
```

==========
ARQUIVO: CHANGELOG.md
==========

```
0001: # Changelog
0002:
0003: ## v1.0.0-beta.1 - 2026-03-24
0004:
0005: Primeira versao organizada para publicacao do projeto como portfolio tecnico e ERP operacional
 em evolucao.
0006:
0007: ### Adicionado
0008:
0009: - shell visual mais proximo de ERP corporativo no frontend
0010: - dashboard operacional com atalhos e imagens por modulo
0011: - modulo operacional de custo por fornecedor
0012: - fluxo de formacao de custo com atualizacao de `B1_CM`
0013: - geracao de `AP` a partir de entrada com fornecedor
0014: - listagem e manutencao de tabelas de preco no frontend
0015: - alertas de negocio no backend com CTA no frontend
```

0016: - navegacao contextual entre AR/AP, categorias e caixa
0017: - auditoria consultavel no frontend
0018: - campo de `CMV unit` por item de pedido
0019: - screenshots e documentacao de portfolio
0020:
0021: ### Alterado
0022:
0023: - backend consolidado como API pura sem camada web/Jinja
0024: - bootstrap protegido por ambiente e estado persistido no banco
0025: - payload de erro padronizado
0026: - limpeza de shims, imports antigos e codigo residual
0027: - frontend com tema unificado e heroes por modulo
0028:
0029: ### Corrigido
0030:
0031: - falha de build no frontend em `parceiros`
0032: - warnings de hooks no frontend
0033: - problemas de encoding/mojibake no codigo e no exportador PDF
0034: - erro de ambiguidade `D_E_L_E_T_` em consultas com joins
0035: - problemas de compatibilidade de coluna `ID` em bases reais
0036: - leitura de auditoria que retornava lista vazia
0037: - geracao de `F1_NUM` para `SF1_AP` quando `SX6_SEQ` estava defasada
0038:
0039: ### Conhecido
0040:
0041: - compras ainda nao existem como modulo completo
0042: - estoque avancado ainda nao cobre lote, validade e armazem
0043: - fiscal e governanca ainda estao em aprofundamento

==========
ARQUIVO: docs/screenshots/README.md
==========
0001: # Screenshots
0002:
0003: Esta pasta concentra os prints usados no README de portfolio do Mini Protheus.
0004:
0005: ## Arquivos esperados
0006:
0007: Swagger / API:
0008:
0009: - `swagger-overview.png`
0010: - `swagger-auth-usuarios-produtos.png`
0011: - `swagger-reserva-financeiro.png`
0012: - `swagger-parceiros-relatorios.png`
0013: - `swagger-clientes-fornecedores.png`
0014: - `swagger-parametros-1.png`
0015: - `swagger-parametros-2.png`
0016: - `swagger-vendas.png`
0017: - `swagger-vendas-nf.png`
0018: - `swagger-schemas.png`
0019:
0020: Frontend:
0021:
0022: - `frontend-login.png`

0023: - `frontend-clientes.png`
0024: - `frontend-fornecedores.png`
0025: - `frontend-produtos.png`
0026: - `frontend-parceiros.png`
0027: - `frontend-pedidos.png`
0028: - `frontend-reservas.png`
0029: - `frontend-financeiro-ar.png`
0030: - `frontend-financeiro-ap.png`
0031: - `frontend-caixa.png`
0032: - `frontend-categorias.png`
0033: - `frontend-dre.png`
0034: - `frontend-margem-produto.png`
0035: - `frontend-auditoria.png`
0036: - `frontend-usuarios.png`
0037:
0038: ## Mapeamento do material capturado
0039:
0040: Swagger:
0041:
0042: - visao geral dos grupos e endpoints: `swagger-overview.png`
0043: - auth, bootstrap, usuarios e produtos: `swagger-auth-usuarios-produtos.png`
0044: - reserva e financeiro: `swagger-reserva-financeiro.png`
0045: - parceiros, relatorios, clientes, fornecedores e AP: `swagger-parceiros-relatorios.png`
0046: - clientes e fornecedores dedicados: `swagger-clientes-fornecedores.png`
0047: - parametros e fechamento: `swagger-parametros-1.png`
0048: - parametros, filiais e inicio de vendas: `swagger-parametros-2.png`
0049: - vendas, pedidos e faturamento: `swagger-vendas.png`
0050: - vendas, NF e schemas: `swagger-vendas-nf.png`
0051: - lista de schemas OpenAPI: `swagger-schemas.png`
0052:
0053: Frontend:
0054:
0055: - login: `frontend-login.png`
0056: - clientes: `frontend-clientes.png`
0057: - fornecedores: `frontend-fornecedores.png`
0058: - produtos: `frontend-produtos.png`
0059: - parceiros: `frontend-parceiros.png`
0060: - pedidos: `frontend-pedidos.png`
0061: - reservas: `frontend-reservas.png`
0062: - contas a receber: `frontend-financeiro-ar.png`
0063: - contas a pagar: `frontend-financeiro-ap.png`
0064: - caixa: `frontend-caixa.png`
0065: - categorias financeiras: `frontend-categorias.png`
0066: - DRE: `frontend-dre.png`
0067: - margem por produto: `frontend-margem-produto.png`
0068: - auditoria: `frontend-auditoria.png`
0069: - usuarios / bootstrap: `frontend-usuarios.png`
0070:
0071: ## Recomendacao de captura
0072:
0073: - usar largura consistente entre as imagens
0074: - preferir capturas com dados de demonstracao
0075: - evitar barras do navegador quando possivel
0076: - manter zoom legivel para tabelas e indicadores

0077:
0078: ## Integracao no README
0079:
0080: Quando os arquivos reais existirem, eles podem ser referenciados diretamente assim:
0081:
0082: ```md
0083: ![Swagger](docs/screenshots/swagger-overview.png)
0084: ![Login](docs/screenshots/frontend-login.png)
0085: ![Pedidos](docs/screenshots/frontend-pedidos.png)
0086: ```
0087:
0088: ## Observacao
0089:
0090: As imagens usadas nesta conversa servem como referencia de catalogacao, mas ainda precisam ser salvas como arquivos PNG dentro desta pasta para entrarem no repositorio.

==========
ARQUIVO: export_code_to_pdf.py
==========
0001: from __future__ import annotations
0002:
0003: from dataclasses import dataclass
0004: from pathlib import Path
0005: from typing import Iterable
0006:
0007:
0008: ROOT = Path(__file__).resolve().parent
0009: OUTPUT = ROOT / "mini_protheus_codigo_fonte.pdf"
0010:
0011: INCLUDE_EXTENSIONS = {
0012:   ".py",
0013:   ".sql",
0014:   ".ts",
0015:   ".tsx",
0016:   ".js",
0017:   ".jsx",
0018:   ".css",
0019:   ".html",
0020:   ".json",
0021:   ".md",
0022:   ".txt",
0023:   ".yml",
0024:   ".yaml",
0025: }
0026:
0027: INCLUDE_FILENAMES = {
0028:   ".env.example",
0029:   ".gitignore",
0030: }
0031:
0032: EXCLUDED_DIRS = {
0033:   ".git",
0034:   ".pytest_cache",
0035:   ".vscode",

```
0036:     "__pycache__",
0037:     "node_modules",
0038:     "dist",
0039:     "build",
0040: }
0041:
0042: EXCLUDED_SUFFIXES = {
0043:     ".pyc",
0044:     ".ico",
0045:     ".svg",
0046:     ".png",
0047:     ".jpg",
0048:     ".jpeg",
0049:     ".gif",
0050:     ".map",
0051:     ".node",
0052:     ".pdf",
0053: }
0054:
0055: MOJIBAKE_MARKERS = ("Ã", "Â", "Æ", "?")
0056:
0057:
0058: @dataclass
0059: class PdfObject:
0060:     number: int
0061:     content: bytes
0062:
0063:
0064: def escape_pdf_text(value: str) -> str:
0065:     return value.replace("\\", "\\\\").replace("(", "\\(").replace(")", "\\)")
0066:
0067:
0068: def normalize_text(value: str) -> str:
0069:     return value.encode("cp1252", "replace").decode("cp1252")
0070:
0071:
0072: def maybe_fix_mojibake(value: str) -> str:
0073:     if not any(marker in value for marker in MOJIBAKE_MARKERS):
0074:         return value
0075:
0076:     repaired = value
0077:     for _ in range(2):
0078:         try:
0079:             candidate = repaired.encode("latin-1").decode("utf-8")
0080:         except UnicodeError:
0081:             break
0082:         if candidate == repaired:
0083:             break
0084:         old_hits = sum(repaired.count(marker) for marker in MOJIBAKE_MARKERS)
0085:         new_hits = sum(candidate.count(marker) for marker in MOJIBAKE_MARKERS)
0086:         if new_hits >= old_hits:
0087:             break
0088:         repaired = candidate
0089:     return repaired
```

```
0090:
0091:
0092: def discover_files(root: Path) -> list[Path]:
0093:     files: list[Path] = []
0094:     for path in sorted(root.rglob("*")):
0095:         if not path.is_file():
0096:             continue
0097:         relative = path.relative_to(root)
0098:         if any(part in EXCLUDED_DIRS for part in relative.parts):
0099:             continue
0100:         if path.suffix.lower() in EXCLUDED_SUFFIXES:
0101:             continue
0102:         if path.name in INCLUDE_FILENAMES or path.suffix.lower() in INCLUDE_EXTENSIONS:
0103:             files.append(path)
0104:     return files
0105:
0106:
0107: def read_source(path: Path) -> str:
0108:     data = path.read_bytes()
0109:     for encoding in ("utf-8-sig", "utf-8", "cp1252", "latin-1"):
0110:         try:
0111:             text = data.decode(encoding)
0112:             break
0113:         except UnicodeDecodeError:
0114:             continue
0115:     else:
0116:         text = data.decode("utf-8", errors="replace")
0117:
0118:     text = maybe_fix_mojibake(text)
0119:     text = text.replace("\r\n", "\n").replace("\r", "\n")
0120:     if not text.endswith("\n"):
0121:         text += "\n"
0122:     return text
0123:
0124:
0125: def build_document_lines(files: Iterable[Path], root: Path) -> list[str]:
0126:     lines = [
0127:         "MINI PROTHEUS - Codigo fonte do projeto",
0128:         "",
0129:         "Documento gerado automaticamente com os arquivos relevantes do repositorio.",
0130:         "Exclusoes aplicadas: node_modules, dist, __pycache__, caches e binarios.",
0131:         "",
0132:     ]
0133:     for path in files:
0134:         relative = path.relative_to(root).as_posix()
0135:         lines.append("=" * 100)
0136:         lines.append(f"ARQUIVO: {relative}")
0137:         lines.append("=" * 100)
0138:         source = read_source(path)
0139:         source_lines = source.split("\n")
0140:         for idx, raw_line in enumerate(source_lines, start=1):
0141:             if idx == len(source_lines) and raw_line == "":
0142:                 continue
0143:             lines.append(f"{idx:04d}: {raw_line}")
```

```
0144:         lines.append("")
0145:     return lines
0146:
0147:
0148: def split_pages(lines: list[str], max_lines: int = 54) -> list[list[str]]:
0149:     pages: list[list[str]] = []
0150:     current: list[str] = []
0151:     line_count = 0
0152:     for line in lines:
0153:         wrapped = wrap_line(line, width=100)
0154:         if line_count + len(wrapped) > max_lines and current:
0155:             pages.append(current)
0156:             current = []
0157:             line_count = 0
0158:         current.extend(wrapped)
0159:         line_count += len(wrapped)
0160:     if current:
0161:         pages.append(current)
0162:     return pages
0163:
0164:
0165: def wrap_line(text: str, width: int) -> list[str]:
0166:     normalized = normalize_text(text)
0167:     if len(normalized) <= width:
0168:         return [normalized]
0169:     parts = []
0170:     remaining = normalized
0171:     while len(remaining) > width:
0172:         parts.append(remaining[:width])
0173:         remaining = remaining[width:]
0174:     if remaining:
0175:         parts.append(remaining)
0176:     return parts
0177:
0178:
0179: def make_stream(page_lines: list[str], page_number: int, total_pages: int) -> bytes:
0180:     commands = ["BT", "/F1 9 Tf", "50 790 Td", "12 TL"]
0181:     for line in page_lines:
0182:         commands.append(f"({escape_pdf_text(line)}) Tj")
0183:         commands.append("T*")
0184:     commands.append("T*")
0185:     commands.append(f"(Pagina {page_number}/{total_pages}) Tj")
0186:     commands.append("ET")
0187:     body = "\n".join(commands).encode("cp1252", "replace")
0188:     return f"<< /Length {len(body)} >>\nstream\n".encode("ascii") + body + b"\nendstream"
0189:
0190:
0191: def build_pdf(pages: list[list[str]], output: Path) -> None:
0192:     objects: list[PdfObject] = []
0193:     font_id = 1
0194:     pages_id = 2
0195:     total_pages = len(pages)
0196:
0197:     objects.append(
```

```
0198:         PdfObject(font_id, b"<< /Type /Font /Subtype /Type1 /BaseFont /Courier /Encoding /WinA
nsiEncoding >>")
0199:     )
0200:
0201:     page_ids: list[int] = []
0202:     next_id = 3
0203:     for _ in pages:
0204:         content_id = next_id
0205:         page_id = next_id + 1
0206:         page_ids.append(page_id)
0207:         next_id += 2
0208:
0209:     kids = " ".join(f"{page_id} 0 R" for page_id in page_ids)
0210:     objects.append(PdfObject(pages_id, f"<< /Type /Pages /Kids [{kids}] /Count {total_pages} >
>".encode("ascii")))
0211:
0212:     next_id = 3
0213:     for index, page in enumerate(pages, start=1):
0214:         content_id = next_id
0215:         page_id = next_id + 1
0216:         objects.append(PdfObject(content_id, make_stream(page, index, total_pages)))
0217:         page_content = (
0218:             f"<< /Type /Page /Parent {pages_id} 0 R /MediaBox [0 0 612 842] "
0219:             f"/Resources << /Font << /F1 {font_id} 0 R >> >> /Contents {content_id} 0 R >>"
0220:         ).encode("ascii")
0221:         objects.append(PdfObject(page_id, page_content))
0222:         next_id += 2
0223:
0224:     catalog_id = next_id
0225:     objects.append(PdfObject(catalog_id, f"<< /Type /Catalog /Pages {pages_id} 0 R >>".encode(
"ascii")))
0226:
0227:     xref_positions: list[int] = [0]
0228:     chunks = [b"%PDF-1.4\n%\xe2\xe3\xcf\xd3\n"]
0229:     current_offset = len(chunks[0])
0230:     for obj in objects:
0231:         xref_positions.append(current_offset)
0232:         obj_header = f"{obj.number} 0 obj\n".encode("ascii")
0233:         obj_footer = b"\nendobj\n"
0234:         chunk = obj_header + obj.content + obj_footer
0235:         chunks.append(chunk)
0236:         current_offset += len(chunk)
0237:
0238:     xref_offset = current_offset
0239:     xref = [f"xref\n0 {len(objects) + 1}\n".encode("ascii"), b"0000000000 65535 f \n"]
0240:     for position in xref_positions[1:]:
0241:         xref.append(f"{position:010d} 00000 n \n".encode("ascii"))
0242:     trailer = (
0243:         f"trailer\n<< /Size {len(objects) + 1} /Root {catalog_id} 0 R >>\n"
0244:         f"startxref\n{xref_offset}\n%%EOF\n"
0245:     ).encode("ascii")
0246:
0247:     output.write_bytes(b"".join(chunks + xref + [trailer]))
0248:
```

```
0249:
0250: def main() -> None:
0251:     files = discover_files(ROOT)
0252:     lines = build_document_lines(files, ROOT)
0253:     pages = split_pages(lines)
0254:     build_pdf(pages, OUTPUT)
0255:     print(f"PDF gerado: {OUTPUT}")
0256:     print(f"Arquivos incluidos: {len(files)}")
0257:     print(f"Paginas: {len(pages)}")
0258:
0259:
0260: if __name__ == "__main__":
0261:     main()
```

==========
ARQUIVO: frontend/.gitignore
==========

```
0001: # Logs
0002: logs
0003: *.log
0004: npm-debug.log*
0005: yarn-debug.log*
0006: yarn-error.log*
0007: pnpm-debug.log*
0008: lerna-debug.log*
0009:
0010: node_modules
0011: dist
0012: dist-ssr
0013: *.local
0014:
0015: # Editor directories and files
0016: .vscode/*
0017: !.vscode/extensions.json
0018: .idea
0019: .DS_Store
0020: *.suo
0021: *.ntvs*
0022: *.njsproj
0023: *.sln
0024: *.sw?
0025:
0026:
0027:
0028: # ========================
0029: # Python
0030: # ========================
0031: __pycache__/
0032: *.py[cod]
0033: *.pyo
0034: *.pyd
0035: .pytest_cache/
0036: .venv/
0037: venv/
```

```
0038:
0039: # ==========================
0040: # Environment variables
0041: # ==========================
0042: .env
0043: .env.*
0044:
0045: # ==========================
0046: # TypeScript / Vite cache
0047: # ==========================
0048: *.tsbuildinfo
0049: node_modules/.tmp/
0050: frontend/node_modules/.tmp/
0051:
0052: # ==========================
0053: # Build frontend
0054: # ==========================
0055: frontend/dist/
0056: dist/
```

==========
ARQUIVO: frontend/eslint.config.js
==========

```
0001: import js from '@eslint/js'
0002: import globals from 'globals'
0003: import reactHooks from 'eslint-plugin-react-hooks'
0004: import reactRefresh from 'eslint-plugin-react-refresh'
0005: import tseslint from 'typescript-eslint'
0006: import { defineConfig, globalIgnores } from 'eslint/config'
0007:
0008: export default defineConfig([
0009:   globalIgnores(['dist']),
0010:   {
0011:     files: ['**/*.{ts,tsx}'],
0012:     extends: [
0013:       js.configs.recommended,
0014:       tseslint.configs.recommended,
0015:       reactHooks.configs.flat.recommended,
0016:       reactRefresh.configs.vite,
0017:     ],
0018:     languageOptions: {
0019:       ecmaVersion: 2020,
0020:       globals: globals.browser,
0021:     },
0022:   },
0023: ])
```

==========
ARQUIVO: frontend/index.html
==========

```
0001: <!doctype html>
0002: <html lang="en">
0003:   <head>
0004:     <meta charset="UTF-8" />
```

```
0005:     <link rel="icon" type="image/svg+xml" href="/vite.svg" />
0006:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
0007:     <title>frontend</title>
0008:   </head>
0009:   <body>
0010:     <div id="root"></div>
0011:     <script type="module" src="/src/main.tsx"></script>
0012:   </body>
0013: </html>
```

==========
ARQUIVO: frontend/package-lock.json
==========

```
0001: {
0002:   "name": "frontend",
0003:   "version": "0.0.0",
0004:   "lockfileVersion": 3,
0005:   "requires": true,
0006:   "packages": {
0007:     "": {
0008:       "name": "frontend",
0009:       "version": "0.0.0",
0010:       "dependencies": {
0011:         "react": "^19.2.0",
0012:         "react-dom": "^19.2.0",
0013:         "react-router-dom": "^7.13.1"
0014:       },
0015:       "devDependencies": {
0016:         "@eslint/js": "^9.39.1",
0017:         "@types/node": "^24.10.1",
0018:         "@types/react": "^19.2.7",
0019:         "@types/react-dom": "^19.2.3",
0020:         "@vitejs/plugin-react": "^5.1.1",
0021:         "eslint": "^9.39.1",
0022:         "eslint-plugin-react-hooks": "^7.0.1",
0023:         "eslint-plugin-react-refresh": "^0.4.24",
0024:         "globals": "^16.5.0",
0025:         "typescript": "~5.9.3",
0026:         "typescript-eslint": "^8.48.0",
0027:         "vite": "^7.3.1"
0028:       }
0029:     },
0030:     "node_modules/@babel/code-frame": {
0031:       "version": "7.29.0",
0032:       "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.29.0.tgz",
0033:       "integrity": "sha512-9NhCeYjq9+3uxgdtp20LSiJXJvN0FeCtNGpJxuMFZ1Kv3cWUNb6DOhJwUvcVCzKGR66
cw4njwM6hrJLqgOwbcw==",
0034:       "dev": true,
0035:       "license": "MIT",
0036:       "dependencies": {
0037:         "@babel/helper-validator-identifier": "^7.28.5",
0038:         "js-tokens": "^4.0.0",
0039:         "picocolors": "^1.1.1"
0040:       },
```

0041:   "engines": {
0042:   "node": ">=6.9.0"
0043:   }
0044:   },
0045:   "node_modules/@babel/compat-data": {
0046:   "version": "7.29.0",
0047:   "resolved": "https://registry.npmjs.org/@babel/compat-data/-/compat-data-7.29.0.tgz",
0048:   "integrity": "sha512-T1NCJqT/j9+cn8fvkt7jtwbLBfLC/1y1c7NtCeXFRgzGTsafi68MRv8yzkYSapBnFA6L3U2VSc02ciDzoAJhJg==",
0049:   "dev": true,
0050:   "license": "MIT",
0051:   "engines": {
0052:   "node": ">=6.9.0"
0053:   }
0054:   },
0055:   "node_modules/@babel/core": {
0056:   "version": "7.29.0",
0057:   "resolved": "https://registry.npmjs.org/@babel/core/-/core-7.29.0.tgz",
0058:   "integrity": "sha512-CGOfOJqWjg2qW/Mb6zNsDm+u5vFQ8DxXfbM09z69p5Z6+mE1ikP2jUXw+j42Pf1XTYED2Rni5f95npYeuwMDQA==",
0059:   "dev": true,
0060:   "license": "MIT",
0061:   "dependencies": {
0062:   "@babel/code-frame": "^7.29.0",
0063:   "@babel/generator": "^7.29.0",
0064:   "@babel/helper-compilation-targets": "^7.28.6",
0065:   "@babel/helper-module-transforms": "^7.28.6",
0066:   "@babel/helpers": "^7.28.6",
0067:   "@babel/parser": "^7.29.0",
0068:   "@babel/template": "^7.28.6",
0069:   "@babel/traverse": "^7.29.0",
0070:   "@babel/types": "^7.29.0",
0071:   "@jridgewell/remapping": "^2.3.5",
0072:   "convert-source-map": "^2.0.0",
0073:   "debug": "^4.1.0",
0074:   "gensync": "^1.0.0-beta.2",
0075:   "json5": "^2.2.3",
0076:   "semver": "^6.3.1"
0077:   },
0078:   "engines": {
0079:   "node": ">=6.9.0"
0080:   },
0081:   "funding": {
0082:   "type": "opencollective",
0083:   "url": "https://opencollective.com/babel"
0084:   }
0085:   },
0086:   "node_modules/@babel/generator": {
0087:   "version": "7.29.1",
0088:   "resolved": "https://registry.npmjs.org/@babel/generator/-/generator-7.29.1.tgz",
0089:   "integrity": "sha512-qsaF+9Qcm2Qv8SRIMMscAvG4O3lJ0F1GuMo5HR/Bp02LopNgnZBC/EkbevHFeGs4ls/oPz9v+Bsmzbkbe+0dUw==",
0090:   "dev": true,
0091:   "license": "MIT",

0092:   "dependencies": {
0093:   "@babel/parser": "^7.29.0",
0094:   "@babel/types": "^7.29.0",
0095:   "@jridgewell/gen-mapping": "^0.3.12",
0096:   "@jridgewell/trace-mapping": "^0.3.28",
0097:   "jsesc": "^3.0.2"
0098:   },
0099:   "engines": {
0100:   "node": ">=6.9.0"
0101:   }
0102:   },
0103:   "node_modules/@babel/helper-compilation-targets": {
0104:   "version": "7.28.6",
0105:   "resolved": "https://registry.npmjs.org/@babel/helper-compilation-targets/-/helper-compilation-targets-7.28.6.tgz",
0106:   "integrity": "sha512-JYtls3hqi15fcx5GaSNL7SCTJ2MNmjrkHXg4FSpOA/grxK8KwyZ5bubHsCq8FXCkua6xhuaaBit+3b7+VZRfcA==",
0107:   "dev": true,
0108:   "license": "MIT",
0109:   "dependencies": {
0110:   "@babel/compat-data": "^7.28.6",
0111:   "@babel/helper-validator-option": "^7.27.1",
0112:   "browserslist": "^4.24.0",
0113:   "lru-cache": "^5.1.1",
0114:   "semver": "^6.3.1"
0115:   },
0116:   "engines": {
0117:   "node": ">=6.9.0"
0118:   }
0119:   },
0120:   "node_modules/@babel/helper-globals": {
0121:   "version": "7.28.0",
0122:   "resolved": "https://registry.npmjs.org/@babel/helper-globals/-/helper-globals-7.28.0.tgz",
0123:   "integrity": "sha512-+W6cISkXFa1jXsDEdYA8HeevQT/FULhxzR99pxphltZcVaugps53THCeiWA8SguxxpSp3gKPiuYfSWopkLQ4hw==",
0124:   "dev": true,
0125:   "license": "MIT",
0126:   "engines": {
0127:   "node": ">=6.9.0"
0128:   }
0129:   },
0130:   "node_modules/@babel/helper-module-imports": {
0131:   "version": "7.28.6",
0132:   "resolved": "https://registry.npmjs.org/@babel/helper-module-imports/-/helper-module-imports-7.28.6.tgz",
0133:   "integrity": "sha512-l5XkZK7r7wa9LucGw9LwZyyCUscb4x37JWTPz7swwFE/0FMQAGpiWUZn8u9DzkSBWEcK25jmvubfpw2dnAMdbw==",
0134:   "dev": true,
0135:   "license": "MIT",
0136:   "dependencies": {
0137:   "@babel/traverse": "^7.28.6",
0138:   "@babel/types": "^7.28.6"
0139:   },

0140:   "engines": {
0141:   "node": ">=6.9.0"
0142:   }
0143:   },
0144:   "node_modules/@babel/helper-module-transforms": {
0145:   "version": "7.28.6",
0146:   "resolved": "https://registry.npmjs.org/@babel/helper-module-transforms/-/helper-module-transforms-7.28.6.tgz",
0147:   "integrity": "sha512-67oXFAYr2cDLDVGLXTEABjdBJZ6drElUSI7WKp70NrpyISso3plG9SAGEF6y7zbha/wOzUByWWTJvEDVNIUGcA==",
0148:   "dev": true,
0149:   "license": "MIT",
0150:   "dependencies": {
0151:   "@babel/helper-module-imports": "^7.28.6",
0152:   "@babel/helper-validator-identifier": "^7.28.5",
0153:   "@babel/traverse": "^7.28.6"
0154:   },
0155:   "engines": {
0156:   "node": ">=6.9.0"
0157:   },
0158:   "peerDependencies": {
0159:   "@babel/core": "^7.0.0"
0160:   }
0161:   },
0162:   "node_modules/@babel/helper-plugin-utils": {
0163:   "version": "7.28.6",
0164:   "resolved": "https://registry.npmjs.org/@babel/helper-plugin-utils/-/helper-plugin-utils-7.28.6.tgz",
0165:   "integrity": "sha512-S9gzZ/bz83GRysI7gAD4wPT/AI3uCnY+9xn+Mx/KPs2JwHJIz1W8PZkg2cqyt3RNOBM8ejcXhV6y8Og7ly/Dug==",
0166:   "dev": true,
0167:   "license": "MIT",
0168:   "engines": {
0169:   "node": ">=6.9.0"
0170:   }
0171:   },
0172:   "node_modules/@babel/helper-string-parser": {
0173:   "version": "7.27.1",
0174:   "resolved": "https://registry.npmjs.org/@babel/helper-string-parser/-/helper-string-parser-7.27.1.tgz",
0175:   "integrity": "sha512-qMlSxKbpRlAridDExk92nSobyDdpPijUq2DW6oDnUqd0iOGxmQjyqhMIihI9+zv4LPyZdRje2cavWPbCbWm3eA==",
0176:   "dev": true,
0177:   "license": "MIT",
0178:   "engines": {
0179:   "node": ">=6.9.0"
0180:   }
0181:   },
0182:   "node_modules/@babel/helper-validator-identifier": {
0183:   "version": "7.28.5",
0184:   "resolved": "https://registry.npmjs.org/@babel/helper-validator-identifier/-/helper-validator-identifier-7.28.5.tgz",
0185:   "integrity": "sha512-qSs4ifwzKJSV39ucNjsvc6WVHs6b7S03sOh2OcHF9UHfVPqWWALUsNUVzhSBiItjRZoLHx7nIarVjqKVusUZ1Q==",

0186:   "dev": true,
0187:   "license": "MIT",
0188:   "engines": {
0189:   "node": ">=6.9.0"
0190:   }
0191:   },
0192:   "node_modules/@babel/helper-validator-option": {
0193:   "version": "7.27.1",
0194:   "resolved": "https://registry.npmjs.org/@babel/helper-validator-option/-/helper-validato
r-option-7.27.1.tgz",
0195:   "integrity": "sha512-YvjJow9FxbhFFKDSuFnVCe2WxXk1zWc22fFePVNEaWJEu8IrZVlda6N0uHwzZrUM1il
7NC9Mlp4MaJYbYd9JSg==",
0196:   "dev": true,
0197:   "license": "MIT",
0198:   "engines": {
0199:   "node": ">=6.9.0"
0200:   }
0201:   },
0202:   "node_modules/@babel/helpers": {
0203:   "version": "7.28.6",
0204:   "resolved": "https://registry.npmjs.org/@babel/helpers/-/helpers-7.28.6.tgz",
0205:   "integrity": "sha512-xOBvwq86HHdB7WUDTfKfT/Vuxh7gElQ+Sfti2Cy6yIWNW05P8iUslOVcZ4/sKbE+/jQ
aukQAdz/gf3724kYdqw==",
0206:   "dev": true,
0207:   "license": "MIT",
0208:   "dependencies": {
0209:   "@babel/template": "^7.28.6",
0210:   "@babel/types": "^7.28.6"
0211:   },
0212:   "engines": {
0213:   "node": ">=6.9.0"
0214:   }
0215:   },
0216:   "node_modules/@babel/parser": {
0217:   "version": "7.29.0",
0218:   "resolved": "https://registry.npmjs.org/@babel/parser/-/parser-7.29.0.tgz",
0219:   "integrity": "sha512-IyDgFV5GeDUVX4YdF/3CPULtVGSXXMLh1xVIgdCgxApktqnQV0r7/8Nqthg+8YLGaAt
dyIlo2qIdZrbCv4+7ww==",
0220:   "dev": true,
0221:   "license": "MIT",
0222:   "dependencies": {
0223:   "@babel/types": "^7.29.0"
0224:   },
0225:   "bin": {
0226:   "parser": "bin/babel-parser.js"
0227:   },
0228:   "engines": {
0229:   "node": ">=6.0.0"
0230:   }
0231:   },
0232:   "node_modules/@babel/plugin-transform-react-jsx-self": {
0233:   "version": "7.27.1",
0234:   "resolved": "https://registry.npmjs.org/@babel/plugin-transform-react-jsx-self/-/plugin-
transform-react-jsx-self-7.27.1.tgz",

0235:   "integrity": "sha512-6UzkCs+ejGdZ5mFFC/OCUrv028ab2fp1znZmCZjAOBKiBK2jXD1O+BPSfX8X2qjJ75fZBMSnQn3Rq2mrBJK2mw==",
0236:   "dev": true,
0237:   "license": "MIT",
0238:   "dependencies": {
0239:   "@babel/helper-plugin-utils": "^7.27.1"
0240:   },
0241:   "engines": {
0242:   "node": ">=6.9.0"
0243:   },
0244:   "peerDependencies": {
0245:   "@babel/core": "^7.0.0-0"
0246:   }
0247:   },
0248:   "node_modules/@babel/plugin-transform-react-jsx-source": {
0249:   "version": "7.27.1",
0250:   "resolved": "https://registry.npmjs.org/@babel/plugin-transform-react-jsx-source/-/plugin-transform-react-jsx-source-7.27.1.tgz",
0251:   "integrity": "sha512-zbwoTsBruTeKB9hSq73ha66iFeJHuaFkUbwvqElnygoNbj/jHRsSeokowZFN3CZ64IvEqcmmkVe89OPXc7ldAw==",
0252:   "dev": true,
0253:   "license": "MIT",
0254:   "dependencies": {
0255:   "@babel/helper-plugin-utils": "^7.27.1"
0256:   },
0257:   "engines": {
0258:   "node": ">=6.9.0"
0259:   },
0260:   "peerDependencies": {
0261:   "@babel/core": "^7.0.0-0"
0262:   }
0263:   },
0264:   "node_modules/@babel/template": {
0265:   "version": "7.28.6",
0266:   "resolved": "https://registry.npmjs.org/@babel/template/-/template-7.28.6.tgz",
0267:   "integrity": "sha512-YA6Ma2KsCdGb+WC6UpBVFJGXL58MDA6oyONbjyF/+5sBgxY/dwkhLogbMT2GXXyU84/IhRw/2D1Os1B/giz+BQ==",
0268:   "dev": true,
0269:   "license": "MIT",
0270:   "dependencies": {
0271:   "@babel/code-frame": "^7.28.6",
0272:   "@babel/parser": "^7.28.6",
0273:   "@babel/types": "^7.28.6"
0274:   },
0275:   "engines": {
0276:   "node": ">=6.9.0"
0277:   }
0278:   },
0279:   "node_modules/@babel/traverse": {
0280:   "version": "7.29.0",
0281:   "resolved": "https://registry.npmjs.org/@babel/traverse/-/traverse-7.29.0.tgz",
0282:   "integrity": "sha512-4HPiQr0X7+waHfyXPZpWPfWL/J7dcN1mx9gL6WdQVMbPnF3+ZhSMs8tCxN7oHddJE9fhNE7+lxdnlyemKfJRuA==",
0283:   "dev": true,

0284:   "license": "MIT",
0285:   "dependencies": {
0286:   "@babel/code-frame": "^7.29.0",
0287:   "@babel/generator": "^7.29.0",
0288:   "@babel/helper-globals": "^7.28.0",
0289:   "@babel/parser": "^7.29.0",
0290:   "@babel/template": "^7.28.6",
0291:   "@babel/types": "^7.29.0",
0292:   "debug": "^4.3.1"
0293:   },
0294:   "engines": {
0295:   "node": ">=6.9.0"
0296:   }
0297:   },
0298:   "node_modules/@babel/types": {
0299:   "version": "7.29.0",
0300:   "resolved": "https://registry.npmjs.org/@babel/types/-/types-7.29.0.tgz",
0301:   "integrity": "sha512-LwdZHpScM4Qz8Xw2iKSzS+cfglZzJGvofQICy7W7v4caru4EaAmyUuO6BGrbyQ2mYV1
1W0U8j5mBhd14dd3B0A==",
0302:   "dev": true,
0303:   "license": "MIT",
0304:   "dependencies": {
0305:   "@babel/helper-string-parser": "^7.27.1",
0306:   "@babel/helper-validator-identifier": "^7.28.5"
0307:   },
0308:   "engines": {
0309:   "node": ">=6.9.0"
0310:   }
0311:   },
0312:   "node_modules/@esbuild/aix-ppc64": {
0313:   "version": "0.27.3",
0314:   "resolved": "https://registry.npmjs.org/@esbuild/aix-ppc64/-/aix-ppc64-0.27.3.tgz",
0315:   "integrity": "sha512-9fJMTNFTWZMh5qwrBIituziu834eOCUcEqymSH7pY+zoMVEZg3gcPuBNxH1EvfVYe9h0
x/Ptw8KBzv7qxb7l8dg==",
0316:   "cpu": [
0317:   "ppc64"
0318:   ],
0319:   "dev": true,
0320:   "license": "MIT",
0321:   "optional": true,
0322:   "os": [
0323:   "aix"
0324:   ],
0325:   "engines": {
0326:   "node": ">=18"
0327:   }
0328:   },
0329:   "node_modules/@esbuild/android-arm": {
0330:   "version": "0.27.3",
0331:   "resolved": "https://registry.npmjs.org/@esbuild/android-arm/-/android-arm-0.27.3.tgz",
0332:   "integrity": "sha512-i5D1hPY7GIQmXlXhs2w8AWHhenb00+GxjxRncS2ZM7YNVGNfaMxgzSGuO8o8SJzRc/o
ZwU2bcScvVERk03QhzA==",
0333:   "cpu": [
0334:   "arm"

0335:   ],
0336:   "dev": true,
0337:   "license": "MIT",
0338:   "optional": true,
0339:   "os": [
0340:   "android"
0341:   ],
0342:   "engines": {
0343:   "node": ">=18"
0344:   }
0345:   },
0346:   "node_modules/@esbuild/android-arm64": {
0347:   "version": "0.27.3",
0348:   "resolved": "https://registry.npmjs.org/@esbuild/android-arm64/-/android-arm64-0.27.3.tg
z",
0349:   "integrity": "sha512-YdghPYUmj/FX2SYKJ0OZxf+iaKgMsKHVPF1MAq/P8WirnSpCStzKJFjOjzsW0QQ7oIA
iccHdcqjbHmJxRb/dmg==",
0350:   "cpu": [
0351:   "arm64"
0352:   ],
0353:   "dev": true,
0354:   "license": "MIT",
0355:   "optional": true,
0356:   "os": [
0357:   "android"
0358:   ],
0359:   "engines": {
0360:   "node": ">=18"
0361:   }
0362:   },
0363:   "node_modules/@esbuild/android-x64": {
0364:   "version": "0.27.3",
0365:   "resolved": "https://registry.npmjs.org/@esbuild/android-x64/-/android-x64-0.27.3.tgz",
0366:   "integrity": "sha512-IN/0BNTkHtk8lkOM8JWAYFg4ORxBkZQf9zXiEOfERX/CzxW3Vg1ewAhU7QSWQpVIzTW
+b8Xy+lGzdYXV6UZObQ==",
0367:   "cpu": [
0368:   "x64"
0369:   ],
0370:   "dev": true,
0371:   "license": "MIT",
0372:   "optional": true,
0373:   "os": [
0374:   "android"
0375:   ],
0376:   "engines": {
0377:   "node": ">=18"
0378:   }
0379:   },
0380:   "node_modules/@esbuild/darwin-arm64": {
0381:   "version": "0.27.3",
0382:   "resolved": "https://registry.npmjs.org/@esbuild/darwin-arm64/-/darwin-arm64-0.27.3.tgz"
,
0383:   "integrity": "sha512-Re491k7ByTVRy0t3EKWajdLIr0gz2kKKfzafkth4Q8A5n1xTHrkqZgLLjFEHVD+AXdU
GgQMq+Godfq45mGpCKg==",

0384:   "cpu": [
0385:   "arm64"
0386:   ],
0387:   "dev": true,
0388:   "license": "MIT",
0389:   "optional": true,
0390:   "os": [
0391:   "darwin"
0392:   ],
0393:   "engines": {
0394:   "node": ">=18"
0395:   }
0396:   },
0397:   "node_modules/@esbuild/darwin-x64": {
0398:   "version": "0.27.3",
0399:   "resolved": "https://registry.npmjs.org/@esbuild/darwin-x64/-/darwin-x64-0.27.3.tgz",
0400:   "integrity": "sha512-vHk/hA7/1AckjGzRqi6wbo+jaShzRowYip6rt6q7VYEDX4LEy1pZfDpdxCBnGtl+A5zq8iXDcyuxwtv3hNtHFg==",
0401:   "cpu": [
0402:   "x64"
0403:   ],
0404:   "dev": true,
0405:   "license": "MIT",
0406:   "optional": true,
0407:   "os": [
0408:   "darwin"
0409:   ],
0410:   "engines": {
0411:   "node": ">=18"
0412:   }
0413:   },
0414:   "node_modules/@esbuild/freebsd-arm64": {
0415:   "version": "0.27.3",
0416:   "resolved": "https://registry.npmjs.org/@esbuild/freebsd-arm64/-/freebsd-arm64-0.27.3.tgz",
0417:   "integrity": "sha512-ipTYM2fjt3kQAYOvo6vcxJx3nBYAzPjgTCk7QEgZG8AUO3ydUhvelmhrbOheMnGOlaSFUoHXB6un+A7q4ygY9w==",
0418:   "cpu": [
0419:   "arm64"
0420:   ],
0421:   "dev": true,
0422:   "license": "MIT",
0423:   "optional": true,
0424:   "os": [
0425:   "freebsd"
0426:   ],
0427:   "engines": {
0428:   "node": ">=18"
0429:   }
0430:   },
0431:   "node_modules/@esbuild/freebsd-x64": {
0432:   "version": "0.27.3",
0433:   "resolved": "https://registry.npmjs.org/@esbuild/freebsd-x64/-/freebsd-x64-0.27.3.tgz",

"integrity": "sha512-dDk0X87T7mI6U3K9VjWtHOXqwAMJBNN2r7bejDsc+j03SEjtD9HrOl8gVFByeM0aJksoUuUVU9TBaZa2rgj0oA==",
"cpu": [
"x64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"freebsd"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/linux-arm": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/linux-arm/-/linux-arm-0.27.3.tgz",
"integrity": "sha512-s6nPv2QkSupJwLYyfS+gwdirm0ukyTFNl3KTgZEAiJDd+iHZcbTPPcWCcRYH+WlNbwChgH2QkE9NSlNrMT8Gfw==",
"cpu": [
"arm"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"linux"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/linux-arm64": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/linux-arm64/-/linux-arm64-0.27.3.tgz",
"integrity": "sha512-sZOuFz/xWnZ4KH3YfFrKCf1WyPZHakVzTiqji3WDc0BCl2kBwiJLCXpzLzUBLgmp4veFZdvN5ChW4Eq/8Fc2Fg==",
"cpu": [
"arm64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"linux"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/linux-ia32": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/linux-ia32/-/linux-ia32-0.27.3.tgz",

"integrity": "sha512-yGlQYjdxtLdh0a3jHjuwOrxQjOZYD/C9PfdbgJJF3TIZWnm/tMd/RcNiLngiu4iwcBAOezdnSLAwQDPqTmtTYg==",
"cpu": [
"ia32"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"linux"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/linux-loong64": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/linux-loong64/-/linux-loong64-0.27.3.tgz",
"integrity": "sha512-WO60Sn8ly3gtzhyjATDgieJNet/KqsDlX5nRC5Y3oTFcS1l0KWba+SEa9Ja1GfDqSF1z6hif/SkpQJbL63cgOA==",
"cpu": [
"loong64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"linux"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/linux-mips64el": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/linux-mips64el/-/linux-mips64el-0.27.3.tgz",
"integrity": "sha512-APsymYA6sGcZ4pD6k+UxbDjOFSvPWyZhjaiPyl/f79xKxwTnrn5QUnXR5prvetuaSMsb4jgeHewIDCIWljrSxw==",
"cpu": [
"mips64el"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"linux"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/linux-ppc64": {

0534:   "version": "0.27.3",
0535:   "resolved": "https://registry.npmjs.org/@esbuild/linux-ppc64/-/linux-ppc64-0.27.3.tgz",
0536:   "integrity": "sha512-eizBnTeBefojtDb9nSh4vvVQ3V9Qf9Df01PfawPcRzJH4gFSgrObw+LveUyDoKU3kxi
5+9RJTCWlj4FjYXVPEA==",
0537:   "cpu": [
0538:   "ppc64"
0539:   ],
0540:   "dev": true,
0541:   "license": "MIT",
0542:   "optional": true,
0543:   "os": [
0544:   "linux"
0545:   ],
0546:   "engines": {
0547:   "node": ">=18"
0548:   }
0549:   },
0550:   "node_modules/@esbuild/linux-riscv64": {
0551:   "version": "0.27.3",
0552:   "resolved": "https://registry.npmjs.org/@esbuild/linux-riscv64/-/linux-riscv64-0.27.3.tg
z",
0553:   "integrity": "sha512-3Emwh0r5wmfm3ssTWRQSyVhbOHvqegUDRd0WhmXKX2mkHJe1SFCMJhagUleMq+Uci34
wLSipf8Lagt4LlpRFWQ==",
0554:   "cpu": [
0555:   "riscv64"
0556:   ],
0557:   "dev": true,
0558:   "license": "MIT",
0559:   "optional": true,
0560:   "os": [
0561:   "linux"
0562:   ],
0563:   "engines": {
0564:   "node": ">=18"
0565:   }
0566:   },
0567:   "node_modules/@esbuild/linux-s390x": {
0568:   "version": "0.27.3",
0569:   "resolved": "https://registry.npmjs.org/@esbuild/linux-s390x/-/linux-s390x-0.27.3.tgz",
0570:   "integrity": "sha512-pBHUx9LzXWBc7MFIEEL0yD/ZVtNgLytvx60gES28GcWMqil8ElCYR4kvbV2BDqsHOvV
DRrOxGySBM9Fcv744hw==",
0571:   "cpu": [
0572:   "s390x"
0573:   ],
0574:   "dev": true,
0575:   "license": "MIT",
0576:   "optional": true,
0577:   "os": [
0578:   "linux"
0579:   ],
0580:   "engines": {
0581:   "node": ">=18"
0582:   }
0583:   },

"node_modules/@esbuild/linux-x64": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/linux-x64/-/linux-x64-0.27.3.tgz",
"integrity": "sha512-Czi8yzXUWIQYAtL/2y6vogER8pvcsOsk5cpwL4Gk5nJqH5UZiVByIY8Eorm5R13gq+DQKYg0+JyQoytLQas4dA==",
"cpu": [
"x64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"linux"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/netbsd-arm64": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/netbsd-arm64/-/netbsd-arm64-0.27.3.tgz",
"integrity": "sha512-sDpk0RgmTCR/5HguIZa9n9u+HVKf40fbEUt+iTzSnCaGvY9kFP0YKBWZtJaraonFnqef5SlJ8/TiPAxzyS+UoA==",
"cpu": [
"arm64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"netbsd"
],
"engines": {
"node": ">=18"
}
},
"node_modules/@esbuild/netbsd-x64": {
"version": "0.27.3",
"resolved": "https://registry.npmjs.org/@esbuild/netbsd-x64/-/netbsd-x64-0.27.3.tgz",
"integrity": "sha512-P14lFKJl/DdaE00LItAukUdZO5iqNH7+PjoBm+fLQjtxfcfFE20Xf5CrLsmZdq5LFFZzb5JMZ9grUwvtVYzjiA==",
"cpu": [
"x64"
],
"dev": true,
"license": "MIT",
"optional": true,
"os": [
"netbsd"
],
"engines": {
"node": ">=18"
}

0634:   },
0635:   "node_modules/@esbuild/openbsd-arm64": {
0636:   "version": "0.27.3",
0637:   "resolved": "https://registry.npmjs.org/@esbuild/openbsd-arm64/-/openbsd-arm64-0.27.3.tgz",
0638:   "integrity": "sha512-AIcMP77AvirGbRl/UZFTq5hjXK+2wC7qFRGoHSDrZ5v5b8DK/GYpXW3CPRL53NkvDqb9D+alBiC/dV0Fb7eJcw==",
0639:   "cpu": [
0640:   "arm64"
0641:   ],
0642:   "dev": true,
0643:   "license": "MIT",
0644:   "optional": true,
0645:   "os": [
0646:   "openbsd"
0647:   ],
0648:   "engines": {
0649:   "node": ">=18"
0650:   }
0651:   },
0652:   "node_modules/@esbuild/openbsd-x64": {
0653:   "version": "0.27.3",
0654:   "resolved": "https://registry.npmjs.org/@esbuild/openbsd-x64/-/openbsd-x64-0.27.3.tgz",
0655:   "integrity": "sha512-DnW2sRrBzA+YnE70LKqnM3P+z8vehfJWHXECbwBmH/CU51z6FiqTQTHFenPlHmo3a8UgpLyH3PT+87OViOh1AQ==",
0656:   "cpu": [
0657:   "x64"
0658:   ],
0659:   "dev": true,
0660:   "license": "MIT",
0661:   "optional": true,
0662:   "os": [
0663:   "openbsd"
0664:   ],
0665:   "engines": {
0666:   "node": ">=18"
0667:   }
0668:   },
0669:   "node_modules/@esbuild/openharmony-arm64": {
0670:   "version": "0.27.3",
0671:   "resolved": "https://registry.npmjs.org/@esbuild/openharmony-arm64/-/openharmony-arm64-0.27.3.tgz",
0672:   "integrity": "sha512-NinAEgr/etERPTsZJ7aEZQvvg/A6IsZG/LgZy+81wON2huV7SrK3e63dU0XhyZP4RKGyTm7aOgmQk0bGp0fy2g==",
0673:   "cpu": [
0674:   "arm64"
0675:   ],
0676:   "dev": true,
0677:   "license": "MIT",
0678:   "optional": true,
0679:   "os": [
0680:   "openharmony"
0681:   ],
0682:   "engines": {

0683:   "node": ">=18"
0684:   }
0685:   },
0686:   "node_modules/@esbuild/sunos-x64": {
0687:   "version": "0.27.3",
0688:   "resolved": "https://registry.npmjs.org/@esbuild/sunos-x64/-/sunos-x64-0.27.3.tgz",
0689:   "integrity": "sha512-PanZ+nEz+eWoBJ8/f8HKxTTD172SKwdXebZ0ndd953gt1HRBbhMsaNqjTyYLGLPdoWH
y4zLU7bDVJztF5f3BHA==",
0690:   "cpu": [
0691:   "x64"
0692:   ],
0693:   "dev": true,
0694:   "license": "MIT",
0695:   "optional": true,
0696:   "os": [
0697:   "sunos"
0698:   ],
0699:   "engines": {
0700:   "node": ">=18"
0701:   }
0702:   },
0703:   "node_modules/@esbuild/win32-arm64": {
0704:   "version": "0.27.3",
0705:   "resolved": "https://registry.npmjs.org/@esbuild/win32-arm64/-/win32-arm64-0.27.3.tgz",
0706:   "integrity": "sha512-B2t59lWWYrbRDw/tjiWOuzSsFh1Y/E95ofKz7rIVYSQkUYBjfSgf6oeYPNWHToFRr2z
x52JKApIcAS/D5TUBnA==",
0707:   "cpu": [
0708:   "arm64"
0709:   ],
0710:   "dev": true,
0711:   "license": "MIT",
0712:   "optional": true,
0713:   "os": [
0714:   "win32"
0715:   ],
0716:   "engines": {
0717:   "node": ">=18"
0718:   }
0719:   },
0720:   "node_modules/@esbuild/win32-ia32": {
0721:   "version": "0.27.3",
0722:   "resolved": "https://registry.npmjs.org/@esbuild/win32-ia32/-/win32-ia32-0.27.3.tgz",
0723:   "integrity": "sha512-QLKSFeXNS8+tHW7tZpMtjlNb7HKau0QDpwm49u0vUp9y1WOF+PEzkU84y9GqYaAVW8a
H8f3GcBck26jh54cX4Q==",
0724:   "cpu": [
0725:   "ia32"
0726:   ],
0727:   "dev": true,
0728:   "license": "MIT",
0729:   "optional": true,
0730:   "os": [
0731:   "win32"
0732:   ],
0733:   "engines": {

0734:   "node": ">=18"
0735:   }
0736:   },
0737:   "node_modules/@esbuild/win32-x64": {
0738:   "version": "0.27.3",
0739:   "resolved": "https://registry.npmjs.org/@esbuild/win32-x64/-/win32-x64-0.27.3.tgz",
0740:   "integrity": "sha512-4uJGhsxuptu3OcpVAzli+/gWusVGwZZHTlS63hh++ehExkVT8SgiEf7/uC/PclrPPkL
hZqGgCTjd0VWLo6xMqA==",
0741:   "cpu": [
0742:   "x64"
0743:   ],
0744:   "dev": true,
0745:   "license": "MIT",
0746:   "optional": true,
0747:   "os": [
0748:   "win32"
0749:   ],
0750:   "engines": {
0751:   "node": ">=18"
0752:   }
0753:   },
0754:   "node_modules/@eslint-community/eslint-utils": {
0755:   "version": "4.9.1",
0756:   "resolved": "https://registry.npmjs.org/@eslint-community/eslint-utils/-/eslint-utils-4.
9.1.tgz",
0757:   "integrity": "sha512-phrYmNiYppR7znFEdqgfWHXR6NCkZEK7hwWDHZUjit/2/U0r6XvkDl0SYnoM51Hq7Fh
CGdLDT6zxCCOY1hexsQ==",
0758:   "dev": true,
0759:   "license": "MIT",
0760:   "dependencies": {
0761:   "eslint-visitor-keys": "^3.4.3"
0762:   },
0763:   "engines": {
0764:   "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
0765:   },
0766:   "funding": {
0767:   "url": "https://opencollective.com/eslint"
0768:   },
0769:   "peerDependencies": {
0770:   "eslint": "^6.0.0 || ^7.0.0 || >=8.0.0"
0771:   }
0772:   },
0773:   "node_modules/@eslint-community/eslint-utils/node_modules/eslint-visitor-keys": {
0774:   "version": "3.4.3",
0775:   "resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-3.4.3.
tgz",
0776:   "integrity": "sha512-wpc+LXeiyiisxPlEkUzU6svyS1frIO3Mgxj1fdy7Pm8Ygzguax2N3Fa/D/ag1WqbOpr
dI+uY6wMUl8/a2G+iag==",
0777:   "dev": true,
0778:   "license": "Apache-2.0",
0779:   "engines": {
0780:   "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
0781:   },
0782:   "funding": {

0783:   "url": "https://opencollective.com/eslint"
0784:   }
0785:   },
0786:   "node_modules/@eslint-community/regexpp": {
0787:   "version": "4.12.2",
0788:   "resolved": "https://registry.npmjs.org/@eslint-community/regexpp/-/regexpp-4.12.2.tgz",
0789:   "integrity": "sha512-EriSTlt5OC9/7SXkRSCAhfSxxoSUgBm33OH+IkwbdpgoqsSsUg7y3uh+IICI/Qg4BBWr3U2i39RpmycbxMq4ew==",
0790:   "dev": true,
0791:   "license": "MIT",
0792:   "engines": {
0793:   "node": "^12.0.0 || ^14.0.0 || >=16.0.0"
0794:   }
0795:   },
0796:   "node_modules/@eslint/config-array": {
0797:   "version": "0.21.1",
0798:   "resolved": "https://registry.npmjs.org/@eslint/config-array/-/config-array-0.21.1.tgz",
0799:   "integrity": "sha512-aw1gNayWpdI/jSYVgzN5pL0cfzU02GT3NBpeT/DXbx1/1x7ZKxFPd9bwrzygx/qiwIQiJ1sw/zD8qY/kRvlGHA==",
0800:   "dev": true,
0801:   "license": "Apache-2.0",
0802:   "dependencies": {
0803:   "@eslint/object-schema": "^2.1.7",
0804:   "debug": "^4.3.1",
0805:   "minimatch": "^3.1.2"
0806:   },
0807:   "engines": {
0808:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
0809:   }
0810:   },
0811:   "node_modules/@eslint/config-helpers": {
0812:   "version": "0.4.2",
0813:   "resolved": "https://registry.npmjs.org/@eslint/config-helpers/-/config-helpers-0.4.2.tgz",
0814:   "integrity": "sha512-gBrxN88gOIf3R7ja5K9slwNayVcZgK6SOUORm2uBzTeIEfeVaIhOpCtTox3P6R7o2jLFwLFTLnC7kU/RGcYEgw==",
0815:   "dev": true,
0816:   "license": "Apache-2.0",
0817:   "dependencies": {
0818:   "@eslint/core": "^0.17.0"
0819:   },
0820:   "engines": {
0821:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
0822:   }
0823:   },
0824:   "node_modules/@eslint/core": {
0825:   "version": "0.17.0",
0826:   "resolved": "https://registry.npmjs.org/@eslint/core/-/core-0.17.0.tgz",
0827:   "integrity": "sha512-yL/sLrpmtDaFEiUj1osRP4TI2MDz1AddJL+jZ7KSqvBuliN4xqYY54IfdN8qD8Toa6g1iloph1fxQNkjOxrrpQ==",
0828:   "dev": true,
0829:   "license": "Apache-2.0",
0830:   "dependencies": {
0831:   "@types/json-schema": "^7.0.15"

0832:   },
0833:   "engines": {
0834:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
0835:   }
0836:   },
0837:   "node_modules/@eslint/eslintrc": {
0838:   "version": "3.3.4",
0839:   "resolved": "https://registry.npmjs.org/@eslint/eslintrc/-/eslintrc-3.3.4.tgz",
0840:   "integrity": "sha512-4h4MVF8pmBsncB60r0wSJiIeUKTSD4m7FmTFThG8RHlsg9ajqckLm9OraguFGZE4vVd
piI1Q4+hFnisopmG6gQ==",
0841:   "dev": true,
0842:   "license": "MIT",
0843:   "dependencies": {
0844:   "ajv": "^6.14.0",
0845:   "debug": "^4.3.2",
0846:   "espree": "^10.0.1",
0847:   "globals": "^14.0.0",
0848:   "ignore": "^5.2.0",
0849:   "import-fresh": "^3.2.1",
0850:   "js-yaml": "^4.1.1",
0851:   "minimatch": "^3.1.3",
0852:   "strip-json-comments": "^3.1.1"
0853:   },
0854:   "engines": {
0855:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
0856:   },
0857:   "funding": {
0858:   "url": "https://opencollective.com/eslint"
0859:   }
0860:   },
0861:   "node_modules/@eslint/eslintrc/node_modules/globals": {
0862:   "version": "14.0.0",
0863:   "resolved": "https://registry.npmjs.org/globals/-/globals-14.0.0.tgz",
0864:   "integrity": "sha512-oahGvuMGQlPw/ivIYBjVSrWAfWLBeku5tpPE2fOPLi+WHffIWbuh2tCjhyQhTBPMf5E
9jDEH4FOmTYgYwbKwtQ==",
0865:   "dev": true,
0866:   "license": "MIT",
0867:   "engines": {
0868:   "node": ">=18"
0869:   },
0870:   "funding": {
0871:   "url": "https://github.com/sponsors/sindresorhus"
0872:   }
0873:   },
0874:   "node_modules/@eslint/js": {
0875:   "version": "9.39.3",
0876:   "resolved": "https://registry.npmjs.org/@eslint/js/-/js-9.39.3.tgz",
0877:   "integrity": "sha512-1B1VkCq6FuUNlQvlBYb+1jDu/gV297TIs/OeiaSR9l1H27SVW55ONE1e1Vp16NqP683
+xEGzxYtv4XCiDPaQiw==",
0878:   "dev": true,
0879:   "license": "MIT",
0880:   "engines": {
0881:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
0882:   },

0883:   "funding": {
0884:   "url": "https://eslint.org/donate"
0885:   }
0886:   },
0887:   "node_modules/@eslint/object-schema": {
0888:   "version": "2.1.7",
0889:   "resolved": "https://registry.npmjs.org/@eslint/object-schema/-/object-schema-2.1.7.tgz"
,
0890:   "integrity": "sha512-VtAOaymWVfZcmZbp6E2mympDIHvyjXs/12LqWYjVw6qjrfF+VK+fyG33kChz3nnK+SU
5/NeHOqrTEHS8sXO3OA==",
0891:   "dev": true,
0892:   "license": "Apache-2.0",
0893:   "engines": {
0894:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
0895:   }
0896:   },
0897:   "node_modules/@eslint/plugin-kit": {
0898:   "version": "0.4.1",
0899:   "resolved": "https://registry.npmjs.org/@eslint/plugin-kit/-/plugin-kit-0.4.1.tgz",
0900:   "integrity": "sha512-43/qtrDUokr7LJqoF2c3+RInu/t4zfrpYdoSDfYyhg52rwLV6TnOvdG4fXm7IkSB3wE
rkcmJS9iEhjVtOSEjjA==",
0901:   "dev": true,
0902:   "license": "Apache-2.0",
0903:   "dependencies": {
0904:   "@eslint/core": "^0.17.0",
0905:   "levn": "^0.4.1"
0906:   },
0907:   "engines": {
0908:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
0909:   }
0910:   },
0911:   "node_modules/@humanfs/core": {
0912:   "version": "0.19.1",
0913:   "resolved": "https://registry.npmjs.org/@humanfs/core/-/core-0.19.1.tgz",
0914:   "integrity": "sha512-5DyQ4+1JEUzejeK1JGICcideyfUbGixgS9jNgex5nqkW+cY7WZhxBigmieN5Qnw9Zos
SNVC9KQKyb+GUaGyKUA==",
0915:   "dev": true,
0916:   "license": "Apache-2.0",
0917:   "engines": {
0918:   "node": ">=18.18.0"
0919:   }
0920:   },
0921:   "node_modules/@humanfs/node": {
0922:   "version": "0.16.7",
0923:   "resolved": "https://registry.npmjs.org/@humanfs/node/-/node-0.16.7.tgz",
0924:   "integrity": "sha512-/zUx+yOsIrG4Y43Eh2peDeKCxlRt/gET6aHfaKpuq267qXdYDFViVHfMaLyygZOnl0k
GWxFIgsBy8QFuTLUXEQ==",
0925:   "dev": true,
0926:   "license": "Apache-2.0",
0927:   "dependencies": {
0928:   "@humanfs/core": "^0.19.1",
0929:   "@humanwhocodes/retry": "^0.4.0"
0930:   },
0931:   "engines": {

0932:   "node": ">=18.18.0"
0933:   }
0934:   },
0935:   "node_modules/@humanwhocodes/module-importer": {
0936:   "version": "1.0.1",
0937:   "resolved": "https://registry.npmjs.org/@humanwhocodes/module-importer/-/module-importer
-1.0.1.tgz",
0938:   "integrity": "sha512-bxveV4V8v5Yb4ncFTT3rPSgZBOpCkjfK0y4oVVVJwIuDVBRMDXrPyXRL988i5ap9m9b
nyEEjWfm5WkBmtffLfA==",
0939:   "dev": true,
0940:   "license": "Apache-2.0",
0941:   "engines": {
0942:   "node": ">=12.22"
0943:   },
0944:   "funding": {
0945:   "type": "github",
0946:   "url": "https://github.com/sponsors/nzakas"
0947:   }
0948:   },
0949:   "node_modules/@humanwhocodes/retry": {
0950:   "version": "0.4.3",
0951:   "resolved": "https://registry.npmjs.org/@humanwhocodes/retry/-/retry-0.4.3.tgz",
0952:   "integrity": "sha512-bV0Tgo9K4hfPCek+aMAn81RppFKv2ySDQeMoSZuvTASywNTnVJCArCZE2FWqpvIatKu
7VMRLWlR1EazvVhDyhQ==",
0953:   "dev": true,
0954:   "license": "Apache-2.0",
0955:   "engines": {
0956:   "node": ">=18.18"
0957:   },
0958:   "funding": {
0959:   "type": "github",
0960:   "url": "https://github.com/sponsors/nzakas"
0961:   }
0962:   },
0963:   "node_modules/@jridgewell/gen-mapping": {
0964:   "version": "0.3.13",
0965:   "resolved": "https://registry.npmjs.org/@jridgewell/gen-mapping/-/gen-mapping-0.3.13.tgz
",
0966:   "integrity": "sha512-2kkt/7niJ6MgEPxF0bYdQ6etZaA+fQvDcLKckhy1yIQOzaoKjBBjSj63/aLVjYE3qhR
t5dvM+uUyfCg6UKCBbA==",
0967:   "dev": true,
0968:   "license": "MIT",
0969:   "dependencies": {
0970:   "@jridgewell/sourcemap-codec": "^1.5.0",
0971:   "@jridgewell/trace-mapping": "^0.3.24"
0972:   }
0973:   },
0974:   "node_modules/@jridgewell/remapping": {
0975:   "version": "2.3.5",
0976:   "resolved": "https://registry.npmjs.org/@jridgewell/remapping/-/remapping-2.3.5.tgz",
0977:   "integrity": "sha512-LI9u/+laYG4Ds1TDKSJW2YPrIlcVYOwi2fUC6xB43lueCjgxV4lffOCZCtYFiH6TNOX
+tQKXx97T4IKHbhyHEQ==",
0978:   "dev": true,
0979:   "license": "MIT",

0980:   "dependencies": {
0981:   "@jridgewell/gen-mapping": "^0.3.5",
0982:   "@jridgewell/trace-mapping": "^0.3.24"
0983:   }
0984:   },
0985:   "node_modules/@jridgewell/resolve-uri": {
0986:   "version": "3.1.2",
0987:   "resolved": "https://registry.npmjs.org/@jridgewell/resolve-uri/-/resolve-uri-3.1.2.tgz"
,
0988:   "integrity": "sha512-bRISgCIjP20/tbWSPWMEi54QVPRZExkuD9lJL+UIxUKtwVJA8wW1Trb1jMs1RFXo1CB
TNZ/5hpC9QvmKWdopKw==",
0989:   "dev": true,
0990:   "license": "MIT",
0991:   "engines": {
0992:   "node": ">=6.0.0"
0993:   }
0994:   },
0995:   "node_modules/@jridgewell/sourcemap-codec": {
0996:   "version": "1.5.5",
0997:   "resolved": "https://registry.npmjs.org/@jridgewell/sourcemap-codec/-/sourcemap-codec-1.
5.5.tgz",
0998:   "integrity": "sha512-cYQ9310grqxueWbl+WuIUIaiUaDcj7WOq5fVhEljNVgRfOUhY9fy2zTvfoqWsnebh8S
l70VScFbICvJnLKB0Og==",
0999:   "dev": true,
1000:   "license": "MIT"
1001:   },
1002:   "node_modules/@jridgewell/trace-mapping": {
1003:   "version": "0.3.31",
1004:   "resolved": "https://registry.npmjs.org/@jridgewell/trace-mapping/-/trace-mapping-0.3.31
.tgz",
1005:   "integrity": "sha512-zzNR+SdQSDJzc8joaeP8QQoCQr8NuYx2dIIytl1QeBEZHJ9uW6hebsrYgbz8hJwUQao
3TWCMtmfV8Nu1twOLAw==",
1006:   "dev": true,
1007:   "license": "MIT",
1008:   "dependencies": {
1009:   "@jridgewell/resolve-uri": "^3.1.0",
1010:   "@jridgewell/sourcemap-codec": "^1.4.14"
1011:   }
1012:   },
1013:   "node_modules/@rolldown/pluginutils": {
1014:   "version": "1.0.0-rc.3",
1015:   "resolved": "https://registry.npmjs.org/@rolldown/pluginutils/-/pluginutils-1.0.0-rc.3.t
gz",
1016:   "integrity": "sha512-eybk3TjzzzV97Dlj5c+XrBFW57eTNhzod66y9HrBlzJ6NsCrWCp/2kaPS3K9wJmurBC
0Tdw4yPjXKZqlznim3Q==",
1017:   "dev": true,
1018:   "license": "MIT"
1019:   },
1020:   "node_modules/@rollup/rollup-android-arm-eabi": {
1021:   "version": "4.59.0",
1022:   "resolved": "https://registry.npmjs.org/@rollup/rollup-android-arm-eabi/-/rollup-android
-arm-eabi-4.59.0.tgz",
1023:   "integrity": "sha512-upnNBkA6ZH2VKGcBj9Fyl9IGNPULcjXRlg0LLeaioQWueH30p6IXtJEbKAgvyv+mJaM
xSm1l6xwDXYjpEMiLMg==",

1024:   "cpu": [
1025:   "arm"
1026:   ],
1027:   "dev": true,
1028:   "license": "MIT",
1029:   "optional": true,
1030:   "os": [
1031:   "android"
1032:   ]
1033:   },
1034:   "node_modules/@rollup/rollup-android-arm64": {
1035:   "version": "4.59.0",
1036:   "resolved": "https://registry.npmjs.org/@rollup/rollup-android-arm64/-/rollup-android-arm64-4.59.0.tgz",
1037:   "integrity": "sha512-hZ+Zxj3SySm4A/DylsDKZAeVg0mvi++0PYVceVyX7hemkw7OreKdCvW2oQ3T1FMZvCaQXqOTHb8qmBShoqk69Q==",
1038:   "cpu": [
1039:   "arm64"
1040:   ],
1041:   "dev": true,
1042:   "license": "MIT",
1043:   "optional": true,
1044:   "os": [
1045:   "android"
1046:   ]
1047:   },
1048:   "node_modules/@rollup/rollup-darwin-arm64": {
1049:   "version": "4.59.0",
1050:   "resolved": "https://registry.npmjs.org/@rollup/rollup-darwin-arm64/-/rollup-darwin-arm64-4.59.0.tgz",
1051:   "integrity": "sha512-W2Psnbh1J8ZJw0xKAd8zdNgF9HRLkdWwwdWqubSVk0pUuQkoHnv7rx4GiF9rT4t5DIZGAsConRE3AxCdJ4m8rg==",
1052:   "cpu": [
1053:   "arm64"
1054:   ],
1055:   "dev": true,
1056:   "license": "MIT",
1057:   "optional": true,
1058:   "os": [
1059:   "darwin"
1060:   ]
1061:   },
1062:   "node_modules/@rollup/rollup-darwin-x64": {
1063:   "version": "4.59.0",
1064:   "resolved": "https://registry.npmjs.org/@rollup/rollup-darwin-x64/-/rollup-darwin-x64-4.59.0.tgz",
1065:   "integrity": "sha512-ZW2KkwlS4lwTv7ZVsYDiARfFCnSGhzYPdiOU4IM2fDbL+QGlyAbjgSFuqNRbSthybLbIJ915UtZBtmuLrQAT/w==",
1066:   "cpu": [
1067:   "x64"
1068:   ],
1069:   "dev": true,
1070:   "license": "MIT",
1071:   "optional": true,

1072:   "os": [
1073:   "darwin"
1074:   ]
1075:   },
1076:   "node_modules/@rollup/rollup-freebsd-arm64": {
1077:   "version": "4.59.0",
1078:   "resolved": "https://registry.npmjs.org/@rollup/rollup-freebsd-arm64/-/rollup-freebsd-ar
m64-4.59.0.tgz",
1079:   "integrity": "sha512-EsKaJ5ytAu9jI3lonzn3BgG8iRBjV4LxZexygcQbpiU0wU0ATxhNVEpXKfUa0pS05gT
cSDMKpn3Sx+QB9RlTTA==",
1080:   "cpu": [
1081:   "arm64"
1082:   ],
1083:   "dev": true,
1084:   "license": "MIT",
1085:   "optional": true,
1086:   "os": [
1087:   "freebsd"
1088:   ]
1089:   },
1090:   "node_modules/@rollup/rollup-freebsd-x64": {
1091:   "version": "4.59.0",
1092:   "resolved": "https://registry.npmjs.org/@rollup/rollup-freebsd-x64/-/rollup-freebsd-x64-
4.59.0.tgz",
1093:   "integrity": "sha512-d3DuZi2KzTMjImrxoHIAODUZYoUUMsuUiY4SRRcJy6NJoZ6iIqWnJu9IScV9jXysyGM
VuW+KNzZvBLOcpdl3Vg==",
1094:   "cpu": [
1095:   "x64"
1096:   ],
1097:   "dev": true,
1098:   "license": "MIT",
1099:   "optional": true,
1100:   "os": [
1101:   "freebsd"
1102:   ]
1103:   },
1104:   "node_modules/@rollup/rollup-linux-arm-gnueabihf": {
1105:   "version": "4.59.0",
1106:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-arm-gnueabihf/-/rollup-linu
x-arm-gnueabihf-4.59.0.tgz",
1107:   "integrity": "sha512-t4ONHboXi/3E0rT6OZl1pKbl2Vgxf9vJfWgmUoCEVQVxhW6Cw/c8I6hbbu7DAvgp82R
KiH7TpLwxnJeKv2pbsw==",
1108:   "cpu": [
1109:   "arm"
1110:   ],
1111:   "dev": true,
1112:   "license": "MIT",
1113:   "optional": true,
1114:   "os": [
1115:   "linux"
1116:   ]
1117:   },
1118:   "node_modules/@rollup/rollup-linux-arm-musleabihf": {
1119:   "version": "4.59.0",

1120:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-arm-musleabihf/-/rollup-lin
ux-arm-musleabihf-4.59.0.tgz",
1121:   "integrity": "sha512-CikFT7aYPA2ufMD086cVORBYGHffBo4K8MQ4uPS/ZnY54GKj36i196u8U+aDVT2LX4e
SMbyHtyOh7D7Zvk2VvA==",
1122:   "cpu": [
1123:   "arm"
1124:   ],
1125:   "dev": true,
1126:   "license": "MIT",
1127:   "optional": true,
1128:   "os": [
1129:   "linux"
1130:   ]
1131:   },
1132:   "node_modules/@rollup/rollup-linux-arm64-gnu": {
1133:   "version": "4.59.0",
1134:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-arm64-gnu/-/rollup-linux-ar
m64-gnu-4.59.0.tgz",
1135:   "integrity": "sha512-jYgUGk5aLd1nUb1CtQ8E+t5JhLc9x5WdBKew9ZgAXg7DBk0ZHErLHdXM24rfX+bKrFe
+Xp5YuJo54I5HFjGDAA==",
1136:   "cpu": [
1137:   "arm64"
1138:   ],
1139:   "dev": true,
1140:   "license": "MIT",
1141:   "optional": true,
1142:   "os": [
1143:   "linux"
1144:   ]
1145:   },
1146:   "node_modules/@rollup/rollup-linux-arm64-musl": {
1147:   "version": "4.59.0",
1148:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-arm64-musl/-/rollup-linux-a
rm64-musl-4.59.0.tgz",
1149:   "integrity": "sha512-peZRVEdnFWZ5Bh2KeumKG9ty7aCXzzEsHShOZEFiCQlDEepP1dpUl/SrUNXNg13UmZl
+gzVDPsiCwnV1uI0RUA==",
1150:   "cpu": [
1151:   "arm64"
1152:   ],
1153:   "dev": true,
1154:   "license": "MIT",
1155:   "optional": true,
1156:   "os": [
1157:   "linux"
1158:   ]
1159:   },
1160:   "node_modules/@rollup/rollup-linux-loong64-gnu": {
1161:   "version": "4.59.0",
1162:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-loong64-gnu/-/rollup-linux-
loong64-gnu-4.59.0.tgz",
1163:   "integrity": "sha512-gbUSW/97f7+r4gHy3Jlup8zDG190AuodsWnNiXErp9mT90iCy9NKKU0Xwx5k8VlRAIV
2uU9CsMnEFg/xXaOfXg==",
1164:   "cpu": [
1165:   "loong64"

1166:   ],
1167:   "dev": true,
1168:   "license": "MIT",
1169:   "optional": true,
1170:   "os": [
1171:   "linux"
1172:   ]
1173:   },
1174:   "node_modules/@rollup/rollup-linux-loong64-musl": {
1175:   "version": "4.59.0",
1176:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-loong64-musl/-/rollup-linux
-loong64-musl-4.59.0.tgz",
1177:   "integrity": "sha512-yTRONe79E+o0FWFijasoTjtzG9EBedFXJMl888NBEDCDV9I2wGbFFfJQQe63OijbFCU
ZqxpHz1GzpbtSFikJ4Q==",
1178:   "cpu": [
1179:   "loong64"
1180:   ],
1181:   "dev": true,
1182:   "license": "MIT",
1183:   "optional": true,
1184:   "os": [
1185:   "linux"
1186:   ]
1187:   },
1188:   "node_modules/@rollup/rollup-linux-ppc64-gnu": {
1189:   "version": "4.59.0",
1190:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-ppc64-gnu/-/rollup-linux-pp
c64-gnu-4.59.0.tgz",
1191:   "integrity": "sha512-sw1o3tfyk12k3OEpRddF68a1unZ5VCN7zoTNtSn2KndUE+ea3m3ROOKRCZxEpmT9nsG
nogpFP9x6mnLTCaoLkA==",
1192:   "cpu": [
1193:   "ppc64"
1194:   ],
1195:   "dev": true,
1196:   "license": "MIT",
1197:   "optional": true,
1198:   "os": [
1199:   "linux"
1200:   ]
1201:   },
1202:   "node_modules/@rollup/rollup-linux-ppc64-musl": {
1203:   "version": "4.59.0",
1204:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-ppc64-musl/-/rollup-linux-p
pc64-musl-4.59.0.tgz",
1205:   "integrity": "sha512-+2kLtQ4xT3AiIxkzFVFXfsmlZiG5FXYW7ZyIIvGA7Bdeuh9Z0aN4hVyXS/G1E9bTP/v
qszNIN/pUKCk/BTHsKA==",
1206:   "cpu": [
1207:   "ppc64"
1208:   ],
1209:   "dev": true,
1210:   "license": "MIT",
1211:   "optional": true,
1212:   "os": [
1213:   "linux"

1214:   ]
1215:   },
1216:   "node_modules/@rollup/rollup-linux-riscv64-gnu": {
1217:   "version": "4.59.0",
1218:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-riscv64-gnu/-/rollup-linux-riscv64-gnu-4.59.0.tgz",
1219:   "integrity": "sha512-NDYMpsXYJJaj+I7UdwIuHHNxXZ/b/N2hR15NyH3m2qAtb/hHPA4g4SuuvrdxetTdndfj9b1WOmy73kcPRoERUg==",
1220:   "cpu": [
1221:   "riscv64"
1222:   ],
1223:   "dev": true,
1224:   "license": "MIT",
1225:   "optional": true,
1226:   "os": [
1227:   "linux"
1228:   ]
1229:   },
1230:   "node_modules/@rollup/rollup-linux-riscv64-musl": {
1231:   "version": "4.59.0",
1232:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-riscv64-musl/-/rollup-linux-riscv64-musl-4.59.0.tgz",
1233:   "integrity": "sha512-nLckB8WOqHIf1bhymk+oHxvM9D3tyPndZH8i8+35p/1YiVoVswPid2yLzgX7ZJP0KQvnkhM4H6QZ5m0LzbyIAg==",
1234:   "cpu": [
1235:   "riscv64"
1236:   ],
1237:   "dev": true,
1238:   "license": "MIT",
1239:   "optional": true,
1240:   "os": [
1241:   "linux"
1242:   ]
1243:   },
1244:   "node_modules/@rollup/rollup-linux-s390x-gnu": {
1245:   "version": "4.59.0",
1246:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-s390x-gnu/-/rollup-linux-s390x-gnu-4.59.0.tgz",
1247:   "integrity": "sha512-oF87Ie3uAIvORFBpwnCvUzdeYUqi2wY6jRFWJAy1qus/udHFYIkplYRW+wo+GRUP4sKzYdmE1Y3+rY5Gc4ZO+w==",
1248:   "cpu": [
1249:   "s390x"
1250:   ],
1251:   "dev": true,
1252:   "license": "MIT",
1253:   "optional": true,
1254:   "os": [
1255:   "linux"
1256:   ]
1257:   },
1258:   "node_modules/@rollup/rollup-linux-x64-gnu": {
1259:   "version": "4.59.0",
1260:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-x64-gnu/-/rollup-linux-x64-gnu-4.59.0.tgz",

1261:   "integrity": "sha512-3AHmtQq/ppNuUspKAlvA8HtLybkDflkMuLK4DPo77DfthRb71V84/c4MlWJXixZz4ur
uIH4uaa07IqoAkG64fg==",
1262:   "cpu": [
1263:   "x64"
1264:   ],
1265:   "dev": true,
1266:   "license": "MIT",
1267:   "optional": true,
1268:   "os": [
1269:   "linux"
1270:   ]
1271:   },
1272:   "node_modules/@rollup/rollup-linux-x64-musl": {
1273:   "version": "4.59.0",
1274:   "resolved": "https://registry.npmjs.org/@rollup/rollup-linux-x64-musl/-/rollup-linux-x64
-musl-4.59.0.tgz",
1275:   "integrity": "sha512-2UdiwS/9cTAx7qIUZB/fWtToJwvt0Vbo0zmnYt7ED35KPg13Q0ym1g442THLC7VyI6J
fYTP4PiSOWyoMdV2/xg==",
1276:   "cpu": [
1277:   "x64"
1278:   ],
1279:   "dev": true,
1280:   "license": "MIT",
1281:   "optional": true,
1282:   "os": [
1283:   "linux"
1284:   ]
1285:   },
1286:   "node_modules/@rollup/rollup-openbsd-x64": {
1287:   "version": "4.59.0",
1288:   "resolved": "https://registry.npmjs.org/@rollup/rollup-openbsd-x64/-/rollup-openbsd-x64-
4.59.0.tgz",
1289:   "integrity": "sha512-M3bLRAVk6GOwFlPTIxVBSYKUaqfLrn8l0psKinkCFxl4lQvOSz8ZrKDz2gxcBwHFpci
0B6rttydI4IpS4IS/jQ==",
1290:   "cpu": [
1291:   "x64"
1292:   ],
1293:   "dev": true,
1294:   "license": "MIT",
1295:   "optional": true,
1296:   "os": [
1297:   "openbsd"
1298:   ]
1299:   },
1300:   "node_modules/@rollup/rollup-openharmony-arm64": {
1301:   "version": "4.59.0",
1302:   "resolved": "https://registry.npmjs.org/@rollup/rollup-openharmony-arm64/-/rollup-openha
rmony-arm64-4.59.0.tgz",
1303:   "integrity": "sha512-tt9KBJqaqp5i5HUZzoafHZX8b5Q2Fe7UjYERADll83O4fGqJ49O1FsL6LpdzVFQcpwv
nyd0i+K/VSwu/o/nWlA==",
1304:   "cpu": [
1305:   "arm64"
1306:   ],
1307:   "dev": true,

1308:   "license": "MIT",
1309:   "optional": true,
1310:   "os": [
1311:   "openharmony"
1312:   ]
1313:   },
1314:   "node_modules/@rollup/rollup-win32-arm64-msvc": {
1315:   "version": "4.59.0",
1316:   "resolved": "https://registry.npmjs.org/@rollup/rollup-win32-arm64-msvc/-/rollup-win32-a
rm64-msvc-4.59.0.tgz",
1317:   "integrity": "sha512-V5B6mG7OrGTwnxaNUzZTDTjDS7F75PO1ae6MJYdiMu60sq0CqN5CVeVsbhPxalupvTX
8gXVSU9gq+Rx1/hvu6A==",
1318:   "cpu": [
1319:   "arm64"
1320:   ],
1321:   "dev": true,
1322:   "license": "MIT",
1323:   "optional": true,
1324:   "os": [
1325:   "win32"
1326:   ]
1327:   },
1328:   "node_modules/@rollup/rollup-win32-ia32-msvc": {
1329:   "version": "4.59.0",
1330:   "resolved": "https://registry.npmjs.org/@rollup/rollup-win32-ia32-msvc/-/rollup-win32-ia
32-msvc-4.59.0.tgz",
1331:   "integrity": "sha512-UKFMHPuM9R0iBegwzKF4y0C4J9u8C6MEJgFuXTBerMk7EJ92GFVFYBfOZaSGLu6COf7
FxpQNqhNS4c4icUPqxA==",
1332:   "cpu": [
1333:   "ia32"
1334:   ],
1335:   "dev": true,
1336:   "license": "MIT",
1337:   "optional": true,
1338:   "os": [
1339:   "win32"
1340:   ]
1341:   },
1342:   "node_modules/@rollup/rollup-win32-x64-gnu": {
1343:   "version": "4.59.0",
1344:   "resolved": "https://registry.npmjs.org/@rollup/rollup-win32-x64-gnu/-/rollup-win32-x64-
gnu-4.59.0.tgz",
1345:   "integrity": "sha512-laBkYlSS1n2L8fSo1thDNGrCTQMmxjYY5G0WFWjFFYZkKPjsMBsgJfGf4TLxXrF6Ryh
I60L8TMOjBMvXiTcxeA==",
1346:   "cpu": [
1347:   "x64"
1348:   ],
1349:   "dev": true,
1350:   "license": "MIT",
1351:   "optional": true,
1352:   "os": [
1353:   "win32"
1354:   ]
1355:   },

1356:   "node_modules/@rollup/rollup-win32-x64-msvc": {
1357:   "version": "4.59.0",
1358:   "resolved": "https://registry.npmjs.org/@rollup/rollup-win32-x64-msvc/-/rollup-win32-x64
-msvc-4.59.0.tgz",
1359:   "integrity": "sha512-2HRCml6OztYXyJXAvdDXPKcawukWY2GpR5/nxKp4iBgiO3wcoEGkAaqctIbZcNB6KlU
QBIqt8VYkNSj2397EfA==",
1360:   "cpu": [
1361:   "x64"
1362:   ],
1363:   "dev": true,
1364:   "license": "MIT",
1365:   "optional": true,
1366:   "os": [
1367:   "win32"
1368:   ]
1369:   },
1370:   "node_modules/@types/babel__core": {
1371:   "version": "7.20.5",
1372:   "resolved": "https://registry.npmjs.org/@types/babel__core/-/babel__core-7.20.5.tgz",
1373:   "integrity": "sha512-qoQprZvz5wQFJwMDqeseRXWv3rqMvhgpbXFfVyWhbx9X47POIA6i/+dXefEmZKoAgOa
TdaIgNSMqMIU61yRyzA==",
1374:   "dev": true,
1375:   "license": "MIT",
1376:   "dependencies": {
1377:   "@babel/parser": "^7.20.7",
1378:   "@babel/types": "^7.20.7",
1379:   "@types/babel__generator": "*",
1380:   "@types/babel__template": "*",
1381:   "@types/babel__traverse": "*"
1382:   }
1383:   },
1384:   "node_modules/@types/babel__generator": {
1385:   "version": "7.27.0",
1386:   "resolved": "https://registry.npmjs.org/@types/babel__generator/-/babel__generator-7.27.
0.tgz",
1387:   "integrity": "sha512-ufFd2Xi92OAVPYsy+P4n7/U7e68fex0+Ee8gSG9KX7eo084CWiQ4sdxktvdl0bOPupX
tVJPY19zk6EwWqUQ8lg==",
1388:   "dev": true,
1389:   "license": "MIT",
1390:   "dependencies": {
1391:   "@babel/types": "^7.0.0"
1392:   }
1393:   },
1394:   "node_modules/@types/babel__template": {
1395:   "version": "7.4.4",
1396:   "resolved": "https://registry.npmjs.org/@types/babel__template/-/babel__template-7.4.4.t
gz",
1397:   "integrity": "sha512-h/NUaSyG5EyxBIp8YRxo4RMe2/qQgvyowRwVMzhYhBCONbW8PUsg4lkFMrhgZhUe5z3
L3MiLDuvyJ/CaPa2A8A==",
1398:   "dev": true,
1399:   "license": "MIT",
1400:   "dependencies": {
1401:   "@babel/parser": "^7.1.0",
1402:   "@babel/types": "^7.0.0"

1403:   }
1404:   },
1405:   "node_modules/@types/babel__traverse": {
1406:   "version": "7.28.0",
1407:   "resolved": "https://registry.npmjs.org/@types/babel__traverse/-/babel__traverse-7.28.0.
tgz",
1408:   "integrity": "sha512-8PvcXf70gTDZBgt9ptxJ8elBeBjcLOAcOtoO/mPJjtji1+CdGbHgm77om1GrsPxsiE+
uXIpNSK64UYaIwQXd4Q==",
1409:   "dev": true,
1410:   "license": "MIT",
1411:   "dependencies": {
1412:   "@babel/types": "^7.28.2"
1413:   }
1414:   },
1415:   "node_modules/@types/estree": {
1416:   "version": "1.0.8",
1417:   "resolved": "https://registry.npmjs.org/@types/estree/-/estree-1.0.8.tgz",
1418:   "integrity": "sha512-dWHzHa2WqEXI/O1E9OjrocMTKJl2mSrEolh1Iomrv6U+JuNwaHXsXx9bLu5gG7BUWFI
N0skIQJQ/L1rIex4X6w==",
1419:   "dev": true,
1420:   "license": "MIT"
1421:   },
1422:   "node_modules/@types/json-schema": {
1423:   "version": "7.0.15",
1424:   "resolved": "https://registry.npmjs.org/@types/json-schema/-/json-schema-7.0.15.tgz",
1425:   "integrity": "sha512-5+fP8P8MFNC+AyZCDxrB2pkZFPGzqQWUzpSeuuVLvm8VMcorNYavBqoFcxK8bQz4Qsb
n4oUEEem4wDLfcysGHA==",
1426:   "dev": true,
1427:   "license": "MIT"
1428:   },
1429:   "node_modules/@types/node": {
1430:   "version": "24.10.15",
1431:   "resolved": "https://registry.npmjs.org/@types/node/-/node-24.10.15.tgz",
1432:   "integrity": "sha512-BgjLoRuSr0MTI5wA6gMw9Xy0sFudAaUuvrnjgGx9wZ522fYYLA5SYJ+1Y30vTcJEG+D
RCyDHx/gzQVfofYzSdg==",
1433:   "dev": true,
1434:   "license": "MIT",
1435:   "dependencies": {
1436:   "undici-types": "~7.16.0"
1437:   }
1438:   },
1439:   "node_modules/@types/react": {
1440:   "version": "19.2.14",
1441:   "resolved": "https://registry.npmjs.org/@types/react/-/react-19.2.14.tgz",
1442:   "integrity": "sha512-ilcTH/UniCkMdtexkoCN0bI7pMcJDvmQFPvuPvmEaYA/NSfFTAgdUSLAoVjaRJm7+6P
vcM+q1zYOwS4wTYMF9w==",
1443:   "dev": true,
1444:   "license": "MIT",
1445:   "dependencies": {
1446:   "csstype": "^3.2.2"
1447:   }
1448:   },
1449:   "node_modules/@types/react-dom": {
1450:   "version": "19.2.3",

1451:   "resolved": "https://registry.npmjs.org/@types/react-dom/-/react-dom-19.2.3.tgz",
1452:   "integrity": "sha512-jp2L/eY6fn+KgVVQAOqYItbF0VY/YApe5Mz2F0aykSO8gx31bYCZyvSeYxCHKvzHG5eZjc+zyaS5BrBWya2+kQ==",
1453:   "dev": true,
1454:   "license": "MIT",
1455:   "peerDependencies": {
1456:   "@types/react": "^19.2.0"
1457:   }
1458:   },
1459:   "node_modules/@typescript-eslint/eslint-plugin": {
1460:   "version": "8.56.1",
1461:   "resolved": "https://registry.npmjs.org/@typescript-eslint/eslint-plugin/-/eslint-plugin-8.56.1.tgz",
1462:   "integrity": "sha512-Jz9ZztpB37dNC+HU2HI28Bs9QXpzCz+y/twHOwhyrIRdbuVDxSytJNDl6z/aAKlaRIwC7y8wJdkBv7FxYGgi0A==",
1463:   "dev": true,
1464:   "license": "MIT",
1465:   "dependencies": {
1466:   "@eslint-community/regexpp": "^4.12.2",
1467:   "@typescript-eslint/scope-manager": "8.56.1",
1468:   "@typescript-eslint/type-utils": "8.56.1",
1469:   "@typescript-eslint/utils": "8.56.1",
1470:   "@typescript-eslint/visitor-keys": "8.56.1",
1471:   "ignore": "^7.0.5",
1472:   "natural-compare": "^1.4.0",
1473:   "ts-api-utils": "^2.4.0"
1474:   },
1475:   "engines": {
1476:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1477:   },
1478:   "funding": {
1479:   "type": "opencollective",
1480:   "url": "https://opencollective.com/typescript-eslint"
1481:   },
1482:   "peerDependencies": {
1483:   "@typescript-eslint/parser": "^8.56.1",
1484:   "eslint": "^8.57.0 || ^9.0.0 || ^10.0.0",
1485:   "typescript": ">=4.8.4 <6.0.0"
1486:   }
1487:   },
1488:   "node_modules/@typescript-eslint/eslint-plugin/node_modules/ignore": {
1489:   "version": "7.0.5",
1490:   "resolved": "https://registry.npmjs.org/ignore/-/ignore-7.0.5.tgz",
1491:   "integrity": "sha512-Hs59xBNfUIunMFgWAbGX5cq6893IbWg4KnrjbYwX3tx0ztorVgTDA6B2sxf8ejHJ4wz8BqGUMYlnzNBer5NvGg==",
1492:   "dev": true,
1493:   "license": "MIT",
1494:   "engines": {
1495:   "node": ">= 4"
1496:   }
1497:   },
1498:   "node_modules/@typescript-eslint/parser": {
1499:   "version": "8.56.1",
1500:   "resolved": "https://registry.npmjs.org/@typescript-eslint/parser/-/parser-8.56.1.tgz",

1501:   "integrity": "sha512-klQbnPAAiGYFyI02+znpBRLyjL4/BrBd0nyWkdC0s/6xFLkXYQ8OoRrSkqacS1ddVxf/LDyODIKbQ5TgKAf/Fg==",
1502:   "dev": true,
1503:   "license": "MIT",
1504:   "dependencies": {
1505:   "@typescript-eslint/scope-manager": "8.56.1",
1506:   "@typescript-eslint/types": "8.56.1",
1507:   "@typescript-eslint/typescript-estree": "8.56.1",
1508:   "@typescript-eslint/visitor-keys": "8.56.1",
1509:   "debug": "^4.4.3"
1510:   },
1511:   "engines": {
1512:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1513:   },
1514:   "funding": {
1515:   "type": "opencollective",
1516:   "url": "https://opencollective.com/typescript-eslint"
1517:   },
1518:   "peerDependencies": {
1519:   "eslint": "^8.57.0 || ^9.0.0 || ^10.0.0",
1520:   "typescript": ">=4.8.4 <6.0.0"
1521:   }
1522:   },
1523:   "node_modules/@typescript-eslint/project-service": {
1524:   "version": "8.56.1",
1525:   "resolved": "https://registry.npmjs.org/@typescript-eslint/project-service/-/project-service-8.56.1.tgz",
1526:   "integrity": "sha512-TAdqQTzHNNvlVFfR+hu2PDJrURiwKsUvxFn1M0h95BB8ah5jejas08jUWG4dBA68jDMI988IvtfdAI53JzEHOQ==",
1527:   "dev": true,
1528:   "license": "MIT",
1529:   "dependencies": {
1530:   "@typescript-eslint/tsconfig-utils": "^8.56.1",
1531:   "@typescript-eslint/types": "^8.56.1",
1532:   "debug": "^4.4.3"
1533:   },
1534:   "engines": {
1535:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1536:   },
1537:   "funding": {
1538:   "type": "opencollective",
1539:   "url": "https://opencollective.com/typescript-eslint"
1540:   },
1541:   "peerDependencies": {
1542:   "typescript": ">=4.8.4 <6.0.0"
1543:   }
1544:   },
1545:   "node_modules/@typescript-eslint/scope-manager": {
1546:   "version": "8.56.1",
1547:   "resolved": "https://registry.npmjs.org/@typescript-eslint/scope-manager/-/scope-manager-8.56.1.tgz",
1548:   "integrity": "sha512-YAi4VDKcIZp0O4tz/haYKhmIDZFEUPOreKbfdAN3SzUDMcPhJ8QI99xQXqX+HoUVq8cs85eRKnD+rne2UAnj2w==",
1549:   "dev": true,

1550:   "license": "MIT",
1551:   "dependencies": {
1552:   "@typescript-eslint/types": "8.56.1",
1553:   "@typescript-eslint/visitor-keys": "8.56.1"
1554:   },
1555:   "engines": {
1556:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1557:   },
1558:   "funding": {
1559:   "type": "opencollective",
1560:   "url": "https://opencollective.com/typescript-eslint"
1561:   }
1562:   },
1563:   "node_modules/@typescript-eslint/tsconfig-utils": {
1564:   "version": "8.56.1",
1565:   "resolved": "https://registry.npmjs.org/@typescript-eslint/tsconfig-utils/-/tsconfig-uti
ls-8.56.1.tgz",
1566:   "integrity": "sha512-qOtCYzKEeyr3aR9f28mPJqBty7+DBqsdd63eO0yyDwc6vgThj2UjWfJIcsFeSucYydq
cuudMOprZ+x1SpF3ZuQ==",
1567:   "dev": true,
1568:   "license": "MIT",
1569:   "engines": {
1570:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1571:   },
1572:   "funding": {
1573:   "type": "opencollective",
1574:   "url": "https://opencollective.com/typescript-eslint"
1575:   },
1576:   "peerDependencies": {
1577:   "typescript": ">=4.8.4 <6.0.0"
1578:   }
1579:   },
1580:   "node_modules/@typescript-eslint/type-utils": {
1581:   "version": "8.56.1",
1582:   "resolved": "https://registry.npmjs.org/@typescript-eslint/type-utils/-/type-utils-8.56.
1.tgz",
1583:   "integrity": "sha512-yB/7dxi7MgTtGhZdaHCemf7PuwrHMenHjmzgUW1aJpO+bBU43OycnM3Wn+DdvDO/8zz
A9HlhaJ0AUGuvri4oGg==",
1584:   "dev": true,
1585:   "license": "MIT",
1586:   "dependencies": {
1587:   "@typescript-eslint/types": "8.56.1",
1588:   "@typescript-eslint/typescript-estree": "8.56.1",
1589:   "@typescript-eslint/utils": "8.56.1",
1590:   "debug": "^4.4.3",
1591:   "ts-api-utils": "^2.4.0"
1592:   },
1593:   "engines": {
1594:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1595:   },
1596:   "funding": {
1597:   "type": "opencollective",
1598:   "url": "https://opencollective.com/typescript-eslint"
1599:   },

1600:   "peerDependencies": {
1601:   "eslint": "^8.57.0 || ^9.0.0 || ^10.0.0",
1602:   "typescript": ">=4.8.4 <6.0.0"
1603:   }
1604:   },
1605:   "node_modules/@typescript-eslint/types": {
1606:   "version": "8.56.1",
1607:   "resolved": "https://registry.npmjs.org/@typescript-eslint/types/-/types-8.56.1.tgz",
1608:   "integrity": "sha512-dbMkdIUkIkchgGDIv7KLUpa0Mda4IYjo4IAMJUZ+3xNoUXxMsk9YtKpTHSChRS85o+H
9ftm51gsK1dZReY9CVw==",
1609:   "dev": true,
1610:   "license": "MIT",
1611:   "engines": {
1612:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1613:   },
1614:   "funding": {
1615:   "type": "opencollective",
1616:   "url": "https://opencollective.com/typescript-eslint"
1617:   }
1618:   },
1619:   "node_modules/@typescript-eslint/typescript-estree": {
1620:   "version": "8.56.1",
1621:   "resolved": "https://registry.npmjs.org/@typescript-eslint/typescript-estree/-/typescrip
t-estree-8.56.1.tgz",
1622:   "integrity": "sha512-qzUL1qgalIvKWAf9C1HpvBjif+Vm6rcT5wZd4VoMb9+Km3iS3Cv9DY6dMRMDtPnwRAF
yAi7YXJpTIEXLvdfPxg==",
1623:   "dev": true,
1624:   "license": "MIT",
1625:   "dependencies": {
1626:   "@typescript-eslint/project-service": "8.56.1",
1627:   "@typescript-eslint/tsconfig-utils": "8.56.1",
1628:   "@typescript-eslint/types": "8.56.1",
1629:   "@typescript-eslint/visitor-keys": "8.56.1",
1630:   "debug": "^4.4.3",
1631:   "minimatch": "^10.2.2",
1632:   "semver": "^7.7.3",
1633:   "tinyglobby": "^0.2.15",
1634:   "ts-api-utils": "^2.4.0"
1635:   },
1636:   "engines": {
1637:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1638:   },
1639:   "funding": {
1640:   "type": "opencollective",
1641:   "url": "https://opencollective.com/typescript-eslint"
1642:   },
1643:   "peerDependencies": {
1644:   "typescript": ">=4.8.4 <6.0.0"
1645:   }
1646:   },
1647:   "node_modules/@typescript-eslint/typescript-estree/node_modules/balanced-match": {
1648:   "version": "4.0.4",
1649:   "resolved": "https://registry.npmjs.org/balanced-match/-/balanced-match-4.0.4.tgz",

1650:   "integrity": "sha512-BLrgEcRTwX2o6gGxGOCNyMvGSp35YofuYzw9h1IMTRmKqttAZZVU67bdb9Pr2vUHA8+
j3i2tJfjO6C6+4myGTA==",
1651:   "dev": true,
1652:   "license": "MIT",
1653:   "engines": {
1654:   "node": "18 || 20 || >=22"
1655:   }
1656:   },
1657:   "node_modules/@typescript-eslint/typescript-estree/node_modules/brace-expansion": {
1658:   "version": "5.0.3",
1659:   "resolved": "https://registry.npmjs.org/brace-expansion/-/brace-expansion-5.0.3.tgz",
1660:   "integrity": "sha512-fy6KJm2RawA5RcHkLa1z/ScpBeA762UF9KmZQxwIbDtRJrgLzM10depAiEQ+CXYcoiq
W1/m96OAAoke2nE9EeA==",
1661:   "dev": true,
1662:   "license": "MIT",
1663:   "dependencies": {
1664:   "balanced-match": "^4.0.2"
1665:   },
1666:   "engines": {
1667:   "node": "18 || 20 || >=22"
1668:   }
1669:   },
1670:   "node_modules/@typescript-eslint/typescript-estree/node_modules/minimatch": {
1671:   "version": "10.2.4",
1672:   "resolved": "https://registry.npmjs.org/minimatch/-/minimatch-10.2.4.tgz",
1673:   "integrity": "sha512-oRjTw/97aTBN0RHbYCdtF1MQfvusSIBQM0IZEgzl6426+8jSC0nF1a/GmnVLpfB9yyr
6g6FTqWqiZVbxrtaCIg==",
1674:   "dev": true,
1675:   "license": "BlueOak-1.0.0",
1676:   "dependencies": {
1677:   "brace-expansion": "^5.0.2"
1678:   },
1679:   "engines": {
1680:   "node": "18 || 20 || >=22"
1681:   },
1682:   "funding": {
1683:   "url": "https://github.com/sponsors/isaacs"
1684:   }
1685:   },
1686:   "node_modules/@typescript-eslint/typescript-estree/node_modules/semver": {
1687:   "version": "7.7.4",
1688:   "resolved": "https://registry.npmjs.org/semver/-/semver-7.7.4.tgz",
1689:   "integrity": "sha512-vFKC2IEtQnVhpT78h1Yp8wzwrf8CM+MzKMHGJZfBtzhZNycRFnXsHk6E5TxIkkMsgNS
7mdX3AGB7x2QM2di4lA==",
1690:   "dev": true,
1691:   "license": "ISC",
1692:   "bin": {
1693:   "semver": "bin/semver.js"
1694:   },
1695:   "engines": {
1696:   "node": ">=10"
1697:   }
1698:   },
1699:   "node_modules/@typescript-eslint/utils": {

1700:   "version": "8.56.1",
1701:   "resolved": "https://registry.npmjs.org/@typescript-eslint/utils/-/utils-8.56.1.tgz",
1702:   "integrity": "sha512-HPAVNIME3tABJ61siYlHzSWCGtOoeP2RTIaHXFMPqjrQKCGB9OgUVdiNgH7TJS2JNIQ
5qQ4RsAUDuGaGme/KOA==",
1703:   "dev": true,
1704:   "license": "MIT",
1705:   "dependencies": {
1706:   "@eslint-community/eslint-utils": "^4.9.1",
1707:   "@typescript-eslint/scope-manager": "8.56.1",
1708:   "@typescript-eslint/types": "8.56.1",
1709:   "@typescript-eslint/typescript-estree": "8.56.1"
1710:   },
1711:   "engines": {
1712:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1713:   },
1714:   "funding": {
1715:   "type": "opencollective",
1716:   "url": "https://opencollective.com/typescript-eslint"
1717:   },
1718:   "peerDependencies": {
1719:   "eslint": "^8.57.0 || ^9.0.0 || ^10.0.0",
1720:   "typescript": ">=4.8.4 <6.0.0"
1721:   }
1722:   },
1723:   "node_modules/@typescript-eslint/visitor-keys": {
1724:   "version": "8.56.1",
1725:   "resolved": "https://registry.npmjs.org/@typescript-eslint/visitor-keys/-/visitor-keys-8
.56.1.tgz",
1726:   "integrity": "sha512-KiROIzYdEV85YygXw6BI/Dx4fnBlFQu6Mq4QE4MOH9fFnhohw6wX/OAvDY2/C+ut0I3
RSPKenvZJIVYqJNkhEw==",
1727:   "dev": true,
1728:   "license": "MIT",
1729:   "dependencies": {
1730:   "@typescript-eslint/types": "8.56.1",
1731:   "eslint-visitor-keys": "^5.0.0"
1732:   },
1733:   "engines": {
1734:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
1735:   },
1736:   "funding": {
1737:   "type": "opencollective",
1738:   "url": "https://opencollective.com/typescript-eslint"
1739:   }
1740:   },
1741:   "node_modules/@typescript-eslint/visitor-keys/node_modules/eslint-visitor-keys": {
1742:   "version": "5.0.1",
1743:   "resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-5.0.1.
tgz",
1744:   "integrity": "sha512-tD40eHxA35h0PEIZNeIjkHoDR4YjjJp34biM0mDvplBe//mB+IHCqHDGV7pxF+7MklT
vighcCPPZC7ynWyjdTA==",
1745:   "dev": true,
1746:   "license": "Apache-2.0",
1747:   "engines": {
1748:   "node": "^20.19.0 || ^22.13.0 || >=24"

1749:   },
1750:   "funding": {
1751:   "url": "https://opencollective.com/eslint"
1752:   }
1753:   },
1754:   "node_modules/@vitejs/plugin-react": {
1755:   "version": "5.1.4",
1756:   "resolved": "https://registry.npmjs.org/@vitejs/plugin-react/-/plugin-react-5.1.4.tgz",
1757:   "integrity": "sha512-VIcFLdRi/VYRU8OL/puL7QXMYafHmqOnwTZY50U1JPlCNj30PxCMx65c494b1K9be9h
X83KVt0+gTEwTWLqToA==",
1758:   "dev": true,
1759:   "license": "MIT",
1760:   "dependencies": {
1761:   "@babel/core": "^7.29.0",
1762:   "@babel/plugin-transform-react-jsx-self": "^7.27.1",
1763:   "@babel/plugin-transform-react-jsx-source": "^7.27.1",
1764:   "@rolldown/pluginutils": "1.0.0-rc.3",
1765:   "@types/babel__core": "^7.20.5",
1766:   "react-refresh": "^0.18.0"
1767:   },
1768:   "engines": {
1769:   "node": "^20.19.0 || >=22.12.0"
1770:   },
1771:   "peerDependencies": {
1772:   "vite": "^4.2.0 || ^5.0.0 || ^6.0.0 || ^7.0.0"
1773:   }
1774:   },
1775:   "node_modules/acorn": {
1776:   "version": "8.16.0",
1777:   "resolved": "https://registry.npmjs.org/acorn/-/acorn-8.16.0.tgz",
1778:   "integrity": "sha512-UVJyE9MttOsBQIDKw1skb9nAwQuR5wuGD3+82K6JgJlm/Y+KI92oNsMNGZCYdDsVtRH
Sak0pcV5Dno5+4jh9sw==",
1779:   "dev": true,
1780:   "license": "MIT",
1781:   "bin": {
1782:   "acorn": "bin/acorn"
1783:   },
1784:   "engines": {
1785:   "node": ">=0.4.0"
1786:   }
1787:   },
1788:   "node_modules/acorn-jsx": {
1789:   "version": "5.3.2",
1790:   "resolved": "https://registry.npmjs.org/acorn-jsx/-/acorn-jsx-5.3.2.tgz",
1791:   "integrity": "sha512-rq9s+JNhf0IChjtDXxllJ7g41oZk5SlXtp0LHwyA5cejwn7vKmKp4pPri6YEePv2PU6
5sAsegbXtIinmDFDXgQ==",
1792:   "dev": true,
1793:   "license": "MIT",
1794:   "peerDependencies": {
1795:   "acorn": "^6.0.0 || ^7.0.0 || ^8.0.0"
1796:   }
1797:   },
1798:   "node_modules/ajv": {
1799:   "version": "6.14.0",

```
1800:       "resolved": "https://registry.npmjs.org/ajv/-/ajv-6.14.0.tgz",
1801:       "integrity": "sha512-IWrosm/yrn43eiKqkfkHis7QioDleaXQHdDVPKg0FSwwd/DuvyX79TZnFOnYpB7dcsF
AMmtFztZuXPDvSePkFw==",
1802:       "dev": true,
1803:       "license": "MIT",
1804:       "dependencies": {
1805:         "fast-deep-equal": "^3.1.1",
1806:         "fast-json-stable-stringify": "^2.0.0",
1807:         "json-schema-traverse": "^0.4.1",
1808:         "uri-js": "^4.2.2"
1809:       },
1810:       "funding": {
1811:         "type": "github",
1812:         "url": "https://github.com/sponsors/epoberezkin"
1813:       }
1814:     },
1815:     "node_modules/ansi-styles": {
1816:       "version": "4.3.0",
1817:       "resolved": "https://registry.npmjs.org/ansi-styles/-/ansi-styles-4.3.0.tgz",
1818:       "integrity": "sha512-zbB9rCJAT1rbjiVDb2hqKFHNYLxgtk8NURxZ3IZwD3F6NtxbXZQCnnSi1Lkx+IDohdP
lFp222wVALIheZJQSEg==",
1819:       "dev": true,
1820:       "license": "MIT",
1821:       "dependencies": {
1822:         "color-convert": "^2.0.1"
1823:       },
1824:       "engines": {
1825:         "node": ">=8"
1826:       },
1827:       "funding": {
1828:         "url": "https://github.com/chalk/ansi-styles?sponsor=1"
1829:       }
1830:     },
1831:     "node_modules/argparse": {
1832:       "version": "2.0.1",
1833:       "resolved": "https://registry.npmjs.org/argparse/-/argparse-2.0.1.tgz",
1834:       "integrity": "sha512-8+9WqebbFzpX9OR+Wa6O29asIogeRMzcGtAINdpMHHyAg10f05aSFVBbcEqGf/PXw1E
jAZ+q2/bEBg3DvurK3Q==",
1835:       "dev": true,
1836:       "license": "Python-2.0"
1837:     },
1838:     "node_modules/balanced-match": {
1839:       "version": "1.0.2",
1840:       "resolved": "https://registry.npmjs.org/balanced-match/-/balanced-match-1.0.2.tgz",
1841:       "integrity": "sha512-3oSeUO0TMV67hN1AmbXsK4yaqU7tjiHlbxRDZOpH0KW9+CeX4bRAaX0Anxt0tx2MrpR
pWwQaPwIlISEJhYU5Pw==",
1842:       "dev": true,
1843:       "license": "MIT"
1844:     },
1845:     "node_modules/baseline-browser-mapping": {
1846:       "version": "2.10.0",
1847:       "resolved": "https://registry.npmjs.org/baseline-browser-mapping/-/baseline-browser-mapp
ing-2.10.0.tgz",
```

"integrity": "sha512-lIyg0szRfYbiy67j9KN8IyeD7q7hcmqnJ1ddWmNt19ItGpNN64mnllmxUNFIOdOm6by97jlL6wfpTTJrmnjWAA==",
"dev": true,
"license": "Apache-2.0",
"bin": {
"baseline-browser-mapping": "dist/cli.cjs"
},
"engines": {
"node": ">=6.0.0"
}
},
"node_modules/brace-expansion": {
"version": "1.1.12",
"resolved": "https://registry.npmjs.org/brace-expansion/-/brace-expansion-1.1.12.tgz",
"integrity": "sha512-9T9UjW3r0UW5c1Q7GTwllptXwhvYmEzFhzMfZ9H7FQWt+uZePjZPjBP/W1ZEyZ1twGWom5/56TF4lPcqjnDHcg==",
"dev": true,
"license": "MIT",
"dependencies": {
"balanced-match": "^1.0.0",
"concat-map": "0.0.1"
}
},
"node_modules/browserslist": {
"version": "4.28.1",
"resolved": "https://registry.npmjs.org/browserslist/-/browserslist-4.28.1.tgz",
"integrity": "sha512-ZC5Bd0LgJXgwGqUknZY/vkUQ04r8NXnJZ3yYi4vDmSiZmC/pdSN0NbNRPxZpbtO4uAfDUAFffO8IZoM3Gj8IkA==",
"dev": true,
"funding": [
{
"type": "opencollective",
"url": "https://opencollective.com/browserslist"
},
{
"type": "tidelift",
"url": "https://tidelift.com/funding/github/npm/browserslist"
},
{
"type": "github",
"url": "https://github.com/sponsors/ai"
}
],
"license": "MIT",
"dependencies": {
"baseline-browser-mapping": "^2.9.0",
"caniuse-lite": "^1.0.30001759",
"electron-to-chromium": "^1.5.263",
"node-releases": "^2.0.27",
"update-browserslist-db": "^1.2.0"
},
"bin": {
"browserslist": "cli.js"
},

```
      "engines": {
        "node": "^6 || ^7 || ^8 || ^9 || ^10 || ^11 || ^12 || >=13.7"
      }
    },
    "node_modules/callsites": {
      "version": "3.1.0",
      "resolved": "https://registry.npmjs.org/callsites/-/callsites-3.1.0.tgz",
      "integrity": "sha512-P8BjAsXvZS+VIDUI11hHCQEv74YT67YUi5JJFNWIqL235sBmjX4+qx9Muvls5ivyNENctx46xQLQ3aTuE7ssaQ==",
      "dev": true,
      "license": "MIT",
      "engines": {
        "node": ">=6"
      }
    },
    "node_modules/caniuse-lite": {
      "version": "1.0.30001774",
      "resolved": "https://registry.npmjs.org/caniuse-lite/-/caniuse-lite-1.0.30001774.tgz",
      "integrity": "sha512-DDdwPGz99nmIEv216hKSgLD+D4ikHQHjBC/seF98N9CPqRX4M5mSxT9eTV6oyisnJcuzxtZy4n17yKKQYmYQOA==",
      "dev": true,
      "funding": [
        {
          "type": "opencollective",
          "url": "https://opencollective.com/browserslist"
        },
        {
          "type": "tidelift",
          "url": "https://tidelift.com/funding/github/npm/caniuse-lite"
        },
        {
          "type": "github",
          "url": "https://github.com/sponsors/ai"
        }
      ],
      "license": "CC-BY-4.0"
    },
    "node_modules/chalk": {
      "version": "4.1.2",
      "resolved": "https://registry.npmjs.org/chalk/-/chalk-4.1.2.tgz",
      "integrity": "sha512-oKnbhFyRIXpUuez8iBMmyEa4nbj4IOQyuhc/wy9kY7/WVPcwIO9VA668Pu8RkO7+0G76SLROeyw9CpQ061i4mA==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "ansi-styles": "^4.1.0",
        "supports-color": "^7.1.0"
      },
      "engines": {
        "node": ">=10"
      },
      "funding": {
        "url": "https://github.com/chalk/chalk?sponsor=1"
      }
```

1950:   },
1951:   "node_modules/color-convert": {
1952:   "version": "2.0.1",
1953:   "resolved": "https://registry.npmjs.org/color-convert/-/color-convert-2.0.1.tgz",
1954:   "integrity": "sha512-RRECPsj7iu/xb5oKYcsFHSppFNnsj/52OVTRKb4zP5onXwVF3zVmmToNcOfGC+CRDpf
K/U584fMg38ZHCaElKQ==",
1955:   "dev": true,
1956:   "license": "MIT",
1957:   "dependencies": {
1958:   "color-name": "~1.1.4"
1959:   },
1960:   "engines": {
1961:   "node": ">=7.0.0"
1962:   }
1963:   },
1964:   "node_modules/color-name": {
1965:   "version": "1.1.4",
1966:   "resolved": "https://registry.npmjs.org/color-name/-/color-name-1.1.4.tgz",
1967:   "integrity": "sha512-dOy+3AuW3a2wNbZHIuMZpTcgjGuLU/uBL/ubcZF9OXbDo8ff4O8yVp5Bf0efS8uEoYo
5q4Fx7dY9OgQGXgAsQA==",
1968:   "dev": true,
1969:   "license": "MIT"
1970:   },
1971:   "node_modules/concat-map": {
1972:   "version": "0.0.1",
1973:   "resolved": "https://registry.npmjs.org/concat-map/-/concat-map-0.0.1.tgz",
1974:   "integrity": "sha512-/Srv4dswyQNBfohGpz9o6Yb3Gz3SrUDqBH5rTuhGR7ahtlbYKnVxw2bCFMRljaA7EXH
aXZ8wsHdodFvbkhKmqg==",
1975:   "dev": true,
1976:   "license": "MIT"
1977:   },
1978:   "node_modules/convert-source-map": {
1979:   "version": "2.0.0",
1980:   "resolved": "https://registry.npmjs.org/convert-source-map/-/convert-source-map-2.0.0.tg
z",
1981:   "integrity": "sha512-Kvp459HrV2FEJ1CAsi1Ku+MY3kasH19TFykTz2xWmMeq6bk2NU3XXvfJ+Q61m0xktWw
t+1HSYf3JZsTms3aRJg==",
1982:   "dev": true,
1983:   "license": "MIT"
1984:   },
1985:   "node_modules/cookie": {
1986:   "version": "1.1.1",
1987:   "resolved": "https://registry.npmjs.org/cookie/-/cookie-1.1.1.tgz",
1988:   "integrity": "sha512-ei8Aos7ja0weRpFzJnEA9UHJ/7XQmqglbRwnf2ATjcB9Wq874VKH9kfjjirM6UhU2/E
5fFYadylyhFldcqSidQ==",
1989:   "license": "MIT",
1990:   "engines": {
1991:   "node": ">=18"
1992:   },
1993:   "funding": {
1994:   "type": "opencollective",
1995:   "url": "https://opencollective.com/express"
1996:   }
1997:   },

1998:   "node_modules/cross-spawn": {
1999:   "version": "7.0.6",
2000:   "resolved": "https://registry.npmjs.org/cross-spawn/-/cross-spawn-7.0.6.tgz",
2001:   "integrity": "sha512-uV2QOWP2nWzsy2aMp8aRibhi9dlzF5Hgh5SHaB9OiTGEyDTiJJyx0uy51QXdyWbtAHN
ua4XJzUKca3OzKUd3vA==",
2002:   "dev": true,
2003:   "license": "MIT",
2004:   "dependencies": {
2005:   "path-key": "^3.1.0",
2006:   "shebang-command": "^2.0.0",
2007:   "which": "^2.0.1"
2008:   },
2009:   "engines": {
2010:   "node": ">= 8"
2011:   }
2012:   },
2013:   "node_modules/csstype": {
2014:   "version": "3.2.3",
2015:   "resolved": "https://registry.npmjs.org/csstype/-/csstype-3.2.3.tgz",
2016:   "integrity": "sha512-z1HGKcYy2xA8AGQfwrn0PAy+PB7X/GSj3UVJW9qKyn43xWa+gl5nXmU4qqLMRzWVLFC
8KusUX8T/0kCiOYpAIQ==",
2017:   "dev": true,
2018:   "license": "MIT"
2019:   },
2020:   "node_modules/debug": {
2021:   "version": "4.4.3",
2022:   "resolved": "https://registry.npmjs.org/debug/-/debug-4.4.3.tgz",
2023:   "integrity": "sha512-RGwwWnwQvkVfavKVt22FGLw+xYSdzARwm0ru6DhTVA3umU5hZc28V3kO4stgYryrTlL
puvgI9GiijltAjNbcqA==",
2024:   "dev": true,
2025:   "license": "MIT",
2026:   "dependencies": {
2027:   "ms": "^2.1.3"
2028:   },
2029:   "engines": {
2030:   "node": ">=6.0"
2031:   },
2032:   "peerDependenciesMeta": {
2033:   "supports-color": {
2034:   "optional": true
2035:   }
2036:   }
2037:   },
2038:   "node_modules/deep-is": {
2039:   "version": "0.1.4",
2040:   "resolved": "https://registry.npmjs.org/deep-is/-/deep-is-0.1.4.tgz",
2041:   "integrity": "sha512-oIPzksmTg4/MriiaYGO+okXDT7ztn/w3Eptv/+gSIdMdKsJo0u4CfYNFJPy+4SKMuCq
Gw2wxnA+URMg3t8a/bQ==",
2042:   "dev": true,
2043:   "license": "MIT"
2044:   },
2045:   "node_modules/electron-to-chromium": {
2046:   "version": "1.5.302",

2047:   "resolved": "https://registry.npmjs.org/electron-to-chromium/-/electron-to-chromium-1.5.
302.tgz",
2048:   "integrity": "sha512-sM6HAN2LyK82IyPBpznDRqlTQAtuSaO+ShzFiWTvoMJLHyZ+Y39r8VMfHzwbU8MVBzQ
4Wdn85+wlZl2TLGIlwg==",
2049:   "dev": true,
2050:   "license": "ISC"
2051:   },
2052:   "node_modules/esbuild": {
2053:   "version": "0.27.3",
2054:   "resolved": "https://registry.npmjs.org/esbuild/-/esbuild-0.27.3.tgz",
2055:   "integrity": "sha512-8VwMnyGCONIs6cWue2IdpHxHnAjzxnw2Zr7MkVxB2vjmQ2ivqGFb4LEG3SMnv0Gb2F/
G/2yA8zUaiL1gywDCCg==",
2056:   "dev": true,
2057:   "hasInstallScript": true,
2058:   "license": "MIT",
2059:   "bin": {
2060:   "esbuild": "bin/esbuild"
2061:   },
2062:   "engines": {
2063:   "node": ">=18"
2064:   },
2065:   "optionalDependencies": {
2066:   "@esbuild/aix-ppc64": "0.27.3",
2067:   "@esbuild/android-arm": "0.27.3",
2068:   "@esbuild/android-arm64": "0.27.3",
2069:   "@esbuild/android-x64": "0.27.3",
2070:   "@esbuild/darwin-arm64": "0.27.3",
2071:   "@esbuild/darwin-x64": "0.27.3",
2072:   "@esbuild/freebsd-arm64": "0.27.3",
2073:   "@esbuild/freebsd-x64": "0.27.3",
2074:   "@esbuild/linux-arm": "0.27.3",
2075:   "@esbuild/linux-arm64": "0.27.3",
2076:   "@esbuild/linux-ia32": "0.27.3",
2077:   "@esbuild/linux-loong64": "0.27.3",
2078:   "@esbuild/linux-mips64el": "0.27.3",
2079:   "@esbuild/linux-ppc64": "0.27.3",
2080:   "@esbuild/linux-riscv64": "0.27.3",
2081:   "@esbuild/linux-s390x": "0.27.3",
2082:   "@esbuild/linux-x64": "0.27.3",
2083:   "@esbuild/netbsd-arm64": "0.27.3",
2084:   "@esbuild/netbsd-x64": "0.27.3",
2085:   "@esbuild/openbsd-arm64": "0.27.3",
2086:   "@esbuild/openbsd-x64": "0.27.3",
2087:   "@esbuild/openharmony-arm64": "0.27.3",
2088:   "@esbuild/sunos-x64": "0.27.3",
2089:   "@esbuild/win32-arm64": "0.27.3",
2090:   "@esbuild/win32-ia32": "0.27.3",
2091:   "@esbuild/win32-x64": "0.27.3"
2092:   }
2093:   },
2094:   "node_modules/escalade": {
2095:   "version": "3.2.0",
2096:   "resolved": "https://registry.npmjs.org/escalade/-/escalade-3.2.0.tgz",

2097:   "integrity": "sha512-WUj2qlxaQtO4g6Pq5c29GTcWGDyd8itL8zTlipgECz3JesAiiOKotd8JU6otB3PACgG
6xkJUyVhboMS+bje/jA==",
2098:   "dev": true,
2099:   "license": "MIT",
2100:   "engines": {
2101:   "node": ">=6"
2102:   }
2103:   },
2104:   "node_modules/escape-string-regexp": {
2105:   "version": "4.0.0",
2106:   "resolved": "https://registry.npmjs.org/escape-string-regexp/-/escape-string-regexp-4.0.
0.tgz",
2107:   "integrity": "sha512-TtpcNJ3XAzx3Gq8sWRzJaVajRs0uVxA2YAkdb1jm2YkPz4G6egUFAyA3n5vtEIZefPk
5Wa4UXbKuS5fKkJWdgA==",
2108:   "dev": true,
2109:   "license": "MIT",
2110:   "engines": {
2111:   "node": ">=10"
2112:   },
2113:   "funding": {
2114:   "url": "https://github.com/sponsors/sindresorhus"
2115:   }
2116:   },
2117:   "node_modules/eslint": {
2118:   "version": "9.39.3",
2119:   "resolved": "https://registry.npmjs.org/eslint/-/eslint-9.39.3.tgz",
2120:   "integrity": "sha512-VmQ+sifHUbI/IcSopBCF/HO3YiHQx/AVd3UVyYL6weuwW+HvON9VYn5l6Zl1WZzPWXP
NZrSQpxwkkZ/VuvJZzg==",
2121:   "dev": true,
2122:   "license": "MIT",
2123:   "dependencies": {
2124:   "@eslint-community/eslint-utils": "^4.8.0",
2125:   "@eslint-community/regexpp": "^4.12.1",
2126:   "@eslint/config-array": "^0.21.1",
2127:   "@eslint/config-helpers": "^0.4.2",
2128:   "@eslint/core": "^0.17.0",
2129:   "@eslint/eslintrc": "^3.3.1",
2130:   "@eslint/js": "9.39.3",
2131:   "@eslint/plugin-kit": "^0.4.1",
2132:   "@humanfs/node": "^0.16.6",
2133:   "@humanwhocodes/module-importer": "^1.0.1",
2134:   "@humanwhocodes/retry": "^0.4.2",
2135:   "@types/estree": "^1.0.6",
2136:   "ajv": "^6.12.4",
2137:   "chalk": "^4.0.0",
2138:   "cross-spawn": "^7.0.6",
2139:   "debug": "^4.3.2",
2140:   "escape-string-regexp": "^4.0.0",
2141:   "eslint-scope": "^8.4.0",
2142:   "eslint-visitor-keys": "^4.2.1",
2143:   "espree": "^10.4.0",
2144:   "esquery": "^1.5.0",
2145:   "esutils": "^2.0.2",
2146:   "fast-deep-equal": "^3.1.3",

2147:   "file-entry-cache": "^8.0.0",
2148:   "find-up": "^5.0.0",
2149:   "glob-parent": "^6.0.2",
2150:   "ignore": "^5.2.0",
2151:   "imurmurhash": "^0.1.4",
2152:   "is-glob": "^4.0.0",
2153:   "json-stable-stringify-without-jsonify": "^1.0.1",
2154:   "lodash.merge": "^4.6.2",
2155:   "minimatch": "^3.1.2",
2156:   "natural-compare": "^1.4.0",
2157:   "optionator": "^0.9.3"
2158:   },
2159:   "bin": {
2160:   "eslint": "bin/eslint.js"
2161:   },
2162:   "engines": {
2163:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
2164:   },
2165:   "funding": {
2166:   "url": "https://eslint.org/donate"
2167:   },
2168:   "peerDependencies": {
2169:   "jiti": "*"
2170:   },
2171:   "peerDependenciesMeta": {
2172:   "jiti": {
2173:   "optional": true
2174:   }
2175:   }
2176:   },
2177:   "node_modules/eslint-plugin-react-hooks": {
2178:   "version": "7.0.1",
2179:   "resolved": "https://registry.npmjs.org/eslint-plugin-react-hooks/-/eslint-plugin-react-
hooks-7.0.1.tgz",
2180:   "integrity": "sha512-O0d0m04evaNzEPoSW+59Mezf8Qt0InfgGIBJnpC0h3NH/WjUAR7BIKUfysC6todmtiZ
/A0oUVS8Gce0WhBrHsA==",
2181:   "dev": true,
2182:   "license": "MIT",
2183:   "dependencies": {
2184:   "@babel/core": "^7.24.4",
2185:   "@babel/parser": "^7.24.4",
2186:   "hermes-parser": "^0.25.1",
2187:   "zod": "^3.25.0 || ^4.0.0",
2188:   "zod-validation-error": "^3.5.0 || ^4.0.0"
2189:   },
2190:   "engines": {
2191:   "node": ">=18"
2192:   },
2193:   "peerDependencies": {
2194:   "eslint": "^3.0.0 || ^4.0.0 || ^5.0.0 || ^6.0.0 || ^7.0.0 || ^8.0.0-0 || ^9.0.0"
2195:   }
2196:   },
2197:   "node_modules/eslint-plugin-react-refresh": {
2198:   "version": "0.4.26",

2199:   "resolved": "https://registry.npmjs.org/eslint-plugin-react-refresh/-/eslint-plugin-reac
t-refresh-0.4.26.tgz",
2200:   "integrity": "sha512-1RETEylht2O6FM/MvgnyvT+8K21wLqDNg4qD51Zj3guhjt433XbnnkVttHMyaVyAFD0
3QSV4LPS5iE3VQmO7XQ==",
2201:   "dev": true,
2202:   "license": "MIT",
2203:   "peerDependencies": {
2204:   "eslint": ">=8.40"
2205:   }
2206:   },
2207:   "node_modules/eslint-scope": {
2208:   "version": "8.4.0",
2209:   "resolved": "https://registry.npmjs.org/eslint-scope/-/eslint-scope-8.4.0.tgz",
2210:   "integrity": "sha512-sNXOfKCn74rt8RICKMvJS7XKV/Xk9kA7DyJr8mJik3S7Cwgy3qlkkmyS2uQB3jiJg6V
NdZd/pDBJu0nvG2NlTg==",
2211:   "dev": true,
2212:   "license": "BSD-2-Clause",
2213:   "dependencies": {
2214:   "esrecurse": "^4.3.0",
2215:   "estraverse": "^5.2.0"
2216:   },
2217:   "engines": {
2218:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
2219:   },
2220:   "funding": {
2221:   "url": "https://opencollective.com/eslint"
2222:   }
2223:   },
2224:   "node_modules/eslint-visitor-keys": {
2225:   "version": "4.2.1",
2226:   "resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-4.2.1.
tgz",
2227:   "integrity": "sha512-Uhdk5sfqcee/9H/rCOJikYz67o0a2Tw2hGRPOG2Y1R2dg7brRe1uG0yaNQDHu+TO/uQ
PF/5eCapvYSmHUjt7JQ==",
2228:   "dev": true,
2229:   "license": "Apache-2.0",
2230:   "engines": {
2231:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
2232:   },
2233:   "funding": {
2234:   "url": "https://opencollective.com/eslint"
2235:   }
2236:   },
2237:   "node_modules/espree": {
2238:   "version": "10.4.0",
2239:   "resolved": "https://registry.npmjs.org/espree/-/espree-10.4.0.tgz",
2240:   "integrity": "sha512-j6PAQ2uUr79PZhBjP5C5fhl8e39FmRnOjsD5lGnWrFU8i2G776tBK7+nP8KuQUTTyAZ
UwfQqXAgrVH5MbH9CYQ==",
2241:   "dev": true,
2242:   "license": "BSD-2-Clause",
2243:   "dependencies": {
2244:   "acorn": "^8.15.0",
2245:   "acorn-jsx": "^5.3.2",
2246:   "eslint-visitor-keys": "^4.2.1"

2247:   },
2248:   "engines": {
2249:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
2250:   },
2251:   "funding": {
2252:   "url": "https://opencollective.com/eslint"
2253:   }
2254:   },
2255:   "node_modules/esquery": {
2256:   "version": "1.7.0",
2257:   "resolved": "https://registry.npmjs.org/esquery/-/esquery-1.7.0.tgz",
2258:   "integrity": "sha512-Ap6G0WQwcU/LHsvLwON1fAQX9Zp0A2Y6Y/cJBl9r/JbW90Zyg4/zbG6zzKa2OTALELa
rYHmKu0GhpM5EO+7T0g==",
2259:   "dev": true,
2260:   "license": "BSD-3-Clause",
2261:   "dependencies": {
2262:   "estraverse": "^5.1.0"
2263:   },
2264:   "engines": {
2265:   "node": ">=0.10"
2266:   }
2267:   },
2268:   "node_modules/esrecurse": {
2269:   "version": "4.3.0",
2270:   "resolved": "https://registry.npmjs.org/esrecurse/-/esrecurse-4.3.0.tgz",
2271:   "integrity": "sha512-KmfKL3b6G+RXvP8N1vr3Tq1kL/oCFgn2NYXEtqP8/L3pKapUA4G8cFVaoF3SU323CD4
XypR/ffioHmkti6/Tag==",
2272:   "dev": true,
2273:   "license": "BSD-2-Clause",
2274:   "dependencies": {
2275:   "estraverse": "^5.2.0"
2276:   },
2277:   "engines": {
2278:   "node": ">=4.0"
2279:   }
2280:   },
2281:   "node_modules/estraverse": {
2282:   "version": "5.3.0",
2283:   "resolved": "https://registry.npmjs.org/estraverse/-/estraverse-5.3.0.tgz",
2284:   "integrity": "sha512-MMdARuVEQziNTeJD8DgMqmhwR11BRQ/cBP+pLtYdSTnf3MIO8fFeiINEbX36ZdNlfU/
7A9f3gUw49B3oQsvwBA==",
2285:   "dev": true,
2286:   "license": "BSD-2-Clause",
2287:   "engines": {
2288:   "node": ">=4.0"
2289:   }
2290:   },
2291:   "node_modules/esutils": {
2292:   "version": "2.0.3",
2293:   "resolved": "https://registry.npmjs.org/esutils/-/esutils-2.0.3.tgz",
2294:   "integrity": "sha512-kVscqXk4OCp68SZ0dkgEKVi6/8ij300KBWTJq32P/dYeWTSwK41WyTxalN1eRmA5Z9U
U/LX9D7FWSmV9SAYx6g==",
2295:   "dev": true,
2296:   "license": "BSD-2-Clause",

2297:   "engines": {
2298:   "node": ">=0.10.0"
2299:   }
2300:   },
2301:   "node_modules/fast-deep-equal": {
2302:   "version": "3.1.3",
2303:   "resolved": "https://registry.npmjs.org/fast-deep-equal/-/fast-deep-equal-3.1.3.tgz",
2304:   "integrity": "sha512-f3qQ9oQy9j2AhBe/H9VC91wLmKBCCU/gDOnKNAYG5hswO7BLKj09Hc5HYNz9cGI++xl
pDCIgDaitVs03ATR84Q==",
2305:   "dev": true,
2306:   "license": "MIT"
2307:   },
2308:   "node_modules/fast-json-stable-stringify": {
2309:   "version": "2.1.0",
2310:   "resolved": "https://registry.npmjs.org/fast-json-stable-stringify/-/fast-json-stable-st
ringify-2.1.0.tgz",
2311:   "integrity": "sha512-lhd/wF+Lk98HZoTCtlVraHtfh5XYijIjalXck7saUtuanSDyLMxnHhSXEDJqHxD7msR
8D0uCmqlkwjCV8xvwHw==",
2312:   "dev": true,
2313:   "license": "MIT"
2314:   },
2315:   "node_modules/fast-levenshtein": {
2316:   "version": "2.0.6",
2317:   "resolved": "https://registry.npmjs.org/fast-levenshtein/-/fast-levenshtein-2.0.6.tgz",
2318:   "integrity": "sha512-DCXu6Ifhqcks7TZKY3Hxp3y6qphY5SJZmrWMDrKcERSOXWQdMhU9Ig/PYrzyw/ul9jO
Iyh0N4M0tbC5hodg8dw==",
2319:   "dev": true,
2320:   "license": "MIT"
2321:   },
2322:   "node_modules/fdir": {
2323:   "version": "6.5.0",
2324:   "resolved": "https://registry.npmjs.org/fdir/-/fdir-6.5.0.tgz",
2325:   "integrity": "sha512-tIbYtZbucOs0BRGqPJkshJUYdL+SDH7dVM8gjy+ERp3WAUjLEFJE+02kanyHtwjWOnw
rKYBiwAmM0p4kLJAnXg==",
2326:   "dev": true,
2327:   "license": "MIT",
2328:   "engines": {
2329:   "node": ">=12.0.0"
2330:   },
2331:   "peerDependencies": {
2332:   "picomatch": "^3 || ^4"
2333:   },
2334:   "peerDependenciesMeta": {
2335:   "picomatch": {
2336:   "optional": true
2337:   }
2338:   }
2339:   },
2340:   "node_modules/file-entry-cache": {
2341:   "version": "8.0.0",
2342:   "resolved": "https://registry.npmjs.org/file-entry-cache/-/file-entry-cache-8.0.0.tgz",
2343:   "integrity": "sha512-XXTUwCvisa5oacNGRP9SfNtYBNAMi+RPwBFmblZEF7N7swHYQS6/Zfk7SRwx4D5j3CH
211YNRco1DEMNVfZCnQ==",
2344:   "dev": true,

```
      "license": "MIT",
      "dependencies": {
        "flat-cache": "^4.0.0"
      },
      "engines": {
        "node": ">=16.0.0"
      }
    },
    "node_modules/find-up": {
      "version": "5.0.0",
      "resolved": "https://registry.npmjs.org/find-up/-/find-up-5.0.0.tgz",
      "integrity": "sha512-78/PXT1wlLLDgTzDs7sjq9hzz0vXD+zn+7wypEe4fXQxCmdmqfGsEPQxmiCSQI3ajFV91bVSsvNtrJRiW6nGng==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "locate-path": "^6.0.0",
        "path-exists": "^4.0.0"
      },
      "engines": {
        "node": ">=10"
      },
      "funding": {
        "url": "https://github.com/sponsors/sindresorhus"
      }
    },
    "node_modules/flat-cache": {
      "version": "4.0.1",
      "resolved": "https://registry.npmjs.org/flat-cache/-/flat-cache-4.0.1.tgz",
      "integrity": "sha512-f7ccFPK3SXFHpx15UIGyRJ/FJQctuKZ0zVuN3frBo4HnK3cay9VEW0R6yPYFHC0AgqhukPzKjq22t5DmAyqGyw==",
      "dev": true,
      "license": "MIT",
      "dependencies": {
        "flatted": "^3.2.9",
        "keyv": "^4.5.4"
      },
      "engines": {
        "node": ">=16"
      }
    },
    "node_modules/flatted": {
      "version": "3.3.3",
      "resolved": "https://registry.npmjs.org/flatted/-/flatted-3.3.3.tgz",
      "integrity": "sha512-GX+ysw4PBCz0PzosHDepZGANEuFCMLrnRTiEy9McGjmkCQYwRq4A/X786G/fjM/+OjsWSU1ZrY5qyARZmO/uwg==",
      "dev": true,
      "license": "ISC"
    },
    "node_modules/fsevents": {
      "version": "2.3.3",
      "resolved": "https://registry.npmjs.org/fsevents/-/fsevents-2.3.3.tgz",
      "integrity": "sha512-5xoDfX+fL7faATnagmWPpbFtwh/R77WmMMqqHGS65C3vvB0YHrgF+B1YmZ3441tMj5n63k0212XNoJwzlhffQw==",
```

2395:   "dev": true,
2396:   "hasInstallScript": true,
2397:   "license": "MIT",
2398:   "optional": true,
2399:   "os": [
2400:   "darwin"
2401:   ],
2402:   "engines": {
2403:   "node": "^8.16.0 || ^10.6.0 || >=11.0.0"
2404:   }
2405:   },
2406:   "node_modules/gensync": {
2407:   "version": "1.0.0-beta.2",
2408:   "resolved": "https://registry.npmjs.org/gensync/-/gensync-1.0.0-beta.2.tgz",
2409:   "integrity": "sha512-3hN7NaskYvMDLQY55gnW3NQ+mesEAepTqlg+VEbj7zzqEMBVNhzcGYYeqFo/TlYz6eQ
iFcp1HcsCZO+nGgS8zg==",
2410:   "dev": true,
2411:   "license": "MIT",
2412:   "engines": {
2413:   "node": ">=6.9.0"
2414:   }
2415:   },
2416:   "node_modules/glob-parent": {
2417:   "version": "6.0.2",
2418:   "resolved": "https://registry.npmjs.org/glob-parent/-/glob-parent-6.0.2.tgz",
2419:   "integrity": "sha512-XxwI8EOhVQgWp6iDL+3b0r86f4d6AX6zSU55HfB4ydCEuXLXc5FcYeOu+nnGftS4TEj
u/11rt4KJPTMgbfmv4A==",
2420:   "dev": true,
2421:   "license": "ISC",
2422:   "dependencies": {
2423:   "is-glob": "^4.0.3"
2424:   },
2425:   "engines": {
2426:   "node": ">=10.13.0"
2427:   }
2428:   },
2429:   "node_modules/globals": {
2430:   "version": "16.5.0",
2431:   "resolved": "https://registry.npmjs.org/globals/-/globals-16.5.0.tgz",
2432:   "integrity": "sha512-c/c15i26VrJ4IRt5Z89DnIzCGDn9EcebibhAOjw5ibqEHsE1wLUgkPn9RDmNcUKyU87
GeaL633nyJ+pplFR2ZQ==",
2433:   "dev": true,
2434:   "license": "MIT",
2435:   "engines": {
2436:   "node": ">=18"
2437:   },
2438:   "funding": {
2439:   "url": "https://github.com/sponsors/sindresorhus"
2440:   }
2441:   },
2442:   "node_modules/has-flag": {
2443:   "version": "4.0.0",
2444:   "resolved": "https://registry.npmjs.org/has-flag/-/has-flag-4.0.0.tgz",

2445:   "integrity": "sha512-EykJT/Q1KjTWctppgIAgfSO0tKVuZUjhgMr17kqTumMl6Afv3EISleU7qZUzoXDFTAH
TDC4NOoG/ZxU3EvlMPQ==",
2446:   "dev": true,
2447:   "license": "MIT",
2448:   "engines": {
2449:   "node": ">=8"
2450:   }
2451:   },
2452:   "node_modules/hermes-estree": {
2453:   "version": "0.25.1",
2454:   "resolved": "https://registry.npmjs.org/hermes-estree/-/hermes-estree-0.25.1.tgz",
2455:   "integrity": "sha512-0wUoCcLp+5Ev5pDW2OriHC2MJCbwLwuRx+gAqMTOkGKJJiBCLjtrvy4PWUGn6MIVefe
cRpzoOZ/UV6iGdOr+Cw==",
2456:   "dev": true,
2457:   "license": "MIT"
2458:   },
2459:   "node_modules/hermes-parser": {
2460:   "version": "0.25.1",
2461:   "resolved": "https://registry.npmjs.org/hermes-parser/-/hermes-parser-0.25.1.tgz",
2462:   "integrity": "sha512-6pEjquH3rqaI6cYAXYPcz9MS4rY6R4ngRgrgfDshRptUZIc3lw0MCIJIGDj9++mfySO
uPTHB4nrSW99BCvOPIA==",
2463:   "dev": true,
2464:   "license": "MIT",
2465:   "dependencies": {
2466:   "hermes-estree": "0.25.1"
2467:   }
2468:   },
2469:   "node_modules/ignore": {
2470:   "version": "5.3.2",
2471:   "resolved": "https://registry.npmjs.org/ignore/-/ignore-5.3.2.tgz",
2472:   "integrity": "sha512-hsBTNUqQTDwkWtcdYI2i06Y/nUBEsNEDJKjWdigLvegy8kDuJAS8uRlpkkcQpyEXL0Z
/pjDy5HBmMjRCJ2gq+g==",
2473:   "dev": true,
2474:   "license": "MIT",
2475:   "engines": {
2476:   "node": ">= 4"
2477:   }
2478:   },
2479:   "node_modules/import-fresh": {
2480:   "version": "3.3.1",
2481:   "resolved": "https://registry.npmjs.org/import-fresh/-/import-fresh-3.3.1.tgz",
2482:   "integrity": "sha512-TR3KfrTZTYLPB6jUjfx6MF9WcWrHL9su5TObK4ZkYgBdWKPOFoSoQIdEuTuR82pmtxH
2spWG9h6etwfr1pLBqQ==",
2483:   "dev": true,
2484:   "license": "MIT",
2485:   "dependencies": {
2486:   "parent-module": "^1.0.0",
2487:   "resolve-from": "^4.0.0"
2488:   },
2489:   "engines": {
2490:   "node": ">=6"
2491:   },
2492:   "funding": {
2493:   "url": "https://github.com/sponsors/sindresorhus"

2494:   }
2495:   },
2496:   "node_modules/imurmurhash": {
2497:   "version": "0.1.4",
2498:   "resolved": "https://registry.npmjs.org/imurmurhash/-/imurmurhash-0.1.4.tgz",
2499:   "integrity": "sha512-JmXMZ6wuvDmLiHEml9ykzqO6lwFbof0GG4IkcGaENdCRDDmMVnny7s5HsIgHCbaq0w2
MyPhDqkhTUgS2LU2PHA==",
2500:   "dev": true,
2501:   "license": "MIT",
2502:   "engines": {
2503:   "node": ">=0.8.19"
2504:   }
2505:   },
2506:   "node_modules/is-extglob": {
2507:   "version": "2.1.1",
2508:   "resolved": "https://registry.npmjs.org/is-extglob/-/is-extglob-2.1.1.tgz",
2509:   "integrity": "sha512-SbKbANkN603Vi4jEZv49LeVJMn4yGwsbzZworEoyEiutsN3nJYdbO36zfhGJ6QEDpOZ
IFkDtnq5JRxmvl3jsoQ==",
2510:   "dev": true,
2511:   "license": "MIT",
2512:   "engines": {
2513:   "node": ">=0.10.0"
2514:   }
2515:   },
2516:   "node_modules/is-glob": {
2517:   "version": "4.0.3",
2518:   "resolved": "https://registry.npmjs.org/is-glob/-/is-glob-4.0.3.tgz",
2519:   "integrity": "sha512-xelSayHH36ZgE7ZWhli7pW34hNbNl8Ojv5KVmkJD4hBdD3th8Tfk9vYasLM+mXWOZhF
kgZfxhLSnrwRr4elSSg==",
2520:   "dev": true,
2521:   "license": "MIT",
2522:   "dependencies": {
2523:   "is-extglob": "^2.1.1"
2524:   },
2525:   "engines": {
2526:   "node": ">=0.10.0"
2527:   }
2528:   },
2529:   "node_modules/isexe": {
2530:   "version": "2.0.0",
2531:   "resolved": "https://registry.npmjs.org/isexe/-/isexe-2.0.0.tgz",
2532:   "integrity": "sha512-RHxMLp9lnKHGHRng9QFhRCMbYAcVpn69smSGcq3f36xjgVVWThj4qqLbTLlq7Ssj8B+
fIQ1EuCEGI2lKsyQeIw==",
2533:   "dev": true,
2534:   "license": "ISC"
2535:   },
2536:   "node_modules/js-tokens": {
2537:   "version": "4.0.0",
2538:   "resolved": "https://registry.npmjs.org/js-tokens/-/js-tokens-4.0.0.tgz",
2539:   "integrity": "sha512-RdJUflcE3cUzKiMqQgsCu06FPu9UdIJO0beYbPhHN4k6apgJtifcoCtT9bcxOpYBtpD
2kCM6Sbzg4CausW/PKQ==",
2540:   "dev": true,
2541:   "license": "MIT"
2542:   },

2543:   "node_modules/js-yaml": {
2544:   "version": "4.1.1",
2545:   "resolved": "https://registry.npmjs.org/js-yaml/-/js-yaml-4.1.1.tgz",
2546:   "integrity": "sha512-qQKT4zQxXl8lLwBtHMWwaTcGfFOZviOJet3Oy/xmGk2gZH677CJM9EvtfdSkgWcATZhj/55JZ0rmy3myCT5lsA==",
2547:   "dev": true,
2548:   "license": "MIT",
2549:   "dependencies": {
2550:   "argparse": "^2.0.1"
2551:   },
2552:   "bin": {
2553:   "js-yaml": "bin/js-yaml.js"
2554:   }
2555:   },
2556:   "node_modules/jsesc": {
2557:   "version": "3.1.0",
2558:   "resolved": "https://registry.npmjs.org/jsesc/-/jsesc-3.1.0.tgz",
2559:   "integrity": "sha512-/sM3dO2FOzXjKQhJuo0Q173wf2KOo8t4I8vHy6lF9poUp7bKT0/NHE8fPX23Pwfhnykfqn C2xRxOnVw5XuGIaA==",
2560:   "dev": true,
2561:   "license": "MIT",
2562:   "bin": {
2563:   "jsesc": "bin/jsesc"
2564:   },
2565:   "engines": {
2566:   "node": ">=6"
2567:   }
2568:   },
2569:   "node_modules/json-buffer": {
2570:   "version": "3.0.1",
2571:   "resolved": "https://registry.npmjs.org/json-buffer/-/json-buffer-3.0.1.tgz",
2572:   "integrity": "sha512-4bV5BfR2mqfQTJm+V5tPPdf+ZpuhiIvTuAB5g8kcrXOZpTT/QwwVRWBywX1ozr6lEuPdbHxwaJlm9G6mI2sfSQ==",
2573:   "dev": true,
2574:   "license": "MIT"
2575:   },
2576:   "node_modules/json-schema-traverse": {
2577:   "version": "0.4.1",
2578:   "resolved": "https://registry.npmjs.org/json-schema-traverse/-/json-schema-traverse-0.4.1.tgz",
2579:   "integrity": "sha512-xbbCH5dCYU5T8LcEhhuh7HJ88HXuW3qsI3Y0zOZFKfZEHcpWiHU/Jxzk629Brsab/mMiHQti9wMP+845RPe3Vg==",
2580:   "dev": true,
2581:   "license": "MIT"
2582:   },
2583:   "node_modules/json-stable-stringify-without-jsonify": {
2584:   "version": "1.0.1",
2585:   "resolved": "https://registry.npmjs.org/json-stable-stringify-without-jsonify/-/json-stable-stringify-without-jsonify-1.0.1.tgz",
2586:   "integrity": "sha512-Bdboy+l7tA3OGW6FjyFHWkP5LuByj1Tk33Ljyq0axyzdk9//JSi2u3fP1QSmd1KNwq6VOKYGlAu87CisVir6Pw==",
2587:   "dev": true,
2588:   "license": "MIT"
2589:   },

"node_modules/json5": {
"version": "2.2.3",
"resolved": "https://registry.npmjs.org/json5/-/json5-2.2.3.tgz",
"integrity": "sha512-XmOWe7eyHYH14cLdVPoyg+GOH3rYX++KpzrylJwSW98t3Nk+U8XOl8FWKOgwtzdb8lXGf6zYwDUzeHMWfxasyg==",
"dev": true,
"license": "MIT",
"bin": {
"json5": "lib/cli.js"
},
"engines": {
"node": ">=6"
}
},
"node_modules/keyv": {
"version": "4.5.4",
"resolved": "https://registry.npmjs.org/keyv/-/keyv-4.5.4.tgz",
"integrity": "sha512-oxVHkHR/EJf2CNXnWxRLW6mg7JyCCUcG0DtEGmL2ctUo1PNTin1PUil+r/+4r5MpVgC/fn1kjsx7mjSujKqIpw==",
"dev": true,
"license": "MIT",
"dependencies": {
"json-buffer": "3.0.1"
}
},
"node_modules/levn": {
"version": "0.4.1",
"resolved": "https://registry.npmjs.org/levn/-/levn-0.4.1.tgz",
"integrity": "sha512-+bT2uH4E5LGE7h/n3evcS/sQlJXCpIp6ym8OWJ5eV6+67Dsql/LaaT7qJBAt2rzfoa/5QBGBhxDix1dMt2kQKQ==",
"dev": true,
"license": "MIT",
"dependencies": {
"prelude-ls": "^1.2.1",
"type-check": "~0.4.0"
},
"engines": {
"node": ">= 0.8.0"
}
},
"node_modules/locate-path": {
"version": "6.0.0",
"resolved": "https://registry.npmjs.org/locate-path/-/locate-path-6.0.0.tgz",
"integrity": "sha512-iPZK6eYjbxRu3uB4/WZ3EsEIMJFMqAoopl3R+zuq0UjcAm/MO6KCweDgPfP3elTztoKP3KtnVHxTn2NHBSDVUw==",
"dev": true,
"license": "MIT",
"dependencies": {
"p-locate": "^5.0.0"
},
"engines": {
"node": ">=10"
},
"funding": {

2640:   "url": "https://github.com/sponsors/sindresorhus"
2641:   }
2642:   },
2643:   "node_modules/lodash.merge": {
2644:   "version": "4.6.2",
2645:   "resolved": "https://registry.npmjs.org/lodash.merge/-/lodash.merge-4.6.2.tgz",
2646:   "integrity": "sha512-0KpjqXRVvrYyCsX1swR/XTK0va6VQkQM6MNo7PqW77ByjAhoARA8EfrP1N4+KlKj8YS0ZUCtRT/YUuhyYDujIQ==",
2647:   "dev": true,
2648:   "license": "MIT"
2649:   },
2650:   "node_modules/lru-cache": {
2651:   "version": "5.1.1",
2652:   "resolved": "https://registry.npmjs.org/lru-cache/-/lru-cache-5.1.1.tgz",
2653:   "integrity": "sha512-KpNARQA3Iwv+jTA0utUVVbrh+Jlrr1Fv0e56GGzAFOXN7dk/FviaDW8LHmK52DlcH4WP2n6gI8vN1aesBFgo9w==",
2654:   "dev": true,
2655:   "license": "ISC",
2656:   "dependencies": {
2657:   "yallist": "^3.0.2"
2658:   }
2659:   },
2660:   "node_modules/minimatch": {
2661:   "version": "3.1.5",
2662:   "resolved": "https://registry.npmjs.org/minimatch/-/minimatch-3.1.5.tgz",
2663:   "integrity": "sha512-VgjWUsnnT6n+NUk6eZq77zeFdpW2LWDzP6zFGrCbHXiYNul5Dzqk2HHQ5uFH2DNW5Xbp8+jVzaeNt94ssEEl4w==",
2664:   "dev": true,
2665:   "license": "ISC",
2666:   "dependencies": {
2667:   "brace-expansion": "^1.1.7"
2668:   },
2669:   "engines": {
2670:   "node": "*"
2671:   }
2672:   },
2673:   "node_modules/ms": {
2674:   "version": "2.1.3",
2675:   "resolved": "https://registry.npmjs.org/ms/-/ms-2.1.3.tgz",
2676:   "integrity": "sha512-6FlzubTLZG3J2a/NVCAleEhjzq5oxgHyaCU9yYXvcLsvoVaHJq/s5xXI6/XXP6tz7R9xAOtHnSO/tXtF3WRTlA==",
2677:   "dev": true,
2678:   "license": "MIT"
2679:   },
2680:   "node_modules/nanoid": {
2681:   "version": "3.3.11",
2682:   "resolved": "https://registry.npmjs.org/nanoid/-/nanoid-3.3.11.tgz",
2683:   "integrity": "sha512-N8SpfPUnUp1bK+PMYW8qSWdl9U+wwNWI4QKxOYDy9JAro3WMX7p2OeVRF9v+347pnakNevPmiHhNmZ2HbFA76w==",
2684:   "dev": true,
2685:   "funding": [
2686:   {
2687:   "type": "github",
2688:   "url": "https://github.com/sponsors/ai"

```
2689:         }
2690:       ],
2691:       "license": "MIT",
2692:       "bin": {
2693:         "nanoid": "bin/nanoid.cjs"
2694:       },
2695:       "engines": {
2696:         "node": "^10 || ^12 || ^13.7 || ^14 || >=15.0.1"
2697:       }
2698:     },
2699:     "node_modules/natural-compare": {
2700:       "version": "1.4.0",
2701:       "resolved": "https://registry.npmjs.org/natural-compare/-/natural-compare-1.4.0.tgz",
2702:       "integrity": "sha512-OWND8ei3VtNC9h7V60qff3SVobHr996CTwgxubgyQYEpg290h9J0buyECNNJexkFm5s
Oajh5G116RYA1c8ZMSw==",
2703:       "dev": true,
2704:       "license": "MIT"
2705:     },
2706:     "node_modules/node-releases": {
2707:       "version": "2.0.27",
2708:       "resolved": "https://registry.npmjs.org/node-releases/-/node-releases-2.0.27.tgz",
2709:       "integrity": "sha512-nmh3lCkYZ3grZvqcCH+fjmQ7X+H0OeZgP40OierEaAptX4XofMh5kwNbWh7lBduUzCc
V/8kZ+NDLCwm2iorIlA==",
2710:       "dev": true,
2711:       "license": "MIT"
2712:     },
2713:     "node_modules/optionator": {
2714:       "version": "0.9.4",
2715:       "resolved": "https://registry.npmjs.org/optionator/-/optionator-0.9.4.tgz",
2716:       "integrity": "sha512-6IpQ7mKUxRcZNLIObR0hz7lxsapSSIYNZJwXPGeF0mTVqGKFIXj1DQcMoT22S3ROcLy
Y/rz0PWaWZ9ayWmad9g==",
2717:       "dev": true,
2718:       "license": "MIT",
2719:       "dependencies": {
2720:         "deep-is": "^0.1.3",
2721:         "fast-levenshtein": "^2.0.6",
2722:         "levn": "^0.4.1",
2723:         "prelude-ls": "^1.2.1",
2724:         "type-check": "^0.4.0",
2725:         "word-wrap": "^1.2.5"
2726:       },
2727:       "engines": {
2728:         "node": ">= 0.8.0"
2729:       }
2730:     },
2731:     "node_modules/p-limit": {
2732:       "version": "3.1.0",
2733:       "resolved": "https://registry.npmjs.org/p-limit/-/p-limit-3.1.0.tgz",
2734:       "integrity": "sha512-TYOanM3wGwNGsZN2cVTYPArw454xnXj5qmWF1bEoAc4+cU/ol7GVh7odevjp1FNHduH
c3KZMcFduxU5Xc6uJRQ==",
2735:       "dev": true,
2736:       "license": "MIT",
2737:       "dependencies": {
2738:         "yocto-queue": "^0.1.0"
```

2739:   },
2740:   "engines": {
2741:   "node": ">=10"
2742:   },
2743:   "funding": {
2744:   "url": "https://github.com/sponsors/sindresorhus"
2745:   }
2746:   },
2747:   "node_modules/p-locate": {
2748:   "version": "5.0.0",
2749:   "resolved": "https://registry.npmjs.org/p-locate/-/p-locate-5.0.0.tgz",
2750:   "integrity": "sha512-LaNjtRWUBY++zB5nE/NwcaoMylSPk+S+ZHNB1TzdbMJMny6dynpAGt7X/tl/QYq3TIe
E6nxHppbo2LGymrG5Pw==",
2751:   "dev": true,
2752:   "license": "MIT",
2753:   "dependencies": {
2754:   "p-limit": "^3.0.2"
2755:   },
2756:   "engines": {
2757:   "node": ">=10"
2758:   },
2759:   "funding": {
2760:   "url": "https://github.com/sponsors/sindresorhus"
2761:   }
2762:   },
2763:   "node_modules/parent-module": {
2764:   "version": "1.0.1",
2765:   "resolved": "https://registry.npmjs.org/parent-module/-/parent-module-1.0.1.tgz",
2766:   "integrity": "sha512-GQ2EWRpQV8/o+Aw8YqtfZZPfNRWZYkbidE9k5rpl/hC3vtHHBfGm2Ifi6qWV+coDGkr
UKZAxE3Lot5kcsRlh+g==",
2767:   "dev": true,
2768:   "license": "MIT",
2769:   "dependencies": {
2770:   "callsites": "^3.0.0"
2771:   },
2772:   "engines": {
2773:   "node": ">=6"
2774:   }
2775:   },
2776:   "node_modules/path-exists": {
2777:   "version": "4.0.0",
2778:   "resolved": "https://registry.npmjs.org/path-exists/-/path-exists-4.0.0.tgz",
2779:   "integrity": "sha512-ak9Qy5Q7jYb2Wwcey5Fpvg2KoAc/ZIhLSLOSBmRmygPsGwkVVt0fZa0qrtMz+m6tJTA
HfZQ8FnmB4MG4LWy7/w==",
2780:   "dev": true,
2781:   "license": "MIT",
2782:   "engines": {
2783:   "node": ">=8"
2784:   }
2785:   },
2786:   "node_modules/path-key": {
2787:   "version": "3.1.1",
2788:   "resolved": "https://registry.npmjs.org/path-key/-/path-key-3.1.1.tgz",

"integrity": "sha512-ojmeN0qd+y0jszEtoY48r0Peq5dwMEkIlCOu6Q5f41lfkswXuKtYrhgoTpLnyIcHm24Uhqx+5Tqm2InSwLhE6Q==",
"dev": true,
"license": "MIT",
"engines": {
"node": ">=8"
}
},
"node_modules/picocolors": {
"version": "1.1.1",
"resolved": "https://registry.npmjs.org/picocolors/-/picocolors-1.1.1.tgz",
"integrity": "sha512-xceH2snhtb5M9liqDsmEw56le376mTZkEX/jEb/RxNFyegNul7eNslCXP9FDj/Lcu0X8KEyMceP2ntpaHrDEVA==",
"dev": true,
"license": "ISC"
},
"node_modules/picomatch": {
"version": "4.0.3",
"resolved": "https://registry.npmjs.org/picomatch/-/picomatch-4.0.3.tgz",
"integrity": "sha512-5gTmgEY/sqK6gFXLIsQNH19lWb4ebPDLA4SdLP7dsWkIXHWlG66oPuVvXSGFPppYZz8ZDZq0dYYrbHfBCVUb1Q==",
"dev": true,
"license": "MIT",
"engines": {
"node": ">=12"
},
"funding": {
"url": "https://github.com/sponsors/jonschlinkert"
}
},
"node_modules/postcss": {
"version": "8.5.6",
"resolved": "https://registry.npmjs.org/postcss/-/postcss-8.5.6.tgz",
"integrity": "sha512-3Ybi1tAuwAP9s0r1UQ2J4n5Y0G05bJkpUIO0/bI9MhwmD70S5aTWbXGBwxHrelT+XM1k6dM0pk+SwNkpTRN7Pg==",
"dev": true,
"funding": [
{
"type": "opencollective",
"url": "https://opencollective.com/postcss/"
},
{
"type": "tidelift",
"url": "https://tidelift.com/funding/github/npm/postcss"
},
{
"type": "github",
"url": "https://github.com/sponsors/ai"
}
],
"license": "MIT",
"dependencies": {
"nanoid": "^3.3.11",
"picocolors": "^1.1.1",

2839:   "source-map-js": "^1.2.1"
2840:   },
2841:   "engines": {
2842:   "node": "^10 || ^12 || >=14"
2843:   }
2844:   },
2845:   "node_modules/prelude-ls": {
2846:   "version": "1.2.1",
2847:   "resolved": "https://registry.npmjs.org/prelude-ls/-/prelude-ls-1.2.1.tgz",
2848:   "integrity": "sha512-vkcDPrRZo1QZLbn5RLGPpg/WmIQ65qoWWhcGKf/b5eplkkarX0m9z8ppCat4mlOqUsWpyNuYgO3VRyrYHSzX5g==",
2849:   "dev": true,
2850:   "license": "MIT",
2851:   "engines": {
2852:   "node": ">= 0.8.0"
2853:   }
2854:   },
2855:   "node_modules/punycode": {
2856:   "version": "2.3.1",
2857:   "resolved": "https://registry.npmjs.org/punycode/-/punycode-2.3.1.tgz",
2858:   "integrity": "sha512-vYt7UD1U9Wg6138shLtLOvdAu+8DsC/ilFtEVHcH+wydcSpNE20AfSOduf6MkRFahL5FY7X1oU7nKVZFtfq8Fg==",
2859:   "dev": true,
2860:   "license": "MIT",
2861:   "engines": {
2862:   "node": ">=6"
2863:   }
2864:   },
2865:   "node_modules/react": {
2866:   "version": "19.2.4",
2867:   "resolved": "https://registry.npmjs.org/react/-/react-19.2.4.tgz",
2868:   "integrity": "sha512-9nfp2hYpCwOjAN+8TZFGhtWEwgvWHXqESH8qT89AT/lWklpLON22Lc8pEtnpsZz7VmawabSU0gCjnj8aC0euHQ==",
2869:   "license": "MIT",
2870:   "engines": {
2871:   "node": ">=0.10.0"
2872:   }
2873:   },
2874:   "node_modules/react-dom": {
2875:   "version": "19.2.4",
2876:   "resolved": "https://registry.npmjs.org/react-dom/-/react-dom-19.2.4.tgz",
2877:   "integrity": "sha512-AXJdLo8kgMbimY95O2aKQqsz2iWi9jMgKJhRBAxECE4IFxfcazB2LmzloIoibJI3C12IlY20+KFaLv+71bUJeQ==",
2878:   "license": "MIT",
2879:   "dependencies": {
2880:   "scheduler": "^0.27.0"
2881:   },
2882:   "peerDependencies": {
2883:   "react": "^19.2.4"
2884:   }
2885:   },
2886:   "node_modules/react-refresh": {
2887:   "version": "0.18.0",
2888:   "resolved": "https://registry.npmjs.org/react-refresh/-/react-refresh-0.18.0.tgz",

2889:   "integrity": "sha512-QgT5//D3jfjJb6Gsjxv0Slpj23ip+HtOpnNgnb2S5zU3CB26G/IDPGoy4RJB42wzFE4
6DRsstbW6tKHoKbhAxw==",
2890:   "dev": true,
2891:   "license": "MIT",
2892:   "engines": {
2893:   "node": ">=0.10.0"
2894:   }
2895:   },
2896:   "node_modules/react-router": {
2897:   "version": "7.13.1",
2898:   "resolved": "https://registry.npmjs.org/react-router/-/react-router-7.13.1.tgz",
2899:   "integrity": "sha512-td+xP4X2/6BJvZoX6xw++A2DdEi++YypA69bJUV5oVvqf6/9/9nNlD70YO1e9d3Myam
JEBQFEzk6mbfDYbqrSA==",
2900:   "license": "MIT",
2901:   "dependencies": {
2902:   "cookie": "^1.0.1",
2903:   "set-cookie-parser": "^2.6.0"
2904:   },
2905:   "engines": {
2906:   "node": ">=20.0.0"
2907:   },
2908:   "peerDependencies": {
2909:   "react": ">=18",
2910:   "react-dom": ">=18"
2911:   },
2912:   "peerDependenciesMeta": {
2913:   "react-dom": {
2914:   "optional": true
2915:   }
2916:   }
2917:   },
2918:   "node_modules/react-router-dom": {
2919:   "version": "7.13.1",
2920:   "resolved": "https://registry.npmjs.org/react-router-dom/-/react-router-dom-7.13.1.tgz",
2921:   "integrity": "sha512-UJnV3Rxc5TgUPJt2KJpo1Jpy0OKQr0AjgbZzBFjaPJcFOb2Y8jA5H3LT8HUJAiRLlWr
EXWHbF1Z4SCZaQjWDHw==",
2922:   "license": "MIT",
2923:   "dependencies": {
2924:   "react-router": "7.13.1"
2925:   },
2926:   "engines": {
2927:   "node": ">=20.0.0"
2928:   },
2929:   "peerDependencies": {
2930:   "react": ">=18",
2931:   "react-dom": ">=18"
2932:   }
2933:   },
2934:   "node_modules/resolve-from": {
2935:   "version": "4.0.0",
2936:   "resolved": "https://registry.npmjs.org/resolve-from/-/resolve-from-4.0.0.tgz",
2937:   "integrity": "sha512-pb/MYmXstAkysRFx8piNI1tGFNQIFA3vkE3Gq4EuA1dF6gHp/+vgZqsCGJapvy8N3Q+
4o7FwvquPJcnZ7RYy4g==",
2938:   "dev": true,

```
2939:       "license": "MIT",
2940:       "engines": {
2941:         "node": ">=4"
2942:       }
2943:     },
2944:     "node_modules/rollup": {
2945:       "version": "4.59.0",
2946:       "resolved": "https://registry.npmjs.org/rollup/-/rollup-4.59.0.tgz",
2947:       "integrity": "sha512-2oMpl67a3zCH9H79LeMcbDhXW/UmWG/y2zuqnF2jQq5uq9TbM9TVyXvA4+t+ne2IIkB
drLpAaRQAvo7YI/Yyeg==",
2948:       "dev": true,
2949:       "license": "MIT",
2950:       "dependencies": {
2951:         "@types/estree": "1.0.8"
2952:       },
2953:       "bin": {
2954:         "rollup": "dist/bin/rollup"
2955:       },
2956:       "engines": {
2957:         "node": ">=18.0.0",
2958:         "npm": ">=8.0.0"
2959:       },
2960:       "optionalDependencies": {
2961:         "@rollup/rollup-android-arm-eabi": "4.59.0",
2962:         "@rollup/rollup-android-arm64": "4.59.0",
2963:         "@rollup/rollup-darwin-arm64": "4.59.0",
2964:         "@rollup/rollup-darwin-x64": "4.59.0",
2965:         "@rollup/rollup-freebsd-arm64": "4.59.0",
2966:         "@rollup/rollup-freebsd-x64": "4.59.0",
2967:         "@rollup/rollup-linux-arm-gnueabihf": "4.59.0",
2968:         "@rollup/rollup-linux-arm-musleabihf": "4.59.0",
2969:         "@rollup/rollup-linux-arm64-gnu": "4.59.0",
2970:         "@rollup/rollup-linux-arm64-musl": "4.59.0",
2971:         "@rollup/rollup-linux-loong64-gnu": "4.59.0",
2972:         "@rollup/rollup-linux-loong64-musl": "4.59.0",
2973:         "@rollup/rollup-linux-ppc64-gnu": "4.59.0",
2974:         "@rollup/rollup-linux-ppc64-musl": "4.59.0",
2975:         "@rollup/rollup-linux-riscv64-gnu": "4.59.0",
2976:         "@rollup/rollup-linux-riscv64-musl": "4.59.0",
2977:         "@rollup/rollup-linux-s390x-gnu": "4.59.0",
2978:         "@rollup/rollup-linux-x64-gnu": "4.59.0",
2979:         "@rollup/rollup-linux-x64-musl": "4.59.0",
2980:         "@rollup/rollup-openbsd-x64": "4.59.0",
2981:         "@rollup/rollup-openharmony-arm64": "4.59.0",
2982:         "@rollup/rollup-win32-arm64-msvc": "4.59.0",
2983:         "@rollup/rollup-win32-ia32-msvc": "4.59.0",
2984:         "@rollup/rollup-win32-x64-gnu": "4.59.0",
2985:         "@rollup/rollup-win32-x64-msvc": "4.59.0",
2986:         "fsevents": "~2.3.2"
2987:       }
2988:     },
2989:     "node_modules/scheduler": {
2990:       "version": "0.27.0",
2991:       "resolved": "https://registry.npmjs.org/scheduler/-/scheduler-0.27.0.tgz",
```

2992:   "integrity": "sha512-eNv+WrVbKu1f3vbYJT/xtiF5syA5HPIMtf9IgY/nKg0sWqzAUEvqY/xm7OcZc/qafLx
/iO9FgOmeSAp4v5ti/Q==",
2993:   "license": "MIT"
2994:   },
2995:   "node_modules/semver": {
2996:   "version": "6.3.1",
2997:   "resolved": "https://registry.npmjs.org/semver/-/semver-6.3.1.tgz",
2998:   "integrity": "sha512-BR7VvDCVHO+q2xBEWskxS6DJE1qRnb7DxzUrogb71CWoSficBxYsiAGd+Kl0mmq/Mpr
G9yArRkyrQxTO6XjMzA==",
2999:   "dev": true,
3000:   "license": "ISC",
3001:   "bin": {
3002:   "semver": "bin/semver.js"
3003:   }
3004:   },
3005:   "node_modules/set-cookie-parser": {
3006:   "version": "2.7.2",
3007:   "resolved": "https://registry.npmjs.org/set-cookie-parser/-/set-cookie-parser-2.7.2.tgz"
,
3008:   "integrity": "sha512-oeM1lpU/UvhTxw+g3cIfxXHyJRc/uidd3yK1P242gzHds0udQBYzs3y8j4gCCW+ZJ7a
d0yctld8RYO+bdurlvw==",
3009:   "license": "MIT"
3010:   },
3011:   "node_modules/shebang-command": {
3012:   "version": "2.0.0",
3013:   "resolved": "https://registry.npmjs.org/shebang-command/-/shebang-command-2.0.0.tgz",
3014:   "integrity": "sha512-kHxr2zZpYtdmrN1qDjrrX/Z1rR1kG8Dx+gkpK1G4eXmvXswmcE1hTWBWYUzlraYw1/y
Zp6YuDY77YtvbN0dmDA==",
3015:   "dev": true,
3016:   "license": "MIT",
3017:   "dependencies": {
3018:   "shebang-regex": "^3.0.0"
3019:   },
3020:   "engines": {
3021:   "node": ">=8"
3022:   }
3023:   },
3024:   "node_modules/shebang-regex": {
3025:   "version": "3.0.0",
3026:   "resolved": "https://registry.npmjs.org/shebang-regex/-/shebang-regex-3.0.0.tgz",
3027:   "integrity": "sha512-7++dFhtcx3353uBaq8DDR4NuxBetBzC7ZQOhmTQInHEd6bSrXdiEyzCvG07Z44UYdLS
hWUyXt5M/yhz8ekcb1A==",
3028:   "dev": true,
3029:   "license": "MIT",
3030:   "engines": {
3031:   "node": ">=8"
3032:   }
3033:   },
3034:   "node_modules/source-map-js": {
3035:   "version": "1.2.1",
3036:   "resolved": "https://registry.npmjs.org/source-map-js/-/source-map-js-1.2.1.tgz",
3037:   "integrity": "sha512-UXWMKhLOwVKb728IUtQPXxfYU+usdybtUrK/8uGE8CQMvrhOpwvzDBwj0QhSL7MQc7v
IsISBG8VQ8+IDQxpfQA==",
3038:   "dev": true,

3039:   "license": "BSD-3-Clause",
3040:   "engines": {
3041:   "node": ">=0.10.0"
3042:   }
3043:   },
3044:   "node_modules/strip-json-comments": {
3045:   "version": "3.1.1",
3046:   "resolved": "https://registry.npmjs.org/strip-json-comments/-/strip-json-comments-3.1.1.
tgz",
3047:   "integrity": "sha512-6fPc+R4ihwqP6N/aIv2f1gMH8lOVtWQHoqC4yK6oSDVVocumAsfCqjkXnqiYMhmMwS/
mEHLp7Vehlt3ql6lEig==",
3048:   "dev": true,
3049:   "license": "MIT",
3050:   "engines": {
3051:   "node": ">=8"
3052:   },
3053:   "funding": {
3054:   "url": "https://github.com/sponsors/sindresorhus"
3055:   }
3056:   },
3057:   "node_modules/supports-color": {
3058:   "version": "7.2.0",
3059:   "resolved": "https://registry.npmjs.org/supports-color/-/supports-color-7.2.0.tgz",
3060:   "integrity": "sha512-qpCAvRl9stuOHveKsn7HncJRvv501qIacKzQlO/+Lwxc9+0q2wLyv4Dfvt80/DPn2pq
OBsJdDiogXGR9+OvwRw==",
3061:   "dev": true,
3062:   "license": "MIT",
3063:   "dependencies": {
3064:   "has-flag": "^4.0.0"
3065:   },
3066:   "engines": {
3067:   "node": ">=8"
3068:   }
3069:   },
3070:   "node_modules/tinyglobby": {
3071:   "version": "0.2.15",
3072:   "resolved": "https://registry.npmjs.org/tinyglobby/-/tinyglobby-0.2.15.tgz",
3073:   "integrity": "sha512-j2Zq4NyQYG5XMST4cbs02Ak8iJUdxRM0XI5QyxXuZOzKOINmWurp3smXu3y5wDcJrpt
wpSjgXHzIQxR0omXljQ==",
3074:   "dev": true,
3075:   "license": "MIT",
3076:   "dependencies": {
3077:   "fdir": "^6.5.0",
3078:   "picomatch": "^4.0.3"
3079:   },
3080:   "engines": {
3081:   "node": ">=12.0.0"
3082:   },
3083:   "funding": {
3084:   "url": "https://github.com/sponsors/SuperchupuDev"
3085:   }
3086:   },
3087:   "node_modules/ts-api-utils": {
3088:   "version": "2.4.0",

3089:   "resolved": "https://registry.npmjs.org/ts-api-utils/-/ts-api-utils-2.4.0.tgz",
3090:   "integrity": "sha512-3TaVTaAv2gTiMB35i3FiGJaRfwb3Pyn/j3m/bfAvGe8FB7CF6u+LMYqYlDh7reQf7UNvoTvdfAqHGmPGOSsPmA==",
3091:   "dev": true,
3092:   "license": "MIT",
3093:   "engines": {
3094:   "node": ">=18.12"
3095:   },
3096:   "peerDependencies": {
3097:   "typescript": ">=4.8.4"
3098:   }
3099:   },
3100:   "node_modules/type-check": {
3101:   "version": "0.4.0",
3102:   "resolved": "https://registry.npmjs.org/type-check/-/type-check-0.4.0.tgz",
3103:   "integrity": "sha512-XleUoc9uwGXqjWwXaUTZAmzMcFZ5858QA2vvx1Ur5xIcixXIP+8LnFDgRplU30us6teqdlskFfu+ae4K79Ooew==",
3104:   "dev": true,
3105:   "license": "MIT",
3106:   "dependencies": {
3107:   "prelude-ls": "^1.2.1"
3108:   },
3109:   "engines": {
3110:   "node": ">= 0.8.0"
3111:   }
3112:   },
3113:   "node_modules/typescript": {
3114:   "version": "5.9.3",
3115:   "resolved": "https://registry.npmjs.org/typescript/-/typescript-5.9.3.tgz",
3116:   "integrity": "sha512-jl1vZzPDinLr9eUt3J/t7V6FgNEw9QjvBPdysz9KfQDD41fQrC2Y4vKQdiaUpFT4bXlb1RHhLpp8wtm6M5TgSw==",
3117:   "dev": true,
3118:   "license": "Apache-2.0",
3119:   "bin": {
3120:   "tsc": "bin/tsc",
3121:   "tsserver": "bin/tsserver"
3122:   },
3123:   "engines": {
3124:   "node": ">=14.17"
3125:   }
3126:   },
3127:   "node_modules/typescript-eslint": {
3128:   "version": "8.56.1",
3129:   "resolved": "https://registry.npmjs.org/typescript-eslint/-/typescript-eslint-8.56.1.tgz",
3130:   "integrity": "sha512-U4lM6pjmBX7J5wk4szltF7I1cGBHXZopnAXCMXb3+fZ3B/0Z3hq3wS/CCUB2NZBNAExK92mCU2tEohWuwVMsDQ==",
3131:   "dev": true,
3132:   "license": "MIT",
3133:   "dependencies": {
3134:   "@typescript-eslint/eslint-plugin": "8.56.1",
3135:   "@typescript-eslint/parser": "8.56.1",
3136:   "@typescript-eslint/typescript-estree": "8.56.1",
3137:   "@typescript-eslint/utils": "8.56.1"

3138:   },
3139:   "engines": {
3140:   "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
3141:   },
3142:   "funding": {
3143:   "type": "opencollective",
3144:   "url": "https://opencollective.com/typescript-eslint"
3145:   },
3146:   "peerDependencies": {
3147:   "eslint": "^8.57.0 || ^9.0.0 || ^10.0.0",
3148:   "typescript": ">=4.8.4 <6.0.0"
3149:   }
3150:   },
3151:   "node_modules/undici-types": {
3152:   "version": "7.16.0",
3153:   "resolved": "https://registry.npmjs.org/undici-types/-/undici-types-7.16.0.tgz",
3154:   "integrity": "sha512-Zz+aZWSj8LE6zoxD+xrjh4VfkIG8Ya6LvYkZqtUQGJPZjYl53ypCaUwWqo7eI0x66KB
GeRo+mlBEkMSeSZ38Nw==",
3155:   "dev": true,
3156:   "license": "MIT"
3157:   },
3158:   "node_modules/update-browserslist-db": {
3159:   "version": "1.2.3",
3160:   "resolved": "https://registry.npmjs.org/update-browserslist-db/-/update-browserslist-db-
1.2.3.tgz",
3161:   "integrity": "sha512-Js0m9cx+qOgDxo0eMiFGEueWztz+d4+M3rGlmKPT+T4IS/jP4ylw3Nwpu6cpTTP8R1M
AC1kF4VbdLt3ARf209w==",
3162:   "dev": true,
3163:   "funding": [
3164:   {
3165:   "type": "opencollective",
3166:   "url": "https://opencollective.com/browserslist"
3167:   },
3168:   {
3169:   "type": "tidelift",
3170:   "url": "https://tidelift.com/funding/github/npm/browserslist"
3171:   },
3172:   {
3173:   "type": "github",
3174:   "url": "https://github.com/sponsors/ai"
3175:   }
3176:   ],
3177:   "license": "MIT",
3178:   "dependencies": {
3179:   "escalade": "^3.2.0",
3180:   "picocolors": "^1.1.1"
3181:   },
3182:   "bin": {
3183:   "update-browserslist-db": "cli.js"
3184:   },
3185:   "peerDependencies": {
3186:   "browserslist": ">= 4.21.0"
3187:   }
3188:   },

3189:   "node_modules/uri-js": {
3190:   "version": "4.4.1",
3191:   "resolved": "https://registry.npmjs.org/uri-js/-/uri-js-4.4.1.tgz",
3192:   "integrity": "sha512-7rKUyy33Q1yc98pQ1DAmLtwX109F7TIfWlW1Ydo8Wl1ii1SeHieeh0HHfPeL2fMXK6z
0s8ecKs9frCuLJvndBg==",
3193:   "dev": true,
3194:   "license": "BSD-2-Clause",
3195:   "dependencies": {
3196:   "punycode": "^2.1.0"
3197:   }
3198:   },
3199:   "node_modules/vite": {
3200:   "version": "7.3.1",
3201:   "resolved": "https://registry.npmjs.org/vite/-/vite-7.3.1.tgz",
3202:   "integrity": "sha512-w+N7Hifpc3gRjZ63vYBXA56dvvRlNWRczTdmCBBa+CotUzAPf5b7YMdMR/8CQoeYE5L
X3W4wj6RYTgonm1b9DA==",
3203:   "dev": true,
3204:   "license": "MIT",
3205:   "dependencies": {
3206:   "esbuild": "^0.27.0",
3207:   "fdir": "^6.5.0",
3208:   "picomatch": "^4.0.3",
3209:   "postcss": "^8.5.6",
3210:   "rollup": "^4.43.0",
3211:   "tinyglobby": "^0.2.15"
3212:   },
3213:   "bin": {
3214:   "vite": "bin/vite.js"
3215:   },
3216:   "engines": {
3217:   "node": "^20.19.0 || >=22.12.0"
3218:   },
3219:   "funding": {
3220:   "url": "https://github.com/vitejs/vite?sponsor=1"
3221:   },
3222:   "optionalDependencies": {
3223:   "fsevents": "~2.3.3"
3224:   },
3225:   "peerDependencies": {
3226:   "@types/node": "^20.19.0 || >=22.12.0",
3227:   "jiti": ">=1.21.0",
3228:   "less": "^4.0.0",
3229:   "lightningcss": "^1.21.0",
3230:   "sass": "^1.70.0",
3231:   "sass-embedded": "^1.70.0",
3232:   "stylus": ">=0.54.8",
3233:   "sugarss": "^5.0.0",
3234:   "terser": "^5.16.0",
3235:   "tsx": "^4.8.1",
3236:   "yaml": "^2.4.2"
3237:   },
3238:   "peerDependenciesMeta": {
3239:   "@types/node": {
3240:   "optional": true

3241:   },
3242:   "jiti": {
3243:   "optional": true
3244:   },
3245:   "less": {
3246:   "optional": true
3247:   },
3248:   "lightningcss": {
3249:   "optional": true
3250:   },
3251:   "sass": {
3252:   "optional": true
3253:   },
3254:   "sass-embedded": {
3255:   "optional": true
3256:   },
3257:   "stylus": {
3258:   "optional": true
3259:   },
3260:   "sugarss": {
3261:   "optional": true
3262:   },
3263:   "terser": {
3264:   "optional": true
3265:   },
3266:   "tsx": {
3267:   "optional": true
3268:   },
3269:   "yaml": {
3270:   "optional": true
3271:   }
3272:   }
3273:   },
3274:   "node_modules/which": {
3275:   "version": "2.0.2",
3276:   "resolved": "https://registry.npmjs.org/which/-/which-2.0.2.tgz",
3277:   "integrity": "sha512-BLI3Tl1TW3Pvl70l3yq3Y64i+awpwXqsGBYWkkqMtnbXgrMD+yj7rhW0kuEDxzJaYXG
jEW5ogapKNMEKNMjibA==",
3278:   "dev": true,
3279:   "license": "ISC",
3280:   "dependencies": {
3281:   "isexe": "^2.0.0"
3282:   },
3283:   "bin": {
3284:   "node-which": "bin/node-which"
3285:   },
3286:   "engines": {
3287:   "node": ">= 8"
3288:   }
3289:   },
3290:   "node_modules/word-wrap": {
3291:   "version": "1.2.5",
3292:   "resolved": "https://registry.npmjs.org/word-wrap/-/word-wrap-1.2.5.tgz",

3293:   "integrity": "sha512-BN22B5eaMMI9UMtjrGd5g5eCYPpCPDUy0FJXbYsaT5zYxjFOckS53SQDE3pWkVoWpHX
Vb3BrYcEN4Twa55B5cA==",
3294:   "dev": true,
3295:   "license": "MIT",
3296:   "engines": {
3297:   "node": ">=0.10.0"
3298:   }
3299:   },
3300:   "node_modules/yallist": {
3301:   "version": "3.1.1",
3302:   "resolved": "https://registry.npmjs.org/yallist/-/yallist-3.1.1.tgz",
3303:   "integrity": "sha512-a4UGQaWPH59mOXUYnAG2ewncQS4i4F43Tv3JoAM+s2VDAmS9NsK8GpDMLrCHPksFT7h
3K6TOoUNn2pb7RoXx4g==",
3304:   "dev": true,
3305:   "license": "ISC"
3306:   },
3307:   "node_modules/yocto-queue": {
3308:   "version": "0.1.0",
3309:   "resolved": "https://registry.npmjs.org/yocto-queue/-/yocto-queue-0.1.0.tgz",
3310:   "integrity": "sha512-rVksvsnNCdJ/ohGc6xgPwyN8eheCxsiLM8mxuE/t/mOVqJewPuO1miLpTHQiRgTKCLe
xL4MeAFVagts7HmNZ2Q==",
3311:   "dev": true,
3312:   "license": "MIT",
3313:   "engines": {
3314:   "node": ">=10"
3315:   },
3316:   "funding": {
3317:   "url": "https://github.com/sponsors/sindresorhus"
3318:   }
3319:   },
3320:   "node_modules/zod": {
3321:   "version": "4.3.6",
3322:   "resolved": "https://registry.npmjs.org/zod/-/zod-4.3.6.tgz",
3323:   "integrity": "sha512-rftlrkhHZOcjDwkGlnUtZZkvaPHCsDATp4pGpuOOMDaTdDDXF91wuVDJoWoPsKX/3YP
Q5fHuF3STjcYyKr+Qhg==",
3324:   "dev": true,
3325:   "license": "MIT",
3326:   "funding": {
3327:   "url": "https://github.com/sponsors/colinhacks"
3328:   }
3329:   },
3330:   "node_modules/zod-validation-error": {
3331:   "version": "4.0.2",
3332:   "resolved": "https://registry.npmjs.org/zod-validation-error/-/zod-validation-error-4.0.
2.tgz",
3333:   "integrity": "sha512-Q6/nZLe6jxuU80qb/4uJ4t5v2VEZ44lzQjPDhYJNztRQ4wyWc6VF3D3Kb/fAuPetZQn
hS3hnajCf9CsWesghLQ==",
3334:   "dev": true,
3335:   "license": "MIT",
3336:   "engines": {
3337:   "node": ">=18.0.0"
3338:   },
3339:   "peerDependencies": {
3340:   "zod": "^3.25.0 || ^4.0.0"

```
3341:       }
3342:     }
3343:   }
3344: }
```

==========
ARQUIVO: frontend/package.json
==========

```
0001: {
0002:   "name": "frontend",
0003:   "private": true,
0004:   "version": "1.0.0-beta.1",
0005:   "type": "module",
0006:   "scripts": {
0007:     "dev": "vite",
0008:     "build": "tsc -b && vite build",
0009:     "lint": "eslint .",
0010:     "preview": "vite preview"
0011:   },
0012:   "dependencies": {
0013:     "react": "^19.2.0",
0014:     "react-dom": "^19.2.0",
0015:     "react-router-dom": "^7.13.1"
0016:   },
0017:   "devDependencies": {
0018:     "@eslint/js": "^9.39.1",
0019:     "@types/node": "^24.10.1",
0020:     "@types/react": "^19.2.7",
0021:     "@types/react-dom": "^19.2.3",
0022:     "@vitejs/plugin-react": "^5.1.1",
0023:     "eslint": "^9.39.1",
0024:     "eslint-plugin-react-hooks": "^7.0.1",
0025:     "eslint-plugin-react-refresh": "^0.4.24",
0026:     "globals": "^16.5.0",
0027:     "typescript": "~5.9.3",
0028:     "typescript-eslint": "^8.48.0",
0029:     "vite": "^7.3.1"
0030:   }
0031: }
```

==========
ARQUIVO: frontend/README.md
==========

```
0001: # Frontend - Mini Protheus
0002:
0003: Aplicacao React + Vite + TypeScript.
0004:
0005: ## Requisitos
0006:
0007: - Node.js 20+
0008:
0009: ## Configuracao
0010:
0011: Arquivo `frontend/.env`:
```

0012:
0013: ```env
0014: VITE_API_BASE_URL=http://127.0.0.1:8000
0015: ```
0016:
0017: ## Comandos
0018:
0019: Instalar dependencias:
0020:
0021: ```powershell
0022: npm install
0023: ```
0024:
0025: Desenvolvimento:
0026:
0027: ```powershell
0028: npm run dev
0029: ```
0030:
0031: Lint:
0032:
0033: ```powershell
0034: npm run lint
0035: ```
0036:
0037: Build de producao:
0038:
0039: ```powershell
0040: npm run build
0041: ```
0042:
0043: Preview local do build:
0044:
0045: ```powershell
0046: npm run preview
0047: ```

==========
ARQUIVO: frontend/src/api/alertas.ts
==========
0001: import { http } from "./http";
0002:
0003: export type BackendAlertItem = {
0004:   code: string;
0005:   severity: "critical" | "warning" | "info" | string;
0006:   title: string;
0007:   text: string;
0008:   route?: string | null;
0009:   count?: number | null;
0010:   action_label?: string | null;
0011:   business_context?: string | null;
0012: };
0013:
0014: export type BackendAlertSummary = {

```
0015:   critical: number;
0016:   warning: number;
0017:   info: number;
0018:   total: number;
0019: };
0020:
0021: export type BackendAlertsResponse = {
0022:   filial: string;
0023:   usuario: string;
0024:   generated_at: string;
0025:   summary: BackendAlertSummary;
0026:   items: BackendAlertItem[];
0027: };
0028:
0029: export function listarAlertas(params: { filial: string }) {
0030:   const query = new URLSearchParams({ filial: params.filial }).toString();
0031:   return http<BackendAlertsResponse>(`/alertas?${query}`);
0032: }
```

==========
ARQUIVO: frontend/src/api/auditoria.ts
==========

```
0001: import { http } from "./http";
0002:
0003: export type LogItem = {
0004:   ID: number;
0005:   L_USUARIO?: string | null;
0006:   L_ACAO?: string | null;
0007:   L_DATA?: string | null;
0008:   [k: string]: unknown;
0009: };
0010:
0011: export async function listarAuditoria() {
0012:   return http<LogItem[]>("/auditoria");
0013: }
```

==========
ARQUIVO: frontend/src/api/auth.ts
==========

```
0001: export type LoginResponse = {
0002:   access_token: string;
0003:   token_type: string;
0004: };
0005:
0006: const API_BASE_URL = import.meta.env.VITE_API_BASE_URL as string;
0007:
0008: function isLoginResponse(data: unknown): data is LoginResponse {
0009:   if (!data || typeof data !== "object") return false;
0010:   const value = data as Record<string, unknown>;
0011:   return typeof value.access_token === "string" && typeof value.token_type === "string";
0012: }
0013:
0014: export async function login(username: string, password: string): Promise<LoginResponse> {
0015:   const form = new URLSearchParams();
```

```
0016:   form.set("username", username);
0017:   form.set("password", password);
0018:
0019:   let res: Response;
0020:   try {
0021:     res = await fetch(`${API_BASE_URL}/login`, {
0022:       method: "POST",
0023:       headers: { "Content-Type": "application/x-www-form-urlencoded" },
0024:       body: form.toString(),
0025:     });
0026:   } catch {
0027:     throw new Error("Nao foi possivel conectar ao servidor.");
0028:   }
0029:
0030:   const text = await res.text();
0031:   let data: unknown = null;
0032:
0033:   try {
0034:     data = text ? JSON.parse(text) : null;
0035:   } catch {
0036:     data = null;
0037:   }
0038:
0039:   if (!res.ok) {
0040:     const detail =
0041:       data && typeof data === "object" && typeof (data as Record<string, unknown>).detail ===
"string"
0042:         ? ((data as Record<string, unknown>).detail as string)
0043:         : "Usuario ou senha invalidos.";
0044:     throw new Error(detail);
0045:   }
0046:
0047:   if (!isLoginResponse(data)) {
0048:     throw new Error("Resposta de login invalida.");
0049:   }
0050:
0051:   return data;
0052: }
```

==========
ARQUIVO: frontend/src/api/clients.ts
==========

```
0001: import { http } from "./http";
0002: import { getFilial } from "../state/filial";
0003:
0004: export type Cliente = {
0005:   id?: number;
0006:   cod: string;
0007:   nome: string;
0008:   doc?: string | null;
0009:   email?: string | null;
0010:   fone?: string | null;
0011:   filial: string;
0012:   ativo_sn: "S" | "N";
```

```
0013:   criado_em?: string;
0014: };
0015:
0016: export type ClienteCreateInput = {
0017:   cod: string;
0018:   nome: string;
0019:   doc?: string | null;
0020:   email?: string | null;
0021:   fone?: string | null;
0022:   filial?: string;
0023: };
0024:
0025: export type ClienteUpdateInput = {
0026:   nome?: string | null;
0027:   doc?: string | null;
0028:   email?: string | null;
0029:   fone?: string | null;
0030:   filial?: string;
0031: };
0032:
0033: type RawRecord = Record<string, unknown>;
0034:
0035: function asRecord(value: unknown): RawRecord {
0036:   return value && typeof value === "object" ? (value as RawRecord) : {};
0037: }
0038:
0039: function asString(value: unknown): string | undefined {
0040:   return typeof value === "string" ? value : undefined;
0041: }
0042:
0043: function asNumber(value: unknown): number | undefined {
0044:   return typeof value === "number" ? value : undefined;
0045: }
0046:
0047: function normalizeCliente(value: unknown): Cliente {
0048:   const x = asRecord(value);
0049:   const ativoRaw = x.A1_ATIVO ?? x.ativo ?? false;
0050:
0051:   return {
0052:     id: asNumber(x.ID),
0053:     filial: asString(x.A1_FILIAL) ?? asString(x.filial) ?? getFilial(),
0054:     cod: asString(x.A1_COD) ?? asString(x.cod) ?? "",
0055:     nome: asString(x.A1_NOME) ?? asString(x.nome) ?? "",
0056:     doc: asString(x.A1_DOC) ?? asString(x.doc) ?? null,
0057:     email: asString(x.A1_EMAIL) ?? asString(x.email) ?? null,
0058:     fone: asString(x.A1_FONE) ?? asString(x.A1_TEL) ?? asString(x.fone) ?? null,
0059:     ativo_sn: ativoRaw ? "S" : "N",
0060:     criado_em: asString(x.A1_CRIADO_EM) ?? asString(x.criado_em),
0061:   };
0062: }
0063:
0064: export async function listClientes(params: { ativos: boolean; filial?: string }) {
0065:   const filial = params.filial ?? getFilial();
0066:   const qs = new URLSearchParams({
```

```
0067:     ativos: params.ativos ? "true" : "false",
0068:     filial,
0069:   });
0070:
0071:   const data = await http<unknown[]>(`/clientes?${qs.toString()}`);
0072:   return data.map(normalizeCliente);
0073: }
0074:
0075: export async function buscarClientes(params: { q: string; filial?: string }) {
0076:   const filial = params.filial ?? getFilial();
0077:   const qs = new URLSearchParams({ q: params.q, filial });
0078:
0079:   const data = await http<unknown[]>(`/clientes/buscar?${qs.toString()}`);
0080:   return data.map(normalizeCliente);
0081: }
0082:
0083: export async function createCliente(payload: ClienteCreateInput) {
0084:   const body = { ...payload, filial: payload.filial ?? getFilial() };
0085:   return http(`/clientes`, { method: "POST", body });
0086: }
0087:
0088: export async function updateCliente(cod: string, payload: ClienteUpdateInput) {
0089:   const body = { ...payload, filial: payload.filial ?? getFilial() };
0090:   return http(`/clientes/${encodeURIComponent(cod)}`, { method: "PUT", body });
0091: }
0092:
0093: export async function setClienteAtivo(cod: string, payload: { ativo: boolean; filial?: string
}) {
0094:   const body = { ativo: payload.ativo, filial: payload.filial ?? getFilial() };
0095:   return http(`/clientes/${encodeURIComponent(cod)}/ativo`, { method: "POST", body });
0096: }
```

==========
ARQUIVO: frontend/src/api/estoque.ts
==========

```
0001: import { http } from "./http";
0002:
0003: export type EstoqueSnapshot = {
0004:   B1_COD: string;
0005:   B1_DESC: string;
0006:   B1_PRECO: number;
0007:   B1_ESTOQUE: number;
0008:   B1_RESERVADO: number;
0009:   B1_CM: number;
0010:   B1_FILIAL: string;
0011:   B1_NCM?: string | null;
0012:   DISPONIVEL?: number;
0013:   saldo_sd3?: number;
0014: };
0015:
0016: export type EntradaEstoqueInput = {
0017:   cod: string;
0018:   qtd: number;
0019:   filial?: string;
```

```
0020:   custo_unit: number;
0021:   forn?: string | null;
0022:   venc_dias?: number;
0023: };
0024:
0025: export type Sd3Movimento = {
0026:   ID: number;
0027:   D3_DATA: string;
0028:   D3_TIPO: string;
0029:   D3_QTD: number;
0030:   D3_CUSTO_UNIT: number;
0031:   D3_VALOR: number;
0032:   D3_ORIGEM: string;
0033:   D3_REF?: string | null;
0034:   D3_USUARIO?: string | null;
0035: };
0036:
0037: export async function consultarEstoque(params: { cod: string; filial: string }) {
0038:   const qs = new URLSearchParams({ filial: params.filial });
0039:   return http<EstoqueSnapshot>(`/estoque/${encodeURIComponent(params.cod)}?${qs.toString()}`);
0040: }
0041:
0042: export async function entradaEstoque(payload: EntradaEstoqueInput) {
0043:   return http<EstoqueSnapshot>("/estoque/entrada", {
0044:     method: "POST",
0045:     body: payload,
0046:   });
0047: }
0048:
0049: export async function extratoEstoque(params: { cod: string; filial: string; limite?: number })
 {
0050:   const qs = new URLSearchParams({
0051:     filial: params.filial,
0052:     limite: String(params.limite ?? 20),
0053:   });
0054:   return http<Sd3Movimento[]>(`/estoque/extrato/${encodeURIComponent(params.cod)}?${qs.toStrin
g()}`);
0055: }

==========================================================================================
ARQUIVO: frontend/src/api/financeiro/ap.ts
==========================================================================================
0001: import { http } from "../http";
0002:
0003: export type ApTitulo = {
0004:   ID: number;
0005:   F1_FORN?: string | null;
0006:   F1_FORN_COD?: string | null;
0007:   F1_VALOR?: number | null;
0008:   F1_STATUS?: string | null;
0009:   F1_VENC?: string | null;
0010:   F1_DATA?: string | null;
0011:   F1_REF?: string | null;
0012:   F1_FILIAL?: string | null;
```

```
0013:   F1_SE5_ID?: number | null;
0014:   [k: string]: unknown;
0015: };
0016:
0017: export type LiquidarInput = {
0018:   titulo_id: number;
0019:   conta_id: number;
0020: };
0021:
0022: function qs(params: Record<string, unknown>) {
0023:   const sp = new URLSearchParams();
0024:   Object.entries(params).forEach(([k, v]) => {
0025:     if (v === undefined || v === null || v === "") return;
0026:     sp.set(k, String(v));
0027:   });
0028:   return sp.toString();
0029: }
0030:
0031: export async function listarAP(params: { filial?: string; status?: string } = {}) {
0032:   const query = qs(params);
0033:   return http<ApTitulo[]>(`/financeiro/ap${query ? `?${query}` : ""}`);
0034: }
0035:
0036: export async function baixarAP(payload: LiquidarInput) {
0037:   return http<{ mov_id?: number; se5_id?: number }>("/financeiro/ap/baixar", { method: "POST",
 body: payload });
0038: }
0039:
0040: export async function pagarAP(payload: LiquidarInput) {
0041:   return http<{ mov_id?: number; se5_id?: number }>("/financeiro/ap/pagar", { method: "POST",
body: payload });
0042: }
```

==========
ARQUIVO: frontend/src/api/financeiro/ar.ts
==========

```
0001: import { http } from "../http";
0002:
0003: export type ArTitulo = {
0004:   ID: number;
0005:   E1_CLIENTE?: string | null;
0006:   E1_CLIENTE_COD?: string | null;
0007:   E1_VALOR?: number | null;
0008:   E1_STATUS?: string | null;
0009:   E1_VENC?: string | null;
0010:   E1_DATA?: string | null;
0011:   E1_REF?: string | null;
0012:   E1_FILIAL?: string | null;
0013:   E1_SE5_ID?: number | null;
0014:   [k: string]: unknown;
0015: };
0016:
0017: export type LiquidarInput = {
0018:   titulo_id: number;
```

```
0019:   conta_id: number;
0020: };
0021:
0022: function qs(params: Record<string, unknown>) {
0023:   const sp = new URLSearchParams();
0024:   Object.entries(params).forEach(([k, v]) => {
0025:     if (v === undefined || v === null || v === "") return;
0026:     sp.set(k, String(v));
0027:   });
0028:   return sp.toString();
0029: }
0030:
0031: export async function listarAR(params: { filial?: string; status?: string } = {}) {
0032:   const query = qs(params);
0033:   return http<ArTitulo[]>(`/financeiro/ar${query ? `?${query}` : ""}`);
0034: }
0035:
0036: export async function baixarAR(payload: LiquidarInput) {
0037:   return http<{ mov_id?: number; se5_id?: number }>("/financeiro/ar/baixar", { method: "POST",
 body: payload });
0038: }
0039:
0040: export async function receberAR(payload: LiquidarInput) {
0041:   return http<{ mov_id?: number; se5_id?: number }>("/financeiro/ar/receber", { method: "POST"
, body: payload });
0042: }
```

==========
ARQUIVO: frontend/src/api/financeiro/caixa.ts
==========

```
0001: import { http } from "../http";
0002:
0003: export type CaixaConta = {
0004:   ID: number;
0005:   E5_FILIAL?: string | null;
0006:   E5_NOME?: string | null;
0007:   E5_TIPO?: string | null;
0008:   E5_ATIVA?: number | boolean | null;
0009:   [k: string]: unknown;
0010: };
0011:
0012: export type CaixaMovimento = {
0013:   ID: number;
0014:   E5_DATA?: string | null;
0015:   E5_FILIAL?: string | null;
0016:   E5_CONTA_ID?: number | null;
0017:   E5_TIPO?: string | null;
0018:   E5_VALOR?: number | null;
0019:   E5_VALOR_ASSINADO?: number | null;
0020:   E5_SALDO_ACUMULADO?: number | null;
0021:   E5_HIST?: string | null;
0022:   E5_ORIGEM_TIPO?: string | null;
0023:   E5_ORIGEM_ID?: number | null;
0024:   E5_USUARIO?: string | null;
```

```
0025:   E5_CATEG_ID?: number | null;
0026:   CATEG_NOME?: string | null;
0027:   CATEG_TIPO?: string | null;
0028:   [k: string]: unknown;
0029: };
0030:
0031: export type CaixaSaldo = {
0032:   entradas: number;
0033:   saidas: number;
0034:   saldo: number;
0035: };
0036:
0037: function qs(params: Record<string, unknown>) {
0038:   const sp = new URLSearchParams();
0039:   Object.entries(params).forEach(([k, v]) => {
0040:     if (v === undefined || v === null || v === "") return;
0041:     sp.set(k, String(v));
0042:   });
0043:   return sp.toString();
0044: }
0045:
0046: export async function listarContasCaixa(params: { filial?: string } = {}) {
0047:   const query = qs(params);
0048:   return http<CaixaConta[]>(`/financeiro/caixa/contas${query ? `?${query}` : ""}`);
0049: }
0050:
0051: export async function extratoCaixa(params: {
0052:   filial?: string;
0053:   conta_id: number;
0054:   de?: string | null;
0055:   ate?: string | null;
0056: }) {
0057:   const query = qs(params);
0058:   return http<CaixaMovimento[]>(`/financeiro/caixa/extrato${query ? `?${query}` : ""}`);
0059: }
0060:
0061: export async function saldoCaixa(params: {
0062:   filial?: string;
0063:   conta_id: number;
0064:   ate?: string | null;
0065: }) {
0066:   const query = qs(params);
0067:   return http<CaixaSaldo>(`/financeiro/caixa/saldo${query ? `?${query}` : ""}`);
0068: }
```

==========
ARQUIVO: frontend/src/api/financeiro/categorias.ts
==========

```
0001: import { http } from "../http";
0002:
0003: export type Categoria = {
0004:   ID: number;
0005:   C5_FILIAL?: string | null;
0006:   C5_NOME?: string | null;
```

```
0007:   C5_TIPO?: string | null;
0008:   C5_ATIVA?: number | boolean | null;
0009:   [k: string]: unknown;
0010: };
0011:
0012: export type CategoriaCreateInput = {
0013:   nome: string;
0014:   tipo: string;
0015: };
0016:
0017: export type MovSetCategoriaInput = {
0018:   mov_id: number;
0019:   categ_id?: number | null;
0020: };
0021:
0022: export type MovimentoFinanceiro = {
0023:   ID: number;
0024:   E5_FILIAL?: string | null;
0025:   E5_DATA?: string | null;
0026:   E5_TIPO?: string | null;
0027:   E5_VALOR?: number | null;
0028:   E5_HIST?: string | null;
0029:   E5_ORIGEM_TIPO?: string | null;
0030:   E5_ORIGEM_ID?: number | null;
0031:   E5_USUARIO?: string | null;
0032:   E5_CATEG_ID?: number | null;
0033:   CATEG_NOME?: string | null;
0034:   CATEG_TIPO?: string | null;
0035:   [k: string]: unknown;
0036: };
0037:
0038: function qs(params: Record<string, unknown>) {
0039:   const sp = new URLSearchParams();
0040:   Object.entries(params).forEach(([k, v]) => {
0041:     if (v === undefined || v === null || v === "") return;
0042:     sp.set(k, String(v));
0043:   });
0044:   return sp.toString();
0045: }
0046:
0047: export async function listarCategorias(params: { filial?: string; ativas?: boolean } = {}) {
0048:   const query = qs(params);
0049:   return http<Categoria[]>(`/financeiro/categorias${query ? `?${query}` : ""}`);
0050: }
0051:
0052: export async function criarCategoria(payload: CategoriaCreateInput & { filial?: string }) {
0053:   const { filial, ...rest } = payload;
0054:   const query = qs({ filial });
0055:   return http<{ id?: number }>(`/financeiro/categorias${query ? `?${query}` : ""}`, {
0056:     method: "POST",
0057:     body: rest,
0058:   });
0059: }
0060:
```

```
0061: export async function definirCategoriaMov(payload: MovSetCategoriaInput) {
0062:   return http<Record<string, unknown>>("/financeiro/movimentos/categoria", {
0063:     method: "POST",
0064:     body: payload,
0065:   });
0066: }
0067:
0068: export async function obterMovimento(movId: number) {
0069:   return http<MovimentoFinanceiro>(`/financeiro/movimentos/${movId}`);
0070: }
```

==========
ARQUIVO: frontend/src/api/fornecedores.ts
==========

```
0001: import { http } from "./http";
0002:
0003: export type Fornecedor = {
0004:   cod: string;
0005:   nome: string;
0006:   doc?: string | null;
0007:   email?: string | null;
0008:   fone?: string | null;
0009:   filial?: string;
0010:   ativo_sn?: "S" | "N";
0011: };
0012:
0013: type RawRecord = Record<string, unknown>;
0014:
0015: function asRecord(value: unknown): RawRecord {
0016:   return value && typeof value === "object" ? (value as RawRecord) : {};
0017: }
0018:
0019: function asString(value: unknown): string | undefined {
0020:   return typeof value === "string" ? value : undefined;
0021: }
0022:
0023: function normalizeFornecedor(value: unknown): Fornecedor {
0024:   const r = asRecord(value);
0025:   const ativo = asString(r.ativo_sn) ?? asString(r.A2_ATIVO_SN) ?? asString(r.a2_ativo_sn) ??
"S";
0026:
0027:   return {
0028:     cod: asString(r.cod) ?? asString(r.A2_COD) ?? asString(r.a2_cod) ?? "",
0029:     nome: asString(r.nome) ?? asString(r.A2_NOME) ?? asString(r.a2_nome) ?? "",
0030:     doc: asString(r.doc) ?? asString(r.A2_DOC) ?? asString(r.a2_doc) ?? null,
0031:     email: asString(r.email) ?? asString(r.A2_EMAIL) ?? asString(r.a2_email) ?? null,
0032:     fone: asString(r.fone) ?? asString(r.A2_FONE) ?? asString(r.a2_fone) ?? null,
0033:     filial: asString(r.filial) ?? asString(r.A2_FILIAL) ?? asString(r.a2_filial),
0034:     ativo_sn: ativo === "N" ? "N" : "S",
0035:   };
0036: }
0037:
0038: export async function listFornecedores(params: { ativos: boolean; filial: string }) {
0039:   const qs = new URLSearchParams({
```

```
0040:     ativos: String(params.ativos),
0041:     filial: params.filial,
0042:   });
0043:
0044:   const data = await http<unknown[]>(`/fornecedores?${qs.toString()}`);
0045:   return data.map(normalizeFornecedor);
0046: }
0047:
0048: export async function buscarFornecedores(params: { q: string; filial: string }) {
0049:   const qs = new URLSearchParams({ q: params.q, filial: params.filial });
0050:
0051:   const data = await http<unknown[]>(`/fornecedores/buscar?${qs.toString()}`);
0052:   return data.map(normalizeFornecedor);
0053: }
0054:
0055: export async function createFornecedor(payload: {
0056:   filial: string;
0057:   cod: string;
0058:   nome: string;
0059:   doc?: string | null;
0060:   email?: string | null;
0061:   fone?: string | null;
0062: }) {
0063:   const created = await http<unknown>("/fornecedores", { method: "POST", body: payload });
0064:   return normalizeFornecedor(created);
0065: }
0066:
0067: export async function updateFornecedor(
0068:   cod: string,
0069:   payload: { filial: string; nome?: string; doc?: string | null; email?: string | null; fone?:
 string | null }
0070: ) {
0071:   const updated = await http<unknown>(`/fornecedores/${encodeURIComponent(cod)}`, {
0072:     method: "PUT",
0073:     body: payload,
0074:   });
0075:   return normalizeFornecedor(updated);
0076: }
0077:
0078: export async function setFornecedorAtivo(cod: string, payload: { ativo: boolean; filial: strin
g }) {
0079:   return http<{ ok: true }>(`/fornecedores/${encodeURIComponent(cod)}/ativo`, {
0080:     method: "POST",
0081:     body: payload,
0082:   });
0083: }
```

==========
ARQUIVO: frontend/src/api/http.ts
==========

```
0001: const API_BASE_URL = import.meta.env.VITE_API_BASE_URL as string;
0002:
0003: type HttpMethod = "GET" | "POST" | "PUT" | "PATCH" | "DELETE";
0004: type JsonObject = Record<string, unknown>;
```

```
0005: type HttpOptions = { method?: HttpMethod; body?: unknown; headers?: Record<string, string> };
0006:
0007: function forceLogout() {
0008:   localStorage.removeItem("token");
0009:   if (window.location.pathname !== "/login") {
0010:     window.location.assign("/login");
0011:   }
0012: }
0013:
0014: function normalizeFastApiError(data: unknown, fallback: string) {
0015:   const obj = data && typeof data === "object" ? (data as JsonObject) : null;
0016:   const detailOrMessage = obj?.detail ?? obj?.message;
0017:
0018:   if (typeof detailOrMessage === "string" && detailOrMessage.trim()) return detailOrMessage;
0019:
0020:   if (Array.isArray(detailOrMessage) && detailOrMessage.length > 0) {
0021:     const first = detailOrMessage[0];
0022:     if (typeof first === "string") return first;
0023:     if (first && typeof first === "object" && "msg" in first) {
0024:       const msg = (first as JsonObject).msg;
0025:       if (typeof msg === "string") return msg;
0026:     }
0027:     return JSON.stringify(detailOrMessage);
0028:   }
0029:
0030:   if (typeof obj?.error === "string" && obj.error.trim()) return obj.error;
0031:   return fallback;
0032: }
0033:
0034: async function safeReadBody(res: Response): Promise<{ text: string; data: unknown | null }> {
0035:   const text = await res.text();
0036:   if (!text) return { text: "", data: null };
0037:
0038:   try {
0039:     return { text, data: JSON.parse(text) };
0040:   } catch {
0041:     return { text, data: null };
0042:   }
0043: }
0044:
0045: export async function http<T>(path: string, opts: HttpOptions = {}): Promise<T> {
0046:   const method = opts.method ?? "GET";
0047:   const headers: Record<string, string> = { ...(opts.headers ?? {}) };
0048:
0049:   const token = localStorage.getItem("token");
0050:   if (token) headers.Authorization = `Bearer ${token}`;
0051:
0052:   const hasBody = opts.body !== undefined && opts.body !== null;
0053:   if (hasBody) headers["Content-Type"] = headers["Content-Type"] ?? "application/json";
0054:
0055:   let res: Response;
0056:   try {
0057:     res = await fetch(`${API_BASE_URL}${path}`, {
0058:       method,
```

```
0059:       headers,
0060:       body: hasBody ? JSON.stringify(opts.body) : undefined,
0061:     });
0062:   } catch {
0063:     throw new Error("Falha de conexao com o servidor (backend). Verifique se o uvicorn esta ro
dando.");
0064:   }
0065:
0066:   if (res.status === 401) {
0067:     forceLogout();
0068:     throw new Error("Sessao expirada. Faca login novamente.");
0069:   }
0070:
0071:   const { text, data } = await safeReadBody(res);
0072:
0073:   if (res.ok && (!text || res.status === 204)) {
0074:     return null as T;
0075:   }
0076:
0077:   if (!res.ok) {
0078:     const msg = normalizeFastApiError(data ?? { message: text }, `HTTP ${res.status}`);
0079:     throw new Error(msg);
0080:   }
0081:
0082:   if (data !== null) return data as T;
0083:   return text as unknown as T;
0084: }
```

==========
ARQUIVO: frontend/src/api/parceiros.ts
==========

```
0001: import { http } from "./http";
0002:
0003: type ClienteInput = {
0004:   cod: string;
0005:   nome: string;
0006:   doc?: string | null;
0007:   email?: string | null;
0008:   tel?: string | null;
0009:   end?: string | null;
0010:   mun?: string | null;
0011:   uf?: string | null;
0012:   cep?: string | null;
0013:   tabela_id?: number | null;
0014: };
0015:
0016: type FornecedorInput = {
0017:   cod: string;
0018:   nome: string;
0019:   doc?: string | null;
0020:   email?: string | null;
0021:   tel?: string | null;
0022:   end?: string | null;
0023:   mun?: string | null;
```

```
0024:   uf?: string | null;
0025:   cep?: string | null;
0026: };
0027:
0028: function qs(params: Record<string, unknown>) {
0029:   const sp = new URLSearchParams();
0030:   Object.entries(params).forEach(([k, v]) => {
0031:     if (v === undefined || v === null || v === "") return;
0032:     sp.set(k, String(v));
0033:   });
0034:   return sp.toString();
0035: }
0036:
0037: export async function listarSa1Clientes(params: { filial?: string; ativo?: boolean; q?: string
 } = {}) {
0038:   const query = qs(params);
0039:   return http<Record<string, unknown>[]>(`/parceiros/sa1/clientes${query ? `?${query}` : ""}`)
;
0040: }
0041:
0042: export async function obterSa1Cliente(cod: string, params: { filial?: string } = {}) {
0043:   const query = qs(params);
0044:   return http<Record<string, unknown>>(
0045:     `/parceiros/sa1/clientes/${encodeURIComponent(cod)}${query ? `?${query}` : ""}`
0046:   );
0047: }
0048:
0049: export async function criarSa1Cliente(payload: ClienteInput & { filial?: string }) {
0050:   const { filial, ...body } = payload;
0051:   const query = qs({ filial });
0052:   return http<Record<string, unknown>>(
0053:     `/parceiros/sa1/clientes${query ? `?${query}` : ""}`,
0054:     { method: "POST", body }
0055:   );
0056: }
0057:
0058: export async function inativarSa1Cliente(cod: string, params: { filial?: string } = {}) {
0059:   const query = qs(params);
0060:   return http<Record<string, unknown>>(
0061:     `/parceiros/sa1/clientes/${encodeURIComponent(cod)}${query ? `?${query}` : ""}`,
0062:     { method: "DELETE" }
0063:   );
0064: }
0065:
0066: export async function setTabelaSa1Cliente(payload: { cod: string; tabela_id?: number | null; f
ilial?: string }) {
0067:   const { filial, ...body } = payload;
0068:   const query = qs({ filial });
0069:   return http<Record<string, unknown>>(
0070:     `/parceiros/sa1/clientes/set-tabela${query ? `?${query}` : ""}`,
0071:     { method: "POST", body }
0072:   );
0073: }
0074:
```

```
0075: export async function listarSa2Fornecedores(params: { filial?: string; ativo?: boolean; q?: st
ring } = {}) {
0076:   const query = qs(params);
0077:   return http<Record<string, unknown>[]>(`/parceiros/sa2/fornecedores${query ? `?${query}` : "
"}`);
0078: }
0079:
0080: export async function obterSa2Fornecedor(cod: string, params: { filial?: string } = {}) {
0081:   const query = qs(params);
0082:   return http<Record<string, unknown>>(
0083:     `/parceiros/sa2/fornecedores/${encodeURIComponent(cod)}${query ? `?${query}` : ""}`
0084:   );
0085: }
0086:
0087: export async function criarSa2Fornecedor(payload: FornecedorInput & { filial?: string }) {
0088:   const { filial, ...body } = payload;
0089:   const query = qs({ filial });
0090:   return http<Record<string, unknown>>(
0091:     `/parceiros/sa2/fornecedores${query ? `?${query}` : ""}`,
0092:     { method: "POST", body }
0093:   );
0094: }
0095:
0096: export async function inativarSa2Fornecedor(cod: string, params: { filial?: string } = {}) {
0097:   const query = qs(params);
0098:   return http<Record<string, unknown>>(
0099:     `/parceiros/sa2/fornecedores/${encodeURIComponent(cod)}${query ? `?${query}` : ""}`,
0100:     { method: "DELETE" }
0101:   );
0102: }
```

==========
ARQUIVO: frontend/src/api/precos.ts
==========

```
0001: import { http } from "./http";
0002:
0003: function qs(params: Record<string, unknown>) {
0004:   const sp = new URLSearchParams();
0005:   Object.entries(params).forEach(([k, v]) => {
0006:     if (v === undefined || v === null || v === "") return;
0007:     sp.set(k, String(v));
0008:   });
0009:   return sp.toString();
0010: }
0011:
0012: export type TabelaPreco = {
0013:   id: number;
0014:   filial: string;
0015:   codigo: string;
0016:   descricao: string;
0017:   ativa: boolean;
0018: };
0019:
0020: export type ItemPrecoTabela = {
```

```
0021:   id: number;
0022:   filial: string;
0023:   tabela_id: number;
0024:   produto: string;
0025:   preco: number;
0026:   dt_ini: string | null;
0027:   dt_fim: string | null;
0028:   vigente: boolean;
0029: };
0030:
0031: export async function listarTabelasPreco(params: { filial?: string } = {}) {
0032:   const query = qs(params);
0033:   return http<TabelaPreco[]>(`/precos/tabelas${query ? `?${query}` : ""}`);
0034: }
0035:
0036: export async function criarTabelaPreco(payload: { codigo: string; descricao: string; filial?:
string }) {
0037:   const { filial, ...body } = payload;
0038:   const query = qs({ filial });
0039:   return http<Record<string, unknown>>(`/precos/tabelas${query ? `?${query}` : ""}`, {
0040:     method: "POST",
0041:     body,
0042:   });
0043: }
0044:
0045: export async function listarItensTabelaPreco(tabelaId: number, params: { filial?: string } = {
}) {
0046:   const query = qs(params);
0047:   return http<ItemPrecoTabela[]>(`/precos/tabelas/${tabelaId}/itens${query ? `?${query}` : ""}
`);
0048: }
0049:
0050: export async function definirPrecoTabela(payload: {
0051:   tabela_id: number;
0052:   produto: string;
0053:   preco: number;
0054:   dt_ini: string;
0055:   filial?: string;
0056: }) {
0057:   const { filial, ...body } = payload;
0058:   const query = qs({ filial });
0059:   return http<Record<string, unknown>>(`/precos/definir${query ? `?${query}` : ""}`, {
0060:     method: "POST",
0061:     body,
0062:   });
0063: }
0064:
0065: export async function buscarPrecoTabela(params: {
0066:   produto: string;
0067:   tabela_id: number;
0068:   data: string;
0069:   filial?: string;
0070: }) {
0071:   const query = qs(params);
```

```
0072:   return http<Record<string, unknown>>(`/precos/buscar${query ? `?${query}` : ""}`);
0073: }
```

==========
ARQUIVO: frontend/src/api/produtos.ts
==========

```
0001: import { http } from "./http";
0002:
0003: export type ProdutoInput = {
0004:   cod: string;
0005:   desc: string;
0006:   preco: number;
0007:   filial?: string;
0008: };
0009:
0010: export type ProdutoResumo = {
0011:   cod: string;
0012:   desc: string;
0013:   preco: number;
0014:   estoque: number;
0015:   reservado: number;
0016:   cm: number;
0017:   filial?: string;
0018:   ncm?: string | null;
0019: };
0020:
0021: type RawRecord = Record<string, unknown>;
0022:
0023: function asRecord(value: unknown): RawRecord {
0024:   return value && typeof value === "object" ? (value as RawRecord) : {};
0025: }
0026:
0027: function asString(value: unknown): string {
0028:   return typeof value === "string" ? value : "";
0029: }
0030:
0031: function asNumber(value: unknown): number {
0032:   const n = Number(value);
0033:   return Number.isFinite(n) ? n : 0;
0034: }
0035:
0036: function normalizeProduto(value: unknown): ProdutoResumo {
0037:   const r = asRecord(value);
0038:   return {
0039:     cod: asString(r.cod) || asString(r.B1_COD),
0040:     desc: asString(r.desc) || asString(r.B1_DESC),
0041:     preco: asNumber(r.preco ?? r.B1_PRECO),
0042:     estoque: asNumber(r.estoque ?? r.B1_ESTOQUE),
0043:     reservado: asNumber(r.reservado ?? r.B1_RESERVADO),
0044:     cm: asNumber(r.cm ?? r.B1_CM),
0045:     filial: asString(r.filial) || asString(r.B1_FILIAL),
0046:     ncm: asString(r.ncm) || asString(r.B1_NCM) || null,
0047:   };
0048: }
```

```
0049:
0050: export async function upsertProduto(payload: ProdutoInput) {
0051:   return http<Record<string, unknown>>("/produto", { method: "POST", body: payload });
0052: }
0053:
0054: export async function buscarProdutos(params: { q: string; filial: string; limite?: number }) {
0055:   const qs = new URLSearchParams({
0056:     q: params.q,
0057:     filial: params.filial,
0058:     limite: String(params.limite ?? 10),
0059:   });
0060:   const data = await http<unknown[]>(`/produtos/buscar?${qs.toString()}`);
0061:   return data.map(normalizeProduto);
0062: }
```

==========
ARQUIVO: frontend/src/api/relatorios/dre.ts
==========

```
0001: import { http } from "../http";
0002:
0003: export type DreLinha = {
0004:   grupo?: string | null;
0005:   valor?: number | null;
0006:   [k: string]: unknown;
0007: };
0008:
0009: export type DreResponse = {
0010:   filial: string;
0011:   de?: string | null;
0012:   ate?: string | null;
0013:   linhas: DreLinha[];
0014:   total: number;
0015: };
0016:
0017: function qs(params: Record<string, unknown>) {
0018:   const sp = new URLSearchParams();
0019:   Object.entries(params).forEach(([k, v]) => {
0020:     if (v === undefined || v === null || v === "") return;
0021:     sp.set(k, String(v));
0022:   });
0023:   return sp.toString();
0024: }
0025:
0026: export async function obterDre(params: { filial?: string; de?: string | null; ate?: string | n
ull } = {}) {
0027:   const query = qs(params);
0028:   return http<DreResponse>(`/financeiro/relatorios/dre${query ? `?${query}` : ""}`);
0029: }
```

==========
ARQUIVO: frontend/src/api/relatorios/margem-produto.ts
==========

```
0001: import { http } from "../http";
0002:
```

```
0003: export type MargemProdutoRow = {
0004:   produto: string;
0005:   receita: number;
0006:   cmv: number;
0007:   margem: number;
0008:   [k: string]: unknown;
0009: };
0010:
0011: function qs(params: Record<string, unknown>) {
0012:   const sp = new URLSearchParams();
0013:   Object.entries(params).forEach(([k, v]) => {
0014:     if (v === undefined || v === null || v === "") return;
0015:     sp.set(k, String(v));
0016:   });
0017:   return sp.toString();
0018: }
0019:
0020: export async function obterMargemProduto(params: { filial?: string; de?: string | null; ate?:
string | null } = {}) {
0021:   const query = qs(params);
0022:   return http<MargemProdutoRow[]>(`/relatorios/margem-produto${query ? `?${query}` : ""}`);
0023: }
```

==========
ARQUIVO: frontend/src/api/reservas.ts
==========

```
0001: import { http } from "./http";
0002:
0003: export type ReservaInput = {
0004:   cod: string;
0005:   qtd: number;
0006:   filial?: string;
0007:   cliente_cod?: string | null;
0008:   tabela_cod?: string;
0009: };
0010:
0011: export type CancelarInput = {
0012:   reserva_id: number;
0013: };
0014:
0015: export type ConfirmarInput = {
0016:   reserva_id: number;
0017:   cliente_cod?: string | null;
0018:   venc_dias?: number;
0019: };
0020:
0021: export async function criarReserva(payload: ReservaInput) {
0022:   return http<Record<string, unknown>>("/reserva", { method: "POST", body: payload });
0023: }
0024:
0025: export async function cancelarReserva(payload: CancelarInput) {
0026:   return http<Record<string, unknown>>("/reserva/cancelar", { method: "POST", body: payload })
;
0027: }
```

```
0028:
0029: export async function confirmarReserva(payload: ConfirmarInput) {
0030:   return http<Record<string, unknown>>("/reserva/confirmar", { method: "POST", body: payload }
);
0031: }
```

==========
ARQUIVO: frontend/src/api/usuarios.ts
==========

```
0001: import { http } from "./http";
0002:
0003: export type BootstrapInput = {
0004:   login: string;
0005:   senha: string;
0006: };
0007:
0008: export type UsuarioCreateInput = {
0009:   login: string;
0010:   senha: string;
0011:   perfil: string;
0012: };
0013:
0014: export async function bootstrapAdmin(payload: BootstrapInput) {
0015:   return http<{ status: string; login: string }>("/bootstrap", { method: "POST", body: payload
 });
0016: }
0017:
0018: export async function criarUsuario(payload: UsuarioCreateInput) {
0019:   return http<{ status: string; login: string }>("/usuarios", { method: "POST", body: payload
});
0020: }
```

==========
ARQUIVO: frontend/src/api/vendas.ts
==========

```
0001: import { http } from "./http";
0002:
0003: export type PedidoStatus = "ABERTO" | "RESERVA_CRIADA" | "RESERVA_FATURADA" | "CANCELADO";
0004:
0005: export type Pedido = {
0006:   id: number;
0007:   filial: string;
0008:   cliente_cod?: string | null;
0009:   status?: PedidoStatus | string;
0010:   origem?: string | null;
0011:   total?: number | null;
0012:   total_bruto?: number | null;
0013:   icms?: number | null;
0014:   ipi?: number | null;
0015:   criado_em?: string | null;
0016:   atualizado_em?: string | null;
0017:   [k: string]: unknown;
0018: };
0019:
```

```
0020: export type PedidoEnriquecido = Pedido & {
0021:   cliente_nome?: string | null;
0022:   itens?: ItemPedido[];
0023:   [k: string]: unknown;
0024: };
0025:
0026: export type ItemPedido = {
0027:   id: number;
0028:   pedido_id: number;
0029:   filial: string;
0030:   produto: string;
0031:   qtd: number;
0032:   total: number;
0033:   preco_unit?: number | null;
0034:   cmv_unit?: number | null;
0035:   status?: string;
0036:   ativo_sn?: "S" | "N";
0037:   [k: string]: unknown;
0038: };
0039:
0040: export type PedidoCriarInput = {
0041:   filial: string;
0042:   valor_total: number;
0043:   status?: string;
0044:   icms?: number | null;
0045:   ipi?: number | null;
0046:   total_bruto?: number | null;
0047: };
0048:
0049: export type ItemCriarInput = {
0050:   filial: string;
0051:   produto: string;
0052:   qtd: number;
0053:   total: number;
0054:   preco_unit?: number | null;
0055:   cmv_unit?: number | null;
0056: };
0057:
0058: export type FaturarPedidoInput = {
0059:   filial: string;
0060:   cliente_cod?: string | null;
0061:   venc_dias?: number;
0062: };
0063:
0064: export type CancelarPedidoInput = {
0065:   filial: string;
0066:   modo?: "AUTO" | "MANUAL" | string;
0067:   reativar_reserva?: boolean;
0068: };
0069:
0070: export type RecalcularPedidoInput = {
0071:   forcar?: boolean;
0072: };
0073:
```

```
0074: export type EditarItemInput = {
0075:   filial: string;
0076:   qtd: number;
0077:   total: number;
0078:   preco_unit?: number | null;
0079:   cmv_unit?: number | null;
0080: };
0081:
0082: export type ExcluirItemInput = {
0083:   filial: string;
0084: };
0085:
0086: export type EstornarPedidoInput = {
0087:   filial: string;
0088:   motivo?: string | null;
0089: };
0090:
0091: export type PagedResponse<T> = {
0092:   total: number;
0093:   limit: number;
0094:   offset: number;
0095:   items: T[];
0096: };
0097:
0098: function qs(params: Record<string, unknown>) {
0099:   const sp = new URLSearchParams();
0100:   Object.entries(params).forEach(([k, v]) => {
0101:     if (v === undefined || v === null || v === "") return;
0102:     sp.set(k, String(v));
0103:   });
0104:   return sp.toString();
0105: }
0106:
0107: export async function criarPedido(payload: PedidoCriarInput) {
0108:   return http<Pedido>("/vendas/pedidos", { method: "POST", body: payload });
0109: }
0110:
0111: export async function listarPedidos(
0112:   params: {
0113:     filial?: string;
0114:     status?: string;
0115:     origem?: string;
0116:     de?: string;
0117:     ate?: string;
0118:     limit?: number;
0119:     offset?: number;
0120:   } = {}
0121: ) {
0122:   const query = qs(params);
0123:   return http<PagedResponse<Pedido>>(`/vendas/pedidos${query ? `?${query}` : ""}`);
0124: }
0125:
0126: export async function listarPedidosEnriquecido(
0127:   params: {
```

```
0128:     filial?: string;
0129:     status?: string;
0130:     origem?: string;
0131:     de?: string;
0132:     ate?: string;
0133:     limit?: number;
0134:     offset?: number;
0135:   } = {}
0136: ) {
0137:   const query = qs(params);
0138:   return http<PagedResponse<PedidoEnriquecido>>(`/vendas/pedidos/enriquecido${query ? `?${quer
y}` : ""}`);
0139: }
0140:
0141: export async function obterPedido(pedidoId: number) {
0142:   return http<PedidoEnriquecido | Pedido>(`/vendas/pedidos/${encodeURIComponent(String(pedidoI
d))}`);
0143: }
0144:
0145: export async function adicionarItem(pedidoId: number, payload: ItemCriarInput) {
0146:   return http<{ pedido_id: number; item_id: number; novo_total?: number; qtd_itens?: number }>
(
0147:     `/vendas/pedidos/${encodeURIComponent(String(pedidoId))}/itens`,
0148:     { method: "POST", body: payload }
0149:   );
0150: }
0151:
0152: export async function editarItem(itemId: number, payload: EditarItemInput) {
0153:   return http<{ pedido_id: number; item_id: number; novo_total?: number; qtd_itens?: number }>
(
0154:     `/vendas/itens/${encodeURIComponent(String(itemId))}`,
0155:     { method: "PUT", body: payload }
0156:   );
0157: }
0158:
0159: export async function excluirItem(itemId: number, payload: ExcluirItemInput) {
0160:   return http<{ pedido_id: number; item_id: number; novo_total?: number; qtd_itens?: number }>
(
0161:     `/vendas/itens/${encodeURIComponent(String(itemId))}`,
0162:     { method: "DELETE", body: payload }
0163:   );
0164: }
0165:
0166: export async function recalcularPedido(pedidoId: number, payload: RecalcularPedidoInput = {})
{
0167:   return http<{ pedido_id: number; novo_total?: number; qtd_itens?: number }>(
0168:     `/vendas/pedidos/${encodeURIComponent(String(pedidoId))}/recalcular`,
0169:     { method: "POST", body: payload }
0170:   );
0171: }
0172:
0173: export async function faturarPedido(pedidoId: number, payload: FaturarPedidoInput) {
0174:   return http<Record<string, unknown>>(
0175:     `/vendas/pedidos/${encodeURIComponent(String(pedidoId))}/faturar`,
```

```
0176:     { method: "POST", body: payload }
0177:   );
0178: }
0179:
0180: export async function cancelarPedido(pedidoId: number, payload: CancelarPedidoInput) {
0181:   return http<Record<string, unknown>>(
0182:     `/vendas/pedidos/${encodeURIComponent(String(pedidoId))}/cancelar`,
0183:     { method: "POST", body: payload }
0184:   );
0185: }
0186:
0187: export async function estornarPedido(pedidoId: number, payload: EstornarPedidoInput) {
0188:   return http<Record<string, unknown>>(
0189:     `/vendas/pedidos/${encodeURIComponent(String(pedidoId))}/estornar`,
0190:     { method: "POST", body: payload }
0191:   );
0192: }
```

==========
ARQUIVO: frontend/src/App.tsx
==========

```
0001: import { BrowserRouter, Navigate, Route, Routes, Outlet } from "react-router-dom";
0002:
0003: import { AuthProvider } from "./auth/AuthContext";
0004: import { RequireAuth } from "./auth/RequireAuth";
0005:
0006: import { Login } from "./pages/Login";
0007: import { ClientsList } from "./pages/clientsList";
0008: import { FornecedoresList } from "./pages/FornecedoresList";
0009: import { PedidosList } from "./pages/pedidosList";
0010: import { FinanceiroAR } from "./pages/financeiroAR";
0011: import { FinanceiroAP } from "./pages/financeiroAP";
0012: import { FinanceiroCaixa } from "./pages/financeiroCaixa";
0013: import { FinanceiroCategorias } from "./pages/financeiroCategorias";
0014: import { RelatorioDRE } from "./pages/relatorioDRE";
0015: import { ProdutosList } from "./pages/produtosList";
0016: import { EstoqueEntradaFornecedor } from "./pages/estoqueEntradaFornecedor";
0017: import { Parceiros } from "./pages/parceiros";
0018: import { ReservasList } from "./pages/reservasList";
0019: import { AuditoriaList } from "./pages/auditoriaList";
0020: import { UsuariosList } from "./pages/usuariosList";
0021: import { RelatorioMargemProduto } from "./pages/relatorioMargemProduto";
0022: import { Dashboard } from "./pages/Dashboard";
0023:
0024: // Se você já tem componentes de layout, use os seus imports aqui:
0025: import { Sidebar } from "./layout/Sidebar";
0026: import { Topbar } from "./layout/Topbar";
0027:
0028: function AppLayout() {
0029:   return (
0030:     <div className="layout">
0031:       <Sidebar />
0032:       <main className="content">
0033:         <Topbar />
```

```
0034:         <section className="page">
0035:           <Outlet />
0036:         </section>
0037:       </main>
0038:     </div>
0039:   );
0040: }
0041:
0042: export default function App() {
0043:   return (
0044:     <AuthProvider>
0045:       <BrowserRouter>
0046:         <Routes>
0047:           {/* Public */}
0048:           <Route path="/login" element={<Login />} />
0049:
0050:           {/* Protected */}
0051:           <Route
0052:             element={
0053:               <RequireAuth>
0054:                 <AppLayout />
0055:               </RequireAuth>
0056:             }
0057:           >
0058:             <Route path="/" element={<Navigate to="/dashboard" replace />} />
0059:             <Route path="/dashboard" element={<Dashboard />} />
0060:             <Route path="/clientes" element={<ClientsList />} />
0061:             <Route path="/fornecedores" element={<FornecedoresList />} />
0062:             <Route path="/pedidos" element={<PedidosList />} />
0063:             <Route path="/vendas/pedidos" element={<PedidosList />} />
0064:             <Route path="/financeiro/ar" element={<FinanceiroAR />} />
0065:             <Route path="/financeiro/ap" element={<FinanceiroAP />} />
0066:             <Route path="/financeiro/caixa" element={<FinanceiroCaixa />} />
0067:             <Route path="/financeiro/categorias" element={<FinanceiroCategorias />} />
0068:             <Route path="/relatorios/dre" element={<RelatorioDRE />} />
0069:             <Route path="/produtos" element={<ProdutosList />} />
0070:             <Route path="/custos/fornecedor" element={<EstoqueEntradaFornecedor />} />
0071:             <Route path="/estoque/entrada-fornecedor" element={<EstoqueEntradaFornecedor />} /
>
0072:             <Route path="/parceiros" element={<Parceiros />} />
0073:             <Route path="/reservas" element={<ReservasList />} />
0074:             <Route path="/auditoria" element={<AuditoriaList />} />
0075:             <Route path="/usuarios" element={<UsuariosList />} />
0076:             <Route path="/relatorios/margem-produto" element={<RelatorioMargemProduto />} />
0077:
0078:             {/* Adicione próximos módulos aqui */}
0079:             {/* <Route path="/fornecedores" element={<FornecedoresList />} /> */}
0080:             {/* <Route path="/produtos" element={<ProdutosList />} /> */}
0081:           </Route>
0082:
0083:           {/* 404 */}
0084:           <Route path="*" element={<Navigate to="/clientes" replace />} />
0085:         </Routes>
0086:       </BrowserRouter>
```

```
0087:     </AuthProvider>
0088:   );
0089: }
```

==========
ARQUIVO: frontend/src/auth/AuthContext.tsx
==========

```
0001: import React, { createContext, useContext, useEffect, useMemo, useState } from "react";
0002: import { login as apiLogin, type LoginResponse } from "../api/auth";
0003:
0004: type AuthState = {
0005:   token: string | null;
0006:   isAuthenticated: boolean;
0007:   login: (u: string, p: string) => Promise<void>;
0008:   logout: () => void;
0009: };
0010:
0011: const Ctx = createContext<AuthState | null>(null);
0012:
0013: function getToken() {
0014:   return localStorage.getItem("token");
0015: }
0016:
0017: export function AuthProvider({ children }: { children: React.ReactNode }) {
0018:   const [token, setToken] = useState<string | null>(() => getToken());
0019:
0020:   // ? sincroniza token entre abas/janelas
0021:   useEffect(() => {
0022:     const onStorage = (e: StorageEvent) => {
0023:       if (e.key === "token") setToken(getToken());
0024:     };
0025:     window.addEventListener("storage", onStorage);
0026:     return () => window.removeEventListener("storage", onStorage);
0027:   }, []);
0028:
0029:   const value = useMemo<AuthState>(
0030:     () => ({
0031:       token,
0032:       isAuthenticated: !!token,
0033:
0034:       login: async (u, p) => {
0035:         const r: LoginResponse = await apiLogin(u, p);
0036:         localStorage.setItem("token", r.access_token);
0037:         setToken(r.access_token);
0038:       },
0039:
0040:       logout: () => {
0041:         localStorage.removeItem("token");
0042:         setToken(null);
0043:         // ? UX: manda pro login
0044:         if (window.location.pathname !== "/login") window.location.assign("/login");
0045:       },
0046:     }),
0047:     [token]
```

```
0048:   );
0049:
0050:   return <Ctx.Provider value={value}>{children}</Ctx.Provider>;
0051: }
0052:
0053: // eslint-disable-next-line react-refresh/only-export-components
0054: export function useAuth() {
0055:   const v = useContext(Ctx);
0056:   if (!v) throw new Error("useAuth must be used within AuthProvider");
0057:   return v;
0058: }
```

==========
ARQUIVO: frontend/src/auth/ProtectedRoute.tsx
==========

```
0001: import React from "react";
0002: import { Navigate } from "react-router-dom";
0003: import { useAuth } from "./AuthContext";
0004:
0005: export function ProtectedRoute({ children }: { children: React.ReactNode }) {
0006:   const { token } = useAuth();
0007:   if (!token) return <Navigate to="/login" replace />;
0008:   return <>{children}</>;
0009: }
```

==========
ARQUIVO: frontend/src/auth/RequireAuth.tsx
==========

```
0001: import React from "react";
0002: import { Navigate, useLocation } from "react-router-dom";
0003: import { useAuth } from "./AuthContext";
0004:
0005: export function RequireAuth({ children }: { children: React.ReactNode }) {
0006:   const { isAuthenticated } = useAuth();
0007:   const loc = useLocation();
0008:
0009:   if (!isAuthenticated) {
0010:     return <Navigate to="/login" replace state={{ from: loc.pathname }} />;
0011:   }
0012:
0013:   return <>{children}</>;
0014: }
```

==========
ARQUIVO: frontend/src/components/ConfirmDialog.tsx
==========

```
0001: import { Modal } from "./Modal";
0002:
0003: type Props = {
0004:   title?: string;
0005:   message: string;
0006:   confirmText?: string;
0007:   cancelText?: string;
0008:   danger?: boolean;
```

```
0009:   loading?: boolean;
0010:   onConfirm: () => void;
0011:   onClose: () => void;
0012: };
0013:
0014: export function ConfirmDialog({
0015:   title = "Confirmar",
0016:   message,
0017:   confirmText = "Confirmar",
0018:   cancelText = "Cancelar",
0019:   danger = false,
0020:   loading = false,
0021:   onConfirm,
0022:   onClose,
0023: }: Props) {
0024:   return (
0025:     <Modal title={title} onClose={onClose}>
0026:       <div className="hint" style={{ marginBottom: 14 }}>
0027:         {message}
0028:       </div>
0029:
0030:       <div className="form-actions">
0031:         <button className="btn" onClick={onClose} disabled={loading}>
0032:           {cancelText}
0033:         </button>
0034:
0035:         <button
0036:           className={`btn ${danger ? "danger" : "primary"}`}
0037:           onClick={onConfirm}
0038:           disabled={loading}
0039:         >
0040:           {confirmText}
0041:         </button>
0042:       </div>
0043:     </Modal>
0044:   );
0045: }
```

==========
ARQUIVO: frontend/src/components/DataTable.tsx
==========

```
0001: import { isValidElement, useMemo, type ReactNode } from "react";
0002:
0003: export type SortDir = "asc" | "desc";
0004:
0005: export type Column<Row extends Record<string, unknown>> = {
0006:   key: string;
0007:   header: string | ReactNode;
0008:   sortable?: boolean;
0009:   width?: number | string;
0010:   align?: "left" | "center" | "right";
0011:   className?: string;
0012:   value?: (row: Row) => unknown;
0013:   render?: (row: Row) => ReactNode;
```

```
0014: };
0015:
0016: type Props<Row extends Record<string, unknown>> = {
0017:   rows: Row[];
0018:   columns: Column<Row>[];
0019:   sortKey: string;
0020:   sortDir: SortDir;
0021:   onSortChange: (key: string, dir: SortDir) => void;
0022:   emptyText?: string;
0023:   loading?: boolean;
0024:   className?: string;
0025:   wrapperClassName?: string;
0026: };
0027:
0028: function normalize(v: unknown) {
0029:   if (v === null || v === undefined) return "";
0030:   if (typeof v === "boolean") return v ? "1" : "0";
0031:   if (typeof v === "number") return String(v).padStart(20, "0");
0032:   return String(v).toLowerCase().trim();
0033: }
0034:
0035: function toReactNode(value: unknown): ReactNode {
0036:   if (value === null || value === undefined) return "";
0037:   if (typeof value === "string" || typeof value === "number") return value;
0038:   if (typeof value === "boolean") return value ? "true" : "false";
0039:   if (isValidElement(value)) return value;
0040:   return String(value);
0041: }
0042:
0043: export function DataTable<Row extends Record<string, unknown>>({
0044:   rows,
0045:   columns,
0046:   sortKey,
0047:   sortDir,
0048:   onSortChange,
0049:   emptyText = "Nenhum registro encontrado.",
0050:   loading = false,
0051:   className,
0052:   wrapperClassName,
0053: }: Props<Row>) {
0054:   const sortedRows = useMemo(() => {
0055:     const col = columns.find((c) => c.key === sortKey);
0056:     const valueFn = col?.value ?? ((r: Row) => r[sortKey]);
0057:
0058:     const copy = [...rows];
0059:     copy.sort((a, b) => {
0060:       const va = normalize(valueFn(a));
0061:       const vb = normalize(valueFn(b));
0062:       if (va < vb) return sortDir === "asc" ? -1 : 1;
0063:       if (va > vb) return sortDir === "asc" ? 1 : -1;
0064:       return 0;
0065:     });
0066:     return copy;
0067:   }, [rows, columns, sortKey, sortDir]);
```

```
0068:
0069:   function toggleSort(key: string) {
0070:     if (sortKey === key) {
0071:       onSortChange(key, sortDir === "asc" ? "desc" : "asc");
0072:     } else {
0073:       onSortChange(key, "asc");
0074:     }
0075:   }
0076:
0077:   function indicator(key: string) {
0078:     if (sortKey !== key) return "";
0079:     return sortDir === "asc" ? " ?" : " ?";
0080:   }
0081:
0082:   const table = (
0083:     <table className={`table mt-12 ${className ?? ""}`.trim()}>
0084:       <thead>
0085:         <tr>
0086:           {columns.map((c) => {
0087:             const clickable = !!c.sortable;
0088:             return (
0089:               <th
0090:                 key={c.key}
0091:                 onClick={clickable ? () => toggleSort(c.key) : undefined}
0092:                 style={{
0093:                   cursor: clickable ? "pointer" : undefined,
0094:                   width: c.width,
0095:                   textAlign: c.align ?? "left",
0096:                 }}
0097:                 className={c.className}
0098:                 aria-sort={
0099:                   sortKey === c.key ? (sortDir === "asc" ? "ascending" : "descending") : "none
"
0100:                 }
0101:               >
0102:                 {c.header}
0103:                 {clickable ? indicator(c.key) : ""}
0104:               </th>
0105:             );
0106:           })}
0107:         </tr>
0108:       </thead>
0109:
0110:       <tbody>
0111:         {sortedRows.length === 0 ? (
0112:           <tr>
0113:             <td colSpan={columns.length} className="p-14">
0114:               {loading ? "Carregando..." : emptyText}
0115:             </td>
0116:           </tr>
0117:         ) : (
0118:           sortedRows.map((row, idx) => (
0119:             <tr key={idx}>
0120:               {columns.map((c) => {
```

```
0121:                 const raw = c.render?.(row) ?? (c.value ? c.value(row) : row[c.key] ?? "");
0122:                 const content = toReactNode(raw);
0123:                 return (
0124:                   <td key={c.key} style={{ textAlign: c.align ?? "left" }} className={c.classN
ame}>
0125:                     {content}
0126:                   </td>
0127:                 );
0128:               })}
0129:             </tr>
0130:           ))
0131:         )}
0132:       </tbody>
0133:     </table>
0134:   );
0135:
0136:   if (wrapperClassName) {
0137:     return <div className={wrapperClassName}>{table}</div>;
0138:   }
0139:
0140:   return table;
0141: }
```

==========
ARQUIVO: frontend/src/components/FormField.tsx
==========

```
0001: import type { ReactNode } from "react";
0002:
0003: export function FormField({ label, children }: { label: string; children: ReactNode }) {
0004:   return (
0005:     <div className="form-field">
0006:       <label>{label}</label>
0007:       {children}
0008:     </div>
0009:   );
0010: }
```

==========
ARQUIVO: frontend/src/components/Modal.tsx
==========

```
0001: import type { ReactNode } from "react";
0002:
0003: export function Modal({
0004:   title,
0005:   onClose,
0006:   children,
0007: }: {
0008:   title: string;
0009:   onClose: () => void;
0010:   children: ReactNode;
0011: }) {
0012:   return (
0013:     <div className="modal-backdrop" onMouseDown={(e) => {
0014:       if ((e.target as HTMLElement).classList.contains("modal-backdrop")) onClose();
```

```
0015:     }}>
0016:       <div className="modal" role="dialog" aria-modal="true" aria-label={title}>
0017:         <div className="modal-header">
0018:           <h3 style={{ margin: 0 }}>{title}</h3>
0019:           <button className="modal-close" type="button" onClick={onClose} aria-label="Fechar">
?</button>
0020:         </div>
0021:         <div className="modal-body">{children}</div>
0022:       </div>
0023:     </div>
0024:   );
0025: }
```

==========
ARQUIVO: frontend/src/components/PageHero.tsx
==========

```
0001: type HeroMetric = {
0002:   label: string;
0003:   value: string;
0004: };
0005:
0006: type Props = {
0007:   icon: string;
0008:   eyebrow: string;
0009:   title: string;
0010:   description: string;
0011:   metrics?: HeroMetric[];
0012:   actions?: React.ReactNode;
0013:   backgroundImage?: string;
0014:   backgroundPosition?: string;
0015:   backgroundSize?: string;
0016: };
0017:
0018: export function PageHero({
0019:   icon,
0020:   eyebrow,
0021:   title,
0022:   description,
0023:   metrics = [],
0024:   actions,
0025:   backgroundImage,
0026:   backgroundPosition,
0027:   backgroundSize,
0028: }: Props) {
0029:   return (
0030:     <section
0031:       className={`hero-panel ${backgroundImage ? "hero-panel-bg" : ""}`.trim()}
0032:       style={
0033:         backgroundImage
0034:           ? ({
0035:               ["--hero-bg" as string]: `url("${backgroundImage}")`,
0036:               ["--hero-bg-position" as string]: backgroundPosition ?? "center",
0037:               ["--hero-bg-size" as string]: backgroundSize ?? "cover",
0038:             } as React.CSSProperties)
```

```
0039:           : undefined
0040:       }
0041:     >
0042:       <div className="hero-main">
0043:         <div className="hero-icon-wrap">
0044:           <img src={icon} alt="" className="hero-icon" />
0045:         </div>
0046:
0047:         <div className="hero-copy">
0048:           <div className="hero-eyebrow">{eyebrow}</div>
0049:           <h1 className="hero-title">{title}</h1>
0050:           <p className="hero-description">{description}</p>
0051:
0052:           {metrics.length > 0 ? (
0053:             <div className="hero-metrics">
0054:               {metrics.map((metric) => (
0055:                 <div key={metric.label} className="hero-metric">
0056:                   <div className="hero-metric-label">{metric.label}</div>
0057:                   <div className="hero-metric-value">{metric.value}</div>
0058:                 </div>
0059:               ))}
0060:             </div>
0061:           ) : null}
0062:         </div>
0063:       </div>
0064:
0065:       {actions ? <div className="hero-actions">{actions}</div> : null}
0066:     </section>
0067:   );
0068: }
```

==========
ARQUIVO: frontend/src/components/Pager.tsx
==========

```
0001: type Props = {
0002:   page: number;
0003:   totalPages: number;
0004:   pageSize: number;
0005:   totalItems: number;
0006:
0007:   onPageChange: (page: number) => void;
0008:   onPageSizeChange: (pageSize: number) => void;
0009:
0010:   showPageSize?: boolean; // topo: true, rodapé: false
0011: };
0012:
0013: export function Pager({
0014:   page,
0015:   totalPages,
0016:   pageSize,
0017:   totalItems,
0018:   onPageChange,
0019:   onPageSizeChange,
0020:   showPageSize = false,
```

```
0021: }: Props) {
0022:   const safePage = Math.min(Math.max(1, page), Math.max(1, totalPages));
0023:
0024:   return (
0025:     <div className="row-between mt-12">
0026:       <div className="hint">
0027:         Página <b>{safePage}</b> de <b>{Math.max(1, totalPages)}</b>
0028:         {showPageSize ? (
0029:           <>
0030:             {" "}
0031:             — {totalItems} registros
0032:           </>
0033:         ) : null}
0034:       </div>
0035:
0036:       <div className="row gap-10">
0037:         {showPageSize && (
0038:           <select
0039:             className="select"
0040:             value={pageSize}
0041:             onChange={(e) => onPageSizeChange(Number(e.target.value))}
0042:             aria-label="Itens por página"
0043:           >
0044:             <option value={10}>10</option>
0045:             <option value={20}>20</option>
0046:             <option value={50}>50</option>
0047:           </select>
0048:         )}
0049:
0050:         <button
0051:           className="btn"
0052:           disabled={safePage <= 1}
0053:           onClick={() => onPageChange(Math.max(1, safePage - 1))}
0054:         >
0055:           Anterior
0056:         </button>
0057:
0058:         <button
0059:           className="btn"
0060:           disabled={safePage >= totalPages}
0061:           onClick={() => onPageChange(Math.min(totalPages, safePage + 1))}
0062:         >
0063:           Próxima
0064:         </button>
0065:       </div>
0066:     </div>
0067:   );
0068: }
```

==========
ARQUIVO: frontend/src/components/Toolbar.tsx
==========

```
0001: type Props = {
0002:   left?: React.ReactNode;
```

```
0003:   right?: React.ReactNode;
0004: };
0005:
0006: export function Toolbar({ left, right }: Props) {
0007:   return (
0008:     <div className="toolbar">
0009:       <div className="toolbar-left">{left}</div>
0010:       <div className="toolbar-right">{right}</div>
0011:     </div>
0012:   );
0013: }
```

==========
ARQUIVO: frontend/src/index.css
==========

```
0001: /* index.css (mínimo, sem tema do template) */
0002: * { box-sizing: border-box; }
0003: html, body, #root { height: 100%; }
0004: body { margin: 0; }
```

==========
ARQUIVO: frontend/src/layout/AppLayout.tsx
==========

```
0001: import type { ReactNode } from "react";
0002: import { Sidebar } from "./Sidebar";
0003: import { Topbar } from "./Topbar";
0004:
0005: export function AppLayout({ children }: { children: ReactNode }) {
0006:   return (
0007:     <div className="layout">
0008:       <Sidebar />
0009:       <main>
0010:         <Topbar />
0011:         <div className="page">{children}</div>
0012:       </main>
0013:     </div>
0014:   );
0015: }
```

==========
ARQUIVO: frontend/src/layout/Sidebar.tsx
==========

```
0001: import { useEffect, useState } from "react";
0002: import { NavLink } from "react-router-dom";
0003: import { getArea, onAreaChange } from "../state/uiPrefs";
0004:
0005: type MenuItem = {
0006:   to: string;
0007:   label: string;
0008:   code: string;
0009: };
0010:
0011: type MenuSection = {
0012:   title: string;
```

```
0013:   items: MenuItem[];
0014: };
0015:
0016: const sections: MenuSection[] = [
0017:   {
0018:     title: "Painel",
0019:     items: [{ to: "/dashboard", label: "Dashboard", code: "DB" }],
0020:   },
0021:   {
0022:     title: "Cadastros",
0023:     items: [
0024:       { to: "/clientes", label: "Clientes", code: "CL" },
0025:       { to: "/fornecedores", label: "Fornecedores", code: "FR" },
0026:       { to: "/produtos", label: "Produtos", code: "PR" },
0027:       { to: "/custos/fornecedor", label: "Custo por Fornecedor", code: "CF" },
0028:       { to: "/parceiros", label: "Parceiros", code: "SA" },
0029:     ],
0030:   },
0031:   {
0032:     title: "Vendas",
0033:     items: [
0034:       { to: "/vendas/pedidos", label: "Pedidos", code: "VD" },
0035:       { to: "/reservas", label: "Reservas", code: "RS" },
0036:     ],
0037:   },
0038:   {
0039:     title: "Financeiro",
0040:     items: [
0041:       { to: "/financeiro/ar", label: "Contas a Receber", code: "AR" },
0042:       { to: "/financeiro/ap", label: "Contas a Pagar", code: "AP" },
0043:       { to: "/financeiro/caixa", label: "Caixa", code: "CX" },
0044:       { to: "/financeiro/categorias", label: "Categorias", code: "CT" },
0045:     ],
0046:   },
0047:   {
0048:     title: "Relatorios",
0049:     items: [
0050:       { to: "/relatorios/dre", label: "DRE", code: "DR" },
0051:       { to: "/relatorios/margem-produto", label: "Margem por Produto", code: "MG" },
0052:     ],
0053:   },
0054:   {
0055:     title: "Administracao",
0056:     items: [
0057:       { to: "/auditoria", label: "Auditoria", code: "AU" },
0058:       { to: "/usuarios", label: "Usuarios", code: "US" },
0059:     ],
0060:   },
0061: ];
0062:
0063: function linkClass(isActive: boolean) {
0064:   return `nav-item ${isActive ? "active" : ""}`;
0065: }
0066:
```

```
0067: export function Sidebar() {
0068:   const [area, setArea] = useState(getArea());
0069:
0070:   useEffect(() => onAreaChange(setArea), []);
0071:
0072:   return (
0073:     <aside className="sidebar-shell">
0074:       <div className="sidebar-brand">
0075:         <div className="brand-mark">MP</div>
0076:         <div>
0077:           <div className="brand-title">Mini Protheus</div>
0078:           <div className="brand-subtitle">ERP operacional</div>
0079:         </div>
0080:       </div>
0081:
0082:       <div className="profile-card">
0083:         <div className="profile-avatar">UQ</div>
0084:         <div className="profile-meta">
0085:           <div className="profile-name">Sessao Ativa</div>
0086:           <div className="profile-role">{area}</div>
0087:         </div>
0088:       </div>
0089:
0090:       <div className="sidebar-scroll">
0091:         {sections.map((section) => (
0092:           <section key={section.title} className="nav-section">
0093:             <div className="nav-section-title">{section.title}</div>
0094:             <div className="nav-list">
0095:               {section.items.map((item) => (
0096:                 <NavLink key={item.to} to={item.to} className={({ isActive }) => linkClass(isA
ctive)}>
0097:                   <span className="nav-code">{item.code}</span>
0098:                   <span className="nav-label">{item.label}</span>
0099:                 </NavLink>
0100:               ))}
0101:             </div>
0102:           </section>
0103:         ))}
0104:       </div>
0105:
0106:       <div className="sidebar-footer">
0107:         <div className="sidebar-footer-label">Ambiente</div>
0108:         <div className="sidebar-footer-value">{area}</div>
0109:       </div>
0110:     </aside>
0111:   );
0112: }
```

==========
ARQUIVO: frontend/src/layout/Topbar.tsx
==========

```
0001: import { useEffect, useState } from "react";
0002: import { useLocation, useNavigate } from "react-router-dom";
0003: import { useAuth } from "../auth/AuthContext";
```

```
0004: import { listarAlertas, type BackendAlertItem } from "../api/alertas";
0005: import { getFilial, onFilialChange, setFilial } from "../state/filial";
0006: import { Modal } from "../components/Modal";
0007: import {
0008:   AVAILABLE_AREAS,
0009:   getArea,
0010:   getFavorites,
0011:   onAreaChange,
0012:   onFavoritesChange,
0013:   setArea,
0014:   toggleFavorite,
0015:   type UiArea,
0016: } from "../state/uiPrefs";
0017:
0018: const titles: Record<string, string> = {
0019:   "/dashboard": "Centro de Operacoes",
0020:   "/clientes": "Gestao de Clientes",
0021:   "/fornecedores": "Gestao de Fornecedores",
0022:   "/produtos": "Cadastro de Produtos",
0023:   "/parceiros": "Painel de Parceiros",
0024:   "/pedidos": "Pedidos de Venda",
0025:   "/vendas/pedidos": "Pedidos de Venda",
0026:   "/financeiro/ar": "Contas a Receber",
0027:   "/financeiro/ap": "Contas a Pagar",
0028:   "/financeiro/caixa": "Caixa e Extrato",
0029:   "/financeiro/categorias": "Categorias Financeiras",
0030:   "/relatorios/dre": "Demonstrativo DRE",
0031:   "/relatorios/margem-produto": "Margem por Produto",
0032:   "/reservas": "Reservas de Estoque",
0033:   "/auditoria": "Auditoria Operacional",
0034:   "/usuarios": "Usuarios e Bootstrap",
0035: };
0036:
0037: const routeCatalog = [
0038:   { path: "/dashboard", label: "Dashboard", category: "Painel" },
0039:   { path: "/clientes", label: "Clientes", category: "Cadastros" },
0040:   { path: "/fornecedores", label: "Fornecedores", category: "Cadastros" },
0041:   { path: "/produtos", label: "Produtos", category: "Cadastros" },
0042:   { path: "/parceiros", label: "Parceiros", category: "Cadastros" },
0043:   { path: "/vendas/pedidos", label: "Pedidos", category: "Vendas" },
0044:   { path: "/reservas", label: "Reservas", category: "Vendas" },
0045:   { path: "/financeiro/ar", label: "Contas a Receber", category: "Financeiro" },
0046:   { path: "/financeiro/ap", label: "Contas a Pagar", category: "Financeiro" },
0047:   { path: "/financeiro/caixa", label: "Caixa", category: "Financeiro" },
0048:   { path: "/financeiro/categorias", label: "Categorias", category: "Financeiro" },
0049:   { path: "/relatorios/dre", label: "DRE", category: "Relatorios" },
0050:   { path: "/relatorios/margem-produto", label: "Margem por Produto", category: "Relatorios" },
0051:   { path: "/auditoria", label: "Auditoria", category: "Administracao" },
0052:   { path: "/usuarios", label: "Usuarios", category: "Administracao" },
0053: ];
0054:
0055: type PanelKey = "favorites" | "alerts" | "help" | "area" | null;
0056:
0057: function resolveAlertRoute(alert: BackendAlertItem) {
```

```
0058:   const contextToRoute: Record<string, string> = {
0059:     "financeiro.ar.overdue": "/financeiro/ar?alert=ar_overdue",
0060:     "financeiro.ar.due_today": "/financeiro/ar?alert=ar_due_today",
0061:     "financeiro.ar.due_soon": "/financeiro/ar?alert=ar_due_soon",
0062:     "financeiro.ap.overdue": "/financeiro/ap?alert=ap_overdue",
0063:     "financeiro.ap.due_today": "/financeiro/ap?alert=ap_due_today",
0064:     "financeiro.ap.due_soon": "/financeiro/ap?alert=ap_due_soon",
0065:     "vendas.pedidos.sem_itens": "/vendas/pedidos?alert=pedidos_no_items",
0066:     "vendas.pedidos.abertos": "/vendas/pedidos?alert=pedidos_open",
0067:   };
0068:
0069:   if (alert.business_context && contextToRoute[alert.business_context]) {
0070:     return contextToRoute[alert.business_context];
0071:   }
0072:   return alert.route || "/dashboard";
0073: }
0074:
0075: export function Topbar() {
0076:   const location = useLocation();
0077:   const navigate = useNavigate();
0078:   const { logout } = useAuth();
0079:
0080:   const [filial, setFilialState] = useState(getFilial());
0081:   const [shortcut, setShortcut] = useState("");
0082:   const [area, setAreaState] = useState<UiArea>(getArea());
0083:   const [favorites, setFavorites] = useState<string[]>(getFavorites());
0084:   const [panel, setPanel] = useState<PanelKey>(null);
0085:   const [alertsLoading, setAlertsLoading] = useState(false);
0086:   const [alertsError, setAlertsError] = useState<string | null>(null);
0087:   const [alertsData, setAlertsData] = useState<BackendAlertItem[]>([]);
0088:
0089:   useEffect(() => onFilialChange(setFilialState), []);
0090:   useEffect(() => onAreaChange(setAreaState), []);
0091:   useEffect(() => onFavoritesChange(setFavorites), []);
0092:
0093:   const title = titles[location.pathname] ?? "Centro de Operacoes";
0094:   const currentRoute = routeCatalog.find((item) => item.path === location.pathname);
0095:   const isFavorite = favorites.includes(location.pathname);
0096:
0097:   const favoriteRoutes = routeCatalog.filter((item) => favorites.includes(item.path));
0098:
0099:   async function refreshAlerts(targetFilial: string) {
0100:     setAlertsLoading(true);
0101:     setAlertsError(null);
0102:     try {
0103:       const response = await listarAlertas({ filial: targetFilial });
0104:       setAlertsData(response.items);
0105:     } catch (error: unknown) {
0106:       const message = error instanceof Error ? error.message : "Erro ao carregar alertas";
0107:       setAlertsError(message);
0108:       setAlertsData([]);
0109:     } finally {
0110:       setAlertsLoading(false);
0111:     }
```

```
0112:   }
0113:
0114:   function openRoute(path: string) {
0115:     navigate(path);
0116:     setPanel(null);
0117:   }
0118:
0119:   function onShortcutSubmit() {
0120:     const query = shortcut.trim().toLowerCase();
0121:     if (!query) return;
0122:     const match = routeCatalog.find(
0123:       (item) =>
0124:         item.label.toLowerCase().includes(query) ||
0125:         item.category.toLowerCase().includes(query) ||
0126:         item.path.toLowerCase().includes(query)
0127:     );
0128:     if (match) {
0129:       navigate(match.path);
0130:       setShortcut("");
0131:     }
0132:   }
0133:
0134:   return (
0135:     <>
0136:       <header className="topbar-shell">
0137:         <div className="topbar-title-block">
0138:           <div className="topbar-eyebrow">Mini Protheus ERP</div>
0139:           <div className="topbar-title">{title}</div>
0140:         </div>
0141:
0142:         <div className="topbar-search">
0143:           <input
0144:             className="shell-input"
0145:             value={shortcut}
0146:             onChange={(event) => setShortcut(event.target.value)}
0147:             onKeyDown={(event) => {
0148:               if (event.key === "Enter") onShortcutSubmit();
0149:             }}
0150:             placeholder="Buscar modulo, fluxo ou acao rapida"
0151:             aria-label="Atalho rapido"
0152:           />
0153:         </div>
0154:
0155:         <div className="topbar-actions">
0156:           <button type="button" className="icon-action" aria-label="Favoritos" onClick={() =>
setPanel("favorites")}>
0157:             Favoritos
0158:           </button>
0159:           <button
0160:             type="button"
0161:             className="icon-action"
0162:             aria-label="Alertas"
0163:             onClick={() => {
0164:               setPanel("alerts");
```

```
0165:               void refreshAlerts(filial);
0166:             }}
0167:           >
0168:             Alertas
0169:           </button>
0170:           <button type="button" className="icon-action" aria-label="Ajuda" onClick={() => setP
anel("help")}>
0171:             Ajuda
0172:           </button>
0173:
0174:           <button type="button" className="topbar-pill topbar-pill-button" onClick={() => setP
anel("area")}>
0175:             {area}
0176:           </button>
0177:
0178:           <label className="filial-picker">
0179:             <span>Filial</span>
0180:             <select
0181:               className="shell-select"
0182:               value={filial}
0183:               onChange={(event) => {
0184:                 const nextFilial = event.target.value;
0185:                 setFilial(nextFilial);
0186:                 if (panel === "alerts") {
0187:                   void refreshAlerts(nextFilial);
0188:                 }
0189:               }}
0190:               aria-label="Selecionar filial"
0191:             >
0192:               <option value="01">01</option>
0193:               <option value="02">02</option>
0194:               <option value="03">03</option>
0195:             </select>
0196:           </label>
0197:
0198:           <button
0199:             type="button"
0200:             className="btn shell-logout"
0201:             onClick={() => {
0202:               logout();
0203:               navigate("/login");
0204:             }}
0205:           >
0206:             Sair
0207:           </button>
0208:         </div>
0209:       </header>
0210:
0211:       {panel === "favorites" && (
0212:         <Modal title="Favoritos" onClose={() => setPanel(null)}>
0213:           <div className="panel-stack">
0214:             <div className="topbar-panel-box">
0215:               <div className="topbar-panel-title">Pagina atual</div>
0216:               <div className="topbar-panel-text">
```

```
0217:                 {currentRoute ? `${currentRoute.label} (${currentRoute.category})` : location.
pathname}
0218:               </div>
0219:               <div className="panel-actions">
0220:                 <button type="button" className="btn primary" onClick={() => toggleFavorite(lo
cation.pathname)}>
0221:                   {isFavorite ? "Remover dos favoritos" : "Adicionar aos favoritos"}
0222:                 </button>
0223:               </div>
0224:             </div>
0225:
0226:             <div className="topbar-panel-box">
0227:               <div className="topbar-panel-title">Rotas favoritas</div>
0228:               {favoriteRoutes.length === 0 ? (
0229:                 <div className="topbar-panel-empty">Nenhum favorito salvo no navegador.</div>
0230:               ) : (
0231:                 <div className="panel-route-list">
0232:                   {favoriteRoutes.map((item) => (
0233:                     <button key={item.path} type="button" className="route-chip" onClick={() =
> openRoute(item.path)}>
0234:                       <span>{item.label}</span>
0235:                       <small>{item.category}</small>
0236:                     </button>
0237:                   ))}
0238:                 </div>
0239:               )}
0240:             </div>
0241:           </div>
0242:         </Modal>
0243:       )}
0244:
0245:       {panel === "alerts" && (
0246:         <Modal title="Alertas" onClose={() => setPanel(null)}>
0247:           <div className="panel-stack">
0248:             <div className="topbar-panel-box">
0249:               <div className="topbar-panel-title">Contexto monitorado</div>
0250:               <div className="topbar-panel-text">Filial {filial}. Area visual {area}. Alertas
guiados por regra de negocio.</div>
0251:             </div>
0252:
0253:             {alertsLoading && <div className="topbar-panel-empty">Carregando alertas operacion
ais...</div>}
0254:
0255:             {alertsError && <div className="topbar-alert-card warning">{alertsError}</div>}
0256:
0257:             {!alertsLoading && !alertsError && alertsData.length === 0 && (
0258:               <div className="topbar-panel-empty">Nenhum alerta ativo retornado pelo backend.<
/div>
0259:             )}
0260:
0261:             {alertsData.map((alert) => (
0262:               <div key={`${alert.code}-${alert.title}`} className={`topbar-alert-card ${alert.
severity}`}>
0263:                 <div className="topbar-panel-title">{alert.title}</div>
```

```
0264:                 <div className="topbar-panel-text">{alert.text}</div>
0265:                 {alert.business_context && <div className="topbar-panel-text"><small>{alert.bu
siness_context}</small></div>}
0266:                 {alert.route && (
0267:                   <div className="panel-actions">
0268:                     <button type="button" className="btn" onClick={() => openRoute(resolveAler
tRoute(alert))}>
0269:                       {alert.action_label || "Abrir modulo"}
0270:                     </button>
0271:                   </div>
0272:                 )}
0273:               </div>
0274:             ))}
0275:           </div>
0276:         </Modal>
0277:       )}
0278:
0279:       {panel === "help" && (
0280:         <Modal title="Ajuda" onClose={() => setPanel(null)}>
0281:           <div className="panel-stack">
0282:             <div className="topbar-panel-box">
0283:               <div className="topbar-panel-title">Navegacao rapida</div>
0284:               <div className="topbar-panel-text">
0285:                 Use o campo de busca do topo e pressione Enter para abrir um modulo pelo nome
ou caminho.
0286:               </div>
0287:             </div>
0288:
0289:             <div className="topbar-panel-box">
0290:               <div className="topbar-panel-title">Atalhos uteis</div>
0291:               <ul className="help-list">
0292:                 <li>`Favoritos`: salva a rota atual no navegador para acesso rapido.</li>
0293:                 <li>`Alertas`: mostra lembretes operacionais e contexto da sessao.</li>
0294:                 <li>`Area`: troca o contexto visual padrao da sessao.</li>
0295:                 <li>`Filial`: muda o contexto usado pelas telas que respeitam filial.</li>
0296:               </ul>
0297:             </div>
0298:
0299:             <div className="topbar-panel-box">
0300:               <div className="topbar-panel-title">Modulos principais</div>
0301:               <div className="panel-route-list">
0302:                 {routeCatalog.slice(0, 8).map((item) => (
0303:                   <button key={item.path} type="button" className="route-chip" onClick={() =>
openRoute(item.path)}>
0304:                     <span>{item.label}</span>
0305:                     <small>{item.category}</small>
0306:                   </button>
0307:                 ))}
0308:               </div>
0309:             </div>
0310:           </div>
0311:         </Modal>
0312:       )}
0313:
```

```
0314:       {panel === "area" && (
0315:         <Modal title="Area padrao" onClose={() => setPanel(null)}>
0316:           <div className="panel-stack">
0317:             <div className="topbar-panel-box">
0318:               <div className="topbar-panel-title">Contexto atual</div>
0319:               <div className="topbar-panel-text">
0320:                 A area selecionada ajusta o contexto visual da sessao no frontend.
0321:               </div>
0322:             </div>
0323:
0324:             <div className="panel-route-list">
0325:               {AVAILABLE_AREAS.map((item) => (
0326:                 <button
0327:                   key={item}
0328:                   type="button"
0329:                   className={`route-chip ${item === area ? "active" : ""}`}
0330:                   onClick={() => {
0331:                     setArea(item);
0332:                     setPanel(null);
0333:                   }}
0334:                 >
0335:                   <span>{item}</span>
0336:                   <small>{item === area ? "Selecionada" : "Selecionar"}</small>
0337:                 </button>
0338:               ))}
0339:             </div>
0340:           </div>
0341:         </Modal>
0342:       )}
0343:     </>
0344:   );
0345: }
```

==========
ARQUIVO: frontend/src/main.tsx
==========

```
0001: import { StrictMode } from "react";
0002: import { createRoot } from "react-dom/client";
0003: import "./index.css";        // mínimo (sem tema do Vite)
0004: import "./styles/app.css";   // seu tema do ERP
0005: import App from "./App";
0006:
0007: createRoot(document.getElementById("root")!).render(
0008:   <StrictMode>
0009:     <App />
0010:   </StrictMode>
0011: );
```

==========
ARQUIVO: frontend/src/pages/auditoriaList.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { Toolbar } from "../components/Toolbar";
0003: import { DataTable, type Column, type SortDir } from "../components/DataTable";
```

```
0004: import { Pager } from "../components/Pager";
0005: import { PageHero } from "../components/PageHero";
0006: import { listarAuditoria, type LogItem } from "../api/auditoria";
0007: import usersIcon from "../assets/shell-icons/users.png";
0008: import auditoriaBg from "../assets/hero-backgrounds/auditoria.jpg";
0009:
0010: function toStringValue(value: unknown, fallback = ""): string {
0011:   if (value === null || value === undefined) return fallback;
0012:   const s = String(value).trim();
0013:   return s ? s : fallback;
0014: }
0015:
0016: function normalize(v: unknown) {
0017:   if (v === null || v === undefined) return "";
0018:   if (typeof v === "boolean") return v ? "1" : "0";
0019:   if (typeof v === "number") return String(v).padStart(20, "0");
0020:   return String(v).toLowerCase().trim();
0021: }
0022:
0023: function getErrorMessage(error: unknown, fallback: string): string {
0024:   if (error instanceof Error && error.message) return error.message;
0025:   return fallback;
0026: }
0027:
0028: export function AuditoriaList() {
0029:   const [q, setQ] = useState("");
0030:   const [loading, setLoading] = useState(false);
0031:   const [err, setErr] = useState<string | null>(null);
0032:   const [items, setItems] = useState<LogItem[]>([]);
0033:
0034:   const [page, setPage] = useState(1);
0035:   const [pageSize, setPageSize] = useState(10);
0036:
0037:   const [sortKey, setSortKey] = useState("id");
0038:   const [sortDir, setSortDir] = useState<SortDir>("desc");
0039:
0040:   async function load() {
0041:     setErr(null);
0042:     setLoading(true);
0043:     try {
0044:       const data = await listarAuditoria();
0045:       setItems(data);
0046:     } catch (error: unknown) {
0047:       setErr(getErrorMessage(error, "Erro ao carregar auditoria"));
0048:     } finally {
0049:       setLoading(false);
0050:     }
0051:   }
0052:
0053:   useEffect(() => {
0054:     load();
0055:   }, []);
0056:
0057:   const filtered = useMemo(() => {
```

```
0058:     const query = q.trim().toLowerCase();
0059:     if (!query) return items;
0060:     return items.filter((x) => {
0061:       const usuario = toStringValue(x.L_USUARIO, "");
0062:       const acao = toStringValue(x.L_ACAO, "");
0063:       const data = toStringValue(x.L_DATA, "");
0064:       const id = String(x.ID ?? "");
0065:       return [usuario, acao, data, id].some((v) => v.toLowerCase().includes(query));
0066:     });
0067:   }, [items, q]);
0068:
0069:   const columns: Column<LogItem>[] = useMemo(
0070:     () => [
0071:       { key: "id", header: "ID", sortable: true, value: (x) => x.ID ?? 0 },
0072:       { key: "usuario", header: "Usuario", sortable: true, value: (x) => toStringValue(x.L_USU
ARIO, "-") },
0073:       { key: "acao", header: "Acao", sortable: true, value: (x) => toStringValue(x.L_ACAO, "-"
) },
0074:       { key: "data", header: "Data", sortable: true, value: (x) => toStringValue(x.L_DATA, "-"
), className: "hide-sm" },
0075:     ],
0076:     []
0077:   );
0078:
0079:   const sorted = useMemo(() => {
0080:     const col = columns.find((c) => c.key === sortKey);
0081:     const valueFn = col?.value ?? ((r: LogItem) => (r as Record<string, unknown>)[sortKey]);
0082:     const arr = [...filtered];
0083:     arr.sort((a, b) => {
0084:       const av = normalize(valueFn(a));
0085:       const bv = normalize(valueFn(b));
0086:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0087:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0088:       return 0;
0089:     });
0090:     return arr;
0091:   }, [filtered, columns, sortKey, sortDir]);
0092:
0093:   const totalPages = useMemo(
0094:     () => Math.max(1, Math.ceil(sorted.length / pageSize)),
0095:     [sorted.length, pageSize]
0096:   );
0097:
0098:   const pagedRows = useMemo(() => {
0099:     const safePage = Math.min(Math.max(1, page), totalPages);
0100:     const start = (safePage - 1) * pageSize;
0101:     return sorted.slice(start, start + pageSize);
0102:   }, [sorted, page, pageSize, totalPages]);
0103:
0104:   return (
0105:     <>
0106:       <PageHero
0107:         icon={usersIcon}
0108:         backgroundImage={auditoriaBg}
```

```
0109:         backgroundPosition="center 26%"
0110:         eyebrow="Governanca"
0111:         title="Auditoria operacional"
0112:         description="Rastreie usuario, acao e data das operacoes expostas pelo backend para le
itura administrativa."
0113:         metrics={[
0114:           { label: "Registros", value: String(sorted.length) },
0115:           { label: "Busca", value: q ? "Ativa" : "Nenhuma" },
0116:           { label: "Fonte", value: "/auditoria" },
0117:         ]}
0118:       />
0119:
0120:       <div className="card">
0121:       <Toolbar
0122:         left={<div className="hint">Auditoria (ultimos 200 registros)</div>}
0123:         right={
0124:           <>
0125:             <input
0126:               className="input minw-240"
0127:               value={q}
0128:               onChange={(e) => setQ(e.target.value)}
0129:               placeholder="Buscar por usuario, acao..."
0130:             />
0131:             <button className="btn" onClick={load} disabled={loading}>
0132:               Atualizar
0133:             </button>
0134:             <button
0135:               className="btn ghost"
0136:               onClick={() => setQ("")}
0137:               disabled={loading}
0138:             >
0139:               Limpar
0140:             </button>
0141:           </>
0142:         }
0143:       />
0144:
0145:       {err && <div className="alert mt-12">{err}</div>}
0146:
0147:       <div className="section-title">Resumo</div>
0148:       <div className="stats-grid">
0149:         <div className="stat-card">
0150:           <div>
0151:             <div className="kpi-label">Registros (filtrados)</div>
0152:             <div className="kpi">{sorted.length}</div>
0153:           </div>
0154:           <div className="stat-icon">LOG</div>
0155:         </div>
0156:         <div className="stat-card">
0157:           <div>
0158:             <div className="kpi-label">Busca</div>
0159:             <div className="kpi">{q ? "Ativa" : "Nenhuma"}</div>
0160:           </div>
0161:           <div className="stat-icon solid">SR</div>
```

```
0162:         </div>
0163:         <div className="stat-card">
0164:           <div>
0165:             <div className="kpi-label">Fonte</div>
0166:             <div className="kpi">/auditoria</div>
0167:           </div>
0168:           <div className="stat-icon success">API</div>
0169:         </div>
0170:       </div>
0171:
0172:       <div className="row-between mt-12">
0173:         <Pager
0174:           page={page}
0175:           totalPages={totalPages}
0176:           pageSize={pageSize}
0177:           totalItems={sorted.length}
0178:           onPageChange={setPage}
0179:           onPageSizeChange={(ps) => {
0180:             setPageSize(ps);
0181:             setPage(1);
0182:           }}
0183:           showPageSize
0184:         />
0185:       </div>
0186:
0187:       <DataTable
0188:         rows={pagedRows}
0189:         columns={columns}
0190:         sortKey={sortKey}
0191:         sortDir={sortDir}
0192:         className="compact"
0193:         wrapperClassName="table-scroller"
0194:         onSortChange={(k, d) => {
0195:           setSortKey(k);
0196:           setSortDir(d);
0197:           setPage(1);
0198:         }}
0199:         loading={loading}
0200:         emptyText="Nenhum registro encontrado."
0201:       />
0202:       </div>
0203:     </>
0204:   );
0205: }
```

==========
ARQUIVO: frontend/src/pages/clientsList.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { Toolbar } from "../components/Toolbar";
0003: import { Modal } from "../components/Modal";
0004: import { ConfirmDialog } from "../components/ConfirmDialog";
0005: import { Pager } from "../components/Pager";
0006: import { DataTable, type Column, type SortDir } from "../components/DataTable";
```

```
0007: import { PageHero } from "../components/PageHero";
0008: import { getFilial, onFilialChange } from "../state/filial";
0009: import clientsIcon from "../assets/shell-icons/clients.png";
0010: import clientsBg from "../assets/hero-backgrounds/clients.jpg";
0011: import {
0012:   listClientes,
0013:   buscarClientes,
0014:   createCliente,
0015:   updateCliente,
0016:   setClienteAtivo,
0017:   type Cliente,
0018: } from "../api/clients";
0019:
0020: type FormState = {
0021:   cod: string;
0022:   nome: string;
0023:   doc: string;
0024:   email: string;
0025:   fone: string;
0026:   filial: string;
0027: };
0028:
0029: function emptyForm(): FormState {
0030:   return {
0031:     cod: "",
0032:     nome: "",
0033:     doc: "",
0034:     email: "",
0035:     fone: "",
0036:     filial: getFilial(),
0037:   };
0038: }
0039:
0040: type SortKey = "cod" | "nome" | "doc" | "ativo_sn";
0041:
0042: function getErrorMessage(error: unknown, fallback: string): string {
0043:   if (error instanceof Error && error.message) return error.message;
0044:   return fallback;
0045: }
0046:
0047: function norm(v: unknown) {
0048:   return (v ?? "").toString().toLowerCase().trim();
0049: }
0050:
0051: function getSortValue(item: Cliente, key: SortKey): unknown {
0052:   if (key === "ativo_sn") return item.ativo_sn ?? "S";
0053:   if (key === "cod") return item.cod;
0054:   if (key === "nome") return item.nome;
0055:   return item.doc;
0056: }
0057:
0058: export function ClientsList() {
0059:   const [filialView, setFilialView] = useState(getFilial());
0060:   const [ativos, setAtivos] = useState(true);
```

```
0061:   const [q, setQ] = useState("");
0062:
0063:   const [loading, setLoading] = useState(false);
0064:   const [err, setErr] = useState<string | null>(null);
0065:   const [items, setItems] = useState<Cliente[]>([]);
0066:
0067:   const [page, setPage] = useState(1);
0068:   const [pageSize, setPageSize] = useState(10);
0069:
0070:   const [sortKey, setSortKey] = useState<SortKey>("nome");
0071:   const [sortDir, setSortDir] = useState<SortDir>("asc");
0072:
0073:   const [modalOpen, setModalOpen] = useState(false);
0074:   const [editingCod, setEditingCod] = useState<string | null>(null);
0075:   const [form, setForm] = useState<FormState>(() => emptyForm());
0076:
0077:   const [confirmOpen, setConfirmOpen] = useState(false);
0078:   const [confirmItem, setConfirmItem] = useState<Cliente | null>(null);
0079:
0080:   const modalTitle = useMemo(() => (editingCod ? `Editar: ${editingCod}` : "Novo"), [editingCo
d]);
0081:
0082:   function askToggleAtivo(c: Cliente) {
0083:     setConfirmItem(c);
0084:     setConfirmOpen(true);
0085:   }
0086:
0087:   async function load() {
0088:     setErr(null);
0089:     setLoading(true);
0090:     try {
0091:       const filial = getFilial();
0092:       const data = await listClientes({ ativos, filial });
0093:       setItems(data);
0094:       setPage(1);
0095:     } catch (error: unknown) {
0096:       setErr(getErrorMessage(error, "Erro ao carregar"));
0097:     } finally {
0098:       setLoading(false);
0099:     }
0100:   }
0101:
0102:   async function doSearch() {
0103:     setErr(null);
0104:     setLoading(true);
0105:     try {
0106:       if (!q.trim()) {
0107:         await load();
0108:         return;
0109:       }
0110:       const filial = getFilial();
0111:       const data = await buscarClientes({ q: q.trim(), filial });
0112:       setItems(data);
0113:       setPage(1);
```

```
0114:     } catch (error: unknown) {
0115:       setErr(getErrorMessage(error, "Erro na busca"));
0116:     } finally {
0117:       setLoading(false);
0118:     }
0119:   }
0120:
0121:   useEffect(() => {
0122:     load();
0123:
0124:     const unsub = onFilialChange((novaFilial) => {
0125:       setFilialView(novaFilial);
0126:       setQ("");
0127:       setErr(null);
0128:       setModalOpen(false);
0129:       setEditingCod(null);
0130:       setForm(emptyForm());
0131:       setConfirmOpen(false);
0132:       setConfirmItem(null);
0133:       setPage(1);
0134:       load();
0135:     });
0136:
0137:     return unsub;
0138:     // eslint-disable-next-line react-hooks/exhaustive-deps
0139:   }, []);
0140:
0141:   function openNew() {
0142:     setEditingCod(null);
0143:     setForm(emptyForm());
0144:     setModalOpen(true);
0145:   }
0146:
0147:   function openEdit(c: Cliente) {
0148:     setEditingCod(c.cod);
0149:     setForm({
0150:       cod: c.cod,
0151:       nome: c.nome ?? "",
0152:       doc: c.doc ?? "",
0153:       email: c.email ?? "",
0154:       fone: c.fone ?? "",
0155:       filial: c.filial ?? getFilial(),
0156:     });
0157:     setModalOpen(true);
0158:   }
0159:
0160:   async function save() {
0161:     setErr(null);
0162:     setLoading(true);
0163:     try {
0164:       if (!form.cod.trim() || !form.nome.trim()) {
0165:         throw new Error("Codigo e Nome sao obrigatorios.");
0166:       }
0167:
```

```
0168:       if (editingCod) {
0169:         await updateCliente(editingCod, {
0170:           filial: form.filial,
0171:           nome: form.nome,
0172:           doc: form.doc || null,
0173:           email: form.email || null,
0174:           fone: form.fone || null,
0175:         });
0176:       } else {
0177:         await createCliente({
0178:           filial: form.filial,
0179:           cod: form.cod,
0180:           nome: form.nome,
0181:           doc: form.doc || null,
0182:           email: form.email || null,
0183:           fone: form.fone || null,
0184:         });
0185:       }
0186:
0187:       setModalOpen(false);
0188:       await load();
0189:     } catch (error: unknown) {
0190:       setErr(getErrorMessage(error, "Erro ao salvar"));
0191:     } finally {
0192:       setLoading(false);
0193:     }
0194:   }
0195:
0196:   async function toggleAtivo(c: Cliente) {
0197:     setErr(null);
0198:     setLoading(true);
0199:     try {
0200:       const ativoAtual = (c.ativo_sn ?? "S") === "S";
0201:       await setClienteAtivo(c.cod, { ativo: !ativoAtual, filial: c.filial ?? getFilial() });
0202:       await load();
0203:     } catch (error: unknown) {
0204:       setErr(getErrorMessage(error, "Erro ao alterar ativo"));
0205:     } finally {
0206:       setLoading(false);
0207:     }
0208:   }
0209:
0210:   const sortedItems = useMemo(() => {
0211:     const arr = [...items];
0212:     arr.sort((a, b) => {
0213:       const av = norm(getSortValue(a, sortKey));
0214:       const bv = norm(getSortValue(b, sortKey));
0215:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0216:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0217:       return norm(a.cod).localeCompare(norm(b.cod));
0218:     });
0219:     return arr;
0220:   }, [items, sortKey, sortDir]);
0221:
```

```
0222:   const totalPages = useMemo(
0223:     () => Math.max(1, Math.ceil(sortedItems.length / pageSize)),
0224:     [sortedItems.length, pageSize]
0225:   );
0226:
0227:   const pagedRows = useMemo(() => {
0228:     const safePage = Math.min(Math.max(1, page), totalPages);
0229:     const start = (safePage - 1) * pageSize;
0230:     return sortedItems.slice(start, start + pageSize);
0231:   }, [sortedItems, page, pageSize, totalPages]);
0232:
0233:   const columns: Column<Cliente>[] = useMemo(
0234:     () => [
0235:       { key: "cod", header: "Codigo", sortable: true },
0236:       { key: "nome", header: "Nome", sortable: true },
0237:       { key: "doc", header: "Doc", sortable: true, className: "hide-sm" },
0238:       {
0239:         key: "ativo_sn",
0240:         header: "Ativo",
0241:         sortable: true,
0242:         value: (x) => x.ativo_sn ?? "S",
0243:         render: (x) =>
0244:           (x.ativo_sn ?? "S") === "S" ? (
0245:             <span className="badge success"><span className="badge-dot" /> Ativo</span>
0246:           ) : (
0247:             <span className="badge muted"><span className="badge-dot" /> Inativo</span>
0248:           ),
0249:       },
0250:       {
0251:         key: "acoes",
0252:         header: "Acoes",
0253:         width: 240,
0254:         render: (x) => (
0255:           <div className="row gap-10">
0256:             <button className="btn" onClick={() => openEdit(x)}>Editar</button>
0257:             <button
0258:               className={`btn ${(x.ativo_sn ?? "S") === "S" ? "danger" : ""}`}
0259:               onClick={() => askToggleAtivo(x)}
0260:             >
0261:               {(x.ativo_sn ?? "S") === "S" ? "Inativar" : "Ativar"}
0262:             </button>
0263:           </div>
0264:         ),
0265:       },
0266:     ],
0267:     []
0268:   );
0269:
0270:   return (
0271:     <>
0272:       <PageHero
0273:         icon={clientsIcon}
0274:         backgroundImage={clientsBg}
0275:         backgroundPosition="center 38%"
```

```
0276:         eyebrow="Cadastros"
0277:         title="Clientes"
0278:         description="Base comercial de clientes ativos, com busca, manutencao cadastral e cont
role de status por filial."
0279:         metrics={[
0280:           { label: "Filial", value: filialView },
0281:           { label: "Clientes", value: String(sortedItems.length) },
0282:           { label: "Ativos", value: String(sortedItems.filter((c) => (c.ativo_sn ?? "S") === "
S").length) },
0283:         ]}
0284:         actions={<button type="button" className="btn primary" onClick={openNew}>Novo cliente<
/button>}
0285:       />
0286:
0287:       <div className="card">
0288:       <Toolbar
0289:         left={
0290:           <div className="hint">
0291:             Filial: <b>{filialView}</b> - {ativos ? "Mostrando apenas ativos" : "Mostrando tod
os"}
0292:           </div>
0293:         }
0294:         right={
0295:           <>
0296:             <input
0297:               className="input minw-240"
0298:               value={q}
0299:               onChange={(e) => setQ(e.target.value)}
0300:               placeholder="Buscar (codigo, nome, doc...)"
0301:             />
0302:             <button type="button" className="btn" onClick={doSearch} disabled={loading}>
0303:               Buscar
0304:             </button>
0305:
0306:             <select
0307:               className="select"
0308:               value={ativos ? "true" : "false"}
0309:               onChange={(e) => {
0310:                 setAtivos(e.target.value === "true");
0311:                 setPage(1);
0312:               }}
0313:             >
0314:               <option value="true">Ativos</option>
0315:               <option value="false">Todos</option>
0316:             </select>
0317:
0318:             <button type="button" className="btn" onClick={load} disabled={loading}>
0319:               Aplicar
0320:             </button>
0321:             <button
0322:               type="button"
0323:               className="btn ghost"
0324:               onClick={() => {
0325:                 setQ("");
```

```
0326:                 setAtivos(true);
0327:                 setPage(1);
0328:               }}
0329:               disabled={loading}
0330:             >
0331:               Limpar
0332:             </button>
0333:             <button type="button" className="btn primary" onClick={openNew}>
0334:               Novo
0335:             </button>
0336:           </>
0337:         }
0338:       />
0339:
0340:       {err && <div className="alert mt-12">{err}</div>}
0341:
0342:       <div className="section-title">Resumo</div>
0343:       <div className="stats-grid">
0344:         <div className="stat-card">
0345:           <div>
0346:             <div className="kpi-label">Clientes (filtrados)</div>
0347:             <div className="kpi">{sortedItems.length}</div>
0348:           </div>
0349:           <div className="stat-icon">CL</div>
0350:         </div>
0351:         <div className="stat-card">
0352:           <div>
0353:             <div className="kpi-label">Ativos no filtro</div>
0354:             <div className="kpi">
0355:               {sortedItems.filter((c) => (c.ativo_sn ?? "S") === "S").length}
0356:             </div>
0357:           </div>
0358:           <div className="stat-icon success">OK</div>
0359:         </div>
0360:         <div className="stat-card">
0361:           <div>
0362:             <div className="kpi-label">Filial</div>
0363:             <div className="kpi">{filialView}</div>
0364:           </div>
0365:           <div className="stat-icon solid">F</div>
0366:         </div>
0367:       </div>
0368:
0369:       <div className="row-between mt-12">
0370:         <div className="hint">
0371:           <Pager
0372:             page={page}
0373:             totalPages={totalPages}
0374:             pageSize={pageSize}
0375:             totalItems={sortedItems.length}
0376:             onPageChange={setPage}
0377:             onPageSizeChange={(ps) => {
0378:               setPageSize(ps);
0379:               setPage(1);
```

```
0380:             }}
0381:             showPageSize
0382:           />
0383:         </div>
0384:       </div>
0385:
0386:       <DataTable
0387:         rows={pagedRows}
0388:         columns={columns}
0389:         sortKey={sortKey}
0390:         sortDir={sortDir}
0391:         className="compact"
0392:         wrapperClassName="table-scroller"
0393:         onSortChange={(k, d) => {
0394:           setSortKey(k as SortKey);
0395:           setSortDir(d);
0396:           setPage(1);
0397:         }}
0398:         loading={loading}
0399:         emptyText="Nenhum registro encontrado."
0400:       />
0401:
0402:       {modalOpen && (
0403:         <Modal title={modalTitle} onClose={() => setModalOpen(false)}>
0404:           <div className="form-grid">
0405:             <div className="form-field">
0406:               <label>Codigo</label>
0407:               <input
0408:                 value={form.cod}
0409:                 onChange={(e) => setForm({ ...form, cod: e.target.value })}
0410:                 disabled={!!editingCod}
0411:               />
0412:             </div>
0413:
0414:             <div className="form-field">
0415:               <label>Filial</label>
0416:               <input
0417:                 value={form.filial}
0418:                 onChange={(e) => setForm({ ...form, filial: e.target.value })}
0419:               />
0420:             </div>
0421:
0422:             <div className="form-field col-span-all">
0423:               <label>Nome</label>
0424:               <input value={form.nome} onChange={(e) => setForm({ ...form, nome: e.target.valu
e })} />
0425:             </div>
0426:
0427:             <div className="form-field">
0428:               <label>Documento</label>
0429:               <input value={form.doc} onChange={(e) => setForm({ ...form, doc: e.target.value
})} />
0430:             </div>
0431:
```

```
0432:             <div className="form-field">
0433:               <label>Email</label>
0434:               <input value={form.email} onChange={(e) => setForm({ ...form, email: e.target.va
lue })} />
0435:             </div>
0436:
0437:             <div className="form-field">
0438:               <label>Fone</label>
0439:               <input value={form.fone} onChange={(e) => setForm({ ...form, fone: e.target.valu
e })} />
0440:             </div>
0441:           </div>
0442:
0443:           <div className="form-actions">
0444:             <button type="button" className="btn" onClick={() => setModalOpen(false)} disabled
={loading}>
0445:               Cancelar
0446:             </button>
0447:             <button type="button" className="btn primary" onClick={save} disabled={loading}>
0448:               Salvar
0449:             </button>
0450:           </div>
0451:         </Modal>
0452:       )}
0453:
0454:       {confirmOpen && confirmItem && (
0455:         <ConfirmDialog
0456:           title={(confirmItem.ativo_sn ?? "S") === "S" ? "Inativar" : "Ativar"}
0457:           message={
0458:             (confirmItem.ativo_sn ?? "S") === "S"
0459:               ? `Deseja realmente INATIVAR o codigo ${confirmItem.cod}?`
0460:               : `Deseja realmente ATIVAR o codigo ${confirmItem.cod}?`
0461:           }
0462:           confirmText={(confirmItem.ativo_sn ?? "S") === "S" ? "Inativar" : "Ativar"}
0463:           cancelText="Cancelar"
0464:           danger={(confirmItem.ativo_sn ?? "S") === "S"}
0465:           loading={loading}
0466:           onClose={() => {
0467:             setConfirmOpen(false);
0468:             setConfirmItem(null);
0469:           }}
0470:           onConfirm={async () => {
0471:             await toggleAtivo(confirmItem);
0472:             setConfirmOpen(false);
0473:             setConfirmItem(null);
0474:           }}
0475:         />
0476:       )}
0477:       </div>
0478:     </>
0479:   );
0480: }
```

ARQUIVO: frontend/src/pages/Dashboard.tsx
==========

```
0001: import { Link } from "react-router-dom";
0002: import { PageHero } from "../components/PageHero";
0003: import dashboardIcon from "../assets/shell-icons/dashboard-ref.png";
0004: import ordersIcon from "../assets/shell-icons/orders-ref.png";
0005: import receivablesIcon from "../assets/shell-icons/receivables-ref.png";
0006: import payablesIcon from "../assets/shell-icons/payables-ref.png";
0007: import cashIcon from "../assets/shell-icons/cash-ref.png";
0008: import reportsIcon from "../assets/shell-icons/reports-ref.png";
0009: import partnersIcon from "../assets/shell-icons/partners-ref.png";
0010: import usersIcon from "../assets/shell-icons/users-ref.svg";
0011: import inventoryIcon from "../assets/shell-icons/products-ref.png";
0012:
0013: const quickActions = [
0014:   { to: "/vendas/pedidos", label: "Pedidos", subtitle: "Criacao, itens e faturamento", icon: o
rdersIcon },
0015:   { to: "/financeiro/ar", label: "Contas a Receber", subtitle: "Recebimento e baixa", icon: re
ceivablesIcon },
0016:   { to: "/financeiro/ap", label: "Contas a Pagar", subtitle: "Pagamento operacional", icon: pa
yablesIcon },
0017:   { to: "/financeiro/caixa", label: "Caixa", subtitle: "Saldo e extrato", icon: cashIcon },
0018:   { to: "/parceiros", label: "Parceiros", subtitle: "Clientes e fornecedores", icon: partnersI
con },
0019:   { to: "/produtos", label: "Produtos", subtitle: "Cadastro e base comercial", icon: inventory
Icon },
0020:   { to: "/relatorios/dre", label: "Relatorios", subtitle: "DRE e margem", icon: reportsIcon },
0021:   { to: "/usuarios", label: "Usuarios", subtitle: "Bootstrap e perfis", icon: usersIcon },
0022: ];
0023:
0024: export function Dashboard() {
0025:   return (
0026:     <div className="dashboard-stack">
0027:       <PageHero
0028:         icon={dashboardIcon}
0029:         eyebrow="Centro de operacoes"
0030:         title="Painel executivo"
0031:         description="Atalhos operacionais para vendas, financeiro, cadastro e governanca, com
uma linguagem visual mais proxima de ERP corporativo."
0032:         metrics={[
0033:           { label: "Modulos principais", value: "8" },
0034:           { label: "Camadas", value: "API + Vite + SQL" },
0035:           { label: "Foco", value: "Operacao" },
0036:         ]}
0037:         actions={
0038:           <>
0039:             <Link to="/vendas/pedidos" className="btn primary">
0040:               Ir para pedidos
0041:             </Link>
0042:             <Link to="/financeiro/caixa" className="btn">
0043:               Abrir caixa
0044:             </Link>
0045:           </>
0046:         }
```

```
0047:       />
0048:
0049:       <section className="dashboard-grid">
0050:         <div className="card dashboard-block dashboard-block-wide">
0051:           <div className="dashboard-block-glow" />
0052:           <div className="dashboard-block-header">
0053:             <div>
0054:               <div className="dashboard-block-eyebrow">Acesso rapido</div>
0055:               <div className="dashboard-block-title">Modulos operacionais</div>
0056:             </div>
0057:           </div>
0058:
0059:           <div className="quick-grid">
0060:             {quickActions.map((item) => (
0061:               <Link key={item.to} to={item.to} className="quick-card">
0062:                 <div className="quick-card-media">
0063:                   <img src={item.icon} alt="" className="quick-card-icon" />
0064:                 </div>
0065:                 <div className="quick-card-label">{item.label}</div>
0066:                 <div className="quick-card-subtitle">{item.subtitle}</div>
0067:               </Link>
0068:             ))}
0069:           </div>
0070:         </div>
0071:
0072:         <div className="card dashboard-block">
0073:           <div className="dashboard-block-glow dashboard-block-glow-secondary" />
0074:           <div className="dashboard-block-header">
0075:             <div>
0076:               <div className="dashboard-block-eyebrow">Operacao</div>
0077:               <div className="dashboard-block-title">Leituras de ambiente</div>
0078:             </div>
0079:           </div>
0080:
0081:           <div className="dashboard-summary-grid">
0082:             <article className="summary-tile summary-tile-green">
0083:               <div className="summary-tile-title">Recebiveis</div>
0084:               <div className="summary-tile-value">AR integrado</div>
0085:               <div className="summary-tile-sub">Recebimento e reflexo no caixa</div>
0086:             </article>
0087:
0088:             <article className="summary-tile summary-tile-red">
0089:               <div className="summary-tile-title">Pagamentos</div>
0090:               <div className="summary-tile-value">AP integrado</div>
0091:               <div className="summary-tile-sub">Saida de caixa e baixa controlada</div>
0092:             </article>
0093:
0094:             <article className="summary-tile summary-tile-cyan">
0095:               <div className="summary-tile-title">Comercial</div>
0096:               <div className="summary-tile-value">Pedidos + reserva</div>
0097:               <div className="summary-tile-sub">Fluxo de venda ate o financeiro</div>
0098:             </article>
0099:           </div>
0100:         </div>
```

```
0101:       </section>
0102:     </div>
0103:   );
0104: }
```

==========
ARQUIVO: frontend/src/pages/estoqueEntradaFornecedor.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { useNavigate, useSearchParams } from "react-router-dom";
0003: import { PageHero } from "../components/PageHero";
0004: import { getFilial, onFilialChange } from "../state/filial";
0005: import { listFornecedores, type Fornecedor } from "../api/fornecedores";
0006: import { buscarProdutos, type ProdutoResumo } from "../api/produtos";
0007: import {
0008:   consultarEstoque,
0009:   entradaEstoque,
0010:   extratoEstoque,
0011:   type EstoqueSnapshot,
0012:   type Sd3Movimento,
0013: } from "../api/estoque";
0014: import inventoryIcon from "../assets/shell-icons/inventory.png";
0015: import supplierEntryBg from "../assets/hero-backgrounds/supplier-entry.webp";
0016:
0017: type EntradaForm = {
0018:   cod: string;
0019:   filial: string;
0020:   fornecedorCod: string;
0021:   qtd: string;
0022:   custoUnit: string;
0023:   vencDias: string;
0024: };
0025:
0026: function emptyForm(): EntradaForm {
0027:   return {
0028:     cod: "",
0029:     filial: getFilial(),
0030:     fornecedorCod: "",
0031:     qtd: "1",
0032:     custoUnit: "0",
0033:     vencDias: "30",
0034:   };
0035: }
0036:
0037: function toNumber(value: string, fallback = 0) {
0038:   const n = Number(value);
0039:   return Number.isFinite(n) ? n : fallback;
0040: }
0041:
0042: function formatCurrency(value: unknown) {
0043:   const n = Number(value);
0044:   const safe = Number.isFinite(n) ? n : 0;
0045:   return safe.toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
0046: }
```

```
0047:
0048: function getErrorMessage(error: unknown, fallback: string) {
0049:   if (error instanceof Error && error.message) return error.message;
0050:   return fallback;
0051: }
0052:
0053: function findFornecedorName(items: Fornecedor[], cod: string) {
0054:   const found = items.find((item) => item.cod === cod);
0055:   return found?.nome ?? "Sem fornecedor";
0056: }
0057:
0058: export function EstoqueEntradaFornecedor() {
0059:   const navigate = useNavigate();
0060:   const [searchParams, setSearchParams] = useSearchParams();
0061:   const [form, setForm] = useState<EntradaForm>(emptyForm());
0062:   const [fornecedores, setFornecedores] = useState<Fornecedor[]>([]);
0063:   const [fornecedorQuery, setFornecedorQuery] = useState("");
0064:   const [fornecedorResultados, setFornecedorResultados] = useState<Fornecedor[]>([]);
0065:   const [produtoQuery, setProdutoQuery] = useState("");
0066:   const [produtoResultados, setProdutoResultados] = useState<ProdutoResumo[]>([]);
0067:   const [beforeSnap, setBeforeSnap] = useState<EstoqueSnapshot | null>(null);
0068:   const [afterSnap, setAfterSnap] = useState<EstoqueSnapshot | null>(null);
0069:   const [extrato, setExtrato] = useState<Sd3Movimento[]>([]);
0070:   const [loading, setLoading] = useState(false);
0071:   const [loadingProdutos, setLoadingProdutos] = useState(false);
0072:   const [loadingFornecedores, setLoadingFornecedores] = useState(false);
0073:   const [err, setErr] = useState<string | null>(null);
0074:   const [okMsg, setOkMsg] = useState<string | null>(null);
0075:
0076:   const fornecedorNome = useMemo(
0077:     () => findFornecedorName(fornecedores, form.fornecedorCod),
0078:     [fornecedores, form.fornecedorCod]
0079:   );
0080:
0081:   async function loadFornecedores(filial = getFilial()) {
0082:     try {
0083:       const data = await listFornecedores({ ativos: true, filial });
0084:       setFornecedores(data);
0085:     } catch {
0086:       setFornecedores([]);
0087:     }
0088:   }
0089:
0090:   async function onBuscarProdutos() {
0091:     const query = produtoQuery.trim();
0092:     if (!query) {
0093:       setProdutoResultados([]);
0094:       return;
0095:     }
0096:     setErr(null);
0097:     setLoadingProdutos(true);
0098:     try {
0099:       const data = await buscarProdutos({
0100:         q: query,
```

```
0101:         filial: form.filial || getFilial(),
0102:         limite: 8,
0103:       });
0104:       setProdutoResultados(data);
0105:     } catch (error: unknown) {
0106:       setErr(getErrorMessage(error, "Falha ao buscar produtos"));
0107:       setProdutoResultados([]);
0108:     } finally {
0109:       setLoadingProdutos(false);
0110:     }
0111:   }
0112:
0113:   async function onBuscarFornecedores() {
0114:     const query = fornecedorQuery.trim();
0115:     if (!query) {
0116:       setFornecedorResultados([]);
0117:       return;
0118:     }
0119:     setErr(null);
0120:     setLoadingFornecedores(true);
0121:     try {
0122:       const data = await listFornecedores({
0123:         ativos: true,
0124:         filial: form.filial || getFilial(),
0125:       });
0126:       const normalized = query.toLowerCase();
0127:       const filtered = data.filter((item) => {
0128:         const cod = item.cod.toLowerCase();
0129:         const nome = (item.nome || "").toLowerCase();
0130:         return cod.includes(normalized) || nome.includes(normalized);
0131:       });
0132:       setFornecedorResultados(filtered.slice(0, 8));
0133:     } catch (error: unknown) {
0134:       setErr(getErrorMessage(error, "Falha ao buscar fornecedores"));
0135:       setFornecedorResultados([]);
0136:     } finally {
0137:       setLoadingFornecedores(false);
0138:     }
0139:   }
0140:
0141:   async function refreshProduto(cod = form.cod.trim(), filial = form.filial || getFilial()) {
0142:     if (!cod) return;
0143:     const [snap, movimentos] = await Promise.all([
0144:       consultarEstoque({ cod, filial }),
0145:       extratoEstoque({ cod, filial, limite: 10 }),
0146:     ]);
0147:     setBeforeSnap(snap);
0148:     setExtrato(movimentos);
0149:   }
0150:
0151:   useEffect(() => {
0152:     loadFornecedores();
0153:     const unsub = onFilialChange((novaFilial) => {
0154:       setForm((current) => ({ ...current, filial: novaFilial }));
```

```
0155:       setBeforeSnap(null);
0156:       setAfterSnap(null);
0157:       setExtrato([]);
0158:       setErr(null);
0159:       setOkMsg(null);
0160:       loadFornecedores(novaFilial);
0161:     });
0162:     return unsub;
0163:   }, []);
0164:
0165:   useEffect(() => {
0166:     const fornecedorCod = (searchParams.get("fornecedor") || "").trim().toUpperCase();
0167:     const produtoCod = (searchParams.get("produto") || "").trim().toUpperCase();
0168:     if (!fornecedorCod && !produtoCod) return;
0169:
0170:     setForm((current) => ({
0171:       ...current,
0172:       fornecedorCod: fornecedorCod || current.fornecedorCod,
0173:       cod: produtoCod || current.cod,
0174:     }));
0175:
0176:     if (fornecedorCod) {
0177:       const fornecedor = fornecedores.find((item) => item.cod === fornecedorCod);
0178:       setFornecedorQuery(fornecedor?.nome || fornecedorCod);
0179:     }
0180:     if (produtoCod) {
0181:       setProdutoQuery(produtoCod);
0182:     }
0183:   }, [searchParams, fornecedores]);
0184:
0185:   async function onConsultarProduto() {
0186:     setErr(null);
0187:     setOkMsg(null);
0188:     try {
0189:       await refreshProduto();
0190:     } catch (error: unknown) {
0191:       setBeforeSnap(null);
0192:       setAfterSnap(null);
0193:       setExtrato([]);
0194:       setErr(getErrorMessage(error, "Falha ao consultar produto"));
0195:     }
0196:   }
0197:
0198:   async function onLancarEntrada() {
0199:     setErr(null);
0200:     setOkMsg(null);
0201:     setLoading(true);
0202:     try {
0203:       const cod = form.cod.trim();
0204:       if (!cod) throw new Error("Informe o codigo do produto.");
0205:       const qtd = toNumber(form.qtd, 0);
0206:       const custoUnit = toNumber(form.custoUnit, -1);
0207:       const vencDias = toNumber(form.vencDias, 30);
0208:       if (qtd <= 0) throw new Error("Quantidade deve ser maior que zero.");
```

```
0209:       if (custoUnit < 0) throw new Error("Custo unitario nao pode ser negativo.");
0210:
0211:       const snapAnterior = await consultarEstoque({ cod, filial: form.filial || getFilial() })
;
0212:       setBeforeSnap(snapAnterior);
0213:
0214:       const snapNovo = await entradaEstoque({
0215:         cod,
0216:         qtd,
0217:         filial: form.filial || getFilial(),
0218:         custo_unit: custoUnit,
0219:         forn: form.fornecedorCod.trim() || null,
0220:         venc_dias: vencDias,
0221:       });
0222:       setAfterSnap(snapNovo);
0223:       setOkMsg(
0224:         [
0225:           "Entrada registrada com sucesso.",
0226:           form.fornecedorCod
0227:             ? `Fornecedor ${form.fornecedorCod} vinculado; AP gerado conforme o valor da entra
da.`
0228:             : "Sem fornecedor informado; nenhum AP foi gerado.",
0229:         ].join(" ")
0230:       );
0231:       const movimentos = await extratoEstoque({
0232:         cod,
0233:         filial: form.filial || getFilial(),
0234:         limite: 10,
0235:       });
0236:       setExtrato(movimentos);
0237:     } catch (error: unknown) {
0238:       setErr(getErrorMessage(error, "Falha ao registrar entrada de estoque"));
0239:     } finally {
0240:       setLoading(false);
0241:     }
0242:   }
0243:
0244:   return (
0245:     <>
0246:       <PageHero
0247:         icon={inventoryIcon}
0248:         backgroundImage={supplierEntryBg}
0249:         backgroundPosition="center 46%"
0250:         eyebrow="Custos, Estoque e Compras"
0251:         title="Custo por fornecedor"
0252:         description="Area operacional para formar custo medio por compra. Voce escolhe fornece
dor, produto, quantidade e custo unitario; o sistema registra a entrada, recalcula o B1_CM e pode ge
rar o AP vinculado."
0253:         metrics={[
0254:           { label: "Filial", value: form.filial || getFilial() },
0255:           { label: "Fornecedor", value: form.fornecedorCod ? fornecedorNome : "Nao vinculado"
},
0256:           { label: "Resultado", value: "Atualiza B1_CM" },
0257:         ]}
```

```
0258:       />
0259:
0260:       <div className="card">
0261:         <div className="section-title">Formacao de custo com fornecedor</div>
0262:
0263:         {err ? <div className="alert mt-12">{err}</div> : null}
0264:         {okMsg ? <div className="alert alert-success mt-12">{okMsg}</div> : null}
0265:
0266:         <div className="stats-grid mt-12">
0267:           <div className="stat-card">
0268:             <div>
0269:               <div className="kpi-label">Etapa 1</div>
0270:               <div className="kpi">Fornecedor + Produto</div>
0271:             </div>
0272:             <div className="stat-icon">CF</div>
0273:           </div>
0274:           <div className="stat-card">
0275:             <div>
0276:               <div className="kpi-label">Etapa 2</div>
0277:               <div className="kpi">Qtd + Custo Unitario</div>
0278:             </div>
0279:             <div className="stat-icon solid">CM</div>
0280:           </div>
0281:           <div className="stat-card">
0282:             <div>
0283:               <div className="kpi-label">Etapa 3</div>
0284:               <div className="kpi">Atualiza B1_CM + AP</div>
0285:             </div>
0286:             <div className="stat-icon success">AP</div>
0287:           </div>
0288:         </div>
0289:
0290:         <div className="detail-grid mt-12">
0291:           <div className="card soft">
0292:             <div className="kpi-label">Como usar</div>
0293:             <div className="hint">
0294:               Selecione um fornecedor e um produto, informe quantidade e custo unitario, depoi
s lance
0295:               a entrada. Esse fluxo forma o custo medio do produto a partir da compra.
0296:             </div>
0297:           </div>
0298:           <div className="card soft">
0299:             <div className="kpi-label">O que muda no produto</div>
0300:             <div className="hint">
0301:               O sistema recalcula o <strong>B1_CM</strong> e aumenta o estoque disponivel na f
ilial.
0302:             </div>
0303:           </div>
0304:           <div className="card soft">
0305:             <div className="kpi-label">O que muda no financeiro</div>
0306:             <div className="hint">
0307:               Com fornecedor informado, a entrada pode abrir um titulo em Contas a Pagar para
seguir o
0308:               fluxo operacional completo.
```

```
0309:             </div>
0310:           </div>
0311:         </div>
0312:
0313:         <div className="form-grid mt-12">
0314:           <div className="form-field">
0315:             <label>Buscar fornecedor</label>
0316:             <input
0317:               value={fornecedorQuery}
0318:               onChange={(e) => setFornecedorQuery(e.target.value)}
0319:               placeholder="Codigo ou nome"
0320:             />
0321:           </div>
0322:           <div className="form-field">
0323:             <label> </label>
0324:             <button
0325:               className="btn"
0326:               onClick={onBuscarFornecedores}
0327:               disabled={loading || loadingFornecedores}
0328:             >
0329:               {loadingFornecedores ? "Buscando..." : "Buscar fornecedor"}
0330:             </button>
0331:           </div>
0332:           <div className="form-field">
0333:             <label>Buscar produto</label>
0334:             <input
0335:               value={produtoQuery}
0336:               onChange={(e) => setProdutoQuery(e.target.value)}
0337:               placeholder="Codigo ou descricao"
0338:             />
0339:           </div>
0340:           <div className="form-field">
0341:             <label> </label>
0342:             <button className="btn" onClick={onBuscarProdutos} disabled={loading || loadingPro
dutos}>
0343:               {loadingProdutos ? "Buscando..." : "Buscar produto"}
0344:             </button>
0345:           </div>
0346:           <div className="form-field">
0347:             <label>Produto (codigo)</label>
0348:             <input
0349:               value={form.cod}
0350:               onChange={(e) => setForm({ ...form, cod: e.target.value.toUpperCase() })}
0351:               placeholder="CEL124"
0352:             />
0353:           </div>
0354:           <div className="form-field">
0355:             <label>Fornecedor</label>
0356:             <select
0357:               className="select"
0358:               value={form.fornecedorCod}
0359:               onChange={(e) => setForm({ ...form, fornecedorCod: e.target.value })}
0360:             >
0361:               <option value="">Sem fornecedor (nao gerar AP)</option>
```

```
0362:                {fornecedores.map((item) => (
0363:                  <option key={item.cod} value={item.cod}>
0364:                    {item.cod} - {item.nome}
0365:                  </option>
0366:                ))}
0367:              </select>
0368:            </div>
0369:            <div className="form-field">
0370:              <label>Quantidade</label>
0371:              <input
0372:                type="number"
0373:                min="1"
0374:                value={form.qtd}
0375:                onChange={(e) => setForm({ ...form, qtd: e.target.value })}
0376:              />
0377:            </div>
0378:            <div className="form-field">
0379:              <label>Custo unitario</label>
0380:              <input
0381:                type="number"
0382:                step="0.01"
0383:                min="0"
0384:                value={form.custoUnit}
0385:                onChange={(e) => setForm({ ...form, custoUnit: e.target.value })}
0386:              />
0387:              <div className="hint">Valor de compra usado para recalcular o custo medio do produ
to.</div>
0388:            </div>
0389:            <div className="form-field">
0390:              <label>Filial</label>
0391:              <input
0392:                value={form.filial}
0393:                onChange={(e) => setForm({ ...form, filial: e.target.value })}
0394:              />
0395:            </div>
0396:            <div className="form-field">
0397:              <label>Vencimento AP (dias)</label>
0398:              <input
0399:                type="number"
0400:                min="1"
0401:                value={form.vencDias}
0402:                onChange={(e) => setForm({ ...form, vencDias: e.target.value })}
0403:              />
0404:              <div className="hint">Usado somente quando houver fornecedor e geracao de AP.</div
>
0405:            </div>
0406:            <div className="form-field">
0407:              <label> </label>
0408:              <button className="btn" onClick={onConsultarProduto} disabled={loading}>
0409:                Consultar produto
0410:              </button>
0411:            </div>
0412:            <div className="form-field">
0413:              <label> </label>
```

```
0414:             <button className="btn primary" onClick={onLancarEntrada} disabled={loading}>
0415:               Lançar entrada
0416:             </button>
0417:           </div>
0418:         </div>
0419:
0420:         {fornecedorResultados.length > 0 && (
0421:           <>
0422:             <div className="section-title">Sugestoes de fornecedor</div>
0423:             <div className="table-scroller">
0424:               <table className="table mt-12 compact">
0425:                 <thead>
0426:                   <tr>
0427:                     <th>Codigo</th>
0428:                     <th>Nome</th>
0429:                     <th>Documento</th>
0430:                     <th>Acoes</th>
0431:                   </tr>
0432:                 </thead>
0433:                 <tbody>
0434:                   {fornecedorResultados.map((item) => (
0435:                     <tr key={item.cod}>
0436:                       <td>{item.cod}</td>
0437:                       <td>{item.nome}</td>
0438:                       <td>{item.doc ?? "-"}</td>
0439:                       <td>
0440:                         <button
0441:                           className="btn"
0442:                           onClick={() => {
0443:                             setForm((current) => ({ ...current, fornecedorCod: item.cod }));
0444:                             setFornecedorQuery(item.nome || item.cod);
0445:                             setFornecedorResultados([]);
0446:                           }}
0447:                         >
0448:                           Usar fornecedor
0449:                         </button>
0450:                       </td>
0451:                     </tr>
0452:                   ))}
0453:                 </tbody>
0454:               </table>
0455:             </div>
0456:           </>
0457:         )}
0458:
0459:         {produtoResultados.length > 0 && (
0460:           <>
0461:             <div className="section-title">Sugestoes de produto</div>
0462:             <div className="table-scroller">
0463:               <table className="table mt-12 compact">
0464:                 <thead>
0465:                   <tr>
0466:                     <th>Codigo</th>
0467:                     <th>Descricao</th>
```

```
0468:                     <th>Preco</th>
0469:                     <th>CM</th>
0470:                     <th>Estoque</th>
0471:                     <th>Acoes</th>
0472:                   </tr>
0473:                 </thead>
0474:                 <tbody>
0475:                   {produtoResultados.map((item) => (
0476:                     <tr key={item.cod}>
0477:                       <td>{item.cod}</td>
0478:                       <td>{item.desc}</td>
0479:                       <td>{formatCurrency(item.preco)}</td>
0480:                       <td>{formatCurrency(item.cm)}</td>
0481:                       <td>{item.estoque}</td>
0482:                       <td>
0483:                         <button
0484:                           className="btn"
0485:                           onClick={() => {
0486:                             setForm((current) => ({ ...current, cod: item.cod }));
0487:                             setProdutoQuery(item.desc || item.cod);
0488:                             setProdutoResultados([]);
0489:                           }}
0490:                         >
0491:                           Usar produto
0492:                         </button>
0493:                       </td>
0494:                     </tr>
0495:                   ))}
0496:                 </tbody>
0497:               </table>
0498:             </div>
0499:           </>
0500:         )}
0501:
0502:         {(beforeSnap || afterSnap) && (
0503:           <>
0504:             <div className="section-title">Antes e depois do custo medio</div>
0505:             <div className="detail-grid mt-12">
0506:               <div className="card soft">
0507:                 <div className="kpi-label">Produto</div>
0508:                 <div className="kpi">{(afterSnap ?? beforeSnap)?.B1_COD ?? "-"}</div>
0509:                 <div className="hint">{(afterSnap ?? beforeSnap)?.B1_DESC ?? "-"}</div>
0510:               </div>
0511:               <div className="card soft">
0512:                 <div className="kpi-label">CM antes</div>
0513:                 <div className="kpi">{formatCurrency(beforeSnap?.B1_CM ?? 0)}</div>
0514:                 <div className="hint">Base consultada antes da entrada</div>
0515:               </div>
0516:               <div className="card soft">
0517:                 <div className="kpi-label">CM depois</div>
0518:                 <div className="kpi">{formatCurrency(afterSnap?.B1_CM ?? beforeSnap?.B1_CM ??
0)}</div>
0519:                 <div className="hint">B1_CM atualizado no estoque</div>
0520:               </div>
```

```
0521:                <div className="card soft">
0522:                  <div className="kpi-label">Estoque antes</div>
0523:                  <div className="kpi">{beforeSnap?.B1_ESTOQUE ?? "-"}</div>
0524:                  <div className="hint">Disponivel {beforeSnap?.DISPONIVEL ?? "-"}</div>
0525:                </div>
0526:                <div className="card soft">
0527:                  <div className="kpi-label">Estoque depois</div>
0528:                  <div className="kpi">{afterSnap?.B1_ESTOQUE ?? beforeSnap?.B1_ESTOQUE ?? "-"}<
/div>
0529:                  <div className="hint">Disponivel {afterSnap?.DISPONIVEL ?? "-"}</div>
0530:                </div>
0531:                <div className="card soft">
0532:                <div className="kpi-label">Valor da entrada</div>
0533:                  <div className="kpi">
0534:                    {formatCurrency(toNumber(form.qtd, 0) * toNumber(form.custoUnit, 0))}
0535:                  </div>
0536:                  <div className="hint">
0537:                    {form.fornecedorCod ? "Com AP vinculado ao fornecedor" : "Sem AP automatico"
}
0538:                  </div>
0539:                  {form.fornecedorCod ? (
0540:                    <div className="mt-12">
0541:                      <button
0542:                        className="btn"
0543:                        onClick={() =>
0544:                          navigate(
0545:                            `/financeiro/ap?status=ABERTO&q=${encodeURIComponent(
0546:                              `ENTRADA:${(afterSnap ?? beforeSnap)?.B1_COD ?? form.cod.trim()}`
0547:                            )}`
0548:                          )
0549:                        }
0550:                      >
0551:                        Abrir AP da entrada
0552:                      </button>
0553:                      <button
0554:                        className="btn mt-12"
0555:                        onClick={() => {
0556:                          const next = new URLSearchParams(searchParams);
0557:                          next.set("fornecedor", form.fornecedorCod);
0558:                          next.set("produto", (afterSnap ?? beforeSnap)?.B1_COD ?? form.cod.trim
());
0559:                          setSearchParams(next);
0560:                        }}
0561:                      >
0562:                        Fixar contexto
0563:                      </button>
0564:                    </div>
0565:                  ) : null}
0566:                </div>
0567:              </div>
0568:            </>
0569:          )}
0570:
0571:          {extrato.length > 0 && (
```

```
0572:           <>
0573:             <div className="section-title">Ultimos movimentos SD3 do produto</div>
0574:             <div className="table-scroller">
0575:               <table className="table mt-12 compact">
0576:                 <thead>
0577:                   <tr>
0578:                     <th>ID</th>
0579:                     <th>Data</th>
0580:                     <th>Tipo</th>
0581:                     <th>Qtd</th>
0582:                     <th>Custo Unit</th>
0583:                     <th>Valor</th>
0584:                     <th>Origem</th>
0585:                     <th>Usuario</th>
0586:                   </tr>
0587:                 </thead>
0588:                 <tbody>
0589:                   {extrato.map((item) => (
0590:                     <tr key={item.ID}>
0591:                       <td>{item.ID}</td>
0592:                       <td>{item.D3_DATA}</td>
0593:                       <td>{item.D3_TIPO}</td>
0594:                       <td>{item.D3_QTD}</td>
0595:                       <td>{formatCurrency(item.D3_CUSTO_UNIT)}</td>
0596:                       <td>{formatCurrency(item.D3_VALOR)}</td>
0597:                       <td>{item.D3_ORIGEM}</td>
0598:                       <td>{item.D3_USUARIO ?? "-"}</td>
0599:                     </tr>
0600:                   ))}
0601:                 </tbody>
0602:               </table>
0603:             </div>
0604:           </>
0605:         )}
0606:       </div>
0607:     </>
0608:   );
0609: }
```

==========
ARQUIVO: frontend/src/pages/financeiroAP.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { useNavigate, useSearchParams } from "react-router-dom";
0003: import { Toolbar } from "../components/Toolbar";
0004: import { Pager } from "../components/Pager";
0005: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0006: import { PageHero } from "../components/PageHero";
0007: import { getFilial, onFilialChange } from "../state/filial";
0008: import { listarAP, pagarAP, baixarAP, type ApTitulo } from "../api/financeiro/ap";
0009: import { listarContasCaixa, type CaixaConta } from "../api/financeiro/caixa";
0010: import payablesIcon from "../assets/shell-icons/payables.png";
0011: import payablesBg from "../assets/hero-backgrounds/payables.webp";
0012:
```

```
0013: function toNumber(value: unknown, fallback = 0): number {
0014:   const n = Number(value);
0015:   return Number.isFinite(n) ? n : fallback;
0016: }
0017:
0018: function toStringValue(value: unknown, fallback = ""): string {
0019:   if (value === null || value === undefined) return fallback;
0020:   const s = String(value).trim();
0021:   return s ? s : fallback;
0022: }
0023:
0024: function formatCurrency(value: unknown) {
0025:   return toNumber(value, 0).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
0026: }
0027:
0028: function getId(item: ApTitulo) {
0029:   return toNumber(item.ID ?? (item as Record<string, unknown>).id);
0030: }
0031:
0032: function getStatus(item: ApTitulo) {
0033:   return toStringValue(item.F1_STATUS ?? (item as Record<string, unknown>).status, "ABERTO");
0034: }
0035:
0036: function getMovId(item: ApTitulo) {
0037:   return toNumber(item.F1_SE5_ID ?? (item as Record<string, unknown>).se5_id, 0);
0038: }
0039:
0040: function getSortValue(item: ApTitulo, key: string) {
0041:   switch (key) {
0042:     case "id":
0043:       return getId(item);
0044:     case "forn":
0045:       return toStringValue(item.F1_FORN ?? (item as Record<string, unknown>).forn, "-");
0046:     case "valor":
0047:       return toNumber(item.F1_VALOR ?? (item as Record<string, unknown>).valor, 0);
0048:     case "venc":
0049:       return toStringValue(item.F1_VENC ?? (item as Record<string, unknown>).venc, "-");
0050:     case "status":
0051:       return getStatus(item);
0052:     case "ref":
0053:       return toStringValue(item.F1_REF ?? (item as Record<string, unknown>).ref, "-");
0054:     case "mov":
0055:       return getMovId(item);
0056:     default:
0057:       return (item as Record<string, unknown>)[key];
0058:   }
0059: }
0060:
0061: export function FinanceiroAP() {
0062:   const navigate = useNavigate();
0063:   const [searchParams, setSearchParams] = useSearchParams();
0064:   const [filialView, setFilialView] = useState(getFilial());
0065:   const [status, setStatus] = useState(searchParams.get("status") ?? "ABERTO");
0066:   const [q, setQ] = useState(searchParams.get("q") ?? "");
```

```
0067:
0068:   const [loading, setLoading] = useState(false);
0069:   const [err, setErr] = useState<string | null>(null);
0070:   const [okMsg, setOkMsg] = useState<string | null>(null);
0071:   const [items, setItems] = useState<ApTitulo[]>([]);
0072:
0073:   const [contas, setContas] = useState<CaixaConta[]>([]);
0074:   const [contaId, setContaId] = useState<number | "">("");
0075:
0076:   const [page, setPage] = useState(1);
0077:   const [pageSize, setPageSize] = useState(10);
0078:
0079:   const [sortKey, setSortKey] = useState("id");
0080:   const [sortDir, setSortDir] = useState<SortDir>("desc");
0081:   const alertFilter = searchParams.get("alert") ?? "";
0082:
0083:   const alertLabelMap: Record<string, string> = {
0084:     ap_overdue: "Exibindo apenas titulos de AP vencidos.",
0085:     ap_due_today: "Exibindo apenas titulos de AP com vencimento hoje.",
0086:     ap_due_soon: "Exibindo apenas titulos de AP que vencem nos proximos 3 dias.",
0087:   };
0088:
0089:   function getErrorMessage(error: unknown, fallback: string): string {
0090:     if (error instanceof Error && error.message) return error.message;
0091:     return fallback;
0092:   }
0093:
0094:   async function load() {
0095:     setErr(null);
0096:     setLoading(true);
0097:     try {
0098:       const filial = getFilial();
0099:       const data = await listarAP({ filial, status });
0100:       setItems(data);
0101:       setQ("");
0102:       setPage(1);
0103:     } catch (error: unknown) {
0104:       setErr(getErrorMessage(error, "Erro ao carregar AP"));
0105:     } finally {
0106:       setLoading(false);
0107:     }
0108:   }
0109:
0110:   async function loadContas() {
0111:     setErr(null);
0112:     try {
0113:       const filial = getFilial();
0114:       const data = await listarContasCaixa({ filial });
0115:       setContas(data);
0116:       if (!contaId && data.length > 0) {
0117:         setContaId(toNumber(data[0].ID));
0118:       }
0119:     } catch (error: unknown) {
0120:       setErr(getErrorMessage(error, "Erro ao carregar contas de caixa"));
```

```
0121:     }
0122:   }
0123:
0124:   async function pagar(item: ApTitulo) {
0125:     setErr(null);
0126:     setOkMsg(null);
0127:     setLoading(true);
0128:     try {
0129:       if (!contaId) throw new Error("Selecione a conta de caixa.");
0130:       const result = await pagarAP({ titulo_id: getId(item), conta_id: Number(contaId) });
0131:       const se5Id = toNumber(result.se5_id ?? result.mov_id, 0);
0132:       setOkMsg(
0133:         se5Id > 0
0134:           ? `Pagamento realizado. Movimento de caixa gerado: ${se5Id}.`
0135:           : "Pagamento realizado com sucesso."
0136:       );
0137:       await load();
0138:     } catch (error: unknown) {
0139:       setErr(getErrorMessage(error, "Erro ao pagar AP"));
0140:     } finally {
0141:       setLoading(false);
0142:     }
0143:   }
0144:
0145:   async function baixar(item: ApTitulo) {
0146:     setErr(null);
0147:     setOkMsg(null);
0148:     setLoading(true);
0149:     try {
0150:       if (!contaId) throw new Error("Selecione a conta de caixa.");
0151:       const result = await baixarAP({ titulo_id: getId(item), conta_id: Number(contaId) });
0152:       const se5Id = toNumber(result.se5_id ?? result.mov_id, 0);
0153:       setOkMsg(
0154:         se5Id > 0
0155:           ? `Baixa realizada. Movimento de caixa vinculado: ${se5Id}.`
0156:           : "Baixa realizada com sucesso."
0157:       );
0158:       await load();
0159:     } catch (error: unknown) {
0160:       setErr(getErrorMessage(error, "Erro ao baixar AP"));
0161:     } finally {
0162:       setLoading(false);
0163:     }
0164:   }
0165:
0166:   useEffect(() => {
0167:     load();
0168:     loadContas();
0169:
0170:     const unsub = onFilialChange((novaFilial) => {
0171:       setFilialView(novaFilial);
0172:       setErr(null);
0173:       setStatus("ABERTO");
0174:       setContaId("");
```

```
0175:       setQ("");
0176:       setPage(1);
0177:       load();
0178:       loadContas();
0179:     });
0180:     return unsub;
0181:     // eslint-disable-next-line react-hooks/exhaustive-deps
0182:   }, []);
0183:
0184:   useEffect(() => {
0185:     const nextStatus = searchParams.get("status");
0186:     const nextQ = searchParams.get("q");
0187:     if (nextStatus && nextStatus !== status) setStatus(nextStatus);
0188:     if ((nextQ ?? "") !== q) setQ(nextQ ?? "");
0189:   }, [searchParams, q, status]);
0190:
0191:   useEffect(() => {
0192:     if (!alertFilter) return;
0193:     setStatus("ABERTO");
0194:     setQ("");
0195:     setPage(1);
0196:   }, [alertFilter]);
0197:
0198:   const columns: Column<ApTitulo>[] = [
0199:       { key: "id", header: "ID", sortable: true, value: (x) => getId(x) },
0200:       {
0201:         key: "forn",
0202:         header: "Fornecedor",
0203:         sortable: true,
0204:         value: (x) => toStringValue(x.F1_FORN ?? (x as Record<string, unknown>).forn, "-"),
0205:       },
0206:       {
0207:         key: "valor",
0208:         header: "Valor",
0209:         sortable: true,
0210:         align: "right",
0211:         value: (x) => toNumber(x.F1_VALOR ?? (x as Record<string, unknown>).valor, 0),
0212:         render: (x) => formatCurrency(x.F1_VALOR ?? (x as Record<string, unknown>).valor),
0213:       },
0214:       {
0215:         key: "venc",
0216:         header: "Venc.",
0217:         sortable: true,
0218:         className: "hide-sm",
0219:         value: (x) => toStringValue(x.F1_VENC ?? (x as Record<string, unknown>).venc, "-"),
0220:       },
0221:       {
0222:         key: "status",
0223:         header: "Status",
0224:         sortable: true,
0225:         className: "hide-sm",
0226:         value: (x) => getStatus(x),
0227:       },
0228:       {
```

```
0229:         key: "ref",
0230:         header: "Ref",
0231:         sortable: true,
0232:         className: "hide-sm",
0233:         value: (x) => toStringValue(x.F1_REF ?? (x as Record<string, unknown>).ref, "-"),
0234:       },
0235:       {
0236:         key: "mov",
0237:         header: "Mov. Caixa",
0238:         sortable: true,
0239:         className: "hide-sm",
0240:         value: (x) => getMovId(x),
0241:         render: (x) => {
0242:           const movId = getMovId(x);
0243:           return movId > 0 ? movId : "-";
0244:         },
0245:       },
0246:       {
0247:         key: "acoes",
0248:         header: "Acoes",
0249:         render: (x) => {
0250:           const s = getStatus(x);
0251:           const movId = getMovId(x);
0252:           const disabled = loading || s !== "ABERTO";
0253:           return (
0254:             <div className="row gap-10">
0255:               <button className="btn primary" disabled={disabled} onClick={() => pagar(x)}>
0256:                 Pagar
0257:               </button>
0258:               <button className="btn" disabled={disabled} onClick={() => baixar(x)}>
0259:                 Baixar
0260:               </button>
0261:               <button
0262:                 className="btn ghost"
0263:                 disabled={movId <= 0}
0264:                 onClick={() => navigate(`/financeiro/categorias?mov_id=${movId}`)}
0265:               >
0266:                 Categorizar
0267:               </button>
0268:               <button
0269:                 className="btn ghost"
0270:                 disabled={movId <= 0}
0271:                 onClick={() =>
0272:                   navigate(
0273:                     `/financeiro/caixa?q=${encodeURIComponent(
0274:                       toStringValue(x.F1_REF ?? (x as Record<string, unknown>).ref, "")
0275:                         ? `SF1#${getId(x)}`
0276:                         : String(movId)
0277:                     )}`
0278:                   )
0279:                 }
0280:               >
0281:                 Abrir caixa
0282:               </button>
```

```
0283:             </div>
0284:           );
0285:         },
0286:       },
0287:   ];
0288:
0289:   function normalize(v: unknown) {
0290:     if (v === null || v === undefined) return "";
0291:     if (typeof v === "boolean") return v ? "1" : "0";
0292:     if (typeof v === "number") return String(v).padStart(20, "0");
0293:     return String(v).toLowerCase().trim();
0294:   }
0295:
0296:   const filteredItems = useMemo(() => {
0297:     const query = q.trim().toLowerCase();
0298:     return items.filter((x) => {
0299:       if (alertFilter) {
0300:         const venc = new Date(toStringValue(x.F1_VENC ?? (x as Record<string, unknown>).venc,
"")));
0301:         if (Number.isNaN(venc.getTime())) return false;
0302:
0303:         const today = new Date();
0304:         today.setHours(0, 0, 0, 0);
0305:         venc.setHours(0, 0, 0, 0);
0306:         const diffDays = Math.round((venc.getTime() - today.getTime()) / 86400000);
0307:
0308:         if (alertFilter === "ap_overdue" && diffDays >= 0) return false;
0309:         if (alertFilter === "ap_due_today" && diffDays !== 0) return false;
0310:         if (alertFilter === "ap_due_soon" && (diffDays < 1 || diffDays > 3)) return false;
0311:       }
0312:       if (!query) return true;
0313:       const forn = toStringValue(x.F1_FORN ?? (x as Record<string, unknown>).forn, "");
0314:       const ref = toStringValue(x.F1_REF ?? (x as Record<string, unknown>).ref, "");
0315:       const id = String(getId(x));
0316:       const movId = String(getMovId(x));
0317:       return [forn, ref, id, movId].some((v) => v.toLowerCase().includes(query));
0318:     });
0319:   }, [items, q, alertFilter]);
0320:
0321:   const sortedItems = useMemo(() => {
0322:     const arr = [...filteredItems];
0323:     arr.sort((a, b) => {
0324:       const av = normalize(getSortValue(a, sortKey));
0325:       const bv = normalize(getSortValue(b, sortKey));
0326:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0327:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0328:       return 0;
0329:     });
0330:     return arr;
0331:   }, [filteredItems, sortKey, sortDir]);
0332:
0333:   const totalPages = useMemo(
0334:     () => Math.max(1, Math.ceil(sortedItems.length / pageSize)),
0335:     [sortedItems.length, pageSize]
```

```
0336:   );
0337:
0338:   const pagedRows = useMemo(() => {
0339:     const safePage = Math.min(Math.max(1, page), totalPages);
0340:     const start = (safePage - 1) * pageSize;
0341:     return sortedItems.slice(start, start + pageSize);
0342:   }, [sortedItems, page, pageSize, totalPages]);
0343:
0344:   const totalValor = useMemo(
0345:     () => filteredItems.reduce((acc, x) => acc + toNumber(x.F1_VALOR ?? (x as Record<string, u
nknown>).valor, 0), 0),
0346:     [filteredItems]
0347:   );
0348:
0349:   return (
0350:     <>
0351:       <PageHero
0352:         icon={payablesIcon}
0353:         backgroundImage={payablesBg}
0354:         backgroundPosition="center 34%"
0355:         eyebrow="Financeiro"
0356:         title="Contas a pagar"
0357:         description="Pagamento e baixa de titulos com reflexo no caixa e leitura rapida da pos
icao financeira."
0358:         metrics={[
0359:           { label: "Filial", value: filialView },
0360:           { label: "Status", value: status },
0361:           { label: "Valor filtrado", value: formatCurrency(totalValor) },
0362:         ]}
0363:       />
0364:
0365:       <div className="card">
0366:       {alertFilter && (
0367:         <div className="context-banner">
0368:           <div>
0369:             <div className="context-banner-title">Filtro aplicado por alerta</div>
0370:             <div className="context-banner-text">{alertLabelMap[alertFilter] ?? "Filtro contex
tual de alerta ativo."}</div>
0371:           </div>
0372:           <button
0373:             type="button"
0374:             className="btn ghost"
0375:             onClick={() => {
0376:               const next = new URLSearchParams(searchParams);
0377:               next.delete("alert");
0378:               setSearchParams(next);
0379:             }}
0380:           >
0381:             Limpar filtro do alerta
0382:           </button>
0383:         </div>
0384:       )}
0385:
0386:       <Toolbar
```

```
0387:         left={
0388:           <div className="hint">
0389:             Filial: <b>{filialView}</b> | Status: <b>{status}</b>
0390:           </div>
0391:         }
0392:         right={
0393:           <>
0394:             <input
0395:               className="input minw-240"
0396:               value={q}
0397:               onChange={(e) => {
0398:                 setQ(e.target.value);
0399:                 setPage(1);
0400:               }}
0401:               placeholder="Buscar por fornecedor, ref ou ID..."
0402:             />
0403:
0404:             <select className="select" value={status} onChange={(e) => { setStatus(e.target.va
lue); setPage(1); }}>
0405:               <option value="ABERTO">ABERTO</option>
0406:               <option value="BAIXADO">BAIXADO</option>
0407:               <option value="CANCELADO">CANCELADO</option>
0408:             </select>
0409:
0410:             <select
0411:               className="select"
0412:               value={contaId}
0413:               onChange={(e) => setContaId(e.target.value ? Number(e.target.value) : "")}
0414:             >
0415:               <option value="">Conta de Caixa</option>
0416:               {contas.map((c) => (
0417:                 <option key={c.ID} value={c.ID}>
0418:                   {toStringValue(c.E5_NOME, "Conta")} ({toStringValue(c.E5_FILIAL, filialView)
})
0419:                 </option>
0420:               ))}
0421:             </select>
0422:
0423:             <button
0424:               type="button"
0425:               className="btn"
0426:               onClick={() => {
0427:                 const next = new URLSearchParams(searchParams);
0428:                 next.set("status", status);
0429:                 if (q.trim()) next.set("q", q.trim());
0430:                 else next.delete("q");
0431:                 setSearchParams(next);
0432:                 load();
0433:               }}
0434:               disabled={loading}
0435:             >
0436:               Aplicar
0437:             </button>
0438:             <button
```

```
0439:               type="button"
0440:               className="btn ghost"
0441:               onClick={() => {
0442:                 setQ("");
0443:                 setStatus("ABERTO");
0444:                 setPage(1);
0445:                 const next = new URLSearchParams(searchParams);
0446:                 next.delete("q");
0447:                 next.delete("status");
0448:                 setSearchParams(next);
0449:               }}
0450:               disabled={loading}
0451:             >
0452:               Limpar
0453:             </button>
0454:           </>
0455:         }
0456:       />
0457:
0458:       {okMsg && <div className="alert alert-success mt-12">{okMsg}</div>}
0459:       {err && <div className="alert mt-12">{err}</div>}
0460:
0461:       <div className="section-title">Resumo</div>
0462:       <div className="stats-grid">
0463:         <div className="stat-card">
0464:           <div>
0465:             <div className="kpi-label">Titulos (filtrados)</div>
0466:             <div className="kpi">{filteredItems.length}</div>
0467:           </div>
0468:           <div className="stat-icon">AP</div>
0469:         </div>
0470:         <div className="stat-card">
0471:           <div>
0472:             <div className="kpi-label">Valor total (filtrado)</div>
0473:             <div className="kpi">{formatCurrency(totalValor)}</div>
0474:           </div>
0475:           <div className="stat-icon success">R$</div>
0476:         </div>
0477:         <div className="stat-card">
0478:           <div>
0479:             <div className="kpi-label">Status atual</div>
0480:             <div className="kpi">{status}</div>
0481:           </div>
0482:           <div className="stat-icon solid">ST</div>
0483:         </div>
0484:       </div>
0485:
0486:       <div className="row-between mt-12">
0487:         <Pager
0488:           page={page}
0489:           totalPages={totalPages}
0490:           pageSize={pageSize}
0491:           totalItems={sortedItems.length}
0492:           onPageChange={setPage}
```

```
0493:           onPageSizeChange={(ps) => {
0494:             setPageSize(ps);
0495:             setPage(1);
0496:           }}
0497:           showPageSize
0498:         />
0499:       </div>
0500:
0501:       <DataTable
0502:         rows={pagedRows}
0503:         columns={columns}
0504:         sortKey={sortKey}
0505:         sortDir={sortDir}
0506:         className="compact"
0507:         wrapperClassName="table-scroller"
0508:         onSortChange={(k, d) => {
0509:           setSortKey(k);
0510:           setSortDir(d);
0511:           setPage(1);
0512:         }}
0513:         loading={loading}
0514:         emptyText="Nenhum titulo encontrado."
0515:       />
0516:       </div>
0517:     </>
0518:   );
0519: }
```

==========
ARQUIVO: frontend/src/pages/financeiroAR.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { useNavigate, useSearchParams } from "react-router-dom";
0003: import { Toolbar } from "../components/Toolbar";
0004: import { Pager } from "../components/Pager";
0005: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0006: import { PageHero } from "../components/PageHero";
0007: import { getFilial, onFilialChange } from "../state/filial";
0008: import { listarAR, receberAR, baixarAR, type ArTitulo } from "../api/financeiro/ar";
0009: import { listarContasCaixa, type CaixaConta } from "../api/financeiro/caixa";
0010: import receivablesIcon from "../assets/shell-icons/receivables.png";
0011: import receivablesBg from "../assets/hero-backgrounds/receivables.jpg";
0012:
0013: function toNumber(value: unknown, fallback = 0): number {
0014:   const n = Number(value);
0015:   return Number.isFinite(n) ? n : fallback;
0016: }
0017:
0018: function toStringValue(value: unknown, fallback = ""): string {
0019:   if (value === null || value === undefined) return fallback;
0020:   const s = String(value).trim();
0021:   return s ? s : fallback;
0022: }
0023:
```

```
0024: function formatCurrency(value: unknown) {
0025:   return toNumber(value, 0).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
0026: }
0027:
0028: function getId(item: ArTitulo) {
0029:   return toNumber(item.ID ?? (item as Record<string, unknown>).id);
0030: }
0031:
0032: function getStatus(item: ArTitulo) {
0033:   return toStringValue(item.E1_STATUS ?? (item as Record<string, unknown>).status, "ABERTO");
0034: }
0035:
0036: function getMovId(item: ArTitulo) {
0037:   return toNumber(item.E1_SE5_ID ?? (item as Record<string, unknown>).se5_id, 0);
0038: }
0039:
0040: function getSortValue(item: ArTitulo, key: string) {
0041:   switch (key) {
0042:     case "id":
0043:       return getId(item);
0044:     case "cliente":
0045:       return toStringValue(item.E1_CLIENTE ?? (item as Record<string, unknown>).cliente, "-");
0046:     case "valor":
0047:       return toNumber(item.E1_VALOR ?? (item as Record<string, unknown>).valor, 0);
0048:     case "venc":
0049:       return toStringValue(item.E1_VENC ?? (item as Record<string, unknown>).venc, "-");
0050:     case "status":
0051:       return getStatus(item);
0052:     case "ref":
0053:       return toStringValue(item.E1_REF ?? (item as Record<string, unknown>).ref, "-");
0054:     case "mov":
0055:       return getMovId(item);
0056:     default:
0057:       return (item as Record<string, unknown>)[key];
0058:   }
0059: }
0060:
0061: export function FinanceiroAR() {
0062:   const navigate = useNavigate();
0063:   const [searchParams, setSearchParams] = useSearchParams();
0064:   const [filialView, setFilialView] = useState(getFilial());
0065:   const [status, setStatus] = useState("ABERTO");
0066:   const [q, setQ] = useState("");
0067:
0068:   const [loading, setLoading] = useState(false);
0069:   const [err, setErr] = useState<string | null>(null);
0070:   const [okMsg, setOkMsg] = useState<string | null>(null);
0071:   const [items, setItems] = useState<ArTitulo[]>([]);
0072:
0073:   const [contas, setContas] = useState<CaixaConta[]>([]);
0074:   const [contaId, setContaId] = useState<number | "">("");
0075:
0076:   const [page, setPage] = useState(1);
0077:   const [pageSize, setPageSize] = useState(10);
```

```
0078:
0079:   const [sortKey, setSortKey] = useState("id");
0080:   const [sortDir, setSortDir] = useState<SortDir>("desc");
0081:   const alertFilter = searchParams.get("alert") ?? "";
0082:
0083:   const alertLabelMap: Record<string, string> = {
0084:     ar_overdue: "Exibindo apenas titulos de AR vencidos.",
0085:     ar_due_today: "Exibindo apenas titulos de AR com vencimento hoje.",
0086:     ar_due_soon: "Exibindo apenas titulos de AR que vencem nos proximos 3 dias.",
0087:   };
0088:
0089:   function getErrorMessage(error: unknown, fallback: string): string {
0090:     if (error instanceof Error && error.message) return error.message;
0091:     return fallback;
0092:   }
0093:
0094:   async function load() {
0095:     setErr(null);
0096:     setLoading(true);
0097:     try {
0098:       const filial = getFilial();
0099:       const data = await listarAR({ filial, status });
0100:       setItems(data);
0101:       setQ("");
0102:       setPage(1);
0103:     } catch (error: unknown) {
0104:       setErr(getErrorMessage(error, "Erro ao carregar AR"));
0105:     } finally {
0106:       setLoading(false);
0107:     }
0108:   }
0109:
0110:   async function loadContas() {
0111:     setErr(null);
0112:     try {
0113:       const filial = getFilial();
0114:       const data = await listarContasCaixa({ filial });
0115:       setContas(data);
0116:       if (!contaId && data.length > 0) {
0117:         setContaId(toNumber(data[0].ID));
0118:       }
0119:     } catch (error: unknown) {
0120:       setErr(getErrorMessage(error, "Erro ao carregar contas de caixa"));
0121:     }
0122:   }
0123:
0124:   async function receber(item: ArTitulo) {
0125:     setErr(null);
0126:     setOkMsg(null);
0127:     setLoading(true);
0128:     try {
0129:       if (!contaId) throw new Error("Selecione a conta de caixa.");
0130:       const result = await receberAR({ titulo_id: getId(item), conta_id: Number(contaId) });
0131:       const se5Id = toNumber(result.se5_id ?? result.mov_id, 0);
```

```
0132:       setOkMsg(
0133:         se5Id > 0
0134:           ? `Recebimento realizado. Movimento de caixa gerado: ${se5Id}.`
0135:           : "Recebimento realizado com sucesso."
0136:       );
0137:       await load();
0138:     } catch (error: unknown) {
0139:       setErr(getErrorMessage(error, "Erro ao receber AR"));
0140:     } finally {
0141:       setLoading(false);
0142:     }
0143:   }
0144:
0145:   async function baixar(item: ArTitulo) {
0146:     setErr(null);
0147:     setOkMsg(null);
0148:     setLoading(true);
0149:     try {
0150:       if (!contaId) throw new Error("Selecione a conta de caixa.");
0151:       const result = await baixarAR({ titulo_id: getId(item), conta_id: Number(contaId) });
0152:       const se5Id = toNumber(result.se5_id ?? result.mov_id, 0);
0153:       setOkMsg(
0154:         se5Id > 0
0155:           ? `Baixa realizada. Movimento de caixa vinculado: ${se5Id}.`
0156:           : "Baixa realizada com sucesso."
0157:       );
0158:       await load();
0159:     } catch (error: unknown) {
0160:       setErr(getErrorMessage(error, "Erro ao baixar AR"));
0161:     } finally {
0162:       setLoading(false);
0163:     }
0164:   }
0165:
0166:   useEffect(() => {
0167:     load();
0168:     loadContas();
0169:
0170:     const unsub = onFilialChange((novaFilial) => {
0171:       setFilialView(novaFilial);
0172:       setErr(null);
0173:       setStatus("ABERTO");
0174:       setContaId("");
0175:       setQ("");
0176:       setPage(1);
0177:       load();
0178:       loadContas();
0179:     });
0180:     return unsub;
0181:     // eslint-disable-next-line react-hooks/exhaustive-deps
0182:   }, []);
0183:
0184:   useEffect(() => {
0185:     if (!alertFilter) return;
```

```
0186:     setStatus("ABERTO");
0187:     setQ("");
0188:     setPage(1);
0189:   }, [alertFilter]);
0190:
0191:   const columns: Column<ArTitulo>[] = [
0192:       { key: "id", header: "ID", sortable: true, value: (x) => getId(x) },
0193:       {
0194:         key: "cliente",
0195:         header: "Cliente",
0196:         sortable: true,
0197:         value: (x) => toStringValue(x.E1_CLIENTE ?? (x as Record<string, unknown>).cliente, "-
"),
0198:       },
0199:       {
0200:         key: "valor",
0201:         header: "Valor",
0202:         sortable: true,
0203:         align: "right",
0204:         value: (x) => toNumber(x.E1_VALOR ?? (x as Record<string, unknown>).valor, 0),
0205:         render: (x) => formatCurrency(x.E1_VALOR ?? (x as Record<string, unknown>).valor),
0206:       },
0207:       {
0208:         key: "venc",
0209:         header: "Venc.",
0210:         sortable: true,
0211:         className: "hide-sm",
0212:         value: (x) => toStringValue(x.E1_VENC ?? (x as Record<string, unknown>).venc, "-"),
0213:       },
0214:       {
0215:         key: "status",
0216:         header: "Status",
0217:         sortable: true,
0218:         className: "hide-sm",
0219:         value: (x) => getStatus(x),
0220:       },
0221:       {
0222:         key: "ref",
0223:         header: "Ref",
0224:         sortable: true,
0225:         className: "hide-sm",
0226:         value: (x) => toStringValue(x.E1_REF ?? (x as Record<string, unknown>).ref, "-"),
0227:       },
0228:       {
0229:         key: "mov",
0230:         header: "Mov. Caixa",
0231:         sortable: true,
0232:         className: "hide-sm",
0233:         value: (x) => getMovId(x),
0234:         render: (x) => {
0235:           const movId = getMovId(x);
0236:           return movId > 0 ? movId : "-";
0237:         },
0238:       },
```

```
0239:       {
0240:         key: "acoes",
0241:         header: "Acoes",
0242:         render: (x) => {
0243:           const s = getStatus(x);
0244:           const movId = getMovId(x);
0245:           const disabled = loading || s !== "ABERTO";
0246:           return (
0247:             <div className="row gap-10">
0248:               <button className="btn primary" disabled={disabled} onClick={() => receber(x)}>
0249:                 Receber
0250:               </button>
0251:               <button className="btn" disabled={disabled} onClick={() => baixar(x)}>
0252:                 Baixar
0253:               </button>
0254:               <button
0255:                 className="btn ghost"
0256:                 disabled={movId <= 0}
0257:                 onClick={() => navigate(`/financeiro/categorias?mov_id=${movId}`)}
0258:               >
0259:                 Categorizar
0260:               </button>
0261:             </div>
0262:           );
0263:         },
0264:       },
0265:   ];
0266:
0267:   function normalize(v: unknown) {
0268:     if (v === null || v === undefined) return "";
0269:     if (typeof v === "boolean") return v ? "1" : "0";
0270:     if (typeof v === "number") return String(v).padStart(20, "0");
0271:     return String(v).toLowerCase().trim();
0272:   }
0273:
0274:   const filteredItems = useMemo(() => {
0275:     const query = q.trim().toLowerCase();
0276:     return items.filter((x) => {
0277:       if (alertFilter) {
0278:         const venc = new Date(toStringValue(x.E1_VENC ?? (x as Record<string, unknown>).venc,
"")));
0279:         if (Number.isNaN(venc.getTime())) return false;
0280:
0281:         const today = new Date();
0282:         today.setHours(0, 0, 0, 0);
0283:         venc.setHours(0, 0, 0, 0);
0284:         const diffDays = Math.round((venc.getTime() - today.getTime()) / 86400000);
0285:
0286:         if (alertFilter === "ar_overdue" && diffDays >= 0) return false;
0287:         if (alertFilter === "ar_due_today" && diffDays !== 0) return false;
0288:         if (alertFilter === "ar_due_soon" && (diffDays < 1 || diffDays > 3)) return false;
0289:       }
0290:       if (!query) return true;
```

```
0291:       const cliente = toStringValue(x.E1_CLIENTE ?? (x as Record<string, unknown>).cliente, ""
);
0292:       const ref = toStringValue(x.E1_REF ?? (x as Record<string, unknown>).ref, "");
0293:       const id = String(getId(x));
0294:       const movId = String(getMovId(x));
0295:       return [cliente, ref, id, movId].some((v) => v.toLowerCase().includes(query));
0296:     });
0297:   }, [items, q, alertFilter]);
0298:
0299:   const sortedItems = useMemo(() => {
0300:     const arr = [...filteredItems];
0301:     arr.sort((a, b) => {
0302:       const av = normalize(getSortValue(a, sortKey));
0303:       const bv = normalize(getSortValue(b, sortKey));
0304:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0305:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0306:       return 0;
0307:     });
0308:     return arr;
0309:   }, [filteredItems, sortKey, sortDir]);
0310:
0311:   const totalPages = useMemo(
0312:     () => Math.max(1, Math.ceil(sortedItems.length / pageSize)),
0313:     [sortedItems.length, pageSize]
0314:   );
0315:
0316:   const pagedRows = useMemo(() => {
0317:     const safePage = Math.min(Math.max(1, page), totalPages);
0318:     const start = (safePage - 1) * pageSize;
0319:     return sortedItems.slice(start, start + pageSize);
0320:   }, [sortedItems, page, pageSize, totalPages]);
0321:
0322:   const totalValor = useMemo(
0323:     () => filteredItems.reduce((acc, x) => acc + toNumber(x.E1_VALOR ?? (x as Record<string, u
nknown>).valor, 0), 0),
0324:     [filteredItems]
0325:   );
0326:
0327:   return (
0328:     <>
0329:       <PageHero
0330:         icon={receivablesIcon}
0331:         backgroundImage={receivablesBg}
0332:         backgroundPosition="center 40%"
0333:         eyebrow="Financeiro"
0334:         title="Contas a receber"
0335:         description="Titulos abertos, recebimento operacional e baixa assistida com conta de c
aixa vinculada."
0336:         metrics={[
0337:           { label: "Filial", value: filialView },
0338:           { label: "Status", value: status },
0339:           { label: "Valor filtrado", value: formatCurrency(totalValor) },
0340:         ]}
0341:       />
```

```
0342:
0343:       <div className="card">
0344:       {alertFilter && (
0345:         <div className="context-banner">
0346:           <div>
0347:             <div className="context-banner-title">Filtro aplicado por alerta</div>
0348:             <div className="context-banner-text">{alertLabelMap[alertFilter] ?? "Filtro contex
tual de alerta ativo."}</div>
0349:           </div>
0350:           <button
0351:             type="button"
0352:             className="btn ghost"
0353:             onClick={() => {
0354:               const next = new URLSearchParams(searchParams);
0355:               next.delete("alert");
0356:               setSearchParams(next);
0357:             }}
0358:           >
0359:             Limpar filtro do alerta
0360:           </button>
0361:         </div>
0362:       )}
0363:
0364:       <Toolbar
0365:         left={
0366:           <div className="hint">
0367:             Filial: <b>{filialView}</b> | Status: <b>{status}</b>
0368:           </div>
0369:         }
0370:         right={
0371:           <>
0372:             <input
0373:               className="input minw-240"
0374:               value={q}
0375:               onChange={(e) => {
0376:                 setQ(e.target.value);
0377:                 setPage(1);
0378:               }}
0379:               placeholder="Buscar por cliente, ref ou ID..."
0380:             />
0381:
0382:             <select className="select" value={status} onChange={(e) => { setStatus(e.target.va
lue); setPage(1); }}>
0383:               <option value="ABERTO">ABERTO</option>
0384:               <option value="BAIXADO">BAIXADO</option>
0385:               <option value="CANCELADO">CANCELADO</option>
0386:             </select>
0387:
0388:             <select
0389:               className="select"
0390:               value={contaId}
0391:               onChange={(e) => setContaId(e.target.value ? Number(e.target.value) : "")}
0392:             >
0393:               <option value="">Conta de Caixa</option>
```

```
0394:               {contas.map((c) => (
0395:                 <option key={c.ID} value={c.ID}>
0396:                   {toStringValue(c.E5_NOME, "Conta")} ({toStringValue(c.E5_FILIAL, filialView)
})
0397:                 </option>
0398:               ))}
0399:             </select>
0400:
0401:             <button type="button" className="btn" onClick={load} disabled={loading}>
0402:               Aplicar
0403:             </button>
0404:             <button
0405:               type="button"
0406:               className="btn ghost"
0407:               onClick={() => {
0408:                 setQ("");
0409:                 setStatus("ABERTO");
0410:                 setPage(1);
0411:               }}
0412:               disabled={loading}
0413:             >
0414:               Limpar
0415:             </button>
0416:           </>
0417:         }
0418:       />
0419:
0420:       {okMsg && <div className="alert alert-success mt-12">{okMsg}</div>}
0421:       {err && <div className="alert mt-12">{err}</div>}
0422:
0423:       <div className="section-title">Resumo</div>
0424:       <div className="stats-grid">
0425:         <div className="stat-card">
0426:           <div>
0427:             <div className="kpi-label">Titulos (filtrados)</div>
0428:             <div className="kpi">{filteredItems.length}</div>
0429:           </div>
0430:           <div className="stat-icon">AR</div>
0431:         </div>
0432:         <div className="stat-card">
0433:           <div>
0434:             <div className="kpi-label">Valor total (filtrado)</div>
0435:             <div className="kpi">{formatCurrency(totalValor)}</div>
0436:           </div>
0437:           <div className="stat-icon success">R$</div>
0438:         </div>
0439:         <div className="stat-card">
0440:           <div>
0441:             <div className="kpi-label">Status atual</div>
0442:             <div className="kpi">{status}</div>
0443:           </div>
0444:           <div className="stat-icon solid">ST</div>
0445:         </div>
0446:       </div>
```

```
0447:
0448:       <div className="row-between mt-12">
0449:         <Pager
0450:           page={page}
0451:           totalPages={totalPages}
0452:           pageSize={pageSize}
0453:           totalItems={sortedItems.length}
0454:           onPageChange={setPage}
0455:           onPageSizeChange={(ps) => {
0456:             setPageSize(ps);
0457:             setPage(1);
0458:           }}
0459:           showPageSize
0460:         />
0461:       </div>
0462:
0463:       <DataTable
0464:         rows={pagedRows}
0465:         columns={columns}
0466:         sortKey={sortKey}
0467:         sortDir={sortDir}
0468:         className="compact"
0469:         wrapperClassName="table-scroller"
0470:         onSortChange={(k, d) => {
0471:           setSortKey(k);
0472:           setSortDir(d);
0473:           setPage(1);
0474:         }}
0475:         loading={loading}
0476:         emptyText="Nenhum titulo encontrado."
0477:       />
0478:       </div>
0479:     </>
0480:   );
0481: }
```

==========
ARQUIVO: frontend/src/pages/financeiroCaixa.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { useSearchParams } from "react-router-dom";
0003: import { Toolbar } from "../components/Toolbar";
0004: import { Pager } from "../components/Pager";
0005: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0006: import { PageHero } from "../components/PageHero";
0007: import { getFilial, onFilialChange } from "../state/filial";
0008: import {
0009:   listarContasCaixa,
0010:   extratoCaixa,
0011:   saldoCaixa,
0012:   type CaixaConta,
0013:   type CaixaMovimento,
0014:   type CaixaSaldo,
0015: } from "../api/financeiro/caixa";
```

```
0016: import cashIcon from "../assets/shell-icons/cash.png";
0017: import cashBg from "../assets/hero-backgrounds/cash.jpg";
0018:
0019: function toNumber(value: unknown, fallback = 0): number {
0020:   const n = Number(value);
0021:   return Number.isFinite(n) ? n : fallback;
0022: }
0023:
0024: function toStringValue(value: unknown, fallback = ""): string {
0025:   if (value === null || value === undefined) return fallback;
0026:   const s = String(value).trim();
0027:   return s ? s : fallback;
0028: }
0029:
0030: function formatCurrency(value: unknown) {
0031:   return toNumber(value, 0).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
0032: }
0033:
0034: export function FinanceiroCaixa() {
0035:   const [searchParams, setSearchParams] = useSearchParams();
0036:   const [filialView, setFilialView] = useState(getFilial());
0037:   const [contas, setContas] = useState<CaixaConta[]>([]);
0038:   const [contaId, setContaId] = useState<number | "">("");
0039:
0040:   const [de, setDe] = useState("");
0041:   const [ate, setAte] = useState("");
0042:   const [q, setQ] = useState(searchParams.get("q") ?? "");
0043:
0044:   const [loading, setLoading] = useState(false);
0045:   const [err, setErr] = useState<string | null>(null);
0046:   const [saldo, setSaldo] = useState<CaixaSaldo | null>(null);
0047:   const [movs, setMovs] = useState<CaixaMovimento[]>([]);
0048:
0049:   const [page, setPage] = useState(1);
0050:   const [pageSize, setPageSize] = useState(10);
0051:
0052:   const [sortKey, setSortKey] = useState("data");
0053:   const [sortDir, setSortDir] = useState<SortDir>("desc");
0054:
0055:   function getErrorMessage(error: unknown, fallback: string): string {
0056:     if (error instanceof Error && error.message) return error.message;
0057:     return fallback;
0058:   }
0059:
0060:   async function loadContas() {
0061:     setErr(null);
0062:     try {
0063:       const filial = getFilial();
0064:       const data = await listarContasCaixa({ filial });
0065:       setContas(data);
0066:       if (!contaId && data.length > 0) {
0067:         setContaId(toNumber(data[0].ID));
0068:       }
0069:     } catch (error: unknown) {
```

```
0070:       setErr(getErrorMessage(error, "Erro ao carregar contas de caixa"));
0071:     }
0072:   }
0073:
0074:   async function loadExtrato() {
0075:     setErr(null);
0076:     setLoading(true);
0077:     try {
0078:       if (!contaId) return;
0079:       const filial = getFilial();
0080:       const data = await extratoCaixa({
0081:         filial,
0082:         conta_id: Number(contaId),
0083:         de: de || null,
0084:         ate: ate || null,
0085:       });
0086:       setMovs(data);
0087:       setPage(1);
0088:     } catch (error: unknown) {
0089:       setErr(getErrorMessage(error, "Erro ao carregar extrato"));
0090:     } finally {
0091:       setLoading(false);
0092:     }
0093:   }
0094:
0095:   async function loadSaldo() {
0096:     setErr(null);
0097:     setLoading(true);
0098:     try {
0099:       if (!contaId) return;
0100:       const filial = getFilial();
0101:       const data = await saldoCaixa({
0102:         filial,
0103:         conta_id: Number(contaId),
0104:         ate: ate || null,
0105:       });
0106:       setSaldo(data);
0107:     } catch (error: unknown) {
0108:       setErr(getErrorMessage(error, "Erro ao carregar saldo"));
0109:     } finally {
0110:       setLoading(false);
0111:     }
0112:   }
0113:
0114:   useEffect(() => {
0115:     loadContas();
0116:
0117:     const unsub = onFilialChange((novaFilial) => {
0118:       setFilialView(novaFilial);
0119:       setErr(null);
0120:       setContaId("");
0121:       setMovs([]);
0122:       setSaldo(null);
0123:       setQ("");
```

```
0124:       setPage(1);
0125:       loadContas();
0126:     });
0127:     return unsub;
0128:     // eslint-disable-next-line react-hooks/exhaustive-deps
0129:   }, []);
0130:
0131:   useEffect(() => {
0132:     const nextQ = searchParams.get("q") ?? "";
0133:     if (nextQ !== q) setQ(nextQ);
0134:   }, [searchParams, q]);
0135:
0136:   useEffect(() => {
0137:     if (!contaId) return;
0138:     loadExtrato();
0139:     loadSaldo();
0140:     // eslint-disable-next-line react-hooks/exhaustive-deps
0141:   }, [contaId]);
0142:
0143:   const columns: Column<CaixaMovimento>[] = useMemo(
0144:     () => [
0145:       {
0146:         key: "data",
0147:         header: "Data",
0148:         sortable: true,
0149:         value: (x) => toStringValue(x.E5_DATA ?? (x as Record<string, unknown>).data, "-"),
0150:       },
0151:       {
0152:         key: "tipo",
0153:         header: "Tipo",
0154:         sortable: true,
0155:         value: (x) => toStringValue(x.E5_TIPO ?? (x as Record<string, unknown>).tipo, "-"),
0156:       },
0157:       {
0158:         key: "valor",
0159:         header: "Valor",
0160:         sortable: true,
0161:         align: "right",
0162:         value: (x) => toNumber(x.E5_VALOR ?? (x as Record<string, unknown>).valor, 0),
0163:         render: (x) => formatCurrency(x.E5_VALOR ?? (x as Record<string, unknown>).valor),
0164:       },
0165:       {
0166:         key: "saldo",
0167:         header: "Saldo",
0168:         sortable: true,
0169:         align: "right",
0170:         value: (x) => toNumber(x.E5_SALDO_ACUMULADO ?? (x as Record<string, unknown>).saldo, 0
),
0171:         render: (x) => formatCurrency(x.E5_SALDO_ACUMULADO ?? (x as Record<string, unknown>).s
aldo),
0172:       },
0173:       {
0174:         key: "hist",
0175:         header: "Historico",
```

```
0176:         sortable: true,
0177:         className: "hide-sm",
0178:         value: (x) => toStringValue(x.E5_HIST ?? (x as Record<string, unknown>).hist, "-"),
0179:       },
0180:       {
0181:         key: "origem",
0182:         header: "Origem",
0183:         sortable: true,
0184:         className: "hide-sm",
0185:         value: (x) => {
0186:           const tipo = toStringValue(x.E5_ORIGEM_TIPO ?? (x as Record<string, unknown>).origem
_tipo, "");
0187:           const id = toStringValue(x.E5_ORIGEM_ID ?? (x as Record<string, unknown>).origem_id,
 "");
0188:           return [tipo, id].filter(Boolean).join("#") || "-";
0189:         },
0190:       },
0191:       {
0192:         key: "categoria",
0193:         header: "Categoria",
0194:         sortable: true,
0195:         className: "hide-sm",
0196:         value: (x) =>
0197:           toStringValue(x.CATEG_NOME ?? (x as Record<string, unknown>).categoria, "-"),
0198:       },
0199:       {
0200:         key: "usuario",
0201:         header: "Usuario",
0202:         sortable: true,
0203:         className: "hide-sm",
0204:         value: (x) => toStringValue(x.E5_USUARIO ?? (x as Record<string, unknown>).usuario, "-
"),
0205:       },
0206:     ],
0207:     []
0208:   );
0209:
0210:   function normalize(v: unknown) {
0211:     if (v === null || v === undefined) return "";
0212:     if (typeof v === "boolean") return v ? "1" : "0";
0213:     if (typeof v === "number") return String(v).padStart(20, "0");
0214:     return String(v).toLowerCase().trim();
0215:   }
0216:
0217:   const filteredItems = useMemo(() => {
0218:     const query = q.trim().toLowerCase();
0219:     if (!query) return movs;
0220:     return movs.filter((x) => {
0221:       const hist = toStringValue(x.E5_HIST ?? (x as Record<string, unknown>).hist, "");
0222:       const origem = toStringValue(x.E5_ORIGEM_TIPO ?? (x as Record<string, unknown>).origem_t
ipo, "");
0223:       const user = toStringValue(x.E5_USUARIO ?? (x as Record<string, unknown>).usuario, "");
0224:       const cat = toStringValue(x.CATEG_NOME ?? (x as Record<string, unknown>).categoria, "");
0225:       return [hist, origem, user, cat].some((v) => v.toLowerCase().includes(query));
```

```
0226:     });
0227:   }, [movs, q]);
0228:
0229:   const sortedItems = useMemo(() => {
0230:     const col = columns.find((c) => c.key === sortKey);
0231:     const valueFn = col?.value ?? ((r: CaixaMovimento) => (r as Record<string, unknown>)[sortK
ey]);
0232:
0233:     const arr = [...filteredItems];
0234:     arr.sort((a, b) => {
0235:       const av = normalize(valueFn(a));
0236:       const bv = normalize(valueFn(b));
0237:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0238:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0239:       return 0;
0240:     });
0241:     return arr;
0242:   }, [filteredItems, columns, sortKey, sortDir]);
0243:
0244:   const totalPages = useMemo(
0245:     () => Math.max(1, Math.ceil(sortedItems.length / pageSize)),
0246:     [sortedItems.length, pageSize]
0247:   );
0248:
0249:   const pagedRows = useMemo(() => {
0250:     const safePage = Math.min(Math.max(1, page), totalPages);
0251:     const start = (safePage - 1) * pageSize;
0252:     return sortedItems.slice(start, start + pageSize);
0253:   }, [sortedItems, page, pageSize, totalPages]);
0254:
0255:   return (
0256:     <>
0257:       <PageHero
0258:         icon={cashIcon}
0259:         backgroundImage={cashBg}
0260:         backgroundPosition="center 42%"
0261:         eyebrow="Tesouraria"
0262:         title="Caixa e extrato"
0263:         description="Consulta de contas, composicao de saldo e rastreio completo das entradas
e saidas da operacao."
0264:         metrics={[
0265:           { label: "Filial", value: filialView },
0266:           { label: "Conta", value: contaId ? String(contaId) : "Nao definida" },
0267:           { label: "Saldo", value: saldo ? formatCurrency(saldo.saldo) : "Aguardando" },
0268:         ]}
0269:       />
0270:
0271:       <div className="card">
0272:       <Toolbar
0273:         left={
0274:           <div className="hint">
0275:             Filial: <b>{filialView}</b>
0276:           </div>
0277:         }
```

```
0278:         right={
0279:           <>
0280:             <select
0281:               className="select"
0282:               value={contaId}
0283:               onChange={(e) => setContaId(e.target.value ? Number(e.target.value) : "")}
0284:             >
0285:               <option value="">Conta de Caixa</option>
0286:               {contas.map((c) => (
0287:                 <option key={c.ID} value={c.ID}>
0288:                   {toStringValue(c.E5_NOME, "Conta")} ({toStringValue(c.E5_FILIAL, filialView)
})
0289:                 </option>
0290:               ))}
0291:             </select>
0292:
0293:             <input
0294:               className="input"
0295:               type="date"
0296:               value={de}
0297:               onChange={(e) => setDe(e.target.value)}
0298:               aria-label="Data inicial"
0299:             />
0300:             <input
0301:               className="input"
0302:               type="date"
0303:               value={ate}
0304:               onChange={(e) => setAte(e.target.value)}
0305:               aria-label="Data final"
0306:             />
0307:
0308:             <input
0309:               className="input minw-240"
0310:               value={q}
0311:               onChange={(e) => {
0312:                 setQ(e.target.value);
0313:                 setPage(1);
0314:               }}
0315:               placeholder="Filtrar extrato (hist, origem, usuario...)"
0316:             />
0317:
0318:             <button
0319:               type="button"
0320:               className="btn"
0321:               onClick={() => {
0322:                 const next = new URLSearchParams(searchParams);
0323:                 if (q.trim()) next.set("q", q.trim());
0324:                 else next.delete("q");
0325:                 setSearchParams(next);
0326:                 loadExtrato();
0327:               }}
0328:               disabled={loading}
0329:             >
0330:               Extrato
```

```
0331:             </button>
0332:             <button type="button" className="btn" onClick={loadSaldo} disabled={loading}>
0333:               Saldo
0334:             </button>
0335:           </>
0336:         }
0337:       />
0338:
0339:       {err && <div className="alert mt-12">{err}</div>}
0340:
0341:       <div className="section-title">Resumo</div>
0342:       {saldo && (
0343:         <div className="stats-grid">
0344:           <div className="stat-card">
0345:             <div>
0346:               <div className="kpi-label">Entradas</div>
0347:               <div className="kpi">{formatCurrency(saldo.entradas)}</div>
0348:             </div>
0349:             <div className="stat-icon success">E</div>
0350:           </div>
0351:           <div className="stat-card">
0352:             <div>
0353:               <div className="kpi-label">Saidas</div>
0354:               <div className="kpi">{formatCurrency(saldo.saidas)}</div>
0355:             </div>
0356:             <div className="stat-icon">S</div>
0357:           </div>
0358:           <div className="stat-card">
0359:             <div>
0360:               <div className="kpi-label">Saldo</div>
0361:               <div className="kpi">{formatCurrency(saldo.saldo)}</div>
0362:             </div>
0363:             <div className="stat-icon solid">$</div>
0364:           </div>
0365:         </div>
0366:       )}
0367:
0368:       <div className="section-title">Extrato</div>
0369:
0370:       <div className="row-between mt-12">
0371:         <Pager
0372:           page={page}
0373:           totalPages={totalPages}
0374:           pageSize={pageSize}
0375:           totalItems={sortedItems.length}
0376:           onPageChange={setPage}
0377:           onPageSizeChange={(ps) => {
0378:             setPageSize(ps);
0379:             setPage(1);
0380:           }}
0381:           showPageSize
0382:         />
0383:       </div>
0384:
```

```
0385:       <DataTable
0386:         rows={pagedRows}
0387:         columns={columns}
0388:         sortKey={sortKey}
0389:         sortDir={sortDir}
0390:         className="compact"
0391:         wrapperClassName="table-scroller"
0392:         onSortChange={(k, d) => {
0393:           setSortKey(k);
0394:           setSortDir(d);
0395:           setPage(1);
0396:         }}
0397:         loading={loading}
0398:         emptyText="Nenhum movimento encontrado."
0399:       />
0400:       </div>
0401:     </>
0402:   );
0403: }
```

==========
ARQUIVO: frontend/src/pages/financeiroCategorias.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { useSearchParams } from "react-router-dom";
0003: import { Toolbar } from "../components/Toolbar";
0004: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0005: import { Pager } from "../components/Pager";
0006: import { PageHero } from "../components/PageHero";
0007: import { getFilial, onFilialChange } from "../state/filial";
0008: import categoriesIcon from "../assets/shell-icons/categories.png";
0009: import categoriesBg from "../assets/hero-backgrounds/categories.jpg";
0010: import {
0011:   listarCategorias,
0012:   criarCategoria,
0013:   definirCategoriaMov,
0014:   obterMovimento,
0015:   type Categoria,
0016:   type MovimentoFinanceiro,
0017: } from "../api/financeiro/categorias";
0018:
0019: function toNumber(value: unknown, fallback = 0): number {
0020:   const n = Number(value);
0021:   return Number.isFinite(n) ? n : fallback;
0022: }
0023:
0024: function toStringValue(value: unknown, fallback = ""): string {
0025:   if (value === null || value === undefined) return fallback;
0026:   const s = String(value).trim();
0027:   return s ? s : fallback;
0028: }
0029:
0030: export function FinanceiroCategorias() {
0031:   const [searchParams, setSearchParams] = useSearchParams();
```

```
0032:   const [filialView, setFilialView] = useState(getFilial());
0033:   const [ativas, setAtivas] = useState(true);
0034:   const [loading, setLoading] = useState(false);
0035:   const [err, setErr] = useState<string | null>(null);
0036:   const [okMsg, setOkMsg] = useState<string | null>(null);
0037:   const [items, setItems] = useState<Categoria[]>([]);
0038:
0039:   const [page, setPage] = useState(1);
0040:   const [pageSize, setPageSize] = useState(10);
0041:
0042:   const [nome, setNome] = useState("");
0043:   const [tipo, setTipo] = useState("RECEITA");
0044:
0045:   const [movId, setMovId] = useState("");
0046:   const [categId, setCategId] = useState("");
0047:   const [movimento, setMovimento] = useState<MovimentoFinanceiro | null>(null);
0048:
0049:   const [sortKey, setSortKey] = useState("nome");
0050:   const [sortDir, setSortDir] = useState<SortDir>("asc");
0051:
0052:   function getErrorMessage(error: unknown, fallback: string): string {
0053:     if (error instanceof Error && error.message) return error.message;
0054:     return fallback;
0055:   }
0056:
0057:   async function load() {
0058:     setErr(null);
0059:     setLoading(true);
0060:     try {
0061:       const filial = getFilial();
0062:       const data = await listarCategorias({ filial, ativas });
0063:       setItems(data);
0064:       setPage(1);
0065:     } catch (error: unknown) {
0066:       setErr(getErrorMessage(error, "Erro ao carregar categorias"));
0067:     } finally {
0068:       setLoading(false);
0069:     }
0070:   }
0071:
0072:   async function saveCategoria() {
0073:     setErr(null);
0074:     setOkMsg(null);
0075:     setLoading(true);
0076:     try {
0077:       if (!nome.trim()) throw new Error("Nome da categoria e obrigatorio.");
0078:       const result = await criarCategoria({ filial: getFilial(), nome: nome.trim(), tipo });
0079:       setNome("");
0080:       if (result.id) {
0081:         setOkMsg(`Categoria criada com sucesso. ID gerado: ${result.id}.`);
0082:       } else {
0083:         setOkMsg("Categoria criada com sucesso.");
0084:       }
0085:       await load();
```

```
0086:     } catch (error: unknown) {
0087:       setErr(getErrorMessage(error, "Erro ao criar categoria"));
0088:     } finally {
0089:       setLoading(false);
0090:     }
0091:   }
0092:
0093:   async function aplicarCategoria() {
0094:     setErr(null);
0095:     setOkMsg(null);
0096:     setLoading(true);
0097:     try {
0098:       const mid = toNumber(movId);
0099:       if (!mid) throw new Error("Informe o ID do movimento.");
0100:       const cid = categId ? toNumber(categId) : null;
0101:       const result = await definirCategoriaMov({ mov_id: mid, categ_id: cid || null });
0102:       const resultMovId = toNumber((result as Record<string, unknown>).mov_id, mid);
0103:       const resultCategId = (result as Record<string, unknown>).categ_id;
0104:       setOkMsg(
0105:         resultCategId
0106:           ? `Categoria aplicada com sucesso. Movimento ${resultMovId} vinculado a categoria ${
resultCategId}.`
0107:           : `Categoria removida com sucesso do movimento ${resultMovId}.`
0108:       );
0109:       setMovimento(await obterMovimento(resultMovId));
0110:       setMovId("");
0111:       setCategId("");
0112:     } catch (error: unknown) {
0113:       setErr(getErrorMessage(error, "Erro ao definir categoria"));
0114:     } finally {
0115:       setLoading(false);
0116:     }
0117:   }
0118:
0119:   async function consultarMovimento() {
0120:     setErr(null);
0121:     setOkMsg(null);
0122:     setLoading(true);
0123:     try {
0124:       const mid = toNumber(movId);
0125:       if (!mid) throw new Error("Informe o ID do movimento.");
0126:       setMovimento(await obterMovimento(mid));
0127:       setOkMsg(`Movimento ${mid} consultado com sucesso.`);
0128:     } catch (error: unknown) {
0129:       setMovimento(null);
0130:       setErr(getErrorMessage(error, "Erro ao consultar movimento"));
0131:     } finally {
0132:       setLoading(false);
0133:     }
0134:   }
0135:
0136:   useEffect(() => {
0137:     load();
0138:
```

```
0139:     const unsub = onFilialChange((novaFilial) => {
0140:       setFilialView(novaFilial);
0141:       setErr(null);
0142:       setAtivas(true);
0143:       setPage(1);
0144:       load();
0145:     });
0146:     return unsub;
0147:     // eslint-disable-next-line react-hooks/exhaustive-deps
0148:   }, []);
0149:
0150:   useEffect(() => {
0151:     const movIdParam = searchParams.get("mov_id") ?? "";
0152:     if (!movIdParam) return;
0153:     setMovId(movIdParam);
0154:     setErr(null);
0155:     setOkMsg(null);
0156:     obterMovimento(Number(movIdParam))
0157:       .then((mov) => {
0158:         setMovimento(mov);
0159:         setOkMsg(`Movimento ${movIdParam} carregado a partir da origem financeira.`);
0160:       })
0161:       .catch((error: unknown) => {
0162:         setMovimento(null);
0163:         setErr(getErrorMessage(error, "Erro ao consultar movimento"));
0164:       });
0165:   }, [searchParams]);
0166:
0167:   const columns: Column<Categoria>[] = useMemo(
0168:     () => [
0169:       { key: "id", header: "ID", sortable: true, value: (x) => toNumber(x.ID) },
0170:       {
0171:         key: "nome",
0172:         header: "Nome",
0173:         sortable: true,
0174:         value: (x) => toStringValue(x.C5_NOME ?? (x as Record<string, unknown>).nome, "-"),
0175:       },
0176:       {
0177:         key: "tipo",
0178:         header: "Tipo",
0179:         sortable: true,
0180:         value: (x) => toStringValue(x.C5_TIPO ?? (x as Record<string, unknown>).tipo, "-"),
0181:       },
0182:       {
0183:         key: "ativa",
0184:         header: "Ativa",
0185:         sortable: true,
0186:         className: "hide-sm",
0187:         value: (x) => String(x.C5_ATIVA ?? (x as Record<string, unknown>).ativa ?? 0),
0188:       },
0189:     ],
0190:     []
0191:   );
0192:
```

```
0193:   return (
0194:     <>
0195:       <PageHero
0196:         icon={categoriesIcon}
0197:         backgroundImage={categoriesBg}
0198:         backgroundPosition="center 40%"
0199:         eyebrow="Financeiro"
0200:         title="Categorias financeiras"
0201:         description="Estruture receitas, despesas, custos e transferencias para enriquecer cai
xa, AR, AP e demonstrativos."
0202:         metrics={[
0203:           { label: "Filial", value: filialView },
0204:           { label: "Categorias", value: String(items.length) },
0205:           { label: "Filtro", value: ativas ? "Ativas" : "Todas" },
0206:         ]}
0207:       />
0208:
0209:       <div className="card">
0210:       <Toolbar
0211:         left={
0212:           <div className="hint">
0213:             Filial: <b>{filialView}</b>
0214:           </div>
0215:         }
0216:         right={
0217:           <>
0218:             <select
0219:               className="select"
0220:               value={ativas ? "true" : "false"}
0221:               onChange={(e) => { setAtivas(e.target.value === "true"); setPage(1); }}
0222:             >
0223:               <option value="true">Ativas</option>
0224:               <option value="false">Todas</option>
0225:             </select>
0226:             <button type="button" className="btn" onClick={load} disabled={loading}>
0227:               Aplicar
0228:             </button>
0229:             <button
0230:               type="button"
0231:               className="btn ghost"
0232:               onClick={() => {
0233:                 setAtivas(true);
0234:                 setNome("");
0235:                 setTipo("RECEITA");
0236:                 setPage(1);
0237:               }}
0238:               disabled={loading}
0239:             >
0240:               Limpar
0241:             </button>
0242:             <button
0243:               type="button"
0244:               className="btn ghost"
0245:               onClick={() => {
```

```
0246:                 setMovimento(null);
0247:                 setMovId("");
0248:                 const next = new URLSearchParams(searchParams);
0249:                 next.delete("mov_id");
0250:                 setSearchParams(next);
0251:               }}
0252:               disabled={loading || !movimento}
0253:             >
0254:               Limpar consulta
0255:             </button>
0256:           </>
0257:         }
0258:       />
0259:
0260:       {okMsg && <div className="alert alert-success mt-12">{okMsg}</div>}
0261:       {err && <div className="alert mt-12">{err}</div>}
0262:
0263:       <div className="section-title">Resumo</div>
0264:       <div className="stats-grid">
0265:         <div className="stat-card">
0266:           <div>
0267:             <div className="kpi-label">Categorias</div>
0268:             <div className="kpi">{items.length}</div>
0269:           </div>
0270:           <div className="stat-icon">CT</div>
0271:         </div>
0272:         <div className="stat-card">
0273:           <div>
0274:             <div className="kpi-label">Filtro</div>
0275:             <div className="kpi">{ativas ? "Ativas" : "Todas"}</div>
0276:           </div>
0277:           <div className="stat-icon solid">FL</div>
0278:         </div>
0279:         <div className="stat-card">
0280:           <div>
0281:             <div className="kpi-label">Filial</div>
0282:             <div className="kpi">{filialView}</div>
0283:           </div>
0284:           <div className="stat-icon success">F</div>
0285:         </div>
0286:       </div>
0287:
0288:       <div className="form-grid mt-12">
0289:         <div className="form-field">
0290:           <label>Nome</label>
0291:           <input value={nome} onChange={(e) => setNome(e.target.value)} placeholder="Categoria
" />
0292:         </div>
0293:         <div className="form-field">
0294:           <label>Tipo</label>
0295:           <select className="select" value={tipo} onChange={(e) => setTipo(e.target.value)}>
0296:             <option value="RECEITA">RECEITA</option>
0297:             <option value="DESPESA">DESPESA</option>
0298:             <option value="CUSTO">CUSTO</option>
```

```
0299:             <option value="TRANSFERENCIA">TRANSFERENCIA</option>
0300:             <option value="OUTROS">OUTROS</option>
0301:           </select>
0302:         </div>
0303:         <div className="form-field">
0304:           <label> </label>
0305:           <button type="button" className="btn primary" onClick={saveCategoria} disabled={load
ing}>
0306:             Criar categoria
0307:           </button>
0308:         </div>
0309:       </div>
0310:
0311:       <div className="form-grid mt-12">
0312:         <div className="form-field">
0313:           <label>ID do movimento</label>
0314:           <input value={movId} onChange={(e) => setMovId(e.target.value)} placeholder="Ex: 123
" />
0315:         </div>
0316:         <div className="form-field">
0317:           <label>ID da categoria (opcional)</label>
0318:           <input
0319:             value={categId}
0320:             onChange={(e) => setCategId(e.target.value)}
0321:             placeholder="Ex: 5"
0322:           />
0323:         </div>
0324:         <div className="form-field">
0325:           <label> </label>
0326:           <div className="row gap-10">
0327:             <button type="button" className="btn" onClick={aplicarCategoria} disabled={loading
}>
0328:               Definir categoria
0329:             </button>
0330:             <button type="button" className="btn ghost" onClick={consultarMovimento} disabled=
{loading}>
0331:               Consultar movimento
0332:             </button>
0333:           </div>
0334:         </div>
0335:       </div>
0336:
0337:       {movimento && (
0338:         <>
0339:           <div className="section-title mt-12">Movimento consultado</div>
0340:           <div className="stats-grid">
0341:             <div className="stat-card">
0342:               <div>
0343:                 <div className="kpi-label">Movimento</div>
0344:                 <div className="kpi">{toNumber(movimento.ID)}</div>
0345:               </div>
0346:               <div className="stat-icon">MV</div>
0347:             </div>
0348:             <div className="stat-card">
```

```
0349:                <div>
0350:                  <div className="kpi-label">Categoria atual</div>
0351:                  <div className="kpi">
0352:                    {toNumber(movimento.E5_CATEG_ID) > 0
0353:                      ? `${toNumber(movimento.E5_CATEG_ID)} - ${toStringValue(movimento.CATEG_NO
ME, "-")}`
0354:                      : "Sem categoria"}
0355:                  </div>
0356:                </div>
0357:                <div className="stat-icon solid">CT</div>
0358:              </div>
0359:              <div className="stat-card">
0360:                <div>
0361:                  <div className="kpi-label">Valor</div>
0362:                  <div className="kpi">
0363:                    {toNumber(movimento.E5_VALOR).toLocaleString("pt-BR", {
0364:                      style: "currency",
0365:                      currency: "BRL",
0366:                    })}
0367:                  </div>
0368:                </div>
0369:                <div className="stat-icon success">{toStringValue(movimento.E5_TIPO, "-")}</div>
0370:              </div>
0371:            </div>
0372:
0373:            <div className="card mt-12">
0374:              <div className="detail-grid">
0375:                <div>
0376:                  <div className="kpi-label">Filial</div>
0377:                  <div>{toStringValue(movimento.E5_FILIAL, "-")}</div>
0378:                </div>
0379:                <div>
0380:                  <div className="kpi-label">Data</div>
0381:                  <div>{toStringValue(movimento.E5_DATA, "-")}</div>
0382:                </div>
0383:                <div>
0384:                  <div className="kpi-label">Origem</div>
0385:                  <div>
0386:                    {toStringValue(movimento.E5_ORIGEM_TIPO, "-")} #{toStringValue(movimento.E5_
ORIGEM_ID, "-")}
0387:                  </div>
0388:                </div>
0389:                <div>
0390:                  <div className="kpi-label">Usuario</div>
0391:                  <div>{toStringValue(movimento.E5_USUARIO, "-")}</div>
0392:                </div>
0393:                <div style={{ gridColumn: "1 / -1" }}>
0394:                  <div className="kpi-label">Historico</div>
0395:                  <div>{toStringValue(movimento.E5_HIST, "-")}</div>
0396:                </div>
0397:                <div>
0398:                  <div className="kpi-label">Tipo da categoria</div>
0399:                  <div>{toStringValue(movimento.CATEG_TIPO, "-")}</div>
0400:                </div>
```

```
0401:             </div>
0402:           </div>
0403:         </>
0404:       )}
0405:
0406:       <div className="row-between mt-12">
0407:         <Pager
0408:           page={page}
0409:           totalPages={Math.max(1, Math.ceil(items.length / pageSize))}
0410:           pageSize={pageSize}
0411:           totalItems={items.length}
0412:           onPageChange={setPage}
0413:           onPageSizeChange={(ps) => {
0414:             setPageSize(ps);
0415:             setPage(1);
0416:           }}
0417:           showPageSize
0418:         />
0419:       </div>
0420:
0421:       <DataTable
0422:         rows={items.slice((page - 1) * pageSize, (page - 1) * pageSize + pageSize)}
0423:         columns={columns}
0424:         sortKey={sortKey}
0425:         sortDir={sortDir}
0426:         className="compact"
0427:         wrapperClassName="table-scroller"
0428:         onSortChange={(k, d) => {
0429:           setSortKey(k);
0430:           setSortDir(d);
0431:           setPage(1);
0432:         }}
0433:         loading={loading}
0434:         emptyText="Nenhuma categoria encontrada."
0435:       />
0436:       </div>
0437:     </>
0438:   );
0439: }
```

==========
ARQUIVO: frontend/src/pages/FornecedoresList.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { Link, useNavigate } from "react-router-dom";
0003: import { Toolbar } from "../components/Toolbar";
0004: import { Modal } from "../components/Modal";
0005: import { ConfirmDialog } from "../components/ConfirmDialog";
0006: import { Pager } from "../components/Pager";
0007: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0008: import { PageHero } from "../components/PageHero";
0009: import partnersIcon from "../assets/shell-icons/partners.png";
0010: import suppliersBg from "../assets/hero-backgrounds/suppliers.webp";
0011:
```

```
0012: import {
0013:   listFornecedores,
0014:   buscarFornecedores,
0015:   createFornecedor,
0016:   updateFornecedor,
0017:   setFornecedorAtivo,
0018:   type Fornecedor,
0019: } from "../api/fornecedores";
0020: import { getFilial, onFilialChange } from "../state/filial";
0021:
0022: type FormState = {
0023:   cod: string;
0024:   nome: string;
0025:   doc: string;
0026:   email: string;
0027:   fone: string;
0028:   filial: string;
0029: };
0030:
0031: function emptyForm(): FormState {
0032:   return {
0033:     cod: "",
0034:     nome: "",
0035:     doc: "",
0036:     email: "",
0037:     fone: "",
0038:     filial: getFilial(),
0039:   };
0040: }
0041:
0042: type SortKey = "cod" | "nome" | "doc" | "ativo_sn";
0043:
0044: export function FornecedoresList() {
0045:   const navigate = useNavigate();
0046:   const [filialView, setFilialView] = useState(getFilial());
0047:
0048:   const [ativos, setAtivos] = useState(true);
0049:   const [q, setQ] = useState("");
0050:
0051:   const [loading, setLoading] = useState(false);
0052:   const [err, setErr] = useState<string | null>(null);
0053:   const [items, setItems] = useState<Fornecedor[]>([]);
0054:
0055:   const [page, setPage] = useState(1);
0056:   const [pageSize, setPageSize] = useState(10);
0057:
0058:   const [sortKey, setSortKey] = useState<SortKey>("nome");
0059:   const [sortDir, setSortDir] = useState<SortDir>("asc");
0060:
0061:   const [modalOpen, setModalOpen] = useState(false);
0062:   const [editingCod, setEditingCod] = useState<string | null>(null);
0063:   const [form, setForm] = useState<FormState>(() => emptyForm());
0064:
0065:   const [confirmOpen, setConfirmOpen] = useState(false);
```

```
0066:   const [confirmFornecedor, setConfirmFornecedor] = useState<Fornecedor | null>(null);
0067:
0068:   const modalTitle = useMemo(
0069:     () => (editingCod ? `Editar Fornecedor: ${editingCod}` : "Novo Fornecedor"),
0070:     [editingCod]
0071:   );
0072:
0073:   function getErrorMessage(error: unknown, fallback: string): string {
0074:     if (error instanceof Error && error.message) return error.message;
0075:     return fallback;
0076:   }
0077:
0078:   function askToggleAtivo(f: Fornecedor) {
0079:     setConfirmFornecedor(f);
0080:     setConfirmOpen(true);
0081:   }
0082:
0083:   async function load() {
0084:     setErr(null);
0085:     setLoading(true);
0086:     try {
0087:       const filial = getFilial();
0088:       const data = await listFornecedores({ ativos, filial });
0089:       setItems(data);
0090:       setPage(1);
0091:     } catch (error: unknown) {
0092:       setErr(getErrorMessage(error, "Erro ao carregar"));
0093:     } finally {
0094:       setLoading(false);
0095:     }
0096:   }
0097:
0098:   async function doSearch() {
0099:     setErr(null);
0100:     setLoading(true);
0101:     try {
0102:       if (!q.trim()) {
0103:         await load();
0104:         return;
0105:       }
0106:       const filial = getFilial();
0107:       const data = await buscarFornecedores({ q: q.trim(), filial });
0108:       setItems(data);
0109:       setPage(1);
0110:     } catch (error: unknown) {
0111:       setErr(getErrorMessage(error, "Erro na busca"));
0112:     } finally {
0113:       setLoading(false);
0114:     }
0115:   }
0116:
0117:   useEffect(() => {
0118:     load();
0119:
```

```
0120:     const unsub = onFilialChange((novaFilial) => {
0121:       setFilialView(novaFilial);
0122:       setQ("");
0123:       setErr(null);
0124:
0125:       setModalOpen(false);
0126:       setEditingCod(null);
0127:       setForm(emptyForm());
0128:
0129:       setConfirmOpen(false);
0130:       setConfirmFornecedor(null);
0131:
0132:       setPage(1);
0133:       load();
0134:     });
0135:
0136:     return unsub;
0137:     // eslint-disable-next-line react-hooks/exhaustive-deps
0138:   }, []);
0139:
0140:   function openNew() {
0141:     setEditingCod(null);
0142:     setForm(emptyForm());
0143:     setModalOpen(true);
0144:   }
0145:
0146:   function openEdit(f: Fornecedor) {
0147:     setEditingCod(f.cod);
0148:     setForm({
0149:       cod: f.cod,
0150:       nome: f.nome ?? "",
0151:       doc: f.doc ?? "",
0152:       email: f.email ?? "",
0153:       fone: f.fone ?? "",
0154:       filial: f.filial ?? getFilial(),
0155:     });
0156:     setModalOpen(true);
0157:   }
0158:
0159:   async function save() {
0160:     setErr(null);
0161:     setLoading(true);
0162:     try {
0163:       if (!form.cod.trim() || !form.nome.trim()) {
0164:         throw new Error("Código e Nome são obrigatórios.");
0165:       }
0166:
0167:       if (editingCod) {
0168:         await updateFornecedor(editingCod, {
0169:           filial: form.filial,
0170:           nome: form.nome,
0171:           doc: form.doc || null,
0172:           email: form.email || null,
0173:           fone: form.fone || null,
```

```
0174:         });
0175:       } else {
0176:         await createFornecedor({
0177:           filial: form.filial,
0178:           cod: form.cod,
0179:           nome: form.nome,
0180:           doc: form.doc || null,
0181:           email: form.email || null,
0182:           fone: form.fone || null,
0183:         });
0184:       }
0185:
0186:       setModalOpen(false);
0187:       await load();
0188:     } catch (error: unknown) {
0189:       setErr(getErrorMessage(error, "Erro ao salvar"));
0190:     } finally {
0191:       setLoading(false);
0192:     }
0193:   }
0194:
0195:   async function toggleAtivo(f: Fornecedor) {
0196:     setErr(null);
0197:     setLoading(true);
0198:     try {
0199:       const ativoAtual = (f.ativo_sn ?? "S") === "S";
0200:       await setFornecedorAtivo(f.cod, {
0201:         ativo: !ativoAtual,
0202:         filial: f.filial ?? getFilial(),
0203:       });
0204:       await load();
0205:     } catch (error: unknown) {
0206:       setErr(getErrorMessage(error, "Erro ao alterar ativo"));
0207:     } finally {
0208:       setLoading(false);
0209:     }
0210:   }
0211:
0212:   function norm(v: unknown) {
0213:     return (v ?? "").toString().toLowerCase().trim();
0214:   }
0215:
0216:   function getSortValue(item: Fornecedor, key: SortKey): unknown {
0217:     if (key === "ativo_sn") return item.ativo_sn ?? "S";
0218:     if (key === "cod") return item.cod;
0219:     if (key === "nome") return item.nome;
0220:     return item.doc;
0221:   }
0222:
0223:   const sortedItems = useMemo(() => {
0224:     const arr = [...items];
0225:     arr.sort((a, b) => {
0226:       const av = norm(getSortValue(a, sortKey));
0227:       const bv = norm(getSortValue(b, sortKey));
```

```
0228:
0229:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0230:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0231:
0232:       return norm(a.cod).localeCompare(norm(b.cod));
0233:     });
0234:     return arr;
0235:   }, [items, sortKey, sortDir]);
0236:
0237:   const totalPages = useMemo(() => {
0238:     return Math.max(1, Math.ceil(sortedItems.length / pageSize));
0239:   }, [sortedItems.length, pageSize]);
0240:
0241:   const pagedRows = useMemo(() => {
0242:     const safePage = Math.min(Math.max(1, page), totalPages);
0243:     const start = (safePage - 1) * pageSize;
0244:     return sortedItems.slice(start, start + pageSize);
0245:   }, [sortedItems, page, pageSize, totalPages]);
0246:
0247:   const columns: Column<Fornecedor>[] = useMemo(
0248:     () => [
0249:       { key: "cod", header: "Código", sortable: true },
0250:       { key: "nome", header: "Nome", sortable: true },
0251:       { key: "doc", header: "Doc", sortable: true, className: "hide-sm" },
0252:       {
0253:         key: "ativo_sn",
0254:         header: "Ativo",
0255:         sortable: true,
0256:         value: (f) => f.ativo_sn ?? "S",
0257:         render: (f) =>
0258:           (f.ativo_sn ?? "S") === "S" ? (
0259:             <span className="badge success">
0260:               <span className="badge-dot" /> Ativo
0261:             </span>
0262:           ) : (
0263:             <span className="badge muted">
0264:               <span className="badge-dot" /> Inativo
0265:             </span>
0266:           ),
0267:       },
0268:       {
0269:         key: "acoes",
0270:         header: "Ações",
0271:         width: 340,
0272:         render: (f) => (
0273:           <div className="row gap-10">
0274:             <button className="btn" onClick={() => openEdit(f)}>
0275:               Editar
0276:             </button>
0277:             <button
0278:               className="btn"
0279:               onClick={() =>
0280:                 navigate(`/custos/fornecedor?fornecedor=${encodeURIComponent(f.cod)}`)
0281:               }
```

```
0282:             >
0283:               Custo
0284:             </button>
0285:             <button
0286:               className={`btn ${(f.ativo_sn ?? "S") === "S" ? "danger" : ""}`}
0287:               onClick={() => askToggleAtivo(f)}
0288:             >
0289:               {(f.ativo_sn ?? "S") === "S" ? "Inativar" : "Ativar"}
0290:             </button>
0291:           </div>
0292:         ),
0293:       },
0294:     ],
0295:     [navigate]
0296:   );
0297:
0298:   return (
0299:     <>
0300:       <PageHero
0301:         icon={partnersIcon}
0302:         backgroundImage={suppliersBg}
0303:         backgroundPosition="center 34%"
0304:         eyebrow="Cadastros"
0305:         title="Fornecedores"
0306:         description="Consulta operacional de fornecedores, com manutencao cadastral, filtros p
or status e controle por filial."
0307:         metrics={[
0308:           { label: "Filial", value: filialView },
0309:           { label: "Fornecedores", value: String(sortedItems.length) },
0310:           { label: "Ativos", value: String(sortedItems.filter((f) => (f.ativo_sn ?? "S") === "
S").length) },
0311:         ]}
0312:         actions={
0313:           <div className="row gap-10">
0314:             <Link to="/custos/fornecedor" className="btn">
0315:               Abrir custo
0316:             </Link>
0317:             <button type="button" className="btn primary" onClick={openNew}>Novo fornecedor</b
utton>
0318:           </div>
0319:         }
0320:       />
0321:
0322:       <div className="card">
0323:       <Toolbar
0324:         left={
0325:           <div className="hint">
0326:             Filial: <b>{filialView}</b> — {ativos ? "Mostrando apenas ativos" : "Mostrando tod
os"}
0327:           </div>
0328:         }
0329:         right={
0330:           <>
0331:             <input
```

```
0332:               className="input minw-240"
0333:               value={q}
0334:               onChange={(e) => setQ(e.target.value)}
0335:               placeholder="Buscar (código, nome, doc...)"
0336:               aria-label="Buscar fornecedores"
0337:             />
0338:
0339:             <button className="btn" onClick={doSearch} disabled={loading}>
0340:               Buscar
0341:             </button>
0342:
0343:             <select
0344:               className="select"
0345:               value={ativos ? "true" : "false"}
0346:               onChange={(e) => {
0347:                 setAtivos(e.target.value === "true");
0348:                 setPage(1);
0349:               }}
0350:               aria-label="Filtro ativos"
0351:             >
0352:               <option value="true">Ativos</option>
0353:               <option value="false">Todos</option>
0354:             </select>
0355:
0356:             <button className="btn" onClick={load} disabled={loading}>
0357:               Aplicar
0358:             </button>
0359:             <button
0360:               className="btn ghost"
0361:               onClick={() => {
0362:                 setQ("");
0363:                 setAtivos(true);
0364:                 setPage(1);
0365:               }}
0366:               disabled={loading}
0367:             >
0368:               Limpar
0369:             </button>
0370:
0371:             <button className="btn primary" onClick={openNew}>
0372:               Novo
0373:             </button>
0374:           </>
0375:         }
0376:       />
0377:
0378:       {err && <div className="alert mt-12">{err}</div>}
0379:
0380:       <div className="section-title">Resumo</div>
0381:       <div className="stats-grid">
0382:         <div className="stat-card">
0383:           <div>
0384:             <div className="kpi-label">Fornecedores (filtrados)</div>
0385:             <div className="kpi">{sortedItems.length}</div>
```

```
0386:           </div>
0387:           <div className="stat-icon">FR</div>
0388:         </div>
0389:         <div className="stat-card">
0390:           <div>
0391:             <div className="kpi-label">Ativos no filtro</div>
0392:             <div className="kpi">
0393:               {sortedItems.filter((f) => (f.ativo_sn ?? "S") === "S").length}
0394:             </div>
0395:           </div>
0396:           <div className="stat-icon success">OK</div>
0397:         </div>
0398:         <div className="stat-card">
0399:           <div>
0400:             <div className="kpi-label">Filial</div>
0401:             <div className="kpi">{filialView}</div>
0402:           </div>
0403:           <div className="stat-icon solid">F</div>
0404:         </div>
0405:       </div>
0406:
0407:       <div className="row-between mt-12">
0408:         <div className="hint">
0409:           <Pager
0410:             page={page}
0411:             totalPages={totalPages}
0412:             pageSize={pageSize}
0413:             totalItems={sortedItems.length}
0414:             onPageChange={(p) => setPage(p)}
0415:             onPageSizeChange={(ps) => {
0416:               setPageSize(ps);
0417:               setPage(1);
0418:             }}
0419:             showPageSize
0420:           />
0421:         </div>
0422:       </div>
0423:
0424:       <DataTable
0425:         rows={pagedRows}
0426:         columns={columns}
0427:         sortKey={sortKey}
0428:         sortDir={sortDir}
0429:         className="compact"
0430:         wrapperClassName="table-scroller"
0431:         onSortChange={(k, d) => {
0432:           setSortKey(k as SortKey);
0433:           setSortDir(d);
0434:           setPage(1);
0435:         }}
0436:         loading={loading}
0437:         emptyText="Nenhum fornecedor encontrado."
0438:       />
0439:
```

```
0440:       {modalOpen && (
0441:         <Modal title={modalTitle} onClose={() => setModalOpen(false)}>
0442:           <div className="form-grid">
0443:             <div className="form-field">
0444:               <label>Código</label>
0445:               <input
0446:                 value={form.cod}
0447:                 onChange={(e) => setForm({ ...form, cod: e.target.value })}
0448:                 disabled={!!editingCod}
0449:               />
0450:             </div>
0451:
0452:             <div className="form-field">
0453:               <label>Filial</label>
0454:               <input
0455:                 value={form.filial}
0456:                 onChange={(e) => setForm({ ...form, filial: e.target.value })}
0457:               />
0458:             </div>
0459:
0460:             <div className="form-field col-span-all">
0461:               <label>Nome</label>
0462:               <input value={form.nome} onChange={(e) => setForm({ ...form, nome: e.target.valu
e })} />
0463:             </div>
0464:
0465:             <div className="form-field">
0466:               <label>Documento</label>
0467:               <input value={form.doc} onChange={(e) => setForm({ ...form, doc: e.target.value
})} />
0468:             </div>
0469:
0470:             <div className="form-field">
0471:               <label>Email</label>
0472:               <input value={form.email} onChange={(e) => setForm({ ...form, email: e.target.va
lue })} />
0473:             </div>
0474:
0475:             <div className="form-field">
0476:               <label>Fone</label>
0477:               <input value={form.fone} onChange={(e) => setForm({ ...form, fone: e.target.valu
e })} />
0478:             </div>
0479:           </div>
0480:
0481:           <div className="form-actions">
0482:             <button className="btn" onClick={() => setModalOpen(false)} disabled={loading}>
0483:               Cancelar
0484:             </button>
0485:             <button className="btn primary" onClick={save} disabled={loading}>
0486:               Salvar
0487:             </button>
0488:           </div>
0489:         </Modal>
```

```
0490:       )}
0491:
0492:       {confirmOpen && confirmFornecedor && (
0493:         <ConfirmDialog
0494:           title={(confirmFornecedor.ativo_sn ?? "S") === "S" ? "Inativar fornecedor" : "Ativar
 fornecedor"}
0495:           message={
0496:             (confirmFornecedor.ativo_sn ?? "S") === "S"
0497:               ? `Deseja realmente INATIVAR o fornecedor ${confirmFornecedor.cod} - ${confirmFo
rnecedor.nome}?`
0498:               : `Deseja realmente ATIVAR o fornecedor ${confirmFornecedor.cod} - ${confirmForn
ecedor.nome}?`
0499:           }
0500:           confirmText={(confirmFornecedor.ativo_sn ?? "S") === "S" ? "Inativar" : "Ativar"}
0501:           cancelText="Cancelar"
0502:           danger={(confirmFornecedor.ativo_sn ?? "S") === "S"}
0503:           loading={loading}
0504:           onClose={() => {
0505:             setConfirmOpen(false);
0506:             setConfirmFornecedor(null);
0507:           }}
0508:           onConfirm={async () => {
0509:             await toggleAtivo(confirmFornecedor);
0510:             setConfirmOpen(false);
0511:             setConfirmFornecedor(null);
0512:           }}
0513:         />
0514:       )}
0515:       </div>
0516:     </>
0517:   );
0518: }
```

==========
ARQUIVO: frontend/src/pages/Login.tsx
==========

```
0001: import { useState } from "react";
0002: import { useNavigate } from "react-router-dom";
0003: import { useAuth } from "../auth/AuthContext";
0004:
0005: export function Login() {
0006:   const { login } = useAuth();
0007:   const navigate = useNavigate();
0008:   const [username, setUsername] = useState("");
0009:   const [password, setPassword] = useState("");
0010:   const [err, setErr] = useState<string | null>(null);
0011:   const [loading, setLoading] = useState(false);
0012:
0013:   function getErrorMessage(error: unknown, fallback: string): string {
0014:     if (error instanceof Error && error.message) return error.message;
0015:     return fallback;
0016:   }
0017:
0018:   return (
```

```
0019:     <div className="login-shell">
0020:       <div className="login-backdrop" />
0021:
0022:       <section className="login-showcase">
0023:         <div className="login-showcase-inner">
0024:           <div className="login-badge">ERP operacional com vendas, financeiro e reservas</div>
0025:           <h1 className="login-title">Mini Protheus</h1>
0026:           <p className="login-copy">
0027:             Ambiente de operacao para pedidos, parceiros, contas a receber, contas a pagar,
0028:             caixa, auditoria e relatorios gerenciais.
0029:           </p>
0030:
0031:           <div className="showcase-grid">
0032:             <article className="showcase-card">
0033:               <div className="showcase-card-label">Comercial</div>
0034:               <div className="showcase-card-value">Pedidos, faturamento e reservas</div>
0035:             </article>
0036:             <article className="showcase-card">
0037:               <div className="showcase-card-label">Financeiro</div>
0038:               <div className="showcase-card-value">AR, AP, caixa e categorias</div>
0039:             </article>
0040:             <article className="showcase-card">
0041:               <div className="showcase-card-label">Governanca</div>
0042:               <div className="showcase-card-value">Usuarios, bootstrap e auditoria</div>
0043:             </article>
0044:           </div>
0045:         </div>
0046:       </section>
0047:
0048:       <section className="login-panel">
0049:         <div className="login-card">
0050:           <div className="login-card-top">
0051:             <div className="login-card-mark">MP</div>
0052:             <div>
0053:               <div className="login-card-title">Acesso ao sistema</div>
0054:               <div className="login-card-subtitle">Use sua credencial para entrar no ambiente
ERP</div>
0055:             </div>
0056:           </div>
0057:
0058:           {err && <div className="alert login-alert">{err}</div>}
0059:
0060:           <div className="form-field">
0061:             <label>Usuario</label>
0062:             <input
0063:               className="input"
0064:               value={username}
0065:               onChange={(event) => setUsername(event.target.value)}
0066:               placeholder="admin"
0067:             />
0068:           </div>
0069:
0070:           <div className="form-field">
0071:             <label>Senha</label>
```

```
0072:             <input
0073:               className="input"
0074:               type="password"
0075:               value={password}
0076:               onChange={(event) => setPassword(event.target.value)}
0077:               placeholder="Digite sua senha"
0078:             />
0079:           </div>
0080:
0081:           <div className="login-helper-row">
0082:             <span className="login-helper-label">Backend</span>
0083:             <span className="login-helper-value">JWT + FastAPI + SQL Server</span>
0084:           </div>
0085:
0086:           <button
0087:             className="btn primary login-submit"
0088:             disabled={loading}
0089:             onClick={async () => {
0090:               setErr(null);
0091:               setLoading(true);
0092:               try {
0093:                 await login(username, password);
0094:                 navigate("/clientes");
0095:               } catch (error: unknown) {
0096:                 setErr(getErrorMessage(error, "Falha no login"));
0097:               } finally {
0098:                 setLoading(false);
0099:               }
0100:             }}
0101:           >
0102:             {loading ? "Entrando..." : "Entrar no ERP"}
0103:           </button>
0104:         </div>
0105:       </section>
0106:     </div>
0107:   );
0108: }
```

==========
ARQUIVO: frontend/src/pages/parceiros.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { Toolbar } from "../components/Toolbar";
0003: import { Pager } from "../components/Pager";
0004: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0005: import { PageHero } from "../components/PageHero";
0006: import { getFilial, onFilialChange } from "../state/filial";
0007: import partnersIcon from "../assets/shell-icons/partners.png";
0008: import partnersBg from "../assets/hero-backgrounds/partners.jpg";
0009: import {
0010:   listarSa1Clientes,
0011:   obterSa1Cliente,
0012:   criarSa1Cliente,
0013:   inativarSa1Cliente,
```

```
0014:   setTabelaSa1Cliente,
0015:   listarSa2Fornecedores,
0016:   obterSa2Fornecedor,
0017:   criarSa2Fornecedor,
0018:   inativarSa2Fornecedor,
0019: } from "../api/parceiros";
0020: import {
0021:   buscarPrecoTabela,
0022:   criarTabelaPreco,
0023:   definirPrecoTabela,
0024:   listarItensTabelaPreco,
0025:   listarTabelasPreco,
0026:   type ItemPrecoTabela,
0027:   type TabelaPreco,
0028: } from "../api/precos";
0029:
0030: type TabKey = "clientes" | "fornecedores" | "precos";
0031:
0032: type ClienteForm = {
0033:   cod: string;
0034:   nome: string;
0035:   doc: string;
0036:   email: string;
0037:   tel: string;
0038:   end: string;
0039:   mun: string;
0040:   uf: string;
0041:   cep: string;
0042:   tabelaId: string;
0043:   filial: string;
0044: };
0045:
0046: type FornForm = {
0047:   cod: string;
0048:   nome: string;
0049:   doc: string;
0050:   email: string;
0051:   tel: string;
0052:   end: string;
0053:   mun: string;
0054:   uf: string;
0055:   cep: string;
0056:   filial: string;
0057: };
0058:
0059: type TabelaForm = {
0060:   codigo: string;
0061:   descricao: string;
0062: };
0063:
0064: type PrecoForm = {
0065:   tabelaId: string;
0066:   produto: string;
0067:   preco: string;
```

```
0068:   dtIni: string;
0069: };
0070:
0071: function emptyCliente(): ClienteForm {
0072:   return {
0073:     cod: "",
0074:     nome: "",
0075:     doc: "",
0076:     email: "",
0077:     tel: "",
0078:     end: "",
0079:     mun: "",
0080:     uf: "",
0081:     cep: "",
0082:     tabelaId: "",
0083:     filial: getFilial(),
0084:   };
0085: }
0086:
0087: function emptyFornecedor(): FornForm {
0088:   return {
0089:     cod: "",
0090:     nome: "",
0091:     doc: "",
0092:     email: "",
0093:     tel: "",
0094:     end: "",
0095:     mun: "",
0096:     uf: "",
0097:     cep: "",
0098:     filial: getFilial(),
0099:   };
0100: }
0101:
0102: function emptyTabela(): TabelaForm {
0103:   return {
0104:     codigo: "",
0105:     descricao: "",
0106:   };
0107: }
0108:
0109: function todayIso() {
0110:   return new Date().toISOString().slice(0, 10);
0111: }
0112:
0113: function emptyPrecoForm(): PrecoForm {
0114:   return {
0115:     tabelaId: "",
0116:     produto: "",
0117:     preco: "",
0118:     dtIni: todayIso(),
0119:   };
0120: }
0121:
```

```
0122: function toNumber(value: string, fallback = 0) {
0123:   const n = Number(value);
0124:   return Number.isFinite(n) ? n : fallback;
0125: }
0126:
0127: function toStringValue(value: unknown, fallback = ""): string {
0128:   if (value === null || value === undefined) return fallback;
0129:   const s = String(value).trim();
0130:   return s ? s : fallback;
0131: }
0132:
0133: function normalize(v: unknown) {
0134:   if (v === null || v === undefined) return "";
0135:   if (typeof v === "boolean") return v ? "1" : "0";
0136:   if (typeof v === "number") return String(v).padStart(20, "0");
0137:   return String(v).toLowerCase().trim();
0138: }
0139:
0140: function getErrorMessage(error: unknown, fallback: string): string {
0141:   if (error instanceof Error && error.message) return error.message;
0142:   return fallback;
0143: }
0144:
0145: export function Parceiros() {
0146:   const [tab, setTab] = useState<TabKey>("clientes");
0147:   const [filialView, setFilialView] = useState(getFilial());
0148:
0149:   const [loading, setLoading] = useState(false);
0150:   const [err, setErr] = useState<string | null>(null);
0151:   const [lastResp, setLastResp] = useState<Record<string, unknown> | null>(null);
0152:
0153:   const [clientes, setClientes] = useState<Record<string, unknown>[]>([]);
0154:   const [fornecedores, setFornecedores] = useState<Record<string, unknown>[]>([]);
0155:   const [tabelas, setTabelas] = useState<TabelaPreco[]>([]);
0156:   const [itensTabela, setItensTabela] = useState<ItemPrecoTabela[]>([]);
0157:
0158:   const [cliAtivo, setCliAtivo] = useState(true);
0159:   const [cliQ, setCliQ] = useState("");
0160:   const [fornAtivo, setFornAtivo] = useState(true);
0161:   const [fornQ, setFornQ] = useState("");
0162:
0163:   const [cliPage, setCliPage] = useState(1);
0164:   const [cliPageSize, setCliPageSize] = useState(10);
0165:   const [fornPage, setFornPage] = useState(1);
0166:   const [fornPageSize, setFornPageSize] = useState(10);
0167:
0168:   const [cliForm, setCliForm] = useState<ClienteForm>(emptyCliente());
0169:   const [fornForm, setFornForm] = useState<FornForm>(emptyFornecedor());
0170:   const [tabelaForm, setTabelaForm] = useState<TabelaForm>(emptyTabela());
0171:   const [precoForm, setPrecoForm] = useState<PrecoForm>(emptyPrecoForm());
0172:
0173:   const [cliBuscaCod, setCliBuscaCod] = useState("");
0174:   const [fornBuscaCod, setFornBuscaCod] = useState("");
0175:   const [cliDelCod, setCliDelCod] = useState("");
```

```
0176:   const [fornDelCod, setFornDelCod] = useState("");
0177:   const [cliSetTabelaCod, setCliSetTabelaCod] = useState("");
0178:   const [cliSetTabelaId, setCliSetTabelaId] = useState("");
0179:   const [precoLookupProduto, setPrecoLookupProduto] = useState("");
0180:   const [precoLookupTabelaId, setPrecoLookupTabelaId] = useState("");
0181:   const [precoLookupData, setPrecoLookupData] = useState(todayIso());
0182:
0183:   const [sortKey, setSortKey] = useState("cod");
0184:   const [sortDir, setSortDir] = useState<SortDir>("asc");
0185:
0186:   async function loadClientes() {
0187:     setErr(null);
0188:     setLoading(true);
0189:     try {
0190:       const data = await listarSa1Clientes({ filial: filialView, ativo: cliAtivo, q: cliQ || u
ndefined });
0191:       setClientes(Array.isArray(data) ? data : []);
0192:     } catch (error: unknown) {
0193:       setErr(getErrorMessage(error, "Erro ao carregar clientes SA1"));
0194:     } finally {
0195:       setLoading(false);
0196:     }
0197:   }
0198:
0199:   async function loadFornecedores() {
0200:     setErr(null);
0201:     setLoading(true);
0202:     try {
0203:       const data = await listarSa2Fornecedores({ filial: filialView, ativo: fornAtivo, q: forn
Q || undefined });
0204:       setFornecedores(Array.isArray(data) ? data : []);
0205:     } catch (error: unknown) {
0206:       setErr(getErrorMessage(error, "Erro ao carregar fornecedores SA2"));
0207:     } finally {
0208:       setLoading(false);
0209:     }
0210:   }
0211:
0212:   async function loadTabelas() {
0213:     setErr(null);
0214:     setLoading(true);
0215:     try {
0216:       const data = await listarTabelasPreco({ filial: filialView });
0217:       setTabelas(Array.isArray(data) ? data : []);
0218:     } catch (error: unknown) {
0219:       setErr(getErrorMessage(error, "Erro ao carregar tabelas de preco"));
0220:     } finally {
0221:       setLoading(false);
0222:     }
0223:   }
0224:
0225:   async function loadItensTabela(tabelaId: number) {
0226:     setErr(null);
0227:     setLoading(true);
```

```
0228:     try {
0229:       const data = await listarItensTabelaPreco(tabelaId, { filial: filialView });
0230:       setItensTabela(Array.isArray(data) ? data : []);
0231:     } catch (error: unknown) {
0232:       setErr(getErrorMessage(error, "Erro ao carregar itens da tabela"));
0233:     } finally {
0234:       setLoading(false);
0235:     }
0236:   }
0237:
0238:   useEffect(() => {
0239:     if (tab === "clientes") loadClientes();
0240:     if (tab === "fornecedores") loadFornecedores();
0241:     if (tab === "precos") loadTabelas();
0242:     // eslint-disable-next-line react-hooks/exhaustive-deps
0243:   }, [tab, filialView, cliAtivo, cliQ, fornAtivo, fornQ]);
0244:
0245:   useEffect(() => {
0246:     const unsub = onFilialChange((nova) => {
0247:       setFilialView(nova);
0248:       setCliForm((f) => ({ ...f, filial: nova }));
0249:       setFornForm((f) => ({ ...f, filial: nova }));
0250:       setPrecoForm((f) => ({ ...f, tabelaId: "", dtIni: f.dtIni || todayIso() }));
0251:       setPrecoLookupTabelaId("");
0252:       setItensTabela([]);
0253:     });
0254:     return unsub;
0255:   }, []);
0256:
0257:   async function onCriarCliente() {
0258:     setErr(null);
0259:     setLoading(true);
0260:     try {
0261:       if (!cliForm.cod.trim() || !cliForm.nome.trim()) {
0262:         throw new Error("Codigo e nome sao obrigatorios.");
0263:       }
0264:       const resp = await criarSa1Cliente({
0265:         filial: cliForm.filial || filialView,
0266:         cod: cliForm.cod.trim(),
0267:         nome: cliForm.nome.trim(),
0268:         doc: cliForm.doc.trim() || null,
0269:         email: cliForm.email.trim() || null,
0270:         tel: cliForm.tel.trim() || null,
0271:         end: cliForm.end.trim() || null,
0272:         mun: cliForm.mun.trim() || null,
0273:         uf: cliForm.uf.trim() || null,
0274:         cep: cliForm.cep.trim() || null,
0275:         tabela_id: cliForm.tabelaId ? toNumber(cliForm.tabelaId, 0) : null,
0276:       });
0277:       setLastResp(resp);
0278:       setCliForm(emptyCliente());
0279:       await loadClientes();
0280:     } catch (error: unknown) {
0281:       setErr(getErrorMessage(error, "Falha ao criar cliente"));
```

```
0282:     } finally {
0283:       setLoading(false);
0284:     }
0285:   }
0286:
0287:   async function onCriarFornecedor() {
0288:     setErr(null);
0289:     setLoading(true);
0290:     try {
0291:       if (!fornForm.cod.trim() || !fornForm.nome.trim()) {
0292:         throw new Error("Codigo e nome sao obrigatorios.");
0293:       }
0294:       const resp = await criarSa2Fornecedor({
0295:         filial: fornForm.filial || filialView,
0296:         cod: fornForm.cod.trim(),
0297:         nome: fornForm.nome.trim(),
0298:         doc: fornForm.doc.trim() || null,
0299:         email: fornForm.email.trim() || null,
0300:         tel: fornForm.tel.trim() || null,
0301:         end: fornForm.end.trim() || null,
0302:         mun: fornForm.mun.trim() || null,
0303:         uf: fornForm.uf.trim() || null,
0304:         cep: fornForm.cep.trim() || null,
0305:       });
0306:       setLastResp(resp);
0307:       setFornForm(emptyFornecedor());
0308:       await loadFornecedores();
0309:     } catch (error: unknown) {
0310:       setErr(getErrorMessage(error, "Falha ao criar fornecedor"));
0311:     } finally {
0312:       setLoading(false);
0313:     }
0314:   }
0315:
0316:   async function onBuscarCliente(cod?: string) {
0317:     const target = cod ?? cliBuscaCod.trim();
0318:     setErr(null);
0319:     setLoading(true);
0320:     try {
0321:       if (!target) throw new Error("Informe o codigo do cliente.");
0322:       const resp = await obterSa1Cliente(target, { filial: filialView });
0323:       setLastResp(resp);
0324:     } catch (error: unknown) {
0325:       setErr(getErrorMessage(error, "Falha ao buscar cliente"));
0326:     } finally {
0327:       setLoading(false);
0328:     }
0329:   }
0330:
0331:   async function onBuscarFornecedor(cod?: string) {
0332:     const target = cod ?? fornBuscaCod.trim();
0333:     setErr(null);
0334:     setLoading(true);
0335:     try {
```

```
0336:       if (!target) throw new Error("Informe o codigo do fornecedor.");
0337:       const resp = await obterSa2Fornecedor(target, { filial: filialView });
0338:       setLastResp(resp);
0339:     } catch (error: unknown) {
0340:       setErr(getErrorMessage(error, "Falha ao buscar fornecedor"));
0341:     } finally {
0342:       setLoading(false);
0343:     }
0344:   }
0345:
0346:   async function onInativarCliente(cod?: string) {
0347:     const target = cod ?? cliDelCod.trim();
0348:     if (!target) {
0349:       setErr("Informe o codigo do cliente.");
0350:       return;
0351:     }
0352:     setErr(null);
0353:     setLoading(true);
0354:     try {
0355:       const resp = await inativarSa1Cliente(target, { filial: filialView });
0356:       setLastResp(resp);
0357:       setCliDelCod("");
0358:       await loadClientes();
0359:     } catch (error: unknown) {
0360:       setErr(getErrorMessage(error, "Falha ao inativar cliente"));
0361:     } finally {
0362:       setLoading(false);
0363:     }
0364:   }
0365:
0366:   async function onInativarFornecedor(cod?: string) {
0367:     const target = cod ?? fornDelCod.trim();
0368:     if (!target) {
0369:       setErr("Informe o codigo do fornecedor.");
0370:       return;
0371:     }
0372:     setErr(null);
0373:     setLoading(true);
0374:     try {
0375:       const resp = await inativarSa2Fornecedor(target, { filial: filialView });
0376:       setLastResp(resp);
0377:       setFornDelCod("");
0378:       await loadFornecedores();
0379:     } catch (error: unknown) {
0380:       setErr(getErrorMessage(error, "Falha ao inativar fornecedor"));
0381:     } finally {
0382:       setLoading(false);
0383:     }
0384:   }
0385:
0386:   async function onSetTabela() {
0387:     setErr(null);
0388:     setLoading(true);
0389:     try {
```

```
0390:       if (!cliSetTabelaCod.trim()) throw new Error("Informe o codigo do cliente.");
0391:       const resp = await setTabelaSa1Cliente({
0392:         cod: cliSetTabelaCod.trim(),
0393:         tabela_id: cliSetTabelaId ? toNumber(cliSetTabelaId, 0) : null,
0394:         filial: filialView,
0395:       });
0396:       setLastResp(resp);
0397:       setCliSetTabelaCod("");
0398:       setCliSetTabelaId("");
0399:       await loadClientes();
0400:     } catch (error: unknown) {
0401:       setErr(getErrorMessage(error, "Falha ao definir tabela"));
0402:     } finally {
0403:       setLoading(false);
0404:     }
0405:   }
0406:
0407:   async function onCriarTabela() {
0408:     setErr(null);
0409:     setLoading(true);
0410:     try {
0411:       if (!tabelaForm.codigo.trim() || !tabelaForm.descricao.trim()) {
0412:         throw new Error("Codigo e descricao da tabela sao obrigatorios.");
0413:       }
0414:       const resp = await criarTabelaPreco({
0415:         filial: filialView,
0416:         codigo: tabelaForm.codigo.trim(),
0417:         descricao: tabelaForm.descricao.trim(),
0418:       });
0419:       setLastResp(resp);
0420:       setTabelaForm(emptyTabela());
0421:       await loadTabelas();
0422:       if (resp && typeof resp.id === "number") {
0423:         setPrecoForm((f) => ({ ...f, tabelaId: String(resp.id) }));
0424:         setPrecoLookupTabelaId(String(resp.id));
0425:         await loadItensTabela(Number(resp.id));
0426:       }
0427:     } catch (error: unknown) {
0428:       setErr(getErrorMessage(error, "Falha ao criar tabela de preco"));
0429:     } finally {
0430:       setLoading(false);
0431:     }
0432:   }
0433:
0434:   async function onDefinirPreco() {
0435:     setErr(null);
0436:     setLoading(true);
0437:     try {
0438:       if (!precoForm.tabelaId || !precoForm.produto.trim() || !precoForm.preco || !precoForm.d
tIni) {
0439:         throw new Error("Tabela, produto, preco e data inicial sao obrigatorios.");
0440:       }
0441:       const resp = await definirPrecoTabela({
0442:         filial: filialView,
```

```
0443:         tabela_id: toNumber(precoForm.tabelaId, 0),
0444:         produto: precoForm.produto.trim(),
0445:         preco: toNumber(precoForm.preco, 0),
0446:         dt_ini: precoForm.dtIni,
0447:       });
0448:       setLastResp(resp);
0449:       setPrecoForm((f) => ({ ...emptyPrecoForm(), tabelaId: f.tabelaId }));
0450:       await loadItensTabela(toNumber(precoForm.tabelaId, 0));
0451:     } catch (error: unknown) {
0452:       setErr(getErrorMessage(error, "Falha ao definir preco"));
0453:     } finally {
0454:       setLoading(false);
0455:     }
0456:   }
0457:
0458:   async function onBuscarPrecoVigente() {
0459:     setErr(null);
0460:     setLoading(true);
0461:     try {
0462:       if (!precoLookupTabelaId || !precoLookupProduto.trim() || !precoLookupData) {
0463:         throw new Error("Tabela, produto e data sao obrigatorios.");
0464:       }
0465:       const resp = await buscarPrecoTabela({
0466:         filial: filialView,
0467:         tabela_id: toNumber(precoLookupTabelaId, 0),
0468:         produto: precoLookupProduto.trim(),
0469:         data: precoLookupData,
0470:       });
0471:       setLastResp(resp);
0472:     } catch (error: unknown) {
0473:       setErr(getErrorMessage(error, "Falha ao buscar preco vigente"));
0474:     } finally {
0475:       setLoading(false);
0476:     }
0477:   }
0478:
0479:   const clienteColumns: Column<Record<string, unknown>>[] = [
0480:       { key: "cod", header: "Codigo", sortable: true, value: (x) => toStringValue(x.A1_COD ??
x.cod, "") },
0481:       { key: "nome", header: "Nome", sortable: true, value: (x) => toStringValue(x.A1_NOME ??
x.nome, "") },
0482:       { key: "doc", header: "Doc", sortable: true, className: "hide-sm", value: (x) => toStrin
gValue(x.A1_DOC ?? x.doc, "-") },
0483:       { key: "email", header: "Email", sortable: true, className: "hide-sm", value: (x) => toS
tringValue(x.A1_EMAIL ?? x.email, "-") },
0484:       { key: "tel", header: "Tel", sortable: true, className: "hide-sm", value: (x) => toStrin
gValue(x.A1_TEL ?? x.tel, "-") },
0485:       {
0486:         key: "tabela",
0487:         header: "Tabela",
0488:         sortable: true,
0489:         className: "hide-sm",
0490:         value: (x) =>
```

```
0491:           `${toStringValue(x.TABELA_COD ?? x.tabela_cod ?? "-", "-")} ${toStringValue(x.TABELA
_DESC ?? x.tabela_desc ?? "", "")}`.trim(),
0492:       },
0493:       {
0494:         key: "acoes",
0495:         header: "Acoes",
0496:         render: (x) => (
0497:           <div className="row gap-10">
0498:             <button className="btn" onClick={() => onBuscarCliente(toStringValue(x.A1_COD ?? x
.cod, ""))}>
0499:               Detalhar
0500:             </button>
0501:             <button className="btn danger" onClick={() => onInativarCliente(toStringValue(x.A1
_COD ?? x.cod, ""))}>
0502:               Inativar
0503:             </button>
0504:           </div>
0505:         ),
0506:       },
0507:   ];
0508:
0509:   const fornecedorColumns: Column<Record<string, unknown>>[] = [
0510:       { key: "cod", header: "Codigo", sortable: true, value: (x) => toStringValue(x.A2_COD ??
x.cod, "") },
0511:       { key: "nome", header: "Nome", sortable: true, value: (x) => toStringValue(x.A2_NOME ??
x.nome, "") },
0512:       { key: "doc", header: "Doc", sortable: true, className: "hide-sm", value: (x) => toStrin
gValue(x.A2_DOC ?? x.doc, "-") },
0513:       { key: "email", header: "Email", sortable: true, className: "hide-sm", value: (x) => toS
tringValue(x.A2_EMAIL ?? x.email, "-") },
0514:       { key: "tel", header: "Tel", sortable: true, className: "hide-sm", value: (x) => toStrin
gValue(x.A2_TEL ?? x.tel, "-") },
0515:       {
0516:         key: "acoes",
0517:         header: "Acoes",
0518:         render: (x) => (
0519:           <div className="row gap-10">
0520:             <button className="btn" onClick={() => onBuscarFornecedor(toStringValue(x.A2_COD ?
? x.cod, ""))}>
0521:               Detalhar
0522:             </button>
0523:             <button className="btn danger" onClick={() => onInativarFornecedor(toStringValue(x
.A2_COD ?? x.cod, ""))}>
0524:               Inativar
0525:             </button>
0526:           </div>
0527:         ),
0528:       },
0529:   ];
0530:
0531:   const tabelaColumns: Column<TabelaPreco>[] = [
0532:     { key: "codigo", header: "Codigo", sortable: true, value: (x) => x.codigo },
0533:     { key: "descricao", header: "Descricao", sortable: true, value: (x) => x.descricao },
0534:     { key: "id", header: "ID", sortable: true, value: (x) => String(x.id) },
```

```
0535:     {
0536:       key: "ativa",
0537:       header: "Status",
0538:       sortable: true,
0539:       render: (x) => (
0540:         <span className={`badge ${x.ativa ? "success" : "muted"}`}>
0541:           <span className="badge-dot" />
0542:           {x.ativa ? "Ativa" : "Inativa"}
0543:         </span>
0544:       ),
0545:     },
0546:     {
0547:       key: "acoes",
0548:       header: "Acoes",
0549:       render: (x) => (
0550:         <button
0551:           className="btn"
0552:           onClick={() => {
0553:             setPrecoForm((f) => ({ ...f, tabelaId: String(x.id) }));
0554:             setPrecoLookupTabelaId(String(x.id));
0555:             void loadItensTabela(x.id);
0556:           }}
0557:         >
0558:           Usar tabela
0559:         </button>
0560:       ),
0561:     },
0562:   ];
0563:
0564:   const itemTabelaColumns: Column<ItemPrecoTabela>[] = [
0565:     { key: "produto", header: "Produto", sortable: true, value: (x) => x.produto },
0566:     {
0567:       key: "preco",
0568:       header: "Preco",
0569:       sortable: true,
0570:       value: (x) => x.preco,
0571:       render: (x) => x.preco.toLocaleString("pt-BR", { style: "currency", currency: "BRL" }),
0572:     },
0573:     { key: "dt_ini", header: "Dt. ini", sortable: true, value: (x) => x.dt_ini ?? "", render:
(x) => x.dt_ini ?? "-" },
0574:     { key: "dt_fim", header: "Dt. fim", sortable: true, value: (x) => x.dt_fim ?? "", render:
(x) => x.dt_fim ?? "Vigente" },
0575:     {
0576:       key: "vigente",
0577:       header: "Status",
0578:       sortable: true,
0579:       render: (x) => (
0580:         <span className={`badge ${x.vigente ? "success" : "muted"}`}>
0581:           <span className="badge-dot" />
0582:           {x.vigente ? "Vigente" : "Historico"}
0583:         </span>
0584:       ),
0585:     },
0586:   ];
```

```
0587:
0588:   const currentRows: Array<Record<string, unknown> | TabelaPreco> =
0589:     tab === "clientes" ? clientes : tab === "fornecedores" ? fornecedores : tabelas;
0590:   const currentCols: Column<Record<string, unknown> | TabelaPreco>[] =
0591:     tab === "clientes"
0592:       ? (clienteColumns as Column<Record<string, unknown> | TabelaPreco>[])
0593:       : tab === "fornecedores"
0594:         ? (fornecedorColumns as Column<Record<string, unknown> | TabelaPreco>[])
0595:         : (tabelaColumns as Column<Record<string, unknown> | TabelaPreco>[]);
0596:
0597:   const sortedRows = useMemo(() => {
0598:     const col = currentCols.find((c) => c.key === sortKey);
0599:     const valueFn = col?.value ?? ((r: Record<string, unknown>) => r[sortKey]);
0600:     const arr = [...currentRows];
0601:     arr.sort((a, b) => {
0602:       const av = normalize(valueFn(a));
0603:       const bv = normalize(valueFn(b));
0604:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0605:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0606:       return 0;
0607:     });
0608:     return arr;
0609:   }, [currentRows, currentCols, sortKey, sortDir]);
0610:
0611:   const totalPages = useMemo(() => {
0612:     const pageSize = tab === "clientes" ? cliPageSize : fornPageSize;
0613:     return Math.max(1, Math.ceil(sortedRows.length / pageSize));
0614:   }, [sortedRows.length, tab, cliPageSize, fornPageSize]);
0615:
0616:   const pagedRows = useMemo(() => {
0617:     const page = tab === "clientes" ? cliPage : fornPage;
0618:     const pageSize = tab === "clientes" ? cliPageSize : fornPageSize;
0619:     const safePage = Math.min(Math.max(1, page), totalPages);
0620:     const start = (safePage - 1) * pageSize;
0621:     return sortedRows.slice(start, start + pageSize);
0622:   }, [sortedRows, tab, cliPage, cliPageSize, fornPage, fornPageSize, totalPages]);
0623:
0624:   return (
0625:     <>
0626:       <PageHero
0627:         icon={partnersIcon}
0628:         backgroundImage={partnersBg}
0629:         backgroundPosition="center 38%"
0630:         eyebrow="Relacionamento"
0631:         title="Parceiros e precificacao"
0632:         description="Painel operacional para clientes, fornecedores e tabelas de preco em estr
utura Protheus, incluindo busca, inativacao, vinculo comercial e precificacao por vigencia."
0633:         metrics={[
0634:           { label: "Filial", value: filialView },
0635:           { label: "Aba", value: tab === "clientes" ? "Clientes" : tab === "fornecedores" ? "F
ornecedores" : "Tabelas de preco" },
0636:           { label: "Registros", value: String(currentRows.length) },
0637:         ]}
0638:       />
```

```
0639:
0640:       <div className="card">
0641:       <Toolbar
0642:         left={<div className="hint">Filial: <b>{filialView}</b></div>}
0643:         right={
0644:           <>
0645:             <button className={`btn ${tab === "clientes" ? "primary" : ""}`} onClick={() => se
tTab("clientes")}>
0646:               Clientes (SA1)
0647:             </button>
0648:             <button className={`btn ${tab === "fornecedores" ? "primary" : ""}`} onClick={() =
> setTab("fornecedores")}>
0649:               Fornecedores (SA2)
0650:             </button>
0651:             <button className={`btn ${tab === "precos" ? "primary" : ""}`} onClick={() => setT
ab("precos")}>
0652:               Tabelas de preco
0653:             </button>
0654:           </>
0655:         }
0656:       />
0657:
0658:       {err && <div className="alert mt-12">{err}</div>}
0659:
0660:       {tab === "clientes" && (
0661:         <>
0662:           <div className="section-title">Clientes SA1</div>
0663:           <div className="row-between mt-12">
0664:             <div className="row gap-10">
0665:               <input
0666:                 className="input minw-240"
0667:                 value={cliQ}
0668:                 onChange={(e) => { setCliQ(e.target.value); setCliPage(1); }}
0669:                 placeholder="Buscar por codigo, nome, doc..."
0670:               />
0671:               <select className="select" value={cliAtivo ? "true" : "false"} onChange={(e) =>
{ setCliAtivo(e.target.value === "true"); setCliPage(1); }}>
0672:                 <option value="true">Ativos</option>
0673:                 <option value="false">Todos</option>
0674:               </select>
0675:             </div>
0676:             <div className="row gap-10">
0677:               <button className="btn" onClick={() => { setCliPage(1); loadClientes(); }} disab
led={loading}>Atualizar</button>
0678:               <button className="btn ghost" onClick={() => { setCliQ(""); setCliPage(1); }} di
sabled={loading}>Limpar</button>
0679:             </div>
0680:           </div>
0681:
0682:           <div className="row-between mt-12">
0683:             <Pager
0684:               page={cliPage}
0685:               totalPages={totalPages}
0686:               pageSize={cliPageSize}
```

```
0687:                totalItems={sortedRows.length}
0688:                onPageChange={setCliPage}
0689:                onPageSizeChange={(ps) => { setCliPageSize(ps); setCliPage(1); }}
0690:                showPageSize
0691:              />
0692:            </div>
0693:
0694:            <DataTable
0695:              rows={pagedRows}
0696:              columns={clienteColumns}
0697:              sortKey={sortKey}
0698:              sortDir={sortDir}
0699:              className="compact"
0700:              wrapperClassName="table-scroller"
0701:              onSortChange={(k, d) => {
0702:                setSortKey(k);
0703:                setSortDir(d);
0704:                setCliPage(1);
0705:              }}
0706:              loading={loading}
0707:              emptyText="Nenhum cliente encontrado."
0708:            />
0709:
0710:            <div className="section-title">Criar cliente</div>
0711:            <div className="form-grid mt-12">
0712:              <div className="form-field"><label>Codigo</label><input value={cliForm.cod} onChan
ge={(e) => setCliForm({ ...cliForm, cod: e.target.value })} /></div>
0713:              <div className="form-field"><label>Nome</label><input value={cliForm.nome} onChang
e={(e) => setCliForm({ ...cliForm, nome: e.target.value })} /></div>
0714:              <div className="form-field"><label>Documento</label><input value={cliForm.doc} onC
hange={(e) => setCliForm({ ...cliForm, doc: e.target.value })} /></div>
0715:              <div className="form-field"><label>Email</label><input value={cliForm.email} onCha
nge={(e) => setCliForm({ ...cliForm, email: e.target.value })} /></div>
0716:              <div className="form-field"><label>Telefone</label><input value={cliForm.tel} onCh
ange={(e) => setCliForm({ ...cliForm, tel: e.target.value })} /></div>
0717:              <div className="form-field"><label>Endereco</label><input value={cliForm.end} onCh
ange={(e) => setCliForm({ ...cliForm, end: e.target.value })} /></div>
0718:              <div className="form-field"><label>Municipio</label><input value={cliForm.mun} onC
hange={(e) => setCliForm({ ...cliForm, mun: e.target.value })} /></div>
0719:              <div className="form-field"><label>UF</label><input value={cliForm.uf} onChange={(
e) => setCliForm({ ...cliForm, uf: e.target.value })} /></div>
0720:              <div className="form-field"><label>CEP</label><input value={cliForm.cep} onChange=
{(e) => setCliForm({ ...cliForm, cep: e.target.value })} /></div>
0721:              <div className="form-field"><label>Tabela ID (opcional)</label><input value={cliFo
rm.tabelaId} onChange={(e) => setCliForm({ ...cliForm, tabelaId: e.target.value })} /></div>
0722:              <div className="form-field"><label>Filial</label><input value={cliForm.filial} onC
hange={(e) => setCliForm({ ...cliForm, filial: e.target.value })} /></div>
0723:              <div className="form-field"><label> </label><button className="btn primary" o
nClick={onCriarCliente} disabled={loading}>Criar cliente</button></div>
0724:            </div>
0725:
0726:            <div className="section-title">Operacoes rapidas</div>
0727:            <div className="form-grid mt-12">
```

```
0728:             <div className="form-field"><label>Buscar por codigo</label><input value={cliBusca
Cod} onChange={(e) => setCliBuscaCod(e.target.value)} /></div>
0729:             <div className="form-field"><label> </label><button className="btn" onClick={
() => onBuscarCliente()} disabled={loading}>Buscar</button></div>
0730:             <div className="form-field"><label>Inativar por codigo</label><input value={cliDel
Cod} onChange={(e) => setCliDelCod(e.target.value)} /></div>
0731:             <div className="form-field"><label> </label><button className="btn danger" on
Click={() => onInativarCliente()} disabled={loading}>Inativar</button></div>
0732:             <div className="form-field"><label>Set tabela (codigo)</label><input value={cliSet
TabelaCod} onChange={(e) => setCliSetTabelaCod(e.target.value)} /></div>
0733:             <div className="form-field"><label>Tabela ID</label><input value={cliSetTabelaId}
onChange={(e) => setCliSetTabelaId(e.target.value)} /></div>
0734:             <div className="form-field"><label> </label><button className="btn" onClick={
onSetTabela} disabled={loading}>Definir tabela</button></div>
0735:           </div>
0736:         </>
0737:       )}
0738:
0739:       {tab === "fornecedores" && (
0740:         <>
0741:           <div className="section-title">Fornecedores SA2</div>
0742:           <div className="row-between mt-12">
0743:             <div className="row gap-10">
0744:               <input
0745:                 className="input minw-240"
0746:                 value={fornQ}
0747:                 onChange={(e) => { setFornQ(e.target.value); setFornPage(1); }}
0748:                 placeholder="Buscar por codigo, nome, doc..."
0749:               />
0750:               <select className="select" value={fornAtivo ? "true" : "false"} onChange={(e) =>
 { setFornAtivo(e.target.value === "true"); setFornPage(1); }}>
0751:                 <option value="true">Ativos</option>
0752:                 <option value="false">Todos</option>
0753:               </select>
0754:             </div>
0755:             <div className="row gap-10">
0756:               <button className="btn" onClick={() => { setFornPage(1); loadFornecedores(); }}
disabled={loading}>Atualizar</button>
0757:               <button className="btn ghost" onClick={() => { setFornQ(""); setFornPage(1); }}
disabled={loading}>Limpar</button>
0758:             </div>
0759:           </div>
0760:
0761:           <div className="row-between mt-12">
0762:             <Pager
0763:               page={fornPage}
0764:               totalPages={totalPages}
0765:               pageSize={fornPageSize}
0766:               totalItems={sortedRows.length}
0767:               onPageChange={setFornPage}
0768:               onPageSizeChange={(ps) => { setFornPageSize(ps); setFornPage(1); }}
0769:               showPageSize
0770:             />
0771:           </div>
```

```
0772:
0773:           <DataTable
0774:             rows={pagedRows}
0775:             columns={fornecedorColumns}
0776:             sortKey={sortKey}
0777:             sortDir={sortDir}
0778:             className="compact"
0779:             wrapperClassName="table-scroller"
0780:             onSortChange={(k, d) => {
0781:               setSortKey(k);
0782:               setSortDir(d);
0783:               setFornPage(1);
0784:             }}
0785:             loading={loading}
0786:             emptyText="Nenhum fornecedor encontrado."
0787:           />
0788:
0789:           <div className="section-title">Criar fornecedor</div>
0790:           <div className="form-grid mt-12">
0791:             <div className="form-field"><label>Codigo</label><input value={fornForm.cod} onCha
nge={(e) => setFornForm({ ...fornForm, cod: e.target.value })} /></div>
0792:             <div className="form-field"><label>Nome</label><input value={fornForm.nome} onChan
ge={(e) => setFornForm({ ...fornForm, nome: e.target.value })} /></div>
0793:             <div className="form-field"><label>Documento</label><input value={fornForm.doc} on
Change={(e) => setFornForm({ ...fornForm, doc: e.target.value })} /></div>
0794:             <div className="form-field"><label>Email</label><input value={fornForm.email} onCh
ange={(e) => setFornForm({ ...fornForm, email: e.target.value })} /></div>
0795:             <div className="form-field"><label>Telefone</label><input value={fornForm.tel} onC
hange={(e) => setFornForm({ ...fornForm, tel: e.target.value })} /></div>
0796:             <div className="form-field"><label>Endereco</label><input value={fornForm.end} onC
hange={(e) => setFornForm({ ...fornForm, end: e.target.value })} /></div>
0797:             <div className="form-field"><label>Municipio</label><input value={fornForm.mun} on
Change={(e) => setFornForm({ ...fornForm, mun: e.target.value })} /></div>
0798:             <div className="form-field"><label>UF</label><input value={fornForm.uf} onChange={
(e) => setFornForm({ ...fornForm, uf: e.target.value })} /></div>
0799:             <div className="form-field"><label>CEP</label><input value={fornForm.cep} onChange
={(e) => setFornForm({ ...fornForm, cep: e.target.value })} /></div>
0800:             <div className="form-field"><label>Filial</label><input value={fornForm.filial} on
Change={(e) => setFornForm({ ...fornForm, filial: e.target.value })} /></div>
0801:             <div className="form-field"><label> </label><button className="btn primary" o
nClick={onCriarFornecedor} disabled={loading}>Criar fornecedor</button></div>
0802:           </div>
0803:
0804:           <div className="section-title">Operacoes rapidas</div>
0805:           <div className="form-grid mt-12">
0806:             <div className="form-field"><label>Buscar por codigo</label><input value={fornBusc
aCod} onChange={(e) => setFornBuscaCod(e.target.value)} /></div>
0807:             <div className="form-field"><label> </label><button className="btn" onClick={
() => onBuscarFornecedor()} disabled={loading}>Buscar</button></div>
0808:             <div className="form-field"><label>Inativar por codigo</label><input value={fornDe
lCod} onChange={(e) => setFornDelCod(e.target.value)} /></div>
0809:             <div className="form-field"><label> </label><button className="btn danger" on
Click={() => onInativarFornecedor()} disabled={loading}>Inativar</button></div>
0810:           </div>
```

```
0811:         </>
0812:       )}
0813:
0814:       {tab === "precos" && (
0815:         <>
0816:           <div className="section-title">Tabelas de preco</div>
0817:           <div className="row-between mt-12">
0818:             <div className="hint">Filial: <b>{filialView}</b> | Reserva e parceiros usam a tab
ela comercial vigente.</div>
0819:             <div className="row gap-10">
0820:               <button className="btn" onClick={loadTabelas} disabled={loading}>Atualizar</butt
on>
0821:             </div>
0822:           </div>
0823:
0824:           <DataTable
0825:             rows={tabelas}
0826:             columns={tabelaColumns}
0827:             sortKey={sortKey}
0828:             sortDir={sortDir}
0829:             className="compact"
0830:             wrapperClassName="table-scroller"
0831:             onSortChange={(k, d) => {
0832:               setSortKey(k);
0833:               setSortDir(d);
0834:             }}
0835:             loading={loading}
0836:             emptyText="Nenhuma tabela de preco encontrada."
0837:           />
0838:
0839:           <div className="section-title">Criar tabela</div>
0840:           <div className="form-grid mt-12">
0841:             <div className="form-field"><label>Codigo</label><input value={tabelaForm.codigo}
onChange={(e) => setTabelaForm({ ...tabelaForm, codigo: e.target.value })} /></div>
0842:             <div className="form-field"><label>Descricao</label><input value={tabelaForm.descr
icao} onChange={(e) => setTabelaForm({ ...tabelaForm, descricao: e.target.value })} /></div>
0843:             <div className="form-field"><label> </label><button className="btn primary" o
nClick={onCriarTabela} disabled={loading}>Criar tabela</button></div>
0844:           </div>
0845:
0846:           <div className="section-title">Definir preco vigente</div>
0847:           <div className="form-grid mt-12">
0848:             <div className="form-field">
0849:               <label>Tabela</label>
0850:               <select value={precoForm.tabelaId} onChange={(e) => setPrecoForm({ ...precoForm,
 tabelaId: e.target.value })}>
0851:                 <option value="">Selecione</option>
0852:                 {tabelas.map((t) => (
0853:                   <option key={t.id} value={String(t.id)}>{t.codigo} - {t.descricao}</option>
0854:                 ))}
0855:               </select>
0856:             </div>
0857:             <div className="form-field"><label>Produto</label><input value={precoForm.produto}
 onChange={(e) => setPrecoForm({ ...precoForm, produto: e.target.value })} /></div>
```

```
0858:             <div className="form-field"><label>Preco</label><input type="number" step="0.01" v
alue={precoForm.preco} onChange={(e) => setPrecoForm({ ...precoForm, preco: e.target.value })} /></d
iv>
0859:             <div className="form-field"><label>Data inicial</label><input type="date" value={p
recoForm.dtIni} onChange={(e) => setPrecoForm({ ...precoForm, dtIni: e.target.value })} /></div>
0860:             <div className="form-field"><label> </label><button className="btn primary" o
nClick={onDefinirPreco} disabled={loading}>Salvar preco</button></div>
0861:           </div>
0862:
0863:           <div className="section-title">Consultar preco vigente</div>
0864:           <div className="form-grid mt-12">
0865:             <div className="form-field">
0866:               <label>Tabela</label>
0867:               <select value={precoLookupTabelaId} onChange={(e) => setPrecoLookupTabelaId(e.ta
rget.value)}>
0868:                 <option value="">Selecione</option>
0869:                 {tabelas.map((t) => (
0870:                   <option key={t.id} value={String(t.id)}>{t.codigo} - {t.descricao}</option>
0871:                 ))}
0872:               </select>
0873:             </div>
0874:             <div className="form-field"><label>Produto</label><input value={precoLookupProduto
} onChange={(e) => setPrecoLookupProduto(e.target.value)} /></div>
0875:             <div className="form-field"><label>Data</label><input type="date" value={precoLook
upData} onChange={(e) => setPrecoLookupData(e.target.value)} /></div>
0876:             <div className="form-field"><label> </label><button className="btn" onClick={
onBuscarPrecoVigente} disabled={loading}>Buscar preco</button></div>
0877:           </div>
0878:
0879:           <div className="section-title">Itens da tabela</div>
0880:           <div className="row-between mt-12">
0881:             <div className="hint">
0882:               {precoLookupTabelaId
0883:                 ? <>Tabela selecionada: <b>{precoLookupTabelaId}</b></>
0884:                 : "Selecione uma tabela para listar os precos cadastrados."}
0885:             </div>
0886:             <div className="row gap-10">
0887:               <button
0888:                 className="btn"
0889:                 onClick={() => {
0890:                   const tabelaId = precoLookupTabelaId || precoForm.tabelaId;
0891:                   if (!tabelaId) {
0892:                     setErr("Selecione uma tabela para listar os itens.");
0893:                     return;
0894:                   }
0895:                   void loadItensTabela(toNumber(tabelaId, 0));
0896:                 }}
0897:                 disabled={loading}
0898:               >
0899:                 Listar itens
0900:               </button>
0901:             </div>
0902:           </div>
0903:
```

```
0904:           <DataTable
0905:             rows={itensTabela}
0906:             columns={itemTabelaColumns}
0907:             sortKey="produto"
0908:             sortDir="asc"
0909:             className="compact"
0910:             wrapperClassName="table-scroller"
0911:             onSortChange={() => {}}
0912:             loading={loading}
0913:             emptyText="Nenhum preco cadastrado para a tabela selecionada."
0914:           />
0915:         </>
0916:       )}
0917:
0918:       {lastResp && (
0919:         <div className="card soft mt-12">
0920:           <div style={{ fontWeight: 700 }}>Ultima resposta da API</div>
0921:           <pre style={{ margin: 0, whiteSpace: "pre-wrap" }}>{JSON.stringify(lastResp, null, 2
)}</pre>
0922:         </div>
0923:       )}
0924:       </div>
0925:     </>
0926:   );
0927: }
```

==========
ARQUIVO: frontend/src/pages/pedidoDetalhe.tsx
==========

```
0001: import { DataTable, type Column } from "../components/DataTable";
0002:
0003: type ItemResumo = {
0004:   id: number;
0005:   pedidoId: number;
0006:   produto: string;
0007:   qtd: number;
0008:   total: number;
0009:   precoUnit: number;
0010:   cmvUnit: number;
0011:   ativo: string;
0012: };
0013:
0014: type ItemRow = Record<string, unknown>;
0015:
0016: type Props = {
0017:   selectedPedidoId: number | null;
0018:   loading: boolean;
0019:   pedidoDetalhe: Record<string, unknown> | null;
0020:   arResumo: Record<string, unknown> | null;
0021:   itens: ItemResumo[];
0022:   onRecalcularPedido: () => void;
0023:   onAbrirFaturar: () => void;
0024:   onAbrirCancelar: () => void;
0025:   onAbrirEstornar: () => void;
```

```
0026:   onAbrirAddItem: () => void;
0027:   onAbrirEditItem: (itemId: number, row: ItemRow) => void;
0028:   onConfirmarExcluirItem: (itemId: number) => void;
0029:   onAbrirCustoItem: (produtoCod: string) => void;
0030: };
0031:
0032: function toNumber(value: unknown, fallback = 0): number {
0033:   if (typeof value === "number") return value;
0034:   if (typeof value === "string" && value.trim()) {
0035:     const n = Number(value);
0036:     return Number.isFinite(n) ? n : fallback;
0037:   }
0038:   return fallback;
0039: }
0040:
0041: function toStringValue(value: unknown, fallback = ""): string {
0042:   if (typeof value === "string") return value;
0043:   if (value === null || value === undefined) return fallback;
0044:   return String(value);
0045: }
0046:
0047: export function PedidoDetalhePanel({
0048:   selectedPedidoId,
0049:   loading,
0050:   pedidoDetalhe,
0051:   arResumo,
0052:   itens,
0053:   onRecalcularPedido,
0054:   onAbrirFaturar,
0055:   onAbrirCancelar,
0056:   onAbrirEstornar,
0057:   onAbrirAddItem,
0058:   onAbrirEditItem,
0059:   onConfirmarExcluirItem,
0060:   onAbrirCustoItem,
0061: }: Props) {
0062:   const itemColumns: Column<ItemRow>[] = [
0063:     { key: "id", header: "Item ID", sortable: true },
0064:     { key: "produto", header: "Produto", sortable: true },
0065:     { key: "qtd", header: "Qtd", sortable: true },
0066:     {
0067:       key: "total",
0068:       header: "Total",
0069:       sortable: true,
0070:       render: (r) => toNumber(r.total).toLocaleString("pt-BR", { style: "currency", currency:
"BRL" }),
0071:     },
0072:     {
0073:       key: "precoUnit",
0074:       header: "Preco Unit",
0075:       sortable: true,
0076:       className: "hide-sm",
0077:       render: (r) => toNumber(r.precoUnit).toLocaleString("pt-BR", { style: "currency", curren
cy: "BRL" }),
```

```
0078:     },
0079:     {
0080:       key: "cmvUnit",
0081:       header: "CMV Unit",
0082:       sortable: true,
0083:       className: "hide-sm",
0084:       render: (r) => toNumber(r.cmvUnit).toLocaleString("pt-BR", { style: "currency", currency
: "BRL" }),
0085:     },
0086:     {
0087:       key: "acoes",
0088:       header: "Acoes",
0089:       width: 320,
0090:       render: (r) => {
0091:         const itemId = toNumber(r.id);
0092:         const produtoCod = toStringValue(r.produto, "");
0093:         return (
0094:           <div className="row gap-10">
0095:             <button className="btn" onClick={() => onAbrirEditItem(itemId, r)}>Editar</button>
0096:             <button className="btn" onClick={() => onAbrirCustoItem(produtoCod)}>Custo</button
>
0097:             <button className="btn danger" onClick={() => onConfirmarExcluirItem(itemId)}>Excl
uir</button>
0098:           </div>
0099:         );
0100:       },
0101:     },
0102:   ];
0103:
0104:   return (
0105:     <div className="card mt-12">
0106:       <div className="row-between">
0107:         <div>
0108:           <div style={{ fontWeight: 700 }}>Detalhe do pedido</div>
0109:           <div className="hint">{selectedPedidoId ? `Pedido selecionado: ${selectedPedidoId}`
: "Selecione um pedido na tabela"}</div>
0110:         </div>
0111:         <div className="row gap-10">
0112:           <button className="btn" disabled={!selectedPedidoId || loading} onClick={onRecalcula
rPedido}>Recalcular</button>
0113:           <button className="btn" disabled={!selectedPedidoId || loading} onClick={onAbrirFatu
rar}>Faturar</button>
0114:           <button className="btn danger" disabled={!selectedPedidoId || loading} onClick={onAb
rirCancelar}>Cancelar</button>
0115:           <button className="btn danger" disabled={!selectedPedidoId || loading} onClick={onAb
rirEstornar}>Estornar</button>
0116:         </div>
0117:       </div>
0118:
0119:       {pedidoDetalhe && (
0120:         <div className="mt-12">
0121:           <div className="hint">Status: <b>{toStringValue(pedidoDetalhe.C5_STATUS, "-")}</b> |
 Origem: <b>{toStringValue(pedidoDetalhe.C5_ORIGEM, "-")}</b></div>
```

```
0122:           <div className="hint">Filial: <b>{toStringValue(pedidoDetalhe.C5_FILIAL, "-")}</b> |
 Total: <b>{toNumber(pedidoDetalhe.C5_VALOR_TOTAL).toLocaleString("pt-BR", { style: "currency", curr
ency: "BRL" })}</b></div>
0123:           {arResumo && <div className="hint">AR: <b>{toStringValue(arResumo.status, "-")}</b>
| Valor: <b>{toNumber(arResumo.valor).toLocaleString("pt-BR", { style: "currency", currency: "BRL" }
)}</b></div>}
0124:         </div>
0125:       )}
0126:
0127:       <div className="section-title">Resumo</div>
0128:       <div className="stats-grid">
0129:         <div className="stat-card">
0130:           <div>
0131:             <div className="kpi-label">Itens</div>
0132:             <div className="kpi">{itens.length}</div>
0133:           </div>
0134:           <div className="stat-icon">IT</div>
0135:         </div>
0136:         <div className="stat-card">
0137:           <div>
0138:             <div className="kpi-label">Total dos itens</div>
0139:             <div className="kpi">
0140:               {itens.reduce((acc, x) => acc + toNumber(x.total, 0), 0).toLocaleString("pt-BR",
 { style: "currency", currency: "BRL" })}
0141:             </div>
0142:           </div>
0143:           <div className="stat-icon success">R$</div>
0144:         </div>
0145:         <div className="stat-card">
0146:           <div>
0147:             <div className="kpi-label">Status</div>
0148:             <div className="kpi">{toStringValue(pedidoDetalhe?.C5_STATUS ?? "-", "-")}</div>
0149:           </div>
0150:           <div className="stat-icon solid">ST</div>
0151:         </div>
0152:       </div>
0153:
0154:       <div className="row-between mt-12">
0155:         <div style={{ fontWeight: 700 }}>Itens do pedido</div>
0156:         <button className="btn primary" disabled={!selectedPedidoId || loading} onClick={onAbr
irAddItem}>Adicionar item</button>
0157:       </div>
0158:
0159:       <DataTable
0160:         rows={itens as unknown as ItemRow[]}
0161:         columns={itemColumns}
0162:         sortKey="id"
0163:         sortDir="asc"
0164:         className="compact"
0165:         wrapperClassName="table-scroller"
0166:         onSortChange={() => undefined}
0167:         loading={loading}
0168:         emptyText="Sem itens."
0169:       />
```

```
0170:     </div>
0171:   );
0172: }
```

==========
ARQUIVO: frontend/src/pages/pedidosList.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { useNavigate, useSearchParams } from "react-router-dom";
0003: import { Toolbar } from "../components/Toolbar";
0004: import { Modal } from "../components/Modal";
0005: import { ConfirmDialog } from "../components/ConfirmDialog";
0006: import { Pager } from "../components/Pager";
0007: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0008: import { PageHero } from "../components/PageHero";
0009: import { PedidoDetalhePanel } from "./pedidoDetalhe";
0010: import { getFilial, onFilialChange } from "../state/filial";
0011: import ordersIcon from "../assets/shell-icons/orders.png";
0012: import ordersBg from "../assets/hero-backgrounds/orders.jpg";
0013: import { buscarProdutos, type ProdutoResumo } from "../api/produtos";
0014: import {
0015:   adicionarItem,
0016:   cancelarPedido,
0017:   criarPedido,
0018:   editarItem,
0019:   estornarPedido,
0020:   excluirItem,
0021:   faturarPedido,
0022:   listarPedidos,
0023:   listarPedidosEnriquecido,
0024:   obterPedido,
0025:   recalcularPedido,
0026:   type CancelarPedidoInput,
0027:   type EstornarPedidoInput,
0028:   type FaturarPedidoInput,
0029: } from "../api/vendas";
0030:
0031: type PedidoRow = Record<string, unknown>;
0032: type PedidoResumo = { id: number; filial: string; status: string; origem: string; valorTotal:
number; data: string; qtdItens: number };
0033: type ItemResumo = { id: number; pedidoId: number; produto: string; qtd: number; total: number;
 precoUnit: number; cmvUnit: number; ativo: string };
0034:
0035: type PedidoForm = { filial: string; valorTotal: string; icms: string; ipi: string; totalBruto:
 string };
0036: type AddItemForm = { filial: string; produto: string; qtd: string; total: string; precoUnit: s
tring; cmvUnit: string };
0037: type EditItemForm = { filial: string; qtd: string; total: string; precoUnit: string; cmvUnit:
string };
0038: type FaturarForm = { filial: string; clienteCod: string; vencDias: string };
0039: type CancelarForm = { filial: string; modo: "AUTO" | "SIMPLES" | "ESTORNAR"; reativarReserva:
boolean };
0040: type EstornarForm = { filial: string; motivo: string };
```

```
0041: type ConfirmState = { title: string; message: string; danger?: boolean; confirmText?: string;
run: () => Promise<void> };
0042: type SortKey = "id" | "data" | "filial" | "status" | "origem" | "valorTotal";
0043:
0044: function toRecord(value: unknown): Record<string, unknown> {
0045:   return value && typeof value === "object" ? (value as Record<string, unknown>) : {};
0046: }
0047:
0048: function toNumber(value: unknown, fallback = 0): number {
0049:   if (typeof value === "number") return value;
0050:   if (typeof value === "string" && value.trim()) {
0051:     const n = Number(value);
0052:     return Number.isFinite(n) ? n : fallback;
0053:   }
0054:   return fallback;
0055: }
0056:
0057: function toStringValue(value: unknown, fallback = ""): string {
0058:   if (typeof value === "string") return value;
0059:   if (value === null || value === undefined) return fallback;
0060:   return String(value);
0061: }
0062:
0063: function getErrorMessage(error: unknown, fallback: string): string {
0064:   if (error instanceof Error && error.message) return error.message;
0065:   return fallback;
0066: }
0067:
0068: function emptyPedidoForm(): PedidoForm {
0069:   return { filial: getFilial(), valorTotal: "0", icms: "", ipi: "", totalBruto: "" };
0070: }
0071:
0072: function emptyItemForm(): AddItemForm {
0073:   return { filial: getFilial(), produto: "", qtd: "1", total: "0", precoUnit: "", cmvUnit: ""
};
0074: }
0075:
0076: function emptyFaturarForm(): FaturarForm {
0077:   return { filial: getFilial(), clienteCod: "", vencDias: "30" };
0078: }
0079:
0080: function emptyCancelarForm(): CancelarForm {
0081:   return { filial: getFilial(), modo: "AUTO", reativarReserva: true };
0082: }
0083:
0084: function emptyEstornarForm(): EstornarForm {
0085:   return { filial: getFilial(), motivo: "" };
0086: }
0087:
0088: function normalizePedido(row: unknown): PedidoResumo {
0089:   const x = toRecord(row);
0090:   return {
0091:     id: toNumber(x.ID ?? x.id),
0092:     filial: toStringValue(x.C5_FILIAL ?? x.filial, "01"),
```

```
0093:     status: toStringValue(x.C5_STATUS ?? x.status, "ABERTO"),
0094:     origem: toStringValue(x.C5_ORIGEM ?? x.origem, "VENDA"),
0095:     valorTotal: toNumber(x.C5_VALOR_TOTAL ?? x.valor_total ?? x.total ?? x.valorTotal),
0096:     data: toStringValue(x.C5_DATA ?? x.criado_em ?? x.data ?? ""),
0097:     qtdItens: toNumber(x.qtd_itens ?? x.qtdItens ?? x.QTD_ITENS, 0),
0098:   };
0099: }
0100:
0101: function normalizeItem(row: unknown): ItemResumo {
0102:   const x = toRecord(row);
0103:   return {
0104:     id: toNumber(x.ID ?? x.id),
0105:     pedidoId: toNumber(x.C6_PEDIDO_ID ?? x.pedido_id),
0106:     produto: toStringValue(x.C6_PRODUTO ?? x.produto),
0107:     qtd: toNumber(x.C6_QTD ?? x.qtd),
0108:     total: toNumber(x.C6_TOTAL ?? x.total ?? x.C6_VALOR),
0109:     precoUnit: toNumber(x.C6_PRECO_UNIT ?? x.preco_unit),
0110:     cmvUnit: toNumber(x.C6_CMV_UNIT ?? x.cmv_unit),
0111:     ativo: toStringValue(x.C6_ATIVO ?? x.ativo_sn ?? "1"),
0112:   };
0113: }
0114:
0115: export function PedidosList() {
0116:   const navigate = useNavigate();
0117:   const [searchParams, setSearchParams] = useSearchParams();
0118:   const [filialView, setFilialView] = useState(getFilial());
0119:   const [status, setStatus] = useState("");
0120:   const [origem, setOrigem] = useState("");
0121:   const [de, setDe] = useState("");
0122:   const [ate, setAte] = useState("");
0123:   const [modoEnriquecido, setModoEnriquecido] = useState(false);
0124:
0125:   const [rows, setRows] = useState<PedidoResumo[]>([]);
0126:   const [total, setTotal] = useState(0);
0127:   const [loading, setLoading] = useState(false);
0128:   const [loadingItemProduto, setLoadingItemProduto] = useState(false);
0129:   const [itemProdutoResultados, setItemProdutoResultados] = useState<ProdutoResumo[]>([]);
0130:   const [loadingBuscaProdutoItem, setLoadingBuscaProdutoItem] = useState(false);
0131:   const [err, setErr] = useState<string | null>(null);
0132:   const [lastAction, setLastAction] = useState<Record<string, unknown> | null>(null);
0133:
0134:   const [page, setPage] = useState(1);
0135:   const [pageSize, setPageSize] = useState(10);
0136:   const [sortKey, setSortKey] = useState<SortKey>("id");
0137:   const [sortDir, setSortDir] = useState<SortDir>("desc");
0138:
0139:   const [selectedPedidoId, setSelectedPedidoId] = useState<number | null>(null);
0140:   const [pedidoDetalhe, setPedidoDetalhe] = useState<Record<string, unknown> | null>(null);
0141:   const [itens, setItens] = useState<ItemResumo[]>([]);
0142:   const [arResumo, setArResumo] = useState<Record<string, unknown> | null>(null);
0143:
0144:   const [modalNovoPedido, setModalNovoPedido] = useState(false);
0145:   const [pedidoForm, setPedidoForm] = useState<PedidoForm>(emptyPedidoForm());
0146:
```

```
0147:   const [modalAddItem, setModalAddItem] = useState(false);
0148:   const [itemForm, setItemForm] = useState<AddItemForm>(emptyItemForm());
0149:
0150:   const [editItemId, setEditItemId] = useState<number | null>(null);
0151:   const [modalEditItem, setModalEditItem] = useState(false);
0152:   const [editItemForm, setEditItemForm] = useState<EditItemForm>({ filial: getFilial(), qtd: "
1", total: "0", precoUnit: "", cmvUnit: "" });
0153:
0154:   const [modalFaturar, setModalFaturar] = useState(false);
0155:   const [faturarForm, setFaturarForm] = useState<FaturarForm>(emptyFaturarForm());
0156:
0157:   const [modalCancelar, setModalCancelar] = useState(false);
0158:   const [cancelarForm, setCancelarForm] = useState<CancelarForm>(emptyCancelarForm());
0159:
0160:   const [modalEstornar, setModalEstornar] = useState(false);
0161:   const [estornarForm, setEstornarForm] = useState<EstornarForm>(emptyEstornarForm());
0162:
0163:   const [confirmState, setConfirmState] = useState<ConfirmState | null>(null);
0164:   const alertFilter = searchParams.get("alert") ?? "";
0165:
0166:   const alertLabelMap: Record<string, string> = {
0167:     pedidos_no_items: "Exibindo apenas pedidos abertos sem itens.",
0168:     pedidos_open: "Exibindo apenas pedidos abertos com itens aguardando faturamento.",
0169:   };
0170:
0171:   const offset = (page - 1) * pageSize;
0172:
0173:   async function carregarPedidos() {
0174:     setErr(null);
0175:     setLoading(true);
0176:     try {
0177:       const params = {
0178:         filial: filialView,
0179:         status: status || undefined,
0180:         origem: origem || undefined,
0181:         de: de || undefined,
0182:         ate: ate || undefined,
0183:         limit: pageSize,
0184:         offset,
0185:       };
0186:       const resp = modoEnriquecido ? await listarPedidosEnriquecido(params) : await listarPedi
dos(params);
0187:       const items = Array.isArray(resp.items) ? resp.items : [];
0188:       setRows(items.map(normalizePedido));
0189:       setTotal(toNumber(resp.total, items.length));
0190:     } catch (error: unknown) {
0191:       setErr(getErrorMessage(error, "Erro ao carregar pedidos"));
0192:     } finally {
0193:       setLoading(false);
0194:     }
0195:   }
0196:
0197:   async function carregarDetalhe(pedidoId: number) {
0198:     setErr(null);
```

```
0199:     setLoading(true);
0200:     try {
0201:       const resp = await obterPedido(pedidoId);
0202:       const obj = toRecord(resp);
0203:       const pedido = toRecord(obj.pedido ?? obj);
0204:       const itensRaw = Array.isArray(obj.itens) ? obj.itens : [];
0205:       const ar = toRecord(obj.ar_resumo);
0206:       setSelectedPedidoId(pedidoId);
0207:       setPedidoDetalhe(pedido);
0208:       setItens(itensRaw.map(normalizeItem));
0209:       setArResumo(Object.keys(ar).length ? ar : null);
0210:     } catch (error: unknown) {
0211:       setErr(getErrorMessage(error, "Erro ao obter pedido"));
0212:     } finally {
0213:       setLoading(false);
0214:     }
0215:   }
0216:
0217:   useEffect(() => {
0218:     carregarPedidos();
0219:     // eslint-disable-next-line react-hooks/exhaustive-deps
0220:   }, [filialView, status, origem, de, ate, page, pageSize, modoEnriquecido]);
0221:
0222:   useEffect(() => {
0223:     const unsub = onFilialChange((novaFilial) => {
0224:       setFilialView(novaFilial);
0225:       setPage(1);
0226:       setSelectedPedidoId(null);
0227:       setPedidoDetalhe(null);
0228:       setItens([]);
0229:       setArResumo(null);
0230:       setPedidoForm(emptyPedidoForm());
0231:       setItemForm(emptyItemForm());
0232:       setFaturarForm(emptyFaturarForm());
0233:       setCancelarForm(emptyCancelarForm());
0234:       setEstornarForm(emptyEstornarForm());
0235:     });
0236:     return unsub;
0237:   }, []);
0238:
0239:   useEffect(() => {
0240:     if (!alertFilter) return;
0241:     setStatus("ABERTO");
0242:     setPage(1);
0243:     if (alertFilter === "pedidos_no_items" || alertFilter === "pedidos_open") {
0244:       setModoEnriquecido(true);
0245:     }
0246:   }, [alertFilter]);
0247:
0248:   const businessFilteredRows = useMemo(() => {
0249:     if (alertFilter === "pedidos_no_items") {
0250:       return rows.filter((row) => row.qtdItens === 0);
0251:     }
0252:     if (alertFilter === "pedidos_open") {
```

```
0253:       return rows.filter((row) => row.qtdItens > 0);
0254:     }
0255:     return rows;
0256:   }, [rows, alertFilter]);
0257:
0258:   const sortedRows = useMemo(() => {
0259:     const arr = [...businessFilteredRows];
0260:     arr.sort((a, b) => {
0261:       const av = sortKey === "id" || sortKey === "valorTotal" ? (a[sortKey] as number) : Strin
g(a[sortKey] ?? "").toLowerCase();
0262:       const bv = sortKey === "id" || sortKey === "valorTotal" ? (b[sortKey] as number) : Strin
g(b[sortKey] ?? "").toLowerCase();
0263:       if (av < bv) return sortDir === "asc" ? -1 : 1;
0264:       if (av > bv) return sortDir === "asc" ? 1 : -1;
0265:       return 0;
0266:     });
0267:     return arr;
0268:   }, [businessFilteredRows, sortKey, sortDir]);
0269:
0270:   const totalPages = Math.max(1, Math.ceil(sortedRows.length / pageSize));
0271:
0272:   const totalValor = useMemo(
0273:     () => sortedRows.reduce((acc, r) => acc + toNumber(r.valorTotal, 0), 0),
0274:     [sortedRows]
0275:   );
0276:
0277:   const columns: Column<PedidoRow>[] = useMemo(
0278:     () => [
0279:       { key: "id", header: "ID", sortable: true },
0280:       { key: "data", header: "Data", sortable: true },
0281:       { key: "filial", header: "Filial", sortable: true, className: "hide-sm" },
0282:       { key: "status", header: "Status", sortable: true, className: "hide-sm" },
0283:       { key: "origem", header: "Origem", sortable: true, className: "hide-sm" },
0284:       {
0285:         key: "valorTotal",
0286:         header: "Valor",
0287:         sortable: true,
0288:         render: (r) => toNumber(r.valorTotal).toLocaleString("pt-BR", { style: "currency", cur
rency: "BRL" }),
0289:       },
0290:       {
0291:         key: "acoes",
0292:         header: "Acoes",
0293:         width: 240,
0294:         render: (r) => {
0295:           const pedidoId = toNumber(r.id);
0296:           return (
0297:             <div className="row gap-10">
0298:               <button className="btn" onClick={() => carregarDetalhe(pedidoId)}>Detalhar</butt
on>
0299:               <button className="btn" onClick={() => abrirModalAddItem(pedidoId)}>Add Item</bu
tton>
0300:             </div>
0301:           );
```

```
0302:         },
0303:       },
0304:     ],
0305:     []
0306:   );
0307:
0308:   function abrirModalAddItem(pedidoId: number) {
0309:     setSelectedPedidoId(pedidoId);
0310:     setItemForm(emptyItemForm());
0311:     setItemProdutoResultados([]);
0312:     setModalAddItem(true);
0313:   }
0314:
0315:   async function buscarProdutosParaItem(queryValue?: string) {
0316:     const query = (queryValue ?? itemForm.produto).trim();
0317:     if (!query) {
0318:       setItemProdutoResultados([]);
0319:       return;
0320:     }
0321:     setLoadingBuscaProdutoItem(true);
0322:     try {
0323:       const results = await buscarProdutos({ q: query, filial: itemForm.filial || filialView,
limite: 8 });
0324:       setItemProdutoResultados(results);
0325:     } catch {
0326:       setItemProdutoResultados([]);
0327:     } finally {
0328:       setLoadingBuscaProdutoItem(false);
0329:     }
0330:   }
0331:
0332:   function usarProdutoNoItem(produto: ProdutoResumo) {
0333:     setItemForm((current) => ({
0334:       ...current,
0335:       produto: produto.cod,
0336:       cmvUnit: current.cmvUnit || String(produto.cm ?? 0),
0337:     }));
0338:     setItemProdutoResultados([]);
0339:   }
0340:
0341:   async function carregarCmvAtualProduto(produtoCod: string) {
0342:     const cod = produtoCod.trim();
0343:     if (!cod) return;
0344:     setLoadingItemProduto(true);
0345:     try {
0346:       const results = await buscarProdutos({ q: cod, filial: itemForm.filial || filialView, li
mite: 8 });
0347:       const match =
0348:         results.find((item) => item.cod.trim().toUpperCase() === cod.toUpperCase()) ??
0349:         results[0];
0350:       if (!match) return;
0351:       setItemForm((current) => ({
0352:         ...current,
0353:         produto: current.produto || match.cod,
```

```
0354:         cmvUnit: current.cmvUnit || String(match.cm ?? 0),
0355:       }));
0356:     } catch {
0357:       // mantem fluxo manual do operador se a consulta falhar
0358:     } finally {
0359:       setLoadingItemProduto(false);
0360:     }
0361:   }
0362:
0363:   function abrirModalEditItem(itemId: number, row: Record<string, unknown>) {
0364:     setEditItemId(itemId);
0365:     setEditItemForm({
0366:       filial: toStringValue(row.filial, filialView),
0367:       qtd: String(toNumber(row.qtd, 1)),
0368:       total: String(toNumber(row.total, 0)),
0369:       precoUnit: String(toNumber(row.precoUnit, 0)),
0370:       cmvUnit: String(toNumber(row.cmvUnit, 0)),
0371:     });
0372:     setModalEditItem(true);
0373:   }
0374:
0375:   async function onCriarPedido() {
0376:     setErr(null);
0377:     setLoading(true);
0378:     try {
0379:       const resp = await criarPedido({
0380:         filial: pedidoForm.filial || filialView,
0381:         valor_total: toNumber(pedidoForm.valorTotal, 0),
0382:         icms: pedidoForm.icms ? toNumber(pedidoForm.icms) : null,
0383:         ipi: pedidoForm.ipi ? toNumber(pedidoForm.ipi) : null,
0384:         total_bruto: pedidoForm.totalBruto ? toNumber(pedidoForm.totalBruto) : null,
0385:       });
0386:       setLastAction(toRecord(resp));
0387:       setModalNovoPedido(false);
0388:       await carregarPedidos();
0389:     } catch (error: unknown) {
0390:       setErr(getErrorMessage(error, "Erro ao criar pedido"));
0391:     } finally {
0392:       setLoading(false);
0393:     }
0394:   }
0395:
0396:   async function onAdicionarItem() {
0397:     if (!selectedPedidoId) return;
0398:     setErr(null);
0399:     setLoading(true);
0400:     try {
0401:       const resp = await adicionarItem(selectedPedidoId, {
0402:         filial: itemForm.filial || filialView,
0403:         produto: itemForm.produto,
0404:         qtd: toNumber(itemForm.qtd, 1),
0405:         total: toNumber(itemForm.total, 0),
0406:         preco_unit: itemForm.precoUnit ? toNumber(itemForm.precoUnit) : null,
0407:         cmv_unit: itemForm.cmvUnit ? toNumber(itemForm.cmvUnit) : null,
```

```
0408:       });
0409:       setLastAction(toRecord(resp));
0410:       setModalAddItem(false);
0411:       await carregarPedidos();
0412:       await carregarDetalhe(selectedPedidoId);
0413:     } catch (error: unknown) {
0414:       setErr(getErrorMessage(error, "Erro ao adicionar item"));
0415:     } finally {
0416:       setLoading(false);
0417:     }
0418:   }
0419:
0420:   async function onEditarItem() {
0421:     if (!editItemId || !selectedPedidoId) return;
0422:     setErr(null);
0423:     setLoading(true);
0424:     try {
0425:       const resp = await editarItem(editItemId, {
0426:         filial: editItemForm.filial || filialView,
0427:         qtd: toNumber(editItemForm.qtd, 1),
0428:         total: toNumber(editItemForm.total, 0),
0429:         preco_unit: editItemForm.precoUnit ? toNumber(editItemForm.precoUnit) : null,
0430:         cmv_unit: editItemForm.cmvUnit ? toNumber(editItemForm.cmvUnit) : null,
0431:       });
0432:       setLastAction(toRecord(resp));
0433:       setModalEditItem(false);
0434:       await carregarPedidos();
0435:       await carregarDetalhe(selectedPedidoId);
0436:     } catch (error: unknown) {
0437:       setErr(getErrorMessage(error, "Erro ao editar item"));
0438:     } finally {
0439:       setLoading(false);
0440:     }
0441:   }
0442:
0443:   function confirmarExcluirItem(itemId: number) {
0444:     setConfirmState({
0445:       title: "Excluir item",
0446:       message: `Deseja excluir o item ${itemId}?`,
0447:       danger: true,
0448:       confirmText: "Excluir",
0449:       run: async () => {
0450:         if (!selectedPedidoId) return;
0451:         const resp = await excluirItem(itemId, { filial: filialView });
0452:         setLastAction(toRecord(resp));
0453:         await carregarPedidos();
0454:         await carregarDetalhe(selectedPedidoId);
0455:       },
0456:     });
0457:   }
0458:
0459:   async function onFaturarPedido() {
0460:     if (!selectedPedidoId) return;
0461:     setErr(null);
```

```
0462:     setLoading(true);
0463:     try {
0464:       const payload: FaturarPedidoInput = {
0465:         filial: faturarForm.filial || filialView,
0466:         cliente_cod: faturarForm.clienteCod || null,
0467:         venc_dias: toNumber(faturarForm.vencDias, 30),
0468:       };
0469:       const resp = await faturarPedido(selectedPedidoId, payload);
0470:       setLastAction(toRecord(resp));
0471:       setModalFaturar(false);
0472:       await carregarPedidos();
0473:       await carregarDetalhe(selectedPedidoId);
0474:     } catch (error: unknown) {
0475:       setErr(getErrorMessage(error, "Erro ao faturar pedido"));
0476:     } finally {
0477:       setLoading(false);
0478:     }
0479:   }
0480:
0481:   async function onCancelarPedido() {
0482:     if (!selectedPedidoId) return;
0483:     setErr(null);
0484:     setLoading(true);
0485:     try {
0486:       const payload: CancelarPedidoInput = {
0487:         filial: cancelarForm.filial || filialView,
0488:         modo: cancelarForm.modo,
0489:         reativar_reserva: cancelarForm.reativarReserva,
0490:       };
0491:       const resp = await cancelarPedido(selectedPedidoId, payload);
0492:       setLastAction(toRecord(resp));
0493:       setModalCancelar(false);
0494:       await carregarPedidos();
0495:       await carregarDetalhe(selectedPedidoId);
0496:     } catch (error: unknown) {
0497:       setErr(getErrorMessage(error, "Erro ao cancelar pedido"));
0498:     } finally {
0499:       setLoading(false);
0500:     }
0501:   }
0502:
0503:   async function onRecalcularPedido() {
0504:     if (!selectedPedidoId) return;
0505:     setErr(null);
0506:     setLoading(true);
0507:     try {
0508:       const resp = await recalcularPedido(selectedPedidoId, { forcar: false });
0509:       setLastAction(toRecord(resp));
0510:       await carregarPedidos();
0511:       await carregarDetalhe(selectedPedidoId);
0512:     } catch (error: unknown) {
0513:       setErr(getErrorMessage(error, "Erro ao recalcular pedido"));
0514:     } finally {
0515:       setLoading(false);
```

```
0516:     }
0517:   }
0518:
0519:   async function onEstornarPedido() {
0520:     if (!selectedPedidoId) return;
0521:     setErr(null);
0522:     setLoading(true);
0523:     try {
0524:       const payload: EstornarPedidoInput = {
0525:         filial: estornarForm.filial || filialView,
0526:         motivo: estornarForm.motivo || null,
0527:       };
0528:       const resp = await estornarPedido(selectedPedidoId, payload);
0529:       setLastAction(toRecord(resp));
0530:       setModalEstornar(false);
0531:       await carregarPedidos();
0532:       await carregarDetalhe(selectedPedidoId);
0533:     } catch (error: unknown) {
0534:       setErr(getErrorMessage(error, "Erro ao estornar pedido"));
0535:     } finally {
0536:       setLoading(false);
0537:     }
0538:   }
0539:
0540:   async function confirmarAcaoAtual() {
0541:     if (!confirmState) return;
0542:     setErr(null);
0543:     setLoading(true);
0544:     try {
0545:       await confirmState.run();
0546:       setConfirmState(null);
0547:     } catch (error: unknown) {
0548:       setErr(getErrorMessage(error, "Falha na operacao"));
0549:     } finally {
0550:       setLoading(false);
0551:     }
0552:   }
0553:
0554:   return (
0555:     <>
0556:       <PageHero
0557:         icon={ordersIcon}
0558:         backgroundImage={ordersBg}
0559:         backgroundPosition="center 46%"
0560:         eyebrow="Comercial"
0561:         title="Pedidos de venda"
0562:         description="Controle de criacao, itens, faturamento, cancelamento e estorno em uma un
ica trilha operacional."
0563:         metrics={[
0564:           { label: "Filial", value: filialView },
0565:           { label: "Pedidos", value: String(sortedRows.length) },
0566:           { label: "Modo", value: modoEnriquecido ? "Enriquecido" : "Simples" },
0567:         ]}
0568:         actions={
```

```
0569:           <>
0570:             <button className="btn primary" onClick={() => { setPedidoForm(emptyPedidoForm());
 setModalNovoPedido(true); }}>
0571:               Novo pedido
0572:             </button>
0573:             <button className="btn" onClick={() => { setModoEnriquecido((v) => !v); setPage(1)
; }}>
0574:               Alternar modo
0575:             </button>
0576:           </>
0577:         }
0578:       />
0579:
0580:       <div className="card">
0581:       {alertFilter && (
0582:         <div className="context-banner">
0583:           <div>
0584:             <div className="context-banner-title">Filtro aplicado por alerta</div>
0585:             <div className="context-banner-text">{alertLabelMap[alertFilter] ?? "Filtro contex
tual de alerta ativo."}</div>
0586:           </div>
0587:           <button
0588:             type="button"
0589:             className="btn ghost"
0590:             onClick={() => {
0591:               const next = new URLSearchParams(searchParams);
0592:               next.delete("alert");
0593:               setSearchParams(next);
0594:             }}
0595:           >
0596:             Limpar filtro do alerta
0597:           </button>
0598:         </div>
0599:       )}
0600:
0601:       <Toolbar
0602:         left={<div className="hint">Filial: <b>{filialView}</b></div>}
0603:         right={
0604:           <>
0605:             <select className="select" value={modoEnriquecido ? "enriquecido" : "simples"} onC
hange={(e) => {
0606:               setModoEnriquecido(e.target.value === "enriquecido");
0607:               setPage(1);
0608:             }}>
0609:               <option value="simples">Lista simples</option>
0610:               <option value="enriquecido">Lista enriquecida</option>
0611:             </select>
0612:             <select className="select" value={status} onChange={(e) => { setStatus(e.target.va
lue); setPage(1); }}>
0613:               <option value="">Status: todos</option>
0614:               <option value="ABERTO">ABERTO</option>
0615:               <option value="FATURADO">FATURADO</option>
0616:               <option value="CANCELADO">CANCELADO</option>
0617:             </select>
```

```
0618:             <select className="select" value={origem} onChange={(e) => { setOrigem(e.target.va
lue); setPage(1); }}>
0619:               <option value="">Origem: todas</option>
0620:               <option value="VENDA">VENDA</option>
0621:               <option value="RESERVA">RESERVA</option>
0622:             </select>
0623:             <input className="input" type="date" value={de} onChange={(e) => { setDe(e.target.
value); setPage(1); }} />
0624:             <input className="input" type="date" value={ate} onChange={(e) => { setAte(e.targe
t.value); setPage(1); }} />
0625:             <button className="btn" onClick={() => { setPage(1); carregarPedidos(); }} disable
d={loading}>Aplicar</button>
0626:             <button
0627:               className="btn ghost"
0628:               onClick={() => {
0629:                 setStatus("");
0630:                 setOrigem("");
0631:                 setDe("");
0632:                 setAte("");
0633:                 setPage(1);
0634:               }}
0635:               disabled={loading}
0636:             >
0637:               Limpar
0638:             </button>
0639:             <button className="btn primary" onClick={() => { setPedidoForm(emptyPedidoForm());
 setModalNovoPedido(true); }}>Novo pedido</button>
0640:           </>
0641:         }
0642:       />
0643:
0644:       {err && <div className="alert mt-12">{err}</div>}
0645:
0646:       <div className="section-title">Resumo</div>
0647:       <div className="stats-grid">
0648:         <div className="stat-card">
0649:           <div>
0650:             <div className="kpi-label">Pedidos (total)</div>
0651:             <div className="kpi">{total}</div>
0652:           </div>
0653:           <div className="stat-icon">PD</div>
0654:         </div>
0655:         <div className="stat-card">
0656:           <div>
0657:             <div className="kpi-label">Valor na pagina</div>
0658:             <div className="kpi">
0659:               {totalValor.toLocaleString("pt-BR", { style: "currency", currency: "BRL" })}
0660:             </div>
0661:           </div>
0662:           <div className="stat-icon success">R$</div>
0663:         </div>
0664:         <div className="stat-card">
0665:           <div>
0666:             <div className="kpi-label">Modo</div>
```

```
0667:             <div className="kpi">{modoEnriquecido ? "Enriq." : "Simples"}</div>
0668:           </div>
0669:           <div className="stat-icon solid">MD</div>
0670:         </div>
0671:       </div>
0672:
0673:       <Pager
0674:         page={page}
0675:         totalPages={totalPages}
0676:         pageSize={pageSize}
0677:         totalItems={sortedRows.length}
0678:         onPageChange={setPage}
0679:         onPageSizeChange={(size) => { setPageSize(size); setPage(1); }}
0680:         showPageSize
0681:       />
0682:
0683:       <DataTable
0684:         rows={sortedRows as unknown as PedidoRow[]}
0685:         columns={columns}
0686:         sortKey={sortKey}
0687:         sortDir={sortDir}
0688:         className="compact"
0689:         wrapperClassName="table-scroller"
0690:         onSortChange={(k, d) => { setSortKey(k as SortKey); setSortDir(d); }}
0691:         loading={loading}
0692:         emptyText="Nenhum pedido encontrado."
0693:       />
0694:
0695:       <PedidoDetalhePanel
0696:         selectedPedidoId={selectedPedidoId}
0697:         loading={loading}
0698:         pedidoDetalhe={pedidoDetalhe}
0699:         arResumo={arResumo}
0700:         itens={itens}
0701:         onRecalcularPedido={onRecalcularPedido}
0702:         onAbrirFaturar={() => {
0703:           setFaturarForm(emptyFaturarForm());
0704:           setModalFaturar(true);
0705:         }}
0706:         onAbrirCancelar={() => {
0707:           setCancelarForm(emptyCancelarForm());
0708:           setModalCancelar(true);
0709:         }}
0710:         onAbrirEstornar={() => {
0711:           setEstornarForm(emptyEstornarForm());
0712:           setModalEstornar(true);
0713:         }}
0714:         onAbrirAddItem={() => {
0715:           if (selectedPedidoId) abrirModalAddItem(selectedPedidoId);
0716:         }}
0717:         onAbrirEditItem={abrirModalEditItem}
0718:         onConfirmarExcluirItem={confirmarExcluirItem}
0719:         onAbrirCustoItem={(produtoCod) => {
0720:           if (!produtoCod) return;
```

```
0721:           navigate(`/custos/fornecedor?produto=${encodeURIComponent(produtoCod)}`);
0722:         }}
0723:       />
0724:
0725:       {lastAction && (
0726:         <div className="card mt-12">
0727:           <div style={{ fontWeight: 700 }}>Ultima resposta da API</div>
0728:           <pre style={{ margin: 0, whiteSpace: "pre-wrap" }}>{JSON.stringify(lastAction, null,
 2)}</pre>
0729:         </div>
0730:       )}
0731:
0732:       {modalNovoPedido && (
0733:         <Modal title="Criar pedido" onClose={() => setModalNovoPedido(false)}>
0734:           <div className="form-grid">
0735:             <div className="form-field"><label>Filial</label><input value={pedidoForm.filial}
onChange={(e) => setPedidoForm({ ...pedidoForm, filial: e.target.value })} /></div>
0736:             <div className="form-field"><label>Valor total</label><input type="number" step="0
.01" value={pedidoForm.valorTotal} onChange={(e) => setPedidoForm({ ...pedidoForm, valorTotal: e.tar
get.value })} /></div>
0737:             <div className="form-field"><label>ICMS (opcional)</label><input type="number" ste
p="0.01" value={pedidoForm.icms} onChange={(e) => setPedidoForm({ ...pedidoForm, icms: e.target.valu
e })} /></div>
0738:             <div className="form-field"><label>IPI (opcional)</label><input type="number" step
="0.01" value={pedidoForm.ipi} onChange={(e) => setPedidoForm({ ...pedidoForm, ipi: e.target.value }
)} /></div>
0739:             <div className="form-field"><label>Total bruto (opcional)</label><input type="numb
er" step="0.01" value={pedidoForm.totalBruto} onChange={(e) => setPedidoForm({ ...pedidoForm, totalB
ruto: e.target.value })} /></div>
0740:           </div>
0741:           <div className="form-actions">
0742:             <button className="btn" onClick={() => setModalNovoPedido(false)} disabled={loadin
g}>Cancelar</button>
0743:             <button className="btn primary" onClick={onCriarPedido} disabled={loading}>Criar</
button>
0744:           </div>
0745:         </Modal>
0746:       )}
0747:
0748:       {modalAddItem && (
0749:         <Modal title={`Adicionar item ao pedido ${selectedPedidoId ?? ""}`} onClose={() => set
ModalAddItem(false)}>
0750:       <div className="form-grid">
0751:         <div className="form-field"><label>Filial</label><input value={itemForm.filial} onChan
ge={(e) => setItemForm({ ...itemForm, filial: e.target.value })} /></div>
0752:             <div className="form-field">
0753:               <label>Produto</label>
0754:               <div className="row gap-10">
0755:                 <input
0756:                   value={itemForm.produto}
0757:                   onChange={(e) => {
0758:                     const next = e.target.value;
0759:                     setItemForm({ ...itemForm, produto: next });
0760:                     if (!next.trim()) setItemProdutoResultados([]);
```

```
0761:                    }}
0762:                    onBlur={() => carregarCmvAtualProduto(itemForm.produto)}
0763:                  />
0764:                  <button
0765:                    type="button"
0766:                    className="btn"
0767:                    disabled={loadingBuscaProdutoItem || !itemForm.produto.trim()}
0768:                    onClick={() => buscarProdutosParaItem(itemForm.produto)}
0769:                  >
0770:                    {loadingBuscaProdutoItem ? "Buscando..." : "Buscar"}
0771:                  </button>
0772:                  <button
0773:                    type="button"
0774:                    className="btn"
0775:                    disabled={loadingItemProduto || !itemForm.produto.trim()}
0776:                    onClick={() => carregarCmvAtualProduto(itemForm.produto)}
0777:                  >
0778:                    {loadingItemProduto ? "Carregando..." : "CM atual"}
0779:                  </button>
0780:                </div>
0781:                {itemProdutoResultados.length > 0 ? (
0782:                  <div className="search-results-panel mt-12">
0783:                    <table className="inline-search-table">
0784:                      <thead>
0785:                        <tr>
0786:                          <th>Codigo</th>
0787:                          <th>Descricao</th>
0788:                          <th>CM Atual</th>
0789:                          <th>Estoque</th>
0790:                          <th></th>
0791:                        </tr>
0792:                      </thead>
0793:                      <tbody>
0794:                        {itemProdutoResultados.map((produto) => (
0795:                          <tr key={`${produto.filial ?? "01"}:${produto.cod}`}>
0796:                            <td>{produto.cod}</td>
0797:                            <td>{produto.desc}</td>
0798:                            <td>{Number(produto.cm ?? 0).toLocaleString("pt-BR", { style: "curre
ncy", currency: "BRL" })}</td>
0799:                            <td>{Number(produto.estoque ?? 0)}</td>
0800:                            <td>
0801:                              <button type="button" className="btn" onClick={() => usarProdutoNo
Item(produto)}>
0802:                                Usar produto
0803:                              </button>
0804:                            </td>
0805:                          </tr>
0806:                        ))}
0807:                      </tbody>
0808:                    </table>
0809:                  </div>
0810:                ) : null}
0811:              </div>
```

```
0812:         <div className="form-field"><label>Qtd</label><input type="number" value={itemForm.qtd
} onChange={(e) => setItemForm({ ...itemForm, qtd: e.target.value })} /></div>
0813:         <div className="form-field"><label>Total</label><input type="number" step="0.01" value
={itemForm.total} onChange={(e) => setItemForm({ ...itemForm, total: e.target.value })} /></div>
0814:         <div className="form-field"><label>Preco unit (opcional)</label><input type="number" s
tep="0.01" value={itemForm.precoUnit} onChange={(e) => setItemForm({ ...itemForm, precoUnit: e.targe
t.value })} /></div>
0815:             <div className="form-field">
0816:               <label>CMV unit (opcional)</label>
0817:               <input
0818:                 type="number"
0819:                 step="0.01"
0820:                 value={itemForm.cmvUnit}
0821:                 onChange={(e) => setItemForm({ ...itemForm, cmvUnit: e.target.value })}
0822:               />
0823:               <div className="hint">Se vazio, o faturamento usa o B1_CM do produto. Se preench
ido, usa o CMV do item.</div>
0824:             </div>
0825:       </div>
0826:           <div className="form-actions">
0827:             <button className="btn" onClick={() => setModalAddItem(false)} disabled={loading}>
Cancelar</button>
0828:             <button className="btn primary" onClick={onAdicionarItem} disabled={loading}>Adici
onar</button>
0829:           </div>
0830:         </Modal>
0831:       )}
0832:
0833:       {modalEditItem && (
0834:         <Modal title={`Editar item ${editItemId ?? ""}`} onClose={() => setModalEditItem(false
)}>
0835:       <div className="form-grid">
0836:         <div className="form-field"><label>Filial</label><input value={editItemForm.filial} on
Change={(e) => setEditItemForm({ ...editItemForm, filial: e.target.value })} /></div>
0837:         <div className="form-field"><label>Qtd</label><input type="number" value={editItemForm
.qtd} onChange={(e) => setEditItemForm({ ...editItemForm, qtd: e.target.value })} /></div>
0838:         <div className="form-field"><label>Total</label><input type="number" step="0.01" value
={editItemForm.total} onChange={(e) => setEditItemForm({ ...editItemForm, total: e.target.value })}
/></div>
0839:         <div className="form-field"><label>Preco unit (opcional)</label><input type="number" s
tep="0.01" value={editItemForm.precoUnit} onChange={(e) => setEditItemForm({ ...editItemForm, precoU
nit: e.target.value })} /></div>
0840:         <div className="form-field"><label>CMV unit (opcional)</label><input type="number" ste
p="0.01" value={editItemForm.cmvUnit} onChange={(e) => setEditItemForm({ ...editItemForm, cmvUnit: e
.target.value })} /></div>
0841:       </div>
0842:           <div className="form-actions">
0843:             <button className="btn" onClick={() => setModalEditItem(false)} disabled={loading}
>Cancelar</button>
0844:             <button className="btn primary" onClick={onEditarItem} disabled={loading}>Salvar</
button>
0845:           </div>
0846:         </Modal>
0847:       )}
```

```
0848:
0849:       {modalFaturar && (
0850:         <Modal title={`Faturar pedido ${selectedPedidoId ?? ""}`} onClose={() => setModalFatur
ar(false)}>
0851:           <div className="form-grid">
0852:             <div className="form-field"><label>Filial</label><input value={faturarForm.filial}
 onChange={(e) => setFaturarForm({ ...faturarForm, filial: e.target.value })} /></div>
0853:             <div className="form-field"><label>Cliente cod (opcional)</label><input value={fat
urarForm.clienteCod} onChange={(e) => setFaturarForm({ ...faturarForm, clienteCod: e.target.value })
} /></div>
0854:             <div className="form-field"><label>Vencimento (dias)</label><input type="number" v
alue={faturarForm.vencDias} onChange={(e) => setFaturarForm({ ...faturarForm, vencDias: e.target.val
ue })} /></div>
0855:           </div>
0856:           <div className="form-actions">
0857:             <button className="btn" onClick={() => setModalFaturar(false)} disabled={loading}>
Cancelar</button>
0858:             <button className="btn primary" onClick={onFaturarPedido} disabled={loading}>Fatur
ar</button>
0859:           </div>
0860:         </Modal>
0861:       )}
0862:
0863:       {modalCancelar && (
0864:         <Modal title={`Cancelar pedido ${selectedPedidoId ?? ""}`} onClose={() => setModalCanc
elar(false)}>
0865:           <div className="form-grid">
0866:             <div className="form-field"><label>Filial</label><input value={cancelarForm.filial
} onChange={(e) => setCancelarForm({ ...cancelarForm, filial: e.target.value })} /></div>
0867:             <div className="form-field"><label>Modo</label><select value={cancelarForm.modo} o
nChange={(e) => setCancelarForm({ ...cancelarForm, modo: e.target.value as CancelarForm["modo"] })}>
<option value="AUTO">AUTO</option><option value="SIMPLES">SIMPLES</option><option value="ESTORNAR">E
STORNAR</option></select></div>
0868:             <div className="form-field"><label>Reativar reserva</label><select value={cancelar
Form.reativarReserva ? "true" : "false"} onChange={(e) => setCancelarForm({ ...cancelarForm, reativa
rReserva: e.target.value === "true" })}><option value="true">true</option><option value="false">fals
e</option></select></div>
0869:           </div>
0870:           <div className="form-actions">
0871:             <button className="btn" onClick={() => setModalCancelar(false)} disabled={loading}
>Voltar</button>
0872:             <button className="btn danger" onClick={onCancelarPedido} disabled={loading}>Cance
lar pedido</button>
0873:           </div>
0874:         </Modal>
0875:       )}
0876:
0877:       {modalEstornar && (
0878:         <Modal title={`Estornar pedido ${selectedPedidoId ?? ""}`} onClose={() => setModalEsto
rnar(false)}>
0879:           <div className="form-grid">
0880:             <div className="form-field"><label>Filial</label><input value={estornarForm.filial
} onChange={(e) => setEstornarForm({ ...estornarForm, filial: e.target.value })} /></div>
```

```
0881:             <div className="form-field col-span-all"><label>Motivo (opcional)</label><input va
lue={estornarForm.motivo} onChange={(e) => setEstornarForm({ ...estornarForm, motivo: e.target.value
 })} /></div>
0882:           </div>
0883:           <div className="form-actions">
0884:             <button className="btn" onClick={() => setModalEstornar(false)} disabled={loading}
>Voltar</button>
0885:             <button className="btn danger" onClick={onEstornarPedido} disabled={loading}>Estor
nar</button>
0886:           </div>
0887:         </Modal>
0888:       )}
0889:
0890:       {confirmState && (
0891:         <ConfirmDialog
0892:           title={confirmState.title}
0893:           message={confirmState.message}
0894:           danger={confirmState.danger}
0895:           confirmText={confirmState.confirmText}
0896:           loading={loading}
0897:           onClose={() => setConfirmState(null)}
0898:           onConfirm={confirmarAcaoAtual}
0899:         />
0900:       )}
0901:       </div>
0902:     </>
0903:   );
0904: }
```

==========
ARQUIVO: frontend/src/pages/produtosList.tsx
==========

```
0001: import { useState } from "react";
0002: import { Link } from "react-router-dom";
0003: import { getFilial } from "../state/filial";
0004: import { upsertProduto } from "../api/produtos";
0005: import { PageHero } from "../components/PageHero";
0006: import productsIcon from "../assets/shell-icons/products-ref.png";
0007: import productsBg from "../assets/hero-backgrounds/products.webp";
0008:
0009: type ProdutoForm = {
0010:   cod: string;
0011:   desc: string;
0012:   preco: string;
0013:   filial: string;
0014: };
0015:
0016: function emptyProduto(): ProdutoForm {
0017:   return { cod: "", desc: "", preco: "0", filial: getFilial() };
0018: }
0019:
0020: function toNumber(value: string, fallback = 0) {
0021:   const n = Number(value);
0022:   return Number.isFinite(n) ? n : fallback;
```

```
0023: }
0024:
0025: function getErrorMessage(error: unknown, fallback: string): string {
0026:   if (error instanceof Error && error.message) return error.message;
0027:   return fallback;
0028: }
0029:
0030: export function ProdutosList() {
0031:   const [form, setForm] = useState<ProdutoForm>(emptyProduto());
0032:   const [loading, setLoading] = useState(false);
0033:   const [err, setErr] = useState<string | null>(null);
0034:   const [lastResp, setLastResp] = useState<Record<string, unknown> | null>(null);
0035:
0036:   async function onSalvar() {
0037:     setErr(null);
0038:     setLoading(true);
0039:     try {
0040:       if (!form.cod.trim() || !form.desc.trim()) {
0041:         throw new Error("Codigo e descricao sao obrigatorios.");
0042:       }
0043:       const resp = await upsertProduto({
0044:         cod: form.cod.trim(),
0045:         desc: form.desc.trim(),
0046:         preco: toNumber(form.preco, 0),
0047:         filial: form.filial || getFilial(),
0048:       });
0049:       setLastResp(resp);
0050:       setForm(emptyProduto());
0051:     } catch (error: unknown) {
0052:       setErr(getErrorMessage(error, "Falha ao salvar produto"));
0053:     } finally {
0054:       setLoading(false);
0055:     }
0056:   }
0057:
0058:   return (
0059:     <>
0060:       <PageHero
0061:         icon={productsIcon}
0062:         backgroundImage={productsBg}
0063:         backgroundPosition="center 42%"
0064:         eyebrow="Cadastro e Estoque"
0065:         title="Produtos"
0066:         description="Cadastro base de produtos e ponto de partida para operacoes de estoque, c
usto medio e entrada por fornecedor."
0067:         metrics={[
0068:           { label: "Filial", value: form.filial || getFilial() },
0069:           { label: "Acao", value: "Upsert" },
0070:           { label: "Operacao", value: "Cadastro + Estoque" },
0071:         ]}
0072:         actions={
0073:           <Link
0074:             to={`/custos/fornecedor${form.cod.trim() ? `?produto=${encodeURIComponent(form.cod
.trim())}` : ""}`}
```

```
0075:             className="btn primary"
0076:           >
0077:             Formar custo
0078:           </Link>
0079:         }
0080:       />
0081:
0082:       <div className="card">
0083:       <div className="section-title">Produto</div>
0084:
0085:       {err && <div className="alert mt-12">{err}</div>}
0086:
0087:       <div className="stats-grid mt-12">
0088:         <div className="stat-card">
0089:           <div>
0090:             <div className="kpi-label">Acao</div>
0091:             <div className="kpi">Upsert</div>
0092:           </div>
0093:           <div className="stat-icon">PR</div>
0094:         </div>
0095:         <div className="stat-card">
0096:           <div>
0097:             <div className="kpi-label">Endpoint</div>
0098:             <div className="kpi">/produto</div>
0099:           </div>
0100:           <div className="stat-icon solid">API</div>
0101:         </div>
0102:         <div className="stat-card">
0103:           <div>
0104:             <div className="kpi-label">Perfil</div>
0105:             <div className="kpi">Admin</div>
0106:           </div>
0107:           <div className="stat-icon success">AUTH</div>
0108:         </div>
0109:       </div>
0110:
0111:       <div className="form-grid mt-12">
0112:         <div className="form-field">
0113:           <label>Codigo</label>
0114:           <input
0115:             value={form.cod}
0116:             onChange={(e) => setForm({ ...form, cod: e.target.value })}
0117:             placeholder="PRD001"
0118:           />
0119:         </div>
0120:         <div className="form-field">
0121:           <label>Descricao</label>
0122:           <input
0123:             value={form.desc}
0124:             onChange={(e) => setForm({ ...form, desc: e.target.value })}
0125:             placeholder="Produto X"
0126:           />
0127:         </div>
0128:         <div className="form-field">
```

```
0129:           <label>Preco</label>
0130:           <input
0131:             type="number"
0132:             step="0.01"
0133:             value={form.preco}
0134:             onChange={(e) => setForm({ ...form, preco: e.target.value })}
0135:           />
0136:         </div>
0137:         <div className="form-field">
0138:           <label>Filial</label>
0139:           <input
0140:             value={form.filial}
0141:             onChange={(e) => setForm({ ...form, filial: e.target.value })}
0142:           />
0143:         </div>
0144:         <div className="form-field">
0145:           <label> </label>
0146:           <div className="row gap-10">
0147:             <button className="btn primary" onClick={onSalvar} disabled={loading}>
0148:               Salvar produto
0149:             </button>
0150:             <Link
0151:               to={`/custos/fornecedor${form.cod.trim() ? `?produto=${encodeURIComponent(form.c
od.trim())}` : ""}`}
0152:               className="btn"
0153:             >
0154:               Ir para custo
0155:             </Link>
0156:           </div>
0157:         </div>
0158:       </div>
0159:
0160:       {lastResp && (
0161:         <div className="card soft mt-12">
0162:           <div style={{ fontWeight: 700 }}>Ultima resposta da API</div>
0163:           <pre style={{ margin: 0, whiteSpace: "pre-wrap" }}>{JSON.stringify(lastResp, null, 2
)}</pre>
0164:         </div>
0165:       )}
0166:       </div>
0167:     </>
0168:   );
0169: }
```

==========
ARQUIVO: frontend/src/pages/relatorioDRE.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
0002: import { Toolbar } from "../components/Toolbar";
0003: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0004: import { Pager } from "../components/Pager";
0005: import { PageHero } from "../components/PageHero";
0006: import { getFilial, onFilialChange } from "../state/filial";
0007: import { obterDre, type DreResponse, type DreLinha } from "../api/relatorios/dre";
```

```
0008: import reportsIcon from "../assets/shell-icons/reports.png";
0009: import dreBg from "../assets/hero-backgrounds/dre.png";
0010:
0011: function toNumber(value: unknown, fallback = 0): number {
0012:   const n = Number(value);
0013:   return Number.isFinite(n) ? n : fallback;
0014: }
0015:
0016: function toStringValue(value: unknown, fallback = ""): string {
0017:   if (value === null || value === undefined) return fallback;
0018:   const s = String(value).trim();
0019:   return s ? s : fallback;
0020: }
0021:
0022: function formatCurrency(value: unknown) {
0023:   return toNumber(value, 0).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
0024: }
0025:
0026: export function RelatorioDRE() {
0027:   const [filialView, setFilialView] = useState(getFilial());
0028:   const [de, setDe] = useState("");
0029:   const [ate, setAte] = useState("");
0030:   const [q, setQ] = useState("");
0031:
0032:   const [loading, setLoading] = useState(false);
0033:   const [err, setErr] = useState<string | null>(null);
0034:   const [dre, setDre] = useState<DreResponse | null>(null);
0035:
0036:   const [page, setPage] = useState(1);
0037:   const [pageSize, setPageSize] = useState(10);
0038:
0039:   const [sortKey, setSortKey] = useState("grupo");
0040:   const [sortDir, setSortDir] = useState<SortDir>("asc");
0041:
0042:   function getErrorMessage(error: unknown, fallback: string): string {
0043:     if (error instanceof Error && error.message) return error.message;
0044:     return fallback;
0045:   }
0046:
0047:   async function load() {
0048:     setErr(null);
0049:     setLoading(true);
0050:     try {
0051:       const filial = getFilial();
0052:       const data = await obterDre({ filial, de: de || null, ate: ate || null });
0053:       setDre(data);
0054:       setPage(1);
0055:     } catch (error: unknown) {
0056:       setErr(getErrorMessage(error, "Erro ao carregar DRE"));
0057:     } finally {
0058:       setLoading(false);
0059:     }
0060:   }
0061:
```

```
0062:   useEffect(() => {
0063:     load();
0064:
0065:     const unsub = onFilialChange((novaFilial) => {
0066:       setFilialView(novaFilial);
0067:       setErr(null);
0068:       setQ("");
0069:       setPage(1);
0070:       load();
0071:     });
0072:     return unsub;
0073:     // eslint-disable-next-line react-hooks/exhaustive-deps
0074:   }, []);
0075:
0076:   const rows = useMemo(() => {
0077:     const all = dre?.linhas ?? [];
0078:     const query = q.trim().toLowerCase();
0079:     if (!query) return all;
0080:     return all.filter((x) =>
0081:       toStringValue(x.grupo ?? (x as Record<string, unknown>).Grupo, "").toLowerCase().include
s(query)
0082:     );
0083:   }, [dre, q]);
0084:
0085:   const totalPages = useMemo(
0086:     () => Math.max(1, Math.ceil(rows.length / pageSize)),
0087:     [rows.length, pageSize]
0088:   );
0089:
0090:   const pagedRows = useMemo(() => {
0091:     const safePage = Math.min(Math.max(1, page), totalPages);
0092:     const start = (safePage - 1) * pageSize;
0093:     return rows.slice(start, start + pageSize);
0094:   }, [rows, page, pageSize, totalPages]);
0095:
0096:   const columns: Column<DreLinha>[] = useMemo(
0097:     () => [
0098:       {
0099:         key: "grupo",
0100:         header: "Grupo",
0101:         sortable: true,
0102:         value: (x) => toStringValue(x.grupo ?? (x as Record<string, unknown>).Grupo, "-"),
0103:       },
0104:       {
0105:         key: "valor",
0106:         header: "Valor",
0107:         sortable: true,
0108:         align: "right",
0109:         value: (x) => toNumber(x.valor ?? (x as Record<string, unknown>).Valor, 0),
0110:         render: (x) => formatCurrency(x.valor ?? (x as Record<string, unknown>).Valor),
0111:       },
0112:     ],
0113:     []
0114:   );
```

```
0115:
0116:   return (
0117:     <>
0118:       <PageHero
0119:         icon={reportsIcon}
0120:         backgroundImage={dreBg}
0121:         backgroundPosition="center 26%"
0122:         eyebrow="Relatorios gerenciais"
0123:         title="DRE"
0124:         description="Visao consolidada do resultado por periodo, com leitura rapida de grupos
e total acumulado."
0125:         metrics={[
0126:           { label: "Filial", value: filialView },
0127:           { label: "Linhas", value: String(rows.length) },
0128:           { label: "Total", value: dre ? formatCurrency(dre.total) : "Aguardando" },
0129:         ]}
0130:       />
0131:
0132:       <div className="card">
0133:       <Toolbar
0134:         left={
0135:           <div className="hint">
0136:             Filial: <b>{filialView}</b>
0137:           </div>
0138:         }
0139:         right={
0140:           <>
0141:             <input
0142:               className="input minw-240"
0143:               value={q}
0144:               onChange={(e) => { setQ(e.target.value); setPage(1); }}
0145:               placeholder="Filtrar grupo..."
0146:             />
0147:             <input
0148:               className="input"
0149:               type="date"
0150:               value={de}
0151:               onChange={(e) => setDe(e.target.value)}
0152:               aria-label="Data inicial"
0153:             />
0154:             <input
0155:               className="input"
0156:               type="date"
0157:               value={ate}
0158:               onChange={(e) => setAte(e.target.value)}
0159:               aria-label="Data final"
0160:             />
0161:             <button type="button" className="btn" onClick={load} disabled={loading}>
0162:               Aplicar
0163:             </button>
0164:           </>
0165:         }
0166:       />
0167:
```

```
0168:       {err && <div className="alert mt-12">{err}</div>}
0169:
0170:       <div className="section-title">Resumo</div>
0171:       {dre && (
0172:         <div className="stats-grid">
0173:           <div className="stat-card">
0174:             <div>
0175:               <div className="kpi-label">Total</div>
0176:               <div className="kpi">{formatCurrency(dre.total)}</div>
0177:             </div>
0178:             <div className="stat-icon success">R$</div>
0179:           </div>
0180:         </div>
0181:       )}
0182:
0183:       <div className="row-between mt-12">
0184:         <Pager
0185:           page={page}
0186:           totalPages={totalPages}
0187:           pageSize={pageSize}
0188:           totalItems={rows.length}
0189:           onPageChange={setPage}
0190:           onPageSizeChange={(ps) => {
0191:             setPageSize(ps);
0192:             setPage(1);
0193:           }}
0194:           showPageSize
0195:         />
0196:       </div>
0197:
0198:       <DataTable
0199:         rows={pagedRows}
0200:         columns={columns}
0201:         sortKey={sortKey}
0202:         sortDir={sortDir}
0203:         className="compact"
0204:         wrapperClassName="table-scroller"
0205:         onSortChange={(k, d) => {
0206:           setSortKey(k);
0207:           setSortDir(d);
0208:           setPage(1);
0209:         }}
0210:         loading={loading}
0211:         emptyText="Nenhuma linha no periodo."
0212:       />
0213:       </div>
0214:     </>
0215:   );
0216: }
```

==========
ARQUIVO: frontend/src/pages/relatorioMargemProduto.tsx
==========

```
0001: import { useEffect, useMemo, useState } from "react";
```

```
0002: import { Toolbar } from "../components/Toolbar";
0003: import { DataTable, type Column, type SortDir } from "../components/DataTable";
0004: import { Pager } from "../components/Pager";
0005: import { PageHero } from "../components/PageHero";
0006: import { getFilial, onFilialChange } from "../state/filial";
0007: import { obterMargemProduto, type MargemProdutoRow } from "../api/relatorios/margem-produto";
0008: import reportsIcon from "../assets/shell-icons/reports.png";
0009: import marginBg from "../assets/hero-backgrounds/margin.webp";
0010:
0011: function toNumber(value: unknown, fallback = 0): number {
0012:   const n = Number(value);
0013:   return Number.isFinite(n) ? n : fallback;
0014: }
0015:
0016: function toStringValue(value: unknown, fallback = ""): string {
0017:   if (value === null || value === undefined) return fallback;
0018:   const s = String(value).trim();
0019:   return s ? s : fallback;
0020: }
0021:
0022: function formatCurrency(value: unknown) {
0023:   return toNumber(value, 0).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
0024: }
0025:
0026: export function RelatorioMargemProduto() {
0027:   const [filialView, setFilialView] = useState(getFilial());
0028:   const [de, setDe] = useState("");
0029:   const [ate, setAte] = useState("");
0030:   const [q, setQ] = useState("");
0031:
0032:   const [loading, setLoading] = useState(false);
0033:   const [err, setErr] = useState<string | null>(null);
0034:   const [rows, setRows] = useState<MargemProdutoRow[]>([]);
0035:
0036:   const [page, setPage] = useState(1);
0037:   const [pageSize, setPageSize] = useState(10);
0038:
0039:   const [sortKey, setSortKey] = useState("margem");
0040:   const [sortDir, setSortDir] = useState<SortDir>("desc");
0041:
0042:   function getErrorMessage(error: unknown, fallback: string): string {
0043:     if (error instanceof Error && error.message) return error.message;
0044:     return fallback;
0045:   }
0046:
0047:   async function load() {
0048:     setErr(null);
0049:     setLoading(true);
0050:     try {
0051:       const filial = getFilial();
0052:       const data = await obterMargemProduto({ filial, de: de || null, ate: ate || null });
0053:       setRows(Array.isArray(data) ? data : []);
0054:       setPage(1);
0055:     } catch (error: unknown) {
```

```
0056:       setErr(getErrorMessage(error, "Erro ao carregar margem por produto"));
0057:     } finally {
0058:       setLoading(false);
0059:     }
0060:   }
0061:
0062:   useEffect(() => {
0063:     load();
0064:
0065:     const unsub = onFilialChange((novaFilial) => {
0066:       setFilialView(novaFilial);
0067:       setErr(null);
0068:       setQ("");
0069:       setPage(1);
0070:       load();
0071:     });
0072:     return unsub;
0073:     // eslint-disable-next-line react-hooks/exhaustive-deps
0074:   }, []);
0075:
0076:   const filtered = useMemo(() => {
0077:     const query = q.trim().toLowerCase();
0078:     if (!query) return rows;
0079:     return rows.filter((x) => toStringValue(x.produto, "").toLowerCase().includes(query));
0080:   }, [rows, q]);
0081:
0082:   const totalPages = useMemo(
0083:     () => Math.max(1, Math.ceil(filtered.length / pageSize)),
0084:     [filtered.length, pageSize]
0085:   );
0086:
0087:   const pagedRows = useMemo(() => {
0088:     const safePage = Math.min(Math.max(1, page), totalPages);
0089:     const start = (safePage - 1) * pageSize;
0090:     return filtered.slice(start, start + pageSize);
0091:   }, [filtered, page, pageSize, totalPages]);
0092:
0093:   const totalReceita = useMemo(() => filtered.reduce((acc, r) => acc + toNumber(r.receita, 0),
 0), [filtered]);
0094:   const totalCmv = useMemo(() => filtered.reduce((acc, r) => acc + toNumber(r.cmv, 0), 0), [fi
ltered]);
0095:   const totalMargem = useMemo(() => filtered.reduce((acc, r) => acc + toNumber(r.margem, 0), 0
), [filtered]);
0096:
0097:   const columns: Column<MargemProdutoRow>[] = useMemo(
0098:     () => [
0099:       { key: "produto", header: "Produto", sortable: true },
0100:       {
0101:         key: "receita",
0102:         header: "Receita",
0103:         sortable: true,
0104:         align: "right",
0105:         render: (x) => formatCurrency(x.receita),
0106:       },
```

```
0107:       {
0108:         key: "cmv",
0109:         header: "CMV",
0110:         sortable: true,
0111:         align: "right",
0112:         render: (x) => formatCurrency(x.cmv),
0113:       },
0114:       {
0115:         key: "margem",
0116:         header: "Margem",
0117:         sortable: true,
0118:         align: "right",
0119:         render: (x) => formatCurrency(x.margem),
0120:       },
0121:     ],
0122:     []
0123:   );
0124:
0125:   return (
0126:     <>
0127:       <PageHero
0128:         icon={reportsIcon}
0129:         backgroundImage={marginBg}
0130:         backgroundPosition="center 36%"
0131:         eyebrow="Relatorios gerenciais"
0132:         title="Margem por produto"
0133:         description="Leitura sintetica da relacao entre receita, CMV e margem para apoiar prio
rizacao comercial e estoque."
0134:         metrics={[
0135:           { label: "Filial", value: filialView },
0136:           { label: "Receita", value: formatCurrency(totalReceita) },
0137:           { label: "Margem", value: formatCurrency(totalMargem) },
0138:         ]}
0139:       />
0140:
0141:       <div className="card">
0142:       <Toolbar
0143:         left={
0144:           <div className="hint">
0145:             Filial: <b>{filialView}</b>
0146:           </div>
0147:         }
0148:         right={
0149:           <>
0150:             <input
0151:               className="input minw-240"
0152:               value={q}
0153:               onChange={(e) => { setQ(e.target.value); setPage(1); }}
0154:               placeholder="Filtrar produto..."
0155:             />
0156:             <input
0157:               className="input"
0158:               type="date"
0159:               value={de}
```

```
0160:               onChange={(e) => setDe(e.target.value)}
0161:               aria-label="Data inicial"
0162:             />
0163:             <input
0164:               className="input"
0165:               type="date"
0166:               value={ate}
0167:               onChange={(e) => setAte(e.target.value)}
0168:               aria-label="Data final"
0169:             />
0170:             <button type="button" className="btn" onClick={load} disabled={loading}>
0171:               Aplicar
0172:             </button>
0173:             <button
0174:               type="button"
0175:               className="btn ghost"
0176:               onClick={() => {
0177:                 setQ("");
0178:                 setDe("");
0179:                 setAte("");
0180:                 setPage(1);
0181:               }}
0182:               disabled={loading}
0183:             >
0184:               Limpar
0185:             </button>
0186:           </>
0187:         }
0188:       />
0189:
0190:       {err && <div className="alert mt-12">{err}</div>}
0191:
0192:       <div className="section-title">Resumo</div>
0193:       <div className="stats-grid">
0194:         <div className="stat-card">
0195:           <div>
0196:             <div className="kpi-label">Receita</div>
0197:             <div className="kpi">{formatCurrency(totalReceita)}</div>
0198:           </div>
0199:           <div className="stat-icon">R$</div>
0200:         </div>
0201:         <div className="stat-card">
0202:           <div>
0203:             <div className="kpi-label">CMV</div>
0204:             <div className="kpi">{formatCurrency(totalCmv)}</div>
0205:           </div>
0206:           <div className="stat-icon solid">C</div>
0207:         </div>
0208:         <div className="stat-card">
0209:           <div>
0210:             <div className="kpi-label">Margem</div>
0211:             <div className="kpi">{formatCurrency(totalMargem)}</div>
0212:           </div>
0213:           <div className="stat-icon success">M</div>
```

```
0214:         </div>
0215:       </div>
0216:
0217:       <div className="row-between mt-12">
0218:         <Pager
0219:           page={page}
0220:           totalPages={totalPages}
0221:           pageSize={pageSize}
0222:           totalItems={filtered.length}
0223:           onPageChange={setPage}
0224:           onPageSizeChange={(ps) => {
0225:             setPageSize(ps);
0226:             setPage(1);
0227:           }}
0228:           showPageSize
0229:         />
0230:       </div>
0231:
0232:       <DataTable
0233:         rows={pagedRows}
0234:         columns={columns}
0235:         sortKey={sortKey}
0236:         sortDir={sortDir}
0237:         className="compact"
0238:         wrapperClassName="table-scroller"
0239:         onSortChange={(k, d) => {
0240:           setSortKey(k);
0241:           setSortDir(d);
0242:           setPage(1);
0243:         }}
0244:         loading={loading}
0245:         emptyText="Nenhum produto encontrado."
0246:       />
0247:       </div>
0248:     </>
0249:   );
0250: }
```

==========
ARQUIVO: frontend/src/pages/reservasList.tsx
==========

```
0001: import { useState } from "react";
0002: import { getFilial } from "../state/filial";
0003: import { criarReserva, cancelarReserva, confirmarReserva } from "../api/reservas";
0004: import { PageHero } from "../components/PageHero";
0005: import reservationsIcon from "../assets/shell-icons/reservations.png";
0006: import reservationsBg from "../assets/hero-backgrounds/reservations.avif";
0007:
0008: type ReservaForm = {
0009:   cod: string;
0010:   qtd: string;
0011:   filial: string;
0012:   clienteCod: string;
0013:   tabelaCod: string;
```

```
0014: };
0015:
0016: type CancelarForm = {
0017:   reservaId: string;
0018: };
0019:
0020: type ConfirmarForm = {
0021:   reservaId: string;
0022:   clienteCod: string;
0023:   vencDias: string;
0024: };
0025:
0026: function emptyReserva(): ReservaForm {
0027:   return {
0028:     cod: "",
0029:     qtd: "1",
0030:     filial: getFilial(),
0031:     clienteCod: "",
0032:     tabelaCod: "0001",
0033:   };
0034: }
0035:
0036: function emptyCancelar(): CancelarForm {
0037:   return { reservaId: "" };
0038: }
0039:
0040: function emptyConfirmar(): ConfirmarForm {
0041:   return { reservaId: "", clienteCod: "", vencDias: "30" };
0042: }
0043:
0044: function toNumber(value: string, fallback = 0) {
0045:   const n = Number(value);
0046:   return Number.isFinite(n) ? n : fallback;
0047: }
0048:
0049: function getErrorMessage(error: unknown, fallback: string): string {
0050:   if (error instanceof Error && error.message) return error.message;
0051:   return fallback;
0052: }
0053:
0054: export function ReservasList() {
0055:   const [reservaForm, setReservaForm] = useState<ReservaForm>(emptyReserva());
0056:   const [cancelForm, setCancelForm] = useState<CancelarForm>(emptyCancelar());
0057:   const [confirmForm, setConfirmForm] = useState<ConfirmarForm>(emptyConfirmar());
0058:
0059:   const [loading, setLoading] = useState(false);
0060:   const [err, setErr] = useState<string | null>(null);
0061:   const [lastResp, setLastResp] = useState<Record<string, unknown> | null>(null);
0062:
0063:   async function onCriarReserva() {
0064:     setErr(null);
0065:     setLoading(true);
0066:     try {
0067:       if (!reservaForm.cod.trim()) throw new Error("Produto (codigo) e obrigatorio.");
```

```
0068:       const resp = await criarReserva({
0069:         cod: reservaForm.cod.trim(),
0070:         qtd: toNumber(reservaForm.qtd, 1),
0071:         filial: reservaForm.filial || getFilial(),
0072:         cliente_cod: reservaForm.clienteCod.trim() || null,
0073:         tabela_cod: reservaForm.tabelaCod.trim() || "0001",
0074:       });
0075:       setLastResp(resp);
0076:       setReservaForm(emptyReserva());
0077:     } catch (error: unknown) {
0078:       setErr(getErrorMessage(error, "Falha ao criar reserva"));
0079:     } finally {
0080:       setLoading(false);
0081:     }
0082:   }
0083:
0084:   async function onCancelarReserva() {
0085:     setErr(null);
0086:     setLoading(true);
0087:     try {
0088:       const reservaId = toNumber(cancelForm.reservaId, 0);
0089:       if (!reservaId) throw new Error("Informe o ID da reserva.");
0090:       const resp = await cancelarReserva({ reserva_id: reservaId });
0091:       setLastResp(resp);
0092:       setCancelForm(emptyCancelar());
0093:     } catch (error: unknown) {
0094:       setErr(getErrorMessage(error, "Falha ao cancelar reserva"));
0095:     } finally {
0096:       setLoading(false);
0097:     }
0098:   }
0099:
0100:   async function onConfirmarReserva() {
0101:     setErr(null);
0102:     setLoading(true);
0103:     try {
0104:       const reservaId = toNumber(confirmForm.reservaId, 0);
0105:       if (!reservaId) throw new Error("Informe o ID da reserva.");
0106:       const resp = await confirmarReserva({
0107:         reserva_id: reservaId,
0108:         cliente_cod: confirmForm.clienteCod.trim() || null,
0109:         venc_dias: toNumber(confirmForm.vencDias, 30),
0110:       });
0111:       setLastResp(resp);
0112:       setConfirmForm(emptyConfirmar());
0113:     } catch (error: unknown) {
0114:       setErr(getErrorMessage(error, "Falha ao confirmar reserva"));
0115:     } finally {
0116:       setLoading(false);
0117:     }
0118:   }
0119:
0120:   return (
0121:     <>
```

```
0122:       <PageHero
0123:         icon={reservationsIcon}
0124:         backgroundImage={reservationsBg}
0125:         backgroundPosition="center 42%"
0126:         eyebrow="Estoque e Comercial"
0127:         title="Reservas"
0128:         description="Criacao, cancelamento e confirmacao de reservas com apoio de tabela de pr
eco e transicao para o fluxo comercial."
0129:         metrics={[
0130:           { label: "Filial", value: reservaForm.filial || getFilial() },
0131:           { label: "Tabela", value: reservaForm.tabelaCod || "0001" },
0132:           { label: "Perfil", value: "Admin / Operador" },
0133:         ]}
0134:       />
0135:
0136:       <div className="card">
0137:       <div className="section-title">Reservas</div>
0138:
0139:       {err && <div className="alert mt-12">{err}</div>}
0140:
0141:       <div className="stats-grid mt-12">
0142:         <div className="stat-card">
0143:           <div>
0144:             <div className="kpi-label">Acoes</div>
0145:             <div className="kpi">Criar / Cancelar / Confirmar</div>
0146:           </div>
0147:           <div className="stat-icon">RS</div>
0148:         </div>
0149:         <div className="stat-card">
0150:           <div>
0151:             <div className="kpi-label">Endereco</div>
0152:             <div className="kpi">/reserva</div>
0153:           </div>
0154:           <div className="stat-icon solid">API</div>
0155:         </div>
0156:         <div className="stat-card">
0157:           <div>
0158:             <div className="kpi-label">Perfil</div>
0159:             <div className="kpi">Admin/Operador</div>
0160:           </div>
0161:           <div className="stat-icon success">AUTH</div>
0162:         </div>
0163:       </div>
0164:
0165:       <div className="section-title">Criar reserva</div>
0166:       <div className="form-grid mt-12">
0167:         <div className="form-field">
0168:           <label>Produto (cod)</label>
0169:           <input
0170:             value={reservaForm.cod}
0171:             onChange={(e) => setReservaForm({ ...reservaForm, cod: e.target.value })}
0172:             placeholder="PROD001"
0173:           />
0174:         </div>
```

```
0175:         <div className="form-field">
0176:           <label>Quantidade</label>
0177:           <input
0178:             type="number"
0179:             value={reservaForm.qtd}
0180:             onChange={(e) => setReservaForm({ ...reservaForm, qtd: e.target.value })}
0181:           />
0182:         </div>
0183:         <div className="form-field">
0184:           <label>Filial</label>
0185:           <input
0186:             value={reservaForm.filial}
0187:             onChange={(e) => setReservaForm({ ...reservaForm, filial: e.target.value })}
0188:           />
0189:         </div>
0190:         <div className="form-field">
0191:           <label>Tabela (cod)</label>
0192:           <input
0193:             value={reservaForm.tabelaCod}
0194:             onChange={(e) => setReservaForm({ ...reservaForm, tabelaCod: e.target.value })}
0195:             placeholder="0001"
0196:           />
0197:         </div>
0198:         <div className="form-field">
0199:           <label>Cliente (cod, opcional)</label>
0200:           <input
0201:             value={reservaForm.clienteCod}
0202:             onChange={(e) => setReservaForm({ ...reservaForm, clienteCod: e.target.value })}
0203:             placeholder="CLI001"
0204:           />
0205:         </div>
0206:         <div className="form-field">
0207:           <label> </label>
0208:           <button className="btn primary" onClick={onCriarReserva} disabled={loading}>
0209:             Criar reserva
0210:           </button>
0211:         </div>
0212:       </div>
0213:
0214:       <div className="section-title">Cancelar reserva</div>
0215:       <div className="form-grid mt-12">
0216:         <div className="form-field">
0217:           <label>ID da reserva</label>
0218:           <input
0219:             value={cancelForm.reservaId}
0220:             onChange={(e) => setCancelForm({ ...cancelForm, reservaId: e.target.value })}
0221:             placeholder="123"
0222:           />
0223:         </div>
0224:         <div className="form-field">
0225:           <label> </label>
0226:           <button className="btn danger" onClick={onCancelarReserva} disabled={loading}>
0227:             Cancelar
0228:           </button>
```

```
0229:         </div>
0230:       </div>
0231:
0232:       <div className="section-title">Confirmar reserva</div>
0233:       <div className="form-grid mt-12">
0234:         <div className="form-field">
0235:           <label>ID da reserva</label>
0236:           <input
0237:             value={confirmForm.reservaId}
0238:             onChange={(e) => setConfirmForm({ ...confirmForm, reservaId: e.target.value })}
0239:             placeholder="123"
0240:           />
0241:         </div>
0242:         <div className="form-field">
0243:           <label>Cliente (cod, opcional)</label>
0244:           <input
0245:             value={confirmForm.clienteCod}
0246:             onChange={(e) => setConfirmForm({ ...confirmForm, clienteCod: e.target.value })}
0247:             placeholder="CLI001"
0248:           />
0249:         </div>
0250:         <div className="form-field">
0251:           <label>Vencimento (dias)</label>
0252:           <input
0253:             type="number"
0254:             value={confirmForm.vencDias}
0255:             onChange={(e) => setConfirmForm({ ...confirmForm, vencDias: e.target.value })}
0256:           />
0257:         </div>
0258:         <div className="form-field">
0259:           <label> </label>
0260:           <button className="btn" onClick={onConfirmarReserva} disabled={loading}>
0261:             Confirmar
0262:           </button>
0263:         </div>
0264:       </div>
0265:
0266:       {lastResp && (
0267:         <div className="card soft mt-12">
0268:           <div style={{ fontWeight: 700 }}>Ultima resposta da API</div>
0269:           <pre style={{ margin: 0, whiteSpace: "pre-wrap" }}>{JSON.stringify(lastResp, null, 2
)}</pre>
0270:         </div>
0271:       )}
0272:       </div>
0273:     </>
0274:   );
0275: }
```

==========
ARQUIVO: frontend/src/pages/usuariosList.tsx
==========

```
0001: import { useState } from "react";
0002: import { bootstrapAdmin, criarUsuario } from "../api/usuarios";
```

```
0003: import { PageHero } from "../components/PageHero";
0004: import usersIcon from "../assets/shell-icons/users.png";
0005: import usersBg from "../assets/hero-backgrounds/users.webp";
0006:
0007: type BootstrapForm = {
0008:   login: string;
0009:   senha: string;
0010: };
0011:
0012: type UsuarioForm = {
0013:   login: string;
0014:   senha: string;
0015:   perfil: string;
0016: };
0017:
0018: function emptyBootstrap(): BootstrapForm {
0019:   return { login: "", senha: "" };
0020: }
0021:
0022: function emptyUsuario(): UsuarioForm {
0023:   return { login: "", senha: "", perfil: "operador" };
0024: }
0025:
0026: function getErrorMessage(error: unknown, fallback: string): string {
0027:   if (error instanceof Error && error.message) return error.message;
0028:   return fallback;
0029: }
0030:
0031: export function UsuariosList() {
0032:   const [bootForm, setBootForm] = useState<BootstrapForm>(emptyBootstrap());
0033:   const [userForm, setUserForm] = useState<UsuarioForm>(emptyUsuario());
0034:
0035:   const [loading, setLoading] = useState(false);
0036:   const [err, setErr] = useState<string | null>(null);
0037:   const [lastResp, setLastResp] = useState<Record<string, unknown> | null>(null);
0038:
0039:   async function onBootstrap() {
0040:     setErr(null);
0041:     setLoading(true);
0042:     try {
0043:       if (!bootForm.login.trim() || !bootForm.senha.trim()) {
0044:         throw new Error("Login e senha sao obrigatorios.");
0045:       }
0046:       const resp = await bootstrapAdmin({
0047:         login: bootForm.login.trim(),
0048:         senha: bootForm.senha,
0049:       });
0050:       setLastResp(resp as Record<string, unknown>);
0051:       setBootForm(emptyBootstrap());
0052:     } catch (error: unknown) {
0053:       setErr(getErrorMessage(error, "Falha no bootstrap"));
0054:     } finally {
0055:       setLoading(false);
0056:     }
```

```
0057:   }
0058:
0059:   async function onCriarUsuario() {
0060:     setErr(null);
0061:     setLoading(true);
0062:     try {
0063:       if (!userForm.login.trim() || !userForm.senha.trim() || !userForm.perfil.trim()) {
0064:         throw new Error("Login, senha e perfil sao obrigatorios.");
0065:       }
0066:       const resp = await criarUsuario({
0067:         login: userForm.login.trim(),
0068:         senha: userForm.senha,
0069:         perfil: userForm.perfil,
0070:       });
0071:       setLastResp(resp as Record<string, unknown>);
0072:       setUserForm(emptyUsuario());
0073:     } catch (error: unknown) {
0074:       setErr(getErrorMessage(error, "Falha ao criar usuario"));
0075:     } finally {
0076:       setLoading(false);
0077:     }
0078:   }
0079:
0080:   return (
0081:     <>
0082:       <PageHero
0083:         icon={usersIcon}
0084:         backgroundImage={usersBg}
0085:         backgroundPosition="center 24%"
0086:         eyebrow="Administracao"
0087:         title="Usuarios e bootstrap"
0088:         description="Controle do admin inicial e criacao de novos usuarios para operacao, audi
toria e administracao."
0089:         metrics={[
0090:           { label: "Bootstrap", value: "One-time" },
0091:           { label: "Criacao", value: "Admin only" },
0092:           { label: "Contrato", value: "/usuarios" },
0093:         ]}
0094:       />
0095:
0096:       <div className="card">
0097:       <div className="section-title">Usuarios</div>
0098:
0099:       {err && <div className="alert mt-12">{err}</div>}
0100:
0101:       <div className="stats-grid mt-12">
0102:         <div className="stat-card">
0103:           <div>
0104:             <div className="kpi-label">Acoes</div>
0105:             <div className="kpi">Bootstrap / Criar</div>
0106:           </div>
0107:           <div className="stat-icon">US</div>
0108:         </div>
0109:         <div className="stat-card">
```

```
0110:           <div>
0111:             <div className="kpi-label">Requer login</div>
0112:             <div className="kpi">Somente /usuarios</div>
0113:           </div>
0114:           <div className="stat-icon solid">AUTH</div>
0115:         </div>
0116:         <div className="stat-card">
0117:           <div>
0118:             <div className="kpi-label">Bootstrap</div>
0119:             <div className="kpi">Admin inicial</div>
0120:           </div>
0121:           <div className="stat-icon success">ADM</div>
0122:         </div>
0123:       </div>
0124:
0125:       <div className="form-grid mt-12">
0126:         <div className="form-field col-span-all">
0127:           <label>Bootstrap (criar admin inicial)</label>
0128:           <div className="hint">Use apenas uma vez. Depois disso, o endpoint bloqueia.</div>
0129:         </div>
0130:         <div className="form-field">
0131:           <label>Login</label>
0132:           <input
0133:             value={bootForm.login}
0134:             onChange={(e) => setBootForm({ ...bootForm, login: e.target.value })}
0135:             placeholder="admin"
0136:           />
0137:         </div>
0138:         <div className="form-field">
0139:           <label>Senha</label>
0140:           <input
0141:             type="password"
0142:             value={bootForm.senha}
0143:             onChange={(e) => setBootForm({ ...bootForm, senha: e.target.value })}
0144:             placeholder="••••••"
0145:           />
0146:         </div>
0147:         <div className="form-field">
0148:           <label> </label>
0149:           <button className="btn primary" onClick={onBootstrap} disabled={loading}>
0150:             Executar bootstrap
0151:           </button>
0152:         </div>
0153:       </div>
0154:
0155:       <div className="form-grid mt-12">
0156:         <div className="form-field col-span-all">
0157:           <label>Criar usuario</label>
0158:           <div className="hint">Requer login como admin.</div>
0159:         </div>
0160:         <div className="form-field">
0161:           <label>Login</label>
0162:           <input
0163:             value={userForm.login}
```

```
0164:             onChange={(e) => setUserForm({ ...userForm, login: e.target.value })}
0165:             placeholder="usuario"
0166:           />
0167:         </div>
0168:         <div className="form-field">
0169:           <label>Senha</label>
0170:           <input
0171:             type="password"
0172:             value={userForm.senha}
0173:             onChange={(e) => setUserForm({ ...userForm, senha: e.target.value })}
0174:             placeholder="••••••"
0175:           />
0176:         </div>
0177:         <div className="form-field">
0178:           <label>Perfil</label>
0179:           <select
0180:             className="select"
0181:             value={userForm.perfil}
0182:             onChange={(e) => setUserForm({ ...userForm, perfil: e.target.value })}
0183:           >
0184:             <option value="admin">admin</option>
0185:             <option value="operador">operador</option>
0186:             <option value="auditor">auditor</option>
0187:           </select>
0188:         </div>
0189:         <div className="form-field">
0190:           <label> </label>
0191:           <button className="btn" onClick={onCriarUsuario} disabled={loading}>
0192:             Criar usuario
0193:           </button>
0194:         </div>
0195:       </div>
0196:
0197:       {lastResp && (
0198:         <div className="card soft mt-12">
0199:           <div style={{ fontWeight: 700 }}>Ultima resposta da API</div>
0200:           <pre style={{ margin: 0, whiteSpace: "pre-wrap" }}>{JSON.stringify(lastResp, null, 2
)}</pre>
0201:         </div>
0202:       )}
0203:       </div>
0204:     </>
0205:   );
0206: }
```

==========
ARQUIVO: frontend/src/state/filial.ts
==========

```
0001: const KEY = "filial";
0002:
0003: export function getFilial(): string {
0004:   return localStorage.getItem(KEY) ?? "01";
0005: }
0006:
```

```
0007: export function setFilial(filial: string) {
0008:   localStorage.setItem(KEY, filial);
0009:   // notifica telas abertas
0010:   window.dispatchEvent(new CustomEvent("filial:change", { detail: filial }));
0011: }
0012:
0013: export function onFilialChange(cb: (filial: string) => void) {
0014:   const handler = (e: Event) => {
0015:     const custom = e as CustomEvent<string>;
0016:     cb(custom.detail ?? getFilial());
0017:   };
0018:   window.addEventListener("filial:change", handler as EventListener);
0019:   return () => window.removeEventListener("filial:change", handler as EventListener);
0020: }
```

==========
ARQUIVO: frontend/src/state/uiPrefs.ts
==========

```
0001: const AREA_KEY = "ui:area";
0002: const FAVORITES_KEY = "ui:favorites";
0003:
0004: const AREA_EVENT = "ui:area-change";
0005: const FAVORITES_EVENT = "ui:favorites-change";
0006:
0007: export const AVAILABLE_AREAS = [
0008:   "Area padrao",
0009:   "Comercial",
0010:   "Financeiro",
0011:   "Cadastros",
0012:   "Administracao",
0013: ] as const;
0014:
0015: export type UiArea = (typeof AVAILABLE_AREAS)[number];
0016:
0017: export function getArea(): UiArea {
0018:   const value = localStorage.getItem(AREA_KEY) as UiArea | null;
0019:   return value && AVAILABLE_AREAS.includes(value) ? value : "Area padrao";
0020: }
0021:
0022: export function setArea(area: UiArea) {
0023:   localStorage.setItem(AREA_KEY, area);
0024:   window.dispatchEvent(new CustomEvent<UiArea>(AREA_EVENT, { detail: area }));
0025: }
0026:
0027: export function onAreaChange(cb: (area: UiArea) => void) {
0028:   const handler = (event: Event) => {
0029:     const custom = event as CustomEvent<UiArea>;
0030:     cb(custom.detail ?? getArea());
0031:   };
0032:   window.addEventListener(AREA_EVENT, handler as EventListener);
0033:   return () => window.removeEventListener(AREA_EVENT, handler as EventListener);
0034: }
0035:
0036: export function getFavorites(): string[] {
```

```
0037:   try {
0038:     const raw = localStorage.getItem(FAVORITES_KEY);
0039:     const parsed = raw ? JSON.parse(raw) : [];
0040:     return Array.isArray(parsed) ? parsed.filter((item): item is string => typeof item === "st
ring") : [];
0041:   } catch {
0042:     return [];
0043:   }
0044: }
0045:
0046: function setFavoritesInternal(favorites: string[]) {
0047:   localStorage.setItem(FAVORITES_KEY, JSON.stringify(favorites));
0048:   window.dispatchEvent(new CustomEvent<string[]>(FAVORITES_EVENT, { detail: favorites }));
0049: }
0050:
0051: export function toggleFavorite(path: string) {
0052:   const current = getFavorites();
0053:   if (current.includes(path)) {
0054:     setFavoritesInternal(current.filter((item) => item !== path));
0055:     return;
0056:   }
0057:   setFavoritesInternal([...current, path]);
0058: }
0059:
0060: export function onFavoritesChange(cb: (favorites: string[]) => void) {
0061:   const handler = (event: Event) => {
0062:     const custom = event as CustomEvent<string[]>;
0063:     cb(custom.detail ?? getFavorites());
0064:   };
0065:   window.addEventListener(FAVORITES_EVENT, handler as EventListener);
0066:   return () => window.removeEventListener(FAVORITES_EVENT, handler as EventListener);
0067: }
```

==========
ARQUIVO: frontend/src/styles/app.css
==========

```
0001: @import url("https://fonts.googleapis.com/css2?family=Manrope:wght@400;500;600;700;800&family=
Rajdhani:wght@500;600;700&display=swap");
0002:
0003: :root {
0004:   --bg: #091623;
0005:   --bg-soft: #102235;
0006:   --bg-deep: #07111c;
0007:   --panel: rgba(8, 18, 30, 0.82);
0008:   --panel-strong: rgba(9, 22, 38, 0.94);
0009:   --panel-soft: rgba(17, 38, 58, 0.68);
0010:   --line: rgba(150, 197, 255, 0.15);
0011:   --line-strong: rgba(125, 183, 255, 0.28);
0012:   --text: #eef6ff;
0013:   --text-soft: #a8bbcf;
0014:   --text-dim: #7891aa;
0015:   --primary: #1d65f0;
0016:   --primary-strong: #2e7bff;
0017:   --primary-soft: rgba(46, 123, 255, 0.16);
```

```
0018:   --teal: #26c6da;
0019:   --green: #2ecc71;
0020:   --amber: #ffb020;
0021:   --red: #ef5350;
0022:   --shadow: 0 30px 80px rgba(0, 0, 0, 0.35);
0023:   --radius: 22px;
0024:   --radius-sm: 16px;
0025:   --font: "Manrope", ui-sans-serif, system-ui, sans-serif;
0026:   --font-display: "Rajdhani", "Manrope", ui-sans-serif, system-ui, sans-serif;
0027: }
0028:
0029: * {
0030:   box-sizing: border-box;
0031: }
0032:
0033: html,
0034: body {
0035:   min-height: 100%;
0036: }
0037:
0038: body {
0039:   margin: 0;
0040:   font-family: var(--font);
0041:   color: var(--text);
0042:   background:
0043:     linear-gradient(90deg, rgba(7, 17, 28, 0.88) 0 280px, rgba(6, 19, 32, 0.14) 280px 100%),
0044:     radial-gradient(circle at 15% 18%, rgba(44, 122, 255, 0.18), transparent 22%),
0045:     radial-gradient(circle at 85% 14%, rgba(38, 198, 218, 0.16), transparent 20%),
0046:     linear-gradient(180deg, rgba(8, 20, 32, 0.96), rgba(5, 14, 23, 0.98)),
0047:     var(--bg);
0048: }
0049:
0050: body::before {
0051:   content: "";
0052:   position: fixed;
0053:   inset: 0;
0054:   background:
0055:     linear-gradient(90deg, rgba(255, 255, 255, 0.035) 0, rgba(255, 255, 255, 0) 22%, rgba(255,
 255, 255, 0.03) 100%),
0056:     radial-gradient(circle at center, rgba(81, 153, 255, 0.08), transparent 60%);
0057:   pointer-events: none;
0058: }
0059:
0060: a {
0061:   color: #8fc2ff;
0062: }
0063:
0064: a:hover {
0065:   color: #c1deff;
0066: }
0067:
0068: .layout {
0069:   display: grid;
0070:   grid-template-columns: 290px 1fr;
```

```
0071:   min-height: 100vh;
0072:   position: relative;
0073:   z-index: 1;
0074: }
0075:
0076: .sidebar-shell {
0077:   display: flex;
0078:   flex-direction: column;
0079:   gap: 18px;
0080:   padding: 18px 18px 20px;
0081:   background:
0082:     linear-gradient(180deg, rgba(9, 19, 31, 0.96), rgba(7, 16, 27, 0.98)),
0083:     linear-gradient(180deg, rgba(62, 122, 255, 0.12), transparent 26%);
0084:   border-right: 1px solid rgba(135, 178, 226, 0.11);
0085:   box-shadow: inset -1px 0 0 rgba(255, 255, 255, 0.03);
0086: }
0087:
0088: .sidebar-brand {
0089:   display: flex;
0090:   align-items: center;
0091:   gap: 14px;
0092:   padding: 10px 10px 18px;
0093:   border-bottom: 1px solid rgba(125, 183, 255, 0.14);
0094: }
0095:
0096: .brand-mark,
0097: .login-card-mark {
0098:   width: 54px;
0099:   height: 54px;
0100:   border-radius: 18px;
0101:   display: inline-flex;
0102:   align-items: center;
0103:   justify-content: center;
0104:   font-family: var(--font-display);
0105:   font-size: 20px;
0106:   font-weight: 700;
0107:   color: white;
0108:   background:
0109:     linear-gradient(145deg, rgba(61, 132, 255, 0.98), rgba(23, 93, 214, 0.98)),
0110:     var(--primary);
0111:   box-shadow: 0 18px 32px rgba(28, 88, 198, 0.34);
0112: }
0113:
0114: .brand-title {
0115:   font-family: var(--font-display);
0116:   font-size: 30px;
0117:   letter-spacing: 0.04em;
0118:   text-transform: uppercase;
0119: }
0120:
0121: .brand-subtitle {
0122:   color: var(--text-dim);
0123:   font-size: 12px;
0124:   text-transform: uppercase;
```

```
0125:   letter-spacing: 0.18em;
0126: }
0127:
0128: .profile-card {
0129:   display: flex;
0130:   align-items: center;
0131:   gap: 12px;
0132:   padding: 14px;
0133:   border-radius: var(--radius-sm);
0134:   background: linear-gradient(180deg, rgba(17, 39, 58, 0.9), rgba(10, 23, 37, 0.92));
0135:   border: 1px solid rgba(126, 183, 255, 0.14);
0136: }
0137:
0138: .profile-avatar {
0139:   width: 56px;
0140:   height: 56px;
0141:   border-radius: 999px;
0142:   display: inline-flex;
0143:   align-items: center;
0144:   justify-content: center;
0145:   font-family: var(--font-display);
0146:   font-size: 18px;
0147:   font-weight: 700;
0148:   color: white;
0149:   background: linear-gradient(180deg, rgba(87, 170, 255, 0.98), rgba(28, 103, 227, 0.98));
0150:   box-shadow: 0 12px 24px rgba(30, 98, 220, 0.3);
0151: }
0152:
0153: .profile-name {
0154:   font-weight: 700;
0155: }
0156:
0157: .profile-role {
0158:   color: var(--text-dim);
0159:   font-size: 12px;
0160:   text-transform: uppercase;
0161:   letter-spacing: 0.12em;
0162: }
0163:
0164: .sidebar-scroll {
0165:   flex: 1;
0166:   overflow: auto;
0167:   padding-right: 4px;
0168: }
0169:
0170: .nav-section + .nav-section {
0171:   margin-top: 14px;
0172: }
0173:
0174: .nav-section-title {
0175:   margin: 0 8px 8px;
0176:   color: var(--text-dim);
0177:   font-size: 11px;
0178:   letter-spacing: 0.16em;
```

```
0179:   text-transform: uppercase;
0180: }
0181:
0182: .nav-list {
0183:   display: flex;
0184:   flex-direction: column;
0185:   gap: 6px;
0186: }
0187:
0188: .nav-item {
0189:   display: flex;
0190:   align-items: center;
0191:   gap: 12px;
0192:   padding: 12px 14px;
0193:   border-radius: 14px;
0194:   color: var(--text-soft);
0195:   text-decoration: none;
0196:   transition: transform 0.16s ease, background 0.16s ease, color 0.16s ease, border-color 0.16
s ease;
0197:   border: 1px solid transparent;
0198: }
0199:
0200: .nav-item:hover {
0201:   transform: translateX(3px);
0202:   background: rgba(21, 44, 68, 0.66);
0203:   color: var(--text);
0204: }
0205:
0206: .nav-item.active {
0207:   color: white;
0208:   background:
0209:     linear-gradient(90deg, rgba(29, 101, 240, 0.3), rgba(29, 101, 240, 0.05)),
0210:     rgba(10, 28, 45, 0.82);
0211:   border-color: rgba(74, 149, 255, 0.34);
0212:   box-shadow: inset 4px 0 0 rgba(77, 146, 255, 0.9);
0213: }
0214:
0215: .nav-code {
0216:   width: 38px;
0217:   height: 38px;
0218:   border-radius: 12px;
0219:   display: inline-flex;
0220:   align-items: center;
0221:   justify-content: center;
0222:   font-family: var(--font-display);
0223:   font-size: 14px;
0224:   font-weight: 700;
0225:   letter-spacing: 0.08em;
0226:   background: rgba(56, 107, 186, 0.2);
0227:   border: 1px solid rgba(117, 173, 248, 0.16);
0228:   color: #b9d9ff;
0229: }
0230:
0231: .nav-label {
```

```
0232:   font-size: 14px;
0233:   font-weight: 600;
0234: }
0235:
0236: .sidebar-footer {
0237:   padding: 14px;
0238:   border-radius: var(--radius-sm);
0239:   background: rgba(8, 18, 30, 0.74);
0240:   border: 1px solid rgba(120, 169, 226, 0.12);
0241: }
0242:
0243: .sidebar-footer-label {
0244:   color: var(--text-dim);
0245:   font-size: 11px;
0246:   text-transform: uppercase;
0247:   letter-spacing: 0.14em;
0248: }
0249:
0250: .sidebar-footer-value {
0251:   margin-top: 4px;
0252:   font-family: var(--font-display);
0253:   font-size: 18px;
0254:   color: #d7eaff;
0255:   letter-spacing: 0.06em;
0256: }
0257:
0258: .content {
0259:   min-width: 0;
0260:   padding: 18px 22px 24px;
0261: }
0262:
0263: .topbar-shell {
0264:   position: sticky;
0265:   top: 0;
0266:   z-index: 10;
0267:   display: grid;
0268:   grid-template-columns: minmax(220px, 300px) minmax(240px, 1fr) auto;
0269:   align-items: center;
0270:   gap: 16px;
0271:   padding: 14px 18px;
0272:   border-radius: 20px;
0273:   border: 1px solid rgba(125, 183, 255, 0.16);
0274:   background:
0275:     linear-gradient(180deg, rgba(22, 54, 80, 0.86), rgba(11, 27, 44, 0.94)),
0276:     rgba(11, 29, 46, 0.92);
0277:   box-shadow: var(--shadow);
0278:   backdrop-filter: blur(12px);
0279: }
0280:
0281: .topbar-eyebrow {
0282:   color: var(--text-dim);
0283:   font-size: 11px;
0284:   text-transform: uppercase;
0285:   letter-spacing: 0.18em;
```

```
0286: }
0287:
0288: .topbar-title {
0289:   margin-top: 3px;
0290:   font-family: var(--font-display);
0291:   font-size: 28px;
0292:   line-height: 1;
0293:   letter-spacing: 0.04em;
0294:   text-transform: uppercase;
0295: }
0296:
0297: .topbar-search {
0298:   min-width: 0;
0299: }
0300:
0301: .topbar-actions {
0302:   display: flex;
0303:   align-items: center;
0304:   justify-content: flex-end;
0305:   gap: 10px;
0306:   flex-wrap: wrap;
0307: }
0308:
0309: .topbar-pill {
0310:   display: inline-flex;
0311:   align-items: center;
0312:   justify-content: center;
0313:   min-height: 42px;
0314:   padding: 0 14px;
0315:   border-radius: 999px;
0316:   font-size: 12px;
0317:   font-weight: 700;
0318:   letter-spacing: 0.1em;
0319:   text-transform: uppercase;
0320:   color: #d3e7ff;
0321:   background: rgba(255, 255, 255, 0.06);
0322:   border: 1px solid rgba(146, 188, 236, 0.16);
0323: }
0324:
0325: .topbar-pill-button {
0326:   cursor: pointer;
0327: }
0328:
0329: .topbar-pill-button:hover {
0330:   background: rgba(255, 255, 255, 0.11);
0331: }
0332:
0333: .filial-picker {
0334:   display: inline-flex;
0335:   align-items: center;
0336:   gap: 8px;
0337:   padding: 0 6px 0 10px;
0338:   min-height: 40px;
0339:   border-radius: 999px;
```

```
0340:   border: 1px solid rgba(146, 188, 236, 0.16);
0341:   background: rgba(255, 255, 255, 0.06);
0342:   color: var(--text-soft);
0343:   font-size: 11px;
0344:   text-transform: uppercase;
0345:   letter-spacing: 0.12em;
0346: }
0347:
0348: .filial-picker span {
0349:   color: var(--text-dim);
0350: }
0351:
0352: .shell-input,
0353: .shell-select,
0354: .input,
0355: .select,
0356: .form-field input,
0357: .form-field select {
0358:   border: 1px solid rgba(139, 187, 241, 0.16);
0359:   background: rgba(238, 247, 255, 0.06);
0360:   color: var(--text);
0361:   border-radius: 14px;
0362:   outline: none;
0363:   transition: border-color 0.16s ease, box-shadow 0.16s ease, background 0.16s ease;
0364: }
0365:
0366: .shell-input,
0367: .input,
0368: .select,
0369: .shell-select {
0370:   padding: 10px 12px;
0371: }
0372:
0373: .select,
0374: .form-field select {
0375:   min-height: 42px;
0376:   padding-right: 38px;
0377:   appearance: none;
0378:   -webkit-appearance: none;
0379:   -moz-appearance: none;
0380:   background-image:
0381:     linear-gradient(45deg, transparent 50%, #cfe2f8 50%),
0382:     linear-gradient(135deg, #cfe2f8 50%, transparent 50%);
0383:   background-position:
0384:     calc(100% - 18px) calc(50% - 2px),
0385:     calc(100% - 13px) calc(50% - 2px);
0386:   background-size: 5px 5px, 5px 5px;
0387:   background-repeat: no-repeat;
0388:   cursor: pointer;
0389: }
0390:
0391: .select:hover,
0392: .form-field select:hover {
0393:   background-color: rgba(240, 248, 255, 0.09);
```

```
0394: }
0395:
0396: .select option,
0397: .form-field select option {
0398:   background: #183149;
0399:   color: #eef6ff;
0400: }
0401:
0402: .shell-input {
0403:   width: 100%;
0404:   min-height: 44px;
0405: }
0406:
0407: .shell-select {
0408:   min-height: 32px;
0409:   min-width: 58px;
0410:   padding: 4px 24px 4px 4px;
0411:   border: none;
0412:   background: transparent;
0413:   color: #eef6ff;
0414:   font-family: var(--font-display);
0415:   font-size: 15px;
0416:   font-weight: 600;
0417:   letter-spacing: 0.04em;
0418:   box-shadow: none;
0419:   appearance: none;
0420:   -webkit-appearance: none;
0421:   -moz-appearance: none;
0422:   background-image:
0423:     linear-gradient(45deg, transparent 50%, #cfe2f8 50%),
0424:     linear-gradient(135deg, #cfe2f8 50%, transparent 50%);
0425:   background-position:
0426:     calc(100% - 12px) calc(50% - 2px),
0427:     calc(100% - 7px) calc(50% - 2px);
0428:   background-size: 5px 5px, 5px 5px;
0429:   background-repeat: no-repeat;
0430: }
0431:
0432: .shell-select:hover {
0433:   background-color: transparent;
0434: }
0435:
0436: .shell-select:focus {
0437:   background-color: transparent;
0438:   box-shadow: none;
0439: }
0440:
0441: .filial-picker:focus-within {
0442:   border-color: rgba(112, 175, 255, 0.42);
0443:   box-shadow: 0 0 0 4px rgba(36, 101, 205, 0.14);
0444: }
0445:
0446: .shell-select option {
0447:   background: #183149;
```

```
0448:   color: #eef6ff;
0449: }
0450:
0451: .shell-input::placeholder,
0452: .input::placeholder {
0453:   color: #8ca6bf;
0454: }
0455:
0456: .shell-input:focus,
0457: .shell-select:focus,
0458: .input:focus,
0459: .select:focus,
0460: .form-field input:focus,
0461: .form-field select:focus {
0462:   border-color: rgba(112, 175, 255, 0.5);
0463:   box-shadow: 0 0 0 4px rgba(36, 101, 205, 0.18);
0464:   background: rgba(240, 248, 255, 0.1);
0465: }
0466:
0467: .icon-action {
0468:   min-height: 42px;
0469:   padding: 0 14px;
0470:   border-radius: 999px;
0471:   border: 1px solid rgba(136, 186, 242, 0.14);
0472:   background: rgba(255, 255, 255, 0.06);
0473:   color: #d5e8ff;
0474:   font-family: var(--font);
0475:   font-size: 12px;
0476:   font-weight: 700;
0477:   letter-spacing: 0.08em;
0478:   text-transform: uppercase;
0479:   cursor: pointer;
0480: }
0481:
0482: .icon-action:hover {
0483:   background: rgba(255, 255, 255, 0.12);
0484: }
0485:
0486: .page {
0487:   padding-top: 18px;
0488: }
0489:
0490: .page > * + * {
0491:   margin-top: 18px;
0492: }
0493:
0494: .card,
0495: .modal,
0496: .table-scroller {
0497:   background:
0498:     linear-gradient(180deg, rgba(16, 34, 54, 0.92), rgba(9, 22, 35, 0.94)),
0499:     var(--panel);
0500:   border: 1px solid rgba(132, 181, 237, 0.14);
0501:   border-radius: var(--radius);
```

```
0502:   box-shadow: var(--shadow);
0503: }
0504:
0505: .card {
0506:   padding: 16px;
0507:   animation: shell-in 0.26s ease-out;
0508: }
0509:
0510: .card.soft {
0511:   background:
0512:     linear-gradient(180deg, rgba(23, 45, 69, 0.96), rgba(10, 24, 39, 0.96)),
0513:     var(--panel);
0514: }
0515:
0516: .section-title {
0517:   margin: 18px 2px 10px;
0518:   color: #9ab0c4;
0519:   font-family: var(--font-display);
0520:   font-size: 15px;
0521:   letter-spacing: 0.14em;
0522:   text-transform: uppercase;
0523: }
0524:
0525: .stats-grid {
0526:   display: grid;
0527:   grid-template-columns: repeat(3, minmax(0, 1fr));
0528:   gap: 14px;
0529: }
0530:
0531: .stat-card {
0532:   display: flex;
0533:   align-items: center;
0534:   justify-content: space-between;
0535:   gap: 12px;
0536:   padding: 16px;
0537:   border-radius: 18px;
0538:   border: 1px solid rgba(131, 181, 241, 0.12);
0539:   background:
0540:     linear-gradient(180deg, rgba(17, 39, 58, 0.92), rgba(12, 27, 42, 0.94)),
0541:     var(--panel-soft);
0542:   box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.03);
0543: }
0544:
0545: .stat-icon {
0546:   width: 46px;
0547:   height: 46px;
0548:   border-radius: 14px;
0549:   display: inline-flex;
0550:   align-items: center;
0551:   justify-content: center;
0552:   font-family: var(--font-display);
0553:   font-size: 15px;
0554:   font-weight: 700;
0555:   color: #dcecff;
```

```
0556:   background: rgba(43, 110, 222, 0.22);
0557:   border: 1px solid rgba(118, 172, 238, 0.18);
0558: }
0559:
0560: .stat-icon.solid {
0561:   background: rgba(127, 154, 185, 0.16);
0562: }
0563:
0564: .stat-icon.success {
0565:   color: #d7ffe7;
0566:   background: rgba(46, 204, 113, 0.16);
0567:   border-color: rgba(111, 231, 165, 0.24);
0568: }
0569:
0570: .kpi {
0571:   font-family: var(--font-display);
0572:   font-size: 32px;
0573:   line-height: 1;
0574:   letter-spacing: 0.02em;
0575: }
0576:
0577: .kpi-label,
0578: .hint,
0579: .subtle {
0580:   color: var(--text-soft);
0581:   font-size: 13px;
0582: }
0583:
0584: .alert {
0585:   border: 1px solid rgba(239, 83, 80, 0.26);
0586:   background: rgba(125, 27, 26, 0.28);
0587:   color: #ffd6d6;
0588:   padding: 11px 12px;
0589:   border-radius: 14px;
0590: }
0591:
0592: .alert.alert-success {
0593:   border-color: rgba(46, 204, 113, 0.26);
0594:   background: rgba(18, 88, 48, 0.32);
0595:   color: #d8ffe8;
0596: }
0597:
0598: .badge {
0599:   display: inline-flex;
0600:   align-items: center;
0601:   gap: 6px;
0602:   padding: 5px 10px;
0603:   border-radius: 999px;
0604:   font-size: 11px;
0605:   border: 1px solid rgba(132, 181, 237, 0.12);
0606:   background: rgba(255, 255, 255, 0.04);
0607:   color: #d6e8fb;
0608: }
0609:
```

```
0610: .badge-dot {
0611:   width: 7px;
0612:   height: 7px;
0613:   border-radius: 999px;
0614:   background: #7c9abd;
0615:   display: inline-block;
0616: }
0617:
0618: .badge.success {
0619:   border-color: rgba(92, 217, 142, 0.24);
0620:   background: rgba(46, 204, 113, 0.14);
0621:   color: #d7ffe8;
0622: }
0623:
0624: .badge.success .badge-dot {
0625:   background: #48dc86;
0626: }
0627:
0628: .badge.muted {
0629:   border-color: rgba(167, 184, 204, 0.16);
0630:   background: rgba(126, 145, 168, 0.12);
0631:   color: #c2cfdd;
0632: }
0633:
0634: .badge.muted .badge-dot {
0635:   background: #a8bacf;
0636: }
0637:
0638: .row {
0639:   display: flex;
0640:   align-items: center;
0641: }
0642:
0643: .row-between {
0644:   display: flex;
0645:   justify-content: space-between;
0646:   align-items: center;
0647: }
0648:
0649: .gap-10 {
0650:   gap: 10px;
0651: }
0652:
0653: .gap-12 {
0654:   gap: 12px;
0655: }
0656:
0657: .mt-12 {
0658:   margin-top: 12px;
0659: }
0660:
0661: .btn {
0662:   display: inline-flex;
0663:   justify-content: center;
```

```
0664:   align-items: center;
0665:   gap: 8px;
0666:   padding: 10px 14px;
0667:   border-radius: 14px;
0668:   border: 1px solid rgba(133, 183, 240, 0.14);
0669:   background: rgba(255, 255, 255, 0.06);
0670:   color: #f0f6ff;
0671:   text-decoration: none;
0672:   cursor: pointer;
0673:   transition: transform 0.16s ease, background 0.16s ease, border-color 0.16s ease;
0674: }
0675:
0676: .btn:hover {
0677:   transform: translateY(-1px);
0678:   background: rgba(255, 255, 255, 0.12);
0679: }
0680:
0681: .btn.primary {
0682:   background: linear-gradient(180deg, var(--primary-strong), var(--primary));
0683:   border-color: rgba(93, 154, 255, 0.4);
0684:   color: white;
0685: }
0686:
0687: .btn.primary:hover {
0688:   background: linear-gradient(180deg, #458bff, #1f69f3);
0689: }
0690:
0691: .btn.ghost {
0692:   background: transparent;
0693:   color: var(--text-soft);
0694: }
0695:
0696: .btn.ghost:hover {
0697:   background: rgba(255, 255, 255, 0.07);
0698: }
0699:
0700: .btn.danger {
0701:   border-color: rgba(239, 83, 80, 0.22);
0702:   color: #ffd4d2;
0703: }
0704:
0705: .btn.danger:hover {
0706:   background: rgba(239, 83, 80, 0.14);
0707: }
0708:
0709: .shell-logout {
0710:   min-width: 84px;
0711: }
0712:
0713: .toolbar {
0714:   display: flex;
0715:   align-items: center;
0716:   justify-content: space-between;
0717:   gap: 12px;
```

```
0718: }
0719:
0720: .toolbar-left,
0721: .toolbar-right {
0722:   display: flex;
0723:   align-items: center;
0724:   gap: 10px;
0725:   flex-wrap: wrap;
0726: }
0727:
0728: .toolbar-right {
0729:   justify-content: flex-end;
0730: }
0731:
0732: .minw-240 {
0733:   min-width: 240px;
0734: }
0735:
0736: .table-scroller {
0737:   width: 100%;
0738:   overflow: auto;
0739: }
0740:
0741: .table {
0742:   width: 100%;
0743:   border-collapse: collapse;
0744:   margin-top: 12px;
0745: }
0746:
0747: .table thead {
0748:   display: table-header-group !important;
0749: }
0750:
0751: .table thead th {
0752:   background: rgba(255, 255, 255, 0.04);
0753:   color: #b9cee3;
0754:   font-size: 11px;
0755:   text-transform: uppercase;
0756:   letter-spacing: 0.14em;
0757:   text-align: left;
0758: }
0759:
0760: .table th,
0761: .table td {
0762:   padding: 11px 10px;
0763:   border-bottom: 1px solid rgba(128, 173, 225, 0.1);
0764: }
0765:
0766: .table tbody tr:hover {
0767:   background: rgba(48, 102, 176, 0.12);
0768: }
0769:
0770: .table tbody tr:last-child td {
0771:   border-bottom: none;
```

```
0772: }
0773:
0774: .table.compact th,
0775: .table.compact td {
0776:   padding: 9px 10px;
0777: }
0778:
0779: .table th {
0780:   color: inherit !important;
0781:   opacity: 1 !important;
0782:   visibility: visible !important;
0783:   font-size: 12px !important;
0784:   line-height: 1.2 !important;
0785: }
0786:
0787: .modal-backdrop {
0788:   position: fixed;
0789:   inset: 0;
0790:   background: rgba(4, 10, 18, 0.62);
0791:   display: flex;
0792:   align-items: center;
0793:   justify-content: center;
0794:   z-index: 1000;
0795:   backdrop-filter: blur(8px);
0796: }
0797:
0798: .modal {
0799:   width: 560px;
0800:   max-width: calc(100vw - 32px);
0801:   overflow: hidden;
0802: }
0803:
0804: .modal-header {
0805:   display: flex;
0806:   justify-content: space-between;
0807:   align-items: center;
0808:   padding: 16px 18px;
0809:   border-bottom: 1px solid rgba(132, 181, 237, 0.12);
0810:   background: rgba(255, 255, 255, 0.04);
0811: }
0812:
0813: .modal-body {
0814:   padding: 18px;
0815: }
0816:
0817: .panel-stack {
0818:   display: flex;
0819:   flex-direction: column;
0820:   gap: 14px;
0821: }
0822:
0823: .topbar-panel-box {
0824:   padding: 14px;
0825:   border-radius: 16px;
```

```
0826:   border: 1px solid rgba(132, 181, 237, 0.1);
0827:   background: rgba(255, 255, 255, 0.03);
0828: }
0829:
0830: .topbar-panel-title {
0831:   font-family: var(--font-display);
0832:   font-size: 18px;
0833:   letter-spacing: 0.04em;
0834:   text-transform: uppercase;
0835: }
0836:
0837: .topbar-panel-text {
0838:   margin-top: 8px;
0839:   color: var(--text-soft);
0840:   line-height: 1.6;
0841: }
0842:
0843: .topbar-panel-empty {
0844:   margin-top: 8px;
0845:   color: var(--text-dim);
0846: }
0847:
0848: .panel-actions {
0849:   margin-top: 12px;
0850: }
0851:
0852: .panel-route-list {
0853:   display: grid;
0854:   grid-template-columns: repeat(2, minmax(0, 1fr));
0855:   gap: 10px;
0856: }
0857:
0858: .route-chip {
0859:   display: flex;
0860:   flex-direction: column;
0861:   align-items: flex-start;
0862:   gap: 4px;
0863:   padding: 12px 14px;
0864:   border-radius: 16px;
0865:   border: 1px solid rgba(132, 181, 237, 0.12);
0866:   background: rgba(255, 255, 255, 0.05);
0867:   color: var(--text);
0868:   cursor: pointer;
0869: }
0870:
0871: .route-chip:hover {
0872:   background: rgba(255, 255, 255, 0.09);
0873: }
0874:
0875: .route-chip.active {
0876:   border-color: rgba(93, 154, 255, 0.4);
0877:   background: rgba(29, 101, 240, 0.18);
0878: }
0879:
```

```
0880: .route-chip span {
0881:   font-weight: 700;
0882: }
0883:
0884: .route-chip small {
0885:   color: var(--text-soft);
0886: }
0887:
0888: .topbar-alert-card {
0889:   padding: 14px;
0890:   border-radius: 16px;
0891:   border: 1px solid rgba(132, 181, 237, 0.1);
0892: }
0893:
0894: .topbar-alert-card.info {
0895:   background: rgba(35, 114, 232, 0.14);
0896: }
0897:
0898: .topbar-alert-card.critical {
0899:   background: rgba(224, 73, 60, 0.16);
0900:   border-color: rgba(224, 73, 60, 0.28);
0901: }
0902:
0903: .topbar-alert-card.warning {
0904:   background: rgba(255, 176, 32, 0.12);
0905: }
0906:
0907: .context-banner {
0908:   display: flex;
0909:   align-items: center;
0910:   justify-content: space-between;
0911:   gap: 16px;
0912:   padding: 14px 16px;
0913:   margin-bottom: 14px;
0914:   border-radius: 16px;
0915:   border: 1px solid rgba(93, 154, 255, 0.18);
0916:   background: rgba(35, 114, 232, 0.1);
0917: }
0918:
0919: .context-banner-title {
0920:   font-size: 12px;
0921:   letter-spacing: 0.12em;
0922:   text-transform: uppercase;
0923:   color: var(--text-soft);
0924: }
0925:
0926: .context-banner-text {
0927:   color: var(--text);
0928:   margin-top: 4px;
0929: }
0930:
0931: .detail-grid {
0932:   display: grid;
0933:   grid-template-columns: repeat(4, minmax(0, 1fr));
```

```
0934:   gap: 14px 18px;
0935: }
0936:
0937: .help-list {
0938:   margin: 10px 0 0;
0939:   padding-left: 18px;
0940:   color: var(--text-soft);
0941:   line-height: 1.7;
0942: }
0943:
0944: .modal-close {
0945:   background: transparent;
0946:   border: none;
0947:   font-size: 18px;
0948:   cursor: pointer;
0949:   color: var(--text-soft);
0950: }
0951:
0952: .modal-close:hover {
0953:   color: white;
0954: }
0955:
0956: .form-grid {
0957:   display: grid;
0958:   grid-template-columns: 1fr 1fr;
0959:   gap: 12px;
0960: }
0961:
0962: .form-field {
0963:   display: flex;
0964:   flex-direction: column;
0965:   gap: 6px;
0966: }
0967:
0968: .form-field label {
0969:   color: #9fb7cb;
0970:   font-size: 12px;
0971:   text-transform: uppercase;
0972:   letter-spacing: 0.12em;
0973: }
0974:
0975: .form-field input,
0976: .form-field select {
0977:   width: 100%;
0978:   padding: 11px 12px;
0979: }
0980:
0981: .form-field.col-span-all {
0982:   grid-column: 1 / -1;
0983: }
0984:
0985: .form-actions {
0986:   display: flex;
0987:   gap: 10px;
```

```
0988:   margin-top: 16px;
0989:   justify-content: flex-end;
0990: }
0991:
0992: .search-results-panel {
0993:   border: 1px solid rgba(118, 164, 223, 0.18);
0994:   border-radius: 14px;
0995:   background: rgba(8, 20, 35, 0.88);
0996:   overflow: hidden;
0997: }
0998:
0999: .inline-search-table {
1000:   width: 100%;
1001:   border-collapse: collapse;
1002:   font-size: 13px;
1003: }
1004:
1005: .inline-search-table th,
1006: .inline-search-table td {
1007:   padding: 10px 12px;
1008:   text-align: left;
1009:   border-bottom: 1px solid rgba(118, 164, 223, 0.12);
1010: }
1011:
1012: .inline-search-table th {
1013:   color: #9fb7cb;
1014:   text-transform: uppercase;
1015:   letter-spacing: 0.1em;
1016:   font-size: 11px;
1017:   background: rgba(255, 255, 255, 0.03);
1018: }
1019:
1020: .inline-search-table tbody tr:hover {
1021:   background: rgba(72, 130, 255, 0.08);
1022: }
1023:
1024: .empty-state {
1025:   border: 1px dashed rgba(126, 180, 237, 0.16);
1026:   background: rgba(255, 255, 255, 0.03);
1027:   border-radius: 16px;
1028:   padding: 24px;
1029:   text-align: center;
1030:   color: var(--text-soft);
1031: }
1032:
1033: .empty-title {
1034:   font-family: var(--font-display);
1035:   color: white;
1036:   font-size: 22px;
1037:   margin-bottom: 6px;
1038: }
1039:
1040: .empty-sub {
1041:   font-size: 13px;
```

```
1042: }
1043:
1044: .p-14 {
1045:   padding: 14px;
1046: }
1047:
1048: .hero-panel {
1049:   display: grid;
1050:   grid-template-columns: minmax(0, 1fr) auto;
1051:   gap: 18px;
1052:   padding: 20px 22px;
1053:   border-radius: 24px;
1054:   border: 1px solid rgba(133, 183, 240, 0.16);
1055:   background:
1056:     radial-gradient(circle at 14% 24%, rgba(72, 146, 255, 0.14), transparent 20%),
1057:     linear-gradient(90deg, rgba(255, 255, 255, 0.025) 0 1px, transparent 1px),
1058:     linear-gradient(rgba(255, 255, 255, 0.02) 0 1px, transparent 1px),
1059:     linear-gradient(135deg, rgba(18, 45, 70, 0.96), rgba(10, 22, 36, 0.96)),
1060:     var(--panel);
1061:   background-size: auto, 64px 64px, 64px 64px, auto, auto;
1062:   box-shadow: var(--shadow);
1063:   position: relative;
1064:   overflow: hidden;
1065: }
1066:
1067: .hero-panel-bg {
1068:   background:
1069:     linear-gradient(90deg, rgba(3, 13, 24, 0.8), rgba(3, 13, 24, 0.56)),
1070:     linear-gradient(rgba(5, 17, 30, 0.62), rgba(5, 17, 30, 0.82)),
1071:     var(--hero-bg) var(--hero-bg-position, center) / var(--hero-bg-size, cover) no-repeat,
1072:     radial-gradient(circle at 14% 24%, rgba(72, 146, 255, 0.14), transparent 20%),
1073:     linear-gradient(90deg, rgba(255, 255, 255, 0.025) 0 1px, transparent 1px),
1074:     linear-gradient(rgba(255, 255, 255, 0.02) 0 1px, transparent 1px),
1075:     linear-gradient(135deg, rgba(18, 45, 70, 0.96), rgba(10, 22, 36, 0.96)),
1076:     var(--panel);
1077:   background-size: auto, auto, cover, auto, 64px 64px, 64px 64px, auto, auto;
1078:   background-position: center, center, center, center, center, center, center, center;
1079: }
1080:
1081: .hero-main {
1082:   display: flex;
1083:   align-items: center;
1084:   gap: 18px;
1085:   min-width: 0;
1086: }
1087:
1088: .hero-icon-wrap {
1089:   width: 128px;
1090:   height: 128px;
1091:   padding: 14px;
1092:   border-radius: 34px;
1093:   display: flex;
1094:   align-items: center;
1095:   justify-content: center;
```

```
1096:   background:
1097:     linear-gradient(180deg, rgba(255, 255, 255, 0.08), rgba(255, 255, 255, 0.02)),
1098:     rgba(8, 24, 39, 0.68);
1099:   border: 1px solid rgba(141, 187, 240, 0.18);
1100:   box-shadow:
1101:     inset 0 1px 0 rgba(255, 255, 255, 0.05),
1102:     0 22px 40px rgba(0, 0, 0, 0.22);
1103: }
1104:
1105: .hero-icon {
1106:   width: 100%;
1107:   height: 100%;
1108:   object-fit: contain;
1109:   filter: drop-shadow(0 12px 20px rgba(0, 0, 0, 0.22));
1110: }
1111:
1112: .hero-copy {
1113:   min-width: 0;
1114: }
1115:
1116: .hero-eyebrow {
1117:   color: #8fb4d8;
1118:   font-size: 11px;
1119:   text-transform: uppercase;
1120:   letter-spacing: 0.18em;
1121: }
1122:
1123: .hero-title {
1124:   margin: 6px 0 8px;
1125:   font-family: var(--font-display);
1126:   font-size: 40px;
1127:   line-height: 0.95;
1128:   text-transform: uppercase;
1129:   letter-spacing: 0.05em;
1130: }
1131:
1132: .hero-description {
1133:   margin: 0;
1134:   max-width: 780px;
1135:   color: #b3c6d8;
1136:   line-height: 1.7;
1137: }
1138:
1139: .hero-metrics {
1140:   display: flex;
1141:   flex-wrap: wrap;
1142:   gap: 12px;
1143:   margin-top: 16px;
1144: }
1145:
1146: .hero-metric {
1147:   min-width: 140px;
1148:   padding: 12px 14px;
1149:   border-radius: 16px;
```

```
1150:   background: rgba(255, 255, 255, 0.04);
1151:   border: 1px solid rgba(132, 181, 237, 0.1);
1152: }
1153:
1154: .hero-metric-label {
1155:   color: var(--text-dim);
1156:   font-size: 10px;
1157:   text-transform: uppercase;
1158:   letter-spacing: 0.18em;
1159: }
1160:
1161: .hero-metric-value {
1162:   margin-top: 6px;
1163:   font-family: var(--font-display);
1164:   font-size: 24px;
1165:   letter-spacing: 0.04em;
1166: }
1167:
1168: .hero-actions {
1169:   display: flex;
1170:   align-items: flex-start;
1171:   gap: 10px;
1172:   flex-wrap: wrap;
1173: }
1174:
1175: .dashboard-stack {
1176:   display: flex;
1177:   flex-direction: column;
1178:   gap: 18px;
1179: }
1180:
1181: .dashboard-grid {
1182:   display: grid;
1183:   grid-template-columns: 1.3fr 1fr;
1184:   gap: 18px;
1185: }
1186:
1187: .dashboard-block {
1188:   min-height: 100%;
1189:   position: relative;
1190:   overflow: hidden;
1191: }
1192:
1193: .dashboard-block-glow {
1194:   position: absolute;
1195:   inset: -20% auto auto -12%;
1196:   width: 220px;
1197:   height: 220px;
1198:   border-radius: 999px;
1199:   background: radial-gradient(circle, rgba(58, 140, 255, 0.18), transparent 66%);
1200:   pointer-events: none;
1201: }
1202:
1203: .dashboard-block-glow-secondary {
```

```
1204:   inset: auto -10% -18% auto;
1205:   background: radial-gradient(circle, rgba(42, 198, 218, 0.14), transparent 66%);
1206: }
1207:
1208: .dashboard-block-wide {
1209:   min-width: 0;
1210: }
1211:
1212: .dashboard-block-header {
1213:   display: flex;
1214:   align-items: center;
1215:   justify-content: space-between;
1216:   gap: 12px;
1217:   margin-bottom: 18px;
1218: }
1219:
1220: .dashboard-block-eyebrow {
1221:   color: var(--text-dim);
1222:   font-size: 11px;
1223:   text-transform: uppercase;
1224:   letter-spacing: 0.16em;
1225: }
1226:
1227: .dashboard-block-title {
1228:   margin-top: 4px;
1229:   font-family: var(--font-display);
1230:   font-size: 26px;
1231:   text-transform: uppercase;
1232:   letter-spacing: 0.05em;
1233: }
1234:
1235: .quick-grid {
1236:   display: grid;
1237:   grid-template-columns: repeat(4, minmax(0, 1fr));
1238:   gap: 14px;
1239: }
1240:
1241: .quick-card {
1242:   display: flex;
1243:   flex-direction: column;
1244:   align-items: center;
1245:   justify-content: center;
1246:   gap: 16px;
1247:   min-height: 182px;
1248:   padding: 20px 16px;
1249:   border-radius: 24px;
1250:   text-decoration: none;
1251:   color: var(--text);
1252:   background:
1253:     radial-gradient(circle at 50% 18%, rgba(79, 151, 255, 0.08), transparent 30%),
1254:     linear-gradient(180deg, rgba(18, 41, 61, 0.92), rgba(11, 26, 40, 0.96)),
1255:     var(--panel);
1256:   border: 1px solid rgba(133, 183, 240, 0.12);
```

```
1257:   transition: transform 0.16s ease, border-color 0.16s ease, background 0.16s ease, box-shadow
 0.16s ease;
1258: }
1259:
1260: .quick-card:hover {
1261:   transform: translateY(-4px);
1262:   border-color: rgba(113, 175, 255, 0.4);
1263:   background:
1264:     radial-gradient(circle at 50% 18%, rgba(79, 151, 255, 0.12), transparent 30%),
1265:     linear-gradient(180deg, rgba(20, 48, 73, 0.98), rgba(12, 29, 44, 0.98)),
1266:     var(--panel);
1267:   box-shadow: 0 22px 40px rgba(0, 0, 0, 0.22);
1268: }
1269:
1270: .quick-card-media {
1271:   width: 108px;
1272:   height: 108px;
1273:   padding: 12px;
1274:   border-radius: 28px;
1275:   display: flex;
1276:   align-items: center;
1277:   justify-content: center;
1278:   background:
1279:     linear-gradient(180deg, rgba(255, 255, 255, 0.08), rgba(255, 255, 255, 0.02)),
1280:     rgba(7, 20, 33, 0.72);
1281:   border: 1px solid rgba(133, 183, 240, 0.14);
1282:   box-shadow:
1283:     inset 0 1px 0 rgba(255, 255, 255, 0.05),
1284:     0 16px 28px rgba(0, 0, 0, 0.18);
1285: }
1286:
1287: .quick-card-icon {
1288:   width: 100%;
1289:   height: 100%;
1290:   object-fit: contain;
1291:   filter: drop-shadow(0 10px 18px rgba(0, 0, 0, 0.2));
1292: }
1293:
1294: .quick-card-label {
1295:   text-align: center;
1296:   font-weight: 700;
1297:   line-height: 1.35;
1298:   font-size: 18px;
1299:   letter-spacing: 0.01em;
1300: }
1301:
1302: .quick-card-subtitle {
1303:   text-align: center;
1304:   color: var(--text-soft);
1305:   font-size: 12px;
1306:   line-height: 1.45;
1307:   max-width: 180px;
1308: }
1309:
```

```
1310: .dashboard-summary-grid {
1311:   display: grid;
1312:   grid-template-columns: 1fr;
1313:   gap: 14px;
1314: }
1315:
1316: .summary-tile {
1317:   padding: 18px;
1318:   border-radius: 20px;
1319:   color: white;
1320:   box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.06);
1321: }
1322:
1323: .summary-tile-green {
1324:   background: linear-gradient(135deg, #26a85e, #1c6f42);
1325: }
1326:
1327: .summary-tile-red {
1328:   background: linear-gradient(135deg, #d84d42, #8d261f);
1329: }
1330:
1331: .summary-tile-cyan {
1332:   background: linear-gradient(135deg, #35b8ef, #1d6cb7);
1333: }
1334:
1335: .summary-tile-title {
1336:   font-size: 12px;
1337:   text-transform: uppercase;
1338:   letter-spacing: 0.14em;
1339:   opacity: 0.86;
1340: }
1341:
1342: .summary-tile-value {
1343:   margin-top: 10px;
1344:   font-family: var(--font-display);
1345:   font-size: 30px;
1346:   line-height: 1;
1347:   text-transform: uppercase;
1348: }
1349:
1350: .summary-tile-sub {
1351:   margin-top: 10px;
1352:   font-size: 13px;
1353:   opacity: 0.92;
1354: }
1355:
1356: .login-shell {
1357:   min-height: 100vh;
1358:   display: grid;
1359:   grid-template-columns: minmax(360px, 1.15fr) minmax(380px, 0.85fr);
1360:   position: relative;
1361:   overflow: hidden;
1362:   background:
1363:     linear-gradient(120deg, rgba(4, 12, 21, 0.96), rgba(7, 16, 27, 0.78)),
```

```
1364:     linear-gradient(180deg, #091623, #0b1c2d);
1365: }
1366:
1367: .login-backdrop {
1368:   position: absolute;
1369:   inset: 0;
1370:   background:
1371:     radial-gradient(circle at 18% 18%, rgba(38, 198, 218, 0.16), transparent 20%),
1372:     radial-gradient(circle at 82% 22%, rgba(29, 101, 240, 0.2), transparent 22%),
1373:     linear-gradient(90deg, rgba(255, 255, 255, 0.04) 0 1px, transparent 1px),
1374:     linear-gradient(rgba(255, 255, 255, 0.025) 0 1px, transparent 1px);
1375:   background-size: auto, auto, 72px 72px, 72px 72px;
1376:   opacity: 0.86;
1377: }
1378:
1379: .login-showcase,
1380: .login-panel {
1381:   position: relative;
1382:   z-index: 1;
1383:   padding: 48px;
1384: }
1385:
1386: .login-showcase {
1387:   display: flex;
1388:   align-items: center;
1389: }
1390:
1391: .login-showcase-inner {
1392:   max-width: 640px;
1393: }
1394:
1395: .login-badge {
1396:   display: inline-flex;
1397:   align-items: center;
1398:   min-height: 36px;
1399:   padding: 0 14px;
1400:   border-radius: 999px;
1401:   border: 1px solid rgba(140, 188, 242, 0.16);
1402:   background: rgba(255, 255, 255, 0.06);
1403:   color: #cbe2ff;
1404:   font-size: 12px;
1405:   letter-spacing: 0.12em;
1406:   text-transform: uppercase;
1407: }
1408:
1409: .login-title {
1410:   margin: 18px 0 10px;
1411:   font-family: var(--font-display);
1412:   font-size: clamp(56px, 7vw, 88px);
1413:   line-height: 0.92;
1414:   letter-spacing: 0.04em;
1415:   text-transform: uppercase;
1416: }
1417:
```

```
1418: .login-copy {
1419:   max-width: 540px;
1420:   color: #b2c5d8;
1421:   font-size: 17px;
1422:   line-height: 1.7;
1423: }
1424:
1425: .showcase-grid {
1426:   display: grid;
1427:   grid-template-columns: repeat(3, minmax(0, 1fr));
1428:   gap: 14px;
1429:   margin-top: 30px;
1430: }
1431:
1432: .showcase-card {
1433:   padding: 18px;
1434:   border-radius: 20px;
1435:   border: 1px solid rgba(133, 183, 240, 0.14);
1436:   background:
1437:     linear-gradient(180deg, rgba(18, 39, 58, 0.92), rgba(10, 23, 37, 0.94)),
1438:     var(--panel);
1439:   box-shadow: var(--shadow);
1440: }
1441:
1442: .showcase-card-label {
1443:   color: var(--text-dim);
1444:   font-size: 11px;
1445:   letter-spacing: 0.14em;
1446:   text-transform: uppercase;
1447: }
1448:
1449: .showcase-card-value {
1450:   margin-top: 10px;
1451:   font-family: var(--font-display);
1452:   font-size: 24px;
1453:   line-height: 1.1;
1454: }
1455:
1456: .login-panel {
1457:   display: flex;
1458:   align-items: center;
1459:   justify-content: center;
1460: }
1461:
1462: .login-card {
1463:   width: min(520px, calc(100vw - 32px));
1464:   padding: 24px;
1465:   border-radius: 28px;
1466:   border: 1px solid rgba(133, 183, 240, 0.16);
1467:   background:
1468:     linear-gradient(180deg, rgba(16, 34, 53, 0.95), rgba(9, 22, 35, 0.98)),
1469:     var(--panel-strong);
1470:   box-shadow: var(--shadow);
1471: }
```

```
1472:
1473: .login-card-top {
1474:   display: flex;
1475:   align-items: center;
1476:   gap: 16px;
1477:   margin-bottom: 22px;
1478: }
1479:
1480: .login-card-title {
1481:   font-family: var(--font-display);
1482:   font-size: 30px;
1483:   line-height: 1;
1484:   text-transform: uppercase;
1485:   letter-spacing: 0.05em;
1486: }
1487:
1488: .login-card-subtitle {
1489:   margin-top: 6px;
1490:   color: var(--text-soft);
1491:   font-size: 13px;
1492: }
1493:
1494: .login-alert {
1495:   margin-bottom: 14px;
1496: }
1497:
1498: .login-helper-row {
1499:   display: flex;
1500:   justify-content: space-between;
1501:   gap: 12px;
1502:   padding: 12px 14px;
1503:   margin-top: 12px;
1504:   border-radius: 16px;
1505:   background: rgba(255, 255, 255, 0.04);
1506:   border: 1px solid rgba(133, 183, 240, 0.1);
1507: }
1508:
1509: .login-helper-label {
1510:   color: var(--text-dim);
1511:   font-size: 11px;
1512:   text-transform: uppercase;
1513:   letter-spacing: 0.14em;
1514: }
1515:
1516: .login-helper-value {
1517:   font-weight: 700;
1518:   color: #d8e8ff;
1519: }
1520:
1521: .login-submit {
1522:   width: 100%;
1523:   margin-top: 18px;
1524:   min-height: 52px;
1525:   font-family: var(--font-display);
```

```
1526:   font-size: 20px;
1527:   letter-spacing: 0.06em;
1528:   text-transform: uppercase;
1529: }
1530:
1531: @keyframes shell-in {
1532:   from {
1533:     transform: translateY(6px);
1534:     opacity: 0.7;
1535:   }
1536:
1537:   to {
1538:     transform: translateY(0);
1539:     opacity: 1;
1540:   }
1541: }
1542:
1543: @media (max-width: 1240px) {
1544:   .topbar-shell {
1545:     grid-template-columns: 1fr;
1546:     align-items: stretch;
1547:   }
1548:
1549:   .topbar-actions {
1550:     justify-content: flex-start;
1551:   }
1552:
1553:   .showcase-grid {
1554:     grid-template-columns: 1fr;
1555:   }
1556: }
1557:
1558: @media (max-width: 1080px) {
1559:   .layout {
1560:     grid-template-columns: 1fr;
1561:   }
1562:
1563:   .sidebar-shell {
1564:     position: static;
1565:     border-right: none;
1566:     border-bottom: 1px solid rgba(135, 178, 226, 0.11);
1567:   }
1568:
1569:   .content {
1570:     padding: 16px;
1571:   }
1572:
1573:   .stats-grid {
1574:     grid-template-columns: 1fr;
1575:   }
1576:
1577:   .toolbar,
1578:   .row-between {
1579:     flex-direction: column;
```

```
1580:     align-items: stretch;
1581:   }
1582:
1583:   .toolbar-right {
1584:     justify-content: flex-start;
1585:   }
1586:
1587:   .minw-240 {
1588:     min-width: 0;
1589:     width: 100%;
1590:   }
1591:
1592:   .form-grid {
1593:     grid-template-columns: 1fr;
1594:   }
1595:
1596:   .panel-route-list {
1597:     grid-template-columns: 1fr;
1598:   }
1599:
1600:   .hero-panel,
1601:   .dashboard-grid {
1602:     grid-template-columns: 1fr;
1603:   }
1604:
1605:   .hero-main {
1606:     align-items: flex-start;
1607:   }
1608:
1609:   .hero-icon-wrap {
1610:     width: 108px;
1611:     height: 108px;
1612:   }
1613:
1614:   .quick-grid {
1615:     grid-template-columns: repeat(2, minmax(0, 1fr));
1616:   }
1617:
1618:   .hide-sm {
1619:     display: none !important;
1620:   }
1621:
1622:   .login-shell {
1623:     grid-template-columns: 1fr;
1624:   }
1625:
1626:   .login-showcase {
1627:     padding-bottom: 0;
1628:   }
1629: }
1630:
1631: @media (max-width: 720px) {
1632:   .brand-title {
1633:     font-size: 24px;
```

```
1634:   }
1635:
1636:   .topbar-title {
1637:     font-size: 22px;
1638:   }
1639:
1640:   .login-showcase,
1641:   .login-panel {
1642:     padding: 28px 18px;
1643:   }
1644:
1645:   .login-title {
1646:     font-size: 46px;
1647:   }
1648:
1649:   .hero-title {
1650:     font-size: 28px;
1651:   }
1652:
1653:   .hero-actions {
1654:     width: 100%;
1655:   }
1656:
1657:   .quick-grid {
1658:     grid-template-columns: 1fr;
1659:   }
1660:
1661:   .quick-card {
1662:     min-height: 148px;
1663:   }
1664:
1665:   .quick-card-media {
1666:     width: 88px;
1667:     height: 88px;
1668:   }
1669: }
```

==========
ARQUIVO: frontend/tsconfig.app.json
==========

```
0001: {
0002:   "compilerOptions": {
0003:     "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.app.tsbuildinfo",
0004:     "target": "ES2022",
0005:     "useDefineForClassFields": true,
0006:     "lib": ["ES2022", "DOM", "DOM.Iterable"],
0007:     "module": "ESNext",
0008:     "types": ["vite/client"],
0009:     "skipLibCheck": true,
0010:
0011:     /* Bundler mode */
0012:     "moduleResolution": "bundler",
0013:     "allowImportingTsExtensions": true,
0014:     "verbatimModuleSyntax": true,
```

```
0015:     "moduleDetection": "force",
0016:     "noEmit": true,
0017:     "jsx": "react-jsx",
0018:
0019:     /* Linting */
0020:     "strict": true,
0021:     "noUnusedLocals": true,
0022:     "noUnusedParameters": true,
0023:     "erasableSyntaxOnly": true,
0024:     "noFallthroughCasesInSwitch": true,
0025:     "noUncheckedSideEffectImports": true
0026:   },
0027:   "include": ["src"]
0028: }
```

==========
ARQUIVO: frontend/tsconfig.json
==========

```
0001: {
0002:   "files": [],
0003:   "references": [
0004:     { "path": "./tsconfig.app.json" },
0005:     { "path": "./tsconfig.node.json" }
0006:   ]
0007: }
```

==========
ARQUIVO: frontend/tsconfig.node.json
==========

```
0001: {
0002:   "compilerOptions": {
0003:     "tsBuildInfoFile": "./node_modules/.tmp/tsconfig.node.tsbuildinfo",
0004:     "target": "ES2023",
0005:     "lib": ["ES2023"],
0006:     "module": "ESNext",
0007:     "types": ["node"],
0008:     "skipLibCheck": true,
0009:
0010:     /* Bundler mode */
0011:     "moduleResolution": "bundler",
0012:     "allowImportingTsExtensions": true,
0013:     "verbatimModuleSyntax": true,
0014:     "moduleDetection": "force",
0015:     "noEmit": true,
0016:
0017:     /* Linting */
0018:     "strict": true,
0019:     "noUnusedLocals": true,
0020:     "noUnusedParameters": true,
0021:     "erasableSyntaxOnly": true,
0022:     "noFallthroughCasesInSwitch": true,
0023:     "noUncheckedSideEffectImports": true
0024:   },
0025:   "include": ["vite.config.ts"]
```

```
0026: }
```

==========
ARQUIVO: frontend/vite.config.ts
==========

```
0001: import { defineConfig } from 'vite'
0002: import react from '@vitejs/plugin-react'
0003:
0004: // https://vite.dev/config/
0005: export default defineConfig({
0006:   plugins: [react()],
0007: })
```

==========
ARQUIVO: migrar_sqlite_para_sqlserver.py
==========

```
0001: import pandas as pd
0002: import sqlalchemy as sa
0003:
0004: sqlite_engine = sa.create_engine("sqlite:///protheus.db")
0005:
0006: mssql_engine = sa.create_engine(
0007:     "mssql+pyodbc://@localhost/ERP_PROTHEUS"
0008:     "?driver=ODBC+Driver+18+for+SQL+Server"
0009:     "&Trusted_Connection=yes"
0010:     "&Encrypt=yes"
0011:     "&TrustServerCertificate=yes",
0012:     fast_executemany=True
0013: )
0014:
0015: tables = pd.read_sql(
0016:     "SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%';",
0017:     sqlite_engine
0018: )
0019:
0020: for t in tables["name"]:
0021:     df = pd.read_sql(f"SELECT * FROM {t}", sqlite_engine)
0022:     df.to_sql(t, mssql_engine, if_exists="replace", index=False)
0023:     print("Migrado:", t)
0024:
0025: print("Migração concluída.")
```

==========
ARQUIVO: models.sql
==========

```
0001: -- ========================
0002: -- USUÁRIOS
0003: -- ========================
0004: CREATE TABLE IF NOT EXISTS USUARIOS (
0005:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0006:     U_LOGIN TEXT UNIQUE NOT NULL,
0007:     U_SENHA_HASH TEXT NOT NULL,
0008:     U_PERFIL TEXT NOT NULL
0009: );
```

```
0010:
0011: CREATE INDEX IF NOT EXISTS IDX_USUARIOS_LOGIN ON USUARIOS(U_LOGIN);
0012:
0013: -- ========================
0014: -- LOGS (AUDITORIA)
0015: -- ========================
0016: CREATE TABLE IF NOT EXISTS LOGS (
0017:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0018:     L_USUARIO TEXT,
0019:     L_ACAO TEXT,
0020:     L_DATA TEXT
0021: );
0022:
0023: CREATE INDEX IF NOT EXISTS IDX_LOGS_USUARIO_DATA ON LOGS(L_USUARIO, L_DATA);
0024:
0025: -- ========================
0026: -- PRODUTOS (SB1) + CM + RESERVA
0027: -- ========================
0028: CREATE TABLE IF NOT EXISTS SB1_PRODUTOS (
0029:     B1_COD TEXT NOT NULL,
0030:     B1_DESC TEXT,
0031:     B1_PRECO REAL NOT NULL DEFAULT 0,
0032:     B1_ESTOQUE INTEGER NOT NULL DEFAULT 0,
0033:     B1_RESERVADO INTEGER NOT NULL DEFAULT 0,
0034:     B1_CM REAL NOT NULL DEFAULT 0,
0035:     B1_FILIAL TEXT NOT NULL,
0036:     PRIMARY KEY (B1_COD, B1_FILIAL)
0037: );
0038:
0039: CREATE INDEX IF NOT EXISTS IDX_SB1_FILIAL ON SB1_PRODUTOS(B1_FILIAL);
0040:
0041: -- ========================
0042: -- PEDIDOS (SC5/SC6)
0043: -- ========================
0044: CREATE TABLE IF NOT EXISTS SC5_PEDIDOS (
0045:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0046:     C5_DATA TEXT NOT NULL,
0047:     C5_FILIAL TEXT NOT NULL,
0048:     C5_VALOR_TOTAL REAL NOT NULL DEFAULT 0,
0049:     C5_USUARIO TEXT NOT NULL,
0050:     C5_STATUS TEXT NOT NULL DEFAULT 'ABERTO' -- ABERTO | FATURADO | CANCELADO
0051: );
0052:
0053: CREATE INDEX IF NOT EXISTS IDX_SC5_FILIAL_DATA ON SC5_PEDIDOS(C5_FILIAL, C5_DATA);
0054:
0055: CREATE TABLE IF NOT EXISTS SC6_ITENS (
0056:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0057:     C6_PEDIDO_ID INTEGER NOT NULL,
0058:     C6_PRODUTO TEXT NOT NULL,
0059:     C6_QTD INTEGER NOT NULL,
0060:     C6_VALOR REAL NOT NULL DEFAULT 0
0061: );
0062:
0063: CREATE INDEX IF NOT EXISTS IDX_SC6_PEDIDO ON SC6_ITENS(C6_PEDIDO_ID);
```

```
0064: CREATE INDEX IF NOT EXISTS IDX_SC6_PRODUTO ON SC6_ITENS(C6_PRODUTO);
0065:
0066: -- ========================
0067: -- MOVIMENTAÇÃO DE ESTOQUE (SD3)
0068: -- ========================
0069: CREATE TABLE IF NOT EXISTS SD3_MOV (
0070:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0071:     D3_DATA TEXT NOT NULL,
0072:     D3_FILIAL TEXT NOT NULL,
0073:     D3_PRODUTO TEXT NOT NULL,
0074:     D3_TIPO TEXT NOT NULL,          -- 'E' entrada, 'S' saída
0075:     D3_QTD INTEGER NOT NULL,
0076:     D3_CUSTO_UNIT REAL NOT NULL DEFAULT 0,
0077:     D3_VALOR REAL NOT NULL DEFAULT 0,
0078:     D3_ORIGEM TEXT,
0079:     D3_REF TEXT,
0080:     D3_USUARIO TEXT
0081: );
0082:
0083: CREATE INDEX IF NOT EXISTS IDX_SD3_PROD_FILIAL ON SD3_MOV(D3_PRODUTO, D3_FILIAL);
0084: CREATE INDEX IF NOT EXISTS IDX_SD3_FILIAL_DATA ON SD3_MOV(D3_FILIAL, D3_DATA);
0085: CREATE INDEX IF NOT EXISTS IDX_SD3_ORIGEM_REF ON SD3_MOV(D3_ORIGEM, D3_REF);
0086:
0087: -- ========================
0088: -- RESERVAS (pré-venda)
0089: -- ========================
0090: CREATE TABLE IF NOT EXISTS SZ0_RESERVA (
0091:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0092:     Z0_DATA TEXT NOT NULL,
0093:     Z0_FILIAL TEXT NOT NULL,
0094:     Z0_PRODUTO TEXT NOT NULL,
0095:     Z0_QTD INTEGER NOT NULL,
0096:     Z0_USUARIO TEXT NOT NULL,
0097:     Z0_STATUS TEXT NOT NULL DEFAULT 'ABERTA' -- ABERTA | CONFIRMADA | CANCELADA
0098: );
0099:
0100: CREATE INDEX IF NOT EXISTS IDX_SZ0_FILIAL_STATUS ON SZ0_RESERVA(Z0_FILIAL, Z0_STATUS);
0101:
0102: -- ========================
0103: -- TRANSFERÊNCIA (rastreamento opcional)
0104: -- ========================
0105: CREATE TABLE IF NOT EXISTS ST0_TRANSFER (
0106:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0107:     T0_DATA TEXT NOT NULL,
0108:     T0_PRODUTO TEXT NOT NULL,
0109:     T0_QTD INTEGER NOT NULL,
0110:     T0_ORIGEM TEXT NOT NULL,
0111:     T0_DESTINO TEXT NOT NULL,
0112:     T0_USUARIO TEXT NOT NULL,
0113:     T0_REF TEXT NOT NULL
0114: );
0115:
0116: CREATE INDEX IF NOT EXISTS IDX_ST0_REF ON ST0_TRANSFER(T0_REF);
0117:
```

```
0118: -- ========================
0119: -- FINANCEIRO (AR/AP)
0120: -- ========================
0121: CREATE TABLE IF NOT EXISTS SE1_AR (
0122:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0123:     E1_DATA TEXT NOT NULL,
0124:     E1_VENC TEXT NOT NULL,
0125:     E1_CLIENTE TEXT,
0126:     E1_VALOR REAL NOT NULL,
0127:     E1_STATUS TEXT NOT NULL DEFAULT 'ABERTO', -- ABERTO | BAIXADO | CANCELADO
0128:     E1_REF TEXT, -- pedido_id
0129:     E1_FILIAL TEXT NOT NULL
0130: );
0131:
0132: CREATE INDEX IF NOT EXISTS IDX_SE1_STATUS_FILIAL ON SE1_AR(E1_STATUS, E1_FILIAL);
0133:
0134: CREATE TABLE IF NOT EXISTS SF1_AP (
0135:     ID INTEGER PRIMARY KEY AUTOINCREMENT,
0136:     F1_DATA TEXT NOT NULL,
0137:     F1_VENC TEXT NOT NULL,
0138:     F1_FORN TEXT,
0139:     F1_VALOR REAL NOT NULL,
0140:     F1_STATUS TEXT NOT NULL DEFAULT 'ABERTO', -- ABERTO | BAIXADO | CANCELADO
0141:     F1_REF TEXT,
0142:     F1_FILIAL TEXT NOT NULL
0143: );
0144:
0145: CREATE INDEX IF NOT EXISTS IDX_SF1_STATUS_FILIAL ON SF1_AP(F1_STATUS, F1_FILIAL);
```

==========
ARQUIVO: postman_collection.json
==========

```
0001: {
0002:   "info": {
0003:     "name": "Mini Protheus - Testes",
0004:     "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
0005:   },
0006:   "item": [
0007:     {
0008:       "name": "Auth - Login",
0009:       "request": {
0010:         "method": "POST",
0011:         "header": [
0012:           { "key": "Content-Type", "value": "application/x-www-form-urlencoded" }
0013:         ],
0014:         "url": {
0015:           "raw": "http://127.0.0.1:8000/login",
0016:           "protocol": "http",
0017:           "host": ["127.0.0.1"],
0018:           "port": "8000",
0019:           "path": ["login"]
0020:         },
0021:         "body": {
0022:           "mode": "urlencoded",
```

0023:   "urlencoded": [
0024:   { "key": "username", "value": "admin" },
0025:   { "key": "password", "value": "Troque123!" }
0026:   ]
0027:   }
0028:   }
0029:   },
0030:   {
0031:   "name": "Param - SF4 TES Venda",
0032:   "request": {
0033:   "method": "POST",
0034:   "header": [
0035:   { "key": "Content-Type", "value": "application/json" },
0036:   { "key": "Authorization", "value": "Bearer {{token}}" }
0037:   ],
0038:   "url": {
0039:   "raw": "http://127.0.0.1:8000/parametros/sf4",
0040:   "protocol": "http",
0041:   "host": ["127.0.0.1"],
0042:   "port": "8000",
0043:   "path": ["parametros", "sf4"]
0044:   },
0045:   "body": {
0046:   "mode": "raw",
0047:   "raw": "{\n \"filial\": \"01\",\n \"tes_cod\": \"001\",\n \"cfop\": \"5102\",\n
\"icms\": 0.12,\n \"ipi\": 0.04,\n \"tipo\": \"S\",\n \"gera_tit\": true,\n \"gera_est\": true,\
n \"descr\": \"TES VENDA PADRAO\"\n}"
0048:   }
0049:   }
0050:   },
0051:   {
0052:   "name": "Param - SF4 TES Devolução",
0053:   "request": {
0054:   "method": "POST",
0055:   "header": [
0056:   { "key": "Content-Type", "value": "application/json" },
0057:   { "key": "Authorization", "value": "Bearer {{token}}" }
0058:   ],
0059:   "url": {
0060:   "raw": "http://127.0.0.1:8000/parametros/sf4",
0061:   "protocol": "http",
0062:   "host": ["127.0.0.1"],
0063:   "port": "8000",
0064:   "path": ["parametros", "sf4"]
0065:   },
0066:   "body": {
0067:   "mode": "raw",
0068:   "raw": "{\n \"filial\": \"01\",\n \"tes_cod\": \"002\",\n \"cfop\": \"1202\",\n
\"icms\": 0.12,\n \"ipi\": 0.04,\n \"tipo\": \"E\",\n \"gera_tit\": true,\n \"gera_est\": true,\
n \"descr\": \"TES DEVOLUCAO\"\n}"
0069:   }
0070:   }
0071:   },
0072:   {

0073:   "name": "Produtos - Criar",
0074:   "request": {
0075:   "method": "POST",
0076:   "header": [
0077:   { "key": "Content-Type", "value": "application/json" },
0078:   { "key": "Authorization", "value": "Bearer {{token}}" }
0079:   ],
0080:   "url": {
0081:   "raw": "http://127.0.0.1:8000/produtos",
0082:   "protocol": "http",
0083:   "host": ["127.0.0.1"],
0084:   "port": "8000",
0085:   "path": ["produtos"]
0086:   },
0087:   "body": {
0088:   "mode": "raw",
0089:   "raw": "{\n \"cod\": \"PRD001\",\n \"desc\": \"Produto Teste\",\n \"preco\": 100\
n}"
0090:   }
0091:   }
0092:   },
0093:   {
0094:   "name": "Clientes - Criar",
0095:   "request": {
0096:   "method": "POST",
0097:   "header": [
0098:   { "key": "Content-Type", "value": "application/json" },
0099:   { "key": "Authorization", "value": "Bearer {{token}}" }
0100:   ],
0101:   "url": {
0102:   "raw": "http://127.0.0.1:8000/clientes",
0103:   "protocol": "http",
0104:   "host": ["127.0.0.1"],
0105:   "port": "8000",
0106:   "path": ["clientes"]
0107:   },
0108:   "body": {
0109:   "mode": "raw",
0110:   "raw": "{\n \"cod\": \"000123\",\n \"nome\": \"Cliente Teste\",\n \"cpf_cnpj\": \
"12345678900\"\n}"
0111:   }
0112:   }
0113:   },
0114:   {
0115:   "name": "Pedidos - Criar",
0116:   "request": {
0117:   "method": "POST",
0118:   "header": [
0119:   { "key": "Content-Type", "value": "application/json" },
0120:   { "key": "Authorization", "value": "Bearer {{token}}" }
0121:   ],
0122:   "url": {
0123:   "raw": "http://127.0.0.1:8000/vendas/pedidos",
0124:   "protocol": "http",

0125:   "host": ["127.0.0.1"],
0126:   "port": "8000",
0127:   "path": ["vendas", "pedidos"]
0128:   },
0129:   "body": {
0130:   "mode": "raw",
0131:   "raw": "{\n \"filial\": \"01\",\n \"valor_total\": 1000,\n \"status\": \"ABERTO\"
\n}"
0132:   }
0133:   }
0134:   },
0135:   {
0136:   "name": "Pedidos - Adicionar Item",
0137:   "request": {
0138:   "method": "POST",
0139:   "header": [
0140:   { "key": "Content-Type", "value": "application/json" },
0141:   { "key": "Authorization", "value": "Bearer {{token}}" }
0142:   ],
0143:   "url": {
0144:   "raw": "http://127.0.0.1:8000/vendas/pedidos/{{pedido_id}}/itens",
0145:   "protocol": "http",
0146:   "host": ["127.0.0.1"],
0147:   "port": "8000",
0148:   "path": ["vendas", "pedidos", "{{pedido_id}}", "itens"]
0149:   },
0150:   "body": {
0151:   "mode": "raw",
0152:   "raw": "{\n \"filial\": \"01\",\n \"produto\": \"PRD001\",\n \"qtd\": 2,\n \"tot
al\": 200,\n \"preco_unit\": 100\n}"
0153:   }
0154:   }
0155:   },
0156:   {
0157:   "name": "Pedidos - Liberar Tudo",
0158:   "request": {
0159:   "method": "POST",
0160:   "header": [
0161:   { "key": "Content-Type", "value": "application/json" },
0162:   { "key": "Authorization", "value": "Bearer {{token}}" }
0163:   ],
0164:   "url": {
0165:   "raw": "http://127.0.0.1:8000/vendas/pedidos/{{pedido_id}}/liberar-tudo",
0166:   "protocol": "http",
0167:   "host": ["127.0.0.1"],
0168:   "port": "8000",
0169:   "path": ["vendas", "pedidos", "{{pedido_id}}", "liberar-tudo"]
0170:   },
0171:   "body": {
0172:   "mode": "raw",
0173:   "raw": "{\n \"filial\": \"01\",\n \"usar_estoque\": true\n}"
0174:   }
0175:   }
0176:   },

0177:   {
0178:   "name": "Pedidos - Faturar",
0179:   "request": {
0180:   "method": "POST",
0181:   "header": [
0182:   { "key": "Content-Type", "value": "application/json" },
0183:   { "key": "Authorization", "value": "Bearer {{token}}" }
0184:   ],
0185:   "url": {
0186:   "raw": "http://127.0.0.1:8000/vendas/pedidos/{{pedido_id}}/faturar",
0187:   "protocol": "http",
0188:   "host": ["127.0.0.1"],
0189:   "port": "8000",
0190:   "path": ["vendas", "pedidos", "{{pedido_id}}", "faturar"]
0191:   },
0192:   "body": {
0193:   "mode": "raw",
0194:   "raw": "{\n \"filial\": \"01\",\n \"cliente_cod\": \"000123\",\n \"venc_dias\": 3
0,\n \"tes_cod\": \"001\"\n}"
0195:   }
0196:   }
0197:   },
0198:   {
0199:   "name": "NF - Estornar",
0200:   "request": {
0201:   "method": "POST",
0202:   "header": [
0203:   { "key": "Content-Type", "value": "application/json" },
0204:   { "key": "Authorization", "value": "Bearer {{token}}" }
0205:   ],
0206:   "url": {
0207:   "raw": "http://127.0.0.1:8000/vendas/nf/{{nf_id}}/estornar",
0208:   "protocol": "http",
0209:   "host": ["127.0.0.1"],
0210:   "port": "8000",
0211:   "path": ["vendas", "nf", "{{nf_id}}", "estornar"]
0212:   },
0213:   "body": {
0214:   "mode": "raw",
0215:   "raw": "{\n \"filial\": \"01\",\n \"motivo\": \"Teste estorno\"\n}"
0216:   }
0217:   }
0218:   },
0219:   {
0220:   "name": "NF - Devolver",
0221:   "request": {
0222:   "method": "POST",
0223:   "header": [
0224:   { "key": "Content-Type", "value": "application/json" },
0225:   { "key": "Authorization", "value": "Bearer {{token}}" }
0226:   ],
0227:   "url": {
0228:   "raw": "http://127.0.0.1:8000/vendas/nf/{{nf_id}}/devolver",
0229:   "protocol": "http",

```
0230:           "host": ["127.0.0.1"],
0231:           "port": "8000",
0232:           "path": ["vendas", "nf", "{{nf_id}}", "devolver"]
0233:         },
0234:         "body": {
0235:           "mode": "raw",
0236:           "raw": "{\n  \"filial\": \"01\",\n  \"tes_cod\": \"002\",\n  \"venc_dias\": 30\n}"
0237:         }
0238:       }
0239:     },
0240:     {
0241:       "name": "NF - Buscar por Doc/Serie",
0242:       "request": {
0243:         "method": "GET",
0244:         "header": [
0245:           { "key": "Authorization", "value": "Bearer {{token}}" }
0246:         ],
0247:         "url": {
0248:           "raw": "http://127.0.0.1:8000/vendas/nf/buscar?filial=01&doc={{nf_doc}}&serie={{nf_s
erie}}",
0249:           "protocol": "http",
0250:           "host": ["127.0.0.1"],
0251:           "port": "8000",
0252:           "path": ["vendas", "nf", "buscar"],
0253:           "query": [
0254:             { "key": "filial", "value": "01" },
0255:             { "key": "doc", "value": "{{nf_doc}}" },
0256:             { "key": "serie", "value": "{{nf_serie}}" }
0257:           ]
0258:         }
0259:       }
0260:     },
0261:     {
0262:       "name": "Relatórios - Conciliação NF",
0263:       "request": {
0264:         "method": "GET",
0265:         "header": [
0266:           { "key": "Authorization", "value": "Bearer {{token}}" }
0267:         ],
0268:         "url": {
0269:           "raw": "http://127.0.0.1:8000/relatorios/conciliacao-nf?filial=01",
0270:           "protocol": "http",
0271:           "host": ["127.0.0.1"],
0272:           "port": "8000",
0273:           "path": ["relatorios", "conciliacao-nf"],
0274:           "query": [
0275:             { "key": "filial", "value": "01" }
0276:           ]
0277:         }
0278:       }
0279:     }
0280:   ]
0281: }
```

==========
ARQUIVO: README.md
==========

Mini Protheus

> Versao recomendada para publicacao atual: `v1.0.0-beta.1`

ERP enxuto em desenvolvimento, inspirado na organizacao funcional do ecossistema Protheus, com foco em operacao comercial, financeiro, reservas, fiscal e rastreabilidade de fluxo.

O projeto combina:

- backend FastAPI exposto como API HTTP
- frontend React + Vite em TypeScript
- persistencia em SQL Server
- regras de negocio organizadas por modulo e fluxo

Visao Geral

O Mini Protheus nasceu como uma base de ERP compacta para experimentar uma arquitetura de negocio mais proxima de sistemas transacionais reais:

- pedido de venda com itens, liberacao, faturamento e rastreio
- contas a receber e a pagar integradas ao caixa
- reserva de estoque com confirmacao e cancelamento
- regras fiscais e conciliacao entre NF e financeiro
- fechamento de periodo e relatorios gerenciais

Hoje o projeto ja funciona como um backend de API pura com frontend dedicado, sem camada server-side de templates. A direcao atual e evoluir essa base para um produto mais robusto, com contratos de API mais previsiveis, maior cobertura de testes e mais hardening operacional.

Status do Projeto

Este repositorio representa uma versao de desenvolvimento.

Estado atual:

- arquitetura principal definida
- fluxos de negocio criticos implementados
- autenticacao e bootstrap endurecidos no backend
- contrato de erro da API padronizado
- testes automatizados para fluxos de negocio e testes DB-backed opcionais

Observacao importante:

- ainda e uma versao em evolucao
- hardening de seguranca, maturidade de observabilidade e refinamento de contrato seguem em andamento
- o foco atual e consolidacao tecnica e narrativa de produto

Publicacao Atual

0046: Esta base ja pode ser publicada como portfolio tecnico e produto em evolucao, com o seguinte p
osicionamento:
0047:
0048: - `mini ERP inspirado em fluxos operacionais do ecossistema Protheus`
0049: - foco atual em `comercial`, `financeiro`, `estoque basico`, `reservas`, `auditoria`, `precifi
cacao` e `relatorios gerenciais simples`
0050: - versao de publicacao recomendada: `v1.0.0-beta.1`
0051:
0052: Escopo recomendado para a publicacao:
0053:
0054: - demonstrar um ERP operacional funcional
0055: - mostrar arquitetura modular e regras de negocio reais
0056: - deixar explicito que o projeto ainda esta em maturacao
0057:
0058: ## Limitacoes Atuais
0059:
0060: Para publicacao honesta e segura, estas limitacoes devem ser consideradas parte do estado atua
l do projeto:
0061:
0062: - compras ainda nao existem como modulo formal de `pedido de compra`
0063: - custo por fornecedor existe, mas ainda funciona como fluxo operacional de entrada e formacao
 de custo, nao como SIGACOM completo
0064: - estoque ainda nao possui armazem/local, lote, validade e inventario formal
0065: - fiscal cobre cenarios basicos, mas ainda nao representa profundidade completa de TES/CFOP/CS
T de um ERP maduro
0066: - DRE e margem ja funcionam, mas ainda dependem de categorizacao financeira consistente para l
eitura gerencial limpa
0067: - governanca de permissao ainda e basica; nao ha ACL fina por modulo/acao
0068: - a plataforma e propria (`FastAPI + React + SQL Server`), portanto o projeto e inspirado no P
rotheus em fluxo de negocio, nao equivalente tecnico ao Protheus
0069:
0070: ## Modulos Principais
0071:
0072: ### Comercial
0073:
0074: - criacao de pedidos
0075: - inclusao e manutencao de itens
0076: - liberacao parcial e total
0077: - faturamento de pedido
0078: - estorno, cancelamento e devolucao de NF
0079:
0080: ### Parceiros
0081:
0082: - clientes
0083: - fornecedores
0084: - associacao de tabela de preco por cliente
0085:
0086: ### Reservas
0087:
0088: - criacao de reserva de estoque
0089: - confirmacao com geracao de pedido, NF e AR
0090: - cancelamento com reversao de reserva
0091:
0092: ### Financeiro

0093:
0094: - contas a receber
0095: - contas a pagar
0096: - caixa
0097: - categorias financeiras
0098: - DRE simples
0099:
0100: ### Fiscal e Consistencia
0101:
0102: - TES e regras fiscais
0103: - calculo fiscal por item
0104: - conciliacao entre notas e financeiro
0105: - consistencia fiscal e SD2
0106:
0107: ### Parametros e Governanca
0108:
0109: - bootstrap controlado por ambiente
0110: - politica de one-time setup persistida no banco
0111: - consulta administrativa do estado de setup
0112: - fechamento e reabertura de periodo
0113:
0114: ## Fluxos de Negocio
0115:
0116: ### 1. Venda Completa
0117:
0118: - criar pedido
0119: - adicionar item
0120: - liberar pedido
0121: - faturar
0122: - gerar contas a receber
0123: - receber e refletir no caixa
0124:
0125: ### 2. Ciclo de AP
0126:
0127: - lancar conta a pagar
0128: - listar titulos
0129: - pagar AP
0130: - refletir saida no caixa
0131:
0132: ### 3. Reserva de Estoque
0133:
0134: - criar reserva
0135: - precificar pela tabela vigente
0136: - confirmar reserva
0137: - gerar pedido, NF e AR
0138: - cancelar quando aplicavel
0139:
0140: ### 4. Controle Gerencial
0141:
0142: - consultar margem por produto
0143: - gerar DRE simples
0144: - rodar conciliacao NF x financeiro
0145: - fechar e reabrir periodo
0146:

Interface

O frontend ja possui paginas para os principais modulos do sistema:

- login
- pedidos
- detalhe de pedido
- parceiros
- clientes
- fornecedores
- reservas
- financeiro AR
- financeiro AP
- caixa
- categorias
- produtos
- usuarios
- auditoria
- DRE
- margem por produto

Arquivos de tela em `frontend/src/pages/`:

- `Login.tsx`
- `pedidosList.tsx`
- `pedidoDetalhe.tsx`
- `parceiros.tsx`
- `reservasList.tsx`
- `financeiroAR.tsx`
- `financeiroAP.tsx`
- `financeiroCaixa.tsx`
- `relatorioDRE.tsx`
- `relatorioMargemProduto.tsx`

Prints da Interface e da API

O material abaixo mostra a API documentada no Swagger e as telas principais do frontend React.

Swagger / OpenAPI:

![Swagger Overview](docs/screenshots/swagger-overview.png)
![Swagger Vendas](docs/screenshots/swagger-vendas.png)
![Swagger Auth Usuarios Produtos](docs/screenshots/swagger-auth-usuarios-produtos.png)
![Swagger Schemas](docs/screenshots/swagger-schemas.png)

Frontend:

![Frontend Login](docs/screenshots/frontend-login.png)
![Frontend Pedidos](docs/screenshots/frontend-pedidos.png)
![Frontend Financeiro AR](docs/screenshots/frontend-financeiro-ar.png)
![Frontend Caixa](docs/screenshots/frontend-caixa.png)
![Frontend DRE](docs/screenshots/frontend-dre.png)
![Frontend Margem Produto](docs/screenshots/frontend-margem-produto.png)
![Frontend Usuarios](docs/screenshots/frontend-usuarios.png)

Mais telas e a lista completa de arquivos: [docs/screenshots/README.md](/c:/Users/umqua/Desktop/mini_protheus/docs/screenshots/README.md)

Diferenciais Tecnicos

- API pura com frontend desacoplado
- bootstrap protegido por ambiente e bloqueio persistente de setup
- payload de erro uniforme com `detail`, `code`, `error_type` e `request_id`
- testes de fluxo de negocio em nivel de API
- testes DB-backed opcionais com rollback
- CI preparada para execucao com SQL Server
- limpeza de shims, imports implicitos e caminhos legados

Roadmap

Curto Prazo

- consolidar homologacao funcional ponta a ponta
- ampliar documentacao por fluxo
- expandir exemplos OpenAPI para mais endpoints
- consolidar naming e contratos restantes da API
- fechar modulo inicial de custo por fornecedor e AP derivado

Medio Prazo

- criar modulo formal de compras
- aumentar cobertura de testes integrados com banco real
- adicionar mais cenarios negativos por modulo
- endurecer validacoes de periodo, fiscal e conciliacao
- melhorar observabilidade e logs de diagnostico

Longo Prazo

- maturidade de deploy
- controle mais fino de permissao por modulo
- dashboards e indicadores operacionais
- compras, estoque avancado e fiscal mais profundo
- refinamento visual do frontend para apresentacao de produto

Release Notes

Resumo da versao de publicacao `v1.0.0-beta.1`:

- backend consolidado como API pura em FastAPI
- frontend React + Vite com shell visual de ERP e navegacao por modulos
- bootstrap endurecido com politica de one-time setup persistida no banco
- contrato de erro da API padronizado
- auditoria, alertas e filtros contextuais no frontend
- pedidos com CMV por item e faturamento integrado
- AR/AP com reflexo em caixa e categorizacao financeira
- custo por fornecedor com atualizacao de `B1_CM` e geracao de AP
- DRE e margem por produto operacionais
- documentacao de portfolio com screenshots e PDF do codigo-fonte

0254:
0255: ## Checklist de Publicacao
0256:
0257: - `README` revisado e coerente com o estado real do projeto
0258: - screenshots principais versionadas em `docs/screenshots/`
0259: - instrucao de subida local validada
0260: - status de versao e limitacoes declarados
0261: - testes principais executados
0262: - release notes registradas
0263: - bugs conhecidos tratados como backlog, nao mascarados como escopo concluido
0264:
0265: ## Stack
0266:
0267: - Backend: FastAPI, JWT (`python-jose`), `pyodbc`
0268: - Frontend: React, React Router, Vite, TypeScript, ESLint
0269: - Banco: SQL Server (via ODBC Driver 18)
0270:
0271: ## Requisitos
0272:
0273: - Python 3.12+ (testado com 3.14)
0274: - Node.js 20+
0275: - SQL Server acessivel
0276: - ODBC Driver 18 for SQL Server instalado
0277:
0278: ## Configuracao
0279:
0280: No diretorio raiz, crie o arquivo `.env` a partir do exemplo:
0281:
0282: ```powershell
0283: Copy-Item .env.example .env
0284: ```
0285:
0286: Variaveis principais:
0287:
0288: - `APP_ENV` (`production`, `development`, `setup`)
0289: - `BOOTSTRAP_ENABLED`
0290: - `JWT_SECRET_KEY`
0291: - `CORS_ALLOW_ORIGINS`
0292: - `DB_CONN_STR`
0293:
0294: Observacoes de configuracao:
0295:
0296: - a aplicacao e API pura; nao usa sessao server-side, Jinja ou middleware de sessao
0297: - `CORS_ALLOW_ORIGINS` tem default voltado ao frontend local (`http://localhost:5173`) e deve ser ajustado no deploy
0298: - `JWT_SECRET_KEY` tem fallback de desenvolvimento em [settings.py](/c:/Users/umqua/Desktop/mini_protheus/app/core/settings.py), mas em `APP_ENV=production` a aplicacao falha ao subir se a chave estiver ausente ou com valor default
0299:
0300: No frontend, confirme o `VITE_API_BASE_URL` em `frontend/.env`.
0301:
0302: ## Executando o Backend
0303:
0304: Instale dependencias:

0305:
0306: ```powershell
0307: python -m pip install -r requirements-dev.txt
0308: ```
0309:
0310: Suba a API:
0311:
0312: ```powershell
0313: uvicorn app.main:app --reload
0314: ```
0315:
0316: API local padrao: `http://127.0.0.1:8000`
0317:
0318: ## Contrato de Erro da API
0319:
0320: As respostas de erro do backend seguem um payload uniforme:
0321:
0322: ```json
0323: {
0324:   "detail": "Pedido sem itens (SC6)",
0325:   "code": "BUSINESS_ERROR",
0326:   "error_type": "business",
0327:   "request_id": "6df2d507-cf4a-4d85-b8db-7b8f81993db0"
0328: }
0329: ```
0330:
0331: Campos:
0332:
0333: - `detail`: mensagem objetiva para cliente e diagnostico
0334: - `code`: codigo estavel do tipo de erro
0335: - `error_type`: categoria (`business`, `auth`, `authorization`, `conflict`, `validation`, `http`, `internal`)
0336: - `request_id`: mesmo valor devolvido no header `X-Request-Id`
0337:
0338: Padrao HTTP:
0339:
0340: - `400`: regra de negocio
0341: - `401`: autenticacao ou token invalido
0342: - `403`: autorizacao ou perfil insuficiente
0343: - `404`: endpoint ou recurso indisponivel
0344: - `409`: conflito de dados
0345: - `422`: payload invalido
0346: - `500`: erro interno
0347:
0348: ## Primeiro Usuario
0349:
0350: O sistema nao possui `admin/admin` padrao. O primeiro admin deve ser criado via bootstrap.
0351:
0352: Politica de one-time setup no backend:
0353:
0354: - `/bootstrap` so responde quando `APP_ENV` for `development` ou `setup`
0355: - e quando `BOOTSTRAP_ENABLED=true`
0356: - e mesmo assim so funciona enquanto nao existir nenhum usuario

0357: - e apos a primeira execucao bem-sucedida o backend grava `setup concluido` no banco e bloquei a novas tentativas
0358:
0359: Criar admin inicial:
0360:
0361: ```powershell
0362: $env:APP_ENV="setup"
0363: $env:BOOTSTRAP_ENABLED="true"
0364: $body = @{ login = "admin"; senha = "Troque123!" } | ConvertTo-Json
0365: Invoke-RestMethod `
0366:   -Method Post `
0367:   -Uri "http://127.0.0.1:8000/bootstrap" `
0368:   -ContentType "application/json" `
0369:   -Body $body
0370: ```
0371:
0372: Consulta administrativa do estado do setup:
0373:
0374: ```powershell
0375: Invoke-RestMethod `
0376:   -Method Get `
0377:   -Uri "http://127.0.0.1:8000/setup-status" `
0378:   -Headers @{ Authorization = "Bearer $token" }
0379: ```
0380:
0381: Validar login e obter token:
0382:
0383: ```powershell
0384: Invoke-RestMethod `
0385:   -Method Post `
0386:   -Uri "http://127.0.0.1:8000/login" `
0387:   -ContentType "application/x-www-form-urlencoded" `
0388:   -Body "username=admin&password=Troque123!"
0389: ```
0390:
0391: ## Executando o Frontend
0392:
0393: ```powershell
0394: cd frontend
0395: npm install
0396: npm run dev
0397: ```
0398:
0399: Frontend local padrao: `http://localhost:5173`
0400:
0401: ## Qualidade e Testes
0402:
0403: Backend:
0404:
0405: ```powershell
0406: pytest -q
0407: ```
0408:
0409: Testes de integracao com banco real:

0410:
0411: ```powershell
0412: $env:TEST_DB_CONN_STR="DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;DATABASE=ERP_PROTHEUS_TEST;Trusted_Connection=yes;Encrypt=yes;TrustServerCertificate=yes;"
0413: pytest -q tests/test_business_flows_db.py
0414: ```
0415:
0416: Se voce ja tem dados no SQL Server local, crie antes uma base separada de testes.
0417: Existe um script seguro para isso em [sql_create_test_database.sql](/c:/Users/umqua/Desktop/mini_protheus/sql_create_test_database.sql).
0418:
0419: Depois inicialize o schema minimo da base de testes:
0420:
0421: ```powershell
0422: $env:TEST_DB_ADMIN_CONN_STR="DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;DATABASE=master;Trusted_Connection=yes;Encrypt=yes;TrustServerCertificate=yes;"
0423: $env:TEST_DB_CONN_STR="DRIVER={ODBC Driver 18 for SQL Server};SERVER=localhost;DATABASE=ERP_PROTHEUS_TEST;Trusted_Connection=yes;Encrypt=yes;TrustServerCertificate=yes;"
0424: python tests/init_integration_db.py
0425: pytest -q tests/test_business_flows_db.py
0426: ```
0427:
0428: Esses testes rodam em transacao com rollback ao final e hoje cobrem:
0429:
0430: - pedido -> item -> faturamento -> AR -> recebimento -> caixa
0431: - AP -> pagamento -> caixa
0432: - reserva -> confirmacao
0433: - margem por produto
0434: - DRE
0435: - conciliacao NF/financeiro
0436: - consistencia fiscal/SD2
0437: - fechamento e reabertura de periodo
0438: - faturamento bloqueado em periodo fechado
0439: - pagamento duplicado de AP bloqueado
0440: - confirmacao de reserva bloqueada por estoque/reservado insuficiente
0441: - divergencia real entre NF e financeiro detectada na conciliacao
0442:
0443: Sem `TEST_DB_CONN_STR`, eles sao ignorados automaticamente.
0444:
0445: CI:
0446:
0447: - `.github/workflows/tests.yml` roda a suite padrao em todo push/PR
0448: - o job `integration-db-tests` sobe um SQL Server local no GitHub Actions, inicializa um schema minimo via [tests/init_integration_db.py](/c:/Users/umqua/Desktop/mini_protheus/tests/init_integration_db.py) e executa os testes DB-backed de verdade
0449: - opcionalmente, o secret `CI_SQLSERVER_SA_PASSWORD` pode sobrescrever a senha padrao usada no container de CI
0450:
0451: Frontend:
0452:
0453: ```powershell
0454: cd frontend
0455: npm run lint
0456: npm run build

0457: ```
0458:
0459: ## Estrutura
0460:
0461: - `app/`: backend FastAPI
0462: - `app/api/routers/`: rotas da API
0463: - `app/use_cases/`: regras de negocio
0464: - `app/infra/`: acesso a dados e repositorios
0465: - `frontend/src/`: aplicacao React
0466: - `tests/`: testes automatizados do backend
0467:
0468: ## Troubleshooting Rapido
0469:
0470: - `401 Token invalido`: confira `JWT_SECRET_KEY` e expiracao
0471: - erro de conexao SQL Server: valide `DB_CONN_STR` e ODBC Driver 18
0472: - frontend sem conectar no backend: valide `frontend/.env` (`VITE_API_BASE_URL`)

==========
ARQUIVO: RELEASE_CHECKLIST.md
==========
0001: # Release Checklist
0002:
0003: ## Publicacao Recomendada
0004:
0005: - Nome: `Mini Protheus`
0006: - Tag sugerida: `v1.0.0-beta.1`
0007: - Posicionamento: `mini ERP operacional em desenvolvimento, inspirado em fluxos de backoffice Protheus`
0008:
0009: ## Antes de Publicar
0010:
0011: - revisar `README.md`
0012: - revisar `CHANGELOG.md`
0013: - revisar screenshots em `docs/screenshots/`
0014: - garantir que `.env.example` esteja coerente
0015: - validar subida local de backend e frontend
0016: - executar `pytest -q`
0017: - executar `cd frontend && npm run lint`
0018:
0019: ## Comunicacao Recomendada
0020:
0021: - destacar modulos ja operacionais
0022: - declarar limitacoes atuais
0023: - tratar roadmap como evolucao planejada, nao como escopo concluido
0024: - nao posicionar o produto como equivalente tecnico ao TOTVS Protheus
0025:
0026: ## Bugs Conhecidos Aceitaveis para a Publicacao
0027:
0028: - profundidade fiscal parcial
0029: - ausencia de modulo formal de compras
0030: - governanca de permissao ainda basica
0031: - estoque avancado ainda incompleto

==========

ARQUIVO: requirements-dev.txt
==========
0001: -r requirements.txt
0002: pytest
0003: httpx

==========
ARQUIVO: requirements.txt
==========
0001: fastapi
0002: uvicorn
0003: python-jose
0004: passlib
0005: pyodbc
0006: python-multipart

==========
ARQUIVO: sql_create_test_database.sql
==========
0001: /*
0002: Cria uma base separada para testes de integração, sem tocar na base atual.
0003:
0004: Uso no SSMS:
0005: 1) Ajuste @source_db e @test_db.
0006: 2) Execute conectado na instância SQL Server.
0007: 3) Depois aponte TEST_DB_CONN_STR para @test_db.
0008: 4) Rode: python tests/init_integration_db.py
0009:
0010: Observações:
0011: - Este script NÃO altera a base de origem.
0012: - Ele só cria a base de testes se ela ainda não existir.
0013: - A base criada nasce vazia; o schema mínimo usado pelos testes é inicializado
0014:   pelo script Python tests/init_integration_db.py.
0015: */
0016:
0017: SET NOCOUNT ON;
0018:
0019: DECLARE @source_db sysname = N'ERP_PROTHEUS';
0020: DECLARE @test_db   sysname = N'ERP_PROTHEUS_TEST';
0021:
0022: IF DB_ID(@source_db) IS NULL
0023: BEGIN
0024:   RAISERROR('Base de origem não encontrada: %s', 16, 1, @source_db);
0025:   RETURN;
0026: END;
0027:
0028: IF DB_ID(@test_db) IS NOT NULL
0029: BEGIN
0030:   PRINT 'Base de testes já existe: ' + @test_db;
0031:   RETURN;
0032: END;
0033:
0034: DECLARE @data_path nvarchar(4000);
0035: DECLARE @log_path nvarchar(4000);

```
0036:
0037: SELECT TOP 1
0038:     @data_path = LEFT(physical_name, LEN(physical_name) - CHARINDEX('\', REVERSE(physical_name
))) + '\'
0039: FROM sys.master_files
0040: WHERE database_id = DB_ID(@source_db)
0041:   AND type = 0
0042: ORDER BY file_id;
0043:
0044: SELECT TOP 1
0045:     @log_path = LEFT(physical_name, LEN(physical_name) - CHARINDEX('\', REVERSE(physical_name)
)) + '\'
0046: FROM sys.master_files
0047: WHERE database_id = DB_ID(@source_db)
0048:   AND type = 1
0049: ORDER BY file_id;
0050:
0051: IF @data_path IS NULL OR @log_path IS NULL
0052: BEGIN
0053:     RAISERROR('Não foi possível resolver os caminhos físicos da base de origem.', 16, 1);
0054:     RETURN;
0055: END;
0056:
0057: DECLARE @sql nvarchar(max) = N'
0058: CREATE DATABASE [' + REPLACE(@test_db, N']', N']]') + N']
0059: ON PRIMARY
0060: (
0061:     NAME = N''' + REPLACE(@test_db, N'''', N''''''') + N'_data'',
0062:     FILENAME = N''' + REPLACE(@data_path, N'''', N''''''') + REPLACE(@test_db, N'''', N''''''')
+ N'.mdf'',
0063:     SIZE = 128MB,
0064:     FILEGROWTH = 64MB
0065: )
0066: LOG ON
0067: (
0068:     NAME = N''' + REPLACE(@test_db, N'''', N''''''') + N'_log'',
0069:     FILENAME = N''' + REPLACE(@log_path, N'''', N''''''') + REPLACE(@test_db, N'''', N''''''') +
 N'_log.ldf'',
0070:     SIZE = 64MB,
0071:     FILEGROWTH = 64MB
0072: );';
0073:
0074: EXEC sp_executesql @sql;
0075:
0076: PRINT 'Base de testes criada com sucesso: ' + @test_db;
0077: PRINT 'Próximo passo: apontar TEST_DB_CONN_STR para essa base e rodar tests/init_integration_d
b.py';
```

==========
ARQUIVO: sql_step1_2.sql
==========

```
0001: -- Patch 1 e 2 (Protheus-like)
0002: -- 1) Garantir colunas de cancelamento/soft delete (D_E_L_E_T_, R_E_C_N_O_, R_E_C_D_E_L_)
0003: -- 2) Criar sequences RECNO e preencher R_E_C_N_O_ onde estiver NULL
```

```
0004: -- Observação: script idempotente (pode rodar mais de uma vez).
0005:
0006: SET NOCOUNT ON;
0007:
0008: DECLARE @tables TABLE (name sysname);
0009: INSERT INTO @tables (name)
0010: VALUES
0011:  ('SC5_PEDIDOS'),
0012:  ('SC6_ITENS'),
0013:  ('SF2_NOTAS'),
0014:  ('SD2_ITENS'),
0015:  ('SE1_AR'),
0016:  ('SF1_AP'),
0017:  ('SD3_MOV'),
0018:  ('SB1_PRODUTOS'),
0019:  ('SA1_CLIENTES'),
0020:  ('SA2_FORNECEDORES'),
0021:  ('SE5_CONTAS'),
0022:  ('SE5_CATEG'),
0023:  ('SE5_MOV'),
0024:  ('SZ0_RESERVA');
0025:
0026: DECLARE @t sysname;
0027: DECLARE table_cursor CURSOR FOR SELECT name FROM @tables;
0028: OPEN table_cursor;
0029: FETCH NEXT FROM table_cursor INTO @t;
0030:
0031: WHILE @@FETCH_STATUS = 0
0032: BEGIN
0033:     -- D_E_L_E_T_
0034:     IF COL_LENGTH('dbo.' + @t, 'D_E_L_E_T_') IS NULL
0035:     BEGIN
0036:         EXEC('ALTER TABLE dbo.' + @t + ' ADD D_E_L_E_T_ CHAR(1) NULL');
0037:     END
0038:
0039:     -- R_E_C_N_O_
0040:     IF COL_LENGTH('dbo.' + @t, 'R_E_C_N_O_') IS NULL
0041:     BEGIN
0042:         EXEC('ALTER TABLE dbo.' + @t + ' ADD R_E_C_N_O_ INT NULL');
0043:     END
0044:
0045:     -- R_E_C_D_E_L_
0046:     IF COL_LENGTH('dbo.' + @t, 'R_E_C_D_E_L_') IS NULL
0047:     BEGIN
0048:         EXEC('ALTER TABLE dbo.' + @t + ' ADD R_E_C_D_E_L_ INT NULL');
0049:     END
0050:
0051:     -- Sequence RECNO por tabela
0052:     DECLARE @seq sysname = 'SEQ_' + @t + '_RECNO';
0053:     IF OBJECT_ID('dbo.' + @seq, 'SO') IS NULL
0054:     BEGIN
0055:         EXEC('CREATE SEQUENCE dbo.' + @seq + ' AS INT START WITH 1 INCREMENT BY 1');
0056:     END
0057:
```

```
0058:     -- Preenche R_E_C_N_O_ faltante
0059:     EXEC('UPDATE dbo.' + @t + ' SET R_E_C_N_O_ = NEXT VALUE FOR dbo.' + @seq + ' WHERE R_E_C_N
_O_ IS NULL');
0060:
0061:     FETCH NEXT FROM table_cursor INTO @t;
0062: END
0063:
0064: CLOSE table_cursor;
0065: DEALLOCATE table_cursor;
0066:
0067: -- Campos de cancelamento mínimos (SD2/SF2)
0068: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_STATUS') IS NULL
0069:     ALTER TABLE dbo.SD2_ITENS ADD D2_STATUS NVARCHAR(20) NULL;
0070: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_DATA_CANCEL') IS NULL
0071:     ALTER TABLE dbo.SD2_ITENS ADD D2_DATA_CANCEL DATETIME2(0) NULL;
0072: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_USUARIO_CANCEL') IS NULL
0073:     ALTER TABLE dbo.SD2_ITENS ADD D2_USUARIO_CANCEL NVARCHAR(50) NULL;
0074: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_MOTIVO_CANCEL') IS NULL
0075:     ALTER TABLE dbo.SD2_ITENS ADD D2_MOTIVO_CANCEL NVARCHAR(200) NULL;
0076:
0077: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_STATUS') IS NULL
0078:     ALTER TABLE dbo.SF2_NOTAS ADD F2_STATUS NVARCHAR(20) NULL;
0079: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_DATA_CANCEL') IS NULL
0080:     ALTER TABLE dbo.SF2_NOTAS ADD F2_DATA_CANCEL DATETIME2(0) NULL;
0081: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_USUARIO_CANCEL') IS NULL
0082:     ALTER TABLE dbo.SF2_NOTAS ADD F2_USUARIO_CANCEL NVARCHAR(50) NULL;
0083: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_MOTIVO_CANCEL') IS NULL
0084:     ALTER TABLE dbo.SF2_NOTAS ADD F2_MOTIVO_CANCEL NVARCHAR(200) NULL;
0085:
0086: -- Ajustes de item Protheus-like
0087: IF COL_LENGTH('dbo.SC6_ITENS', 'C6_ITEM') IS NULL
0088:     ALTER TABLE dbo.SC6_ITENS ADD C6_ITEM INT NULL;
0089: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_ITEM_NUM') IS NULL
0090:     ALTER TABLE dbo.SD2_ITENS ADD D2_ITEM_NUM CHAR(4) NULL;
```

==========
ARQUIVO: sql_step1_2_b.sql
==========

```
0001: -- Passo 1/2 (complemento)
0002: -- Backfill C6_ITEM por pedido e D2_ITEM_NUM por NF
0003: -- Idempotente (recalcula apenas NULL)
0004:
0005: SET NOCOUNT ON;
0006: GO
0007:
0008: -- Garantir colunas (evita erro de compilação)
0009: IF NOT EXISTS (
0010:     SELECT 1 FROM sys.columns
0011:     WHERE object_id = OBJECT_ID('dbo.SC6_ITENS') AND name = 'C6_ITEM'
0012: )
0013: BEGIN
0014:     ALTER TABLE dbo.SC6_ITENS ADD C6_ITEM INT NULL;
0015: END
0016:
```

```
0017: IF NOT EXISTS (
0018:     SELECT 1 FROM sys.columns
0019:     WHERE object_id = OBJECT_ID('dbo.SD2_ITENS') AND name = 'D2_ITEM_NUM'
0020: )
0021: BEGIN
0022:     ALTER TABLE dbo.SD2_ITENS ADD D2_ITEM_NUM CHAR(4) NULL;
0023: END
0024: GO
0025:
0026: -- 1) C6_ITEM: numeração sequencial por pedido (1..n)
0027: IF EXISTS (
0028:     SELECT 1 FROM sys.columns
0029:     WHERE object_id = OBJECT_ID('dbo.SC6_ITENS') AND name = 'C6_ITEM'
0030: )
0031: BEGIN
0032:     EXEC('
0033:     ;WITH ItensOrdenados AS (
0034:         SELECT
0035:             ID,
0036:             C6_PEDIDO_ID,
0037:             ROW_NUMBER() OVER (PARTITION BY C6_PEDIDO_ID ORDER BY ID ASC) AS rn
0038:         FROM dbo.SC6_ITENS
0039:         WHERE C6_ITEM IS NULL
0040:     )
0041:     UPDATE i
0042:     SET C6_ITEM = rn
0043:     FROM ItensOrdenados i;
0044:     ');
0045: END
0046:
0047: -- 2) D2_ITEM_NUM: 0001, 0002... por NF
0048: IF EXISTS (
0049:     SELECT 1 FROM sys.columns
0050:     WHERE object_id = OBJECT_ID('dbo.SD2_ITENS') AND name = 'D2_ITEM_NUM'
0051: )
0052: BEGIN
0053:     EXEC('
0054:     ;WITH ItensNF AS (
0055:         SELECT
0056:             ID,
0057:             D2_NF_ID,
0058:             ROW_NUMBER() OVER (PARTITION BY D2_NF_ID ORDER BY ID ASC) AS rn
0059:         FROM dbo.SD2_ITENS
0060:         WHERE D2_ITEM_NUM IS NULL
0061:     )
0062:     UPDATE d
0063:     SET D2_ITEM_NUM = RIGHT(''0000'' + CAST(rn AS VARCHAR(4)), 4)
0064:     FROM ItensNF d;
0065:     ');
0066: END
```

==========
ARQUIVO: sql_step4_audit.sql
==========

```
0001: -- Bloco 4.3 - Auditoria (logs) via trigger
0002: -- Requer tabela dbo.LOGS existente
0003:
0004: -- SC5_PEDIDOS
0005: IF OBJECT_ID('dbo.TR_LOG_SC5', 'TR') IS NULL
0006: BEGIN
0007: EXEC('
0008: CREATE TRIGGER dbo.TR_LOG_SC5
0009: ON dbo.SC5_PEDIDOS
0010: AFTER INSERT, UPDATE, DELETE
0011: AS
0012: BEGIN
0013:     SET NOCOUNT ON;
0014:     DECLARE @acao NVARCHAR(20) =
0015:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0016:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0017:              ELSE ''DELETE'' END;
0018:
0019:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0020:     SELECT
0021:       COALESCE(i.C5_USUARIO, d.C5_USUARIO, SYSTEM_USER),
0022:       ''SC5_PEDIDOS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0023:       SYSDATETIME()
0024:     FROM inserted i
0025:     FULL JOIN deleted d ON i.ID = d.ID;
0026: END
0027: ');
0028: END
0029:
0030: -- SC6_ITENS
0031: IF OBJECT_ID('dbo.TR_LOG_SC6', 'TR') IS NULL
0032: BEGIN
0033: EXEC('
0034: CREATE TRIGGER dbo.TR_LOG_SC6
0035: ON dbo.SC6_ITENS
0036: AFTER INSERT, UPDATE, DELETE
0037: AS
0038: BEGIN
0039:     SET NOCOUNT ON;
0040:     DECLARE @acao NVARCHAR(20) =
0041:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0042:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0043:              ELSE ''DELETE'' END;
0044:
0045:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0046:     SELECT
0047:       COALESCE(i.C6_USUARIO_ALT, i.C6_USUARIO_DEL, d.C6_USUARIO_ALT, d.C6_USUARIO_DEL, SYSTEM_
USER),
0048:       ''SC6_ITENS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0049:       SYSDATETIME()
0050:     FROM inserted i
0051:     FULL JOIN deleted d ON i.ID = d.ID;
```

```
0052: END
0053: ');
0054: END
0055:
0056: -- SF2_NOTAS
0057: IF OBJECT_ID('dbo.TR_LOG_SF2', 'TR') IS NULL
0058: BEGIN
0059: EXEC('
0060: CREATE TRIGGER dbo.TR_LOG_SF2
0061: ON dbo.SF2_NOTAS
0062: AFTER INSERT, UPDATE, DELETE
0063: AS
0064: BEGIN
0065:     SET NOCOUNT ON;
0066:     DECLARE @acao NVARCHAR(20) =
0067:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0068:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0069:              ELSE ''DELETE'' END;
0070:
0071:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0072:     SELECT
0073:       COALESCE(i.F2_USUARIO_CANCEL, d.F2_USUARIO_CANCEL, SYSTEM_USER),
0074:       ''SF2_NOTAS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0075:       SYSDATETIME()
0076:     FROM inserted i
0077:     FULL JOIN deleted d ON i.ID = d.ID;
0078: END
0079: ');
0080: END
0081:
0082: -- SD2_ITENS
0083: IF OBJECT_ID('dbo.TR_LOG_SD2', 'TR') IS NULL
0084: BEGIN
0085: EXEC('
0086: CREATE TRIGGER dbo.TR_LOG_SD2
0087: ON dbo.SD2_ITENS
0088: AFTER INSERT, UPDATE, DELETE
0089: AS
0090: BEGIN
0091:     SET NOCOUNT ON;
0092:     DECLARE @acao NVARCHAR(20) =
0093:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0094:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0095:              ELSE ''DELETE'' END;
0096:
0097:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0098:     SELECT
0099:       COALESCE(i.D2_USUARIO_CANCEL, d.D2_USUARIO_CANCEL, SYSTEM_USER),
0100:       ''SD2_ITENS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0101:       SYSDATETIME()
0102:     FROM inserted i
0103:     FULL JOIN deleted d ON i.ID = d.ID;
```

```
0104: END
0105: ');
0106: END
0107:
0108: -- SE1_AR
0109: IF OBJECT_ID('dbo.TR_LOG_SE1', 'TR') IS NULL
0110: BEGIN
0111: EXEC('
0112: CREATE TRIGGER dbo.TR_LOG_SE1
0113: ON dbo.SE1_AR
0114: AFTER INSERT, UPDATE, DELETE
0115: AS
0116: BEGIN
0117:     SET NOCOUNT ON;
0118:     DECLARE @acao NVARCHAR(20) =
0119:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0120:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0121:              ELSE ''DELETE'' END;
0122:
0123:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0124:     SELECT
0125:       SYSTEM_USER,
0126:       ''SE1_AR '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0127:       SYSDATETIME()
0128:     FROM inserted i
0129:     FULL JOIN deleted d ON i.ID = d.ID;
0130: END
0131: ');
0132: END
0133:
0134: -- SF1_AP
0135: IF OBJECT_ID('dbo.TR_LOG_SF1', 'TR') IS NULL
0136: BEGIN
0137: EXEC('
0138: CREATE TRIGGER dbo.TR_LOG_SF1
0139: ON dbo.SF1_AP
0140: AFTER INSERT, UPDATE, DELETE
0141: AS
0142: BEGIN
0143:     SET NOCOUNT ON;
0144:     DECLARE @acao NVARCHAR(20) =
0145:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0146:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0147:              ELSE ''DELETE'' END;
0148:
0149:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0150:     SELECT
0151:       SYSTEM_USER,
0152:       ''SF1_AP '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0153:       SYSDATETIME()
0154:     FROM inserted i
0155:     FULL JOIN deleted d ON i.ID = d.ID;
```

```
0156: END
0157: ');
0158: END
```

==========
ARQUIVO: sql_step4_indices.sql
==========

```
0001: -- Bloco 4.1 - Índices e chaves únicas críticas (Protheus-like)
0002: -- Execute no banco ERP_PROTHEUS
0003:
0004: -- NF (SF2): doc/serie por filial
0005: IF NOT EXISTS (
0006:     SELECT 1 FROM sys.indexes
0007:     WHERE name = 'UX_SF2_FILIAL_DOC_SERIE'
0008:       AND object_id = OBJECT_ID('dbo.SF2_NOTAS')
0009: )
0010: BEGIN
0011:     CREATE UNIQUE INDEX UX_SF2_FILIAL_DOC_SERIE
0012:     ON dbo.SF2_NOTAS (F2_FILIAL, F2_DOC, F2_SERIE)
0013:     WHERE D_E_L_E_T_ IS NULL;
0014: END
0015:
0016: -- AR (SE1): num/serie por filial
0017: IF NOT EXISTS (
0018:     SELECT 1 FROM sys.indexes
0019:     WHERE name = 'UX_SE1_FILIAL_NUM_SERIE'
0020:       AND object_id = OBJECT_ID('dbo.SE1_AR')
0021: )
0022: BEGIN
0023:     CREATE UNIQUE INDEX UX_SE1_FILIAL_NUM_SERIE
0024:     ON dbo.SE1_AR (E1_FILIAL, E1_NUM, E1_SERIE)
0025:     WHERE D_E_L_E_T_ IS NULL;
0026: END
0027:
0028: -- AP (SF1): num/serie por filial
0029: IF NOT EXISTS (
0030:     SELECT 1 FROM sys.indexes
0031:     WHERE name = 'UX_SF1_FILIAL_NUM_SERIE'
0032:       AND object_id = OBJECT_ID('dbo.SF1_AP')
0033: )
0034: BEGIN
0035:     CREATE UNIQUE INDEX UX_SF1_FILIAL_NUM_SERIE
0036:     ON dbo.SF1_AP (F1_FILIAL, F1_NUM, F1_SERIE)
0037:     WHERE D_E_L_E_T_ IS NULL;
0038: END
0039:
0040: -- Índices de status/data
0041: IF NOT EXISTS (
0042:     SELECT 1 FROM sys.indexes
0043:     WHERE name = 'IX_SF2_STATUS_EMISSAO'
0044:       AND object_id = OBJECT_ID('dbo.SF2_NOTAS')
0045: )
0046: BEGIN
0047:     CREATE INDEX IX_SF2_STATUS_EMISSAO
```

```
0048:     ON dbo.SF2_NOTAS (F2_STATUS, F2_EMISSAO, F2_FILIAL);
0049: END
0050:
0051: IF NOT EXISTS (
0052:     SELECT 1 FROM sys.indexes
0053:     WHERE name = 'IX_SE1_STATUS_DATA'
0054:       AND object_id = OBJECT_ID('dbo.SE1_AR')
0055: )
0056: BEGIN
0057:     CREATE INDEX IX_SE1_STATUS_DATA
0058:     ON dbo.SE1_AR (E1_STATUS, E1_DATA, E1_FILIAL);
0059: END
0060:
0061: IF NOT EXISTS (
0062:     SELECT 1 FROM sys.indexes
0063:     WHERE name = 'IX_SF1_STATUS_DATA'
0064:       AND object_id = OBJECT_ID('dbo.SF1_AP')
0065: )
0066: BEGIN
0067:     CREATE INDEX IX_SF1_STATUS_DATA
0068:     ON dbo.SF1_AP (F1_STATUS, F1_DATA, F1_FILIAL);
0069: END
```

==========
ARQUIVO: sql_step5_audit_request.sql
==========

```
0001: -- Bloco 5.3 - Auditoria com request_id e path
0002: -- Recria triggers para incluir REQ e PATH em L_ACAO
0003:
0004: -- SC5_PEDIDOS
0005: IF OBJECT_ID('dbo.TR_LOG_SC5', 'TR') IS NOT NULL
0006:     DROP TRIGGER dbo.TR_LOG_SC5;
0007: EXEC('
0008: CREATE TRIGGER dbo.TR_LOG_SC5
0009: ON dbo.SC5_PEDIDOS
0010: AFTER INSERT, UPDATE, DELETE
0011: AS
0012: BEGIN
0013:     SET NOCOUNT ON;
0014:     DECLARE @acao NVARCHAR(20) =
0015:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0016:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0017:              ELSE ''DELETE'' END;
0018:
0019:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0020:     SELECT
0021:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.C5_USUARIO, d.C5_USUA
RIO, SYSTEM_USER),
0022:       ''SC5_PEDIDOS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20))
0023:         + '' REQ='' + COALESCE(CAST(SESSION_CONTEXT(N''app_req_id'') AS NVARCHAR(64)), ''-'')
0024:         + '' PATH='' + COALESCE(CAST(SESSION_CONTEXT(N''app_path'') AS NVARCHAR(200)), ''-''),
0025:       SYSDATETIME()
0026:     FROM inserted i
```

```
0027:     FULL JOIN deleted d ON i.ID = d.ID;
0028: END
0029: ');
0030:
0031: -- SC6_ITENS
0032: IF OBJECT_ID('dbo.TR_LOG_SC6', 'TR') IS NOT NULL
0033:     DROP TRIGGER dbo.TR_LOG_SC6;
0034: EXEC('
0035: CREATE TRIGGER dbo.TR_LOG_SC6
0036: ON dbo.SC6_ITENS
0037: AFTER INSERT, UPDATE, DELETE
0038: AS
0039: BEGIN
0040:     SET NOCOUNT ON;
0041:     DECLARE @acao NVARCHAR(20) =
0042:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0043:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0044:              ELSE ''DELETE'' END;
0045:
0046:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0047:     SELECT
0048:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.C6_USUARIO_ALT, i.C6_
USUARIO_DEL, d.C6_USUARIO_ALT, d.C6_USUARIO_DEL, SYSTEM_USER),
0049:       ''SC6_ITENS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20))
0050:         + '' REQ='' + COALESCE(CAST(SESSION_CONTEXT(N''app_req_id'') AS NVARCHAR(64)), ''-'')
0051:         + '' PATH='' + COALESCE(CAST(SESSION_CONTEXT(N''app_path'') AS NVARCHAR(200)), ''-''),
0052:       SYSDATETIME()
0053:     FROM inserted i
0054:     FULL JOIN deleted d ON i.ID = d.ID;
0055: END
0056: ');
0057:
0058: -- SF2_NOTAS
0059: IF OBJECT_ID('dbo.TR_LOG_SF2', 'TR') IS NOT NULL
0060:     DROP TRIGGER dbo.TR_LOG_SF2;
0061: EXEC('
0062: CREATE TRIGGER dbo.TR_LOG_SF2
0063: ON dbo.SF2_NOTAS
0064: AFTER INSERT, UPDATE, DELETE
0065: AS
0066: BEGIN
0067:     SET NOCOUNT ON;
0068:     DECLARE @acao NVARCHAR(20) =
0069:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0070:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0071:              ELSE ''DELETE'' END;
0072:
0073:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0074:     SELECT
0075:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.F2_USUARIO_CANCEL, d.
F2_USUARIO_CANCEL, SYSTEM_USER),
0076:       ''SF2_NOTAS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20))
```

```
0077:         + '' REQ='' + COALESCE(CAST(SESSION_CONTEXT(N''app_req_id'') AS NVARCHAR(64)), ''-'')
0078:         + '' PATH='' + COALESCE(CAST(SESSION_CONTEXT(N''app_path'') AS NVARCHAR(200)), ''-''),
0079:       SYSDATETIME()
0080:     FROM inserted i
0081:     FULL JOIN deleted d ON i.ID = d.ID;
0082: END
0083: ');
0084:
0085: -- SD2_ITENS
0086: IF OBJECT_ID('dbo.TR_LOG_SD2', 'TR') IS NOT NULL
0087:     DROP TRIGGER dbo.TR_LOG_SD2;
0088: EXEC('
0089: CREATE TRIGGER dbo.TR_LOG_SD2
0090: ON dbo.SD2_ITENS
0091: AFTER INSERT, UPDATE, DELETE
0092: AS
0093: BEGIN
0094:     SET NOCOUNT ON;
0095:     DECLARE @acao NVARCHAR(20) =
0096:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0097:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0098:              ELSE ''DELETE'' END;
0099:
0100:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0101:     SELECT
0102:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.D2_USUARIO_CANCEL, d.
D2_USUARIO_CANCEL, SYSTEM_USER),
0103:       ''SD2_ITENS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20))
0104:         + '' REQ='' + COALESCE(CAST(SESSION_CONTEXT(N''app_req_id'') AS NVARCHAR(64)), ''-'')
0105:         + '' PATH='' + COALESCE(CAST(SESSION_CONTEXT(N''app_path'') AS NVARCHAR(200)), ''-''),
0106:       SYSDATETIME()
0107:     FROM inserted i
0108:     FULL JOIN deleted d ON i.ID = d.ID;
0109: END
0110: ');
0111:
0112: -- SE1_AR
0113: IF OBJECT_ID('dbo.TR_LOG_SE1', 'TR') IS NOT NULL
0114:     DROP TRIGGER dbo.TR_LOG_SE1;
0115: EXEC('
0116: CREATE TRIGGER dbo.TR_LOG_SE1
0117: ON dbo.SE1_AR
0118: AFTER INSERT, UPDATE, DELETE
0119: AS
0120: BEGIN
0121:     SET NOCOUNT ON;
0122:     DECLARE @acao NVARCHAR(20) =
0123:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0124:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0125:              ELSE ''DELETE'' END;
0126:
0127:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
```

```
0128:     SELECT
0129:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), SYSTEM_USER),
0130:       ''SE1_AR '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20))
0131:         + '' REQ='' + COALESCE(CAST(SESSION_CONTEXT(N''app_req_id'') AS NVARCHAR(64)), ''-'')
0132:         + '' PATH='' + COALESCE(CAST(SESSION_CONTEXT(N''app_path'') AS NVARCHAR(200)), ''-''),
0133:       SYSDATETIME()
0134:     FROM inserted i
0135:     FULL JOIN deleted d ON i.ID = d.ID;
0136: END
0137: ');
0138:
0139: -- SF1_AP
0140: IF OBJECT_ID('dbo.TR_LOG_SF1', 'TR') IS NOT NULL
0141:     DROP TRIGGER dbo.TR_LOG_SF1;
0142: EXEC('
0143: CREATE TRIGGER dbo.TR_LOG_SF1
0144: ON dbo.SF1_AP
0145: AFTER INSERT, UPDATE, DELETE
0146: AS
0147: BEGIN
0148:     SET NOCOUNT ON;
0149:     DECLARE @acao NVARCHAR(20) =
0150:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0151:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0152:              ELSE ''DELETE'' END;
0153:
0154:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0155:     SELECT
0156:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), SYSTEM_USER),
0157:       ''SF1_AP '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20))
0158:         + '' REQ='' + COALESCE(CAST(SESSION_CONTEXT(N''app_req_id'') AS NVARCHAR(64)), ''-'')
0159:         + '' PATH='' + COALESCE(CAST(SESSION_CONTEXT(N''app_path'') AS NVARCHAR(200)), ''-''),
0160:       SYSDATETIME()
0161:     FROM inserted i
0162:     FULL JOIN deleted d ON i.ID = d.ID;
0163: END
0164: ');
```

==========
ARQUIVO: sql_step5_audit_session.sql
==========

```
0001: -- Bloco 5.2 - Auditoria com usuário da sessão (SESSION_CONTEXT)
0002: -- Recria triggers para usar app_user
0003:
0004: DECLARE @sql NVARCHAR(MAX);
0005:
0006: -- SC5_PEDIDOS
0007: IF OBJECT_ID('dbo.TR_LOG_SC5', 'TR') IS NOT NULL
0008:     DROP TRIGGER dbo.TR_LOG_SC5;
0009: EXEC('
0010: CREATE TRIGGER dbo.TR_LOG_SC5
0011: ON dbo.SC5_PEDIDOS
0012: AFTER INSERT, UPDATE, DELETE
```

```
0013: AS
0014: BEGIN
0015:     SET NOCOUNT ON;
0016:     DECLARE @acao NVARCHAR(20) =
0017:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0018:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0019:              ELSE ''DELETE'' END;
0020:
0021:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0022:     SELECT
0023:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.C5_USUARIO, d.C5_USUA
RIO, SYSTEM_USER),
0024:       ''SC5_PEDIDOS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0025:       SYSDATETIME()
0026:     FROM inserted i
0027:     FULL JOIN deleted d ON i.ID = d.ID;
0028: END
0029: ');
0030:
0031: -- SC6_ITENS
0032: IF OBJECT_ID('dbo.TR_LOG_SC6', 'TR') IS NOT NULL
0033:     DROP TRIGGER dbo.TR_LOG_SC6;
0034: EXEC('
0035: CREATE TRIGGER dbo.TR_LOG_SC6
0036: ON dbo.SC6_ITENS
0037: AFTER INSERT, UPDATE, DELETE
0038: AS
0039: BEGIN
0040:     SET NOCOUNT ON;
0041:     DECLARE @acao NVARCHAR(20) =
0042:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0043:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0044:              ELSE ''DELETE'' END;
0045:
0046:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0047:     SELECT
0048:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.C6_USUARIO_ALT, i.C6_
USUARIO_DEL, d.C6_USUARIO_ALT, d.C6_USUARIO_DEL, SYSTEM_USER),
0049:       ''SC6_ITENS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0050:       SYSDATETIME()
0051:     FROM inserted i
0052:     FULL JOIN deleted d ON i.ID = d.ID;
0053: END
0054: ');
0055:
0056: -- SF2_NOTAS
0057: IF OBJECT_ID('dbo.TR_LOG_SF2', 'TR') IS NOT NULL
0058:     DROP TRIGGER dbo.TR_LOG_SF2;
0059: EXEC('
0060: CREATE TRIGGER dbo.TR_LOG_SF2
0061: ON dbo.SF2_NOTAS
0062: AFTER INSERT, UPDATE, DELETE
```

```
0063: AS
0064: BEGIN
0065:     SET NOCOUNT ON;
0066:     DECLARE @acao NVARCHAR(20) =
0067:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0068:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0069:              ELSE ''DELETE'' END;
0070:
0071:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0072:     SELECT
0073:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.F2_USUARIO_CANCEL, d.
F2_USUARIO_CANCEL, SYSTEM_USER),
0074:       ''SF2_NOTAS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0075:       SYSDATETIME()
0076:     FROM inserted i
0077:     FULL JOIN deleted d ON i.ID = d.ID;
0078: END
0079: ');
0080:
0081: -- SD2_ITENS
0082: IF OBJECT_ID('dbo.TR_LOG_SD2', 'TR') IS NOT NULL
0083:     DROP TRIGGER dbo.TR_LOG_SD2;
0084: EXEC('
0085: CREATE TRIGGER dbo.TR_LOG_SD2
0086: ON dbo.SD2_ITENS
0087: AFTER INSERT, UPDATE, DELETE
0088: AS
0089: BEGIN
0090:     SET NOCOUNT ON;
0091:     DECLARE @acao NVARCHAR(20) =
0092:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0093:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0094:              ELSE ''DELETE'' END;
0095:
0096:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0097:     SELECT
0098:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), i.D2_USUARIO_CANCEL, d.
D2_USUARIO_CANCEL, SYSTEM_USER),
0099:       ''SD2_ITENS '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0100:       SYSDATETIME()
0101:     FROM inserted i
0102:     FULL JOIN deleted d ON i.ID = d.ID;
0103: END
0104: ');
0105:
0106: -- SE1_AR
0107: IF OBJECT_ID('dbo.TR_LOG_SE1', 'TR') IS NOT NULL
0108:     DROP TRIGGER dbo.TR_LOG_SE1;
0109: EXEC('
0110: CREATE TRIGGER dbo.TR_LOG_SE1
0111: ON dbo.SE1_AR
0112: AFTER INSERT, UPDATE, DELETE
```

```
0113: AS
0114: BEGIN
0115:     SET NOCOUNT ON;
0116:     DECLARE @acao NVARCHAR(20) =
0117:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0118:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0119:              ELSE ''DELETE'' END;
0120:
0121:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0122:     SELECT
0123:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), SYSTEM_USER),
0124:       ''SE1_AR '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0125:       SYSDATETIME()
0126:     FROM inserted i
0127:     FULL JOIN deleted d ON i.ID = d.ID;
0128: END
0129: ');
0130:
0131: -- SF1_AP
0132: IF OBJECT_ID('dbo.TR_LOG_SF1', 'TR') IS NOT NULL
0133:     DROP TRIGGER dbo.TR_LOG_SF1;
0134: EXEC('
0135: CREATE TRIGGER dbo.TR_LOG_SF1
0136: ON dbo.SF1_AP
0137: AFTER INSERT, UPDATE, DELETE
0138: AS
0139: BEGIN
0140:     SET NOCOUNT ON;
0141:     DECLARE @acao NVARCHAR(20) =
0142:         CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1 FROM deleted) THEN ''UPDA
TE''
0143:              WHEN EXISTS(SELECT 1 FROM inserted) THEN ''INSERT''
0144:              ELSE ''DELETE'' END;
0145:
0146:     INSERT INTO dbo.LOGS (L_USUARIO, L_ACAO, L_DATA)
0147:     SELECT
0148:       COALESCE(CONVERT(NVARCHAR(100), SESSION_CONTEXT(N''app_user'')), SYSTEM_USER),
0149:       ''SF1_AP '' + @acao + '' ID='' + CAST(COALESCE(i.ID, d.ID) AS NVARCHAR(20)),
0150:       SYSDATETIME()
0151:     FROM inserted i
0152:     FULL JOIN deleted d ON i.ID = d.ID;
0153: END
0154: ');
```

==========
ARQUIVO: sql_step5_perf.sql
==========

```
0001: -- Bloco 5.1 - Performance (índices adicionais)
0002:
0003: -- SC6: pedido + produto
0004: IF NOT EXISTS (
0005:     SELECT 1 FROM sys.indexes
0006:     WHERE name = 'IX_SC6_PEDIDO_PRODUTO'
```

```
0007:       AND object_id = OBJECT_ID('dbo.SC6_ITENS')
0008: )
0009: BEGIN
0010:     CREATE INDEX IX_SC6_PEDIDO_PRODUTO
0011:     ON dbo.SC6_ITENS (C6_PEDIDO_ID, C6_PRODUTO);
0012: END
0013:
0014: -- SD2: NF + produto
0015: IF NOT EXISTS (
0016:     SELECT 1 FROM sys.indexes
0017:     WHERE name = 'IX_SD2_NF_PRODUTO'
0018:       AND object_id = OBJECT_ID('dbo.SD2_ITENS')
0019: )
0020: BEGIN
0021:     CREATE INDEX IX_SD2_NF_PRODUTO
0022:     ON dbo.SD2_ITENS (D2_NF_ID, D2_PRODUTO);
0023: END
0024:
0025: -- SC9: pedido + item
0026: IF NOT EXISTS (
0027:     SELECT 1 FROM sys.indexes
0028:     WHERE name = 'IX_SC9_PEDIDO_ITEM'
0029:       AND object_id = OBJECT_ID('dbo.SC9_LIBERACAO')
0030: )
0031: BEGIN
0032:     CREATE INDEX IX_SC9_PEDIDO_ITEM
0033:     ON dbo.SC9_LIBERACAO (C9_PEDIDO_ID, C9_ITEM);
0034: END
0035:
0036: -- SD3: produto + data
0037: IF NOT EXISTS (
0038:     SELECT 1 FROM sys.indexes
0039:     WHERE name = 'IX_SD3_PRODUTO_DATA'
0040:       AND object_id = OBJECT_ID('dbo.SD3_MOV')
0041: )
0042: BEGIN
0043:     CREATE INDEX IX_SD3_PRODUTO_DATA
0044:     ON dbo.SD3_MOV (D3_PRODUTO, D3_DATA, D3_FILIAL);
0045: END
0046:
0047: -- LOGS: usuario + data
0048: IF NOT EXISTS (
0049:     SELECT 1 FROM sys.indexes
0050:     WHERE name = 'IX_LOGS_USUARIO_DATA'
0051:       AND object_id = OBJECT_ID('dbo.LOGS')
0052: )
0053: BEGIN
0054:     CREATE INDEX IX_LOGS_USUARIO_DATA
0055:     ON dbo.LOGS (L_USUARIO, L_DATA);
0056: END
```

==========
ARQUIVO: sql_step6_fix.sql
==========

```
0001: -- Bloco 6.3 - Correções automáticas (consistência)
0002: -- Execute no ERP_PROTHEUS. Script conservador (não mexe em AR baixado).
0003:
0004: SET NOCOUNT ON;
0005:
0006: -- 1) Preencher C6_ITEM faltante (por pedido) - dinâmico
0007: EXEC('
0008: ;WITH ItensOrdenados AS (
0009:     SELECT ID, C6_PEDIDO_ID,
0010:            ROW_NUMBER() OVER (PARTITION BY C6_PEDIDO_ID ORDER BY ID ASC) AS rn
0011:     FROM dbo.SC6_ITENS
0012:     WHERE C6_ITEM IS NULL
0013: )
0014: UPDATE i SET C6_ITEM = rn
0015: FROM ItensOrdenados i;
0016: ');
0017:
0018: -- 2) Preencher D2_ITEM_NUM faltante (por NF) - dinâmico
0019: EXEC('
0020: ;WITH ItensNF AS (
0021:     SELECT ID, D2_NF_ID,
0022:            ROW_NUMBER() OVER (PARTITION BY D2_NF_ID ORDER BY ID ASC) AS rn
0023:     FROM dbo.SD2_ITENS
0024:     WHERE D2_ITEM_NUM IS NULL
0025: )
0026: UPDATE d
0027: SET D2_ITEM_NUM = RIGHT(''0000'' + CAST(rn AS VARCHAR(4)), 4)
0028: FROM ItensNF d;
0029: ');
0030:
0031: -- 3) Se NF cancelada, marca SD2 como cancelado (somente ativos)
0032: UPDATE d
0033: SET d.D2_STATUS='CANCELADO',
0034:     d.D2_DATA_CANCEL=COALESCE(d.D2_DATA_CANCEL, SYSDATETIME()),
0035:     d.D2_MOTIVO_CANCEL=COALESCE(d.D2_MOTIVO_CANCEL, 'AUTO: NF cancelada'),
0036:     d.D_E_L_E_T_='*',
0037:     d.R_E_C_D_E_L_=COALESCE(d.R_E_C_D_E_L_, d.R_E_C_N_O_)
0038: FROM dbo.SD2_ITENS d
0039: JOIN dbo.SF2_NOTAS f2 ON f2.ID = d.D2_NF_ID
0040: WHERE f2.F2_STATUS='CANCELADA'
0041:   AND (d.D2_STATUS IS NULL OR d.D2_STATUS <> 'CANCELADO');
0042:
0043: -- 4) Se NF cancelada, cancela AR aberto vinculado (não mexe em BAIXADO)
0044: UPDATE e1
0045: SET e1.E1_STATUS='CANCELADO',
0046:     e1.D_E_L_E_T_='*',
0047:     e1.R_E_C_D_E_L_=COALESCE(e1.R_E_C_D_E_L_, e1.R_E_C_N_O_)
0048: FROM dbo.SE1_AR e1
0049: JOIN dbo.SF2_NOTAS f2 ON f2.ID = TRY_CAST(e1.E1_REF AS INT)
0050: WHERE f2.F2_STATUS='CANCELADA'
0051:   AND e1.E1_STATUS='ABERTO';
0052:
0053: -- 5) Se pedido cancelado, cancela liberações SC9
0054: UPDATE s
```

```
0055: SET s.C9_STATUS='CANCELADO',
0056:     s.C9_DATA_CANCEL=COALESCE(s.C9_DATA_CANCEL, SYSDATETIME()),
0057:     s.D_E_L_E_T_='*',
0058:     s.R_E_C_D_E_L_=COALESCE(s.R_E_C_D_E_L_, s.R_E_C_N_O_)
0059: FROM dbo.SC9_LIBERACAO s
0060: JOIN dbo.SC5_PEDIDOS p ON p.ID = s.C9_PEDIDO_ID
0061: WHERE p.C5_STATUS='CANCELADO'
0062:   AND (s.C9_STATUS IS NULL OR s.C9_STATUS <> 'CANCELADO');
```

==========
ARQUIVO: sql_step7_fechamento.sql
==========

```
0001: -- Bloco 7.1 - Fechamento mensal por filial
0002:
0003: IF OBJECT_ID('dbo.SF7_FECHAMENTO', 'U') IS NULL
0004: BEGIN
0005:     CREATE TABLE dbo.SF7_FECHAMENTO (
0006:         ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0007:         F7_FILIAL NVARCHAR(10) NOT NULL,
0008:         F7_ANO INT NOT NULL,
0009:         F7_MES INT NOT NULL,
0010:         F7_FECHADO BIT NOT NULL DEFAULT (1),
0011:         F7_USUARIO NVARCHAR(100) NULL,
0012:         F7_DATA DATETIME2(0) NOT NULL DEFAULT (SYSDATETIME()),
0013:         D_E_L_E_T_ CHAR(1) NULL,
0014:         R_E_C_N_O_ INT NULL,
0015:         R_E_C_D_E_L_ INT NULL
0016:     );
0017: END
0018:
0019: IF NOT EXISTS (
0020:     SELECT 1 FROM sys.indexes
0021:     WHERE name = 'UX_SF7_FILIAL_ANO_MES'
0022:       AND object_id = OBJECT_ID('dbo.SF7_FECHAMENTO')
0023: )
0024: BEGIN
0025:     CREATE UNIQUE INDEX UX_SF7_FILIAL_ANO_MES
0026:     ON dbo.SF7_FECHAMENTO (F7_FILIAL, F7_ANO, F7_MES)
0027:     WHERE D_E_L_E_T_ IS NULL;
0028: END
```

==========
ARQUIVO: sql_step7_fiscal_extra.sql
==========

```
0001: -- Bloco 7.2 - Integração fiscal extra (NCM + CFOP por produto)
0002:
0003: IF COL_LENGTH('dbo.SB1_PRODUTOS', 'B1_NCM') IS NULL
0004:     ALTER TABLE dbo.SB1_PRODUTOS ADD B1_NCM NVARCHAR(20) NULL;
0005:
0006: IF OBJECT_ID('dbo.SF8_FISCAL_NCM', 'U') IS NULL
0007: BEGIN
0008:     CREATE TABLE dbo.SF8_FISCAL_NCM (
0009:         ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0010:         F8_FILIAL NVARCHAR(10) NOT NULL,
```

```
0011:         F8_NCM NVARCHAR(20) NOT NULL,
0012:         F8_CFOP NVARCHAR(10) NOT NULL,
0013:         F8_ICMS DECIMAL(9,4) NULL,
0014:         F8_IPI DECIMAL(9,4) NULL,
0015:         F8_ATIVO BIT NOT NULL DEFAULT (1),
0016:         D_E_L_E_T_ CHAR(1) NULL,
0017:         R_E_C_N_O_ INT NULL,
0018:         R_E_C_D_E_L_ INT NULL
0019:     );
0020: END
0021:
0022: IF NOT EXISTS (
0023:     SELECT 1 FROM sys.indexes
0024:     WHERE name = 'UX_SF8_FILIAL_NCM'
0025:       AND object_id = OBJECT_ID('dbo.SF8_FISCAL_NCM')
0026: )
0027: BEGIN
0028:     CREATE UNIQUE INDEX UX_SF8_FILIAL_NCM
0029:     ON dbo.SF8_FISCAL_NCM (F8_FILIAL, F8_NCM)
0030:     WHERE D_E_L_E_T_ IS NULL;
0031: END
```

==========

ARQUIVO: sql_step7_multifilial.sql

==========

```
0001: -- Bloco 7.3 - Multi-filial (SM0_EMPRESA)
0002:
0003: IF COL_LENGTH('dbo.SM0_EMPRESA', 'D_E_L_E_T_') IS NULL
0004:     ALTER TABLE dbo.SM0_EMPRESA ADD D_E_L_E_T_ CHAR(1) NULL;
0005: IF COL_LENGTH('dbo.SM0_EMPRESA', 'R_E_C_N_O_') IS NULL
0006:     ALTER TABLE dbo.SM0_EMPRESA ADD R_E_C_N_O_ INT NULL;
0007: IF COL_LENGTH('dbo.SM0_EMPRESA', 'R_E_C_D_E_L_') IS NULL
0008:     ALTER TABLE dbo.SM0_EMPRESA ADD R_E_C_D_E_L_ INT NULL;
0009:
0010: IF NOT EXISTS (
0011:     SELECT 1 FROM sys.indexes
0012:     WHERE name = 'UX_SM0_FILIAL'
0013:       AND object_id = OBJECT_ID('dbo.SM0_EMPRESA')
0014: )
0015: BEGIN
0016:     CREATE UNIQUE INDEX UX_SM0_FILIAL
0017:     ON dbo.SM0_EMPRESA (M0_FILIAL)
0018:     WHERE D_E_L_E_T_ IS NULL;
0019: END
```

==========

ARQUIVO: sql_step8_fechamento_travas.sql

==========

```
0001: -- Bloco 8 - Travar lançamentos por fechamento (estoque + financeiro)
0002: -- Trigger simples para bloquear SD3 e SE5 em período fechado
0003:
0004: -- Função auxiliar: verifica período fechado
0005: IF OBJECT_ID('dbo.fn_periodo_fechado', 'FN') IS NOT NULL
0006:     DROP FUNCTION dbo.fn_periodo_fechado;
```

```
0007: GO
0008: CREATE FUNCTION dbo.fn_periodo_fechado (@filial NVARCHAR(10), @data DATE)
0009: RETURNS BIT
0010: AS
0011: BEGIN
0012:     DECLARE @fechado BIT = 0;
0013:     SELECT TOP 1 @fechado = F7_FECHADO
0014:     FROM dbo.SF7_FECHAMENTO
0015:     WHERE F7_FILIAL=@filial
0016:       AND F7_ANO=YEAR(@data)
0017:       AND F7_MES=MONTH(@data)
0018:       AND (D_E_L_E_T_ IS NULL OR D_E_L_E_T_ <> '*');
0019:     RETURN ISNULL(@fechado, 0);
0020: END
0021: GO
0022:
0023: -- SD3_MOV: bloqueia inserção em período fechado
0024: IF OBJECT_ID('dbo.TR_SD3_BLOQ_FECH', 'TR') IS NOT NULL
0025:     DROP TRIGGER dbo.TR_SD3_BLOQ_FECH;
0026: GO
0027: CREATE TRIGGER dbo.TR_SD3_BLOQ_FECH
0028: ON dbo.SD3_MOV
0029: AFTER INSERT
0030: AS
0031: BEGIN
0032:     SET NOCOUNT ON;
0033:     IF EXISTS (
0034:         SELECT 1
0035:         FROM inserted i
0036:         WHERE dbo.fn_periodo_fechado(i.D3_FILIAL, CAST(i.D3_DATA AS DATE)) = 1
0037:     )
0038:     BEGIN
0039:         RAISERROR('Período fechado: bloqueado lançamento de estoque (SD3).', 16, 1);
0040:         ROLLBACK TRANSACTION;
0041:     END
0042: END
0043: GO
0044:
0045: -- SE5_MOV: bloqueia inserção em período fechado (caixa)
0046: IF OBJECT_ID('dbo.TR_SE5_BLOQ_FECH', 'TR') IS NOT NULL
0047:     DROP TRIGGER dbo.TR_SE5_BLOQ_FECH;
0048: GO
0049: CREATE TRIGGER dbo.TR_SE5_BLOQ_FECH
0050: ON dbo.SE5_MOV
0051: AFTER INSERT
0052: AS
0053: BEGIN
0054:     SET NOCOUNT ON;
0055:     IF EXISTS (
0056:         SELECT 1
0057:         FROM inserted i
0058:         WHERE dbo.fn_periodo_fechado(i.E5_FILIAL, CAST(i.E5_DATA AS DATE)) = 1
0059:     )
0060:     BEGIN
```

```
0061:         RAISERROR('Período fechado: bloqueado lançamento financeiro (SE5).', 16, 1);
0062:         ROLLBACK TRANSACTION;
0063:     END
0064: END
0065: GO
```

==========
ARQUIVO: sql_step8_fechamento_travas_ext.sql
==========

```
0001: -- Bloco 8 (ext) - Travar lançamentos por fechamento (NF/AR/AP)
0002: USE ERP_PROTHEUS;
0003: GO
0004:
0005: -- SF2_NOTAS: bloqueia INSERT/UPDATE em período fechado
0006: IF OBJECT_ID('dbo.SF2_NOTAS', 'U') IS NOT NULL
0007: BEGIN
0008:     IF OBJECT_ID('dbo.TR_SF2_BLOQ_FECH', 'TR') IS NOT NULL
0009:         DROP TRIGGER dbo.TR_SF2_BLOQ_FECH;
0010:     EXEC('
0011:     CREATE TRIGGER dbo.TR_SF2_BLOQ_FECH
0012:     ON dbo.SF2_NOTAS
0013:     AFTER INSERT, UPDATE
0014:     AS
0015:     BEGIN
0016:         SET NOCOUNT ON;
0017:         IF EXISTS (
0018:             SELECT 1
0019:             FROM inserted i
0020:             WHERE dbo.fn_periodo_fechado(i.F2_FILIAL, CAST(i.F2_EMISSAO AS DATE)) = 1
0021:         )
0022:         BEGIN
0023:             RAISERROR(''Período fechado: bloqueada emissão/alteração de NF (SF2).'',
0024: 16, 1);
0025:             ROLLBACK TRANSACTION;
0026:         END
0027:     END
0028:     ');
0029: END
0030: GO
0031:
0032: -- SD2_ITENS: bloqueia INSERT/UPDATE em período fechado (usa data da NF)
0033: IF OBJECT_ID('dbo.SD2_ITENS', 'U') IS NOT NULL
0034: BEGIN
0035:     IF OBJECT_ID('dbo.TR_SD2_BLOQ_FECH', 'TR') IS NOT NULL
0036:         DROP TRIGGER dbo.TR_SD2_BLOQ_FECH;
0037:     EXEC('
0038:     CREATE TRIGGER dbo.TR_SD2_BLOQ_FECH
0039:     ON dbo.SD2_ITENS
0040:     AFTER INSERT, UPDATE
0041:     AS
0042:     BEGIN
0043:         SET NOCOUNT ON;
0044:         IF EXISTS (
0045:             SELECT 1
```

```
0046:             FROM inserted i
0047:             JOIN dbo.SF2_NOTAS f2 ON f2.ID = i.D2_NF_ID
0048:             WHERE dbo.fn_periodo_fechado(f2.F2_FILIAL, CAST(f2.F2_EMISSAO AS DATE)) = 1
0049:         )
0050:         BEGIN
0051:             RAISERROR(''Período fechado: bloqueada alteração de itens NF (SD2).'', 16, 1);
0052:             ROLLBACK TRANSACTION;
0053:         END
0054:     END
0055:     ');
0056: END
0057: GO
0058:
0059: -- SE1_AR: bloqueia INSERT/UPDATE em período fechado
0060: IF OBJECT_ID('dbo.SE1_AR', 'U') IS NOT NULL
0061: BEGIN
0062:     IF OBJECT_ID('dbo.TR_SE1_BLOQ_FECH', 'TR') IS NOT NULL
0063:         DROP TRIGGER dbo.TR_SE1_BLOQ_FECH;
0064:     EXEC('
0065:     CREATE TRIGGER dbo.TR_SE1_BLOQ_FECH
0066:     ON dbo.SE1_AR
0067:     AFTER INSERT, UPDATE
0068:     AS
0069:     BEGIN
0070:         SET NOCOUNT ON;
0071:         IF EXISTS (
0072:             SELECT 1
0073:             FROM inserted i
0074:             WHERE dbo.fn_periodo_fechado(i.E1_FILIAL, CAST(i.E1_DATA AS DATE)) = 1
0075:         )
0076:         BEGIN
0077:             RAISERROR(''Período fechado: bloqueado lançamento AR (SE1).'', 16, 1);
0078:             ROLLBACK TRANSACTION;
0079:         END
0080:     END
0081:     ');
0082: END
0083: GO
0084:
0085: -- SF1_AP: bloqueia INSERT/UPDATE em período fechado
0086: IF OBJECT_ID('dbo.SF1_AP', 'U') IS NOT NULL
0087: BEGIN
0088:     IF OBJECT_ID('dbo.TR_SF1_BLOQ_FECH', 'TR') IS NOT NULL
0089:         DROP TRIGGER dbo.TR_SF1_BLOQ_FECH;
0090:     EXEC('
0091:     CREATE TRIGGER dbo.TR_SF1_BLOQ_FECH
0092:     ON dbo.SF1_AP
0093:     AFTER INSERT, UPDATE
0094:     AS
0095:     BEGIN
0096:         SET NOCOUNT ON;
0097:         IF EXISTS (
0098:             SELECT 1
0099:             FROM inserted i
```

```
0100:             WHERE dbo.fn_periodo_fechado(i.F1_FILIAL, CAST(i.F1_DATA AS DATE)) = 1
0101:         )
0102:         BEGIN
0103:             RAISERROR(''Período fechado: bloqueado lançamento AP (SF1).'', 16, 1);
0104:             ROLLBACK TRANSACTION;
0105:         END
0106:     END
0107:     ');
0108: END
0109: GO
```

==========

ARQUIVO: sql_step9_tributos.sql

==========

```
0001: -- Bloco 9.1 - Tributos avançados (PIS/COFINS/ICMS ST/DIFAL + CST)
0002: USE ERP_PROTHEUS;
0003: GO
0004:
0005: -- SF4_FISCAL (TES)
0006: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_PIS') IS NULL
0007:     ALTER TABLE dbo.SF4_FISCAL ADD F4_PIS DECIMAL(9,4) NULL;
0008: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_COFINS') IS NULL
0009:     ALTER TABLE dbo.SF4_FISCAL ADD F4_COFINS DECIMAL(9,4) NULL;
0010: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_ICMS_ST') IS NULL
0011:     ALTER TABLE dbo.SF4_FISCAL ADD F4_ICMS_ST DECIMAL(9,4) NULL;
0012: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_DIFAL') IS NULL
0013:     ALTER TABLE dbo.SF4_FISCAL ADD F4_DIFAL DECIMAL(9,4) NULL;
0014: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_CST_ICMS') IS NULL
0015:     ALTER TABLE dbo.SF4_FISCAL ADD F4_CST_ICMS NVARCHAR(3) NULL;
0016: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_CSOSN') IS NULL
0017:     ALTER TABLE dbo.SF4_FISCAL ADD F4_CSOSN NVARCHAR(3) NULL;
0018: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_CST_PIS') IS NULL
0019:     ALTER TABLE dbo.SF4_FISCAL ADD F4_CST_PIS NVARCHAR(3) NULL;
0020: IF COL_LENGTH('dbo.SF4_FISCAL', 'F4_CST_COFINS') IS NULL
0021:     ALTER TABLE dbo.SF4_FISCAL ADD F4_CST_COFINS NVARCHAR(3) NULL;
0022:
0023: -- SF5_FISCAL_REGRAS (override por cliente/produto)
0024: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_PIS') IS NULL
0025:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_PIS DECIMAL(9,4) NULL;
0026: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_COFINS') IS NULL
0027:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_COFINS DECIMAL(9,4) NULL;
0028: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_ICMS_ST') IS NULL
0029:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_ICMS_ST DECIMAL(9,4) NULL;
0030: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_DIFAL') IS NULL
0031:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_DIFAL DECIMAL(9,4) NULL;
0032: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_CST_ICMS') IS NULL
0033:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_CST_ICMS NVARCHAR(3) NULL;
0034: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_CSOSN') IS NULL
0035:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_CSOSN NVARCHAR(3) NULL;
0036: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_CST_PIS') IS NULL
0037:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_CST_PIS NVARCHAR(3) NULL;
0038: IF COL_LENGTH('dbo.SF5_FISCAL_REGRAS', 'F5_CST_COFINS') IS NULL
0039:     ALTER TABLE dbo.SF5_FISCAL_REGRAS ADD F5_CST_COFINS NVARCHAR(3) NULL;
0040:
```

```
0041: -- SF2_NOTAS (totais)
0042: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_PIS') IS NULL
0043:     ALTER TABLE dbo.SF2_NOTAS ADD F2_PIS DECIMAL(18,2) NULL;
0044: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_COFINS') IS NULL
0045:     ALTER TABLE dbo.SF2_NOTAS ADD F2_COFINS DECIMAL(18,2) NULL;
0046: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_ICMS_ST') IS NULL
0047:     ALTER TABLE dbo.SF2_NOTAS ADD F2_ICMS_ST DECIMAL(18,2) NULL;
0048: IF COL_LENGTH('dbo.SF2_NOTAS', 'F2_DIFAL') IS NULL
0049:     ALTER TABLE dbo.SF2_NOTAS ADD F2_DIFAL DECIMAL(18,2) NULL;
0050:
0051: -- SD2_ITENS (por item)
0052: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_PIS') IS NULL
0053:     ALTER TABLE dbo.SD2_ITENS ADD D2_PIS DECIMAL(18,2) NULL;
0054: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_COFINS') IS NULL
0055:     ALTER TABLE dbo.SD2_ITENS ADD D2_COFINS DECIMAL(18,2) NULL;
0056: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_ICMS_ST') IS NULL
0057:     ALTER TABLE dbo.SD2_ITENS ADD D2_ICMS_ST DECIMAL(18,2) NULL;
0058: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_DIFAL') IS NULL
0059:     ALTER TABLE dbo.SD2_ITENS ADD D2_DIFAL DECIMAL(18,2) NULL;
0060: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_CST_ICMS') IS NULL
0061:     ALTER TABLE dbo.SD2_ITENS ADD D2_CST_ICMS NVARCHAR(3) NULL;
0062: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_CSOSN') IS NULL
0063:     ALTER TABLE dbo.SD2_ITENS ADD D2_CSOSN NVARCHAR(3) NULL;
0064: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_CST_PIS') IS NULL
0065:     ALTER TABLE dbo.SD2_ITENS ADD D2_CST_PIS NVARCHAR(3) NULL;
0066: IF COL_LENGTH('dbo.SD2_ITENS', 'D2_CST_COFINS') IS NULL
0067:     ALTER TABLE dbo.SD2_ITENS ADD D2_CST_COFINS NVARCHAR(3) NULL;
```

==========
ARQUIVO: sql_step_fiscal_regras.sql
==========

```
0001: -- Regras fiscais avançadas (SF5_FISCAL_REGRAS)
0002: -- Prioridade: mais específico (cliente+produto) > cliente ou produto > genérico
0003:
0004: IF OBJECT_ID('dbo.SF5_FISCAL_REGRAS', 'U') IS NULL
0005: BEGIN
0006:     CREATE TABLE dbo.SF5_FISCAL_REGRAS (
0007:         ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0008:         F5_FILIAL NVARCHAR(10) NOT NULL,
0009:         F5_TES NVARCHAR(10) NOT NULL,
0010:         F5_CLIENTE_COD NVARCHAR(20) NULL,
0011:         F5_PRODUTO NVARCHAR(50) NULL,
0012:         F5_CFOP NVARCHAR(10) NULL,
0013:         F5_ICMS DECIMAL(9,4) NULL,
0014:         F5_IPI DECIMAL(9,4) NULL,
0015:         F5_GERA_TIT BIT NULL,
0016:         F5_GERA_EST BIT NULL,
0017:         F5_PRIORIDADE INT NOT NULL DEFAULT (0),
0018:         F5_ATIVO BIT NOT NULL DEFAULT (1),
0019:         D_E_L_E_T_ CHAR(1) NULL,
0020:         R_E_C_N_O_ INT NULL,
0021:         R_E_C_D_E_L_ INT NULL
0022:     );
0023: END
```

```
0024:
0025: IF NOT EXISTS (
0026:     SELECT 1 FROM sys.indexes
0027:     WHERE name = 'IX_SF5_FISCAL_REGRAS_FILTRO'
0028:       AND object_id = OBJECT_ID('dbo.SF5_FISCAL_REGRAS')
0029: )
0030: BEGIN
0031:     CREATE INDEX IX_SF5_FISCAL_REGRAS_FILTRO
0032:     ON dbo.SF5_FISCAL_REGRAS (F5_FILIAL, F5_TES, F5_ATIVO, F5_CLIENTE_COD, F5_PRODUTO, F5_PRIO
RIDADE);
0033: END
```

==========
ARQUIVO: tests/conftest.py
==========

```
0001: import sys
0002: from pathlib import Path
0003: import os
0004: from contextlib import contextmanager
0005:
0006: import pyodbc
0007: import pytest
0008:
0009:
0010: ROOT = Path(__file__).resolve().parents[1]
0011: if str(ROOT) not in sys.path:
0012:     sys.path.insert(0, str(ROOT))
0013:
0014: os.environ.setdefault("APP_ENV", "development")
0015: os.environ.setdefault("JWT_SECRET_KEY", "test-only-secret-key")
0016:
0017:
0018: def get_integration_db_conn_str() -> str | None:
0019:     return os.getenv("TEST_DB_CONN_STR") or None
0020:
0021:
0022: @contextmanager
0023: def integration_db_cursor():
0024:     conn_str = get_integration_db_conn_str()
0025:     if not conn_str:
0026:         pytest.skip("TEST_DB_CONN_STR nao configurada; testes de integracao com banco real ign
orados.")
0027:
0028:     try:
0029:         conn = pyodbc.connect(conn_str, autocommit=False, timeout=30)
0030:     except pyodbc.Error as exc:
0031:         pytest.skip(f"Nao foi possivel conectar ao banco de integracao: {exc}")
0032:
0033:     cur = conn.cursor()
0034:     try:
0035:         cur.execute("SET NOCOUNT ON;")
0036:         yield conn, cur
0037:     finally:
0038:         conn.rollback()
```

```
0039:         cur.close()
0040:         conn.close()
```

==========
ARQUIVO: tests/init_integration_db.py
==========

```
0001: from __future__ import annotations
0002:
0003: import os
0004: import re
0005: import time
0006:
0007: import pyodbc
0008:
0009:
0010: def _require_env(name: str) -> str:
0011:     value = os.getenv(name, "").strip()
0012:     if not value:
0013:         raise RuntimeError(f"Environment variable {name} is required")
0014:     return value
0015:
0016:
0017: def _extract_database_name(conn_str: str) -> str:
0018:     match = re.search(r"(?:DATABASE|Initial Catalog)\s*=\s*([^;]+)", conn_str, flags=re.IGNORE
CASE)
0019:     if not match:
0020:         raise RuntimeError("Could not extract database name from TEST_DB_CONN_STR")
0021:     return match.group(1).strip()
0022:
0023:
0024: def _connect_with_retry(conn_str: str, attempts: int = 30, sleep_seconds: int = 2, autocommit:
 bool = False):
0025:     last_error: Exception | None = None
0026:     for _ in range(attempts):
0027:         try:
0028:             return pyodbc.connect(conn_str, timeout=5, autocommit=autocommit)
0029:         except Exception as exc:  # pragma: no cover - infra bootstrap
0030:             last_error = exc
0031:             time.sleep(sleep_seconds)
0032:     raise RuntimeError(f"Could not connect to SQL Server after retries: {last_error}") from la
st_error
0033:
0034:
0035: SCHEMA_STATEMENTS = [
0036:     """
0037:     IF OBJECT_ID('dbo.USUARIOS', 'U') IS NULL
0038:     BEGIN
0039:         CREATE TABLE dbo.USUARIOS (
0040:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0041:             U_LOGIN NVARCHAR(50) NOT NULL,
0042:             U_SENHA_HASH NVARCHAR(255) NOT NULL,
0043:             U_PERFIL NVARCHAR(20) NOT NULL,
0044:             D_E_L_E_T_ CHAR(1) NULL,
0045:             R_E_C_N_O_ INT NULL,
```

```
0046:             R_E_C_D_E_L_ INT NULL
0047:         );
0048:         CREATE UNIQUE INDEX UX_USUARIOS_LOGIN ON dbo.USUARIOS (U_LOGIN);
0049:     END
0050:     """,
0051:     """
0052:     IF OBJECT_ID('dbo.LOGS', 'U') IS NULL
0053:     BEGIN
0054:         CREATE TABLE dbo.LOGS (
0055:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0056:             L_USUARIO NVARCHAR(50) NOT NULL,
0057:             L_ACAO NVARCHAR(1000) NOT NULL,
0058:             L_DATA DATETIME2(0) NOT NULL
0059:         );
0060:     END
0061:     """,
0062:     """
0063:     IF OBJECT_ID('dbo.SM0_EMPRESA', 'U') IS NULL
0064:     BEGIN
0065:         CREATE TABLE dbo.SM0_EMPRESA (
0066:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0067:             M0_FILIAL NVARCHAR(10) NOT NULL,
0068:             M0_SERIE_NF NVARCHAR(10) NULL,
0069:             M0_SERIE_AR NVARCHAR(10) NULL,
0070:             M0_SERIE_AP NVARCHAR(10) NULL,
0071:             D_E_L_E_T_ CHAR(1) NULL,
0072:             R_E_C_N_O_ INT NULL,
0073:             R_E_C_D_E_L_ INT NULL
0074:         );
0075:         CREATE UNIQUE INDEX UX_SM0_FILIAL ON dbo.SM0_EMPRESA (M0_FILIAL) WHERE D_E_L_E_T_ IS N
ULL;
0076:     END
0077:     """,
0078:     """
0079:     IF OBJECT_ID('dbo.SX5_TABELAS', 'U') IS NULL
0080:     BEGIN
0081:         CREATE TABLE dbo.SX5_TABELAS (
0082:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0083:             X5_FILIAL NVARCHAR(10) NOT NULL,
0084:             X5_TABELA NVARCHAR(20) NOT NULL,
0085:             X5_CHAVE NVARCHAR(30) NOT NULL,
0086:             X5_DESCR NVARCHAR(100) NULL,
0087:             X5_ATIVO BIT NOT NULL DEFAULT (1),
0088:             D_E_L_E_T_ CHAR(1) NULL,
0089:             R_E_C_N_O_ INT NULL,
0090:             R_E_C_D_E_L_ INT NULL
0091:         );
0092:         CREATE UNIQUE INDEX UX_SX5_FILIAL_TABELA_CHAVE ON dbo.SX5_TABELAS (X5_FILIAL, X5_TABEL
A, X5_CHAVE);
0093:     END
0094:     """,
0095:     """
0096:     IF OBJECT_ID('dbo.SX6_SEQ', 'U') IS NULL
0097:     BEGIN
```

```
0098:         CREATE TABLE dbo.SX6_SEQ (
0099:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0100:             X6_FILIAL NVARCHAR(10) NOT NULL,
0101:             X6_SERIE NVARCHAR(10) NOT NULL,
0102:             X6_TABELA NVARCHAR(20) NOT NULL,
0103:             X6_SEQ INT NOT NULL
0104:         );
0105:         CREATE UNIQUE INDEX UX_SX6_FILIAL_SERIE_TABELA ON dbo.SX6_SEQ (X6_FILIAL, X6_SERIE, X6
_TABELA);
0106:     END
0107:     """,
0108:     """
0109:     IF OBJECT_ID('dbo.SA1_CLIENTES', 'U') IS NULL
0110:     BEGIN
0111:         CREATE TABLE dbo.SA1_CLIENTES (
0112:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0113:             A1_COD NVARCHAR(20) NOT NULL,
0114:             A1_FILIAL NVARCHAR(10) NOT NULL,
0115:             A1_NOME NVARCHAR(120) NOT NULL,
0116:             A1_DOC NVARCHAR(30) NULL,
0117:             A1_EMAIL NVARCHAR(120) NULL,
0118:             A1_FONE NVARCHAR(30) NULL,
0119:             A1_TABELA_ID INT NULL,
0120:             A1_ATIVO BIT NOT NULL DEFAULT (1),
0121:             D_E_L_E_T_ CHAR(1) NULL,
0122:             R_E_C_N_O_ INT NULL,
0123:             R_E_C_D_E_L_ INT NULL
0124:         );
0125:         CREATE UNIQUE INDEX UX_SA1_FILIAL_COD ON dbo.SA1_CLIENTES (A1_FILIAL, A1_COD);
0126:     END
0127:     """,
0128:     """
0129:     IF OBJECT_ID('dbo.SA2_FORNECEDORES', 'U') IS NULL
0130:     BEGIN
0131:         CREATE TABLE dbo.SA2_FORNECEDORES (
0132:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0133:             A2_COD NVARCHAR(20) NOT NULL,
0134:             A2_FILIAL NVARCHAR(10) NOT NULL,
0135:             A2_NOME NVARCHAR(120) NOT NULL,
0136:             A2_DOC NVARCHAR(30) NULL,
0137:             A2_EMAIL NVARCHAR(120) NULL,
0138:             A2_FONE NVARCHAR(30) NULL,
0139:             A2_ATIVO BIT NOT NULL DEFAULT (1),
0140:             D_E_L_E_T_ CHAR(1) NULL,
0141:             R_E_C_N_O_ INT NULL,
0142:             R_E_C_D_E_L_ INT NULL
0143:         );
0144:         CREATE UNIQUE INDEX UX_SA2_FILIAL_COD ON dbo.SA2_FORNECEDORES (A2_FILIAL, A2_COD);
0145:     END
0146:     """,
0147:     """
0148:     IF OBJECT_ID('dbo.SB1_PRODUTOS', 'U') IS NULL
0149:     BEGIN
0150:         CREATE TABLE dbo.SB1_PRODUTOS (
```

```
0151:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0152:             B1_COD NVARCHAR(50) NOT NULL,
0153:             B1_DESC NVARCHAR(120) NOT NULL,
0154:             B1_PRECO DECIMAL(18,2) NOT NULL DEFAULT (0),
0155:             B1_ESTOQUE INT NOT NULL DEFAULT (0),
0156:             B1_RESERVADO INT NOT NULL DEFAULT (0),
0157:             B1_CM DECIMAL(18,2) NOT NULL DEFAULT (0),
0158:             B1_FILIAL NVARCHAR(10) NOT NULL,
0159:             B1_NCM NVARCHAR(20) NULL,
0160:             D_E_L_E_T_ CHAR(1) NULL,
0161:             R_E_C_N_O_ INT NULL,
0162:             R_E_C_D_E_L_ INT NULL
0163:         );
0164:         CREATE UNIQUE INDEX UX_SB1_FILIAL_COD ON dbo.SB1_PRODUTOS (B1_FILIAL, B1_COD);
0165:     END
0166:     """,
0167:     """
0168:     IF OBJECT_ID('dbo.PR0_TABELA', 'U') IS NULL
0169:     BEGIN
0170:         CREATE TABLE dbo.PR0_TABELA (
0171:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0172:             P0_FILIAL NVARCHAR(10) NOT NULL,
0173:             P0_COD NVARCHAR(20) NOT NULL,
0174:             P0_DESC NVARCHAR(120) NOT NULL,
0175:             P0_ATIVA BIT NOT NULL DEFAULT (1),
0176:             D_E_L_E_T_ CHAR(1) NULL,
0177:             R_E_C_N_O_ INT NULL,
0178:             R_E_C_D_E_L_ INT NULL
0179:         );
0180:         CREATE UNIQUE INDEX UX_PR0_FILIAL_COD ON dbo.PR0_TABELA (P0_FILIAL, P0_COD);
0181:     END
0182:     """,
0183:     """
0184:     IF OBJECT_ID('dbo.PR1_PRECO', 'U') IS NULL
0185:     BEGIN
0186:         CREATE TABLE dbo.PR1_PRECO (
0187:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0188:             P1_FILIAL NVARCHAR(10) NOT NULL,
0189:             P1_TABELA_ID INT NOT NULL,
0190:             P1_PRODUTO NVARCHAR(50) NOT NULL,
0191:             P1_PRECO DECIMAL(18,2) NOT NULL,
0192:             P1_DTINI DATE NOT NULL,
0193:             P1_DTFIM DATE NULL,
0194:             D_E_L_E_T_ CHAR(1) NULL,
0195:             R_E_C_N_O_ INT NULL,
0196:             R_E_C_D_E_L_ INT NULL
0197:         );
0198:         CREATE INDEX IX_PR1_LOOKUP ON dbo.PR1_PRECO (P1_FILIAL, P1_TABELA_ID, P1_PRODUTO, P1_D
TINI);
0199:     END
0200:     """,
0201:     """
0202:     IF OBJECT_ID('dbo.SF4_FISCAL', 'U') IS NULL
0203:     BEGIN
```

```
0204:         CREATE TABLE dbo.SF4_FISCAL (
0205:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0206:             F4_FILIAL NVARCHAR(10) NOT NULL,
0207:             F4_COD NVARCHAR(10) NOT NULL,
0208:             F4_TIPO NVARCHAR(5) NOT NULL,
0209:             F4_CFOP NVARCHAR(10) NOT NULL,
0210:             F4_GERA_TIT BIT NOT NULL DEFAULT (1),
0211:             F4_GERA_EST BIT NOT NULL DEFAULT (1),
0212:             F4_ICMS DECIMAL(18,4) NOT NULL DEFAULT (0),
0213:             F4_IPI DECIMAL(18,4) NOT NULL DEFAULT (0),
0214:             F4_PIS DECIMAL(18,4) NOT NULL DEFAULT (0),
0215:             F4_COFINS DECIMAL(18,4) NOT NULL DEFAULT (0),
0216:             F4_ICMS_ST DECIMAL(18,4) NOT NULL DEFAULT (0),
0217:             F4_DIFAL DECIMAL(18,4) NOT NULL DEFAULT (0),
0218:             F4_CST_ICMS NVARCHAR(10) NULL,
0219:             F4_CSOSN NVARCHAR(10) NULL,
0220:             F4_CST_PIS NVARCHAR(10) NULL,
0221:             F4_CST_COFINS NVARCHAR(10) NULL,
0222:             F4_DESC NVARCHAR(120) NULL,
0223:             F4_ATIVO BIT NOT NULL DEFAULT (1),
0224:             D_E_L_E_T_ CHAR(1) NULL,
0225:             R_E_C_N_O_ INT NULL,
0226:             R_E_C_D_E_L_ INT NULL
0227:         );
0228:         CREATE UNIQUE INDEX UX_SF4_FILIAL_COD ON dbo.SF4_FISCAL (F4_FILIAL, F4_COD);
0229:     END
0230:     """,
0231:     """
0232:     IF OBJECT_ID('dbo.SF5_FISCAL_REGRAS', 'U') IS NULL
0233:     BEGIN
0234:         CREATE TABLE dbo.SF5_FISCAL_REGRAS (
0235:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0236:             F5_FILIAL NVARCHAR(10) NOT NULL,
0237:             F5_TES NVARCHAR(10) NOT NULL,
0238:             F5_CLIENTE_COD NVARCHAR(20) NULL,
0239:             F5_PRODUTO NVARCHAR(50) NULL,
0240:             F5_CFOP NVARCHAR(10) NULL,
0241:             F5_ICMS DECIMAL(18,4) NULL,
0242:             F5_IPI DECIMAL(18,4) NULL,
0243:             F5_PIS DECIMAL(18,4) NULL,
0244:             F5_COFINS DECIMAL(18,4) NULL,
0245:             F5_ICMS_ST DECIMAL(18,4) NULL,
0246:             F5_DIFAL DECIMAL(18,4) NULL,
0247:             F5_CST_ICMS NVARCHAR(10) NULL,
0248:             F5_CSOSN NVARCHAR(10) NULL,
0249:             F5_CST_PIS NVARCHAR(10) NULL,
0250:             F5_CST_COFINS NVARCHAR(10) NULL,
0251:             F5_GERA_TIT BIT NULL,
0252:             F5_GERA_EST BIT NULL,
0253:             F5_PRIORIDADE INT NOT NULL DEFAULT (0),
0254:             F5_ATIVO BIT NOT NULL DEFAULT (1),
0255:             D_E_L_E_T_ CHAR(1) NULL,
0256:             R_E_C_N_O_ INT NULL,
0257:             R_E_C_D_E_L_ INT NULL
```

```
0258:         );
0259:     END
0260:     """,
0261:     """
0262:     IF OBJECT_ID('dbo.SF8_FISCAL_NCM', 'U') IS NULL
0263:     BEGIN
0264:         CREATE TABLE dbo.SF8_FISCAL_NCM (
0265:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0266:             F8_FILIAL NVARCHAR(10) NOT NULL,
0267:             F8_NCM NVARCHAR(20) NOT NULL,
0268:             F8_CFOP NVARCHAR(10) NOT NULL,
0269:             F8_ICMS DECIMAL(18,4) NULL,
0270:             F8_IPI DECIMAL(18,4) NULL,
0271:             F8_ATIVO BIT NOT NULL DEFAULT (1),
0272:             D_E_L_E_T_ CHAR(1) NULL,
0273:             R_E_C_N_O_ INT NULL,
0274:             R_E_C_D_E_L_ INT NULL
0275:         );
0276:     END
0277:     """,
0278:     """
0279:     IF OBJECT_ID('dbo.SC5_PEDIDOS', 'U') IS NULL
0280:     BEGIN
0281:         CREATE TABLE dbo.SC5_PEDIDOS (
0282:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0283:             C5_DATA DATETIME2(0) NOT NULL,
0284:             C5_FILIAL NVARCHAR(10) NOT NULL,
0285:             C5_NUM NVARCHAR(20) NULL,
0286:             C5_VALOR_TOTAL DECIMAL(18,2) NOT NULL DEFAULT (0),
0287:             C5_USUARIO NVARCHAR(50) NOT NULL,
0288:             C5_STATUS NVARCHAR(20) NOT NULL,
0289:             C5_ICMS DECIMAL(18,2) NULL,
0290:             C5_IPI DECIMAL(18,2) NULL,
0291:             C5_TOTAL_BRUTO DECIMAL(18,2) NULL,
0292:             C5_ORIGEM NVARCHAR(20) NULL,
0293:             C5_MOTIVO_CANCEL NVARCHAR(200) NULL,
0294:             C5_DATA_CANCEL DATETIME2(0) NULL,
0295:             C5_USUARIO_CANCEL NVARCHAR(50) NULL,
0296:             D_E_L_E_T_ CHAR(1) NULL,
0297:             R_E_C_N_O_ INT NULL,
0298:             R_E_C_D_E_L_ INT NULL
0299:         );
0300:         CREATE INDEX IX_SC5_FILIAL_DATA_STATUS ON dbo.SC5_PEDIDOS (C5_FILIAL, C5_DATA, C5_STAT
US);
0301:     END
0302:     """,
0303:     """
0304:     IF OBJECT_ID('dbo.SC6_ITENS', 'U') IS NULL
0305:     BEGIN
0306:         CREATE TABLE dbo.SC6_ITENS (
0307:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0308:             C6_PEDIDO_ID INT NOT NULL,
0309:             C6_ITEM INT NULL,
0310:             C6_PRODUTO NVARCHAR(50) NOT NULL,
```

```
0311:             C6_QTD INT NOT NULL,
0312:             C6_VALOR DECIMAL(18,2) NOT NULL,
0313:             C6_PRECO_UNIT DECIMAL(18,2) NULL,
0314:             C6_TOTAL DECIMAL(18,2) NULL,
0315:             D_E_L_E_T_ CHAR(1) NULL,
0316:             R_E_C_N_O_ INT NULL,
0317:             R_E_C_D_E_L_ INT NULL
0318:         );
0319:         CREATE INDEX IX_SC6_PEDIDO ON dbo.SC6_ITENS (C6_PEDIDO_ID, C6_ITEM);
0320:     END
0321:     """,
0322:     """
0323:     IF OBJECT_ID('dbo.SC9_LIBERACAO', 'U') IS NULL
0324:     BEGIN
0325:         CREATE TABLE dbo.SC9_LIBERACAO (
0326:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0327:             C9_PEDIDO_ID INT NOT NULL,
0328:             C9_FILIAL NVARCHAR(10) NOT NULL,
0329:             C9_PRODUTO NVARCHAR(50) NOT NULL,
0330:             C9_QTD INT NOT NULL,
0331:             C9_STATUS NVARCHAR(20) NOT NULL,
0332:             C9_USUARIO NVARCHAR(50) NOT NULL,
0333:             C9_DATA DATETIME2(0) NOT NULL,
0334:             C9_ITEM INT NULL,
0335:             C9_USUARIO_CANCEL NVARCHAR(50) NULL,
0336:             C9_DATA_CANCEL DATETIME2(0) NULL,
0337:             D_E_L_E_T_ CHAR(1) NULL,
0338:             R_E_C_N_O_ INT NULL,
0339:             R_E_C_D_E_L_ INT NULL
0340:         );
0341:         CREATE INDEX IX_SC9_PEDIDO_STATUS ON dbo.SC9_LIBERACAO (C9_PEDIDO_ID, C9_STATUS, C9_IT
EM);
0342:     END
0343:     """,
0344:     """
0345:     IF OBJECT_ID('dbo.SF2_NOTAS', 'U') IS NULL
0346:     BEGIN
0347:         CREATE TABLE dbo.SF2_NOTAS (
0348:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0349:             F2_DOC NVARCHAR(20) NOT NULL,
0350:             F2_SERIE NVARCHAR(10) NOT NULL,
0351:             F2_EMISSAO DATETIME2(0) NOT NULL,
0352:             F2_FILIAL NVARCHAR(10) NOT NULL,
0353:             F2_CLIENTE NVARCHAR(120) NULL,
0354:             F2_CLIENTE_COD NVARCHAR(20) NULL,
0355:             F2_VALOR DECIMAL(18,2) NOT NULL,
0356:             F2_ICMS DECIMAL(18,2) NULL,
0357:             F2_IPI DECIMAL(18,2) NULL,
0358:             F2_PIS DECIMAL(18,2) NULL,
0359:             F2_COFINS DECIMAL(18,2) NULL,
0360:             F2_ICMS_ST DECIMAL(18,2) NULL,
0361:             F2_DIFAL DECIMAL(18,2) NULL,
0362:             F2_TOTAL_BRUTO DECIMAL(18,2) NULL,
0363:             F2_STATUS NVARCHAR(20) NOT NULL,
```

```
0364:             F2_PEDIDO_ID INT NULL,
0365:             F2_TES NVARCHAR(10) NULL,
0366:             F2_CFOP NVARCHAR(10) NULL,
0367:             F2_ORIGEM NVARCHAR(20) NULL,
0368:             F2_DATA_CANCEL DATETIME2(0) NULL,
0369:             F2_USUARIO_CANCEL NVARCHAR(50) NULL,
0370:             F2_MOTIVO_CANCEL NVARCHAR(200) NULL,
0371:             D_E_L_E_T_ CHAR(1) NULL,
0372:             R_E_C_N_O_ INT NULL,
0373:             R_E_C_D_E_L_ INT NULL
0374:         );
0375:         CREATE UNIQUE INDEX UX_SF2_FILIAL_DOC_SERIE ON dbo.SF2_NOTAS (F2_FILIAL, F2_DOC, F2_SE
RIE);
0376:     END
0377:     """,
0378:     """
0379:     IF OBJECT_ID('dbo.SD2_ITENS', 'U') IS NULL
0380:     BEGIN
0381:         CREATE TABLE dbo.SD2_ITENS (
0382:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0383:             D2_NF_ID INT NOT NULL,
0384:             D2_DOC NVARCHAR(20) NOT NULL,
0385:             D2_SERIE NVARCHAR(10) NOT NULL,
0386:             D2_ITEM INT NOT NULL,
0387:             D2_FILIAL NVARCHAR(10) NOT NULL,
0388:             D2_PEDIDO_ID INT NULL,
0389:             D2_PRODUTO NVARCHAR(50) NOT NULL,
0390:             D2_QTD INT NOT NULL,
0391:             D2_PRECO_UNIT DECIMAL(18,2) NULL,
0392:             D2_TOTAL DECIMAL(18,2) NOT NULL,
0393:             D2_ICMS DECIMAL(18,2) NULL,
0394:             D2_IPI DECIMAL(18,2) NULL,
0395:             D2_PIS DECIMAL(18,2) NULL,
0396:             D2_COFINS DECIMAL(18,2) NULL,
0397:             D2_ICMS_ST DECIMAL(18,2) NULL,
0398:             D2_DIFAL DECIMAL(18,2) NULL,
0399:             D2_CST_ICMS NVARCHAR(10) NULL,
0400:             D2_CSOSN NVARCHAR(10) NULL,
0401:             D2_CST_PIS NVARCHAR(10) NULL,
0402:             D2_CST_COFINS NVARCHAR(10) NULL,
0403:             D2_TES NVARCHAR(10) NULL,
0404:             D2_CFOP NVARCHAR(10) NULL,
0405:             D2_ITEM_NUM CHAR(4) NULL,
0406:             D2_STATUS NVARCHAR(20) NULL,
0407:             D2_DATA_CANCEL DATETIME2(0) NULL,
0408:             D2_USUARIO_CANCEL NVARCHAR(50) NULL,
0409:             D2_MOTIVO_CANCEL NVARCHAR(200) NULL,
0410:             D_E_L_E_T_ CHAR(1) NULL,
0411:             R_E_C_N_O_ INT NULL,
0412:             R_E_C_D_E_L_ INT NULL
0413:         );
0414:         CREATE INDEX IX_SD2_NF ON dbo.SD2_ITENS (D2_NF_ID, D2_ITEM);
0415:     END
0416:     """,
```

```
0417:     """
0418:     IF OBJECT_ID('dbo.SE1_AR', 'U') IS NULL
0419:     BEGIN
0420:         CREATE TABLE dbo.SE1_AR (
0421:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0422:             E1_DATA DATETIME2(0) NOT NULL,
0423:             E1_VENC DATE NOT NULL,
0424:             E1_CLIENTE NVARCHAR(120) NULL,
0425:             E1_CLIENTE_COD NVARCHAR(20) NULL,
0426:             E1_VALOR DECIMAL(18,2) NOT NULL,
0427:             E1_STATUS NVARCHAR(20) NOT NULL,
0428:             E1_REF NVARCHAR(30) NULL,
0429:             E1_NUM NVARCHAR(20) NOT NULL,
0430:             E1_SERIE NVARCHAR(10) NOT NULL,
0431:             E1_FILIAL NVARCHAR(10) NOT NULL,
0432:             E1_SE5_ID INT NULL,
0433:             D_E_L_E_T_ CHAR(1) NULL,
0434:             R_E_C_N_O_ INT NULL,
0435:             R_E_C_D_E_L_ INT NULL
0436:         );
0437:         CREATE UNIQUE INDEX UX_SE1_FILIAL_NUM_SERIE ON dbo.SE1_AR (E1_FILIAL, E1_NUM, E1_SERIE
);
0438:     END
0439:     """,
0440:     """
0441:     IF OBJECT_ID('dbo.SF1_AP', 'U') IS NULL
0442:     BEGIN
0443:         CREATE TABLE dbo.SF1_AP (
0444:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0445:             F1_DATA DATETIME2(0) NOT NULL,
0446:             F1_VENC DATE NOT NULL,
0447:             F1_FORN NVARCHAR(120) NULL,
0448:             F1_FORN_COD NVARCHAR(20) NULL,
0449:             F1_VALOR DECIMAL(18,2) NOT NULL,
0450:             F1_STATUS NVARCHAR(20) NOT NULL,
0451:             F1_REF NVARCHAR(50) NULL,
0452:             F1_NUM NVARCHAR(20) NOT NULL,
0453:             F1_SERIE NVARCHAR(10) NOT NULL,
0454:             F1_FILIAL NVARCHAR(10) NOT NULL,
0455:             F1_SE5_ID INT NULL,
0456:             D_E_L_E_T_ CHAR(1) NULL,
0457:             R_E_C_N_O_ INT NULL,
0458:             R_E_C_D_E_L_ INT NULL
0459:         );
0460:         CREATE UNIQUE INDEX UX_SF1_FILIAL_NUM_SERIE ON dbo.SF1_AP (F1_FILIAL, F1_NUM, F1_SERIE
);
0461:     END
0462:     """,
0463:     """
0464:     IF OBJECT_ID('dbo.SE5_CONTAS', 'U') IS NULL
0465:     BEGIN
0466:         CREATE TABLE dbo.SE5_CONTAS (
0467:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0468:             E5_FILIAL NVARCHAR(10) NOT NULL,
```

```
0469:             E5_NOME NVARCHAR(120) NOT NULL,
0470:             E5_TIPO NVARCHAR(20) NOT NULL,
0471:             E5_ATIVA BIT NOT NULL DEFAULT (1),
0472:             D_E_L_E_T_ CHAR(1) NULL,
0473:             R_E_C_N_O_ INT NULL,
0474:             R_E_C_D_E_L_ INT NULL
0475:         );
0476:     END
0477:     """,
0478:     """
0479:     IF OBJECT_ID('dbo.SE5_CATEG', 'U') IS NULL
0480:     BEGIN
0481:         CREATE TABLE dbo.SE5_CATEG (
0482:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0483:             C5_FILIAL NVARCHAR(10) NOT NULL,
0484:             C5_NOME NVARCHAR(120) NOT NULL,
0485:             C5_TIPO NVARCHAR(30) NOT NULL,
0486:             C5_ATIVA BIT NOT NULL DEFAULT (1),
0487:             D_E_L_E_T_ CHAR(1) NULL,
0488:             R_E_C_N_O_ INT NULL,
0489:             R_E_C_D_E_L_ INT NULL
0490:         );
0491:     END
0492:     """,
0493:     """
0494:     IF OBJECT_ID('dbo.SE5_MOV', 'U') IS NULL
0495:     BEGIN
0496:         CREATE TABLE dbo.SE5_MOV (
0497:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0498:             E5_DATA DATETIME2(0) NOT NULL DEFAULT (SYSDATETIME()),
0499:             E5_FILIAL NVARCHAR(10) NOT NULL,
0500:             E5_CONTA_ID INT NOT NULL,
0501:             E5_TIPO NVARCHAR(1) NOT NULL,
0502:             E5_VALOR DECIMAL(18,2) NOT NULL,
0503:             E5_HIST NVARCHAR(255) NULL,
0504:             E5_ORIGEM_TIPO NVARCHAR(20) NULL,
0505:             E5_ORIGEM_ID INT NULL,
0506:             E5_USUARIO NVARCHAR(50) NULL,
0507:             E5_CATEG_ID INT NULL,
0508:             D_E_L_E_T_ CHAR(1) NULL,
0509:             R_E_C_N_O_ INT NULL,
0510:             R_E_C_D_E_L_ INT NULL
0511:         );
0512:         CREATE INDEX IX_SE5_FILIAL_CONTA_DATA ON dbo.SE5_MOV (E5_FILIAL, E5_CONTA_ID, E5_DATA,
 ID);
0513:     END
0514:     """,
0515:     """
0516:     IF OBJECT_ID('dbo.SD3_MOV', 'U') IS NULL
0517:     BEGIN
0518:         CREATE TABLE dbo.SD3_MOV (
0519:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0520:             D3_DATA DATETIME2(0) NOT NULL,
0521:             D3_FILIAL NVARCHAR(10) NOT NULL,
```

```
0522:             D3_PRODUTO NVARCHAR(50) NOT NULL,
0523:             D3_TIPO NVARCHAR(1) NOT NULL,
0524:             D3_QTD INT NOT NULL,
0525:             D3_CUSTO_UNIT DECIMAL(18,2) NOT NULL,
0526:             D3_VALOR DECIMAL(18,2) NOT NULL,
0527:             D3_ORIGEM NVARCHAR(30) NULL,
0528:             D3_REF NVARCHAR(50) NULL,
0529:             D3_USUARIO NVARCHAR(50) NULL,
0530:             D_E_L_E_T_ CHAR(1) NULL,
0531:             R_E_C_N_O_ INT NULL,
0532:             R_E_C_D_E_L_ INT NULL
0533:         );
0534:         CREATE INDEX IX_SD3_FILIAL_PRODUTO_DATA ON dbo.SD3_MOV (D3_FILIAL, D3_PRODUTO, D3_DATA
);
0535:     END
0536:     """,
0537:     """
0538:     IF OBJECT_ID('dbo.SZ0_RESERVA', 'U') IS NULL
0539:     BEGIN
0540:         CREATE TABLE dbo.SZ0_RESERVA (
0541:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0542:             Z0_DATA DATETIME2(0) NOT NULL,
0543:             Z0_FILIAL NVARCHAR(10) NOT NULL,
0544:             Z0_PRODUTO NVARCHAR(50) NOT NULL,
0545:             Z0_QTD INT NOT NULL,
0546:             Z0_USUARIO NVARCHAR(50) NOT NULL,
0547:             Z0_STATUS NVARCHAR(20) NOT NULL,
0548:             Z0_CLIENTE_COD NVARCHAR(20) NULL,
0549:             Z0_TABELA_ID INT NULL,
0550:             Z0_PRECO_UNIT DECIMAL(18,2) NULL,
0551:             Z0_TOTAL DECIMAL(18,2) NULL,
0552:             Z0_PEDIDO_ID INT NULL,
0553:             D_E_L_E_T_ CHAR(1) NULL,
0554:             R_E_C_N_O_ INT NULL,
0555:             R_E_C_D_E_L_ INT NULL
0556:         );
0557:         CREATE INDEX IX_SZ0_FILIAL_STATUS ON dbo.SZ0_RESERVA (Z0_FILIAL, Z0_STATUS);
0558:     END
0559:     """,
0560:     """
0561:     IF OBJECT_ID('dbo.SF7_FECHAMENTO', 'U') IS NULL
0562:     BEGIN
0563:         CREATE TABLE dbo.SF7_FECHAMENTO (
0564:             ID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
0565:             F7_FILIAL NVARCHAR(10) NOT NULL,
0566:             F7_ANO INT NOT NULL,
0567:             F7_MES INT NOT NULL,
0568:             F7_FECHADO BIT NOT NULL DEFAULT (1),
0569:             F7_DATA DATETIME2(0) NOT NULL DEFAULT (SYSDATETIME()),
0570:             F7_USUARIO NVARCHAR(50) NULL,
0571:             D_E_L_E_T_ CHAR(1) NULL,
0572:             R_E_C_N_O_ INT NULL,
0573:             R_E_C_D_E_L_ INT NULL
0574:         );
```

```
0575:         CREATE UNIQUE INDEX UX_SF7_FILIAL_ANO_MES ON dbo.SF7_FECHAMENTO (F7_FILIAL, F7_ANO, F7
_MES) WHERE D_E_L_E_T_ IS NULL;
0576:     END
0577:     """,
0578: ]
0579:
0580:
0581: def _ensure_database(admin_conn_str: str, db_name: str) -> None:
0582:     with _connect_with_retry(admin_conn_str, autocommit=True) as conn:
0583:         cur = conn.cursor()
0584:         cur.execute(
0585:             f"""
0586:             IF DB_ID(N'{db_name}') IS NULL
0587:             BEGIN
0588:                 CREATE DATABASE [{db_name}];
0589:             END
0590:             """
0591:         )
0592:
0593:
0594: def _ensure_schema(test_conn_str: str) -> None:
0595:     with _connect_with_retry(test_conn_str, autocommit=True) as conn:
0596:         cur = conn.cursor()
0597:         cur.execute("SET NOCOUNT ON")
0598:         for statement in SCHEMA_STATEMENTS:
0599:             cur.execute(statement)
0600:
0601:
0602: def main() -> None:
0603:     admin_conn_str = _require_env("TEST_DB_ADMIN_CONN_STR")
0604:     test_conn_str = _require_env("TEST_DB_CONN_STR")
0605:     db_name = _extract_database_name(test_conn_str)
0606:     _ensure_database(admin_conn_str, db_name)
0607:     _ensure_schema(test_conn_str)
0608:     print(f"Integration database ready: {db_name}")
0609:
0610:
0611: if __name__ == "__main__":
0612:     main()
```

==========
ARQUIVO: tests/test_alertas_api.py
==========

```
0001: from contextlib import contextmanager
0002:
0003: from fastapi.testclient import TestClient
0004:
0005: from app.api.deps import get_current_user
0006: from app.api.routers import alertas as alertas_router
0007: from app.main import app
0008:
0009:
0010: @contextmanager
0011: def fake_db_transaction():
```

```
0012:     yield None, object()
0013:
0014:
0015: def test_alertas_retorna_payload_de_negocio(monkeypatch):
0016:     def fake_alertas(_cur, login, perfil, filial):
0017:         assert login == "admin"
0018:         assert perfil == "admin"
0019:         assert filial == "01"
0020:         return {
0021:             "filial": filial,
0022:             "usuario": login,
0023:             "generated_at": "2026-03-23T18:00:00",
0024:             "summary": {"critical": 1, "warning": 1, "info": 0, "total": 2},
0025:             "items": [
0026:                 {
0027:                     "code": "AR_OVERDUE",
0028:                     "severity": "critical",
0029:                     "title": "Recebiveis vencidos",
0030:                     "text": "2 titulo(s) em atraso.",
0031:                     "route": "/financeiro/ar",
0032:                     "count": 2,
0033:                     "action_label": "Abrir recebiveis",
0034:                     "business_context": "financeiro.ar.overdue",
0035:                 },
0036:                 {
0037:                     "code": "RESERVAS_OPEN",
0038:                     "severity": "warning",
0039:                     "title": "Reservas pendentes",
0040:                     "text": "1 reserva aberta.",
0041:                     "route": "/reservas",
0042:                     "count": 1,
0043:                     "action_label": "Abrir reservas",
0044:                     "business_context": "vendas.reservas.abertas",
0045:                 },
0046:             ],
0047:         }
0048:
0049:     monkeypatch.setattr(alertas_router, "db_transaction", fake_db_transaction)
0050:     monkeypatch.setattr(alertas_router, "listar_alertas_uc_tx", fake_alertas)
0051:     app.dependency_overrides[get_current_user] = lambda: {"login": "admin", "perfil": "admin"}
0052:     try:
0053:         client = TestClient(app)
0054:         response = client.get("/alertas", params={"filial": "01"})
0055:     finally:
0056:         app.dependency_overrides.clear()
0057:
0058:     assert response.status_code == 200
0059:     payload = response.json()
0060:     assert payload["summary"]["critical"] == 1
0061:     assert payload["items"][0]["code"] == "AR_OVERDUE"
0062:     assert payload["items"][0]["action_label"] == "Abrir recebiveis"
0063:     assert payload["items"][1]["route"] == "/reservas"
0064:
0065:
```

```
0066: def test_alertas_sem_token_retorna_401():
0067:     client = TestClient(app)
0068:     response = client.get("/alertas")
0069:
0070:     assert response.status_code == 401
0071:     payload = response.json()
0072:     assert payload["code"] == "HTTP_401"
0073:     assert payload["error_type"] == "http"
```

==========
ARQUIVO: tests/test_alertas_uc.py
==========

```
0001: from app.use_cases import alertas_uc
0002:
0003:
0004: def test_alertas_admin_prioriza_setup_periodo_e_financeiro(monkeypatch):
0005:     monkeypatch.setattr(alertas_uc, "contar_ar_aberto_tx", lambda *_: 5)
0006:     monkeypatch.setattr(alertas_uc, "contar_ar_vencido_tx", lambda *_: 2)
0007:     monkeypatch.setattr(alertas_uc, "contar_ar_vence_hoje_tx", lambda *_: 1)
0008:     monkeypatch.setattr(alertas_uc, "contar_ar_vencendo_ate_tx", lambda *_: 3)
0009:     monkeypatch.setattr(alertas_uc, "contar_ap_aberto_tx", lambda *_: 4)
0010:     monkeypatch.setattr(alertas_uc, "contar_ap_vencido_tx", lambda *_: 1)
0011:     monkeypatch.setattr(alertas_uc, "contar_ap_vence_hoje_tx", lambda *_: 0)
0012:     monkeypatch.setattr(alertas_uc, "contar_ap_vencendo_ate_tx", lambda *_: 2)
0013:     monkeypatch.setattr(alertas_uc, "contar_reservas_abertas_tx", lambda *_: 1)
0014:     monkeypatch.setattr(alertas_uc, "contar_pedidos_abertos_sem_itens_tx", lambda *_: 2)
0015:     monkeypatch.setattr(alertas_uc, "contar_pedidos_abertos_com_itens_tx", lambda *_: 3)
0016:     monkeypatch.setattr(alertas_uc, "contar_contas_caixa_ativas_tx", lambda *_: 1)
0017:     monkeypatch.setattr(alertas_uc, "fechamento_periodo_aberto_tx", lambda *_: False)
0018:     monkeypatch.setattr(
0019:         alertas_uc,
0020:         "app_setup_status_tx",
0021:         lambda *_: {"setup_completed": False, "completed_at": None, "completed_by": None},
0022:     )
0023:
0024:     payload = alertas_uc.listar_alertas_uc_tx(object(), login="admin", perfil="admin", filial=
"01")
0025:
0026:     assert payload["summary"]["critical"] == 2
0027:     assert payload["summary"]["warning"] >= 1
0028:     assert payload["items"][0]["code"] == "AR_OVERDUE"
0029:     assert payload["items"][0]["action_label"] == "Abrir recebiveis"
0030:     assert payload["items"][0]["business_context"] == "financeiro.ar.overdue"
0031:     assert payload["items"][1]["code"] == "AP_OVERDUE"
0032:     warning_codes = [item["code"] for item in payload["items"] if item["severity"] == "warning
"]
0033:     assert "SETUP_PENDING" in warning_codes
0034:     assert "PERIODO_FECHADO" in warning_codes
0035:
0036:
0037: def test_alertas_operador_destaca_pedidos_e_reservas(monkeypatch):
0038:     monkeypatch.setattr(alertas_uc, "contar_ar_aberto_tx", lambda *_: 0)
0039:     monkeypatch.setattr(alertas_uc, "contar_ar_vencido_tx", lambda *_: 0)
0040:     monkeypatch.setattr(alertas_uc, "contar_ar_vence_hoje_tx", lambda *_: 0)
```

```
0041:     monkeypatch.setattr(alertas_uc, "contar_ar_vencendo_ate_tx", lambda *_: 0)
0042:     monkeypatch.setattr(alertas_uc, "contar_ap_aberto_tx", lambda *_: 0)
0043:     monkeypatch.setattr(alertas_uc, "contar_ap_vencido_tx", lambda *_: 0)
0044:     monkeypatch.setattr(alertas_uc, "contar_ap_vence_hoje_tx", lambda *_: 0)
0045:     monkeypatch.setattr(alertas_uc, "contar_ap_vencendo_ate_tx", lambda *_: 0)
0046:     monkeypatch.setattr(alertas_uc, "contar_reservas_abertas_tx", lambda *_: 4)
0047:     monkeypatch.setattr(alertas_uc, "contar_pedidos_abertos_sem_itens_tx", lambda *_: 2)
0048:     monkeypatch.setattr(alertas_uc, "contar_pedidos_abertos_com_itens_tx", lambda *_: 3)
0049:     monkeypatch.setattr(alertas_uc, "contar_contas_caixa_ativas_tx", lambda *_: 1)
0050:     monkeypatch.setattr(alertas_uc, "fechamento_periodo_aberto_tx", lambda *_: True)
0051:     monkeypatch.setattr(
0052:         alertas_uc,
0053:         "app_setup_status_tx",
0054:         lambda *_: {"setup_completed": True, "completed_at": "2026-03-23T19:00:00", "completed
_by": "admin"},
0055:     )
0056:
0057:     payload = alertas_uc.listar_alertas_uc_tx(object(), login="operador", perfil="operador", f
ilial="01")
0058:
0059:     codes = [item["code"] for item in payload["items"]]
0060:     assert "PEDIDOS_NO_ITEMS" in codes
0061:     assert "PEDIDOS_OPEN" in codes
0062:     assert "RESERVAS_OPEN" in codes
0063:     pedidos_sem_itens = next(item for item in payload["items"] if item["code"] == "PEDIDOS_NO_
ITEMS")
0064:     assert pedidos_sem_itens["action_label"] == "Revisar pedidos"
0065:     assert pedidos_sem_itens["business_context"] == "vendas.pedidos.sem_itens"
0066:     assert "OPERACAO_ESTAVEL" not in codes
```

==========
ARQUIVO: tests/test_auth_api.py
==========

```
0001: from contextlib import contextmanager
0002:
0003: from fastapi.testclient import TestClient
0004:
0005: from app.api.deps import get_current_user
0006: from app.api.routers import auth as auth_router
0007: from app.api.routers import usuarios as usuarios_router
0008: from app.core.exceptions import AuthzError
0009: from app.core.exceptions import BusinessError
0010: from app.domain.errors import AuthError
0011: from app.main import app
0012:
0013:
0014: @contextmanager
0015: def fake_db_transaction():
0016:     yield None, object()
0017:
0018:
0019: def test_login_invalido_retorna_401(monkeypatch):
0020:     def fake_auth(_cur, _username, _password):
0021:         raise AuthError("Credenciais invalidas")
```

```
0022:
0023:     monkeypatch.setattr(auth_router, "db_transaction", fake_db_transaction)
0024:     monkeypatch.setattr(auth_router, "autenticar_usuario_tx", fake_auth)
0025:
0026:     client = TestClient(app)
0027:     response = client.post("/login", data={"username": "x", "password": "y"})
0028:
0029:     assert response.status_code == 401
0030:     assert response.json()["code"] == "AUTH_ERROR"
0031:
0032:
0033: def test_login_valido_retorna_token(monkeypatch):
0034:     def fake_auth(_cur, _username, _password):
0035:         return {"U_LOGIN": "admin", "U_PERFIL": "admin"}
0036:
0037:     monkeypatch.setattr(auth_router, "db_transaction", fake_db_transaction)
0038:     monkeypatch.setattr(auth_router, "autenticar_usuario_tx", fake_auth)
0039:
0040:     client = TestClient(app)
0041:     response = client.post("/login", data={"username": "admin", "password": "ok"})
0042:
0043:     assert response.status_code == 200
0044:     payload = response.json()
0045:     assert isinstance(payload.get("access_token"), str)
0046:     assert payload.get("token_type") == "bearer"
0047:
0048:
0049: def test_rota_protegida_sem_token_retorna_401():
0050:     client = TestClient(app)
0051:     response = client.post(
0052:         "/usuarios",
0053:         json={"login": "novo", "senha": "123", "perfil": "admin"},
0054:     )
0055:     assert response.status_code == 401
0056:     payload = response.json()
0057:     assert payload["code"] == "HTTP_401"
0058:     assert payload["error_type"] == "http"
0059:     assert isinstance(payload["request_id"], str)
0060:
0061:
0062: def test_bootstrap_desabilitado_retorna_404(monkeypatch):
0063:     monkeypatch.setattr(usuarios_router.settings, "APP_ENV", "production")
0064:     monkeypatch.setattr(usuarios_router.settings, "BOOTSTRAP_ENABLED", False)
0065:
0066:     client = TestClient(app)
0067:     response = client.post("/bootstrap", json={"login": "admin", "senha": "Troque123!"})
0068:
0069:     assert response.status_code == 404
0070:     assert response.json()["detail"] == "Not Found"
0071:     assert response.json()["code"] == "HTTP_404"
0072:
0073:
0074: def test_payload_invalido_retorna_422_padronizado():
0075:     client = TestClient(app)
```

```
0076:     response = client.post(
0077:         "/bootstrap",
0078:         json={"login": "admin"},
0079:     )
0080:
0081:     assert response.status_code == 422
0082:     payload = response.json()
0083:     assert payload["code"] == "VALIDATION_ERROR"
0084:     assert payload["error_type"] == "validation"
0085:     assert payload["detail"].startswith("Payload inválido:")
0086:
0087:
0088: def test_bootstrap_habilitado_sem_usuarios_cria_admin(monkeypatch):
0089:     def fake_bootstrap(_cur, _login, _senha):
0090:         return None
0091:
0092:     monkeypatch.setattr(usuarios_router.settings, "APP_ENV", "setup")
0093:     monkeypatch.setattr(usuarios_router.settings, "BOOTSTRAP_ENABLED", True)
0094:     monkeypatch.setattr(usuarios_router, "db_transaction", fake_db_transaction)
0095:     monkeypatch.setattr(usuarios_router, "criar_usuario_bootstrap_tx", fake_bootstrap)
0096:
0097:     client = TestClient(app)
0098:     response = client.post("/bootstrap", json={"login": "admin", "senha": "Troque123!"})
0099:
0100:     assert response.status_code == 200
0101:     assert response.json()["login"] == "admin"
0102:
0103:
0104: def test_bootstrap_habilitado_mas_bloqueado_por_regra(monkeypatch):
0105:     def fake_bootstrap(_cur, _login, _senha):
0106:         raise BusinessError("Bootstrap bloqueado: já existem usuários.")
0107:
0108:     monkeypatch.setattr(usuarios_router.settings, "APP_ENV", "development")
0109:     monkeypatch.setattr(usuarios_router.settings, "BOOTSTRAP_ENABLED", True)
0110:     monkeypatch.setattr(usuarios_router, "db_transaction", fake_db_transaction)
0111:     monkeypatch.setattr(usuarios_router, "criar_usuario_bootstrap_tx", fake_bootstrap)
0112:
0113:     client = TestClient(app)
0114:     response = client.post("/bootstrap", json={"login": "admin", "senha": "Troque123!"})
0115:
0116:     assert response.status_code == 400
0117:     assert response.json()["code"] == "BUSINESS_ERROR"
0118:
0119:
0120: def test_bootstrap_habilitado_mas_setup_ja_concluido(monkeypatch):
0121:     def fake_bootstrap(_cur, _login, _senha):
0122:         raise BusinessError("Bootstrap bloqueado: setup inicial já foi concluído.")
0123:
0124:     monkeypatch.setattr(usuarios_router.settings, "APP_ENV", "setup")
0125:     monkeypatch.setattr(usuarios_router.settings, "BOOTSTRAP_ENABLED", True)
0126:     monkeypatch.setattr(usuarios_router, "db_transaction", fake_db_transaction)
0127:     monkeypatch.setattr(usuarios_router, "criar_usuario_bootstrap_tx", fake_bootstrap)
0128:
0129:     client = TestClient(app)
```

```
0130:     response = client.post("/bootstrap", json={"login": "admin", "senha": "Troque123!"})
0131:
0132:     assert response.status_code == 400
0133:     assert response.json()["code"] == "BUSINESS_ERROR"
0134:
0135:
0136: def test_setup_status_admin_retorna_estado(monkeypatch):
0137:     def fake_status(_cur, _login):
0138:         return {
0139:             "setup_completed": True,
0140:             "completed_at": "2026-03-21T19:10:00",
0141:             "completed_by": "admin",
0142:         }
0143:
0144:     monkeypatch.setattr(usuarios_router, "db_transaction", fake_db_transaction)
0145:     monkeypatch.setattr(usuarios_router, "obter_setup_status_tx", fake_status)
0146:     app.dependency_overrides[get_current_user] = lambda: {"login": "admin", "perfil": "admin"}
0147:     try:
0148:         client = TestClient(app)
0149:         response = client.get("/setup-status")
0150:     finally:
0151:         app.dependency_overrides.clear()
0152:
0153:     assert response.status_code == 200
0154:     assert response.json()["setup_completed"] is True
0155:     assert response.json()["completed_by"] == "admin"
0156:
0157:
0158: def test_setup_status_usuario_nao_admin_retorna_403(monkeypatch):
0159:     def fake_status(_cur, _login):
0160:         raise AuthzError("Sem permissão. Perfil atual: user")
0161:
0162:     monkeypatch.setattr(usuarios_router, "db_transaction", fake_db_transaction)
0163:     monkeypatch.setattr(usuarios_router, "obter_setup_status_tx", fake_status)
0164:     app.dependency_overrides[get_current_user] = lambda: {"login": "operador", "perfil": "user
"}
0165:     try:
0166:         client = TestClient(app)
0167:         response = client.get("/setup-status")
0168:     finally:
0169:         app.dependency_overrides.clear()
0170:
0171:     assert response.status_code == 403
0172:     assert response.json()["code"] == "AUTHZ_ERROR"
```

==========
ARQUIVO: tests/test_business_flows_api.py
==========

```
0001: from contextlib import contextmanager
0002:
0003: from fastapi.testclient import TestClient
0004:
0005: from app.api.deps import get_current_user
0006: from app.api.routers import ap_lancamento as ap_router
```

```
0007: from app.api.routers import financeiro as financeiro_router
0008: from app.api.routers import reservas as reservas_router
0009: from app.api.routers import vendas as vendas_router
0010: from app.core.exceptions import BusinessError
0011: from app.main import app
0012:
0013:
0014: @contextmanager
0015: def fake_db_transaction():
0016:     yield None, object()
0017:
0018:
0019: def _install_admin_override():
0020:     app.dependency_overrides[get_current_user] = lambda: {"login": "admin", "perfil": "admin"}
0021:
0022:
0023: def _clear_overrides():
0024:     app.dependency_overrides.clear()
0025:
0026:
0027: def _wire_fake_business_flow(monkeypatch):
0028:     state = {
0029:         "pedido_seq": 100,
0030:         "item_seq": 1000,
0031:         "nf_seq": 500,
0032:         "ar_seq": 800,
0033:         "ap_seq": 900,
0034:         "reserva_seq": 300,
0035:         "pedidos": {},
0036:         "itens": {},
0037:         "ar": {},
0038:         "ap": {},
0039:         "reservas": {},
0040:         "caixa_saldo": {1: 0.0},
0041:     }
0042:
0043:     def patch_tx(module, name, fn):
0044:         monkeypatch.setattr(module, "db_transaction", fake_db_transaction)
0045:         monkeypatch.setattr(module, name, fn)
0046:
0047:     def criar_pedido(_cur, usuario, filial, valor_total, status, icms=None, ipi=None, total_br
uto=None):
0048:         state["pedido_seq"] += 1
0049:         pedido_id = state["pedido_seq"]
0050:         pedido = {
0051:             "pedido_id": pedido_id,
0052:             "filial": filial,
0053:             "valor_total": valor_total,
0054:             "status": status,
0055:             "usuario": usuario,
0056:             "icms": icms,
0057:             "ipi": ipi,
0058:             "total_bruto": total_bruto,
0059:             "itens": [],
```

```
0060:         }
0061:         state["pedidos"][pedido_id] = pedido
0062:         return pedido
0063:
0064:     def adicionar_item(_cur, usuario, pedido_id, filial, produto, qtd, total, preco_unit=None)
:
0065:         state["item_seq"] += 1
0066:         item_id = state["item_seq"]
0067:         item = {
0068:             "item_id": item_id,
0069:             "pedido_id": pedido_id,
0070:             "filial": filial,
0071:             "produto": produto,
0072:             "qtd": qtd,
0073:             "total": total,
0074:             "preco_unit": preco_unit,
0075:             "usuario": usuario,
0076:         }
0077:         state["itens"][item_id] = item
0078:         state["pedidos"][pedido_id]["itens"].append(item)
0079:         return item
0080:
0081:     def faturar_pedido(_cur, usuario, pedido_id, filial, cliente_cod, venc_dias, tes_cod):
0082:         state["nf_seq"] += 1
0083:         state["ar_seq"] += 1
0084:         pedido = state["pedidos"][pedido_id]
0085:         pedido["status"] = "FATURADO"
0086:         pedido["nf_id"] = state["nf_seq"]
0087:
0088:         titulo_id = state["ar_seq"]
0089:         titulo = {
0090:             "ID": titulo_id,
0091:             "E1_FILIAL": filial,
0092:             "E1_CLIENTE": cliente_cod or "CLIENTE",
0093:             "E1_VALOR": pedido["valor_total"],
0094:             "E1_STATUS": "ABERTO",
0095:             "E1_REF": f"NF-{pedido['nf_id']}",
0096:             "pedido_id": pedido_id,
0097:             "usuario": usuario,
0098:             "tes_cod": tes_cod,
0099:             "venc_dias": venc_dias,
0100:         }
0101:         state["ar"][titulo_id] = titulo
0102:         return {"pedido_id": pedido_id, "nf_id": pedido["nf_id"], "ar_titulo_id": titulo_id, "
status": "FATURADO"}
0103:
0104:     def listar_ar(_cur, filial, status):
0105:         return [
0106:             titulo
0107:             for titulo in state["ar"].values()
0108:             if titulo["E1_FILIAL"] == filial and titulo["E1_STATUS"] == status
0109:         ]
0110:
0111:     def receber_ar(_cur, usuario, titulo_id, conta_id):
```

```
0112:         titulo = state["ar"][titulo_id]
0113:         titulo["E1_STATUS"] = "BAIXADO"
0114:         titulo["recebido_por"] = usuario
0115:         titulo["conta_id"] = conta_id
0116:         state["caixa_saldo"][conta_id] = state["caixa_saldo"].get(conta_id, 0.0) + float(titul
o["E1_VALOR"])
0117:         return {"titulo_id": titulo_id, "status": "BAIXADO", "conta_id": conta_id}
0118:
0119:     def saldo_caixa(_cur, filial, conta_id, ate=None):
0120:         return {"filial": filial, "conta_id": conta_id, "saldo": state["caixa_saldo"].get(cont
a_id, 0.0), "ate": ate}
0121:
0122:     def lancar_ap(_cur, usuario, filial, forn_cod, valor, ref, venc_dias):
0123:         state["ap_seq"] += 1
0124:         titulo_id = state["ap_seq"]
0125:         titulo = {
0126:             "ID": titulo_id,
0127:             "F1_FILIAL": filial,
0128:             "F1_FORN": forn_cod,
0129:             "F1_VALOR": valor,
0130:             "F1_STATUS": "ABERTO",
0131:             "F1_REF": ref,
0132:             "venc_dias": venc_dias,
0133:             "usuario": usuario,
0134:         }
0135:         state["ap"][titulo_id] = titulo
0136:         return {"titulo_id": titulo_id, "status": "ABERTO", "forn_cod": forn_cod}
0137:
0138:     def listar_ap(_cur, usuario, filial, status):
0139:         return [
0140:             titulo
0141:             for titulo in state["ap"].values()
0142:             if titulo["F1_FILIAL"] == filial and titulo["F1_STATUS"] == status
0143:         ]
0144:
0145:     def pagar_ap(_cur, usuario, titulo_id, conta_id):
0146:         titulo = state["ap"][titulo_id]
0147:         titulo["F1_STATUS"] = "BAIXADO"
0148:         titulo["pago_por"] = usuario
0149:         titulo["conta_id"] = conta_id
0150:         state["caixa_saldo"][conta_id] = state["caixa_saldo"].get(conta_id, 0.0) - float(titul
o["F1_VALOR"])
0151:         return {"titulo_id": titulo_id, "status": "BAIXADO", "conta_id": conta_id}
0152:
0153:     def criar_reserva(_cur, usuario, produto, qtd, filial, cliente_cod, tabela_cod):
0154:         state["reserva_seq"] += 1
0155:         reserva_id = state["reserva_seq"]
0156:         reserva = {
0157:             "reserva_id": reserva_id,
0158:             "produto": produto,
0159:             "qtd": qtd,
0160:             "filial": filial,
0161:             "cliente_cod": cliente_cod,
0162:             "tabela_cod": tabela_cod,
```

```
0163:             "status": "ABERTA",
0164:             "usuario": usuario,
0165:         }
0166:         state["reservas"][reserva_id] = reserva
0167:         return reserva
0168:
0169:     def confirmar_reserva(_cur, usuario, reserva_id, cliente_cod, venc_dias, tes_cod):
0170:         reserva = state["reservas"][reserva_id]
0171:         reserva["status"] = "CONFIRMADA"
0172:         reserva["confirmada_por"] = usuario
0173:         reserva["cliente_cod"] = cliente_cod or reserva["cliente_cod"]
0174:         reserva["venc_dias"] = venc_dias
0175:         reserva["tes_cod"] = tes_cod
0176:         return reserva
0177:
0178:     def cancelar_reserva(_cur, usuario, reserva_id):
0179:         reserva = state["reservas"][reserva_id]
0180:         reserva["status"] = "CANCELADA"
0181:         reserva["cancelada_por"] = usuario
0182:         return reserva
0183:
0184:     patch_tx(vendas_router, "criar_pedido_tx", criar_pedido)
0185:     patch_tx(vendas_router, "adicionar_item_tx", adicionar_item)
0186:     patch_tx(vendas_router, "faturar_pedido_venda_tx", faturar_pedido)
0187:     patch_tx(financeiro_router, "listar_ar_uc_tx", listar_ar)
0188:     patch_tx(financeiro_router, "receber_ar_uc_tx", receber_ar)
0189:     patch_tx(financeiro_router, "saldo_caixa_uc_tx", saldo_caixa)
0190:     patch_tx(financeiro_router, "listar_ap_uc_tx", listar_ap)
0191:     patch_tx(financeiro_router, "pagar_ap_uc_tx", pagar_ap)
0192:     patch_tx(ap_router, "lancar_ap_uc_tx", lancar_ap)
0193:     patch_tx(reservas_router, "criar_reserva_tx", criar_reserva)
0194:     patch_tx(reservas_router, "confirmar_reserva_tx", confirmar_reserva)
0195:     patch_tx(reservas_router, "cancelar_reserva_tx", cancelar_reserva)
0196:
0197:     return state
0198:
0199:
0200: def test_fluxo_vendas_financeiro_e_caixa(monkeypatch):
0201:     state = _wire_fake_business_flow(monkeypatch)
0202:     _install_admin_override()
0203:     try:
0204:         client = TestClient(app)
0205:
0206:         pedido_resp = client.post(
0207:             "/vendas/pedidos",
0208:             json={"filial": "01", "valor_total": 150.0, "status": "ABERTO"},
0209:         )
0210:         assert pedido_resp.status_code == 200
0211:         pedido_id = pedido_resp.json()["pedido_id"]
0212:
0213:         item_resp = client.post(
0214:             f"/vendas/pedidos/{pedido_id}/itens",
0215:             json={"filial": "01", "produto": "PROD001", "qtd": 3, "total": 150.0, "preco_unit"
: 50.0},
```

```
0216:         )
0217:         assert item_resp.status_code == 200
0218:         assert item_resp.json()["pedido_id"] == pedido_id
0219:
0220:         faturar_resp = client.post(
0221:             f"/vendas/pedidos/{pedido_id}/faturar",
0222:             json={"filial": "01", "cliente_cod": "C0001", "venc_dias": 30, "tes_cod": "001"},
0223:         )
0224:         assert faturar_resp.status_code == 200
0225:         ar_titulo_id = faturar_resp.json()["ar_titulo_id"]
0226:         assert state["pedidos"][pedido_id]["status"] == "FATURADO"
0227:
0228:         listar_ar_resp = client.get("/financeiro/ar", params={"filial": "01", "status": "ABERT
O"})
0229:         assert listar_ar_resp.status_code == 200
0230:         ar_payload = listar_ar_resp.json()
0231:         assert len(ar_payload) == 1
0232:         assert ar_payload[0]["ID"] == ar_titulo_id
0233:         assert ar_payload[0]["E1_VALOR"] == 150.0
0234:
0235:         receber_resp = client.post("/financeiro/ar/receber", json={"titulo_id": ar_titulo_id,
"conta_id": 1})
0236:         assert receber_resp.status_code == 200
0237:         assert state["ar"][ar_titulo_id]["E1_STATUS"] == "BAIXADO"
0238:
0239:         saldo_resp = client.get("/financeiro/caixa/saldo", params={"filial": "01", "conta_id":
 1})
0240:         assert saldo_resp.status_code == 200
0241:         assert saldo_resp.json()["saldo"] == 150.0
0242:     finally:
0243:         _clear_overrides()
0244:
0245:
0246: def test_fluxo_ap_e_reservas(monkeypatch):
0247:     state = _wire_fake_business_flow(monkeypatch)
0248:     _install_admin_override()
0249:     try:
0250:         client = TestClient(app)
0251:
0252:         ap_resp = client.post(
0253:             "/ap/lancar",
0254:             json={"forn_cod": "F0001", "valor": 80.0, "ref": "MANUAL-1", "venc_dias": 15, "fil
ial": "01"},
0255:         )
0256:         assert ap_resp.status_code == 200
0257:         ap_titulo_id = ap_resp.json()["titulo_id"]
0258:
0259:         listar_ap_resp = client.get("/financeiro/ap", params={"filial": "01", "status": "ABERT
O"})
0260:         assert listar_ap_resp.status_code == 200
0261:         ap_payload = listar_ap_resp.json()
0262:         assert len(ap_payload) == 1
0263:         assert ap_payload[0]["ID"] == ap_titulo_id
0264:         assert ap_payload[0]["F1_VALOR"] == 80.0
```

```
0265:
0266:         pagar_resp = client.post("/financeiro/ap/pagar", json={"titulo_id": ap_titulo_id, "con
ta_id": 1})
0267:         assert pagar_resp.status_code == 200
0268:         assert state["ap"][ap_titulo_id]["F1_STATUS"] == "BAIXADO"
0269:
0270:         saldo_resp = client.get("/financeiro/caixa/saldo", params={"filial": "01", "conta_id":
 1})
0271:         assert saldo_resp.status_code == 200
0272:         assert saldo_resp.json()["saldo"] == -80.0
0273:
0274:         criar_reserva_resp = client.post(
0275:             "/reserva",
0276:             json={"cod": "PROD002", "qtd": 2, "filial": "01", "cliente_cod": "C0002", "tabela_
cod": "0001"},
0277:         )
0278:         assert criar_reserva_resp.status_code == 200
0279:         reserva_id = criar_reserva_resp.json()["reserva_id"]
0280:         assert state["reservas"][reserva_id]["status"] == "ABERTA"
0281:
0282:         confirmar_resp = client.post(
0283:             "/reserva/confirmar",
0284:             json={"reserva_id": reserva_id, "cliente_cod": "C0002", "venc_dias": 20, "tes_cod"
: "001"},
0285:         )
0286:         assert confirmar_resp.status_code == 200
0287:         assert state["reservas"][reserva_id]["status"] == "CONFIRMADA"
0288:
0289:         cancelar_resp = client.post("/reserva/cancelar", json={"reserva_id": reserva_id})
0290:         assert cancelar_resp.status_code == 200
0291:         assert state["reservas"][reserva_id]["status"] == "CANCELADA"
0292:     finally:
0293:         _clear_overrides()
0294:
0295:
0296: def test_faturar_pedido_sem_itens_retorna_erro_de_negocio(monkeypatch):
0297:     state = _wire_fake_business_flow(monkeypatch)
0298:
0299:     def faturar_sem_itens(_cur, usuario, pedido_id, filial, cliente_cod, venc_dias, tes_cod):
0300:         pedido = state["pedidos"][pedido_id]
0301:         if not pedido["itens"]:
0302:             raise BusinessError("Pedido sem itens")
0303:         return {"pedido_id": pedido_id}
0304:
0305:     monkeypatch.setattr(vendas_router, "faturar_pedido_venda_tx", faturar_sem_itens)
0306:     _install_admin_override()
0307:     try:
0308:         client = TestClient(app, raise_server_exceptions=False)
0309:         pedido_resp = client.post(
0310:             "/vendas/pedidos",
0311:             json={"filial": "01", "valor_total": 150.0, "status": "ABERTO"},
0312:         )
0313:         pedido_id = pedido_resp.json()["pedido_id"]
0314:         response = client.post(
```

```
0315:             f"/vendas/pedidos/{pedido_id}/faturar",
0316:             json={"filial": "01", "cliente_cod": "C0001", "venc_dias": 30, "tes_cod": "001"},
0317:         )
0318:     finally:
0319:         _clear_overrides()
0320:
0321:     assert response.status_code == 400
0322:     assert response.json()["detail"] == "Pedido sem itens"
0323:
0324:
0325: def test_receber_ar_ja_baixado_nao_duplica_caixa(monkeypatch):
0326:     state = _wire_fake_business_flow(monkeypatch)
0327:
0328:     def receber_ar_unico(_cur, usuario, titulo_id, conta_id):
0329:         titulo = state["ar"][titulo_id]
0330:         if titulo["E1_STATUS"] != "ABERTO":
0331:             raise BusinessError("Titulo AR nao esta em aberto")
0332:         titulo["E1_STATUS"] = "BAIXADO"
0333:         state["caixa_saldo"][conta_id] = state["caixa_saldo"].get(conta_id, 0.0) + float(titul
o["E1_VALOR"])
0334:         return {"titulo_id": titulo_id, "status": "BAIXADO", "conta_id": conta_id}
0335:
0336:     monkeypatch.setattr(financeiro_router, "receber_ar_uc_tx", receber_ar_unico)
0337:     _install_admin_override()
0338:     try:
0339:         client = TestClient(app, raise_server_exceptions=False)
0340:         pedido_resp = client.post("/vendas/pedidos", json={"filial": "01", "valor_total": 100.
0, "status": "ABERTO"})
0341:         pedido_id = pedido_resp.json()["pedido_id"]
0342:         client.post(
0343:             f"/vendas/pedidos/{pedido_id}/itens",
0344:             json={"filial": "01", "produto": "PROD001", "qtd": 2, "total": 100.0, "preco_unit"
: 50.0},
0345:         )
0346:         faturar_resp = client.post(
0347:             f"/vendas/pedidos/{pedido_id}/faturar",
0348:             json={"filial": "01", "cliente_cod": "C0001", "venc_dias": 30, "tes_cod": "001"},
0349:         )
0350:         titulo_id = faturar_resp.json()["ar_titulo_id"]
0351:         first = client.post("/financeiro/ar/receber", json={"titulo_id": titulo_id, "conta_id"
: 1})
0352:         second = client.post("/financeiro/ar/receber", json={"titulo_id": titulo_id, "conta_id
": 1})
0353:     finally:
0354:         _clear_overrides()
0355:
0356:     assert first.status_code == 200
0357:     assert second.status_code == 400
0358:     assert second.json()["detail"] == "Titulo AR nao esta em aberto"
0359:     assert state["caixa_saldo"][1] == 100.0
```

==========
ARQUIVO: tests/test_business_flows_db.py
==========

```
0001: from __future__ import annotations
0002:
0003: from datetime import date
0004: from uuid import uuid4
0005:
0006: import pytest
0007:
0008: from app.core.exceptions import BusinessError
0009: from app.core.security import hash_password
0010: from app.infra.repositories.fechamento_repo import fechamento_periodo_aberto_tx
0011: from app.infra.repositories.financeiro_repo import listar_ar_tx
0012: from app.infra.repositories.fiscal_repo import fiscal_upsert_tx
0013: from app.infra.repositories.relatorios_repo import consistencia_fiscal_tx
0014: from app.infra.repositories.sa1_repo import sa1_criar_tx
0015: from app.infra.repositories.sa2_repo import sa2_criar_tx
0016: from app.infra.repositories.sb1_repo import sb1_atualizar_cm_tx, sb1_entrada_tx, sb1_upsert_tx
0017: from app.infra.repositories.serie_repo import seed_series_basicas_tx
0018: from app.infra.repositories.sm0_repo import sm0_criar_tx
0019: from app.infra.repositories.relatorios_repo import conciliacao_nf_financeiro_tx, dre_simples_t
x, margem_por_produto_tx
0020: from app.use_cases.ap_lancamento_uc import lancar_ap_uc_tx
0021: from app.use_cases.fechamento_uc import abrir_periodo_uc_tx, fechar_periodo_uc_tx
0022: from app.use_cases.financeiro_uc import listar_ap_uc_tx, pagar_ap_uc_tx, receber_ar_uc_tx, sal
do_caixa_uc_tx
0023: from app.use_cases.reservas_uc import cancelar_reserva_tx, confirmar_reserva_tx, criar_reserva
_tx
0024: from app.use_cases.vendas.faturar_pedido_uc import faturar_pedido_venda_tx
0025: from app.use_cases.vendas.pedidos_uc import adicionar_item_tx, criar_pedido_tx
0026: from tests.conftest import integration_db_cursor
0027:
0028:
0029: def _seed_admin(cur, login: str) -> None:
0030:     cur.execute(
0031:         """
0032:         INSERT INTO dbo.USUARIOS (U_LOGIN, U_SENHA_HASH, U_PERFIL)
0033:         VALUES (?, ?, 'admin')
0034:         """,
0035:         (login, hash_password("SenhaForte123!")),
0036:     )
0037:
0038:
0039: def _seed_caixa(cur, filial: str, nome: str = "CAIXA TESTE") -> int:
0040:     cur.execute(
0041:         """
0042:         INSERT INTO dbo.SE5_CONTAS (E5_FILIAL, E5_NOME, E5_TIPO, E5_ATIVA)
0043:         VALUES (?, ?, 'CAIXA', 1);
0044:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0045:         """,
0046:         (filial, nome),
0047:     )
0048:     row = cur.fetchone()
0049:     return int(row[0])
0050:
0051:
```

```
0052: def _seed_fornecedor(cur, filial: str, forn_cod: str) -> None:
0053:     sa2_criar_tx(cur, filial=filial, cod=forn_cod, nome="Fornecedor Integracao")
0054:
0055:
0056: def _seed_price_table(cur, filial: str, produto_cod: str, tabela_cod: str = "0001", preco: flo
at = 55.0) -> int:
0057:     cur.execute(
0058:         """
0059:         INSERT INTO dbo.PR0_TABELA (P0_FILIAL, P0_COD, P0_DESC, P0_ATIVA)
0060:         VALUES (?, ?, ?, 1);
0061:         SELECT CAST(SCOPE_IDENTITY() AS INT) AS id;
0062:         """,
0063:         (filial, tabela_cod, f"TABELA {tabela_cod}"),
0064:     )
0065:     row = cur.fetchone()
0066:     tabela_id = int(row[0])
0067:     cur.execute(
0068:         """
0069:         INSERT INTO dbo.PR1_PRECO (P1_FILIAL, P1_TABELA_ID, P1_PRODUTO, P1_PRECO, P1_DTINI, P1
_DTFIM)
0070:         VALUES (?, ?, ?, ?, CAST(GETDATE() AS DATE), NULL)
0071:         """,
0072:         (filial, tabela_id, produto_cod, float(preco)),
0073:     )
0074:     return tabela_id
0075:
0076:
0077: def _seed_sales_minimum(cur, filial: str, admin_login: str, cliente_cod: str, produto_cod: str
) -> int:
0078:     sm0_criar_tx(cur, filial=filial, serie_nf="1", serie_ar="1", serie_ap="1")
0079:     seed_series_basicas_tx(cur, filial=filial)
0080:     fiscal_upsert_tx(
0081:         cur,
0082:         filial=filial,
0083:         tes_cod="001",
0084:         cfop="5102",
0085:         icms=0.18,
0086:         ipi=0.05,
0087:         tipo="S",
0088:         gera_tit=True,
0089:         gera_est=True,
0090:         descr="TES VENDA TESTE",
0091:     )
0092:     sa1_criar_tx(cur, filial=filial, cod=cliente_cod, nome="Cliente Integracao")
0093:     sb1_upsert_tx(cur, cod=produto_cod, desc="Produto Integracao", preco=50.0, filial=filial)
0094:     sb1_entrada_tx(cur, produto=produto_cod, filial=filial, qtd=20)
0095:     sb1_atualizar_cm_tx(cur, produto=produto_cod, filial=filial, cm_novo=20.0)
0096:     return _seed_caixa(cur, filial=filial, nome=f"CAIXA {admin_login}")
0097:
0098:
0099: def test_fluxo_real_venda_ar_caixa_db():
0100:     run_id = uuid4().hex[:8].upper()
0101:     filial = f"T{run_id[:1]}"
0102:     admin_login = f"admin_it_{run_id}"
```

```
0103:     cliente_cod = f"C{run_id[:6]}"
0104:     produto_cod = f"P{run_id[:6]}"
0105:
0106:     with integration_db_cursor() as (_, cur):
0107:         _seed_admin(cur, admin_login)
0108:         conta_id = _seed_sales_minimum(
0109:             cur,
0110:             filial=filial,
0111:             admin_login=admin_login,
0112:             cliente_cod=cliente_cod,
0113:             produto_cod=produto_cod,
0114:         )
0115:
0116:         pedido = criar_pedido_tx(
0117:             cur,
0118:             usuario=admin_login,
0119:             filial=filial,
0120:             valor_total=0.0,
0121:             status="ABERTO",
0122:         )
0123:         pedido_id = int(pedido["pedido_id"])
0124:
0125:         item = adicionar_item_tx(
0126:             cur,
0127:             usuario=admin_login,
0128:             pedido_id=pedido_id,
0129:             filial=filial,
0130:             produto=produto_cod,
0131:             qtd=2,
0132:             total=100.0,
0133:             preco_unit=50.0,
0134:         )
0135:         assert item["novo_total"] == 100.0
0136:
0137:         faturamento = faturar_pedido_venda_tx(
0138:             cur,
0139:             usuario=admin_login,
0140:             pedido_id=pedido_id,
0141:             filial=filial,
0142:             cliente_cod=cliente_cod,
0143:             venc_dias=30,
0144:             tes_cod="001",
0145:         )
0146:         assert faturamento["status"] == "FATURADO"
0147:         assert faturamento["valor_venda"] == 100.0
0148:
0149:         ar_list = listar_ar_tx(cur, filial=filial, status="ABERTO")
0150:         assert len(ar_list) == 1
0151:         titulo_id = int(ar_list[0]["ID"])
0152:         assert float(ar_list[0]["E1_VALOR"]) == 100.0
0153:
0154:         recebimento = receber_ar_uc_tx(
0155:             cur,
0156:             usuario=admin_login,
```

```
0157:             titulo_id=titulo_id,
0158:             conta_id=conta_id,
0159:         )
0160:         assert recebimento["status"] == "BAIXADO"
0161:
0162:         saldo = saldo_caixa_uc_tx(cur, filial=filial, conta_id=conta_id, ate=None)
0163:         assert saldo["saldo"] == 100.0
0164:
0165:
0166: def test_fluxo_real_ap_pagamento_caixa_db():
0167:     run_id = uuid4().hex[:8].upper()
0168:     filial = f"U{run_id[:1]}"
0169:     admin_login = f"admin_it_{run_id}"
0170:     forn_cod = f"F{run_id[:6]}"
0171:
0172:     with integration_db_cursor() as (_, cur):
0173:         _seed_admin(cur, admin_login)
0174:         sm0_criar_tx(cur, filial=filial, serie_nf="1", serie_ar="1", serie_ap="1")
0175:         seed_series_basicas_tx(cur, filial=filial)
0176:         _seed_fornecedor(cur, filial=filial, forn_cod=forn_cod)
0177:         conta_id = _seed_caixa(cur, filial=filial, nome=f"CAIXA AP {admin_login}")
0178:
0179:         lancamento = lancar_ap_uc_tx(
0180:             cur,
0181:             usuario=admin_login,
0182:             filial=filial,
0183:             forn_cod=forn_cod,
0184:             valor=80.0,
0185:             ref="MANUAL-IT",
0186:             venc_dias=15,
0187:         )
0188:         assert lancamento["ok"] is True
0189:
0190:         ap_list = listar_ap_uc_tx(cur, usuario=admin_login, filial=filial, status="ABERTO")
0191:         assert len(ap_list) == 1
0192:         titulo_id = int(ap_list[0]["ID"])
0193:         assert float(ap_list[0]["F1_VALOR"]) == 80.0
0194:
0195:         pagamento = pagar_ap_uc_tx(cur, usuario=admin_login, titulo_id=titulo_id, conta_id=con
ta_id)
0196:         assert pagamento["status"] == "BAIXADO"
0197:
0198:         saldo = saldo_caixa_uc_tx(cur, filial=filial, conta_id=conta_id, ate=None)
0199:         assert saldo["saldo"] == -80.0
0200:
0201:
0202: def test_fluxo_real_reserva_confirmacao_cancelamento_db():
0203:     run_id = uuid4().hex[:8].upper()
0204:     filial = f"R{run_id[:1]}"
0205:     admin_login = f"admin_it_{run_id}"
0206:     cliente_cod = f"C{run_id[:6]}"
0207:     produto_cod = f"P{run_id[:6]}"
0208:
0209:     with integration_db_cursor() as (_, cur):
```

```
0210:         _seed_admin(cur, admin_login)
0211:         _seed_sales_minimum(
0212:             cur,
0213:             filial=filial,
0214:             admin_login=admin_login,
0215:             cliente_cod=cliente_cod,
0216:             produto_cod=produto_cod,
0217:         )
0218:         _seed_price_table(cur, filial=filial, produto_cod=produto_cod, preco=60.0)
0219:
0220:         reserva = criar_reserva_tx(
0221:             cur,
0222:             usuario=admin_login,
0223:             produto=produto_cod,
0224:             qtd=2,
0225:             filial=filial,
0226:             cliente_cod=cliente_cod,
0227:             tabela_cod="0001",
0228:         )
0229:         reserva_id = int(reserva["reserva_id"])
0230:         assert reserva["status"] == "ABERTA"
0231:
0232:         confirmacao = confirmar_reserva_tx(
0233:             cur,
0234:             usuario=admin_login,
0235:             reserva_id=reserva_id,
0236:             cliente_cod=cliente_cod,
0237:             venc_dias=30,
0238:             tes_cod="001",
0239:         )
0240:         assert confirmacao["reserva_id"] == reserva_id
0241:         assert confirmacao["pedido_id"] > 0
0242:         assert confirmacao["nf_id"] > 0
0243:         assert confirmacao["valor_venda"] == 120.0
0244:
0245:         ar_list = listar_ar_tx(cur, filial=filial, status="ABERTO")
0246:         assert len(ar_list) == 1
0247:         assert float(ar_list[0]["E1_VALOR"]) == 120.0
0248:
0249:         with pytest.raises(BusinessError):
0250:             cancelar_reserva_tx(cur, usuario=admin_login, reserva_id=reserva_id)
0251:
0252:
0253: def test_relatorios_e_fechamento_db():
0254:     run_id = uuid4().hex[:8].upper()
0255:     filial = f"D{run_id[:1]}"
0256:     admin_login = f"admin_it_{run_id}"
0257:     cliente_cod = f"C{run_id[:6]}"
0258:     produto_cod = f"P{run_id[:6]}"
0259:
0260:     with integration_db_cursor() as (_, cur):
0261:         _seed_admin(cur, admin_login)
0262:         conta_id = _seed_sales_minimum(
0263:             cur,
```

```
0264:             filial=filial,
0265:             admin_login=admin_login,
0266:             cliente_cod=cliente_cod,
0267:             produto_cod=produto_cod,
0268:         )
0269:
0270:         pedido = criar_pedido_tx(cur, usuario=admin_login, filial=filial, valor_total=0.0, sta
tus="ABERTO")
0271:         pedido_id = int(pedido["pedido_id"])
0272:         adicionar_item_tx(
0273:             cur,
0274:             usuario=admin_login,
0275:             pedido_id=pedido_id,
0276:             filial=filial,
0277:             produto=produto_cod,
0278:             qtd=2,
0279:             total=100.0,
0280:             preco_unit=50.0,
0281:         )
0282:         fat = faturar_pedido_venda_tx(
0283:             cur,
0284:             usuario=admin_login,
0285:             pedido_id=pedido_id,
0286:             filial=filial,
0287:             cliente_cod=cliente_cod,
0288:             venc_dias=30,
0289:             tes_cod="001",
0290:         )
0291:         titulo_id = int(listar_ar_tx(cur, filial=filial, status="ABERTO")[0]["ID"])
0292:         receber_ar_uc_tx(cur, usuario=admin_login, titulo_id=titulo_id, conta_id=conta_id)
0293:
0294:         cur.execute(
0295:             """
0296:             INSERT INTO dbo.SD3_MOV
0297:               (D3_DATA, D3_FILIAL, D3_PRODUTO, D3_TIPO, D3_QTD, D3_CUSTO_UNIT, D3_VALOR, D3_OR
IGEM, D3_REF, D3_USUARIO)
0298:             VALUES (SYSDATETIME(), ?, ?, 'S', ?, ?, ?, 'RESERVA_FATURADA', ?, ?)
0299:             """,
0300:             (filial, produto_cod, 2, 20.0, 40.0, str(fat["nf_id"]), admin_login),
0301:         )
0302:
0303:         dre = dre_simples_tx(cur, filial=filial, de=None, ate=None)
0304:         assert dre["receita"] == 100.0
0305:         assert dre["cmv"] == 40.0
0306:         assert dre["margem_bruta"] == 60.0
0307:
0308:         margem = margem_por_produto_tx(cur, filial=filial, de=None, ate=None)
0309:         assert len(margem) == 1
0310:         assert margem[0]["produto"] == produto_cod
0311:         assert margem[0]["receita"] == 100.0
0312:         assert margem[0]["cmv"] == 40.0
0313:
0314:         conciliacao = conciliacao_nf_financeiro_tx(cur, filial=filial, de=None, ate=None, tole
rancia=0.01, limit=50)
```

```
0315:         assert conciliacao["nf_sem_ar"] == []
0316:         assert conciliacao["ar_sem_nf"] == []
0317:         assert conciliacao["valor_divergente"] == []
0318:         assert conciliacao["nf_cancelada_ar"] == []
0319:
0320:         cur.execute(
0321:             """
0322:             UPDATE dbo.SF2_NOTAS
0323:             SET F2_STATUS='CANCELADA'
0324:             WHERE ID=?
0325:             """,
0326:             (int(fat["nf_id"]),),
0327:         )
0328:         consistencia = consistencia_fiscal_tx(cur, filial=filial, de=None, ate=None, limit=50)
0329:         assert len(consistencia["nf_cancelada_sd2"]) >= 1
0330:
0331:         fechar_periodo_uc_tx(cur, usuario=admin_login, filial=filial, ano=2099, mes=1)
0332:         assert fechamento_periodo_aberto_tx(cur, filial=filial, data_ref=date(2099, 1, 1)) is
False
0333:         abrir_periodo_uc_tx(cur, usuario=admin_login, filial=filial, ano=2099, mes=1)
0334:         assert fechamento_periodo_aberto_tx(cur, filial=filial, data_ref=date(2099, 1, 1)) is
True
0335:
0336:
0337: def test_faturar_pedido_em_periodo_fechado_db():
0338:     run_id = uuid4().hex[:8].upper()
0339:     filial = f"F{run_id[:1]}"
0340:     admin_login = f"admin_it_{run_id}"
0341:     cliente_cod = f"C{run_id[:6]}"
0342:     produto_cod = f"P{run_id[:6]}"
0343:     hoje = date.today()
0344:
0345:     with integration_db_cursor() as (_, cur):
0346:         _seed_admin(cur, admin_login)
0347:         _seed_sales_minimum(
0348:             cur,
0349:             filial=filial,
0350:             admin_login=admin_login,
0351:             cliente_cod=cliente_cod,
0352:             produto_cod=produto_cod,
0353:         )
0354:
0355:         pedido = criar_pedido_tx(cur, usuario=admin_login, filial=filial, valor_total=0.0, sta
tus="ABERTO")
0356:         pedido_id = int(pedido["pedido_id"])
0357:         adicionar_item_tx(
0358:             cur,
0359:             usuario=admin_login,
0360:             pedido_id=pedido_id,
0361:             filial=filial,
0362:             produto=produto_cod,
0363:             qtd=2,
0364:             total=100.0,
0365:             preco_unit=50.0,
```

```
0366:         )
0367:
0368:         fechar_periodo_uc_tx(cur, usuario=admin_login, filial=filial, ano=hoje.year, mes=hoje.
month)
0369:
0370:         with pytest.raises(BusinessError, match="Per"):
0371:             faturar_pedido_venda_tx(
0372:                 cur,
0373:                 usuario=admin_login,
0374:                 pedido_id=pedido_id,
0375:                 filial=filial,
0376:                 cliente_cod=cliente_cod,
0377:                 venc_dias=30,
0378:                 tes_cod="001",
0379:             )
0380:
0381:
0382: def test_pagar_ap_ja_baixado_nao_duplica_caixa_db():
0383:     run_id = uuid4().hex[:8].upper()
0384:     filial = f"A{run_id[:1]}"
0385:     admin_login = f"admin_it_{run_id}"
0386:     forn_cod = f"F{run_id[:6]}"
0387:
0388:     with integration_db_cursor() as (_, cur):
0389:         _seed_admin(cur, admin_login)
0390:         sm0_criar_tx(cur, filial=filial, serie_nf="1", serie_ar="1", serie_ap="1")
0391:         seed_series_basicas_tx(cur, filial=filial)
0392:         _seed_fornecedor(cur, filial=filial, forn_cod=forn_cod)
0393:         conta_id = _seed_caixa(cur, filial=filial, nome=f"CAIXA AP DUP {admin_login}")
0394:
0395:         lancar_ap_uc_tx(
0396:             cur,
0397:             usuario=admin_login,
0398:             filial=filial,
0399:             forn_cod=forn_cod,
0400:             valor=80.0,
0401:             ref="MANUAL-DUP",
0402:             venc_dias=15,
0403:         )
0404:         titulo_id = int(listar_ap_uc_tx(cur, usuario=admin_login, filial=filial, status="ABERT
O")[0]["ID"])
0405:
0406:         primeiro = pagar_ap_uc_tx(cur, usuario=admin_login, titulo_id=titulo_id, conta_id=cont
a_id)
0407:         assert primeiro["status"] == "BAIXADO"
0408:
0409:         with pytest.raises(BusinessError, match="T.tulo AP n.o est. em aberto"):
0410:             pagar_ap_uc_tx(cur, usuario=admin_login, titulo_id=titulo_id, conta_id=conta_id)
0411:
0412:         saldo = saldo_caixa_uc_tx(cur, filial=filial, conta_id=conta_id, ate=None)
0413:         assert saldo["saldo"] == -80.0
0414:
0415:
0416: def test_confirmar_reserva_sem_saldo_efetivo_db():
```

```
0417:     run_id = uuid4().hex[:8].upper()
0418:     filial = f"S{run_id[:1]}"
0419:     admin_login = f"admin_it_{run_id}"
0420:     cliente_cod = f"C{run_id[:6]}"
0421:     produto_cod = f"P{run_id[:6]}"
0422:
0423:     with integration_db_cursor() as (_, cur):
0424:         _seed_admin(cur, admin_login)
0425:         _seed_sales_minimum(
0426:             cur,
0427:             filial=filial,
0428:             admin_login=admin_login,
0429:             cliente_cod=cliente_cod,
0430:             produto_cod=produto_cod,
0431:         )
0432:         _seed_price_table(cur, filial=filial, produto_cod=produto_cod, preco=60.0)
0433:
0434:         reserva = criar_reserva_tx(
0435:             cur,
0436:             usuario=admin_login,
0437:             produto=produto_cod,
0438:             qtd=2,
0439:             filial=filial,
0440:             cliente_cod=cliente_cod,
0441:             tabela_cod="0001",
0442:         )
0443:         reserva_id = int(reserva["reserva_id"])
0444:
0445:         cur.execute(
0446:             """
0447:             UPDATE dbo.SB1_PRODUTOS
0448:             SET B1_ESTOQUE = 1
0449:             WHERE B1_COD=? AND B1_FILIAL=?
0450:             """,
0451:             (produto_cod, filial),
0452:         )
0453:
0454:         with pytest.raises(BusinessError, match="Estoque/reservado insuficiente para confirmar
"):
0455:             confirmar_reserva_tx(
0456:                 cur,
0457:                 usuario=admin_login,
0458:                 reserva_id=reserva_id,
0459:                 cliente_cod=cliente_cod,
0460:                 venc_dias=30,
0461:                 tes_cod="001",
0462:             )
0463:
0464:
0465: def test_conciliacao_detecta_divergencia_nf_financeiro_db():
0466:     run_id = uuid4().hex[:8].upper()
0467:     filial = f"C{run_id[:1]}"
0468:     admin_login = f"admin_it_{run_id}"
0469:     cliente_cod = f"C{run_id[:6]}"
```

```
0470:     produto_cod = f"P{run_id[:6]}"
0471:
0472:     with integration_db_cursor() as (_, cur):
0473:         _seed_admin(cur, admin_login)
0474:         _seed_sales_minimum(
0475:             cur,
0476:             filial=filial,
0477:             admin_login=admin_login,
0478:             cliente_cod=cliente_cod,
0479:             produto_cod=produto_cod,
0480:         )
0481:
0482:         pedido = criar_pedido_tx(cur, usuario=admin_login, filial=filial, valor_total=0.0, sta
tus="ABERTO")
0483:         pedido_id = int(pedido["pedido_id"])
0484:         adicionar_item_tx(
0485:             cur,
0486:             usuario=admin_login,
0487:             pedido_id=pedido_id,
0488:             filial=filial,
0489:             produto=produto_cod,
0490:             qtd=2,
0491:             total=100.0,
0492:             preco_unit=50.0,
0493:         )
0494:         fat = faturar_pedido_venda_tx(
0495:             cur,
0496:             usuario=admin_login,
0497:             pedido_id=pedido_id,
0498:             filial=filial,
0499:             cliente_cod=cliente_cod,
0500:             venc_dias=30,
0501:             tes_cod="001",
0502:         )
0503:
0504:         titulo_id = int(listar_ar_tx(cur, filial=filial, status="ABERTO")[0]["ID"])
0505:         cur.execute(
0506:             """
0507:             UPDATE dbo.SE1_AR
0508:             SET E1_VALOR = 130.0
0509:             WHERE ID = ?
0510:             """,
0511:             (titulo_id,),
0512:         )
0513:
0514:         conciliacao = conciliacao_nf_financeiro_tx(cur, filial=filial, de=None, ate=None, tole
rancia=0.01, limit=50)
0515:         assert conciliacao["nf_sem_ar"] == []
0516:         assert conciliacao["ar_sem_nf"] == []
0517:         assert len(conciliacao["valor_divergente"]) >= 1
0518:         assert any(int(row["nf_id"]) == int(fat["nf_id"]) for row in conciliacao["valor_diverg
ente"])
```

ARQUIVO: tests/test_reservas_uc.py
==========

```
0001: from app.core.exceptions import BusinessError
0002: from app.use_cases import reservas_uc
0003:
0004:
0005: def test_criar_reserva_faz_fallback_para_preco_do_produto(monkeypatch):
0006:     monkeypatch.setattr(reservas_uc, "exigir_perfil_tx", lambda *_: None)
0007:     monkeypatch.setattr(
0008:         reservas_uc,
0009:         "sb1_get_tx",
0010:         lambda *_: {
0011:             "B1_COD": "CEL124",
0012:             "B1_PRECO": 11325.45,
0013:             "B1_ESTOQUE": 5,
0014:             "B1_RESERVADO": 0,
0015:             "B1_FILIAL": "01",
0016:         },
0017:     )
0018:     monkeypatch.setattr(reservas_uc, "_tabela_padrao_id_tx", lambda *_args, **_kwargs: 1)
0019:     monkeypatch.setattr(
0020:         reservas_uc,
0021:         "buscar_preco_vigente_tx",
0022:         lambda *_args, **_kwargs: (_ for _ in ()).throw(BusinessError("Preco nao encontrado"))
,
0023:     )
0024:     monkeypatch.setattr(reservas_uc, "sb1_reservar_atomico_tx", lambda *_: None)
0025:     monkeypatch.setattr(reservas_uc, "reserva_criar_tx", lambda *_args, **_kwargs: 123)
0026:     monkeypatch.setattr(reservas_uc, "log_tx", lambda *_: None)
0027:     monkeypatch.setattr(
0028:         reservas_uc,
0029:         "saldo_sd3_tx",
0030:         lambda *_: 5,
0031:     )
0032:
0033:     payload = reservas_uc.criar_reserva_tx(
0034:         object(),
0035:         usuario="admin",
0036:         produto="CEL124",
0037:         qtd=1,
0038:         filial="01",
0039:         cliente_cod="CLI001",
0040:         tabela_cod="0001",
0041:     )
0042:
0043:     assert payload["reserva_id"] == 123
0044:     assert payload["preco_unit"] == 11325.45
0045:     assert payload["total"] == 11325.45
0046:     assert payload["preco_source"] == "PRODUTO"
0047:
0048:
0049: def test_criar_reserva_sem_preco_em_tabela_nem_produto_continua_bloqueando(monkeypatch):
0050:     monkeypatch.setattr(reservas_uc, "exigir_perfil_tx", lambda *_: None)
0051:     monkeypatch.setattr(
```

```
0052:         reservas_uc,
0053:         "sb1_get_tx",
0054:         lambda *_: {
0055:             "B1_COD": "CEL124",
0056:             "B1_PRECO": 0.0,
0057:             "B1_ESTOQUE": 5,
0058:             "B1_RESERVADO": 0,
0059:             "B1_FILIAL": "01",
0060:         },
0061:     )
0062:     monkeypatch.setattr(reservas_uc, "_tabela_padrao_id_tx", lambda *_args, **_kwargs: 1)
0063:     monkeypatch.setattr(
0064:         reservas_uc,
0065:         "buscar_preco_vigente_tx",
0066:         lambda *_args, **_kwargs: (_ for _ in ()).throw(BusinessError("Preco nao encontrado"))
,
0067:     )
0068:
0069:     try:
0070:         reservas_uc.criar_reserva_tx(
0071:             object(),
0072:             usuario="admin",
0073:             produto="CEL124",
0074:             qtd=1,
0075:             filial="01",
0076:             cliente_cod="CLI001",
0077:             tabela_cod="0001",
0078:         )
0079:         assert False, "Era esperado BusinessError quando nao ha preco disponivel"
0080:     except BusinessError as exc:
0081:         assert str(exc) == "Preco nao encontrado"
```